



MOSEK Fusion API for C++
Release 10.0.44

MOSEK ApS

09 May 2023

Contents

1	Introduction	1
1.1	Why the Fusion API for C++?	2
2	Contact Information	3
3	License Agreement	4
3.1	MOSEK end-user license agreement	4
3.2	Third party licenses	4
4	Installation	10
4.1	Testing the installation and compiling examples	12
4.2	Creating a Visual Studio Project	12
4.3	Troubleshooting	12
5	Design Overview	13
6	Conic Modeling	15
6.1	The model	15
6.2	Variables	16
6.3	Expressions and linear operators	16
6.4	Constraints and objective	17
6.5	Matrices	18
6.6	Parameters	19
6.7	Stacking and views	19
6.8	Vectorization	20
6.9	Reoptimization	21
7	Optimization Tutorials	23
7.1	Linear Optimization	24
7.2	Conic Quadratic Optimization	26
7.3	Power Cone Optimization	29
7.4	Conic Exponential Optimization	31
7.5	Geometric Programming	33
7.6	Semidefinite Optimization	36
7.7	Integer Optimization	42
7.8	Disjunctive constraints	45
7.9	Model Parametrization and Reoptimization	48
7.10	Problem Modification and Reoptimization	51
7.11	Parallel optimization	55
7.12	Retrieving infeasibility certificates	56
8	Solver Interaction Tutorials	60
8.1	Accessing the solution	60
8.2	Errors and exceptions	64
8.3	Input/Output	66
8.4	Setting solver parameters	67
8.5	Retrieving information items	68

8.6	Stopping the solver	69
8.7	Progress and data callback	70
8.8	Optimizer API Task	73
8.9	MOSEK OptServer	73
9	Debugging Tutorials	75
9.1	Understanding optimizer log	75
9.2	Addressing numerical issues	80
9.3	Debugging infeasibility	82
9.4	Python Console	87
10	Technical guidelines	89
10.1	Limitations	89
10.2	Memory management and garbage collection	89
10.3	Names	90
10.4	Multithreading	91
10.5	Efficiency	91
10.6	The license system	93
10.7	Deployment	93
11	Case Studies	94
11.1	Portfolio Optimization	95
11.2	Primal Support-Vector Machine (SVM)	110
11.3	2D Total Variation	116
11.4	Multiprocessor Scheduling	120
11.5	Logistic regression	123
11.6	Inner and outer Löwner-John Ellipsoids	126
11.7	SUDOKU	129
11.8	Travelling Salesman Problem (TSP)	134
11.9	Nearest Correlation Matrix Problem	139
11.10	Semidefinite Relaxation of MIQCQO Problems	143
12	Problem Formulation and Solutions	147
12.1	Linear Optimization	147
12.2	Conic Optimization	150
12.3	Semidefinite Optimization	153
13	Optimizers	157
13.1	Presolve	157
13.2	Linear Optimization	159
13.3	Conic Optimization - Interior-point optimizer	166
13.4	The Optimizer for Mixed-Integer Problems	170
14	<i>Fusion</i> API Reference	180
14.1	<i>Fusion</i> API conventions	180
14.2	Class list	186
14.3	Parameters grouped by topic	271
14.4	Parameters (alphabetical list sorted by type)	279
14.5	Enumerations	307
14.6	Constants	309
14.7	Exceptions	333
14.8	Supported domains	338
14.9	Class LinAlg	340
15	Supported File Formats	344
15.1	The LP File Format	345
15.2	The MPS File Format	349
15.3	The OPF Format	361
15.4	The CBF Format	371

15.5	The PTF Format	388
15.6	The Task Format	394
15.7	The JSON Format	395
15.8	The Solution File Format	401
16	List of examples	404
17	Interface changes	406
17.1	Important changes compared to version 9	406
17.2	Changes compared to version 9	406
17.3	Parameters compared to version 9	406
17.4	Constants compared to version 9	407
	Bibliography	408
	Symbol Index	409
	Index	412

Chapter 1

Introduction

The **MOSEK** Optimization Suite 10.0.44 is a powerful software package capable of solving large-scale optimization problems of the following kind:

- linear,
- conic:
 - conic quadratic (also known as second-order cone),
 - involving the exponential cone,
 - involving the power cone,
 - semidefinite,
- convex quadratic and quadratically constrained,
- integer.

In order to obtain an overview of features in the **MOSEK** Optimization Suite consult the [product introduction](#) guide.

The most widespread class of optimization problems is *linear optimization problems*, where all relations are linear. The tremendous success of both applications and theory of linear optimization can be ascribed to the following factors:

- The required data are simple, i.e. just matrices and vectors.
- Convexity is guaranteed since the problem is convex by construction.
- Linear functions are trivially differentiable.
- There exist very efficient algorithms and software for solving linear problems.
- Duality properties for linear optimization are nice and simple.

Even if the linear optimization model is only an approximation to the true problem at hand, the advantages of linear optimization may outweigh the disadvantages. In some cases, however, the problem formulation is inherently nonlinear and a linear approximation is either intractable or inadequate. *Conic optimization* has proved to be a very expressive and powerful way to introduce nonlinearities, while preserving all the nice properties of linear optimization listed above.

The fundamental expression in linear optimization is a linear expression of the form

$$Ax - b \geq 0.$$

In conic optimization this is replaced with a wider class of constraints

$$Ax - b \in \mathcal{K}$$

where $\mathcal{K}$ is a *convex cone*. For example in 3 dimensions $\mathcal{K}$ may correspond to an ice cream cone. The conic optimizer in **MOSEK** supports a number of different types of cones $\mathcal{K}$, which allows a surprisingly large number of nonlinear relations to be modeled, as described in the **MOSEK** [Modeling Cookbook](#), while preserving the nice algorithmic and theoretical properties of linear optimization.

1.1 Why the Fusion API for C++?

Fusion is an object oriented API specifically designed to build conic optimization models in a simple and expressive manner, using mainstream programming languages.



With focus on usability and compactness, it helps the user focus on modeling instead of coding.

Typically a conic optimization model in *Fusion* can be developed in a fraction of the time compared to using a low-level C API, but of course *Fusion* introduces a computational overhead compared to customized C code. In most cases, however, the overhead is small compared to the overall solution time. Moreover, parametrization makes it possible to construct a *Fusion* model once and then solve it repeatedly for different inputs with almost no overhead.

We generally recommend that *Fusion* is used as a first step for building and verifying new models. Often, the final *Fusion* implementation will be directly suited for production code, and otherwise it readily provides a reference implementation for model verification. *Fusion* always yields readable and easily portable code.

The Fusion API for C++ provides access to Conic Optimization, including:

- Linear Optimization (LO)
- Conic Quadratic (Second-Order Cone) Optimization (CQO, SOCO)
- Power Cone Optimization
- Conic Exponential Optimization (CEO)
- Semidefinite Optimization (SDO)
- Mixed-Integer Optimization (MIO)

as well as to an auxiliary linear algebra library.

Convex Quadratic and Quadratically Constrained (QCQO) problems can be reformulated as Conic Quadratic problems and subsequently solved using *Fusion*. This is the recommended approach, as described in the **MOSEK Modeling Cookbook** and this [whitepaper](#).

Chapter 2

Contact Information

Phone	+45 7174 9373	
Website	mosek.com	
Email		
	sales@mosek.com	Sales, pricing, and licensing
	support@mosek.com	Technical support, questions and bug reports
	info@mosek.com	Everything else.
Mailing Address		
	MOSEK ApS	
	Fruebjergvej 3	
	Symbion Science Park, Box 16	
	2100 Copenhagen O	
	Denmark	

You can get in touch with **MOSEK** using popular social media as well:

Blogger	https://blog.mosek.com/
Google Group	https://groups.google.com/forum/#!forum/mosek
Twitter	https://twitter.com/mosektw
Linkedin	https://www.linkedin.com/company/mosek-aps
Youtube	https://www.youtube.com/channel/UCvIyectEVLp31NXeD5mIbEw

In particular **Twitter** is used for news, updates and release announcements.

Chapter 3

License Agreement

3.1 MOSEK end-user license agreement

Before using the **MOSEK** software, please read the license agreement available in the distribution at <MSKHOME>/mosek/10.0/mosek-eula.pdf or on the **MOSEK** website <https://mosek.com/products/license-agreement>. By using **MOSEK** you agree to the terms of that license agreement.

3.2 Third party licenses

MOSEK uses some third-party open-source libraries. Their license details follow.

zlib

MOSEK uses the *zlib* library obtained from the [zlib website](#). The license agreement for *zlib* is shown in [Listing 3.1](#).

Listing 3.1: *zlib* license.

```
zlib.h -- interface of the 'zlib' general purpose compression library
version 1.2.7, May 2nd, 2012

Copyright (C) 1995-2012 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implied
warranty. In no event will the authors be held liable for any damages
arising from the use of this software.

Permission is granted to anyone to use this software for any purpose,
including commercial applications, and to alter it and redistribute it
freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not
   claim that you wrote the original software. If you use this software
   in a product, an acknowledgment in the product documentation would be
   appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be
   misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

Jean-loup Gailly          Mark Adler
jloup@gzip.org            madler@alumni.caltech.edu
```


fplib

MOSEK uses the floating point formatting library developed by David M. Gay obtained from the [netlib website](#). The license agreement for *fplib* is shown in [Listing 3.2](#).

Listing 3.2: *fplib* license.

```
/*
 *
 * The author of this software is David M. Gay.
 *
 * Copyright (c) 1991, 2000, 2001 by Lucent Technologies.
 *
 * Permission to use, copy, modify, and distribute this software for any
 * purpose without fee is hereby granted, provided that this entire notice
 * is included in all copies of any software which is or includes a copy
 * or modification of this software and in all copies of the supporting
 * documentation for such software.
 *
 * THIS SOFTWARE IS BEING PROVIDED "AS IS", WITHOUT ANY EXPRESS OR IMPLIED
 * WARRANTY. IN PARTICULAR, NEITHER THE AUTHOR NOR LUCENT MAKES ANY
 * REPRESENTATION OR WARRANTY OF ANY KIND CONCERNING THE MERCHANTABILITY
 * OF THIS SOFTWARE OR ITS FITNESS FOR ANY PARTICULAR PURPOSE.
 *
 *****/
```

{fmt}

MOSEK uses the formatting library *{fmt}* developed by Victor Zverovich obtained from [github/fmt](#) and distributed under the MIT license. The license agreement for *{fmt}* is shown in [Listing 3.3](#).

Listing 3.3: *{fmt}* license.

```
Copyright (c) 2012 - present, Victor Zverovich

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the "Software"),
to deal in the Software without restriction, including without limitation
the rights to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell copies of the Software, and to permit persons to whom the Software
is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED,
INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR
A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR
COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Zstandard

MOSEK uses the *Zstandard* library developed by Facebook obtained from [github/zstd](https://github.com/facebook/zstd). The license agreement for *Zstandard* is shown in [Listing 3.4](#).

Listing 3.4: *Zstandard* license.

```
BSD License

For Zstandard software

Copyright (c) 2016-present, Facebook, Inc. All rights reserved.

Redistribution and use in source and binary forms, with or without modification,
are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this
  list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright notice,
  this list of conditions and the following disclaimer in the documentation
  and/or other materials provided with the distribution.

* Neither the name Facebook nor the names of its contributors may be used to
  endorse or promote products derived from this software without specific
  prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR
ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

OpenSSL

MOSEK uses the [LibReSSL](#) library, which is build on *OpenSSL*. *OpenSSL* is included under the *OpenSSL* license, [Listing 3.5](#), and the *LibReSSL* additions are licensed under the *ISC* license, [Listing 3.6](#).

Listing 3.5: *OpenSSL* license

```
=====
Copyright (c) 1998-2011 The OpenSSL Project. All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions
are met:

1. Redistributions of source code must retain the above copyright
   notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright
   notice, this list of conditions and the following disclaimer in
```

(continues on next page)

the documentation and/or other materials provided with the distribution.

3. All advertising materials mentioning features or use of this software must display the following acknowledgment:
"This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit. (<http://www.openssl.org/>)"
4. The names "OpenSSL Toolkit" and "OpenSSL Project" must not be used to endorse or promote products derived from this software without prior written permission. For written permission, please contact openssl-core@openssl.org.
5. Products derived from this software may not be called "OpenSSL" nor may "OpenSSL" appear in their names without prior written permission of the OpenSSL Project.
6. Redistributions of any form whatsoever must retain the following acknowledgment:
"This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit (<http://www.openssl.org/>)"

THIS SOFTWARE IS PROVIDED BY THE OpenSSL PROJECT ``AS IS'' AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE OpenSSL PROJECT OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

=====

This product includes cryptographic software written by Eric Young (ey@cryptsoft.com). This product includes software written by Tim Hudson (tjh@cryptsoft.com).

Listing 3.6: ISC license

Copyright (C) 1994-2017 Free Software Foundation, Inc.
Copyright (c) 2014 Jeremie Courreges-Anglas <jca@openbsd.org>
Copyright (c) 2014-2015 Joel Sing <jsing@openbsd.org>
Copyright (c) 2014 Ted Unangst <tedu@openbsd.org>
Copyright (c) 2015-2016 Bob Beck <beck@openbsd.org>
Copyright (c) 2015 Marko Kreen <markokr@gmail.com>
Copyright (c) 2015 Reyk Floeter <reyk@openbsd.org>
Copyright (c) 2016 Tobias Pape <tobias@netshed.de>

Permission to use, copy, modify, and/or distribute this software for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.

(continued from previous page)

```
THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL
WARRANTIES WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE
AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL
DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR
PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER
TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

mimalloc

MOSEK uses the *mimalloc* memory allocator library from [github/mimalloc](https://github.com/mimalloc). The license agreement for *mimalloc* is shown in [Listing 3.7](#).

Listing 3.7: *mimalloc* license.

```
MIT License

Copyright (c) 2019 Microsoft Corporation, Daan Leijen

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

BLASFEO

MOSEK uses the *BLASFEO* linear algebra library developed by Gianluca Frison, obtained from [github/blasfeo](https://github.com/blasfeo). The license agreement for *BLASFEO* is shown in [Listing 3.8](#).

Listing 3.8: *blasfeo* license.

```
BLASFEO -- BLAS For Embedded Optimization.
Copyright (C) 2019 by Gianluca Frison.
Developed at IMTEK (University of Freiburg) under the supervision of Moritz Diehl.
All rights reserved.

The 2-Clause BSD License

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this
```

(continues on next page)

list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

oneTBB

MOSEK uses the *oneTBB* parallelization library which is part of *oneAPI* developed by Intel, obtained from [github/oneTBB](https://github.com/oneTBB), licensed under the Apache License 2.0. The license agreement for *oneTBB* can be found in <https://github.com/oneapi-src/oneTBB/blob/master/LICENSE.txt> .

Chapter 4

Installation

In this section we discuss how to install and setup the **MOSEK** Fusion API for C++.

Important: Before running this **MOSEK** interface please make sure that you:

- Installed **MOSEK** correctly. Some operating systems require extra steps. See the [Installation guide](#) for instructions and common troubleshooting tips.
 - Set up a license. See the [Licensing guide](#) for instructions.
-

Compatibility

The Fusion API for C++ is compatible with the following compiler tool chains:

Platform	Supported compiler	Framework
Linux 64 bit x86	gcc (≥ 4.5)	glibc (≥ 2.17)
Linux 64 bit ARM	clang (≥ 10)	glibc (≥ 2.29)
macOS 64 bit x86	Xcode (≥ 11)	MAC OS SDK (≥ 10.15)
macOS 64 bit ARM	Xcode (≥ 12)	MAC OS SDK (≥ 11)
Windows 64 bit x86	Visual Studio (≥ 2017)	

In many cases older versions can also be used. In particular Fusion API for C++ requires a C++11 compliant compiler.

Locating files in the MOSEK Optimization Suite

The relevant files of the Fusion API for C++ are organized as reported in [Table 4.1](#).

Table 4.1: Relevant files for the Fusion API for C++.

Relative Path	Description	Label
<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/h	Header files	<HEADERDIR>
<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/src/fusion_cxx	Source files	<SRCDIR>
<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/bin	Shared libraries	<LIBDIR>
<MSKHOME>/mosek/10.0/tools/examples/fusion/cxx	Examples	<EXDIR>
<MSKHOME>/mosek/10.0/tools/examples/fusion/data	Additional data	<MISCDIR>

where

- <MSKHOME> is the folder in which the **MOSEK** Optimization Suite has been installed,
- <PLATFORM> is the actual platform among those supported by **MOSEK**, i.e. win64x86, linux64x86 or osx64x86.

Manual compilation (all platforms)

This step is compulsory on Linux and Mac OS. The implementation of *Fusion* is distributed as C++ source code in `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/src/fusion_cxx`. It can be compiled as follows:

1. Go to `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/src/fusion_cxx`
2. Run `make install` (Linux, macOS) or `nmake install` (Windows).
3. If no error occurs, then the *Fusion* C++ API has been successfully compiled and the corresponding libraries have been copied to `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/bin`.

Using a pre-compiled library (only Windows)

On Windows 64bit the users can skip the compilation step and use a pre-compiled library `fusion64_10_0.lib` available in `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/bin`.

Setting up paths and linking

To compile and link a C++ application using *Fusion*, the user must set paths to header files, compiled *Fusion* libraries, and run-time dependencies must be resolved. Details vary depending on the operating system and compiler. See the `Makefile` included in the distribution under `<MSKHOME>/mosek/10.0/tools/examples/fusion/cxx` for a working example. Typically:

- Linux:

```
g++ -std=c++11 file.cc -o file -I<HEADERDIR> -L<LIBDIR> -Wl,-rpath-link,<LIBDIR>
↪ -Wl,-rpath=<LIBDIR> -lmosek64 -lfusion64
```

The shared libraries `libmosek64.so.10.0`, `libfusion64.so.10.0` must be available at runtime.

- macOS:

```
clang++ -std=c++11 -stdlib=libc++ file.cc -o file -I<HEADERDIR> -L<LIBDIR> -Wl,
↪ -headerpad,128 -lmosek64 -lfusion64
install_name_tool -change libmosek64.10.0.dylib <LIBDIR>/libmosek64.10.0.dylib
↪ file
install_name_tool -change libfusion64.10.0.dylib <LIBDIR>/libfusion64.10.0.dylib
↪ file
```

The shared libraries `libmosek64.10.0.dylib`, `libfusion64.10.0.dylib` must be available at runtime.

- Windows:

```
cl /I<HEADERDIR> file.cc /link /LIBPATH:<LIBDIR> fusion64_10_0.lib mosek64_10_0.
↪ lib
```

The shared library `mosek64_10_0.dll` must be available at runtime.

Importing the source code

Alternatively, the *Fusion* source code from `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/src/fusion_cxx` can be imported into any other project, or compiled into a stand-alone library by the user. This is especially useful if non-standard compiler options should be used.

4.1 Testing the installation and compiling examples

The example directory `<MSKHOME>/mosek/10.0/tools/examples/fusion/cxx` contains a makefile for use with `make` (Linux, macOS) or `nmake` (Windows).

To build the examples, open a shell (Linux, macOS) or a Developer Command Prompt and go to `<EXDIR>`. To compile all examples run the one of the commands

```
make all  
  
nmake all
```

depending on the operating system. To build only a single example, for instance `lo1.cc`, use one of the following:

```
make lo1  
  
nmake lo1.exe
```

4.2 Creating a Visual Studio Project

The following walk-through describes how to set up a 64bit *Fusion* project with Microsoft Visual Studio 2019. Similar steps should be followed with other tools and setup configurations.

- Create a Visual C++ project or open an existing project in Visual Studio.
- Go to **Project** → **Properties**.
- In the selection box **Configuration:** select **Release** and in **Platform:** select **x64**.
- Go to **Configuration Manager** and set **Active solution configuration** to **Release** and **Active solution platform** to **x64**.
- Under **Configuration Properties** → **C/C++** → **General** → **Additional Include Directories** add the full path to `<HEADERDIR>`.
- Under **Configuration Properties** → **C/C++** → **Code Generation** → **Runtime library** select `(/MD)`.
- Under **Configuration Properties** → **Linker** → **Input** → **Additional Dependencies** add the full paths to the files `mosek64_10_0.lib` and `fusion64_10_0.lib`.
- To ensure that `mosek64_10_0.dll` is available in the DLL search path during execution, set **Configuration Properties** → **Debugging** → **Environment** to `PATH=%PATH%;<LIBDIR>`.

4.3 Troubleshooting

error LNK2038

The distributed *Fusion* library is built in `/MD` mode. Attempts to link it with an application built in a different mode, like `/MT`, `/Mdd`, `/MTd`, will result in errors such as:

```
error LNK2038: mismatch detected for 'RuntimeLibrary': value 'MD_DynamicRelease' doesn't match value 'Mdd_DynamicDebug' in lo1.obj
```

However, if you must use different configurations for your project, then you may import the *Fusion* source code from `<MSKHOME>/mosek/10.0/tools/platform/<PLATFORM>/src/fusion_cxx` directly into your project and compile within the project with whatever compiler and linker settings you require.

Chapter 5

Design Overview

Fusion is a result of many years of experience in conic optimization. It is a dedicated API for users who want to enjoy a simpler experience interfacing with the solver. This applies to users who regularly solve conic problems, and to new users who do not want to be too bothered with the technicalities of a low-level optimizer. *Fusion* is designed for fast and clean prototyping of conic problems without suffering excessive performance degradation.

Note that *Fusion* **is** an object-oriented framework for conic-optimization but it **is not** a general purpose modeling language. The main design principles of *Fusion* are:

- **Expressiveness:** we try to make it nice! Despite not being a modeling language, *Fusion* yields readable, easy to maintain code that closely resembles the mathematical formulation of the problem.
- **Seamlessly multi-language :** *Fusion* code can be ported across C++, Python, Java, .NET and with only minimal adaptations to the syntax of each language.
- **What you write is what MOSEK gets:** A *Fusion* model is fed into the solver with (almost) no additional transformations.

Expressiveness

Suppose you have a conic quadratic optimization problem like the efficient frontier in portfolio optimization:

$$\begin{aligned} & \text{maximize} && \mu^T x - \alpha \gamma \\ & \text{subject to} && e^T x = w, \\ & && \gamma \geq \|G^T x\|, \\ & && x \geq 0. \end{aligned}$$

where μ, G are input data and α is an input parameter whose value we want to change between many optimizations. Its representation in *Fusion* is a direct translation of the mathematical model and could look as follows:

```
auto x = M->variable(n);
auto gamma = M->variable();
auto alpha = M->parameter();

M->objective(ObjectiveSense::Maximize, Expr::sub(Expr::dot(mu, x), Expr::
↪mul(alpha, gamma)));

M->constraint(Expr::sub(Expr::sum(x), w), Domain::equalsTo(0.0));
M->constraint(Expr::vstack(gamma, Expr::mul(G->transpose(), x)), Domain::
↪inQCone());
M->constraint(x, Domain::greaterThan(0.0));
```

Seamless multi-language API

Fusion can easily be ported across the five supported languages. All functionalities and naming conventions remain the same in all of them. This has some advantages:

- Simplifies code sharing between developers working in different languages.
- Improves code reusability.
- Simplifies the transition from R&D to production (for instance from fast-prototyping languages used in R&D to more efficient ones used for high performance).

Here is the same code snippet (creation of a variable in the model) in all languages supported by *Fusion*. Careful code design can generate models with only the necessary syntactic differences between implementations.

```
auto x= M->variable("x", 3, Domain::greaterThan(0.0));           // C++
```

```
x = M.variable('x', 3, Domain.greaterThan(0.0))                 # Python
```

```
Variable x = M.variable("x", 3, Domain.greaterThan(0.0))        // Java
```

```
Variable x = M.Variable("x", 3, Domain.GreaterThan(0.0))        // C#
```

What You Write is What MOSEK Gets

Fusion is not a modeling language. Instead it clearly defines the formulation the user must adhere to and only provides functionalities required for that formulation. Users familiar with the concept of DCP (Disciplined Convex Programming) can think of *Fusion* as a language for VDCP - Very Disciplined Convex Programming.

An important upshot is that *Fusion* will not modify the problem provided by the user any more than is required to fit it into the form accepted by the low-level optimizer. In other words, the problem that is actually solved is as close as possible to what the user writes. For example, *Fusion* will transform a multi-dimensional constraint into a sequence of scalar constraints for the linear constraint matrix A , and so on. So, in effect, the *Fusion* mechanism only automates operations that the user would have to carry out anyway (using pencil and paper, presumably). Otherwise the optimizer model is a direct copy of the *Fusion* model.

The main benefits of this approach are:

- The user knows what problem is actually being solved.
- Dual information is readily available for all variables and constraints.
- Only the necessary overhead.
- Better control over numerical stability.

Chapter 6

Conic Modeling

6.1 The model

A model built using *Fusion* is **always** a conic optimization problem and it is convex by definition. These problems can be succinctly characterized as

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && Ax + b \in \mathcal{K} \end{aligned} \tag{6.1}$$

where $\mathcal{K}$ is a product of domains supported by **MOSEK**, in particular:

- *linear*: $\mathbb{R}, \mathbb{R}_+, \{0\}$,
- *quadratic*: $\mathcal{Q}^n = \{x \in \mathbb{R}^n : x_1 \geq \sqrt{x_2^2 + \dots + x_n^2}\}$,
- *rotated quadratic*: $\mathcal{Q}_r^n = \{x \in \mathbb{R}^n : 2x_1x_2 \geq x_3^2 + \dots + x_n^2, x_1, x_2 \geq 0\}$,
- *primal power cone*: $\mathcal{P}_n^{\alpha, 1-\alpha} = \{x \in \mathbb{R}^n : x_1^\alpha x_2^{1-\alpha} \geq \sqrt{x_3^2 + \dots + x_n^2}, x_1, x_2 \geq 0\}$, or its dual,
- *primal exponential*: $K_{\text{exp}} = \{x \in \mathbb{R}^3 : x_1 \geq x_2 \exp(x_3/x_2), x_1, x_2 \geq 0\}$, or its dual,
- *semidefinite*: $\mathcal{S}_+^n = \{X \in \mathbb{R}^{n \times n} : X \text{ is symmetric positive semidefinite}\}$.
- and others, see [Sec. 14.8](#) for a full list.

The main thing about a *Fusion* model is that it can be specified in a convenient way without explicitly constructing the representation (6.1). Instead the user has access to *variables* which are used to construct *linear operators* that appear in *constraints*. The cone types described above are the domains of those constraints. A *Fusion* model can potentially contain many different building blocks of that kind. To facilitate manipulations with a large number of variables *Fusion* defines various *logical views* of parts of the model. To facilitate reoptimizing the same problem with varying input data *Fusion* provides *parameters*.

This section briefly summarizes the constructions and techniques available in *Fusion*. See [Sec. 7](#) for a basic tutorial and [Sec. 11](#) for more advanced case studies. This section is only an introduction: detailed specification of the methods and classes mentioned here can be found in the [API reference](#).

A *Fusion* model is represented by the class *Model* and created by a simple construction

```
Model::t M = new Model(); auto _M = finally([&] () { M->dispose(); });
```

The model object is the user's interface to the optimization problem, used in particular for

- formulating the problem by defining variables, parameters, constraints and objective,
- solving the problem and retrieving the solution status and solutions,
- interacting with the solver: setting up parameters, registering for callbacks, performing I/O, obtaining detailed information from the optimizer etc.

Almost all elements of the model: variables, parameters, constraints and the model itself can be constructed with or without names. If used, the names for each type of object must be unique. Choosing a good naming convention can make the problem more readable when dumped to a file.

6.2 Variables

Continuous variables can be scalars, vectors or higher-dimensional arrays. They are added to the model with the method `Model.variable` which returns a representing object of type `Variable`. The shape of a variable (number of dimensions and length in each dimension) has to be specified at creation. Optionally a variable may be created in a restricted domain (by default variables are unbounded, that is in $\mathbb{R}$). For instance, to declare a variable $x \in \mathbb{R}_+^n$ we could write

```
auto x = M->variable("x", n, Domain::greaterThan(0.));
```

A multi-dimensional variable is declared by specifying an array with all dimension sizes. Here is an $n \times n$ variable:

```
auto x = M->variable( new_array_ptr<int,1>({n,n}), Domain::unbounded() );
```

The specification of dimensions can also be part of the domain, as in this declaration of a symmetric positive semidefinite variable of dimension n :

```
auto v = M->variable(Domain::inPSDCone(n));
```

Integer variables are specified with an additional domain modifier. To add an integer variable $z \in [1, 10]$ we write

```
auto z = M->variable("z", Domain::integral(Domain::inRange(1.,10.)) );
```

The function `Domain.binary` is a shorthand for binary variables often appearing in combinatorial problems:

```
auto y = M->variable("y", Domain::binary());
```

Integrality requirement can be switched on and off using the methods `Variable.makeInteger` and `Variable.makeContinuous`.

A domain usually allows to specify the number of objects to be created. For example here is a definition of m symmetric positive semidefinite variables of dimension n each. The actual variable `x` will be of shape $m \times n \times n$ where each slice with fixed first coordinate is an $n \times n$ PSD:

```
auto x = M->variable(Domain::inPSDCone(n, m));
```

The `Variable` object provides the primal (`Variable.level`) and dual (`Variable.dual`) solution values of the variable after optimization, and it enters in the construction of linear expressions involving the variable.

6.3 Expressions and linear operators

Linear expressions are constructed combining *variables*, *parameters*, *matrices* and other constant values by linear operators. The result is an object that represents the linear expression itself. *Fusion* only allows for those combinations of operators and arguments that yield linear functions of the variables. Expressions have shapes and dimensions in the same fashion as variables. For instance, if $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{m \times n}$, then Ax is a vector expression of length m . Note, however, that the internal size of Ax is mn , because each entry is a linear combination for which m coefficients have to be stored.

Expressions are concrete implementations of the virtual interface `Expression`. In typical situations, however, all operations on expressions can be performed using the static methods and factory methods of the class `Expr`.

Table 6.1: Linear Operators

Method	Description
<i>Expr.add</i>	Element-wise addition of two matrices
<i>Expr.sub</i>	Element-wise subtraction of two matrices
<i>Expr.mul</i>	Matrix or matrix-scalar multiplication
<i>Expr.neg</i>	Sign inversion
<i>Expr.outer</i>	Vector outer-product
<i>Expr.dot</i>	Dot product
<i>Expr.sum</i>	Sum over a given dimension
<i>Expr.mulElm</i>	Element-wise multiplication
<i>Expr.mulDiag</i>	Sum over the diagonal of a matrix which is the result of a matrix multiplication
<i>Expr.constTerm</i>	Return a <i>constant term</i>

Operations on expressions must adhere to the rules of matrix algebra regarding dimensions; otherwise a *DimensionError* exception will be thrown.

Expression can be composed, nested and used as building blocks in new expressions. For instance $Ax + By$ can be implemented as:

```
Expr::add( Expr::mul(A,x), Expr::mul(B,y) );
```

For operations involving multiple variables and expressions the users should consider list-based methods. For instance, a clean way to write $x + y + z + w$ would be:

```
Expr::add( new_array_ptr<Variable::t,1>({x, y, z, w}));
```

Note that a single variable (object of class *Variable*) can also be used as an expression. Once constructed, expressions are immutable.

6.4 Constraints and objective

Constraints are declared within an optimization model using the method *Model.constraint*. Every constraint in *Fusion* has the form

Expression belongs to a *Domain*.

Objects of type *Domain* correspond roughly to the types of convex cones $\mathcal{K}$ mentioned at the beginning of this section. For instance, the following set of linear constraints

$$\begin{array}{rcl} x_1 & + & 2x_2 & = & 0 \\ & & + & x_2 & + & x_3 & = & 0 \\ x_1 & & & & & & = & 0 \end{array} \quad (6.2)$$

could be declared as

```
auto A = new_array_ptr<double,2>({
    { 1.0, 2.0, 0.0},
    { 0.0, 1.0, 1.0},
    { 1.0, 0.0, 0.0} });

auto x = M->variable("x",3,Domain::unbounded());
auto c = M->constraint( Expr::mul(A,x), Domain::equalsTo(0.0));
```

Note that the scalar domain *Domain.equalsTo* consisting of a single point 0 scales up to the dimension of the expression and applies to all its elements. This allows many constraints to be comfortably expressed in a vectorized form. See also [Sec. 6.8](#).

The *Constraint* object provides the dual (*Constraint.dual*) value of the constraint after optimization and the primal value of the constraint expression (*Constraint.level*).

The typical domains used to specify constraints are listed below. Note that they can also be used directly at variable creation, whenever that makes sense.

	Type	Domain
Linear	equality	<i>Domain.equalsTo</i>
	inequality $\leq$	<i>Domain.lessThan</i>
	inequality $\geq$	<i>Domain.greaterThan</i>
	two-sided bound	<i>Domain.inRange</i>
Conic Quadratic	quadratic cone	<i>Domain.inQCone</i>
	rotated quadratic cone	<i>Domain.inRotatedQCone</i>
Other Conic	exponential cone	<i>Domain.inPExpCone</i>
	power cone	<i>Domain.inPPowerCone</i>
	geometric mean	<i>Domain.inPGeoMeanCone</i>
Semidefinite	PSD matrix	<i>Domain.inPSDCone</i>
Integral	Integers in domain D	<i>Domain.integral</i> (D)
	$\{0, 1\}$	<i>Domain.binary</i>

See Sec. 14.8 and the API reference for *Domain* for a full list.

Having discussed variables and constraints we can finish by defining the optimization objective with *Model.objective*. The objective function is an affine expression that evaluates to a scalar (that is, of shape () or (1)) and the objective sense is specified by the enumeration *ObjectiveSense* as either *minimize* or *maximize*. The typical linear objective function $c^T x$ can be declared as

```
M->objective( ObjectiveSense::Minimize, Expr::dot(c,x) );
```

6.5 Matrices

At some point it becomes necessary to specify linear expressions such as Ax where A is a (large) constant data matrix. Such coefficient matrices can be represented in dense or sparse format. Dense matrices can always be represented using the standard data structures for arrays and two-dimensional arrays built into the language. Alternatively, or when sparsity can be exploited, matrices can be constructed as objects of the class *Matrix*. This can have some advantages: a more generic code that can be ported across platforms and can be used with *both* dense and sparse matrices without modifications.

Dense matrices are constructed with a variant of the static factory method *Matrix.dense*. The values of all entries must be specified all at once and the resulting matrix is immutable. For example the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$$

can be defined with:

```
auto A = new_array_ptr<double,2>({ {1,2,3,4}, {5,6,7,8} });
auto Ad= Matrix::dense(A);
```

or from a flattened representation:

```
auto Af = new_array_ptr<double,1>({ 1,2,3,4,5,6,7,8 });
auto Aff= Matrix::dense(2,4,Af);
```

Sparse matrices are constructed with a variant of the static factory method *Matrix.sparse*. This is both speed- and memory-efficient when the matrix has few nonzero entries. A matrix A in sparse format is given by a list of triples (i, j, v) , each defining one entry: $A_{i,j} = v$. The order does not matter. The entries not in the list are assumed to be 0. For example, take the matrix

$$A = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 2.0 \\ 0.0 & 3.0 & 0.0 & 4.0 \end{bmatrix}.$$

Assuming we number rows and columns from 0, the corresponding list of triplets is:

$$A = \{(0, 0, 1.0), (0, 3, 2.0), (1, 1, 3.0), (1, 3, 4.0)\}$$

The *Fusion* definition would be:

```

auto rows    = new_array_ptr<int,1>({ 0, 0, 1, 1 });
auto cols    = new_array_ptr<int,1>({ 0, 3, 1, 3 });
auto values  = new_array_ptr<double,1>({ 1.0, 2.0, 3.0, 4.0 });

auto ms = Matrix::sparse(2, 4, rows, cols, values);

```

The *Matrix* class provides more standard constructions such as the identity matrix, a constant value matrix, block diagonal matrices etc.

6.6 Parameters

A parameter (*Parameter*) is a placeholder for a constant whose value should be specified before the model is optimized. Parameters can have arbitrary shapes, just like variables, and can be used in any place where using a constant, array or matrix of the same shape would be suitable. That means parameters behave like expressions under additive operations and stacking, and can additionally be used in some multiplicative operations where the result is affine in the optimization variables.

For example, we can create a parametrized constraint

$$p^T x + q \leq 0,$$

where $x \in \mathbb{R}^4$, as follows:

```

Variable::t x = M->variable("x", 4);    // Variable

Parameter::t p = M->parameter("p", 4); // Parameter of shape [ 4 ]
Parameter::t q = M->parameter();        // Scalar parameter

M->constraint(Expr::add(Expr::dot(p, x), q), Domain::lessThan(0.0));

```

Later in the code we can initialize the parameters with actual values. For example

```

p->setValue(new_array_ptr<double,1>({ 1.0, 2.0 , 3.0, 4.0 }));
q->setValue(5.0);

```

will make the previously defined constraint evaluate to

$$x_1 + 2x_2 + 3x_3 + 4x_4 + 5 \leq 0.$$

The values of parameters can be changed between optimizations. Therefore one parametrized model with fixed structure can be used to solve many instances of the same optimization problem with varying input data.

6.7 Stacking and views

Fusion provides a way to construct logical views of parts of existing expressions or combinations of existing expressions. They are still represented by objects of type *Variable* or *Expression* that refer to the original ones. This can be useful in some scenarios:

- retrieving only the values of a few variables, and ignoring the remaining auxiliary ones,
- stacking vectors or matrices to perform various matrix operations,
- bundling a number of similar constraints into one; see [Sec. 6.8](#),
- adding constraints between parts of the same variable, etc.

All these operations do not require *new* variables or expressions, but just lightweight *logical views*. In what follows we will concentrate on expressions; the same techniques are available for variables. These techniques will be familiar to the users of numerical tools such as Matlab or NumPy.

Picking and slicing

Expression.pick picks a subset of entries from a variable or expression. Special cases of picking are *Expression.index*, which picks just one scalar entry and *Expression.slice* which picks a *slice*, that is restricts each dimension to a subinterval. Slicing is a frequently used operation.

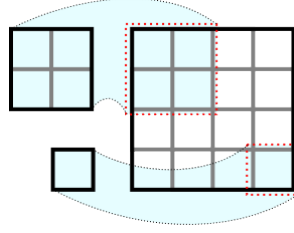


Fig. 6.1: Two dimensional slicing.

Both displayed regions are slices of the two-dimensional 4×4 expression, which can be selected as follows:

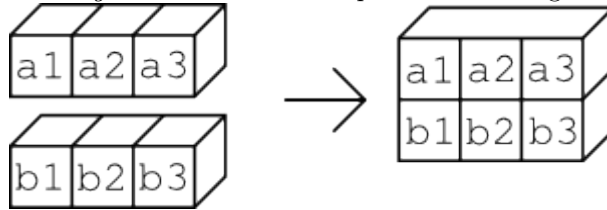
```
auto Axs1 = Ax->slice( new_array_ptr<int,1>({0,0}), new_array_ptr<int,1>({2,2}) );
auto Axs2 = Ax->index( new_array_ptr<int,1>({3,3}) );
```

Reshaping

Expressions can be *reshaped* creating a view with the same number of coordinates arranged in a different way. A particular example of this operation is *flattening*, which converts any multi-dimensional expression into a one-dimensional vector.

Stacking

Stacking refers to the concatenation of expressions to form a new larger one. For example, the next figure depicts the *vertical stacking* of two vectors of shape 1×3 resulting in a matrix of shape 2×3 .



```
auto c = Expr::vstack(new_array_ptr<Expression::t,1>({a,b}));
```

Vertical stacking (*Expr.vstack*) of expressions of shapes $d_1 \times d_2$ and $d'_1 \times d_2$ has shape $(d_1 + d'_1) \times d_2$. Similarly, *horizontal stacking* (*Expr.hstack*) of expressions of shapes $d_1 \times d_2$ and $d_1 \times d'_2$ has shape $d_1 \times (d_2 + d'_2)$. *Fusion* supports also more general versions of stacking for multi-dimensional variables, as described in *Expr.stack*. A special case of stacking is *repetition* (*Expr.repeat*), equivalent to stacking copies of the same expression.

6.8 Vectorization

Using *Fusion* one can compactly express sequences of similar constraints. For example, if we want to express

$$Ax_i = b_i, \quad i = 1, \dots, n$$

we can think of $x_i \in \mathbb{R}^m, b_i \in \mathbb{R}^k$ as the columns of two matrices $X = [x_1, \dots, x_n] \in \mathbb{R}^{m \times n}$, $B = [b_1, \dots, b_n] \in \mathbb{R}^{k \times n}$, and write simply

$$AX - B = 0.$$


```

auto X = Var::hstack( new_array_ptr<Variable::t,1>({xi(0), xi(1), xi(2), xi(3)})
↳);
auto B = Expr::hstack( new_array_ptr<Expression::t,1>({bi(0), bi(1), bi(2), bi(3)}
↳) );

M->constraint(Expr::sub(Expr::mul(A, X), B), Domain::equalsTo(0.0));

```

In this example the domain *Domain.equalsTo* scales to apply to all the entries of the expression.

Another powerful case of vectorization and scaling domains is the ability to define a sequence of conic constraints in one go. Suppose we want to find an upper bound on the 2-norm of a sequence of vectors, that is we want to express

$$t \geq \|y_i\|, \quad i = 1, \dots, n$$

Suppose that the vectors y_i are arranged in the rows of a matrix Y . Then we can simply write:

```

auto t = M->variable();

M->constraint(Expr::hstack(Var::vrepeat(t, n), Y), Domain::inQCone());

```

Here, again, the conic domain *Domain.inQCone* is by default applied to each row of the matrix separately, yielding the desired constraints in a loop-free way (the i -th row is (t, y_i)). The direction along which conic constraints are created within multi-dimensional expressions can be changed with *Domain.axis*.

We recommend vectorizing the code whenever possible. It is not only more elegant and portable but also more efficient — loops are eliminated and the number of *Fusion* API calls is reduced.

6.9 Reoptimization

Between optimizations the user can modify the model in a few ways:

- Set/change values of parameters (*Parameter.setValue*). This is the recommended way to reoptimize multiple models identical structure and varying (parts of) input data. For simplicity, suppose we want to minimize $f(x) = \gamma x + \beta y$, for varying choices of $\gamma > 0$. Then we could write:

```

double gammaValues[] = {0., 0.5, 1.0};           // Choices for gamma
double beta = 2.0;
auto x = M->variable("x", 1, Domain::greaterThan(0.));
auto y = M->variable("y", 1, Domain::greaterThan(0.));
auto gamma = M->parameter("gamma");

M->objective( ObjectiveSense::Minimize, Expr::add(Expr::mul(gamma, x), Expr::
↳:mul(beta, y)) );

for(auto g : gammaValues)
{
    gamma->setValue(g);
    M->solve();
}

```

- Add new constraints with *Model.constraint*. This is useful for solving a sequence of optimization problems with more and more restrictions on the feasible set. See for example [Sec. 11.8](#).
- Add new variables with *Model.variable* or parameters with *Model.parameter*.
- Replace the objective with a completely new one (*Model.objective*).
- Update part of the objective (*Model.updateObjective*).
- Update an existing constraint or replace the constraint expression with a new one (*Constraint.update*).

Otherwise all *Fusion* objects are immutable. See also [Sec. 7.10](#) for more reoptimization examples.

Chapter 7

Optimization Tutorials

In this section we demonstrate how to set up basic types of optimization problems. Each short tutorial contains a working example of formulating problems, defining variables and constraints and retrieving solutions.

- **Model setup and linear optimization tutorial (LO)**

- [Sec. 7.1](#). Linear optimization tutorial, *recommended first reading for all users*. Apart from setting up a linear problem it also demonstrates how to work with a Fusion model: initialize it, add variables and constraints and retrieve the solution.

- **Conic optimization tutorials (CO)**

Basic examples demonstrating various types of conic constraints.

- [Sec. 7.2](#). A basic example with a quadratic cone (CQO).
- [Sec. 7.3](#). A basic example with a power cone.
- [Sec. 7.4](#). A basic example with a exponential cone (CEO).
- [Sec. 7.5](#). A basic tutorial of geometric programming (GP).

- **Semidefinite optimization tutorial (SDO)**

- [Sec. 7.6](#). Examples showing how to solve semidefinite optimization problems with one or more semidefinite variables.

- **Mixed-integer optimization tutorials (MIO)**

- [Sec. 7.7](#). Shows how to declare integer variables for linear and conic problems and how to set an initial solution.
- [Sec. 7.8](#). Demonstrates how to create a problem with disjunctive constraints (DJC).

- **Reoptimization tutorials**

- [Sec. 7.9](#). Shows how to construct a parameterized model.
- [Sec. 7.10](#). Other techniques for modifying the model.

- **Parallel optimization tutorial**

- [Sec. 7.11](#). Shows how to optimize models in parallel.

- **Infeasibility certificates**

- [Sec. 7.12](#). Shows how to retrieve and analyze a primal infeasibility certificate for continuous problems.

7.1 Linear Optimization

The simplest optimization problem is a purely linear problem. A *linear optimization problem* is a problem of the following form:

Minimize or maximize the objective function

$$\sum_{j=0}^{n-1} c_j x_j + c^f$$

subject to the linear constraints

$$l_k^c \leq \sum_{j=0}^{n-1} a_{kj} x_j \leq u_k^c, \quad k = 0, \dots, m-1,$$

and the bounds

$$l_j^x \leq x_j \leq u_j^x, \quad j = 0, \dots, n-1.$$

The problem description consists of the following elements:

- m and n — the number of constraints and variables, respectively,
- x — the variable vector of length n ,
- c — the coefficient vector of length n

$$c = \begin{bmatrix} c_0 \\ \vdots \\ c_{n-1} \end{bmatrix},$$

- c^f — fixed term in the objective,
- A — an $m \times n$ matrix of coefficients

$$A = \begin{bmatrix} a_{0,0} & \cdots & a_{0,(n-1)} \\ \vdots & \cdots & \vdots \\ a_{(m-1),0} & \cdots & a_{(m-1),(n-1)} \end{bmatrix},$$

- l^c and u^c — the lower and upper bounds on constraints,
- l^x and u^x — the lower and upper bounds on variables.

Please note that we are using 0 as the first index: x_0 is the first element in variable vector x .

The *Fusion* user does not need to specify all of the above elements explicitly — they will be assembled from the *Fusion* model.

7.1.1 Example LO1

The following is an example of a small linear optimization problem:

$$\begin{array}{llllll} \text{maximize} & 3x_0 & + & 1x_1 & + & 5x_2 & + & 1x_3 \\ \text{subject to} & 3x_0 & + & 1x_1 & + & 2x_2 & & = & 30, \\ & 2x_0 & + & 1x_1 & + & 3x_2 & + & 1x_3 & \geq & 15, \\ & & & 2x_1 & & & + & 3x_3 & \leq & 25, \end{array} \tag{7.1}$$

under the bounds

$$\begin{array}{llll} 0 & \leq & x_0 & \leq & \infty, \\ 0 & \leq & x_1 & \leq & 10, \\ 0 & \leq & x_2 & \leq & \infty, \\ 0 & \leq & x_3 & \leq & \infty. \end{array}$$

We start our implementation in *Fusion* importing the relevant modules, i.e.

```
#include "fusion.h"
using namespace mosek::fusion;
using namespace monty;
```

Next we declare an optimization model creating an instance of the *Model* class:

```
Model::t M = new Model("lo1"); auto _M = finally([&]() { M->dispose(); });
```

For this simple problem we are going to enter all the linear coefficients directly:

```
auto A1 = new_array_ptr<double, 1>({ 3.0, 1.0, 2.0, 0.0 });
auto A2 = new_array_ptr<double, 1>({ 2.0, 1.0, 3.0, 1.0 });
auto A3 = new_array_ptr<double, 1>({ 0.0, 2.0, 0.0, 3.0 });
auto c = new_array_ptr<double, 1>({ 3.0, 1.0, 5.0, 1.0 });
```

The variables appearing in problem (7.1) can be declared as one 4-dimensional variable:

```
Variable::t x = M->variable("x", 4, Domain::greaterThan(0.0));
```

At this point we already have variables with bounds $0 \leq x_i \leq \infty$, because the domain is applied element-wise to the entries of the variable vector. Next, we impose the upper bound on x_1 :

```
M->constraint(x->index(1), Domain::lessThan(10.0));
```

The linear constraints can now be entered one by one using the dot product of our variable with a coefficient vector:

```
M->constraint("c1", Expr::dot(A1, x), Domain::equalTo(30.0));
M->constraint("c2", Expr::dot(A2, x), Domain::greaterThan(15.0));
M->constraint("c3", Expr::dot(A3, x), Domain::lessThan(25.0));
```

We end the definition of our optimization model setting the objective function in the same way:

```
M->objective("obj", ObjectiveSense::Maximize, Expr::dot(c, x));
```

Finally, we only need to call the *Model.solve* method:

```
M->solve();
```

The solution values can be attained with the method *Variable.level*.

```
auto sol = x->level();
std::cout << "[x0,x1,x2,x3] = " << (*sol) << "\n";
```

Listing 7.1: *Fusion* implementation of model (7.1).

```
#include <iostream>
#include "fusion.h"
using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    auto A1 = new_array_ptr<double, 1>({ 3.0, 1.0, 2.0, 0.0 });
    auto A2 = new_array_ptr<double, 1>({ 2.0, 1.0, 3.0, 1.0 });
    auto A3 = new_array_ptr<double, 1>({ 0.0, 2.0, 0.0, 3.0 });
    auto c = new_array_ptr<double, 1>({ 3.0, 1.0, 5.0, 1.0 });

    // Create a model with the name 'lo1'
    Model::t M = new Model("lo1"); auto _M = finally([&]() { M->dispose(); });

    M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush; }_
    ↪);
```

(continues on next page)

```

// Create variable 'x' of length 4
Variable::t x = M->variable("x", 4, Domain::greaterThan(0.0));

// Create constraints
M->constraint(x->index(1), Domain::lessThan(10.0));
M->constraint("c1", Expr::dot(A1, x), Domain::equalsTo(30.0));
M->constraint("c2", Expr::dot(A2, x), Domain::greaterThan(15.0));
M->constraint("c3", Expr::dot(A3, x), Domain::lessThan(25.0));

// Set the objective function to (c^t * x)
M->objective("obj", ObjectiveSense::Maximize, Expr::dot(c, x));

// Solve the problem
M->solve();

// Get the solution values
auto sol = x->level();
std::cout << "[x0,x1,x2,x3] = " << (*sol) << "\n";
}

```

7.2 Conic Quadratic Optimization

The structure of a typical conic optimization problem is

$$\begin{aligned}
 & \text{minimize} && c^T x + c^f \\
 & \text{subject to} && l^c \leq Ax \leq u^c, \\
 & && l^x \leq x \leq u^x, \\
 & && Fx + g \in \mathcal{D},
 \end{aligned}$$

(see [Sec. 12](#) for detailed formulations). Here we discuss how to set-up problems with the **(rotated) quadratic cones**.

MOSEK supports two types of quadratic cones, namely:

- Quadratic cone:

$$\mathcal{Q}^n = \left\{ x \in \mathbb{R}^n : x_0 \geq \sqrt{\sum_{j=1}^{n-1} x_j^2} \right\}.$$

- Rotated quadratic cone:

$$\mathcal{Q}_r^n = \left\{ x \in \mathbb{R}^n : 2x_0x_1 \geq \sum_{j=2}^{n-1} x_j^2, \quad x_0 \geq 0, \quad x_1 \geq 0 \right\}.$$

For example, consider the following constraint:

$$(x_4, x_0, x_2) \in \mathcal{Q}^3$$

which describes a convex cone in $\mathbb{R}^3$ given by the inequality:

$$x_4 \geq \sqrt{x_0^2 + x_2^2}.$$

For other types of cones supported by **MOSEK**, see [Sec. 14.8](#) and the other tutorials in this chapter. Different cone types can appear together in one optimization problem.

7.2.1 Example CQO1

Consider the following conic quadratic problem which involves some linear constraints, a quadratic cone and a rotated quadratic cone.

$$\begin{aligned}
 &\text{minimize} && y_1 + y_2 + y_3 \\
 &\text{subject to} && x_1 + x_2 + 2.0x_3 = 1.0, \\
 & && x_1, x_2, x_3 \geq 0.0, \\
 & && (y_1, x_1, x_2) \in \mathcal{Q}^3, \\
 & && (y_2, y_3, x_3) \in \mathcal{Q}_r^3.
 \end{aligned} \tag{7.2}$$

We start by creating the optimization model:

```
Model::t M = new Model("cqo1"); auto _M = finally([&]() { M->dispose(); });
```

We then define variables `x` and `y`. Two logical variables (aliases) `z1` and `z2` are introduced to model the quadratic cones. These are not new variables, but map onto parts of `x` and `y` for the sake of convenience.

```
Variable::t x = M->variable("x", 3, Domain::greaterThan(0.0));
Variable::t y = M->variable("y", 3, Domain::unbounded());

// Create the aliases
//      z1 = [ y[0], x[0], x[1] ]
// and z2 = [ y[1], y[2], x[2] ]
Variable::t z1 = Var::vstack(y->index(0), x->slice(0, 2));
Variable::t z2 = Var::vstack(y->slice(1, 3), x->index(2));
```

The linear constraint is defined using the dot product:

```
// Create the constraint
//      x[0] + x[1] + 2.0 x[2] = 1.0
auto aval = new_array_ptr<double>, 1>({1.0, 1.0, 2.0});
M->constraint("lc", Expr::dot(aval, x), Domain::equalsTo(1.0));
```

The conic constraints are defined using the logical views `z1` and `z2` created previously. Note that this is a basic way of defining conic constraints, and that in practice they would have more complicated structure.

```
// Create the constraints
//      z1 belongs to C_3
//      z2 belongs to K_3
// where C_3 and K_3 are respectively the quadratic and
// rotated quadratic cone of size 3, i.e.
//      z1[0] >= sqrt(z1[1]^2 + z1[2]^2)
// and 2.0 z2[0] z2[1] >= z2[2]^2
Constraint::t qc1 = M->constraint("qc1", z1, Domain::inQCone());
Constraint::t qc2 = M->constraint("qc2", z2, Domain::inRotatedQCone());
```

We only need the objective function:

```
// Set the objective function to (y[0] + y[1] + y[2])
M->objective("obj", ObjectiveSense::Minimize, Expr::sum(y));
```

Calling the `Model.solve` method invokes the solver:

```
M->solve();
```

The primal and dual solution values can be retrieved using `Variable.level`, `Constraint.level` and `Variable.dual`, `Constraint.dual`, respectively:

```
// Get the linear solution values
ndarray<double>, 1> xlvl = *(x->level());
ndarray<double>, 1> ylvl = *(y->level());
```

```
// Get conic solution of qc1
ndarray<double, 1> qc1lvl = *(qc1->level());
ndarray<double, 1> qc1dl = *(qc1->dual());
```

Listing 7.2: *Fusion* implementation of model (7.2).

```
#include <iostream>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    Model::t M = new Model("cq01"); auto _M = finally([&]() { M->dispose(); });

    Variable::t x = M->variable("x", 3, Domain::greaterThan(0.0));
    Variable::t y = M->variable("y", 3, Domain::unbounded());

    // Create the aliases
    //      z1 = [ y[0], x[0], x[1] ]
    //      and z2 = [ y[1], y[2], x[2] ]
    Variable::t z1 = Var::vstack(y->index(0), x->slice(0, 2));
    Variable::t z2 = Var::vstack(y->slice(1, 3), x->index(2));

    // Create the constraint
    //      x[0] + x[1] + 2.0 x[2] = 1.0
    auto aval = new_array_ptr<double, 1>({1.0, 1.0, 2.0});
    M->constraint("lc", Expr::dot(aval, x), Domain::equalsTo(1.0));

    // Create the constraints
    //      z1 belongs to C_3
    //      z2 belongs to K_3
    // where C_3 and K_3 are respectively the quadratic and
    // rotated quadratic cone of size 3, i.e.
    //      z1[0] >= sqrt(z1[1]^2 + z1[2]^2)
    //      and 2.0 z2[0] z2[1] >= z2[2]^2
    Constraint::t qc1 = M->constraint("qc1", z1, Domain::inQCone());
    Constraint::t qc2 = M->constraint("qc2", z2, Domain::inRotatedQCone());

    // Set the objective function to (y[0] + y[1] + y[2])
    M->objective("obj", ObjectiveSense::Minimize, Expr::sum(y));

    // Solve the problem
    M->solve();

    // Get the linear solution values
    ndarray<double, 1> xlvl = *(x->level());
    ndarray<double, 1> ylvl = *(y->level());
    // Get conic solution of qc1
    ndarray<double, 1> qc1lvl = *(qc1->level());
    ndarray<double, 1> qc1dl = *(qc1->dual());

    std::cout << "x1,x2,x3 = " << xlvl << std::endl;
    std::cout << "y1,y2,y3 = " << ylvl << std::endl;
    std::cout << "qc1 levels = " << qc1lvl << std::endl;
    std::cout << "qc1 dual conic var levels = " << qc1dl << std::endl;
```

(continues on next page)

}

7.3 Power Cone Optimization

The structure of a typical conic optimization problem is

$$\begin{array}{llll} \text{minimize} & & c^T x + c^f \\ \text{subject to} & l^c \leq & Ax & \leq u^c, \\ & l^x \leq & x & \leq u^x, \\ & & Fx + g & \in \mathcal{D}, \end{array}$$

(see [Sec. 12](#) for detailed formulations). Here we discuss how to set-up problems with the **primal/dual power cones**.

MOSEK supports the primal and dual power cones, defined as below:

- Primal power cone:

$$\mathcal{P}_n^{\alpha_k} = \left\{ x \in \mathbb{R}^n : \prod_{i=0}^{n_\ell-1} x_i^{\beta_i} \geq \sqrt{\sum_{j=n_\ell}^{n-1} x_j^2}, x_0, \dots, x_{n_\ell-1} \geq 0 \right\}$$

where $s = \sum_i \alpha_i$ and $\beta_i = \alpha_i/s$, so that $\sum_i \beta_i = 1$.

- Dual power cone:

$$(\mathcal{P}_n^{\alpha_k})^* = \left\{ x \in \mathbb{R}^n : \prod_{i=0}^{n_\ell-1} \left(\frac{x_i}{\beta_i} \right)^{\beta_i} \geq \sqrt{\sum_{j=n_\ell}^{n-1} x_j^2}, x_0, \dots, x_{n_\ell-1} \geq 0 \right\}$$

where $s = \sum_i \alpha_i$ and $\beta_i = \alpha_i/s$, so that $\sum_i \beta_i = 1$.

Perhaps the most important special case is the three-dimensional power cone family:

$$\mathcal{P}_3^{\alpha, 1-\alpha} = \{x \in \mathbb{R}^3 : x_0^\alpha x_1^{1-\alpha} \geq |x_2|, x_0, x_1 \geq 0\}.$$

which has the corresponding dual cone:

For example, the conic constraint $(x, y, z) \in \mathcal{P}_3^{0.25, 0.75}$ is equivalent to $x^{0.25}y^{0.75} \geq |z|$, or simply $xy^3 \geq z^4$ with $x, y \geq 0$.

For other types of cones supported by **MOSEK**, see [Sec. 14.8](#) and the other tutorials in this chapter. Different cone types can appear together in one optimization problem.

7.3.1 Example POW1

Consider the following optimization problem which involves powers of variables:

$$\begin{array}{ll} \text{maximize} & x_0^{0.2} x_1^{0.8} + x_2^{0.4} - x_0 \\ \text{subject to} & x_0 + x_1 + \frac{1}{2}x_2 = 2, \\ & x_0, x_1, x_2 \geq 0. \end{array} \tag{7.3}$$

We convert (7.3) into affine conic form using auxiliary variables as bounds for the power expressions:

$$\begin{array}{ll} \text{maximize} & x_3 + x_4 - x_0 \\ \text{subject to} & x_0 + x_1 + \frac{1}{2}x_2 = 2, \\ & (x_0, x_1, x_3) \in \mathcal{P}_3^{0.2, 0.8}, \\ & (x_2, 1.0, x_4) \in \mathcal{P}_3^{0.4, 0.6}. \end{array} \tag{7.4}$$

The two conic constraints shown in (7.4) can be expressed in the ACC form as shown in (7.5):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \in \mathcal{P}_3^{0.2,0.8} \times \mathcal{P}_3^{0.4,0.6}. \quad (7.5)$$

We start by creating the optimization model:

```
Model::t M = new Model("pow1"); auto _M = finally([&]() { M->dispose(); });
```

We then define the variable `x` corresponding to the original problem (7.3), and auxiliary variables appearing in the conic reformulation (7.4).

```
Variable::t x = M->variable("x", 3, Domain::unbounded());
Variable::t x3 = M->variable();
Variable::t x4 = M->variable();
```

The linear constraint is defined using the dot product operator *Expr.dot*:

```
// Create the linear constraint
auto aval = new_array_ptr<double>, 1>({1.0, 1.0, 0.5});
M->constraint(Expr::dot(x, aval), Domain::equalsTo(2.0));
```

The primal power cone is referred to via *Domain.inPPowerCone* with an appropriate list of variables or expressions in each case.

```
// Create the conic constraints
M->constraint(Var::vstack(x->slice(0,2), x3), Domain::inPPowerCone(0.2));
M->constraint(Expr::vstack(x->index(2), 1.0, x4), Domain::inPPowerCone(0.4));
```

We only need the objective function:

```
auto cval = new_array_ptr<double>, 1>({1.0, 1.0, -1.0});
M->objective(ObjectiveSense::Maximize, Expr::dot(cval, Var::vstack(x3, x4, x->
->index(0))));
```

Calling the *Model.solve* method invokes the solver:

```
M->solve();
```

The primal and dual solution values can be retrieved using *Variable.level*, *Constraint.level* and *Variable.dual*, *Constraint.dual*. Here we just display the primal solution

```
ndarray<double>, 1> xlv1 = *(x->level());
std::cout << "x,y,z = " << xlv1 << std::endl;
```

which is

```
[ 0.06389298  0.78308564  2.30604283 ]
```

Listing 7.3: *Fusion* implementation of model (7.3).

```
#include <iostream>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
```

(continues on next page)

```

Model::t M = new Model("pow1"); auto _M = finally([&]() { M->dispose(); });

Variable::t x = M->variable("x", 3, Domain::unbounded());
Variable::t x3 = M->variable();
Variable::t x4 = M->variable();

// Create the linear constraint
auto aval = new_array_ptr<double, 1>({1.0, 1.0, 0.5});
M->constraint(Expr::dot(x, aval), Domain::equalsTo(2.0));

// Create the conic constraints
M->constraint(Var::vstack(x->slice(0,2), x3), Domain::inPPowerCone(0.2));
M->constraint(Expr::vstack(x->index(2), 1.0, x4), Domain::inPPowerCone(0.4));

auto cval = new_array_ptr<double, 1>({1.0, 1.0, -1.0});
M->objective(ObjectiveSense::Maximize, Expr::dot(cval, Var::vstack(x3, x4, x->
->index(0))));

// Solve the problem
M->solve();

// Get the linear solution values
ndarray<double, 1> xlv1 = *(x->level());
std::cout << "x,y,z = " << xlv1 << std::endl;
}

```

7.4 Conic Exponential Optimization

The structure of a typical conic optimization problem is

$$\begin{aligned}
 & \text{minimize} && c^T x + c^f \\
 & \text{subject to} && l^c \leq Ax \leq u^c, \\
 & && l^x \leq x \leq u^x, \\
 & && Fx + g \in \mathcal{D},
 \end{aligned}$$

(see [Sec. 12](#) for detailed formulations). Here we discuss how to set-up problems with the **primal/dual exponential cones**.

MOSEK supports two exponential cones, namely:

- Primal exponential cone:

$$K_{\text{exp}} = \{x \in \mathbb{R}^3 : x_0 \geq x_1 \exp(x_2/x_1), x_0, x_1 \geq 0\}.$$

- Dual exponential cone:

$$K_{\text{exp}}^* = \{s \in \mathbb{R}^3 : s_0 \geq -s_2 e^{-1} \exp(s_1/s_2), s_2 \leq 0, s_0 \geq 0\}.$$

For example, consider the following constraint:

$$(x_4, x_0, x_2) \in K_{\text{exp}}$$

which describes a convex cone in $\mathbb{R}^3$ given by the inequalities:

$$x_4 \geq x_0 \exp(x_2/x_0), x_0, x_4 \geq 0.$$

For other types of cones supported by **MOSEK**, see [Sec. 14.8](#) and the other tutorials in this chapter. Different cone types can appear together in one optimization problem.

7.4.1 Example CEO1

Consider the following basic conic exponential problem which involves some linear constraints and an exponential inequality:

$$\begin{aligned} & \text{minimize} && x_0 + x_1 \\ & \text{subject to} && x_0 + x_1 + x_2 = 1, \\ & && x_0 \geq x_1 \exp(x_2/x_1), \\ & && x_0, x_1 \geq 0. \end{aligned} \tag{7.6}$$

The affine conic form of (7.6) is:

$$\begin{aligned} & \text{minimize} && x_0 + x_1 \\ & \text{subject to} && x_0 + x_1 + x_2 = 1, \\ & && Ix \in K_{\text{exp}}, \\ & && x \in \mathbb{R}^3. \end{aligned} \tag{7.7}$$

where I is the 3×3 identity matrix.

We start by creating the optimization model:

```
Model::t M = new Model("ceo1"); auto _M = finally([&]() { M->dispose(); });
```

We then define the variable x .

```
Variable::t x = M->variable("x", 3, Domain::unbounded());
```

The linear constraint is defined using the sum operator *Expr.sum*:

```
// Create the constraint
//      x[0] + x[1] + x[2] = 1.0
M->constraint("lc", Expr::sum(x), Domain::equalsTo(1.0));
```

The conic exponential constraint in this case is very simple as it involves just the variable x . The primal exponential cone is referred to via *Domain.inPExpCone*, and it must be applied to a variable of length 3 or an array of such variables. Note that this is a basic way of defining conic constraints, and that in practice they would have more complicated structure.

```
// Create the exponential conic constraint
Constraint::t expc = M->constraint("expc", x, Domain::inPExpCone());
```

We only need the objective function:

```
// Set the objective function to (x[0] + x[1])
M->objective("obj", ObjectiveSense::Minimize, Expr::sum(x->slice(0,2)));
```

Calling the *Model.solve* method invokes the solver:

```
M->solve();
```

The primal and dual solution values can be retrieved using *Variable.level*, *Constraint.level* and *Variable.dual*, *Constraint.dual*, respectively:

```
// Get the linear solution values
ndarray<double, 1> xlv1 = *(x->level());
```

```
// Get conic solution of expc1
ndarray<double, 1> expclvl = *(expc->level());
ndarray<double, 1> expcdl = *(expc->dual());
```

Listing 7.4: *Fusion* implementation of model (7.6).

```
#include <iostream>
#include "fusion.h"
```

(continues on next page)

```

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    Model::t M = new Model("ceo1"); auto _M = finally([&]() { M->dispose(); });

    Variable::t x = M->variable("x", 3, Domain::unbounded());

    // Create the constraint
    //      x[0] + x[1] + x[2] = 1.0
    M->constraint("lc", Expr::sum(x), Domain::equalsTo(1.0));

    // Create the exponential conic constraint
    Constraint::t expc = M->constraint("expc", x, Domain::inPExpCone());

    // Set the objective function to (x[0] + x[1])
    M->objective("obj", ObjectiveSense::Minimize, Expr::sum(x->slice(0,2)));

    // Solve the problem
    M->solve();

    // Get the linear solution values
    ndarray<double, 1> xlv1 = *(x->level());
    // Get conic solution of expc1
    ndarray<double, 1> expclvl = *(expc->level());
    ndarray<double, 1> expcdl = *(expc->dual());

    std::cout << "x1,x2,x3 = " << xlv1 << std::endl;
    std::cout << "expc levels = " << expclvl << std::endl;
    std::cout << "expc dual conic var levels = " << expcdl << std::endl;
}

```

7.5 Geometric Programming

Geometric programs (GP) are a particular class of optimization problems which can be expressed in special polynomial form as positive sums of generalized monomials. More precisely, a geometric problem in canonical form is

$$\begin{aligned}
 &\text{minimize} && f_0(x) \\
 &\text{subject to} && f_i(x) \leq 1, \quad i = 1, \dots, m, \\
 & && x_j > 0, \quad j = 1, \dots, n,
 \end{aligned} \tag{7.8}$$

where each $f_0, \dots, f_m$ is a *posynomial*, that is a function of the form

$$f(x) = \sum_k c_k x_1^{\alpha_{k1}} x_2^{\alpha_{k2}} \dots x_n^{\alpha_{kn}}$$

with arbitrary real α_{ki} and $c_k > 0$. The standard way to formulate GPs in convex form is to introduce a variable substitution

$$x_i = \exp(y_i).$$

Under this substitution all constraints in a GP can be reduced to the form

$$\log\left(\sum_k \exp(a_k^T y + b_k)\right) \leq 0 \tag{7.9}$$

involving a *log-sum-exp* bound. Moreover, constraints involving only a single monomial in x can be even more simply written as a linear inequality:

$$a_k^T y + b_k \leq 0$$

We refer to the **MOSEK Modeling Cookbook** and to [BKVH07] for more details on this reformulation. A geometric problem formulated in convex form can be entered into **MOSEK** with the help of exponential cones.

7.5.1 Example GP1

The following problem comes from [BKVH07]. Consider maximizing the volume of a $h \times w \times d$ box subject to upper bounds on the area of the floor and of the walls and bounds on the ratios h/w and d/w :

$$\begin{aligned} & \text{maximize} && hwd \\ & \text{subject to} && 2(hw + hd) \leq A_{\text{wall}}, \\ & && wd \leq A_{\text{floor}}, \\ & && \alpha \leq h/w \leq \beta, \\ & && \gamma \leq d/w \leq \delta. \end{aligned} \tag{7.10}$$

The decision variables in the problem are h, w, d . We make a substitution

$$h = \exp(x), w = \exp(y), d = \exp(z)$$

after which (7.10) becomes

$$\begin{aligned} & \text{maximize} && x + y + z \\ & \text{subject to} && \log(\exp(x + y + \log(2/A_{\text{wall}})) + \exp(x + z + \log(2/A_{\text{wall}}))) \leq 0, \\ & && y + z \leq \log(A_{\text{floor}}), \\ & && \log(\alpha) \leq x - y \leq \log(\beta), \\ & && \log(\gamma) \leq z - y \leq \log(\delta). \end{aligned} \tag{7.11}$$

Next, we demonstrate how to implement a log-sum-exp constraint (7.9). It can be written as:

$$u_k \geq \exp(a_k^T y + b_k), \quad (\text{equiv. } (u_k, 1, a_k^T y + b_k) \in K_{\text{exp}}), \tag{7.12}$$

$$\sum_k u_k = 1.$$

This presentation requires one extra variable u_k for each monomial appearing in the original posynomial constraint.

Listing 7.5: Implementation of log-sum-exp as in (7.12).

```
// Models log(sum(exp(Ax+b))) <= 0.
// Each row of [A b] describes one of the exp-terms
void logsumexp(Model::t M,
               std::shared_ptr<ndarray<double, 2>> A,
               Variable::t x,
               std::shared_ptr<ndarray<double, 1>> b)
{
    int k = A->size(0);
    auto u = M->variable(k);
    M->constraint(Expr::sum(u), Domain::equalsTo(1.0));
    M->constraint(Expr::hstack(u,
                               Expr::constTerm(k, 1.0),
                               Expr::add(Expr::mul(A, x), b)), Domain::inPExpCone());
}
```

We can now use this function to assemble all constraints in the model. The linear part of the problem is entered as in Sec. 7.1.

Listing 7.6: Source code solving problem (7.11).

```
std::shared_ptr<ndarray<double, 1>> max_volume_box(double Aw, double Af,
                                                    double alpha, double beta, double
                                                    gamma, double delta)
{
    Model::t M = new Model("max_vol_box"); auto _M = finally([&]() { M->dispose(); });

    auto xyz = M->variable(3);
    M->objective("Objective", ObjectiveSense::Maximize, Expr::sum(xyz));

    logsumexp(M,
               new_array_ptr<double,2>({{1,1,0}, {1,0,1}}),
               xyz,
               new_array_ptr<double,1>({log(2.0/Aw), log(2.0/Aw)}));

    M->constraint(Expr::dot(new_array_ptr<double,1>({0,1,1}), xyz), Domain::
    lessThan(log(Af)));
    M->constraint(Expr::dot(new_array_ptr<double,1>({1,-1,0}), xyz), Domain::
    inRange(log(alpha),log(beta)));
    M->constraint(Expr::dot(new_array_ptr<double,1>({0,-1,1}), xyz), Domain::
    inRange(log(gamma),log(delta)));

    M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; } );
    M->solve();

    return std::make_shared<ndarray<double, 1>>(shape(3), [xyz](ptrdiff_t i) { return
    exp((*(xyz->level()))[i]); });
}
```

Given sample data we obtain the solution h, w, d as follows:

Listing 7.7: Sample data for problem (7.10).

```
int main()
{
    double Aw    = 200.0;
    double Af    = 50.0;
    double alpha = 2.0;
    double beta  = 10.0;
    double gamma = 2.0;
    double delta = 10.0;

    auto hwd = max_volume_box(Aw, Af, alpha, beta, gamma, delta);

    std::cout << std::setprecision(4);
    std::cout << "h=" << (*hwd)[0] << " w=" << (*hwd)[1] << " d=" << (*hwd)[2] << std::
    endl;
}
```

7.6 Semidefinite Optimization

Semidefinite optimization is a generalization of conic optimization, allowing the use of matrix variables belonging to the convex cone of positive semidefinite matrices

$$\mathcal{S}_+^r = \{X \in \mathcal{S}^r : z^T X z \geq 0, \quad \forall z \in \mathbb{R}^r\},$$

where $\mathcal{S}^r$ is the set of $r \times r$ real-valued symmetric matrices.

MOSEK can solve semidefinite optimization problems stated in the **primal** form,

$$\begin{aligned} & \text{minimize} && \sum_{j=0}^{p-1} \langle \overline{C}_j, \overline{X}_j \rangle + \sum_{j=0}^{n-1} c_j x_j + c^f \\ & \text{subject to} && l_i^c \leq \sum_{j=0}^{p-1} \langle \overline{A}_{ij}, \overline{X}_j \rangle + \sum_{j=0}^{n-1} a_{ij} x_j \leq u_i^c, \quad i = 0, \dots, m-1, \\ & && \sum_{j=0}^{p-1} \langle \overline{F}_{ij}, \overline{X}_j \rangle + \sum_{j=0}^{n-1} f_{ij} x_j + g_i \in \mathcal{K}_i, \quad i = 0, \dots, q-1, \\ & && l_j^x \leq \frac{x_j}{x_j} \leq u_j^x, \quad j = 0, \dots, n-1, \\ & && x \in \mathcal{K}, \overline{X}_j \in \mathcal{S}_+^{r_j}, \quad j = 0, \dots, p-1 \end{aligned} \quad (7.13)$$

where the problem has p symmetric positive semidefinite variables $\overline{X}_j \in \mathcal{S}_+^{r_j}$ of dimension r_j . The symmetric coefficient matrices $\overline{C}_j \in \mathcal{S}^{r_j}$ and $\overline{A}_{i,j} \in \mathcal{S}^{r_j}$ are used to specify PSD terms in the linear objective and the linear constraints, respectively. The symmetric coefficient matrices $\overline{F}_{i,j} \in \mathcal{S}^{r_j}$ are used to specify PSD terms in the affine conic constraints. Note that q ((7.13)) is the total dimension of all the cones, i.e. $q = \dim(\mathcal{K}_1 \times \dots \times \mathcal{K}_k)$, given there are k ACCs. We use standard notation for the matrix inner product, i.e., for $A, B \in \mathbb{R}^{m \times n}$ we have

$$\langle A, B \rangle := \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} A_{ij} B_{ij}.$$

In addition to the primal form presented above, semidefinite problems can be expressed in their **dual** form. Constraints in this form are usually called **linear matrix inequalities** (LMIs). LMIs can be easily specified in **MOSEK** using the vectorized positive semidefinite cone which is defined as:

- Vectorized semidefinite domain:

$$\mathcal{S}_+^{d,\text{vec}} = \{(x_1, \dots, x_{d(d+1)/2}) \in \mathbb{R}^n : \text{sMat}(x) \in \mathcal{S}_+^d\},$$

where $n = d(d+1)/2$ and,

$$\text{sMat}(x) = \begin{bmatrix} x_1 & x_2/\sqrt{2} & \cdots & x_d/\sqrt{2} \\ x_2/\sqrt{2} & x_{d+1} & \cdots & x_{2d-1}/\sqrt{2} \\ \cdots & \cdots & \cdots & \cdots \\ x_d/\sqrt{2} & x_{2d-1}/\sqrt{2} & \cdots & x_{d(d+1)/2} \end{bmatrix},$$

or equivalently

$$\mathcal{S}_+^{d,\text{vec}} = \{\text{sVec}(X) : X \in \mathcal{S}_+^d\},$$

where

$$\text{sVec}(X) = (X_{11}, \sqrt{2}X_{21}, \dots, \sqrt{2}X_{d1}, X_{22}, \sqrt{2}X_{32}, \dots, X_{dd}).$$

In other words, the domain consists of vectorizations of the lower-triangular part of a positive semidefinite matrix, with the non-diagonal elements additionally rescaled. LMIs can be expressed by restricting appropriate affine expressions to this cone type.

For other types of cones supported by **MOSEK**, see [Sec. 14.8](#) and the other tutorials in this chapter. Different cone types can appear together in one optimization problem.

In *Fusion* the user can enter the linear expressions in a more convenient way, without having to cast the problem exactly in the above form.

We demonstrate the setup of semidefinite variables and their coefficient matrices in the following examples:

- [Sec. 7.6.1](#): A problem with one semidefinite variable and linear and conic constraints.
- [Sec. 7.6.2](#): A problem with two semidefinite variables with a linear constraint and bound.
- [Sec. 7.6.3](#): Shows how to efficiently set up many semidefinite variables of the same dimension.

7.6.1 Example SDO1

We consider the simple optimization problem with semidefinite and conic quadratic constraints:

$$\begin{aligned}
& \text{minimize} && \left\langle \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \bar{X} \right\rangle + x_0 \\
& \text{subject to} && \left\langle \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \bar{X} \right\rangle + x_0 = 1, \\
& && \left\langle \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \bar{X} \right\rangle + x_1 + x_2 = 1/2, \\
& && x_0 \geq \sqrt{x_1^2 + x_2^2}, \quad \bar{X} \succeq 0,
\end{aligned} \tag{7.14}$$

The problem description contains a 3-dimensional symmetric semidefinite variable which can be written explicitly as:

$$\bar{X} = \begin{bmatrix} \bar{X}_{00} & \bar{X}_{10} & \bar{X}_{20} \\ \bar{X}_{10} & \bar{X}_{11} & \bar{X}_{21} \\ \bar{X}_{20} & \bar{X}_{21} & \bar{X}_{22} \end{bmatrix} \in \mathcal{S}_+^3,$$

and an affine conic constraint (ACC) $(x_0, x_1, x_2) \in \mathcal{Q}^3$. The objective is to minimize

$$2(\bar{X}_{00} + \bar{X}_{10} + \bar{X}_{11} + \bar{X}_{21} + \bar{X}_{22}) + x_0,$$

subject to the two linear constraints

$$\begin{aligned}
& \bar{X}_{00} + \bar{X}_{11} + \bar{X}_{22} + x_0 = 1, \\
& \bar{X}_{00} + \bar{X}_{11} + \bar{X}_{22} + 2(\bar{X}_{10} + \bar{X}_{20} + \bar{X}_{21}) + x_1 + x_2 = 1/2.
\end{aligned}$$

Our implementation in *Fusion* begins with creating a new model:

```
Model::t M = new Model("sdo1"); auto _M = finally([&]() { M->dispose(); });
```

We create a symmetric semidefinite variable $\bar{X}$ and another variable representing x . For simplicity we immediately declare that x belongs to a quadratic cone

```
Variable::t X = M->variable("X", Domain::inPSDCone(3));
Variable::t x = M->variable("x", Domain::inQCone(3));
```

In this elementary example we are going to create an explicit matrix representation of the problem

$$\bar{C} = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \quad \bar{A}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \bar{A}_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

and use it in the model via the dot product operation $\langle \cdot, \cdot \rangle$ which applies to matrices as well as to vectors. This way we create each of the linear constraints and the objective as one expression.

```
// Objective
M->objective(ObjectiveSense::Minimize, Expr::add(Expr::dot(C, X), x->index(0)));

// Constraints
M->constraint("c1", Expr::add(Expr::dot(A1, X), x->index(0)), Domain::equalsTo(1.
->0));
M->constraint("c2", Expr::add(Expr::dot(A2, X), Expr::sum(x->slice(1, 3))), Domain::
->equalsTo(0.5));
```

Now it remains to solve the problem with *Model.solve*.

Listing 7.8: *Fusion* implementation of problem (7.14).

```
#include <iostream>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    Model::t M = new Model("sdo1"); auto _M = finally([&]() { M->dispose(); });

    // Setting up the variables
    Variable::t X = M->variable("X", Domain::inPSDCone(3));
    Variable::t x = M->variable("x", Domain::inQCone(3));

    // Setting up the constant coefficient matrices
    Matrix::t C = Matrix::dense ( new_array_ptr<double, 2>({{2., 1., 0.}, {1., 2., 1.},
    ↪ {0., 1., 2.}}));
    Matrix::t A1 = Matrix::eye(3);
    Matrix::t A2 = Matrix::ones(3, 3);

    // Objective
    M->objective(ObjectiveSense::Minimize, Expr::add(Expr::dot(C, X), x->index(0)));

    // Constraints
    M->constraint("c1", Expr::add(Expr::dot(A1, X), x->index(0)), Domain::equalsTo(1.
    ↪ 0));
    M->constraint("c2", Expr::add(Expr::dot(A2, X), Expr::sum(x->slice(1, 3))), Domain::
    ↪ equalsTo(0.5));

    M->solve();

    std::cout << "Solution : " << std::endl;
    std::cout << "  X = " << *(X->level()) << std::endl;
    std::cout << "  x = " << *(x->level()) << std::endl;

    return 0;
}
```

7.6.2 Example SDO2

We now demonstrate how to define more than one semidefinite variable using the following problem with two matrix variables and two types of constraints:

$$\begin{aligned}
 & \text{minimize} && \langle C_1, \overline{X}_1 \rangle + \langle C_2, \overline{X}_2 \rangle \\
 & \text{subject to} && \langle A_1, \overline{X}_1 \rangle + \langle A_2, \overline{X}_2 \rangle = b, \\
 & && (\overline{X}_2)_{01} \leq k, \\
 & && \overline{X}_1, \overline{X}_2 \succeq 0.
 \end{aligned} \tag{7.15}$$

In our example $\dim(\overline{X}_1) = 3$, $\dim(\overline{X}_2) = 4$, $b = 23$, $k = -3$ and

$$\begin{aligned}
 C_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 6 \end{bmatrix}, A_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 2 \end{bmatrix}, \\
 C_2 &= \begin{bmatrix} 1 & -3 & 0 & 0 \\ -3 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, A_2 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -3 \end{bmatrix},
 \end{aligned}$$

are constant symmetric matrices.

Note that this problem does not contain any scalar variables, but they could be added in the same fashion as in Sec. 7.6.1.

The code representing the above problem is shown below.

Listing 7.9: Implementation of model (7.15).

```
#include <iostream>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

std::shared_ptr<ndarray<int,1>> nint(const std::vector<int> &X) { return new_
    ↪array_ptr<int>(X); }
std::shared_ptr<ndarray<double,1>> ndou(const std::vector<double> &X) { return new_
    ↪array_ptr<double>(X); }

int main(int argc, char ** argv)
{
    // Sample data in sparse, symmetric triplet format
    std::vector<int> C1_k = {0, 2};
    std::vector<int> C1_l = {0, 2};
    std::vector<double> C1_v = {1, 6};
    std::vector<int> A1_k = {0, 2, 0, 2};
    std::vector<int> A1_l = {0, 0, 2, 2};
    std::vector<double> A1_v = {1, 1, 1, 2};
    std::vector<int> C2_k = {0, 1, 0, 1, 2};
    std::vector<int> C2_l = {0, 0, 1, 1, 2};
    std::vector<double> C2_v = {1, -3, -3, 2, 1};
    std::vector<int> A2_k = {1, 0, 1, 3};
    std::vector<int> A2_l = {0, 1, 1, 3};
    std::vector<double> A2_v = {1, 1, -1, -3};
    double b = 23;
    double k = -3;

    // Convert input data into Fusion sparse matrices
    auto C1 = Matrix::sparse(3, 3, nint(C1_k), nint(C1_l), ndou(C1_v));
    auto C2 = Matrix::sparse(4, 4, nint(C2_k), nint(C2_l), ndou(C2_v));
    auto A1 = Matrix::sparse(3, 3, nint(A1_k), nint(A1_l), ndou(A1_v));
    auto A2 = Matrix::sparse(4, 4, nint(A2_k), nint(A2_l), ndou(A2_v));

    // Create model
    Model::t M = new Model("sdo2"); auto _M = finally([&]() { M->dispose(); });

    // Two semidefinite variables
    auto X1 = M->variable(Domain::inPSDCone(3));
    auto X2 = M->variable(Domain::inPSDCone(4));

    // Objective
    M->objective(ObjectiveSense::Minimize, Expr::add(Expr::dot(C1,X1), Expr::dot(C2,
    ↪X2)));

    // Equality constraint
    M->constraint(Expr::add(Expr::dot(A1,X1), Expr::dot(A2,X2)), Domain::equalsTo(b));

    // Inequality constraint
    M->constraint(X2->index(nint({0,1})), Domain::lessThan(k));
}
```

(continues on next page)

```

// Solve
M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush;
→} );
M->solve();

// Retrieve solution
std::cout << "Solution (vectorized) : " << std::endl;
std::cout << "  X1 = " << *(X1->level()) << std::endl;
std::cout << "  X2 = " << *(X2->level()) << std::endl;

return 0;
}

```

7.6.3 Example SDO3

Here we demonstrate how to use the facilities provided in *Fusion* to set up a model with many semidefinite variables of the same dimension more efficiently than via looping. We consider a problem with n semidefinite variables of dimension d and k constraints:

$$\begin{aligned}
 & \text{minimize} && \sum_j \text{tr}(\bar{X}_j) \\
 & \text{subject to} && \sum_j \langle A_{ij}, \bar{X}_j \rangle \geq b_i, \quad i = 1, \dots, k, \\
 & && \bar{X}_j \succeq 0 \quad j = 1, \dots, n,
 \end{aligned} \tag{7.16}$$

with symmetric data matrices A_{ij} .

The key construction is:

Listing 7.10: Creating a stack of semidefinite variables.

```
Variable::t X = M->variable(Domain::inPSDCone(d, n));
```

It creates n symmetric, semidefinite matrix variables of dimension d arranged in a single variable object X of shape (n, d, d) . Individual matrix variables can be accessed as slices from $(i, 0, 0)$ to $(i+1, d, d)$ (reshaped into shape (d, d) if necessary). It is also possible to operate on the full variable X when constructing expressions that involve entries of all the semidefinite matrices in a natural way. The source code example illustrates both these approaches.

Listing 7.11: Implementation of model (7.16).

```

#include <iostream>
#include <random>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

// A helper function which returns a slice corresponding to j-th variable
Variable::t slice(Variable::t X, int d, int j) {
    return
        X->slice(new_array_ptr<int,1>({j,0,0}), new_array_ptr<int,1>({j+1,d,d}))
        ->reshape(new_array_ptr<int,1>({d,d}));
}

int main(int argc, char ** argv)
{
    std::random_device rd;
    std::mt19937 e2(rd());
    std::uniform_real_distribution<> dist(0, 1);
}

```

(continues on next page)

```

// Sample data
int n = 100, d = 4, k = 3;
std::vector<double> b({9, 10, 11});
std::vector< std::shared_ptr<ndarray<double,2>> > A;
for(int i=0; i<k*n; i++) {
    auto Ai = std::make_shared<ndarray<double,2>>(shape(d,d));
    for(int s1=0; s1<d; s1++)
        for(int s2=0; s2<=s1; s2++)
            (*Ai)(s1,s2) = (*Ai)(s2,s1) = dist(e2);
    A.push_back(Ai);
}

// Create a model with n semidefinite variables of dimension d x d
Model::t M = new Model("sdo3"); auto _M = finally([&]() { M->dispose(); });

Variable::t X = M->variable(Domain::inPSDCone(d, n));

// Pick indexes (j, s, s), j=0..n-1, s=0..d, of diagonal entries for the
↪ objective
auto alldiag = std::make_shared<ndarray<int,2>>(
    shape(d*n,3),
    std::function<int(const shape_t<2> &)>([d](const shape_t<2> & p) { return
↪ p[1]==0 ? p[0]/d : p[0]%d; }));

M->objective(ObjectiveSense::Minimize, Expr::sum( X->pick(alldiag) ));

// Each constraint is a sum of inner products
// Each semidefinite variable is a slice of X
for(int i=0; i<k; i++) {
    std::vector<Expression::t> sumlist;
    for(int j=0; j<n ;j++)
        sumlist.push_back( Expr::dot(A[i*n+j], slice(X, d, j)) );

    M->constraint(Expr::add(new_array_ptr(sumlist)), Domain::greaterThan(b[i]));
}

// Solve
M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush;
↪ } ); // Add logging
M->writeTask("sdo3.ptf"); // Save problem in readable format
M->solve();

// Get results. Each variable is a slice of X
std::cout << "Contributing variables:" << std::endl;
for(int j=0; j<n; j++) {
    auto Xj = *(slice(X, d, j)->level());
    double maxval = 0;
    for(int s=0; s<d*d; s++) maxval = std::max(maxval, Xj[s]);
    if (maxval>1e-6) {
        std::cout << "X" << j << "=" << std::endl;
        for(int s1=0; s1<d; s1++) {
            for(int s2=0; s2<d; s2++) std::cout << Xj[s1*d+s2] << " ";
            std::cout << std::endl;
        }
    }
}

```

```

    }

    return 0;
}

```

7.7 Integer Optimization

An optimization problem where one or more of the variables are constrained to integer values is called a (mixed) integer optimization problem. **MOSEK** supports integer variables in combination with linear, quadratic and quadratically constrained and conic problems (except semidefinite). See the previous tutorials for an introduction to how to model these types of problems.

7.7.1 Example MILO1

We use the example

$$\begin{aligned}
 &\text{maximize} && x_0 + 0.64x_1 \\
 &\text{subject to} && 50x_0 + 31x_1 \leq 250, \\
 & && 3x_0 - 2x_1 \geq -4, \\
 & && x_0, x_1 \geq 0 \quad \text{and integer}
 \end{aligned} \tag{7.17}$$

to demonstrate how to set up and solve a problem with integer variables. It has the structure of a linear optimization problem (see [Sec. 7.1](#)) except for integrality constraints on the variables. Therefore, only the specification of the integer constraints requires something new compared to the linear optimization problem discussed previously.

First, the integrality constraints are imposed by modifying any existing domain with *Domain.integral*:

```
Variable::t x = M->variable("x", 2, Domain::integral(Domain::greaterThan(0.0)));
```

Another way to do this is to use the method *Variable.makeInteger* on a selected variable.

Next, the example demonstrates how to set various useful parameters of the mixed-integer optimizer. See [Sec. 13.4](#) for details.

```

// Set max solution time
M->setSolverParam("mioMaxTime", 60.0);
// Set max relative gap (to its default value)
M->setSolverParam("mioTolRelGap", 1e-4);
// Set max absolute gap (to its default value)
M->setSolverParam("mioTolAbsGap", 0.0);

```

The complete source for the example is listed in [Listing 7.12](#).

Listing 7.12: How to solve problem (7.17).

```

#include <iostream>
#include <iomanip>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    auto a1 = new_array_ptr<double>, 1>({ 50.0, 31.0 });
    auto a2 = new_array_ptr<double>, 1>({ 3.0, -2.0 });
    auto c = new_array_ptr<double>, 1>({ 1.0, 0.64 });
}

```

(continues on next page)

```

Model::t M = new Model("milo1"); auto _M = finally([&]() { M->dispose(); });
Variable::t x = M->variable("x", 2, Domain::integral(Domain::greaterThan(0.0)));

// Create the constraints
//      50.0 x[0] + 31.0 x[1] <= 250.0
//      3.0 x[0] - 2.0 x[1] >= -4.0
M->constraint("c1", Expr::dot(a1, x), Domain::lessThan(250.0));
M->constraint("c2", Expr::dot(a2, x), Domain::greaterThan(-4.0));

// Set max solution time
M->setSolverParam("mioMaxTime", 60.0);
// Set max relative gap (to its default value)
M->setSolverParam("mioTolRelGap", 1e-4);
// Set max absolute gap (to its default value)
M->setSolverParam("mioTolAbsGap", 0.0);

// Set the objective function to (c^T * x)
M->objective("obj", ObjectiveSense::Maximize, Expr::dot(c, x));

// Solve the problem
M->solve();

// Get the solution values
auto sol = x->level();
std::cout << std::setiosflags(std::ios::scientific) << std::setprecision(2)
    << "x1 = " << (*sol)[0] << std::endl
    << "x2 = " << (*sol)[1] << std::endl
    << "MIP rel gap = " << M->getSolverDoubleInfo("mioObjRelGap") << " (" <<
M->getSolverDoubleInfo("mioObjAbsGap") << ")" << std::endl;
}

```

7.7.2 Specifying an initial solution

It is a common strategy to provide a starting feasible point (if one is known in advance) to the mixed-integer solver. This can in many cases reduce solution time.

There are two modes for **MOSEK** to utilize an initial solution.

- **A complete solution.** **MOSEK** will first try to check if the current value of the primal variable solution is a feasible point. The solution can either come from a previous solver call or can be entered by the user, however the full solution with values for all variables (both integer and continuous) must be provided. This check is always performed and does not require any extra action from the user. The outcome of this process can be inspected via information items *"mioInitialFeasibleSolution"* and *"mioInitialFeasibleSolutionObj"*, and via the **Initial feasible solution objective** entry in the log.
- **A partial integer solution.** **MOSEK** can also try to construct a feasible solution by fixing integer variables to the values provided by the user (rounding if necessary) and optimizing over the remaining continuous variables. In this setup the user must provide initial values for all integer variables. This action is only performed if the parameter *mioConstructSol* is switched on. The outcome of this process can be inspected via information items *"mioConstructSolution"* and *"mioConstructSolutionObj"*, and via the **Construct solution objective** entry in the log.

In the following example we focus on inputting a partial integer solution.

$$\begin{aligned}
 &\text{maximize} && 7x_0 + 10x_1 + x_2 + 5x_3 \\
 &\text{subject to} && x_0 + x_1 + x_2 + x_3 \leq 2.5 \\
 &&& x_0, x_1, x_2 \in \mathbb{Z} \\
 &&& x_0, x_1, x_2, x_3 \geq 0
 \end{aligned} \tag{7.18}$$

Solution values can be set using `Variable.setLevel` .

Listing 7.13: Implementation of problem (7.18) specifying an initial solution.

```
// Assign values to integer variables.
// We only set a slice of x
auto init_sol = new_array_ptr<double, 1>({ 1.0, 1.0, 0.0 });
x->slice(0,3)->setLevel( init_sol );

// Request constructing the solution from integer variable values
M->setSolverParam("mioConstructSol", "on");
```

A more advanced application of `Variable.setLevel` is presented in the case study on *Multiprocessor scheduling*.

The log output from the optimizer will in this case indicate that the inputted values were used to construct an initial feasible solution:

```
Construct solution objective      : 1.9500000000000e+01
```

The same information can be obtained from the API:

Listing 7.14: Retrieving information about usage of initial solution

```
int constr = M->getSolverIntInfo("mioConstructSolution");
double constrVal = M->getSolverDoubleInfo("mioConstructSolutionObj");
std::cout << "Construct solution utilization: " << constr << std::endl;
std::cout << "Construct solution objective: " << constrVal << std::endl;
```

7.7.3 Example MICO1

Integer variables can also be used arbitrarily in conic problems (except semidefinite). We refer to the previous tutorials for how to set up a conic optimization problem. Here we present sample code that sets up a simple optimization problem:

$$\begin{aligned} & \text{minimize} && x^2 + y^2 \\ & \text{subject to} && x \geq e^y + 3.8, \\ & && x, y \text{ integer.} \end{aligned} \tag{7.19}$$

The canonical conic formulation of (7.19) suitable for Fusion API for C++ is

$$\begin{aligned} & \text{minimize} && t \\ & \text{subject to} && (t, x, y) \in \mathcal{Q}^3 && (t \geq \sqrt{x^2 + y^2}) \\ & && (x - 3.8, 1, y) \in K_{\text{exp}} && (x - 3.8 \geq e^y) \\ & && x, y \text{ integer,} \\ & && t \in \mathbb{R}. \end{aligned} \tag{7.20}$$

Listing 7.15: Implementation of problem (7.20).

```
#include <iostream>
#include <iomanip>
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv)
{
    Model::t M = new Model("mico1"); auto _M = finally([&]() { M->dispose(); });
```

(continues on next page)


```

Variable::t x = M->variable(Domain::integral(Domain::unbounded()));
Variable::t y = M->variable(Domain::integral(Domain::unbounded()));
Variable::t t = M->variable();

M->constraint(Expr::vstack(t, x, y), Domain::inQCone());
M->constraint(Expr::vstack(Expr::sub(x, 3.8), 1, y), Domain::inPExpCone());

M->objective(ObjectiveSense::Minimize, t);

M->solve();

std::cout << std::setprecision(2)
           << "x = " << (*(x->level()))[0] << std::endl
           << "y = " << (*(y->level()))[0] << std::endl ;

return 0;
}

```

Error and solution status handling were omitted for readability.

7.8 Disjunctive constraints

A **disjunctive constraint (DJC)** involves of a number of affine conditions combined with the logical operators or ($\vee$) and optionally and ($\wedge$) into a formula in *disjunctive normal form*, that is a disjunction of conjunctions. Specifically, a disjunctive constraint has the form of a disjunction

$$T_1 \text{ or } T_2 \text{ or } \cdots \text{ or } T_t \quad (7.21)$$

where each T_i is written as a conjunction

$$T_i = T_{i,1} \text{ and } T_{i,2} \text{ and } \cdots \text{ and } T_{i,s_i} \quad (7.22)$$

and each $T_{i,j}$ is an affine condition (affine equation or affine inequality) of the form $D_{ij}x + d_{ij} \in \mathcal{D}_{ij}$ with $\mathcal{D}_{ij}$ being one of the affine domains from [Sec. 14.8.1](#). A disjunctive constraint (DJC) can therefore be succinctly written as

$$\bigvee_{i=1}^t \bigwedge_{j=1}^{s_i} T_{i,j} \quad (7.23)$$

where each $T_{i,j}$ is an affine condition.

Each T_i is called a **term** of the disjunctive constraint and t is the number of terms. Each condition $T_{i,j}$ is called a **simple term** and s_i is called the **size** of the i -th term.

A disjunctive constraint is satisfied if at least one of its terms is satisfied. A term is satisfied if all of its constituent simple terms are satisfied. A problem containing DJCs will be solved by the mixed-integer optimizer.

Note that nonlinear cones are not allowed as one of the domains $\mathcal{D}_{ij}$ inside a DJC.

7.8.1 Applications

Disjunctive constraints are a convenient and expressive syntactical tool. They can be used to phrase many constructions appearing especially in mixed-integer modelling. Here are some examples.

- **Complementarity.** The condition $xy = 0$, where x, y are scalar variables, is equivalent to

$$x = 0 \text{ or } y = 0.$$

It is a DJC with two terms, each of size 1.

- **Semicontinuous variable.** A semicontinuous variable is a scalar variable which takes values in $\{0\} \cup [a, +\infty]$. This can be expressed as

$$x = 0 \text{ or } x \geq a.$$

It is again a DJC with two terms, each of size 1.

- **Exact absolute value.** The constraint $t = |x|$ is not convex, but can be written as

$$(x \geq 0 \text{ and } t = x) \text{ or } (x \leq 0 \text{ and } t = -x)$$

It is a DJC with two terms, each of size 2.

- **Indicator.** Suppose z is a Boolean variable. Then we can write the indicator constraint $z = 1 \implies a^T x \leq b$ as

$$(z = 1 \text{ and } a^T x \leq b) \text{ or } (z = 0)$$

which is a DJC with two terms, of sizes, respectively, 2 and 1.

- **Piecewise linear functions.** Suppose $a_1 \leq \dots \leq a_{k+1}$ and $f : [a_1, a_{k+1}] \rightarrow \mathbb{R}$ is a piecewise linear function, given on the i -th of k intervals $[a_i, a_{i+1}]$ by a different affine expression $f_i(x)$. Then we can write the constraint $y = f(x)$ as

$$\bigvee_{i=1}^k (a_i \leq y \text{ and } y \leq a_{i+1} \text{ and } y - f_i(x) = 0)$$

making it a DJC with k terms, each of size 3.

On the other hand most DJCs are equivalent to a mixed-integer linear program through a big-M reformulation. In some cases, when a suitable big-M is known to the user, writing such a formulation directly may be more efficient than formulating the problem as a DJC. See [Sec. 13.4.6](#) for a discussion of this topic.

Disjunctive constraints can be added to any problem which includes linear constraints, affine conic constraints (without semidefinite domains) or integer variables.

7.8.2 Example DJC1

In this tutorial we will consider the following sample demonstration problem:

$$\begin{aligned} & \text{minimize} && 2x_0 + x_1 + 3x_2 + x_3 \\ & \text{subject to} && x_0 + x_1 + x_2 + x_3 \geq -10, \\ & && \left(\begin{array}{c} x_0 - 2x_1 \leq -1 \\ \text{and} \\ x_2 = x_3 = 0 \end{array} \right) \text{ or } \left(\begin{array}{c} x_2 - 3x_3 \leq -2 \\ \text{and} \\ x_0 = x_1 = 0 \end{array} \right), \\ & && x_i = 2.5 \text{ for at least one } i \in \{0, 1, 2, 3\}. \end{aligned} \tag{7.24}$$

The problem has two DJCs: the first one has 2 terms. The second one, which we can write as $\bigvee_{i=0}^3 (x_i = 2.5)$, has 4 terms.

We refer to [Sec. 7.1](#) for the details of constructing a model and setting up variables and linear constraints. In this tutorial we focus on the two disjunctive constraints. Each of the simple terms

appearing in disjunctions is constructed using *DJC.term* in the form known from ordinary constraints, that is

Expression belongs to a *Domain*.

Therefore the first disjunction in our example can be written as

```

M->disjunction( DJC::AND( DJC::term(Expr::dot(dblarray({1,-2,0,0}), x), Domain::
↳lessThan(-1)), // x0 - 2x1 <= -1
                DJC::term(x->slice(2, 4), Domain::equalsTo(0)) ),
↳
                // x2 = x3 = 0
                DJC::AND( DJC::term(Expr::dot(dblarray({0,0,1,-3}), x), Domain::
↳lessThan(-2)), // x2 - 3x3 <= -2
                DJC::term(x->slice(0, 2), Domain::equalsTo(0)) ) );
↳
                // x0 = x1 = 0

```

The disjunctive constraint is added to them model with *Model.disjunction*. Here we call this method with two terms, each of which is a conjunction (*DJC.AND*) of simple terms.

The second disjunctive constraint is created by passing an array of 4 terms:

```

// Disjunctive constraint from an array of terms reading x_i = 2.5 for i = 0,1,2,3
M->disjunction(std::make_shared<ndarray<Term::t,1>>(shape(4), [x](int i) { return
↳DJC::term(x->index(i), Domain::equalsTo(2.5)); }));

```

The complete code constructing and solving the problem (7.24) is shown below.

Listing 7.16: Source code solving problem (7.24).

```

#include <iostream>
#include "fusion.h"
using namespace mosek::fusion;
using namespace monty;

std::shared_ptr<ndarray<double,1>> dblarray(std::initializer_list<double> x) {
    return new_array_ptr<double,1>(x);
}

int main(int argc, char ** argv)
{
    Model::t M = new Model("djc1"); auto _M = finally([&]() { M->dispose(); });

    // Create variable 'x' of length 4
    Variable::t x = M->variable("x", 4);

    // First disjunctive constraint
    M->disjunction( DJC::AND( DJC::term(Expr::dot(dblarray({1,-2,0,0}), x), Domain::
↳lessThan(-1)), // x0 - 2x1 <= -1
                DJC::term(x->slice(2, 4), Domain::equalsTo(0)) ),
↳
                // x2 = x3 = 0
                DJC::AND( DJC::term(Expr::dot(dblarray({0,0,1,-3}), x), Domain::
↳lessThan(-2)), // x2 - 3x3 <= -2
                DJC::term(x->slice(0, 2), Domain::equalsTo(0)) ) );
↳
                // x0 = x1 = 0

    // Second disjunctive constraint
    // Disjunctive constraint from an array of terms reading x_i = 2.5 for i = 0,1,2,3
    M->disjunction(std::make_shared<ndarray<Term::t,1>>(shape(4), [x](int i) { return
↳DJC::term(x->index(i), Domain::equalsTo(2.5)); }));

    // The linear constraint

```

(continues on next page)

```

M->constraint(Expr::sum(x), Domain::greaterThan(-10));

// Objective
M->objective(ObjectiveSense::Minimize, Expr::dot(dblarray({2,1,3,1}), x));

// Useful for debugging
M->writeTask("djc1.ptf");
M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush; }
→);

// Solve the problem
M->solve();

// Get the solution values
if (M->getPrimalSolutionStatus() == SolutionStatus::Optimal) {
    auto sol = x->level();
    std::cout << "[x0,x1,x2,x3] = " << (*sol) << std::endl;
}
else {
    std::cout << "Another solution status" << std::endl;
}
}

```

7.9 Model Parametrization and Reoptimization

This tutorial demonstrates how to construct a model with a fixed structure and reoptimize it by changing some of the input data. If you instead want to dynamically modify the model structure between optimizations by adding variables, constraints etc., see the other reoptimization tutorial [Sec. 7.10](#).

For this tutorial we solve the following variant of *linear regression with elastic net regularization*:

$$\text{minimize}_x \|Ax - b\|_2 + \lambda_1 \|x\|_1 + \lambda_2 \|x\|_2$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$. The optimization variable is $x \in \mathbb{R}^n$ and λ_1, λ_2 are two nonnegative numbers indicating the tradeoff between the linear regression objective, a *lasso* (ℓ_1 -norm) penalty and a *ridge* (ℓ_2 -norm) regularization. The representation of this problem compatible with **MOSEK** input format is

$$\begin{aligned}
 &\text{minimize} && t + \lambda_1 \sum_i p_i + \lambda_2 q \\
 &\text{subject to} && (t, Ax - b) \in \mathcal{Q}^{m+1}, \\
 & && p_i \geq |x_i|, \quad i = 1, \dots, n, \\
 & && (q, x) \in \mathcal{Q}^{n+1}.
 \end{aligned}$$

7.9.1 Creating a model

Before creating a parametrized model we should analyze which parts of the model are fixed once for all, and which parts do we intend to change between optimizations. Here we make the following assumption:

- the matrix A will not change,
- we want to solve the problem for many target vectors b ,
- we want to experiment with different tradeoffs λ_1, λ_2 .

That leads us to construct the model with A provided from the start as fixed input and declare b, λ_1, λ_2 as parameters. The initial model construction is shown below. Parameters are objects of type *Parameter*, created with the method *Model.parameter*. We exploit the fact that parameters can have shapes, just like variables and expressions, and that they can be used everywhere within an expression where a constant of the same shape would be suitable.

Listing 7.17: Constructing a parametrized model.

```

Model::t initializeModel(int m, int n, std::shared_ptr<ndarray<double,2>> A) {
    Model::t M = new Model();
    auto x = M->variable("x", n);

    // t >= |Ax-b|_2 where b is a parameter
    auto b = M->parameter("b", m);
    auto t = M->variable();
    M->constraint(Expr::vstack(t, Expr::sub(Expr::mul(A, x), b)), Domain::inQCone());

    // p_i >= |x_i|, i=1..n
    auto p = M->variable(n);
    M->constraint(Expr::hstack(p, x), Domain::inQCone());

    // q >= |x|_2
    auto q = M->variable();
    M->constraint(Expr::vstack(q, x), Domain::inQCone());

    // Objective, parametrized with lambda1, lambda2
    // t + lambda1*sum(p) + lambda2*q
    auto lambda1 = M->parameter("lambda1");
    auto lambda2 = M->parameter("lambda2");
    auto obj = Expr::add(new_array_ptr<Expression::t, 1>({t, Expr::mul(lambda1, Expr::
    sum(p)), Expr::mul(lambda2, q)}));
    M->objective(ObjectiveSense::Minimize, obj);

    // Return the ready model
    return M;
}

```

For the purpose of the example we take

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ -2 & -1 \\ -4 & -3 \end{bmatrix}$$

and we initialize the parametrized model:

Listing 7.18: Initializing the model

```

//Create a small example
int m = 4;
int n = 2;
auto A = new_array_ptr<double, 2>({
    {1.0, 2.0},
    {3.0, 4.0},
    {-2.0, -1.0},
    {-4.0, -3.0} });

auto M = initializeModel(m, n, A);

// For convenience retrieve some elements of the model
auto b = M->getParameter("b");
auto lambda1 = M->getParameter("lambda1");
auto lambda2 = M->getParameter("lambda2");
auto x = M->getVariable("x");

```

We made sure to keep references to the interesting elements of the model, in particular the parameter objects we are about to set values of.

7.9.2 Setting parameters

For the first solve we use

$$b = [0.1, 1.2, -1.1, 3.0]^T, \lambda_1 = 0.1, \lambda_2 = 0.01.$$

Parameters are set with method `Parameter.setValue`. We set the parameters and solve the model as follows:

Listing 7.19: Setting parameters and solving the model.

```
// First solve
b->setValue(new_array_ptr<double, 1>({0.1, 1.2, -1.1, 3.0}));
lambda1->setValue(0.1);
lambda2->setValue(0.01);

M->solve();
auto sol = x->level();
std::cout << "Objective " << M->primalObjValue() << ", solution " << (*sol)[0] << ",
↪ " << (*sol)[1] << "\n";
```

7.9.3 Changing parameters

Let us say we now want to increase the weight of the lasso penalty in order to favor sparser solutions. We can simply change that parameter, leave the other ones unchanged, and resolve:

Listing 7.20: Changing a parameter and resolving

```
// Increase lambda1
lambda1->setValue(0.5);

M->solve();
sol = x->level();
std::cout << "Objective " << M->primalObjValue() << ", solution " << (*sol)[0] << ",
↪ " << (*sol)[1] << "\n";
```

Next, we might want to solve a few instances of the problem for another value of b . Again, we reset the relevant parameters and solve:

Listing 7.21: Changing parameters and resolving

```
// Now change the data completely
b->setValue(new_array_ptr<double, 1>({1.0, 1.0, 1.0, 1.0}));
lambda1->setValue(0.0);
lambda2->setValue(0.0);

M->solve();
sol = x->level();
std::cout << "Objective " << M->primalObjValue() << ", solution " << (*sol)[0] << ",
↪ " << (*sol)[1] << "\n";

// And increase lambda2
lambda2->setValue(1.4145);

M->solve();
```

(continues on next page)

```
sol = x->level();
std::cout << "Objective " << M->primalObjValue() << ", solution " << (*sol)[0] << ",
↪ " << (*sol)[1] << "\n";
```

7.9.4 Additional remarks

- Domains cannot be parametrized, therefore to parametrize a bound, such as $x \geq p$, it is necessary to write it as $x - p \geq 0$.
- Coefficients appearing at semidefinite terms cannot be parametrized. If it is necessary to have a parametrized expression such as $p\bar{X}_{i,j}$, introduce an auxiliary scalar variable $x = \bar{X}_{i,j}$ and use px in the model.
- Parametrized models can be found in the following examples: `alan.cc`, `portfolio_2_frontier.cc`, `portfolio_5_card.cc`, `total_variation.cc`.

7.10 Problem Modification and Reoptimization

This tutorial demonstrates how to modify a model by adding new elements and changing existing ones. If instead you want to create one model of fixed structure and reoptimize it for changing input data, see [Sec. 7.9](#).

The example we study is a simple production planning model.

Problem modifications regarding variables, cones, objective function and constraints can be grouped in categories:

- adding constraints and variables,
- modifying existing constraints.

Adding new variables and constraints is very easy. Modifications to existing constraints are more cumbersome, and the user should consider whether it is not worth rebuilding the model from scratch in such case. The amount of work required by *Fusion* to update the optimizer task may outweigh the potential gains.

Depending on the type of modification, **MOSEK** may be able to optimize the modified problem more efficiently exploiting the information and internal state from the previous execution. After optimization, the solution is always stored internally, and is available before next optimization. The former optimal solution may be still feasible, but no longer optimal; or it may remain optimal if the modification of the objective function was small.

In general, **MOSEK** exploits dual information and availability of an optimal basis from the previous execution. The simplex optimizer is well suited for exploiting an existing primal or dual feasible solution. Restarting capabilities for interior-point methods are still not as reliable and effective as those for the simplex algorithm. More information can be found in Chapter 10 of the book [\[Chvatal83\]](#).

Parameter settings (see [Sec. 8.4](#)) can also be changed between optimizations.

7.10.1 Example: Production Planning

A company manufactures three types of products. Suppose the stages of manufacturing can be split into three parts: Assembly, Polishing and Packing. In the table below we show the time required for each stage as well as the profit associated with each product.

Product no.	Assembly (minutes)	Polishing (minutes)	Packing (minutes)	Profit (\$)
0	2	3	2	1.50
1	4	2	3	2.50
2	3	3	2	3.00

With the current resources available, the company has 100,000 minutes of assembly time, 50,000 minutes of polishing time and 60,000 minutes of packing time available per year. We want to know how many items of each product the company should produce each year in order to maximize profit?

Denoting the number of items of each type by x_0, x_1 and x_2 , this problem can be formulated as a linear optimization problem:

$$\begin{array}{llllll} \text{maximize} & 1.5x_0 & + & 2.5x_1 & + & 3.0x_2 \\ \text{subject to} & 2x_0 & + & 4x_1 & + & 3x_2 & \leq & 100000, \\ & 3x_0 & + & 2x_1 & + & 3x_2 & \leq & 50000, \\ & 2x_0 & + & 3x_1 & + & 2x_2 & \leq & 60000, \end{array} \quad (7.25)$$

and

$$x_0, x_1, x_2 \geq 0.$$

Code in [Listing 7.22](#) loads and solves this problem.

Listing 7.22: Setting up and solving problem (7.25)

```
auto c = new_array_ptr<double>, 1>({ 1.5, 2.5, 3.0 });
auto A = new_array_ptr<double>, 2>({ {2, 4, 3},
                                       {3, 2, 3},
                                       {2, 3, 2} });
auto b = new_array_ptr<double>, 1>({ 100000.0, 50000.0, 60000.0 });
int numvar = 3;
int numcon = 3;

// Create a model and input data
Model::t M = new Model(); auto M_ = monty::finally([&]() { M->dispose(); });

auto x = M->variable(numvar, Domain::greaterThan(0.0));
auto con = M->constraint(Expr::mul(A, x), Domain::lessThan(b));
M->objective(ObjectiveSense::Maximize, Expr::dot(c, x));
// Solve the problem
M->solve();
```

7.10.2 Changing the Linear Constraint Matrix

Suppose we want to change the time required for assembly of product 0 to 3 minutes. This corresponds to setting $a_{0,0} = 3$. Now the *Constraint* provides the method *Constraint.update*, which can replace the columns corresponding to a variable with new values (or to replace the whole constraint). In our case the update we need is replacing $1 \cdot x_0$ with $3 \cdot x_0$ in the constraint with index 0.

```
con->index(0)->update(Expr::mul(3.0, x->index(0)), x->index(0));
```

The problem now has the form:

$$\begin{array}{llllll} \text{maximize} & 1.5x_0 & + & 2.5x_1 & + & 3.0x_2 \\ \text{subject to} & 3x_0 & + & 4x_1 & + & 3x_2 & \leq & 100000, \\ & 3x_0 & + & 2x_1 & + & 3x_2 & \leq & 50000, \\ & 2x_0 & + & 3x_1 & + & 2x_2 & \leq & 60000, \end{array} \quad (7.26)$$

and

$$x_0, x_1, x_2 \geq 0.$$

After this operation we can reoptimize the problem.

7.10.3 Appending Variables

We now want to add a new product with the following data:

Product no.	Assembly (minutes)	Polishing (minutes)	Packing (minutes)	Profit (\$)
3	4	0	1	1.00

This corresponds to creating a new variable x_3 , appending a new column to the A matrix and setting a new term in the objective. We do this in [Listing 7.23](#)

Listing 7.23: How to add a new variable (column)

```

/***** Add a new variable *****/
// Create a variable and a compound view of all variables
auto x3 = M->variable(Domain::greaterThan(0.0));
auto xNew = Var::vstack(x, x3);
// Add to the existing constraint
con->update(Expr::mul(x3, new_array_ptr<double, 1>({ 4, 0, 1 })), x3);
// Change the objective to include x3
M->objective(ObjectiveSense::Maximize, Expr::dot(new_array_ptr<double, 1>({1.5, 2.5,
↪ 3.0, 1.0}), xNew));

```

After this operation the new problem is:

$$\begin{aligned}
 &\text{maximize} && 1.5x_0 &+& 2.5x_1 &+& 3.0x_2 &+& 1.0x_3 \\
 &\text{subject to} && 3x_0 &+& 4x_1 &+& 3x_2 &+& 4x_3 &\leq 100000, \\
 & && 3x_0 &+& 2x_1 &+& 3x_2 && &\leq 50000, \\
 & && 2x_0 &+& 3x_1 &+& 2x_2 &+& 1x_3 &\leq 60000,
 \end{aligned} \tag{7.27}$$

and

$$x_0, x_1, x_2, x_3 \geq 0.$$

7.10.4 Appending Constraints

Now suppose we want to add a new stage to the production process called *Quality control* for which 30000 minutes are available. The time requirement for this stage is shown below:

Product no.	Quality control (minutes)
0	1
1	2
2	1
3	1

This corresponds to adding the constraint

$$x_0 + 2x_1 + x_2 + x_3 \leq 30000$$

to the problem. This is done as follows.

Listing 7.24: Adding a new constraint.

```

/***** Add a new constraint *****/
auto con2 = M->constraint(Expr::dot(xNew, new_array_ptr<double, 1>({1, 2, 1, 1})),
↳Domain::lessThan(30000.0));

```

Again, we can continue with re-optimizing the modified problem.

7.10.5 Changing bounds

One typical reoptimization scenario is to change bounds. Suppose for instance that we must operate with limited time resources, and we must change the upper bounds in the problem as follows:

Operation	Time available (before)	Time available (new)
Assembly	100000	80000
Polishing	50000	40000
Packing	60000	50000
Quality control	30000	22000

That means we would like to solve the problem:

$$\begin{aligned}
 &\text{maximize} && 1.5x_0 &+& 2.5x_1 &+& 3.0x_2 &+& 1.0x_3 \\
 &\text{subject to} && 3x_0 &+& 4x_1 &+& 3x_2 &+& 4x_3 &\leq 80000, \\
 & && 3x_0 &+& 2x_1 &+& 3x_2 && &\leq 40000, \\
 & && 2x_0 &+& 3x_1 &+& 2x_2 &+& 1x_3 &\leq 50000, \\
 & && x_0 &+& 2x_1 &+& x_2 &+& x_3 &\leq 22000.
 \end{aligned} \tag{7.28}$$

Since *Domain* objects are immutable, we cannot change the constraints by simply updating the value inside domains. To circumvent this, we add the differences between new and old bounds as fixed terms to the constraint expression. That means, we effectively construct an equivalent problem:

$$\begin{aligned}
 &\text{maximize} && 1.5x_0 &+& 2.5x_1 &+& 3.0x_2 &+& 1.0x_3 \\
 &\text{subject to} && 3x_0 &+& 4x_1 &+& 3x_2 &+& 4x_3 &+& 20000 &\leq 100000, \\
 & && 3x_0 &+& 2x_1 &+& 3x_2 && &+& 10000 &\leq 50000, \\
 & && 2x_0 &+& 3x_1 &+& 2x_2 &+& 1x_3 &+& 10000 &\leq 60000, \\
 & && x_0 &+& 2x_1 &+& x_2 &+& x_3 &+& 8000 &\leq 30000.
 \end{aligned} \tag{7.29}$$

The next listing shows how to do it.

Listing 7.25: Change constraint bounds.

```

/***** Change constraint bounds *****/
// Assemble all constraints we previously defined into one
auto cAll = Constraint::vstack(con, con2);
// Change bounds by effectively updating fixed terms with the difference
cAll->update(new_array_ptr<double, 1>({20000, 10000, 10000, 8000}));

```

Again, we can continue with re-optimizing the modified problem.

7.10.6 Advanced hot-start

If the optimizer used the data from the previous run to hot-start the optimizer for reoptimization, this will be indicated in the log:

```

Optimizer - hotstart : yes

```

When performing re-optimizations, instead of removing a basic variable it may be more efficient to fix the variable at zero and then remove it when the problem is re-optimized and it has left the basis. This makes it easier for **MOSEK** to restart the simplex optimizer.

7.11 Parallel optimization

In this section we demonstrate the method `Model.solveBatch` which is a parallel optimization mechanism built-in in **MOSEK**. It has the following features:

- It allows to fine-tune the balance between the total number of threads in use by the parallel solver and the number of threads used for each individual model.
- It is very efficient for optimizing a large number of models of similar size, for example models obtained by cloning an initial model and changing some coefficients.

In the example below we demonstrate a very standard application of `Model.solveBatch`. We create an initial model, clone it a few times, set different parameter values in each clone and then optimize all the cloned models in parallel. When all models complete we access the status for each of them and, if successfully solved, we gather solutions and other information in the standard way, as if each model was optimized separately.

Listing 7.26: Calling the parallel optimizer.

```
/** Example of how to use Model.solveBatch()
 */
int main(int argc, char ** argv)
{
    // Choose some sample parameters
    int n = 10;                // Number of models to optimize
    int threadpoolsize = 4;    // Total number of threads available
    int threadspersmodel = 1;  // Number of threads per each model

    // Create a toy model for this example
    auto M = makeToyParameterizedModel();

    // Set up n copies of the model with different data
    auto models = std::make_shared<ndarray<Model::t,1>>(shape(n));

    for(int i = 0; i < n ; i++)
    {
        (*models)[i] = M->clone();
        (*models)[i]->getParameter("p")->setValue(i+1);
        // We can set the number of threads individually per model
        (*models)[i]->setSolverParam("numThreads", threadspersmodel);
    }

    // Solve all models in parallel
    auto status = Model::solveBatch(false,          // No race
                                    -1.0,          // No time limit
                                    threadpoolsize,
                                    models);        // Array of Models to solve

    // Access the solutions
    for(int i = 0; i < n; i++)
        if ((*status)[i] == SolverStatus::OK)
            std::cout << "Model " << i << ": "
                        << " Status " << (*status)[i]
                        << " Solution Status " << (*models)[i]->getPrimalSolutionStatus()
                        << " Objective " << (*models)[i]->primalObjValue()
                        << " Time " << (*models)[i]->getSolverDoubleInfo(
->"optimizerTime") << std::endl;
        else
```

(continues on next page)

```
std::cout << "Model " << i << ": not solved" << std::endl;
}
```

7.12 Retrieving infeasibility certificates

When a continuous problem is declared as primal or dual infeasible, **MOSEK** provides a Farkas-type infeasibility certificate. If, as it happens in many cases, the problem is infeasible due to an unintended mistake in the formulation or because some individual constraint is too tight, then it is likely that infeasibility can be isolated to a few linear constraints/bounds that mutually contradict each other. In this case it is easy to identify the source of infeasibility. The tutorial in [Sec. 9.3](#) has instructions on how to deal with this situation and debug it **by hand**. We recommend [Sec. 9.3](#) as an introduction to infeasibility certificates and how to deal with infeasibilities in general.

Some users, however, would prefer to obtain the infeasibility certificate using Fusion API for C++, for example in order to repair the issue automatically, display the information to the user, or perhaps simply because the infeasibility was one of the intended outcomes that should be analyzed in the code.

In this tutorial we show how to obtain such an infeasibility certificate with Fusion API for C++ in the most typical case, that is when the linear part of a problem is primal infeasible. A Farkas-type primal infeasibility certificate consists of the dual values of linear constraints and bounds. Each of the dual values (multipliers) indicates that a certain multiple of the corresponding constraint should be taken into account when forming the collection of mutually contradictory equalities/inequalities.

7.12.1 Example PINFEAS

For the purpose of this tutorial we use the same example as in [Sec. 9.3](#), that is the primal infeasible problem

$$\begin{array}{llllllllll}
\text{minimize} & & x_0 & + & 2x_1 & + & 5x_2 & + & 2x_3 & + & x_4 & + & 2x_5 & + & x_6 \\
\text{subject to} & s_0 : & x_0 & + & x_1 & & & & & & & & & & & \leq & 200, \\
& s_1 : & & & & & x_2 & + & x_3 & & & & & & & \leq & 1000, \\
& s_2 : & & & & & & & & & x_4 & + & x_5 & + & x_6 & \leq & 1000, \\
& d_0 : & x_0 & & & & & & & & + & x_4 & & & & = & 1100, \\
& d_1 : & & & x_1 & & & & & & & & & & & = & 200, \\
& d_2 : & & & & & x_2 & + & & & & & & x_5 & & = & 500, \\
& d_3 : & & & & & & & x_3 & + & & & & & & x_6 & = & 500, \\
& & & & & & & & & & & & & & x_i & \geq & 0.
\end{array} \tag{7.30}$$

Creating the model

In order to fetch the infeasibility certificate we must have access to the objects representing both variables and constraints after optimization. We will implement the problem as having two linear constraints s and d of dimensions 3 and 4, respectively.

```
// Construct the sample model from the example in the manual
auto sMat = Matrix::sparse(3, 7, new_array_ptr<int,1>({0,0,1,1,2,2,2}),
                           new_array_ptr<int,1>({0,1,2,3,4,5,6}),
                           new_array_ptr<double,1>({1,1,1,1,1,1,1}));
auto sBound = new_array_ptr<double,1>({200, 1000, 1000});
auto dMat = Matrix::sparse(4, 7, new_array_ptr<int,1>({0,0,1,2,2,3,3}),
                           new_array_ptr<int,1>({0,4,1,2,5,3,6}),
                           new_array_ptr<double,1>({1,1,1,1,1,1,1}));
auto dBound = new_array_ptr<double,1>({1100, 200, 500, 500});
auto c = new_array_ptr<double,1>({1, 2, 5, 2, 1, 2, 1});

Model::t M = new Model("pinfeas"); auto _M = finally([&]() { M->dispose(); });

Variable::t x = M->variable("x", 7, Domain::greaterThan(0));
```

(continues on next page)

(continued from previous page)

```
Constraint::t s = M->constraint("s", Expr::mul(sMat, x), Domain::lessThan(sBound));
Constraint::t d = M->constraint("d", Expr::mul(dMat, x), Domain::equalsTo(dBound));
M->objective(ObjectiveSense::Minimize, Expr::dot(c,x));
```

Checking infeasible status and adjusting settings

After the model has been solved we check that it is indeed infeasible. If yes, then we choose a threshold for when a certificate value is considered as an important contributor to infeasibility (ideally we would like to list all nonzero duals, but just like an optimal solution, an infeasibility certificate is also subject to floating-point rounding errors). Finally, we declare that we are interested in retrieving certificates and not just optimal solutions by calling `Model.acceptedSolutionStatus`, see Sec. 8.1.4. All these steps are demonstrated in the snippet below:

```
// Check problem status
if (M->getProblemStatus() == ProblemStatus::PrimalInfeasible) {
    // Set the tolerance at which we consider a dual value as essential
    double eps = 1e-7;

    // We want to retrieve infeasibility certificates
    M->acceptedSolutionStatus(AccSolutionStatus::Certificate);
```

Going through the certificate for a single item

We can define a fairly generic function which takes an array of dual values and all other required data and prints out the positions of those entries whose dual values exceed the given threshold. These are precisely the values we are interested in:

```
//Analyzes and prints infeasibility certificate for a single object,
//which can be a variable or constraint
static void analyzeCertificate(std::string name, // name
    // of the analyzed object
    long size, // size
    // of the object
    std::shared_ptr<ndarray<double, 1>> duals, //
    // actual dual values
    double eps) //
    // tolerance determining when a dual value is considered important
{
    for(int i = 0; i < size; i++) {
        if (abs((*duals)[i]) > eps)
            std::cout << name << "[" << i << "], dual = " << (*duals)[i] << std::endl;
    }
}
```

Full source code

All that remains is to call this function for all variable and constraint bounds for which we want to know their contribution to infeasibility. Putting all these pieces together we obtain the following full code:

Listing 7.27: Demonstrates how to retrieve a primal infeasibility certificate.

```
#include <iostream>
#include "fusion.h"
using namespace mosek::fusion;
using namespace monty;
```

(continues on next page)

```

//Analyzes and prints infeasibility certificate for a single object,
//which can be a variable or constraint
static void analyzeCertificate(std::string name,                                // name_
    ↪ of the analyzed object                                long size,                // size_
    ↪ of the object                                         std::shared_ptr<ndarray<double, 1>> duals, //
    ↪ actual dual values                                     double eps)                             //
    ↪ tolerance determining when a dual value is considered important
{
    for(int i = 0; i < size; i++) {
        if (abs((*duals)[i]) > eps)
            std::cout << name << "[" << i << "],    dual = " << (*duals)[i] << std::endl;
    }
}

int main(int argc, char ** argv)
{
    // Construct the sample model from the example in the manual
    auto sMat = Matrix::sparse(3, 7, new_array_ptr<int,1>({0,0,1,1,2,2,2}),
                                new_array_ptr<int,1>({0,1,2,3,4,5,6}),
                                new_array_ptr<double,1>({1,1,1,1,1,1,1}));
    auto sBound = new_array_ptr<double,1>({200, 1000, 1000});
    auto dMat = Matrix::sparse(4, 7, new_array_ptr<int,1>({0,0,1,2,2,3,3}),
                                new_array_ptr<int,1>({0,4,1,2,5,3,6}),
                                new_array_ptr<double,1>({1,1,1,1,1,1,1}));
    auto dBound = new_array_ptr<double,1>({1100, 200, 500, 500});
    auto c = new_array_ptr<double,1>({1, 2, 5, 2, 1, 2, 1});

    Model::t M = new Model("pinfeas"); auto _M = finally([&]() { M->dispose(); });

    Variable::t x = M->variable("x", 7, Domain::greaterThan(0));
    Constraint::t s = M->constraint("s", Expr::mul(sMat, x), Domain::lessThan(sBound));
    Constraint::t d = M->constraint("d", Expr::mul(dMat, x), Domain::equalsTo(dBound));
    M->objective(ObjectiveSense::Minimize, Expr::dot(c,x));

    // Useful for debugging
    M->writeTask("pinfeas.ptf");
    M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush; }_
    ↪);

    // Solve the problem
    M->solve();

    // Check problem status
    if (M->getProblemStatus() == ProblemStatus::PrimalInfeasible) {
        // Set the tolerance at which we consider a dual value as essential
        double eps = 1e-7;

        // We want to retrieve infeasibility certificates
        M->acceptedSolutionStatus(AccSolutionStatus::Certificate);

        // Go through variable bounds

```

(continued from previous page)

```
std::cout << "Variable bounds important for infeasibility: " << std::endl;
analyzeCertificate("x", x->getSize(), x->dual(), eps);

// Go through constraint bounds
std::cout << "Constraint bounds important for infeasibility: " << std::endl;
analyzeCertificate("s", s->getSize(), s->dual(), eps);
analyzeCertificate("d", d->getSize(), d->dual(), eps);
}
else {
    std::cout << "The problem is not primal infeasible, no certificate to show" <<
std::endl;
}
}
```

Running this code will produce the following output:

```
Variable bounds important for infeasibility:
x[5], dual = 1.0
x[6], dual = 1.0
Constraint bounds important for infeasibility:
s[0], dual = -1.0
s[2], dual = -1.0
d[0], dual = 1.0
d[1], dual = 1.0
```

indicating the positions of bounds which appear in the infeasibility certificate with nonzero values. For a more in-depth treatment see the following sections:

- [Sec. 11](#) for more advanced and complicated optimization examples.
- [Sec. 11.1](#) for examples related to portfolio optimization.
- [Sec. 12](#) for formal mathematical formulations of problems **MOSEK** can solve, dual problems and infeasibility certificates.

Chapter 8

Solver Interaction Tutorials

In this section we cover the interaction with the solver.

8.1 Accessing the solution

This section contains important information about the status of the solver and the status of the solution, which must be checked in order to properly interpret the results of the optimization.

8.1.1 Solver termination

If an error occurs during optimization then the method *Model.solve* will throw an exception of type *OptimizeError*. The method *FusionRuntimeException.toString* will produce a description of the error, if available. More about exceptions in [Sec. 8.2](#).

If a runtime error causes the program to crash during optimization, the first debugging step is to enable logging and check the log output. See [Sec. 8.3](#).

If the optimization completes successfully, the next step is to check the solution status, as explained below.

8.1.2 Available solutions

MOSEK uses three kinds of optimizers and provides three types of solutions:

- **basic solution** from the simplex optimizer,
- **interior-point solution** from the interior-point optimizer,
- **integer solution** from the mixed-integer optimizer.

Under standard parameters settings the following solutions will be available for various problem types:

Table 8.1: Types of solutions available from **MOSEK**

	Simplex optimizer	Interior-point optimizer	Mixed-integer optimizer
Linear problem	<i>SolutionType.Basic</i>	<i>SolutionType.Interior</i>	
Conic (nonlinear) problem		<i>SolutionType.Interior</i>	
Problem with integer variables			<i>SolutionType.Integer</i>

For linear problems the user can force a specific optimizer choice making only one of the two solutions available. For example, if the user disables basis identification, then only the interior point solution will be available for a linear problem. Numerical issues may cause one of the solutions to be unknown even if another one is feasible.

Not all components of a solution are always available. For example, there is no dual solution for integer problems and no dual conic variables from the simplex optimizer.

The user will always need to specify which solution should be accessed.

Moreover, the user may be oblivious to the actual solution type by always referring to *SolutionType.Default*, which will automatically select the best available solution, if there is more than one. Moreover, the method *Model.selectedSolution* can be used to fix one solution type for all future references.

8.1.3 Problem and solution status

Assuming that the optimization terminated without errors, the next important step is to check the problem and solution status. There is one for every type of solution, as explained above.

Problem status

Problem status (*ProblemStatus*, retrieved with *Model.getProblemStatus*) determines whether the problem is certified as feasible. Its values can roughly be divided into the following broad categories:

- **feasible** — the problem is feasible. For continuous problems and when the solver is run with default parameters, the feasibility status should ideally be *ProblemStatus.PrimalAndDualFeasible*.
- **primal/dual infeasible** — the problem is infeasible or unbounded or a combination of those. The exact problem status will indicate the type of infeasibility.
- **unknown** — the solver was unable to reach a conclusion, most likely due to numerical issues.

Solution status

Solution status (*SolutionStatus*, retrieved with *Model.getPrimalSolutionStatus* and *Model.getDualSolutionStatus*) provides the information about what the solution values actually contain. The most important broad categories of values are:

- **optimal** (*SolutionStatus.Optimal*) — the solution values are feasible and optimal.
- **certificate** — the solution is in fact a certificate of infeasibility (primal or dual, depending on the solution).
- **unknown** — the status was not determined, typically because of numerical issues, stall etc. Some solution is available, but its quality is not guaranteed.
- **undefined** — this type of solution is not available at all.

The solution status determines the action to be taken. For example, in some cases a suboptimal solution may still be valuable and deserve attention. It is the user's responsibility to check the status and quality of the solution.

Typical status reports

Here are the most typical optimization outcomes described in terms of the problem and solution statuses. Note that these do not cover all possible situations that can occur.

Table 8.2: Continuous problems (solution status for *SolutionType.Interior* or *SolutionType.Basic*)

Outcome	Problem status	Solution status (primal)	Solution status (dual)
Optimal	<i>ProblemStatus.PrimalAndDualFeasible</i>	<i>SolutionStatus.Optimal</i>	<i>SolutionStatus.Optimal</i>
Primal infeasible	<i>ProblemStatus.PrimalInfeasible</i>	<i>SolutionStatus.Undefined</i>	<i>SolutionStatus.Certificate</i>
Dual infeasible (unbounded)	<i>ProblemStatus.DualInfeasible</i>	<i>SolutionStatus.Certificate</i>	<i>SolutionStatus.Undefined</i>
Uncertain (stall, numerical issues, etc.)	<i>ProblemStatus.Unknown</i>	<i>SolutionStatus.Unknown</i>	<i>SolutionStatus.Unknown</i>

Table 8.3: Integer problems (solution status for `SolutionType.Integer`, others undefined)

Outcome	Problem status	Solution status (primal)	Solution status (dual)
Integer optimal	<i>ProblemStatus.PrimalFeasible</i>	<i>SolutionStatus.Optimal</i>	<i>SolutionStatus.Undefined</i>
Infeasible	<i>ProblemStatus.PrimalInfeasible</i>	<i>SolutionStatus.Undefined</i>	<i>SolutionStatus.Undefined</i>
Integer feasible point	<i>ProblemStatus.PrimalFeasible</i>	<i>SolutionStatus.Feasible</i>	<i>SolutionStatus.Undefined</i>
No conclusion	<i>ProblemStatus.Unknown</i>	<i>SolutionStatus.Unknown</i>	<i>SolutionStatus.Undefined</i>

8.1.4 Retrieving solution values

After the meaning and quality of the solution (or certificate) have been established, we can query for the actual numerical values. They can be accessed using:

- `Model.primalObjValue`, `Model.dualObjValue` — the primal and dual objective value.
- `Variable.level` — solution values for the variables.
- `Constraint.level` — values of the constraint expressions in the current solution.
- `Constraint.dual`, `Variable.dual` — dual values.

Remark

By default only *optimal solutions* are returned. An attempt to access a solution with an incompatible status will result in an exception. This can be changed by choosing another level of *acceptable solutions* with the method `Model.acceptedSolutionStatus`. In particular, this method must be called to enable retrieving suboptimal solutions and infeasibility certificates. For instance, one could write

```
M->acceptedSolutionStatus(AccSolutionStatus::Feasible);
```

The current setting of acceptable solutions can be checked with `Model.getAcceptedSolutionStatus`.

8.1.5 Source code example

Below is a source code example with a simple framework for assessing and retrieving the solution to a conic optimization problem.

Listing 8.1: Sample framework for checking optimization result.

```
int main(int argc, char** argv)
{
    Model::t M = new Model(); auto _M = finally([&]() { M->dispose(); });

    // (Optional) set a log stream
    // M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }
    ↪);

    // (Optional) uncomment to see what happens when solution status is unknown
    // M->setSolverParam("intpntMaxIterations", 1);

    // In this example we set up a small conic problem
    setupExample(M);

    // Optimize
    try
```

(continues on next page)

```

{
    M->solve();

    // We expect solution status OPTIMAL (this is also default)
    M->acceptedSolutionStatus(AccSolutionStatus::Optimal);

    auto x = M->getVariable("x");
    auto xsize = x->getSize();
    auto xVal = x->level();
    std::cout << "Optimal value of x = ";
    for(int i = 0; i < xsize; ++i)
        std::cout << (*xVal)[i] << " ";
    std::cout << "\nOptimal primal objective: " << M->primalObjValue() << "\n";
    // .. continue analyzing the solution

}
catch (const OptimizeError& e)
{
    std::cout << "Optimization failed. Error: " << e.what() << "\n";
}
catch (const SolutionError& e)
{
    // The solution with at least the expected status was not available.
    // We try to diagnose why.
    std::cout << "Requested solution was not available.\n";
    auto prosta = M->getProblemStatus();
    switch(prosta)
    {
        case ProblemStatus::DualInfeasible:
            std::cout << "Dual infeasibility certificate found.\n";
            break;

        case ProblemStatus::PrimalInfeasible:
            std::cout << "Primal infeasibility certificate found.\n";
            break;

        case ProblemStatus::Unknown:
            // The solutions status is unknown. The termination code
            // indicates why the optimizer terminated prematurely.
            std::cout << "The solution status is unknown.\n";
            char symname[MSK_MAX_STR_LEN];
            char desc[MSK_MAX_STR_LEN];
            MSK_getcodedesc((MSKrescodee)(M->getSolverIntInfo("optimizeResponse")), &
→symname, desc);
            std::cout << " Termination code: " << symname << " " << desc << "\n";
            break;

        default:
            std::cout << "Another unexpected problem status: " << prosta << "\n";
    }
}
catch (const std::exception& e)
{
    std::cerr << "Unexpected error: " << e.what() << "\n";
}

```

```
M->dispose();
return 0;
}
```

8.2 Errors and exceptions

Exceptions

Almost every method in Fusion API for C++ can throw an exception informing that the requested operation was not performed correctly, and indicating the type of error that occurred. This is the case in situations such as for instance:

- incompatible dimensions in a linear expression,
- defining an invalid value for a parameter,
- accessing an undefined solution,
- repeating a variable name, etc.

It is therefore a good idea to catch exceptions of type *FusionException* and its specific subclasses. The one case where it is *extremely important* to do so is when *Model.solve* is invoked. We will say more about this in [Sec. 8.1](#).

The exception contains a short diagnostic message. They can be accessed as in the following example.

```
try {
    M->setSolverParam("intpntCoTolRelGap", 1.01);
} catch (mosek::fusion::ParameterError& e) {
    std::cout << "Error: " << e.toString() << "\n";
}
```

It will produce as output:

```
Error: Invalid value for parameter (intpntCoTolRelGap)
```

Optimizer errors and warnings

The optimizer may also produce warning messages. They indicate non-critical but important events, that will not prevent solver execution, but may be an indication that something in the optimization problem might be improved. Warning messages are normally printed to a log stream (see [Sec. 8.3](#)). A typical warning is, for example:

```
MOSEK warning 53: A numerically large upper bound value 6.6e+09 is specified for
↳constraint 'C69200' (46020).
```

Error and solution status handling example

Below is a source code example with a simple framework for handling major errors when assessing and retrieving the solution to a conic optimization problem.

Listing 8.2: Sample framework for checking optimization result.

```
int main(int arc, char** argv)
{
    Model::t M = new Model(); auto _M = finally([&]() { M->dispose(); });

    // (Optional) set a log stream
    // M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }
    ↳);
}
```

(continues on next page)

```

// (Optional) uncomment to see what happens when solution status is unknown
// M->setSolverParam("intpntMaxIterations", 1);

// In this example we set up a small conic problem
setupExample(M);

// Optimize
try
{
    M->solve();

    // We expect solution status OPTIMAL (this is also default)
    M->acceptedSolutionStatus(AccSolutionStatus::Optimal);

    auto x = M->getVariable("x");
    auto xsize = x->getSize();
    auto xVal = x->level();
    std::cout << "Optimal value of x = ";
    for(int i = 0; i < xsize; ++i)
        std::cout << (*xVal)[i] << " ";
    std::cout << "\nOptimal primal objective: " << M->primalObjValue() << "\n";
    // .. continue analyzing the solution
}
catch (const OptimizeError& e)
{
    std::cout << "Optimization failed. Error: " << e.what() << "\n";
}
catch (const SolutionError& e)
{
    // The solution with at least the expected status was not available.
    // We try to diagnose why.
    std::cout << "Requested solution was not available.\n";
    auto prosta = M->getProblemStatus();
    switch(prosta)
    {
        case ProblemStatus::DualInfeasible:
            std::cout << "Dual infeasibility certificate found.\n";
            break;

        case ProblemStatus::PrimalInfeasible:
            std::cout << "Primal infeasibility certificate found.\n";
            break;

        case ProblemStatus::Unknown:
            // The solutions status is unknown. The termination code
            // indicates why the optimizer terminated prematurely.
            std::cout << "The solution status is unknown.\n";
            char symname[MSK_MAX_STR_LEN];
            char desc[MSK_MAX_STR_LEN];
            MSK_getcodedesc((MSKrescodee)(M->getSolverIntInfo("optimizeResponse")), &
↪symname, desc);
            std::cout << " Termination code: " << symname << " " << desc << "\n";
            break;
    }
}

```

```

        default:
            std::cout << "Another unexpected problem status: " << prosta << "\n";
        }
    }
    catch (const std::exception& e)
    {
        std::cerr << "Unexpected error: " << e.what() << "\n";
    }

    M->dispose();
    return 0;
}

```

8.3 Input/Output

The *Model* class is also a proxy for input/output operations related to an optimization model.

8.3.1 Stream logging

By default the solver runs silently and does not produce any output to the console or otherwise. However, the log output can be redirected to a user-defined output stream or stream callback function. The log output is analogous to the one produced by the command-line version of **MOSEK**.

To redirect all log messages use the method *Model.setLogHandler*. For instance, we can use the standard output:

```

M->setLogHandler([=](const std::string & msg) { std::cout << msg << std::flush; }  

↪);

```

A log stream can be detached by passing NULL.

8.3.2 Log verbosity

The logging verbosity can be controlled by setting the relevant parameters, as for instance

- *log*,
- *logIntpnt*,
- *logMio*,
- *logCutSecondOpt*,
- *logSim*, and
- *logSimMinor*.

Each parameter controls the output level of a specific functionality or algorithm. The main switch is *log* which affect the whole output. The actual log level for a specific functionality is determined as the minimum between *log* and the relevant parameter. For instance, the log level for the output produce by the interior-point algorithm is tuned by the *logIntpnt*; the actual log level is defined by the minimum between *log* and *logIntpnt*.

Tuning the solver verbosity may require adjusting several parameters. It must be noticed that verbose logging is supposed to be of interest during debugging and tuning. When output is no more of interest, the user can easily disable it globally with *log*. Larger values of *log* do not necessarily result in increased output.

By default **MOSEK** will reduce the amount of log information after the first optimization on a given problem. To get full log output on subsequent re-optimizations set *logCutSecondOpt* to zero.

8.3.3 Saving a problem to a file

An optimization model defined in *Fusion* can be dumped to a file using the method `Model.writeTask`. The file format will be determined from the filename's extension. Supported formats are listed in [Sec. 15](#) together with a table of problem types supported by each.

For instance the problem can be written to a human-readable PTF file (see [Sec. 15.5](#)) with

```
M->writeTask("dump.ptf");
```

All formats can be compressed with `gzip` by appending the `.gz` extension, and with `ZStandard` by appending the `.zst` extension, for example

```
M->writeTask("dump.task.gz");
```

Some remarks:

- The problem is written to the file as it is represented in the underlying *optimizer task*, that is including transformations performed by *Fusion*, if any.
- Unnamed variables are given generic names. It is therefore recommended to use meaningful variable names if the problem file is meant to be human-readable.
- The `task` format is **MOSEK**'s native file format which contains all the problem data as well as solver settings.

8.3.4 Reading a problem from a file

It is not possible to read a file saved with `Model.writeTask` back into *Fusion* because the structure of the high-level optimization model is not saved. However, such problem files can be solved with the command-line tool or read by the low-level Optimizer API if necessary. See the documentation of those interfaces for details.

8.4 Setting solver parameters

MOSEK comes with a large number of parameters that allows the user to tune the behavior of the optimizer. The typical settings which can be changed with solver parameters include:

- choice of the optimizer for linear problems,
- choice of primal/dual solver,
- turning presolve on/off,
- turning heuristics in the mixed-integer optimizer on/off,
- level of multi-threading,
- feasibility tolerances,
- solver termination criteria,
- behaviour of the license manager,

and more. All parameters have default settings which will be suitable for most typical users. The API reference contains:

- *Full list of parameters*
- *List of parameters grouped by topic*

Setting parameters

Each parameter is identified by a unique string name and it can accept either integers, floating point values or symbolic strings. Parameters are set using the method `Model.setSolverParam`. *Fusion* will try to convert the given argument to the exact expected type, and will raise an exception if that fails.

Some parameters accept only symbolic strings from a fixed set of values. The set of accepted values for every parameter is provided in the API reference.

For example, the following piece of code sets up parameters which choose and tune the interior point optimizer before solving a problem.

Listing 8.3: Parameter setting example.

```
// Set log level (integer parameter)
M->setSolverParam("log", 1);
// Select interior-point optimizer... (parameter with symbolic string values)
M->setSolverParam("optimizer", "intpnt");
// ... without basis identification (parameter with symbolic string values)
M->setSolverParam("intpntBasis", "never");
// Set relative gap tolerance (double parameter)
M->setSolverParam("intpntCoTolRelGap", 1.0e-7);

// The same in a different way
M->setSolverParam("intpntCoTolRelGap", "1.0e-7");

// Incorrect value
try {
    M->setSolverParam("intpntCoTolRelGap", -1);
}
catch (mosek::fusion::ParameterError) {
    std::cout << "Wrong parameter value\n";
}
```

8.5 Retrieving information items

After the optimization the user has access to the solution as well as to a report containing a large amount of additional *information items*. For example, one can obtain information about:

- **timing**: total optimization time, time spent in various optimizer subroutines, number of iterations, etc.
- **solution quality**: feasibility measures, solution norms, constraint and bound violations, etc.
- **problem structure**: counts of variables of different types, constraints, nonzeros, etc.
- **integer optimizer**: integrality gap, objective bound, number of cuts, etc.

and more. Information items are numerical values of integer, long integer or double type. The full list can be found in the API reference:

- *Double information items*
- *Integer information items*
- *Long information items*

Certain information items make sense, and are made available, also *during* the optimization process. They can be accessed from a callback function, see [Sec. 8.7](#) for details.

Remark

For efficiency reasons, not all information items are automatically computed after optimization. To force all information items to be updated use the parameter `autoUpdateSolInfo`.

Retrieving the values

Values of information items are fetched using one of the methods

- `Model.getSolverDoubleInfo` for a double information item,
- `Model.getSolverIntInfo` for an integer information item,
- `Model.getSolverLIntInfo` for a long integer information item.

Each information item is identified by a unique name. The example below reads two pieces of data from the solver: total optimization time and the number of interior-point iterations.

Listing 8.4: Information items example.

```
double tm = M->getSolverDoubleInfo("optimizerTime");
int it = M->getSolverIntInfo("intpntIter");

std::cout << "Time: " << tm << "\nIterations: " << it << "\n";
```

8.6 Stopping the solver

The `Model` provides the method `Model.breakSolver` that notifies the solver that it must stop as soon as possible. The solver will not terminate momentarily, as it only periodically checks for such notifications. In any case, it will stop as soon as possible. The typical usage pattern of this method would be:

- build the optimization model `M`,
- create a separate thread in which `M` will run,
- break the solver by calling `Model.breakSolver` from the main thread.

Warnings and comments:

- It is recommended to use the solver parameters to set or modify standard built-in termination criteria (such as maximal running time, solution tolerances etc.). See [Sec. 8.4](#).
- More complicated user-defined termination criteria can be implemented within a callback function. See [Sec. 8.7](#).
- The state of the solver and solution after termination may be undefined.
- This operation is very language dependent and particular care must be taken to avoid stalling or other undesired side effects.

8.6.1 Example: Setting a Time Limit

For the purpose of the tutorial we will implement a busy-waiting breaker with the time limit as a termination criterion. Note that in practice it would be better just to set the parameter `optimizerMaxTime`.

Suppose we built a model `M` that is known to run for quite a long time (in the accompanying example code we create a particular integer program). Then we could create a new thread solving the model:

```
bool alive = true;
std::thread T(std::function<void(void)>([&]() { M->solve(); alive = false; }) );
```

In the main thread we are going to check if a time limit has elapsed. After calling `Model.breakSolver` we should wait for the solver thread to actually return. Altogether this scenario can be implemented as follows:

Listing 8.5: Stopping solver execution.

```
bool alive = true;
std::thread T(std::function<void(void)>([&]() { M->solve(); alive = false; }) );

time_t T0 = time(NULL);
while (true)
{
    if (time(NULL) - T0 > timeout)
    {
        std::cout << "Solver terminated due to timeout!\n";
        M->breakSolver();
        T.join();
        break;
    }
    if (! alive)
    {
        std::cout << "Solver terminated before anything happened!\n";
        T.join();
        break;
    }
}
```

8.7 Progress and data callback

Callbacks are a very useful mechanism that allow the caller to track the progress of the **MOSEK** optimizer. A callback function provided by the user is regularly called during the optimization and can be used to

- obtain a customized log of the solver execution,
- collect information for debugging purposes or
- ask the solver to terminate.

Fusion API for C++ has the following callback mechanisms:

- **progress callback**, which provides only the basic status of the solver.
- **data callback**, which provides the solver status and a complete set of information items that describe the progress of the optimizer in detail.

Warning

The callbacks functions *must not* invoke any functions of the solver, environment or task. Otherwise the state of the solver and its outcome are undefined.

8.7.1 Data callback

In the data callback **MOSEK** passes a callback code and values of all information items to a user-defined function. The callback function is called, in particular, at the beginning of each iteration of the interior-point optimizer. For the simplex optimizers *logSimFreq* controls how frequently the call-back is called. Note that the callback is done quite frequently, which can lead to degraded performance. If the information items are not required, the simpler progress callback may be a better choice.

The data callback is set by calling the method *Model.setDataCallbackHandler*.

The callback function should have the following signature:

```
typedef std::function<bool(MSKcallbackcodee, const double *, const int32_t *, const_
↳int64_t *)> callbackHandler_t;
```

Arguments:

- `caller` - the status of the optimizer.
- `douinf` - values of double information items.
- `intinf` - values of integer information items.
- `lintinf` - values of long information items.

Return value: Non-zero return value of the callback function indicates that the optimizer should be terminated.

8.7.2 Progress callback

In the progress callback **MOSEK** provides a single code indicating the current stage of the optimization process.

The callback is set by calling the method `Model.setCallbackHandler`.

The callback function should have the following signature

```
typedef std::function<int(MSKcallbackcodee)> progressHandler_t;
```

Arguments:

- `caller` - the status of the optimizer.

Return value: Non-zero return value of the callback function indicates that the optimizer should be terminated.

8.7.3 Working example: Data callback

The following example defines a data callback function that prints out some of the information items. It interrupts the solver after a certain time limit. Note that the time limit refers to time spent in the solver and does not include setting up the model in *Fusion*.

Listing 8.6: An example of a data callback function.

```
static int MSKAPI usercallback( MSKcallbackcodee caller,
                                const double * douinf,
                                const int32_t * intinf,
                                const int64_t * lintinf,
                                Model::t mod,
                                const double maxtime)
{
    switch ( caller )
    {
        case MSK_CALLBACK_BEGIN_INTPNT:
            std::cerr << "Starting interior-point optimizer\n";
            break;
        case MSK_CALLBACK_INTPNT:
            std::cerr << "Iterations: " << intinf[MSK_IINF_INTPNT_ITER];
            std::cerr << " (" << douinf[MSK_DINF_OPTIMIZER_TIME] << "/" <<
            std::cerr << douinf[MSK_DINF_INTPNT_TIME] << ")s. \n";
            std::cerr << "Primal obj.: " << douinf[MSK_DINF_INTPNT_PRIMAL_OBJ];
            std::cerr << " Dual obj.: " << douinf[MSK_DINF_INTPNT_DUAL_OBJ] << std::endl;
            break;
        case MSK_CALLBACK_END_INTPNT:
            std::cerr << "Interior-point optimizer finished.\n";
            break;
        case MSK_CALLBACK_BEGIN_PRIMAL_SIMPLEX:
            std::cerr << "Primal simplex optimizer started.\n";
```

(continues on next page)

```

    break;
case MSK_CALLBACK_UPDATE_PRIMAL_SIMPLEX:
    std::cerr << "Iterations: " << intinf[MSK_IINF_SIM_PRIMAL_ITER];
    std::cerr << "   Elapsed time: " << douinf[MSK_DINF_OPTIMIZER_TIME];
    std::cerr << "(" << douinf[MSK_DINF_SIM_TIME] << ")\n";
    std::cerr << "Obj.: " << douinf[MSK_DINF_SIM_OBJ] << std::endl;
    break;
case MSK_CALLBACK_END_PRIMAL_SIMPLEX:
    std::cerr << "Primal simplex optimizer finished.\n";
    break;
case MSK_CALLBACK_BEGIN_DUAL_SIMPLEX:
    std::cerr << "Dual simplex optimizer started.\n";
    break;
case MSK_CALLBACK_UPDATE_DUAL_SIMPLEX:
    std::cerr << "Iterations: " << intinf[MSK_IINF_SIM_DUAL_ITER];
    std::cerr << "   Elapsed time: " << douinf[MSK_DINF_OPTIMIZER_TIME];
    std::cerr << "(" << douinf[MSK_DINF_SIM_TIME] << ")\n";
    std::cerr << "Obj.: " << douinf[MSK_DINF_SIM_OBJ] << std::endl;
    break;
case MSK_CALLBACK_END_DUAL_SIMPLEX:
    std::cerr << "Dual simplex optimizer finished.\n";
    break;
case MSK_CALLBACK_BEGIN_BI:
    std::cerr << "Basis identification started.\n";
    break;
case MSK_CALLBACK_END_BI:
    std::cerr << "Basis identification finished.\n";
    break;
default:
    break;
}
if ( douinf[MSK_DINF_OPTIMIZER_TIME] >= maxtime )
{
    std::cerr << "MOSEK is spending too much time. Terminate it.\n";
    return 1;
}
return 0;
} /* usercallback */

```

Assuming that we have defined a model M and a time limit `maxtime`, the callback function is attached as follows:

Listing 8.7: Attaching the data callback function to the model.

```
callbackHandler_t cllbck = [&](MSKcallbackcodee caller,
                             const double * douinf, const int32_t* intinf, const
→int64_t* lintinf)
{
    return usercallback(caller, douinf, intinf, lintinf, M, maxtime);
};

M->setDataCallbackHandler(cllbck);
```

8.8 Optimizer API Task

This section is intended for advanced users and should normally never be followed unless advanced debugging or very specialized functionalities are required.

The *Model* is a wrapper on top of an underlying **MOSEK** low-level optimizer task. Access to the task is provided by the method *Model.getTask*. The functionalities available from the task are described in the documentation of the relevant Optimizer API.

Warning

Note that the user gets access to the *actual task* in the model, and *not* its clone. Changing the state of the task will most likely invalidate the *Fusion* model.

8.9 MOSEK OptServer

MOSEK provides an easy way to offload optimization problem to a remote server. This section demonstrates related functionalities from the client side, i.e. sending optimization tasks to the remote server and retrieving solutions.

Setting up and configuring the remote server is described in a separate manual for the OptServer.

8.9.1 Synchronous Remote Optimization

In synchronous mode the client sends an optimization problem to the server and blocks, waiting for the optimization to end. Once the result has been received, the program can continue. This is the simplest mode all it takes is to provide the address of the server before starting optimization. The rest of the code remains untouched.

Note that it is impossible to recover the job in case of a broken connection.

Source code example

Listing 8.8: Using the OptServer in synchronous mode.

```
#include <fusion.h>
#include <stdlib.h>
#include <iostream>

using namespace mosek::fusion;
using namespace monty;

int main(int argc, char ** argv) {
    if (argc<2) {
        std::cout << "Missing argument, syntax is:" << std::endl;
        std::cout << "  opt_server_sync addr [certpath]" << std::endl;
        exit(0);
    }
```

(continues on next page)

```

}

std::string serveraddr(argv[1]);
std::string tlscert(argc==3 ? argv[2] : "");

// Setup a simple test problem
Model::t M = new Model("testOptServer"); auto _M = finally([&]() { M->dispose(); }
→ );
Variable::t x = M->variable("x", 3, Domain::greaterThan(0.0));
M->constraint("lc", Expr::dot(new_array_ptr<double>, 1>({1.0, 1.0, 2.0}), x),
→ Domain::equalsTo(1.0));
M->objective("obj", ObjectiveSense::Minimize, Expr::sum(x));

// Attach log handler
M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }
→ );

// Set OptServer URL
M->optserverHost(serveraddr);

// Path to certificate, if any
M->setSolverParam("remoteTlsCertPath", tlscert);

// Solve the problem on the OptServer
M->solve();

// Get the solution
std::cout << "x1,x2,x3 = " << *(x->level()) << std::endl;

return 0;
}

```

Chapter 9

Debugging Tutorials

This collection of tutorials contains basic techniques for debugging optimization problems using tools available in **MOSEK**: optimizer log, solution summary, infeasibility report, command-line tools. It is intended as a first line of technical help for issues such as: Why do I get solution status *unknown* and how can I fix it? Why is my model infeasible while it shouldn't be? Should I change some parameters? Can the model solve faster? etc.

The major steps when debugging a model are always:

- Enable log output. See [Sec. 8.3.1](#) for how to do it. In the simplest case:

```
M->setLogHandler([=](const std::string & msg) { std::cout << msg << std:::  
↪flush; } );
```

- Run the optimization and analyze the log output, see [Sec. 9.1](#). In particular:
 - check if the problem setup (number of constraints/variables etc.) matches your expectation.
 - check solution summary and solution status.
- Dump the problem to disk if necessary to continue analysis. See [Sec. 8.3.3](#).
 - use a human-readable text format, preferably *.ptf if you want to check the problem structure by hand. Assign names to variables and constraints to make them easier to identify.

```
M->writeTask("dump.ptf");
```

- use the **MOSEK** native format *.task.gz when submitting a bug report or support question.

```
M->writeTask("dump.task.gz");
```

- Fix problem setup, improve the model, locate infeasibility or adjust parameters, depending on the diagnosis.

See the following sections for details.

9.1 Understanding optimizer log

The optimizer produces a log which splits roughly into four sections:

1. summary of the input data,
2. presolve and other pre-optimize problem setup stages,
3. actual optimizer iterations,
4. solution summary.

In this tutorial we show how to analyze the most important parts of the log when initially debugging a model: input data (1) and solution summary (4). For the iterations log (3) see [Sec. 13.3.4](#) or [Sec. 13.4.4](#).

9.1.1 Input data

If **MOSEK** behaves very far from expectations it may be due to errors in problem setup. The log file will begin with a summary of the structure of the problem, which looks for instance like:

```
Problem
  Name           :
  Objective sense : minimize
  Type           : CONIC (conic optimization problem)
  Constraints     : 234
  Affine conic cons. : 5348
  Disjunctive cons. : 0
  Cones          : 0
  Scalar variables : 20693
  Matrix variables : 0
  Integer variables : 0
```

This can be consulted to eliminate simple errors: wrong objective sense, wrong number of variables etc. Note that some modeling tools can introduce additional variables and constraints to the model and perturb the model even further (such as by dualizing). In most **MOSEK** APIs the problem dimensions should match exactly what the user specified.

If this is not sufficient a bit more information can be obtained by dumping the problem to a file (see [Sec. 9](#)) and using the `anapro` option of any of the command line tools. This will produce a longer summary similar to:

```
** Variables
scalar: 20414      integer: 0      matrix: 0
low: 2082          up: 5014        ranged: 0      free: 12892      fixed: 426

** Constraints
all: 20413
low: 10028        up: 0           ranged: 0      free: 0          fixed: 10385

** Affine conic constraints (ACC)
QUAD: 1           dims: 2865: 1
RQUAD: 2507       dims: 3: 2507

** Problem data (numerics)
|c|               nnz: 10028      min=2.09e-05   max=1.00e+00
|A|               nnz: 597023     min=1.17e-10   max=1.00e+00
blx               fin: 2508       min=-3.60e+09   max=2.75e+05
bux               fin: 5440       min=0.00e+00   max=2.94e+08
blc               fin: 20413     min=-7.61e+05   max=7.61e+05
buc               fin: 10385     min=-5.00e-01   max=0.00e+00
|F|               nnz: 612301     min=8.29e-06   max=9.31e+01
|g|               nnz: 1203       min=5.00e-03   max=1.00e+00
```

Again, this can be used to detect simple errors, such as:

- Wrong type of conic constraint was used or it has wrong dimension.
- The bounds for variables or constraints are incorrect or incomplete.
- The model is otherwise incomplete.
- Suspicious values of coefficients.
- For various data sizes the model does not scale as expected.

Finally saving the problem in a human-friendly text format such as LP or PTF (see [Sec. 9](#)) and analyzing it by hand can reveal if the model is correct.

Warnings and errors

At this stage the user can encounter warnings which should not be ignored, unless they are well-understood. They can also serve as hints as to numerical issues with the problem data. A typical warning of this kind is

```
MOSEK warning 53: A numerically large upper bound value 2.9e+08 is specified for
↪variable 'absh[107]' (2613).
```

Warnings do not stop the problem setup. If, on the other hand, an error occurs then the model will become invalid. The user should make sure to test for errors/exceptions from all API calls that set up the problem and validate the data. See [Sec. 8.2](#) for more details.

9.1.2 Solution summary

The last item in the log is the solution summary.

Continuous problem

Optimal solution

A typical solution summary for a continuous (linear, conic, quadratic) problem looks like:

```
Problem status : PRIMAL_AND_DUAL_FEASIBLE
Solution status : OPTIMAL
Primal.  obj: 8.7560516107e+01    nrm: 1e+02    Viol.  con: 3e-12    var: 0e+00    ↪
↪acc: 3e-11
Dual.    obj: 8.7560521345e+01    nrm: 1e+00    Viol.  con: 5e-09    var: 9e-11    ↪
↪acc: 0e+00
```

It contains the following elements:

- Problem and solution status. For details see [Sec. 8.1.3](#).
- A summary of the primal solution: objective value, infinity norm of the solution vector and maximal violations of variables and constraints of different types. The violation of a linear constraint such as $a^T x \leq b$ is $\max(a^T x - b, 0)$. The violation of a conic constraint is the distance to the cone.
- The same for the dual solution.

The features of the solution summary which characterize a very good and accurate solution and a well-posed model are:

- **Status:** The solution status is `OPTIMAL`.
- **Duality gap:** The primal and dual objective values are (almost) identical, which proves the solution is (almost) optimal.
- **Norms:** Ideally the norms of the solution and the objective values should not be too large. This of course depends on the input data, but a huge solution norm can be an indicator of issues with the scaling, conditioning and/or well-posedness of the model. It may also indicate that the problem is borderline between feasibility and infeasibility and sensitive to small perturbations in this respect.
- **Violations:** The violations are close to zero, which proves the solution is (almost) feasible. Observe that due to rounding errors it can be expected that the violations are proportional to the norm (`nrm:`) of the solution. It is rarely the case that violations are exactly zero.

Solution status UNKNOWN

A typical example with solution status UNKNOWN due to numerical problems will look like:

```
Problem status : UNKNOWN
Solution status : UNKNOWN
Primal.  obj: 1.3821656824e+01    nrm: 1e+01    Viol.  con: 2e-03    var: 0e+00    ┐
↪acc: 0e+00
Dual.    obj: 3.0119004098e-01    nrm: 5e+07    Viol.  con: 4e-16    var: 1e-01    ┐
↪acc: 0e+00
```

Note that:

- The primal and dual objective are very different.
- The dual solution has very large norm.
- There are considerable violations so the solution is likely far from feasible.

Follow the hints in [Sec. 9.2](#) to resolve the issue.

Solution status UNKNOWN with a potentially useful solution

Solution status UNKNOWN does not necessarily mean that the solution is completely useless. It only means that the solver was unable to make any more progress due to numerical difficulties, and it was not able to reach the accuracy required by the termination criteria (see [Sec. 13.3.2](#)). Consider for instance:

```
Problem status : UNKNOWN
Solution status : UNKNOWN
Primal.  obj: 3.4531019648e+04    nrm: 1e+05    Viol.  con: 7e-02    var: 0e+00    ┐
↪acc: 0e+00
Dual.    obj: 3.4529720645e+04    nrm: 8e+03    Viol.  con: 1e-04    var: 2e-04    ┐
↪acc: 0e+00
```

Such a solution may still be useful, and it is always up to the user to decide. It may be a good enough approximation of the optimal point. For example, the large constraint violation may be due to the fact that one constraint contained a huge coefficient.

Infeasibility certificate

A primal infeasibility certificate is stored in the dual variables:

```
Problem status : PRIMAL_INFEASIBLE
Solution status : PRIMAL_INFEASIBLE_CER
Dual.    obj: 2.9238975853e+02    nrm: 6e+02    Viol.  con: 0e+00    var: 1e-11    ┐
↪acc: 0e+00
```

It is a Farkas-type certificate as described in [Sec. 12.2.2](#). In particular, for a good certificate:

- The dual objective is positive for a minimization problem, negative for a maximization problem. Ideally it is well bounded away from zero.
- The norm is not too big and the violations are small (as for a solution).

If the model was not expected to be infeasible, the likely cause is an error in the problem formulation. Use the hints in [Sec. 9.1.1](#) and [Sec. 9.3](#) to locate the issue.

Just like a solution, the infeasibility certificate can be of better or worse quality. The infeasibility certificate above is very solid. However, there can be less clear-cut cases, such as for example:

```
Problem status : PRIMAL_INFEASIBLE
Solution status : PRIMAL_INFEASIBLE_CER
Dual.    obj: 1.6378689238e-06    nrm: 6e+05    Viol.  con: 7e-03    var: 2e-04    ┐
↪acc: 0e+00
```

This infeasibility certificate is more dubious because the dual objective is positive, but barely so in comparison with the large violations. It also has rather large norm. This is more likely an indication that the problem is borderline between feasibility and infeasibility or simply ill-posed and sensitive to tiny variations in input data. See [Sec. 9.3](#) and [Sec. 9.2](#).

The same remarks apply to dual infeasibility (i.e. unboundedness) certificates. Here the primal objective should be negative a minimization problem and positive for a maximization problem.

9.1.3 Mixed-integer problem

Optimal integer solution

For a mixed-integer problem there is no dual solution and a typical optimal solution report will look as follows:

```
Problem status : PRIMAL_FEASIBLE
Solution status : INTEGER_OPTIMAL
Primal.  obj: 6.0111122960e+06    nrm: 1e+03    Viol.  con: 2e-13    var: 2e-14    ̐
↪itg: 5e-15
```

The interpretation of all elements is as for a continuous problem. The additional field `itg` denotes the maximum violation of an integer variable from being an exact integer.

Feasible integer solution

If the solver found an integer solution but did not prove optimality, for instance because of a time limit, the solution status will be `PRIMAL_FEASIBLE`:

```
Problem status : PRIMAL_FEASIBLE
Solution status : PRIMAL_FEASIBLE
Primal.  obj: 6.0114607792e+06    nrm: 1e+03    Viol.  con: 2e-13    var: 2e-13    ̐
↪itg: 4e-15
```

In this case it is valuable to go back to the optimizer summary to see how good the best solution is:

```
31      35      1      0      6.0114607792e+06      6.0078960892e+06      0.06    ̐
↪      4.1

Objective of best integer solution : 6.011460779193e+06
Best objective bound                : 6.007896089225e+06
```

In this case the best integer solution found has objective value `6.011460779193e+06`, the best proved lower bound is `6.007896089225e+06` and so the solution is guaranteed to be within 0.06% from optimum. The same data can be obtained as information items through an API. See also [Sec. 13.4](#) for more details.

Infeasible problem

If the problem is declared infeasible the summary is simply

```
Problem status : PRIMAL_INFEASIBLE
Solution status : UNKNOWN
Primal.  obj: 0.0000000000e+00    nrm: 0e+00    Viol.  con: 0e+00    var: 0e+00    ̐
↪itg: 0e+00
```

If infeasibility was not expected, consult [Sec. 9.3](#).

9.2 Addressing numerical issues

The suggestions in this section should help diagnose and solve issues with numerical instability, in particular **UNKNOWN** solution status or solutions with large violations. Since numerically stable models tend to solve faster, following these hints can also dramatically shorten solution times.

We always recommend that issues of this kind are addressed by reformulating or rescaling the model, since it is the modeler who has the best insight into the structure of the problem and can fix the cause of the issue.

9.2.1 Formulating problems

Scaling

Make sure that all the data in the problem are of comparable orders of magnitude. This applies especially to the linear constraint matrix. Use [Sec. 9.1.1](#) if necessary. For example a report such as

A	nnz: 597023	min=1.17e-6	max=2.21e+5
---	-------------	-------------	-------------

means that the ratio of largest to smallest elements in **A** is 10^{11} . In this case the user should rescale or reformulate the model to avoid such spread which makes it difficult for **MOSEK** to scale the problem internally. In many cases it may be possible to change the units, i.e. express the model in terms of rescaled variables (for instance work with millions of dollars instead of dollars, etc.).

Similarly, if the objective contains very different coefficients, say

$$\text{maximize } 10^{10}x + y$$

then it is likely to lead to inaccuracies. The objective will be dominated by the contribution from x and y will become insignificant.

Removing huge bounds

Never use a very large number as replacement for ∞ . Instead define the variable or constraint as unbounded from below/above. Similarly, avoid artificial huge bounds if you expect they will not become tight in the optimal solution.

Avoiding linear dependencies

As much as possible try to avoid linear dependencies and near-linear dependencies in the model. See [Example 9.3](#).

Avoiding ill-posedness

Avoid continuous models which are ill-posed: the solution space is degenerate, for example consists of a single point (technically, the Slater condition is not satisfied). In general, this refers to problems which are borderline between feasible and infeasible. See [Example 9.1](#).

Scaling the expected solution

Try to formulate the problem in such a way that the expected solution (both primal and dual) is not very large. Consult the solution summary [Sec. 9.1.2](#) to check the objective values or solution norms.

9.2.2 Further suggestions

Here are other simple suggestions that can help locate the cause of the issues. They can also be used as hints for how to tune the optimizer if fixing the root causes of the issue is not possible.

- Remove the objective and solve the feasibility problem. This can reveal issues with the objective.
- Change the objective or change the objective sense from minimization to maximization (if applicable). If the two objective values are almost identical, this may indicate that the feasible set is very small, possibly degenerate.
- Perturb the data, for instance bounds, very slightly, and compare the results.
- For linear problems: solve the problem using a different optimizer by setting the parameter `optimizer` and compare the results.
- Force the optimizer to solve the primal/dual versions of the problem by setting the parameter `intpntSolveForm` or `simSolveForm`. **MOSEK** has a heuristic to decide whether to dualize, but for some problems the guess is wrong an explicit choice may give better results.
- Solve the problem without presolve or some of its parts by setting the parameter `presolveUse`, see Sec. 13.1.
- Use different numbers of threads (`numThreads`) and compare the results. Very different results indicate numerical issues resulting from round-off errors.

If the problem was dumped to a file, experimenting with various parameters is facilitated with the **MOSEK** Command Line Tool or **MOSEK** Python Console Sec. 9.4.

9.2.3 Typical pitfalls

Example 9.1 (Ill-posedness). A toy example of this situation is the feasibility problem

$$(x - 1)^2 \leq 1, (x + 1)^2 \leq 1$$

whose only solution is $x = 0$ and moreover replacing any 1 on the right hand side by $1 - \varepsilon$ makes the problem infeasible and replacing it by $1 + \varepsilon$ yields a problem whose solution set is an interval (fully-dimensional). This is an example of ill-posedness.

Example 9.2 (Huge solution). If the norm of the expected solution is very large it may lead to numerical issues or infeasibility. For example the problem

$$(10^{-4}, x, 10^3) \in \mathcal{Q}_r^3$$

may be declared infeasible because the expected solution must satisfy $x \geq 5 \cdot 10^9$.

Example 9.3 (Near linear dependency). Consider the following problem:

$$\begin{array}{llllll} \text{minimize} & & & & & \\ \text{subject to} & x_1 & + & x_2 & & = & 1, \\ & & & & x_3 & + & x_4 & = & 1, \\ & - & x_1 & & - & x_3 & & = & -1 + \varepsilon, \\ & & - & x_2 & & - & x_4 & = & -1, \\ & x_1, & & x_2, & & x_3, & & x_4 & \geq & 0. \end{array}$$

If we add the equalities together we obtain:

$$0 = \varepsilon$$

which is infeasible for any $\varepsilon \neq 0$. Here infeasibility is caused by a linear dependency in the constraint matrix coupled with a precision error represented by the ε . Indeed if a problem contains linear dependencies then the problem is either infeasible or contains redundant constraints. In the above case any of the equality constraints can be removed while not changing the set of feasible solutions. To summarize linear dependencies in the constraints can give rise to infeasible problems and therefore it is better to avoid them.

Example 9.4 (Presolving very tight bounds). Next consider the problem

$$\begin{array}{ll} \text{minimize} & \\ \text{subject to} & x_1 - 0.01x_2 = 0, \\ & x_2 - 0.01x_3 = 0, \\ & x_3 - 0.01x_4 = 0, \\ & x_1 \geq -10^{-9}, \\ & x_1 \leq 10^{-9}, \\ & x_4 \geq 10^{-4}. \end{array}$$

Now the **MOSEK** presolve will, for the sake of efficiency, fix variables (and constraints) that have tight bounds where tightness is controlled by the parameter `presolveTolX`. Since the bounds

$$-10^{-9} \leq x_1 \leq 10^{-9}$$

are tight, presolve will set $x_1 = 0$. It is easy to see that this implies $x_4 = 0$, which leads to the incorrect conclusion that the problem is infeasible. However a tiny change of the value 10^{-9} makes the problem feasible. In general it is recommended to avoid ill-posed problems, but if that is not possible then one solution is to reduce parameters such as `presolveTolX` to say 10^{-10} . This will at least make sure that presolve does not make the undesired reduction.

9.3 Debugging infeasibility

When solving an optimization problem one typically expects to get an optimal solution, but in some cases, either by design, or (most frequently) due to an error in the formulation, the problem may become infeasible (have no solution at all).

This section

- describes the intuitions behind infeasibility,
- helps to debug (unexpectedly) infeasible problems using the command line tool and by inspecting infeasibility reports and problem data by hand,
- gives some hints for how to modify the formulation to identify the reasons for infeasibility.

If, instead, you want to fetch an infeasibility certificate directly using Fusion API for C++, see the tutorial in [Sec. 7.12](#).

An infeasibility certificate is only available for continuous problems, however the hints in [Sec. 9.3.4](#) apply to a large extent also to mixed-integer problems.

9.3.1 Numerical issues

Infeasible problem status may be just an artifact of numerical issues appearing when the problem is badly-scaled, barely feasible or otherwise ill-conditioned so that it is unstable under small perturbations of the data or round-off errors. This may be visible in the solution summary if the infeasibility certificate has poor quality. See [Sec. 9.1.2](#) for how to diagnose that and [Sec. 9.2](#) for possible hints. [Sec. 9.2.3](#) contains examples of situations which may lead to infeasibility for numerical reasons.

We refer to [Sec. 9.2](#) for further information on dealing with those sort of issues. For the rest of this section we concentrate on the case when the solution summary leaves little doubt that the problem solved by the optimizer actually is infeasible.

9.3.2 Locating primal infeasibility

As an example of a primal infeasible problem consider minimizing the cost of transportation between a number of production plants and stores: Each plant produces a fixed number of goods, and each store has a fixed demand that must be met. Supply, demand and cost of transportation per unit are given in Fig. 9.1.

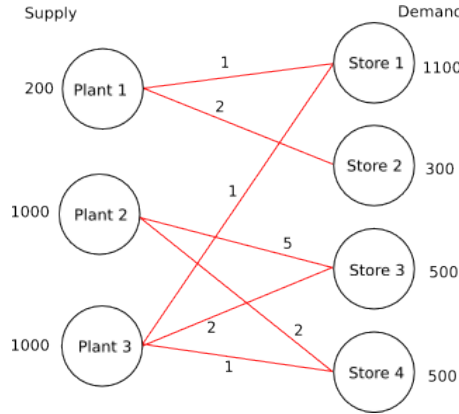


Fig. 9.1: Supply, demand and cost of transportation.

The problem represented in Fig. 9.1 is infeasible, since the total demand

$$2300 = 1100 + 200 + 500 + 500$$

exceeds the total supply

$$2200 = 200 + 1000 + 1000$$

If we denote the number of transported goods from plant i to store j by x_{ij} , the problem can be formulated as the LP:

$$\begin{aligned}
 & \text{minimize} && x_{11} + 2x_{12} + 5x_{23} + 2x_{24} + x_{31} + 2x_{33} + x_{34} \\
 & \text{subject to} && s_0 : x_{11} + x_{12} \leq 200, \\
 & && s_1 : x_{23} + x_{24} \leq 1000, \\
 & && s_2 : x_{31} + x_{33} + x_{34} \leq 1000, \\
 & && d_1 : x_{11} + x_{31} = 1100, \\
 & && d_2 : x_{12} = 200, \\
 & && d_3 : x_{23} + x_{33} = 500, \\
 & && d_4 : x_{24} + x_{34} = 500, \\
 & && x_{ij} \geq 0.
 \end{aligned} \tag{9.1}$$

Solving problem (9.1) using **MOSEK** will result in an infeasibility status. The infeasibility certificate is contained in the dual variables and can be accessed from an API. The variables and constraints with nonzero solution values form an infeasible subproblem, which frequently is very small. See Sec. 12.1.2 or Sec. 12.2.2 for detailed specifications of infeasibility certificates.

A short infeasibility report can also be printed to the log stream. It can be turned on by setting the parameter `MSK_IPAR_INFEAS_REPORT_AUTO` to `MSK_ON` in the command-line tool. This causes **MOSEK** to print a report on variables and constraints which are involved in infeasibility in the above sense, i.e. have nonzero values in the certificate. The parameter `MSK_IPAR_INFEAS_REPORT_LEVEL` controls the amount of information presented in the infeasibility report. The default value is 1. For the above example the report is

MOSEK PRIMAL INFEASIBILITY REPORT.

Problem status: The problem is primal infeasible

(continues on next page)

The following constraints are involved in the primal infeasibility.

Index	Name	Lower bound	Upper bound	Dual lower	Dual upper
0	s0	NONE	2.000000e+002	0.000000e+000	1.000000e+000
2	s2	NONE	1.000000e+003	0.000000e+000	1.000000e+000
3	d1	1.100000e+003	1.100000e+003	1.000000e+000	0.000000e+000
4	d2	2.000000e+002	2.000000e+002	1.000000e+000	0.000000e+000

The following bound constraints are involved in the infeasibility.

Index	Name	Lower bound	Upper bound	Dual lower	Dual upper
8	x33	0.000000e+000	NONE	1.000000e+000	0.000000e+000
10	x34	0.000000e+000	NONE	1.000000e+000	0.000000e+000

The infeasibility report is divided into two sections corresponding to constraints and variables. It is a selection of those lines from the problem solution which are important in understanding primal infeasibility. In this case the constraints s0, s2, d1, d2 and variables x33, x34 are of importance because of nonzero dual values. The columns **Dual lower** and **Dual upper** contain the values of dual variables s_l^c , s_u^c , s_l^x and s_u^x in the primal infeasibility certificate (see Sec. 12.1.2).

In our example the certificate means that an appropriate linear combination of constraints s0, s1 with coefficient $s_u^c = 1$, constraints d1 and d2 with coefficient $s_u^c - s_l^c = 0 - 1 = -1$ and lower bounds on x33 and x34 with coefficient $-s_l^x = -1$ gives a contradiction. Indeed, the combination of the four involved constraints is $x_{33} + x_{34} \leq -100$ (as indicated in the introduction, the difference between supply and demand).

It is also possible to extract the infeasible subproblem with the command-line tool. For an infeasible problem called `infeas.lp` the command:

```
mosek -d MSK_IPAR_INFEAS_REPORT_AUTO MSK_ON infeas.lp -info rinfeas.lp
```

will produce the file `rinfeas.bas.inf.lp` which contains the infeasible subproblem. Because of its size it may be easier to work with than the original problem file.

Returning to the transportation example, we discover that removing the fifth constraint $x_{12} = 200$ makes the problem feasible. Almost all undesired infeasibilities should be fixable at the modeling stage.

9.3.3 Locating dual infeasibility

A problem may also be *dual infeasible*. In this case the primal problem is usually unbounded, meaning that feasible solutions exists such that the objective tends towards infinity. For example, consider the problem

$$\begin{aligned}
 &\text{maximize} && 200y_1 + 1000y_2 + 1000y_3 + 1100y_4 + 200y_5 + 500y_6 + 500y_7 \\
 &\text{subject to} && y_1 + y_4 \leq 1, \quad y_1 + y_5 \leq 2, \quad y_2 + y_6 \leq 5, \quad y_2 + y_7 \leq 2, \\
 &&& y_3 + y_4 \leq 1, \quad y_3 + y_6 \leq 2, \quad y_3 + y_7 \leq 1 \\
 &&& y_1, y_2, y_3 \leq 0
 \end{aligned}$$

which is dual to (9.1) (and therefore is dual infeasible). The dual infeasibility report may look as follows:

MOSEK DUAL INFEASIBILITY REPORT.

Problem status: The problem is dual infeasible

The following constraints are involved in the infeasibility.

Index	Name	Activity	Objective	Lower bound	Upper bound
5	x33	-1.000000e+00	NONE	2.000000e+00	NONE

(continued from previous page)

6	x34	-1.000000e+00	NONE	1.
↪000000e+00				
The following variables are involved in the infeasibility.				
Index	Name	Activity	Objective	Lower bound Upper bound
0	y1	-1.000000e+00	2.000000e+02	NONE 0.
↪000000e+00				
2	y3	-1.000000e+00	1.000000e+03	NONE 0.
↪000000e+00				
3	y4	1.000000e+00	1.100000e+03	NONE NONE
4	y5	1.000000e+00	2.000000e+02	NONE NONE
Interior-point solution summary				
Problem status : DUAL_INFEASIBLE				
Solution status : DUAL_INFEASIBLE_CER				
Primal. obj: 1.0000000000e+02 nrm: 1e+00 Viol. con: 0e+00 var: 0e+00				

In the report we see that the variables y1, y3, y4, y5 and two constraints contribute to infeasibility with non-zero values in the Activity column. Therefore

$$(y_1, \dots, y_7) = (-1, 0, -1, 1, 1, 0, 0)$$

is the dual infeasibility certificate as in [Sec. 12.1.2](#). This just means, that along the ray

$$(0, 0, 0, 0, 0, 0, 0) + t(y_1, \dots, y_7) = (-t, 0, -t, t, t, 0, 0), \quad t > 0,$$

which belongs to the feasible set, the objective value $100t$ can be arbitrarily large, i.e. the problem is unbounded.

In the example problem we could

- Add a lower bound on y3. This will directly invalidate the certificate of dual infeasibility.
- Increase the objective coefficient of y3. Changing the coefficients sufficiently will invalidate the inequality $c^T y^* > 0$ and thus the certificate.

9.3.4 Suggestions

Primal infeasibility

When trying to understand what causes the unexpected primal infeasible status use the following hints:

- Remove the objective function. This does not change the infeasibility status but simplifies the problem, eliminating any possibility of issues related to the objective function.
- Remove cones, semidefinite variables and integer constraints. Solve only the linear part of the problem. Typical simple modeling errors will lead to infeasibility already at this stage.
- Consider whether your problem has some obvious necessary conditions for feasibility and examine if these are satisfied, e.g. total supply should be greater than or equal to total demand.
- Verify that coefficients and bounds are reasonably sized in your problem.
- See if there are any obvious contradictions, for instance a variable is bounded both in the variables and constraints section, and the bounds are contradictory.
- Consider replacing suspicious equality constraints by inequalities. For instance, instead of $x_{12} = 200$ see what happens for $x_{12} \geq 200$ or $x_{12} \leq 200$.
- Relax bounds of the suspicious constraints or variables.

- For integer problems, remove integrality constraints on some/all variables and see if the problem solves.
- Form an **elastic model**: allow to violate constraints at a cost. Introduce slack variables and add them to the objective as penalty. For instance, suppose we have a constraint

$$\begin{array}{ll}\text{minimize} & c^T x, \\ \text{subject to} & a^T x \leq b.\end{array}$$

which might be causing infeasibility. Then create a new variable y and form the problem which contains:

$$\begin{array}{ll}\text{minimize} & c^T x + y, \\ \text{subject to} & a^T x \leq b + y.\end{array}$$

Solving this problem will reveal by how much the constraint needs to be relaxed in order to become feasible. This is equivalent to inspecting the infeasibility certificate but may be more intuitive.

- If you think you have a feasible solution or its part, fix all or some of the variables to those values. Presolve will propagate them through the model and potentially reveal more localized sources of infeasibility.
- Dump the problem in PTF or LP format and verify that the problem that was passed to the optimizer corresponds to the problem expressed in the high-level modeling language, if any such was used.

Dual infeasibility

When trying to understand what causes the unexpected dual infeasible status use the following hints:

- Verify that the objective coefficients are reasonably sized.
- Check if no bounds and constraints are missing, for example if all variables that should be nonnegative have been declared as such etc.
- Strengthen bounds of the suspicious constraints or variables.
- Form an series of models with decreasing bounds on the objective, that is, instead of objective

$$\text{minimize } c^T x$$

solve the problem with an additional constraint such as

$$c^T x = -10^5$$

and inspect the solution to figure out the mechanism behind arbitrarily decreasing objective values. This is equivalent to inspecting the infeasibility certificate but may be more intuitive.

- Dump the problem in PTF or LP format and verify that the problem that was passed to the optimizer corresponds to the problem expressed in the high-level modeling language, if any such was used.

Please note that modifying the problem to invalidate the reported certificate does *not* imply that the problem becomes feasible — the reason for infeasibility may simply *move*, resulting a problem that is still infeasible, but for a different reason. More often, the reported certificate can be used to give a hint about errors or inconsistencies in the model that produced the problem.

9.4 Python Console

The **MOSEK** Python Console is an alternative to the **MOSEK** Command Line Tool. It can be used for interactive loading, solving and debugging optimization problems stored in files, for example **MOSEK** task files. It facilitates debugging techniques described in [Sec. 9](#).

9.4.1 Usage

The tool requires Python 3. The **MOSEK** interface for Python must be installed following the installation instructions for Python API or Python Fusion API. The easiest option is

```
pip install Mosek
```

The Python Console is contained in the file `mosekconsole.py` in the folder with **MOSEK** binaries. It can be copied to an arbitrary location. The file is also available for [download here](#) (`mosekconsole.py`).

To run the console in interactive mode use

```
python mosekconsole.py
```

To run the console in batch mode provide a semicolon-separated list of commands as the second argument of the script, for example:

```
python mosekconsole.py "read data.task.gz; solve form=dual; writesol data"
```

The script is written using the **MOSEK** Python API and can be extended by the user if more specific functionality is required. We refer to the documentation of the Python API.

9.4.2 Examples

To read a problem from `data.task.gz`, solve it, and write solutions to `data.sol`, `data.bas` or `data.itg`:

```
read data.task.gz; solve; writesol data
```

To convert between file formats:

```
read data.task.gz; write data.mps
```

To set a parameter before solving:

```
read data.task.gz; param INTPNT_CO_TOL_DFEAS 1e-9; solve"
```

To list parameter values related to the mixed-integer optimizer in the task file:

```
read data.task.gz; param MIO
```

To print a summary of problem structure:

```
read data.task.gz; anapro
```

To solve a problem forcing the dual and switching off presolve:

```
read data.task.gz; solve form=dual presolve=no
```

To write an infeasible subproblem to a file for debugging purposes:

```
read data.task.gz; solve; infsub; write inf.opf
```

9.4.3 Full list of commands

Below is a brief description of all the available commands. Detailed information about a specific command `cmd` and its options can be obtained with

```
help cmd
```

Table 9.1: List of commands of the MOSEK Python Console.

Command	Description
<code>help [command]</code>	Print list of commands or info about a specific command
<code>log filename</code>	Save the session to a file
<code>intro</code>	Print MOSEK splashscreen
<code>testlic</code>	Test the license system
<code>read filename</code>	Load problem from file
<code>reread</code>	Reload last problem file
<code>solve [options]</code>	Solve current problem
<code>write filename</code>	Write current problem to file
<code>param [name [value]]</code>	Set a parameter or get parameter values
<code>paramdef</code>	Set all parameters to default values
<code>paramdiff</code>	Show parameters with non-default values
<code>info [name]</code>	Get an information item
<code>anapro</code>	Analyze problem data
<code>hist</code>	Plot a histogram of problem data
<code>histsol</code>	Plot a histogram of the solutions
<code>spy</code>	Plot the sparsity pattern of the data matrices
<code>truncate epsilon</code>	Truncate small coefficients down to 0
<code>resobj [fac]</code>	Rescale objective by a factor
<code>anasol</code>	Analyze solutions
<code>removeitg</code>	Remove integrality constraints
<code>removecones</code>	Remove all cones and leave just the linear part
<code>infsub</code>	Replace current problem with its infeasible subproblem
<code>writesol basename</code>	Write solution(s) to file(s) with given basename
<code>del_sol</code>	Remove all solutions from the task
<code>optserver [url]</code>	Use an OptServer to optimize
<code>exit</code>	Leave

Chapter 10

Technical guidelines

This section contains some more in-depth technical guidelines for Fusion API for C++, not strictly necessary for basic use of **MOSEK**.

10.1 Limitations

Fusion imposes some limitations on certain aspects of a model to ensure easier portability:

- Constraints and variables belong to a single model, and cannot as such be used (e.g. stacked) with objects from other models.
- Most objects forming a *Fusion* model are immutable.

The limits on the model size in *Fusion* are as follows:

- The maximum number of variable elements is $2^{31} - 1$.
- The maximum size of a dimension is $2^{31} - 1$.
- The maximum number of structural nonzeros in any single expression object is $2^{31} - 1$.
- The total size of an item (the product of dimensions) is limited to $2^{63} - 1$.

10.2 Memory management and garbage collection

Users who experience memory leaks using *Fusion*, especially:

- memory usage not decreasing after the solver terminates,
- memory usage increasing when solving a sequence of problems,

should make sure that the *Model* objects are properly garbage collected. Since each *Model* object links to a **MOSEK** task resource in a linked library, it is sometimes the case that the garbage collector is unable to reclaim it automatically. This means that substantial amounts of memory may be leaked. For this reason it is very important to make sure that the *Model* object is disposed of manually when it is not used any more. The necessary cleanup is performed by the method *Model.dispose*.

```
{
    Model::t M = new Model();    auto _M = finally([&]() { M->dispose(); });
    // do something with M
}
```

This construction assures that the *Model.dispose* method is called when the object goes out of scope, even if an exception occurred. If this approach cannot be used, e.g. if the *Model* object is returned by a factory function, one should explicitly call the *Model.dispose* method when the object is no longer used.

Furthermore, if the *Model* class is extended, it is necessary to dispose of the superclass if the initialization of the derived subclass fails. One can use a construction such as:

```

class MyModel: Model
{
public:
MyModel(): Model()
{
    bool finished = false;
    try
    {
        //perform initialization here
        finished = true;
    }
    catch(...)
    {
        dispose();
    }
}
};

```

10.3 Names

All elements of an optimization problem in **MOSEK** (objective, constraints, variables, etc.) can be given names. Assigning meaningful names to variables and constraints makes it much easier to understand and debug optimization problems dumped to a file. On the other hand, note that assigning names can substantially increase setup time, so it should be avoided in time-critical applications.

The *Model* object's, variables' and constraints' constructors provide versions with a string name as an optional parameter.

Names introduced in *Fusion* are transformed into names in the underlying low-level optimization task, which in turn can be saved to a file. In particular:

- a scalar variable with name `var` becomes a variable with name `var[]`,
- a one- or more-dimensional variable with name `var` becomes a sequence of scalar variables with names `var[0]`, `var[1]`, etc. or `var[0,0]`, `var[0,1]`, etc., depending on the shape,
- the same applies to constraints,
- a new variable with name 1.0 may be added.

These are the guidelines. No guarantees are made for the exact form of this transformation.

The user can override the default numbering scheme by providing a list of string labels for some or all axes. For example the following code

```

auto itemNames = new_array_ptr<std::string,1>({"ITEM1", "ITEM2", "ITEM3"});
auto slotNames = new_array_ptr<std::string,1>({"SLOT1", "SLOT2"});

auto x = M->variable("price", new_array_ptr<int,1>({ 3, 2 }),
                    Domain::unbounded()->withNamesOnAxis(itemNames, 0)
                    ->withNamesOnAxis(slotNames, 1));

```

will lead to the individual entries of variable `price` being named as `price[ITEM1,SLOT1]`, `price[ITEM1,SLOT2]` and so on instead of `price[0,0]`, `price[0,1]` etc.

Note that file formats impose various restrictions on names, so not all resulting names can be written verbatim to each type of file, and when writing to a file further transformations and character substitutions can be applied, resulting in poor readability. This is particularly true for LP files, so saving *Fusion* problems in LP format is discouraged. The PTF format is recommended instead. See [Sec. 15](#).

10.4 Multithreading

Thread safety

Sharing a *Model* object between threads is safe, as long as it is not accessed from more than one thread at a time. Multiple *Model* objects can be used in parallel without any problems.

Parallelization

The interior-point and mixed-integer optimizers in **MOSEK** are parallelized. By default **MOSEK** will automatically select the number of threads. However, the maximum number of threads allowed can be changed by setting the parameter *numThreads* and related parameters. This should never exceed the number of cores.

The speed-up obtained when using multiple threads is highly problem and hardware dependent. We recommend experimenting with various thread numbers to determine the optimal settings. For small problems using multiple threads may be counter-productive because of the associated overhead. Note also that not all parts of the algorithm can be parallelized, so there are times when CPU utilization is only 1 even if more cores are available.

Determinism

By default the optimizer is run-to-run deterministic, which means that it will return the same answer each time it is run on the same machine with the same input, the same parameter settings (including number of threads) and no time limits.

Setting the number of threads

The number of threads the optimizer uses can be changed with the parameter *numThreads*.

10.5 Efficiency

The following guidelines can help keep the code as efficient as possible.

Decide between sparse and dense matrices

Deciding whether a matrix should be stored in dense or sparse format is not always trivial. First, there are storage considerations. An $n \times m$ matrix with l non zero entries, requires

- $\approx n \cdot m$ storage space in dense format,
- $\approx 3 \cdot l$ storage space in sparse (triplet) format.

Therefore if $l \ll n \cdot m$, then the sparse format has smaller memory requirements. Especially for very sparse density matrices it will also yield much faster expression transformations. Also, this is the format used ultimately by the underlying optimizer task. However, there are borderline cases in which these advantages may vanish due to overhead spent creating the triplet representation.

Sparsity is a key feature of many optimization models and often occurs naturally. For instance, linear constraints arising from networks or multi-period planning are typically sparse. *Fusion* does not detect sparsity but leaves to the user the responsibility of choosing the most appropriate storage format.

Reduce the number of *Fusion* calls and level of nesting

A possible source of performance degradation is an excessive use of nested expressions resulting in a large number of *Fusion* calls with small model updates, where instead the model could be updated in larger chunks at once. In general, loop-free code and reduction of expression nesting are likely to be more efficient. For example the expression

$$\sum_{i=1}^n A_i x_i$$
$$x_i \in \mathbb{R}^k, A_i \in \mathbb{R}^{k \times k},$$

could be implemented in a loop as

```
Expression::t ee = Expr::constTerm(k, 0.0);
for(int i=0;i<n;i++)
    ee = Expr::add( ee, Expr::mul((*A)[i],(*x)[i]) );
```

A better way is to store the intermediate expressions for $A_i x_i$ and sum all of them in one step:

```
auto prods = new ndarray<Expression::t,1>(shape(n));
for(int i=0;i<n;i++) (*prods)[i] = Expr::mul((*A)[i],(*x)[i]);
Expression::t ee = Expr::add( std::shared_ptr<ndarray<Expression::t,1>>(prods) );
```

Fusion design naturally promotes this sort of vectorized implementations. See [Sec. 6.8](#) for more examples.

Parametrize relevant parts of the model

If you intend to reoptimize the same model with changing input data, use a parametrized model and modify it between optimizations by resetting parameter values, see [Sec. 6.6](#). This way the model is constructed only once, and only a few coefficients need to be recomputed each time.

Keep a healthy balance and parametrize only the part of the model you in fact intend to change. For example, using parameters in place of all constants appearing in the model would be an overkill with an adverse effect on efficiency since all coefficients in the problem would still have to be recomputed each time.

Do not fetch the whole solution if not necessary

Fetching a solution from a shaped variable produces a flat array of values. This means that some reshaping has to take place and that the user gets all values even if they are potentially interested only in some of them. In this case, it is better to create a slice variable holding the relevant elements and fetch the solution for this subset. See [Sec. 6.7](#). Fetching the full solution may cause an exception due to memory exhaustion or platform-dependent constraints on array sizes.

Remove names

Variables, constraints and the objective function can be constructed with user-assigned names. While this feature is very useful for debugging and improves the readability of both the code and of problems dumped to files, it also introduces quite some overhead: *Fusion* must check and make sure that names are unique. For optimal performance it is therefore recommended to not specify names at all.

10.6 The license system

MOSEK is a commercial product that **always** needs a valid license to work. **MOSEK** uses a third party license manager to implement license checking. The number of license tokens provided determines the number of optimizations that can be run simultaneously.

By default a license token remains checked out from the first optimization until the end of the **MOSEK** session, i.e.

- a license token is checked out when the method *Model.solve* is called the first time, and
- the token is returned when the process exits.

Starting the optimization when no license tokens are available will result in an error.

Default behaviour of the license system can be changed in several ways:

- Setting the parameter *cacheLicense* to "off" will force **MOSEK** to return the license token immediately after the optimization completed.
- Setting the license wait flag with *Model.putLicenseWait* or with the parameter *licenseWait* will force **MOSEK** to wait until a license token becomes available instead of throwing an exception.
- The default path to the license file can be changed with *Model.putLicensePath*.

10.7 Deployment

When redistributing a C++ application using the **MOSEK** Fusion API for C++ 10.0.44, the following shared libraries from the **MOSEK** bin folder are required:

- Linux : libmosek64, libfusion64, libtbb,
- Windows : mosek64, fusion64, tbb, svml_dispmd,
- OSX : libmosek64, libfusion64, libtbb.

Chapter 11

Case Studies

In this section we present some case studies in which the Fusion API for C++ is used to solve real-life applications. These examples involve some more advanced modeling skills and possibly some input data. The user is strongly recommended to first read the basic tutorials of [Sec. 7](#) before going through these advanced case studies.

- *Portfolio Optimization*
 - **Keywords:** Markowitz model, variance, risk, efficient frontier, factor model, transaction cost, market impact cost, cardinality constraints
 - **Type:** Conic Quadratic, Power Cone, Mixed-Integer, Model parametrization
- *Primal SVM*
 - **Keywords:** machine learning, Support-Vector Machine, hyperplane separation, classifier
 - **Type:** Conic Quadratic
- *2D Total Variation*
 - **Keywords:** denoising, total variation
 - **Type:** Conic Quadratic, Model parametrization
- *Multi Processor Scheduling*
 - **Keywords:** scheduling, job allocation, feasible point heuristic
 - **Type:** Mixed-Integer, Linear Optimization
- *Logistic regression*
 - **Keywords:** machine learning, logistic regression, classifier, log-sum-exp, softplus, regularization
 - **Type:** Exponential Cone, Quadratic Cone
- *Inner and outer Löwner-John Ellipsoids*
 - **Keywords:** volume optimization, ellipsoidal approximation, determinant, geometric mean, eigenvalues
 - **Type:** Geometric Mean Cone, Semidefinite
- *SUDOKU*
 - **Keywords:** combinatorial puzzle, binary variables, integer modeling
 - **Type:** Integer Optimization, Linear Optimization
- *Travelling Salesman*
 - **Keywords:** TSP, cycle elimination
 - **Type:** Mixed-Integer, Linear Optimization

- *Nearest Correlation Matrix Problem*
 - **Keywords:** low-rank matrix approximation, trace, Frobenius norm, correlation matrix
 - **Type:** Semidefinite
- *Semidefinite relaxation of MIQCQO problems*
 - **Keywords:** integer quadratic problems, semidefinite relaxation, approximation, integer least squares
 - **Type:** Semidefinite, Mixed-Integer Conic Quadratic

11.1 Portfolio Optimization

In this section the Markowitz portfolio optimization problem and variants are implemented using Fusion API for C++.

- *Basic Markowitz model*
- *Efficient frontier*
- *Factor model and efficiency*
- *Market impact costs*
- *Transaction costs*
- *Cardinality constraints*

11.1.1 The Basic Model

The classical Markowitz portfolio optimization problem considers investing in n stocks or assets held over a period of time. Let x_j denote the amount invested in asset j , and assume a stochastic model where the return of the assets is a random variable r with known mean

$$\mu = \mathbf{E}r$$

and covariance

$$\Sigma = \mathbf{E}(r - \mu)(r - \mu)^T.$$

The return of the investment is also a random variable $y = r^T x$ with mean (or expected return)

$$\mathbf{E}y = \mu^T x$$

and variance

$$\mathbf{E}(y - \mathbf{E}y)^2 = x^T \Sigma x.$$

The standard deviation

$$\sqrt{x^T \Sigma x}$$

is usually associated with risk.

The problem facing the investor is to rebalance the portfolio to achieve a good compromise between risk and expected return, e.g., maximize the expected return subject to a budget constraint and an upper bound (denoted γ) on the tolerable risk. This leads to the optimization problem

$$\begin{aligned} & \text{maximize} && \mu^T x \\ & \text{subject to} && e^T x = w + e^T x^0, \\ & && x^T \Sigma x \leq \gamma^2, \\ & && x \geq 0. \end{aligned} \tag{11.1}$$

The variables x denote the investment i.e. x_j is the amount invested in asset j and x_j^0 is the initial holding of asset j . Finally, w is the initial amount of cash available.

A popular choice is $x^0 = 0$ and $w = 1$ because then x_j may be interpreted as the relative amount of the total portfolio that is invested in asset j .

Since e is the vector of all ones then

$$e^T x = \sum_{j=1}^n x_j$$

is the total investment. Clearly, the total amount invested must be equal to the initial wealth, which is

$$w + e^T x^0.$$

This leads to the first constraint

$$e^T x = w + e^T x^0.$$

The second constraint

$$x^T \Sigma x \leq \gamma^2$$

ensures that the variance, is bounded by the parameter γ^2 . Therefore, γ specifies an upper bound of the standard deviation (risk) the investor is willing to undertake. Finally, the constraint

$$x_j \geq 0$$

excludes the possibility of short-selling. This constraint can of course be excluded if short-selling is allowed.

The covariance matrix Σ is positive semidefinite by definition and therefore there exist a matrix $G \in \mathbb{R}^{n \times k}$ such that

$$\Sigma = GG^T. \tag{11.2}$$

In general the choice of G is **not** unique and one possible choice of G is the Cholesky factorization of Σ . However, in many cases another choice is better for efficiency reasons as discussed in [Sec. 11.1.3](#). For a given G we have that

$$\begin{aligned} x^T \Sigma x &= x^T G G^T x \\ &= \|G^T x\|^2. \end{aligned}$$

Hence, we may write the risk constraint as

$$\gamma \geq \|G^T x\|$$

or equivalently

$$(\gamma, G^T x) \in \mathcal{Q}^{k+1},$$

where $\mathcal{Q}^{k+1}$ is the $(k+1)$ -dimensional quadratic cone. Note that specifically when G is derived using Cholesky factorization, $k = n$.

Therefore, problem (11.1) can be written as

$$\begin{aligned} & \text{maximize} && \mu^T x \\ & \text{subject to} && e^T x = w + e^T x^0, \\ & && (\gamma, G^T x) \in \mathcal{Q}^{k+1}, \\ & && x \geq 0, \end{aligned} \tag{11.3}$$

which is a conic quadratic optimization problem that can easily be formulated and solved with Fusion API for C++. Subsequently we will use the example data

$$\mu = [0.0720, 0.1552, 0.1754, 0.0898, 0.4290, 0.3929, 0.3217, 0.1838]^T$$

and

$$\Sigma = \begin{bmatrix} 0.0946 & 0.0374 & 0.0349 & 0.0348 & 0.0542 & 0.0368 & 0.0321 & 0.0327 \\ 0.0374 & 0.0775 & 0.0387 & 0.0367 & 0.0382 & 0.0363 & 0.0356 & 0.0342 \\ 0.0349 & 0.0387 & 0.0624 & 0.0336 & 0.0395 & 0.0369 & 0.0338 & 0.0243 \\ 0.0348 & 0.0367 & 0.0336 & 0.0682 & 0.0402 & 0.0335 & 0.0436 & 0.0371 \\ 0.0542 & 0.0382 & 0.0395 & 0.0402 & 0.1724 & 0.0789 & 0.0700 & 0.0501 \\ 0.0368 & 0.0363 & 0.0369 & 0.0335 & 0.0789 & 0.0909 & 0.0536 & 0.0449 \\ 0.0321 & 0.0356 & 0.0338 & 0.0436 & 0.0700 & 0.0536 & 0.0965 & 0.0442 \\ 0.0327 & 0.0342 & 0.0243 & 0.0371 & 0.0501 & 0.0449 & 0.0442 & 0.0816 \end{bmatrix}.$$

Using Cholesky factorization, this implies

$$G^T = \begin{bmatrix} 0.3076 & 0.1215 & 0.1134 & 0.1133 & 0.1763 & 0.1197 & 0.1044 & 0.1064 \\ 0. & 0.2504 & 0.0995 & 0.0916 & 0.0669 & 0.0871 & 0.0917 & 0.0851 \\ 0. & 0. & 0.1991 & 0.0587 & 0.0645 & 0.0737 & 0.0647 & 0.0191 \\ 0. & 0. & 0. & 0.2088 & 0.0493 & 0.0365 & 0.0938 & 0.0774 \\ 0. & 0. & 0. & 0. & 0.3609 & 0.1257 & 0.1016 & 0.0571 \\ 0. & 0. & 0. & 0. & 0. & 0.2155 & 0.0566 & 0.0619 \\ 0. & 0. & 0. & 0. & 0. & 0. & 0.2251 & 0.0333 \\ 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0.2202 \end{bmatrix}$$

In Sec. 11.1.3, we present a different way of obtaining G based on a factor model, that leads to more efficient computation.

Why a Conic Formulation?

Problem (11.1) is a convex quadratically constrained optimization problem that can be solved directly using **MOSEK**. Why then reformulate it as a conic quadratic optimization problem (11.3)? The main reason for choosing a conic model is that it is more robust and usually solves faster and more reliably. For instance it is not always easy to numerically validate that the matrix Σ in (11.1) is positive semidefinite due to the presence of rounding errors. It is also very easy to make a mistake so Σ becomes indefinite. These problems are completely eliminated in the conic formulation.

Moreover, observe the constraint

$$\|G^T x\| \leq \gamma$$

more numerically robust than

$$x^T \Sigma x \leq \gamma^2$$

for very small and very large values of γ . Indeed, if say $\gamma \approx 10^4$ then $\gamma^2 \approx 10^8$, which introduces a scaling issue in the model. Hence, using conic formulation we work with the standard deviation instead of variance, which usually gives rise to a better scaled model.

Example code

Listing 11.1 demonstrates how the basic Markowitz model (11.3) is implemented.

Listing 11.1: Code implementing problem (11.3).

```
double BasicMarkowitz
( int                n,
  std::shared_ptr<ndarray<double, 1>> mu,
  std::shared_ptr<ndarray<double, 2>> GT,
  std::shared_ptr<ndarray<double, 1>> x0,
  double             w,
  double             gamma)
{
  Model::t M = new Model("Basic Markowitz"); auto _M = finally([&]() { M->dispose(); }
  ↪);
  // Redirect log output from the solver to stdout for debugging.
  // M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }
  ↪);

  // Defines the variables (holdings). Shortselling is not allowed.
  Variable::t x = M->variable("x", n, Domain::greaterThan(0.0));

  // Maximize expected return
  M->objective("obj", ObjectiveSense::Maximize, Expr::dot(mu, x));

  // The amount invested must be identical to initial wealth
  M->constraint("budget", Expr::sum(x), Domain::equalsTo(w + sum(x0)));

  // Imposes a bound on the risk
  M->constraint("risk", Expr::vstack(gamma, Expr::mul(GT, x)), Domain::inQCone());

  // Solves the model.
  M->solve();

  return dot(mu, x->level());
}
```

The source code should be self-explanatory except perhaps for

```
M->constraint("risk", Expr::vstack(gamma, Expr::mul(GT, x)), Domain::inQCone());
```

where the linear expression

$$(\gamma, G^T x)$$

is created using the `Expr.vstack` operator. Finally, the linear expression must lie in a quadratic cone implying

$$\gamma \geq \|G^T x\|.$$

11.1.2 The Efficient Frontier

The portfolio computed by the Markowitz model is efficient in the sense that there is no other portfolio giving a strictly higher return for the same amount of risk. An efficient portfolio is also sometimes called a Pareto optimal portfolio. Clearly, an investor should only invest in efficient portfolios and therefore it may be relevant to present the investor with all efficient portfolios so the investor can choose the portfolio that has the desired tradeoff between return and risk.

Given a nonnegative α the problem

$$\begin{aligned} & \text{maximize} && \mu^T x - \alpha x^T \Sigma x \\ & \text{subject to} && e^T x = w + e^T x^0, \\ & && x \geq 0. \end{aligned} \tag{11.4}$$

is one standard way to trade the expected return against penalizing variance. Note that, in contrast to the previous example, we explicitly use the variance ($\|G^T x\|_2^2$) rather than standard deviation ($\|G^T x\|_2$), therefore the conic model includes a rotated quadratic cone:

$$\begin{aligned} & \text{maximize} && \mu^T x - \alpha s \\ & \text{subject to} && e^T x = w + e^T x^0, \\ & && (s, 0.5, G^T x) \in Q_r^{k+2} \quad (\text{equiv. to } s \geq \|G^T x\|_2^2 = x^T \Sigma x), \\ & && x \geq 0. \end{aligned} \tag{11.5}$$

The parameter α specifies the tradeoff between expected return and variance. Ideally the problem (11.4) should be solved for all values $\alpha \geq 0$ but in practice it is impossible. Using the example data from Sec. 11.1.1, the optimal values of return and variance for several values of α are shown in the figure.

Example code

Listing 11.2 demonstrates how to compute the efficient portfolios for several values of α .

Listing 11.2: Code for the computation of the efficient frontier based on problem (11.4).

```
void EfficientFrontier
( int n,
  std::shared_ptr<ndarray<double, 1>> mu,
  std::shared_ptr<ndarray<double, 2>> GT,
  std::shared_ptr<ndarray<double, 1>> x0,
  double w,
  std::vector<double> & alphas,
  std::vector<double> & frontier_mux,
  std::vector<double> & frontier_s)
{
    Model::t M = new Model("Efficient frontier"); auto M_ = finally([&]() { M->
    dispose(); });

    // Defines the variables (holdings). Shortselling is not allowed.
    Variable::t x = M->variable("x", n, Domain::greaterThan(0.0)); // Portfolio
    variables
    Variable::t s = M->variable("s", 1, Domain::unbounded()); // Variance variable

    M->constraint("budget", Expr::sum(x), Domain::equalsTo(w + sum(x0)));

    // Computes the risk
    M->constraint("variance", Expr::vstack(s, 0.5, Expr::mul(GT, x)), Domain::
    inRotatedQCone());

    // Define objective as a weighted combination of return and variance
    Parameter::t alpha = M->parameter();
```

(continues on next page)

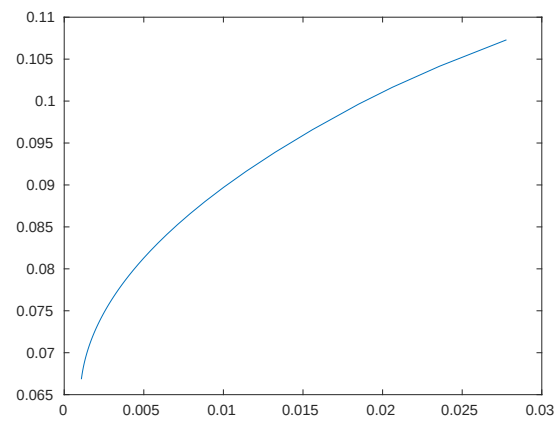


Fig. 11.1: The efficient frontier for the sample data.


```

M->objective("obj", ObjectiveSense::Maximize, Expr::sub(Expr::dot(mu, x), Expr::
↪mul(alpha, s)));

// Solve the same problem for many values of parameter alpha
for (double a : alphas) {
    alpha->setValue(a);
    M->solve();

    frontier_mux.push_back(dot(mu, x->level()));
    frontier_s.push_back((*s->level())[0]);
}
}

```

Note that we defined α as a model parameter and used it to parametrize the objective. This way we were able to reuse the same model for all solves along the efficient frontier, simply changing the value of α between the solves.

11.1.3 Factor model and efficiency

In practice it is often important to solve the portfolio problem very quickly. Therefore, in this section we discuss how to improve computational efficiency at the modeling stage.

The computational cost is of course to some extent dependent on the number of constraints and variables in the optimization problem. However, in practice a more important factor is the sparsity: the number of nonzeros used to represent the problem. Indeed it is often better to focus on the number of nonzeros in G see (11.2) and try to reduce that number by for instance changing the choice of G .

In other words if the computational efficiency should be improved then it is always good idea to start with focusing at the covariance matrix. As an example assume that

$$\Sigma = D + VV^T$$

where D is a positive definite diagonal matrix. Moreover, V is a matrix with n rows and k columns. Such a model for the covariance matrix is called a factor model and usually k is much smaller than n . In practice k tends to be a small number independent of n , say less than 100.

One possible choice for G is the Cholesky factorization of Σ which requires storage proportional to $n(n+1)/2$. However, another choice is

$$G = \begin{bmatrix} D^{1/2} & V \end{bmatrix}$$

because then

$$GG^T = D + VV^T.$$

This choice requires storage proportional to $n + kn$ which is much less than for the Cholesky choice of G . Indeed assuming k is a constant storage requirements are reduced by a factor of n .

The example above exploits the so-called factor structure and demonstrates that an alternative choice of G may lead to a significant reduction in the amount of storage used to represent the problem. This will in most cases also lead to a significant reduction in the solution time.

The lesson to be learned is that it is important to investigate how the covariance matrix is formed. Given this knowledge it might be possible to make a special choice for G that helps reducing the storage requirements and enhance the computational efficiency. More details about this process can be found in [And13].

Factor model in finance

Factor model structure is typical in financial context. It is common to model security returns as the sum of two components using a factor model. The first component is the linear combination of a small number of factors common among a group of securities. The second component is a residual, specific to each security. It can be written as $R = \sum_j \beta_j F_j + \theta$, where R is a random variable representing the return of a security at a particular point in time, F_j is the random variable representing the common factor j , β_j is the exposure of the return to factor j , and θ is the specific component.

Such a model will result in the covariance structure

$$\Sigma = \Sigma_\theta + \beta \Sigma_F \beta^T,$$

where Σ_F is the covariance of the factors and Σ_θ is the residual covariance. This structure is of the form discussed earlier with $D = \Sigma_\theta$ and $V = \beta P$, assuming the decomposition $\Sigma_F = PP^T$. If the number of factors k is low and Σ_θ is diagonal, we get a very sparse G that provides the storage and solution time benefits.

Example code

Here we will work with the example data of a two-factor model ($k = 2$) built using the variables

$$\beta = \begin{bmatrix} 0.4256 & 0.1869 \\ 0.2413 & 0.3877 \\ 0.2235 & 0.3697 \\ 0.1503 & 0.4612 \\ 1.5325 & -0.2633 \\ 1.2741 & -0.2613 \\ 0.6939 & 0.2372 \\ 0.5425 & 0.2116 \end{bmatrix},$$

$$\theta = [0.0720, 0.0508, 0.0377, 0.0394, 0.0663, 0.0224, 0.0417, 0.0459],$$

and the factor covariance matrix is

$$\Sigma_F = \begin{bmatrix} 0.0620 & 0.0577 \\ 0.0577 & 0.0908 \end{bmatrix},$$

giving

$$P = \begin{bmatrix} 0.2491 & 0. \\ 0.2316 & 0.1928 \end{bmatrix}.$$

Then the matrix G would look like

$$G = \begin{bmatrix} \beta P & \Sigma_\theta^{1/2} \end{bmatrix} = \begin{bmatrix} 0.1493 & 0.0360 & 0.2683 & 0. & 0. & 0. & 0. & 0. & 0. & 0. \\ 0.1499 & 0.0747 & 0. & 0.2254 & 0. & 0. & 0. & 0. & 0. & 0. \\ 0.1413 & 0.0713 & 0. & 0. & 0.1942 & 0. & 0. & 0. & 0. & 0. \\ 0.1442 & 0.0889 & 0. & 0. & 0. & 0.1985 & 0. & 0. & 0. & 0. \\ 0.3207 & -0.0508 & 0. & 0. & 0. & 0. & 0.2576 & 0. & 0. & 0. \\ 0.2568 & -0.0504 & 0. & 0. & 0. & 0. & 0. & 0.1497 & 0. & 0. \\ 0.2277 & 0.0457 & 0. & 0. & 0. & 0. & 0. & 0. & 0.2042 & 0. \\ 0.1841 & 0.0408 & 0. & 0. & 0. & 0. & 0. & 0. & 0. & 0.2142 \end{bmatrix}.$$

This matrix is indeed very sparse.

In general, we get an $n \times (n + k)$ size matrix this way with k full columns and an $n \times n$ diagonal part. In order to maintain a sparse representation we do not construct the matrix G explicitly in the code but instead work with two pieces of data: the dense matrix $G_{\text{factor}} = \beta P$ of shape $n \times k$ and the diagonal vector θ of length n .

Example code

In the following we demonstrate how to write code to compute the matrix G_{factor} of the factor model. We start with the inputs

Listing 11.3: Inputs for the computation of the matrix G_{factor} from the factor model.

```
// Factor exposure matrix
auto B = new_array_ptr<double, 2>({
    {0.4256, 0.1869},
    {0.2413, 0.3877},
    {0.2235, 0.3697},
    {0.1503, 0.4612},
    {1.5325, -0.2633},
    {1.2741, -0.2613},
    {0.6939, 0.2372},
    {0.5425, 0.2116}
});

// Factor covariance matrix
auto S_F = new_array_ptr<double, 2>({
    {0.0620, 0.0577},
    {0.0577, 0.0908}
});

// Specific risk components
auto theta = new_array_ptr<double, 1>({0.0720, 0.0508, 0.0377, 0.0394, 0.0663, 0.
↪0224, 0.0417, 0.0459});
```

Then the matrix G_{factor} is obtained as:

```
auto P = cholesky(S_F);
auto G_factor = matrix_mul(B, P);
auto G_factor_T = transpose(G_factor);
```

The functions used above to operate on matrices are defined in the source file that can be downloaded from [Listing 11.3](#).

The code for computing an optimal portfolio in the factor model is very similar to the one from the basic model in [Listing 11.1](#) with one notable exception: we construct the expression $G^T x$ appearing in the conic constraint by stacking together two separate vectors $G_{\text{factor}}^T x$ and $\Sigma_{\theta}^{1/2} x$:

```
// Imposes a bound on the risk
M->constraint("risk", Expr::vstack(new_array_ptr<Expression::t, 1>({Expr::
↪constTerm(gamma),
Expr::mul(G_
↪factor_T, x),
Expr::
↪mulElm(vector_sqrt(theta), x)})), Domain::inQCone());
```

The full code is demonstrated below:

Listing 11.4: Implementation of portfolio optimization in the factor model.

```
double FactorMarkowitz(
    int n,
    farray<double, 1> mu,
    farray<double, 2> G_factor_T,
    farray<double, 1> theta,
```

(continues on next page)

```

farray<double, 1>  x0,
double            w,
double            gamma
)
{
  Model::t M = new Model("Factor Markowitz"); auto _M = finally([&]() { M->dispose();
  ↪});
  // Redirect log output from the solver to stdout for debugging.
  // M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }
  ↪);

  // Defines the variables (holdings). Shortselling is not allowed.
  Variable::t x = M->variable("x", n, Domain::greaterThan(0.0));

  // Maximize expected return
  M->objective("obj", ObjectiveSense::Maximize, Expr::dot(mu, x));

  // The amount invested must be identical to initial wealth
  M->constraint("budget", Expr::sum(x), Domain::equalsTo(w + sum(x0)));

  // Imposes a bound on the risk
  M->constraint("risk", Expr::vstack(new_array_ptr<Expression::t, 1>({Expr::
  ↪constTerm(gamma),
  ↪factor_T, x),
  ↪mulElm(vector_sqrt(theta), x)})), Domain::inQCone());

  // Solves the model.
  M->solve();

  return dot(mu, x->level());
}

```

11.1.4 Slippage Cost

The basic Markowitz model assumes that there are no costs associated with trading the assets and that the returns of the assets are independent of the amount traded. Neither of those assumptions is usually valid in practice. Therefore, a more realistic model is

$$\begin{aligned}
 & \text{maximize} && \mu^T x \\
 & \text{subject to} && e^T x + \sum_{j=1}^n T_j(\Delta x_j) = w + e^T x^0, \\
 & && x^T \Sigma x \leq \gamma^2, \\
 & && x \geq 0.
 \end{aligned} \tag{11.6}$$

Here Δx_j is the change in the holding of asset j i.e.

$$\Delta x_j = x_j - x_j^0$$

and $T_j(\Delta x_j)$ specifies the transaction costs when the holding of asset j is changed from its initial value. In the next two sections we show two different variants of this problem with two nonlinear cost functions T .

11.1.5 Market Impact Costs

If the initial wealth is fairly small and no short selling is allowed, then the holdings will be small and the traded amount of each asset must also be small. Therefore, it is reasonable to assume that the prices of the assets are independent of the amount traded. However, if a large volume of an asset is sold or purchased, the price, and hence return, can be expected to change. This effect is called market impact costs. It is common to assume that the market impact cost for asset j can be modeled by

$$T_j(\Delta x_j) = m_j |\Delta x_j|^{3/2}$$

where m_j is a constant that is estimated in some way by the trader. See [GK00] [p. 452] for details. From the [Modeling Cookbook](#) we know that $t \geq |z|^{3/2}$ can be modeled directly using the power cone $\mathcal{P}_3^{2/3, 1/3}$:

$$\{(t, z) : t \geq |z|^{3/2}\} = \{(t, z) : (t, 1, z) \in \mathcal{P}_3^{2/3, 1/3}\}$$

Hence, it follows that $\sum_{j=1}^n T_j(\Delta x_j) = \sum_{j=1}^n m_j |x_j - x_j^0|^{3/2}$ can be modeled by $\sum_{j=1}^n m_j t_j$ under the constraints

$$\begin{aligned} z_j &= |x_j - x_j^0|, \\ (t_j, 1, z_j) &\in \mathcal{P}_3^{2/3, 1/3}. \end{aligned}$$

Unfortunately this set of constraints is nonconvex due to the constraint

$$z_j = |x_j - x_j^0| \tag{11.7}$$

but in many cases the constraint may be replaced by the relaxed constraint

$$z_j \geq |x_j - x_j^0|, \tag{11.8}$$

For instance if the universe of assets contains a risk free asset then

$$z_j > |x_j - x_j^0| \tag{11.9}$$

cannot hold for an optimal solution.

If the optimal solution has the property (11.9) then the market impact cost within the model is larger than the true market impact cost and hence money are essentially considered garbage and removed by generating transaction costs. This may happen if a portfolio with very small risk is requested because the only way to obtain a small risk is to get rid of some of the assets by generating transaction costs. We generally assume that this is not the case and hence the models (11.7) and (11.8) are equivalent.

The above observations lead to

$$\begin{aligned} &\text{maximize} && \mu^T x \\ &\text{subject to} && e^T x + m^T t = w + e^T x^0, \\ & && (\gamma, G^T x) \in \mathcal{Q}^{k+1}, \\ & && (t_j, 1, x_j - x_j^0) \in \mathcal{P}_3^{2/3, 1/3}, \quad j = 1, \dots, n, \\ & && x \geq 0. \end{aligned} \tag{11.10}$$

The revised budget constraint

$$e^T x + m^T t = w + e^T x^0$$

specifies that the initial wealth covers the investment and the transaction costs. It should be mentioned that transaction costs of the form

$$t_j \geq |z_j|^p$$

where $p > 1$ is a real number can be modeled with the power cone as

$$(t_j, 1, z_j) \in \mathcal{P}_3^{1/p, 1-1/p}.$$

See the [Modeling Cookbook](#) for details.

Example code

Listing 11.5 demonstrates how to compute an optimal portfolio when market impact cost are included.

Listing 11.5: Implementation of model (11.10).

```
void MarkowitzWithMarketImpact
( int n,
  std::shared_ptr<ndarray<double, 1>> mu,
  std::shared_ptr<ndarray<double, 2>> GT,
  std::shared_ptr<ndarray<double, 1>> x0,
  double w,
  double gamma,
  std::shared_ptr<ndarray<double, 1>> m,
  std::vector<double> & xsol,
  std::vector<double> & tsol)
{
  Model::t M = new Model("Markowitz portfolio with market impact"); auto M_ =
  ↪finally([&]() { M->dispose(); });

  // Defines the variables. No shortselling is allowed.
  Variable::t x = M->variable("x", n, Domain::greaterThan(0.0));

  // Variables computing the impact cost
  Variable::t t = M->variable("t", n, Domain::unbounded());

  // Maximize expected return
  M->objective("obj", ObjectiveSense::Maximize, Expr::dot(mu, x));

  // Invested amount + slippage cost = initial wealth
  M->constraint("budget", Expr::add(Expr::sum(x), Expr::dot(m, t)), Domain::
  ↪equalsTo(w + sum(x0)));

  // Imposes a bound on the risk
  M->constraint("risk", Expr::vstack( gamma, Expr::mul(GT, x)),
              Domain::inQCone());

  // t >= |x-x0|1.5, using a power cone
  M->constraint("tz", Expr::hstack(t, Expr::constTerm(n, 1.0), Expr::sub(x, x0)),
  ↪Domain::inPPowerCone(2.0/3.0));

  M->solve();

  xsol.resize(n);
  tsol.resize(n);
  auto xlvl = x->level();
  auto tlv1 = t->level();

  std::copy(xlvl->flat_begin(), xlvl->flat_end(), xsol.begin());
  std::copy(tlv1->flat_begin(), tlv1->flat_end(), tsol.begin());
}
```

11.1.6 Transaction Costs

Now assume there is a cost associated with trading asset j given by

$$T_j(\Delta x_j) = \begin{cases} 0, & \Delta x_j = 0, \\ f_j + g_j |\Delta x_j|, & \text{otherwise.} \end{cases}$$

Hence, whenever asset j is traded we pay a fixed setup cost f_j and a variable cost of g_j per unit traded. Given the assumptions about transaction costs in this section problem (11.6) may be formulated as

$$\begin{aligned} & \text{maximize} && \mu^T x \\ & \text{subject to} && e^T x + f^T y + g^T z = w + e^T x^0, \\ & && (\gamma, G^T x) \in \mathcal{Q}^{k+1}, \\ & && z_j \geq x_j - x_j^0, \quad j = 1, \dots, n, \\ & && z_j \geq x_j^0 - x_j, \quad j = 1, \dots, n, \\ & && z_j \leq U_j y_j, \quad j = 1, \dots, n, \\ & && y_j \in \{0, 1\}, \quad j = 1, \dots, n, \\ & && x \geq 0. \end{aligned} \tag{11.11}$$

First observe that

$$z_j \geq |x_j - x_j^0| = |\Delta x_j|.$$

We choose U_j as some a priori upper bound on the amount of trading in asset j and therefore if $z_j > 0$ then $y_j = 1$ has to be the case. This implies that the transaction cost for asset j is given by

$$f_j y_j + g_j z_j.$$

Example code

The following example code demonstrates how to compute an optimal portfolio when transaction costs are included.

Listing 11.6: Code solving problem (11.11).

```
std::shared_ptr<ndarray<double, 1>> MarkowitzWithTransactionsCost
( int n,
  std::shared_ptr<ndarray<double, 1>> mu,
  std::shared_ptr<ndarray<double, 2>> GT,
  std::shared_ptr<ndarray<double, 1>> x0,
  double w,
  double gamma,
  std::shared_ptr<ndarray<double, 1>> f,
  std::shared_ptr<ndarray<double, 1>> g)
{
    // Upper bound on the traded amount
    std::shared_ptr<ndarray<double, 1>> u(new ndarray<double, 1>(shape_t<1>(n), w + u
    ↪ sum(x0)));

    Model::t M = new Model("Markowitz portfolio with transaction costs"); auto M_ = M
    ↪ finally([&] () { M->dispose(); });

    // Defines the variables. No shortselling is allowed.
    Variable::t x = M->variable("x", n, Domain::greaterThan(0.0));

    // Additional "helper" variables
    Variable::t z = M->variable("z", n, Domain::unbounded());
    // Binary variables
```

(continues on next page)

```

Variable::t y = M->variable("y", n, Domain::binary());

// Maximize expected return
M->objective("obj", ObjectiveSense::Maximize, Expr::dot(mu, x));

// Invest amount + transactions costs = initial wealth
M->constraint("budget", Expr::add(Expr::add(Expr::sum(x), Expr::dot(f, y)), Expr::
→dot(g, z)),
            Domain::equalsTo(w + sum(x0)));

// Imposes a bound on the risk
M->constraint("risk", Expr::vstack( gamma, Expr::mul(GT, x)),
            Domain::inQCone());

// z >= |x-x0|
M->constraint("buy", Expr::sub(z, Expr::sub(x, x0)), Domain::greaterThan(0.0));
M->constraint("sell", Expr::sub(z, Expr::sub(x0, x)), Domain::greaterThan(0.0));

// Constraints for turning y off and on. z-diag(u)*y<=0 i.e. z_j <= u_j*y_j
M->constraint("y_on_off", Expr::sub(z, Expr::mul(Matrix::diag(u), y)), Domain::
→lessThan(0.0));

// Integer optimization problems can be very hard to solve so limiting the
// maximum amount of time is a valuable safe guard
M->setSolverParam("mioMaxTime", 180.0);
M->solve();

return x->level();
}

```

11.1.7 Cardinality constraints

Another method to reduce costs involved with processing transactions is to only change positions in a small number of assets. In other words, at most K of the differences $|\Delta x_j| = |x_j - x_j^0|$ are allowed to be non-zero, where K is (much) smaller than the total number of assets n .

This type of constraint can be again modeled by introducing a binary variable y_j which indicates if $\Delta x_j \neq 0$ and bounding the sum of y_j . The basic Markowitz model then gets updated as follows:

$$\begin{aligned}
 &\text{maximize} && \mu^T x \\
 &\text{subject to} && e^T x = w + e^T x^0, \\
 & && (\gamma, G^T x) \in \mathcal{Q}^{k+1}, \\
 & && z_j \geq x_j - x_j^0, & j = 1, \dots, n, \\
 & && z_j \geq x_j^0 - x_j, & j = 1, \dots, n, \\
 & && z_j \leq U_j y_j, & j = 1, \dots, n, \\
 & && y_j \in \{0, 1\}, & j = 1, \dots, n, \\
 & && e^T y \leq K, \\
 & && x \geq 0,
 \end{aligned} \tag{11.12}$$

where U_j is some a priori chosen upper bound on the amount of trading in asset j .

Example code

The following example code demonstrates how to compute an optimal portfolio with cardinality bounds. Note that we define the maximum cardinality as a parameter in the model and use it to parametrize the cardinality constraint. This way we can use one model to solve many problems with the same structure and data except for the cardinality bound by simply changing this parameter between the solves.

Listing 11.7: Code solving problem (11.12).

```
std::vector<std::vector<double>> MarkowitzWithCardinality
    ( int n,
      std::shared_ptr<ndarray<double, 1>> mu,
      std::shared_ptr<ndarray<double, 2>> GT,
      std::shared_ptr<ndarray<double, 1>> x0,
      double w,
      double gamma,
      std::vector<int> kValues)
{
    // Upper bound on the traded amount
    std::shared_ptr<ndarray<double, 1>> u(new ndarray<double, 1>(shape_t<1>(n), w +
    ↪ sum(x0)));

    Model::t M = new Model("Markowitz portfolio with cardinality constraints"); auto M_
    ↪ = finally([&]() { M->dispose(); });

    // Defines the variables. No shortselling is allowed.
    Variable::t x = M->variable("x", n, Domain::greaterThan(0.0));

    // Additional "helper" variables
    Variable::t z = M->variable("z", n, Domain::unbounded());
    // Binary variables
    Variable::t y = M->variable("y", n, Domain::binary());

    // Maximize expected return
    M->objective("obj", ObjectiveSense::Maximize, Expr::dot(mu, x));

    // The amount invested must be identical to initial wealth
    M->constraint("budget", Expr::sum(x), Domain::equalsTo(w + sum(x0)));

    // Imposes a bound on the risk
    M->constraint("risk", Expr::vstack( gamma, Expr::mul(GT, x)),
        Domain::inQCone());

    // z >= |x-x0|
    M->constraint("buy", Expr::sub(z, Expr::sub(x, x0)), Domain::greaterThan(0.0));
    M->constraint("sell", Expr::sub(z, Expr::sub(x0, x)), Domain::greaterThan(0.0));

    // Constraints for turning y off and on. z-diag(u)*y<=0 i.e. z_j <= u_j*y_j
    M->constraint("y_on_off", Expr::sub(z, Expr::mul(Matrix::diag(u), y)), Domain::
    ↪ lessThan(0.0));

    // At most k assets change position
    auto cardMax = M->parameter();
    M->constraint("cardinality", Expr::sub(Expr::sum(y), cardMax), Domain::lessThan(0));

    // Integer optimization problems can be very hard to solve so limiting the
    // maximum amount of time is a valuable safe guard
    M->setSolverParam("mioMaxTime", 180.0);
```

(continues on next page)

```

// Solve multiple instances by varying the cardinality bound
std::vector<std::vector<double>> results;

for(auto k : kValues) {
    cardMax->setValue(k);
    M->solve();
    auto sol = x->level();
    results.push_back(new_vector_from_array_ptr(sol));
}

return results;
}

```

If we solve our running example with $K = 1, \dots, n$ then we get the following solutions, with increasing expected returns:

Bound 1	Solution:	0.0000e+00	0.0000e+00	1.0000e+00	0.0000e+00	0.0000e+00	┐
↪	0.0000e+00	0.0000e+00	0.0000e+00				
Bound 2	Solution:	0.0000e+00	0.0000e+00	3.5691e-01	0.0000e+00	0.0000e+00	┐
↪	6.4309e-01	-0.0000e+00	0.0000e+00				
Bound 3	Solution:	0.0000e+00	0.0000e+00	1.9258e-01	0.0000e+00	0.0000e+00	┐
↪	5.4592e-01	2.6150e-01	0.0000e+00				
Bound 4	Solution:	0.0000e+00	0.0000e+00	2.0391e-01	0.0000e+00	6.7098e-02	┐
↪	4.9181e-01	2.3718e-01	0.0000e+00				
Bound 5	Solution:	0.0000e+00	3.1970e-02	1.7028e-01	0.0000e+00	7.0741e-02	┐
↪	4.9551e-01	2.3150e-01	0.0000e+00				
Bound 6	Solution:	0.0000e+00	3.1970e-02	1.7028e-01	0.0000e+00	7.0740e-02	┐
↪	4.9551e-01	2.3150e-01	0.0000e+00				
Bound 7	Solution:	0.0000e+00	3.1970e-02	1.7028e-01	0.0000e+00	7.0740e-02	┐
↪	4.9551e-01	2.3150e-01	0.0000e+00				
Bound 8	Solution:	1.9557e-10	2.6992e-02	1.6706e-01	2.9676e-10	7.1245e-02	┐
↪	4.9559e-01	2.2943e-01	9.6905e-03				

11.2 Primal Support-Vector Machine (SVM)

Machine-Learning (ML) has become a common widespread tool in many applications that affect our everyday life. In many cases, at the very core of these techniques there is an optimization problem. This case study focuses on the Support-Vector Machines (SVM).

The basic SVM model can be stated as:

We are given a set of m points in $\mathbb{R}^n$, partitioned into two groups. Find, if any, the separating hyperplane of the two subsets with the largest margin, i.e. as far as possible from the points.

Mathematical Model

Let $x_1, \dots, x_m \in \mathbb{R}^n$ be the given training set and let $y_i \in \{-1, +1\}$ be the labels indicating the group membership of the i -th training example. Then we want to determine an affine hyperplane $w^T x = b$ that separates the group in the strong sense that

$$y_i(w^T x_i - b) \geq 1 \quad (11.13)$$

for all i , the property referred to as *large margin classification*: the strip $\{x \in \mathbb{R}^n : -1 < w^T x - b < 1\}$ does not contain any training example. The width of this strip is $2\|w\|^{-1}$, and maximizing that quantity is equivalent to minimizing $\|w\|$. We get that the large margin classification is the solution of the following optimization problem:

$$\begin{aligned} & \text{minimize}_{b,w} \quad \frac{1}{2}\|w\|^2 \\ & \text{subject to} \quad y_i(w^T x_i - b) \geq 1 \quad i = 1, \dots, m. \end{aligned}$$

If a solution exists, w, b define the separating hyperplane and the sign of $w^T x - b$ can be used to decide the class in which a point x falls.

To allow more flexibility the soft-margin SVM classifier is often used instead. It admits a violation of the large margin requirement (11.13) by a non-negative slack variable which is then penalized in the objective function.

$$\begin{aligned} \text{minimize}_{b,w} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} \quad & y_i(w^T x_i - b) \geq 1 - \xi_i \quad i = 1, \dots, m, \\ & \xi_i \geq 0 \quad i = 1, \dots, m. \end{aligned}$$

In matrix form we have

$$\begin{aligned} \text{minimize}_{b,w,\xi} \quad & \frac{1}{2} \|w\|^2 + C \mathbf{e}^T \xi \\ \text{subject to} \quad & y \star (Xw - b\mathbf{e}) + \xi \geq \mathbf{e}, \\ & \xi \geq 0. \end{aligned}$$

where $\star$ denotes the component-wise product, and $\mathbf{e}$ a vector with all components equal to one. The constant $C \geq 0$ acts both as scaling factor and as weight. Varying C yields different trade-offs between accuracy and robustness.

Implementing the matrix formulation of the soft-margin SVM in *Fusion* is very easy. We only need to cast the problem in conic form, which in this case involves converting the quadratic term of the objective function into a conic constraint:

$$\begin{aligned} \text{minimize}_{b,w,\xi,t} \quad & t + C \mathbf{e}^T \xi \\ \text{subject to} \quad & \xi + y \star (Xw - b\mathbf{e}) \geq \mathbf{e}, \\ & (1, t, w) \in \mathcal{Q}_r^{n+2}, \\ & \xi \geq 0. \end{aligned} \tag{11.14}$$

where $\mathcal{Q}_r^{n+2}$ denotes a rotated cone of dimension $n + 2$.

Fusion implementation

We now demonstrate how implement model (11.14). Let us assume that the training examples are stored in the rows of a matrix X , the labels in a vector y and that we have a set of weights C for which we want to train the model. The implementation in *Fusion* of our conic model starts declaring the model class:

```
Model::t M = new Model("primal SVM"); auto _M = finally([&]() { M->dispose(); });
```

Then we proceed defining the variables :

```
Variable::t w = M->variable( n, Domain::unbounded());
Variable::t t = M->variable( 1, Domain::unbounded());
Variable::t b = M->variable( 1, Domain::unbounded());
Variable::t xi = M->variable( m, Domain::greaterThan(0.));
```

The conic constraint is obtained by stacking the three values:

```
M->constraint( Expr::vstack(1., t, w) , Domain::inRotatedQCone() );
```

Note how the dimension of the cone is deduced from the arguments. The relaxed classification constraints can be expressed using the built-in expressions available in *Fusion*. In particular:

1. element-wise multiplication $\star$ is performed with the [Expr.mulElm](#) function;
2. a vector whose entries are repetitions of b is produced by [Var.repeat](#).

The results is

```
auto ex = Expr::sub( Expr::mul(X, w), Var::repeat(b, m) );
M->constraint( Expr::add(Expr::mulElm( y, ex ), xi ) , Domain::greaterThan( 1. ) );
```

Finally, the objective function is defined as

```
M->objective( ObjectiveSense::Minimize, Expr::add( t, Expr::mul(c, Expr::sum(xi)
↪) ) );
```

To solve a sequence of problems with varying C we can simply iterate along those values changing the objective function:

```
std::cout << "    c    | b          | w\n";
for (int i = 0; i < nc; i++)
{
    double c = 500.0 * i;
    M->objective( ObjectiveSense::Minimize, Expr::add( t, Expr::mul(c, Expr::sum(xi)
↪) ) );
    M->solve();

    try
    {
        std::cout << std::setw(6) << c << " | " << std::setw(8) << (*(b->level())) [0] <
↪< " | ";
        std::cout.width(8);
        auto wlev = w->level();
        std::copy( wlev->begin(), wlev->end() , std::ostream_iterator<double>(std::cout,
↪ " ") );
        std::cout << "\n";
    }
    catch (...) {}
}
```

Source code

Listing 11.8: The code implementing model (11.14)

```
Model::t M = new Model("primal SVM"); auto _M = finally([&]() { M->dispose(); });

Variable::t w = M->variable( n, Domain::unbounded());
Variable::t t = M->variable( 1, Domain::unbounded());
Variable::t b = M->variable( 1, Domain::unbounded());
Variable::t xi = M->variable( m, Domain::greaterThan(0.));

auto ex = Expr::sub( Expr::mul(X, w), Var::repeat(b, m) );
M->constraint( Expr::add(Expr::mulElm( y, ex ), xi ), Domain::greaterThan( 1. ) );

M->constraint( Expr::vstack(1., t, w) , Domain::inRotatedQCone() );

std::cout << "    c    | b          | w\n";
for (int i = 0; i < nc; i++)
{
    double c = 500.0 * i;
    M->objective( ObjectiveSense::Minimize, Expr::add( t, Expr::mul(c, Expr::sum(xi)
↪) ) );
    M->solve();

    try
    {
        std::cout << std::setw(6) << c << " | " << std::setw(8) << (*(b->level())) [0] <
↪< " | ";
        std::cout.width(8);
```

(continues on next page)

```

    auto wlev = w->level();
    std::copy( wlev->begin(), wlev->end() , std::ostream_iterator<double>(std::cout,
↪ " ") );
    std::cout << "\n";
}
catch (...) {}

}

```

Example

We generate a random dataset consisting of two groups of points, each from a Gaussian distribution in $\mathbb{R}^2$ with centres $(1.0, 1.0)$ and $(-1.0, -1.0)$, respectively.

```

int m = 50 ;
int n = 3;
int nc = 10;

int nump = m / 2;
int numm = m - nump;

auto y = new_array_ptr<double, 1> (m);
std::fill( y->begin(), y->begin() + nump, 1.);
std::fill( y->begin() + nump, y->end(), -1.);

double mean = 1.;
double var = 1.;

auto X = std::shared_ptr< ndarray<double, 2> > ( new ndarray<double, 2> ( shape_t
↪ <2>(m, n) ) );

std::mt19937 e2(0);

for (int i = 0; i < nump; i++)
{
    auto ram = std::bind(std::normal_distribution<>(mean, var), e2);
    for ( int j = 0; j < n; j++)
        (*X)(i, j) = ram();
}

std::cout << "Number of data      : " << m << std::endl;
std::cout << "Number of features: " << n << std::endl;

```

With standard deviation $\sigma = 1/2$ we obtain a separable instance of the problem with a solution shown in Fig. 11.2.

For $\sigma = 1$ the two groups are not linearly separable and the we obtain the optimal hyperplane as in Fig. 11.3.

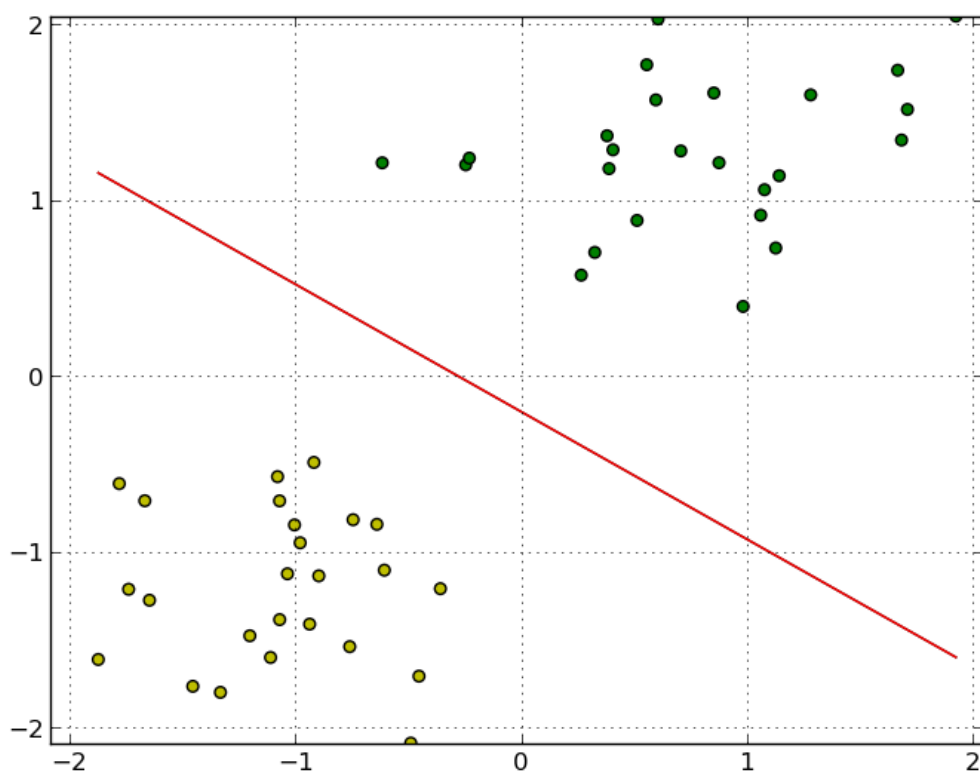


Fig. 11.2: Separating hyperplane for two clusters of points.

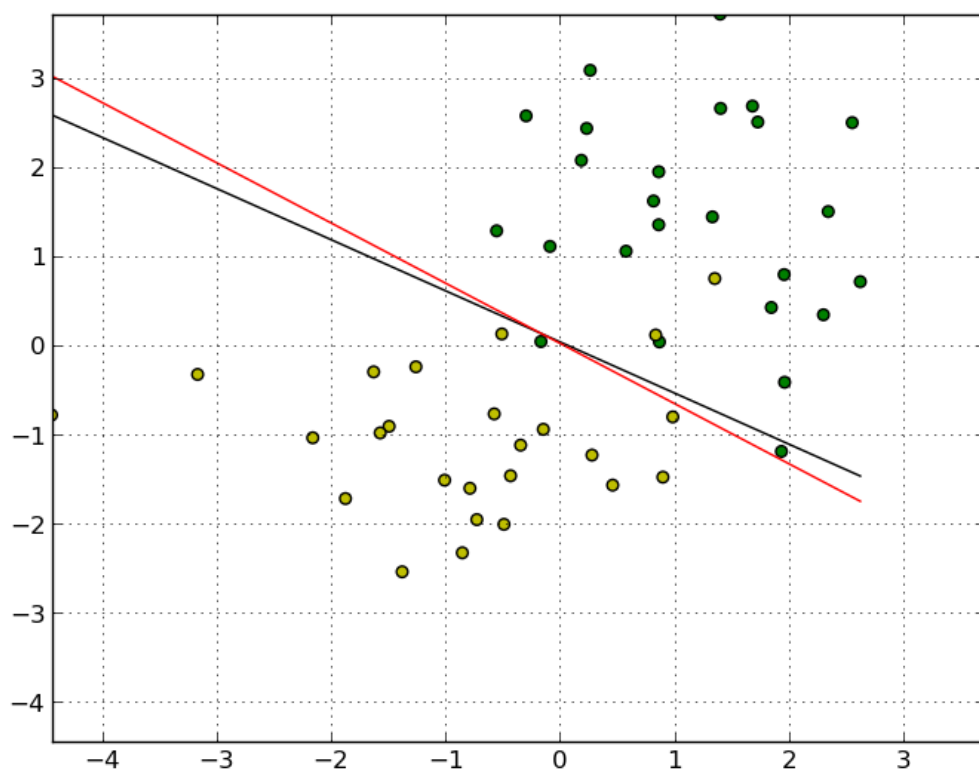


Fig. 11.3: Soft separating hyperplane for two groups of points.

11.3 2D Total Variation

This case study is based mainly on the paper by Goldfarb and Yin [GY05].

Mathematical Formulation

We are given a $n \times m$ grid and for each cell (i, j) an observed value f_{ij} that can be expressed as

$$f_{ij} = u_{ij} + v_{ij},$$

where $u_{ij} \in [0, 1]$ is the actual signal value and v_{ij} is the noise. The aim is to reconstruct u subtracting the noise from the observations.

We assume the 2-norm of the overall noise to be bounded: the corresponding constraint is

$$\|u - f\|_2 \leq \sigma$$

which translates into a simple conic quadratic constraint as

$$(\sigma, u - f) \in \mathcal{Q}.$$

We aim to minimize the change in signal value when moving between adjacent cells. To this end we define the adjacent differences vector as

$$\partial_{ij}^+ = \begin{pmatrix} \partial_{ij}^x \\ \partial_{ij}^y \end{pmatrix} = \begin{pmatrix} u_{i+1,j} - u_{i,j} \\ u_{i,j+1} - u_{i,j} \end{pmatrix}, \quad (11.15)$$

for each cell $1 \leq i, j \leq n$ (we assume that the respective coordinates ∂_{ij}^x and ∂_{ij}^y are zero on the right and bottom boundary of the grid).

For each cell we want to minimize the norm of ∂_{ij}^+ , and therefore we introduce auxiliary variables t_{ij} such that

$$t_{ij} \geq \|\partial_{ij}^+\|_2 \quad \text{or} \quad (t_{ij}, \partial_{ij}^+) \in \mathcal{Q},$$

and minimize the sum of all t_{ij} .

The complete model takes the form:

$$\begin{aligned} \min \quad & \sum_{1 \leq i, j \leq n} t_{ij}, \\ \text{s.t.} \quad & \partial_{ij}^+ = (u_{i+1,j} - u_{i,j}, u_{i,j+1} - u_{i,j})^T, \quad \forall 1 \leq i, j \leq n, \\ & (t_{ij}, \partial_{ij}^+) \in \mathcal{Q}^3, \quad \forall 1 \leq i, j \leq n, \\ & (\sigma, u - f) \in \mathcal{Q}^{nm+1}, \\ & u_{i,j} \in [0, 1]. \quad \forall 1 \leq i, j \leq n. \end{aligned} \quad (11.16)$$

Implementation

The *Fusion* implementation of model (11.16) uses variable and expression slices.

First of all we start by creating the optimization model and variables $\mathbf{t}$ and $\mathbf{u}$. Since we intend to solve the problem many times with various input data we define σ and f as parameters:

```
Model::t M = new Model("TV");

Variable::t u = M->variable("u", new_array_ptr<int, 1>({n + 1, m + 1}), Domain::
→inRange(0.0, 1.0));
Variable::t t = M->variable("t", new_array_ptr<int, 1>({n, m}), Domain::
→unbounded());

// In this example we define sigma and the input image f as parameters
```

(continues on next page)

(continued from previous page)

```
// to demonstrate how to solve the same model with many data variants.
// Of course they could simply be passed as ordinary arrays if that is not needed.
Parameter::t sigma = M->parameter("sigma");
Parameter::t f = M->parameter("f", n, m);
```

Note the dimensions of u is larger than those of the grid to accommodate the boundary conditions later. The actual cells of the grid are defined as a slice of u :

```
Variable::t ucore = u->slice(new_array_ptr<int, 1>({0, 0}), new_array_ptr<int, 1>({n, m}));
```

The next step is to define the partial variation along each axis, as in (11.15):

```
Expression::t deltax = Expr::sub( u->slice( new_array_ptr<int, 1>({1, 0}), new_
array_ptr<int, 1>({n + 1, m}) ), ucore );
Expression::t deltax = Expr::sub( u->slice( new_array_ptr<int, 1>({0, 1}), new_
array_ptr<int, 1>({n, m + 1}) ), ucore );
```

Slices are created on the fly as they will not be reused. Now we can set the conic constraints on the norm of the total variations. To this extent we stack the variables t , $deltax$ and $deltay$ together and demand that each row of the new matrix is in a quadratic cone.

```
M->constraint( Expr::stack(2, t, deltax, deltax), Domain::inQCone()->axis(2) );
```

We now need to bound the norm of the noise. This can be achieved with a conic constraint using f as a one-dimensional array:

```
M->constraint( Expr::vstack(sigma, Expr::flatten( Expr::sub(f, ucore) ) ),
Domain::inQCone() );
```

The objective function is the sum of all t_{ij} :

```
M->objective( ObjectiveSense::Minimize, Expr::sum(t) );
```

Example

Consider the linear signal $u_{ij} = \frac{i+j}{n+m}$ and its modification with random Gaussian noise, as in Fig. 11.4. Various reconstructions of u , obtained with different values of σ , are shown in Fig. 11.5 (where $\bar{\sigma} = \sigma/nm$ is the relative noise bound per cell).

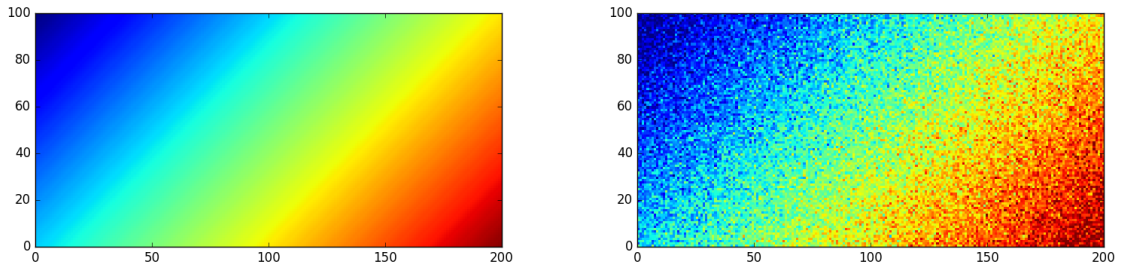


Fig. 11.4: A linear signal and its modification with random Gaussian noise.

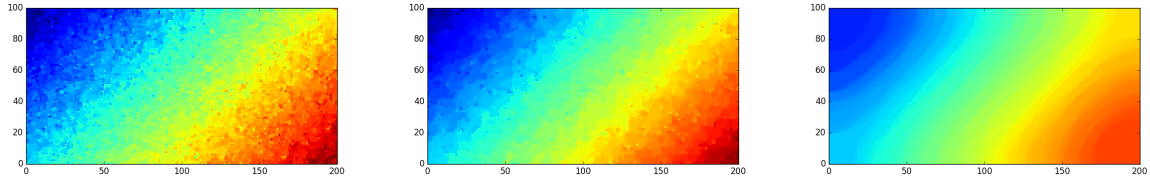


Fig. 11.5: Three reconstructions of the linear signal obtained for $\bar{\sigma} \in \{0.0004, 0.0005, 0.0006\}$, respectively.

Source code

Listing 11.9: The *Fusion* implementation of model (11.16).

```

Model::t total_var(int n, int m) {
    Model::t M = new Model("TV");

    Variable::t u = M->variable("u", new_array_ptr<int, 1>({n + 1, m + 1}), Domain::
    ↪inRange(0.0, 1.0));
    Variable::t t = M->variable("t", new_array_ptr<int, 1>({n, m}), Domain::
    ↪unbounded());

    // In this example we define sigma and the input image f as parameters
    // to demonstrate how to solve the same model with many data variants.
    // Of course they could simply be passed as ordinary arrays if that is not needed.
    Parameter::t sigma = M->parameter("sigma");
    Parameter::t f = M->parameter("f", n, m);

    Variable::t ucore = u->slice(new_array_ptr<int, 1>({0, 0}), new_array_ptr<int, 1>({
    ↪n, m}));

    Expression::t deltax = Expr::sub( u->slice( new_array_ptr<int, 1>({1, 0}), new_
    ↪array_ptr<int, 1>({n + 1, m}) ), ucore );
    Expression::t deltax = Expr::sub( u->slice( new_array_ptr<int, 1>({0, 1}), new_
    ↪array_ptr<int, 1>({n, m + 1}) ), ucore );

    M->constraint( Expr::stack(2, t, deltax, deltax), Domain::inQCone()->axis(2) );

    M->constraint( Expr::vstack(sigma, Expr::flatten( Expr::sub(f, ucore) ) ),
    ↪Domain::inQCone() );

    M->objective( ObjectiveSense::Minimize, Expr::sum(t) );

    return M;
}

int main(int argc, char ** argv)
{
    std::normal_distribution<double> ndistr(0., 1.);
    std::mt19937 engine(0);

    int n = 100;
    int m = 200;

```

(continues on next page)

```

std::vector<double> sigmas({ 0.0004, 0.0005, 0.0006 });

// Create a parametrized model with given shape
Model::t M = total_var(n, m);
Parameter::t sigma = M->getParameter("sigma");
Parameter::t f      = M->getParameter("f");
Variable::t ucore = M->getVariable("u")->slice(new_array_ptr<int, 1>({0, 0}), new_
↪array_ptr<int, 1>({n, m}));

// Example: Linear signal with Gaussian noise
std::vector<std::vector<double>> signal(n, std::vector<double>(m));
std::vector<std::vector<double>> noise(n, std::vector<double>(m));
std::vector<std::vector<double>> fVal(n, std::vector<double>(m));
std::vector<std::vector<double>> sol(n, std::vector<double>(m));

for(int i=0; i<n; i++) for(int j=0; j<m; j++) {
    signal[i][j] = 1.0*(i+j)/(n+m);
    noise[i][j] = ndistr(engine) * 0.08;
    fVal[i][j] = std::max( std::min(1.0, signal[i][j] + noise[i][j]), .0 );
}

// Set value for f
f->setValue(new_array_ptr(fVal));

for(int iter=0; iter<3; iter++) {
    // Set new value for sigma and solve
    sigma->setValue(sigmas[iter]*n*m);

    M->solve();

    // Retrieve solution from ucore
    auto ucoreLev = *(ucore->level());
    for(int i=0; i<n; i++) for(int j=0; j<m; j++)
        sol[i][j] = ucoreLev[i*n+m];

    // Use the solution
    // ...

    std::cout << "rel_sigma = " << sigmas[iter] << " total_var = " << M->
↪primalObjValue() << std::endl;
}

M->dispose();

return 0;
}

```

11.4 Multiprocessor Scheduling

In this case study we consider a simple scheduling problem in which a set of jobs must be assigned to a set of identical machines. We want to minimize the makespan of the overall processing, i.e. the latest machine termination time.

The main aims of this case study are

- to show how to define a Integer Linear Programming model,
- to take advantage of *Fusion* operators to compactly express sets of constraints,
- to provide the solver with an incumbent integer feasible solution.

Mathematical formulation

We are given a set of jobs J with $|J| = n$ to be assigned to a set M of identical machines with $|M| = m$. Each job $j \in J$ has a processing time $T_j > 0$ and can be assigned to any machine. Our aim is to find the job scheduling that minimizes the overall makespan, i.e. the maximum completion time among all machines.

Formally, we introduce a binary variable x_{ij} that takes value 1 if the job j is assigned to the machine i , zero otherwise. The only constraint we need to set is the requirement that a job must be assigned to a single machine. The optimization model takes the following form:

$$\begin{aligned} \min \quad & \max_{i \in M} \sum_{j \in J} T_j x_{ij} \\ \text{s.t.} \quad & \sum_{i \in M} x_{ij} = 1, & j \in J, \\ & x_{ij} \in \{0, 1\} & \forall i \in M, j \in J. \end{aligned} \quad (11.17)$$

Model (11.17) can be easily transformed into an integer linear programming model as follows:

$$\begin{aligned} \min \quad & t \\ \text{s.t.} \quad & \sum_{i \in M} x_{ij} = 1, & j \in J, \\ & t \geq \sum_{j \in J} T_j x_{ij}, & i \in M, \\ & x_{ij} \in \{0, 1\}, & \forall i \in M, j \in J. \end{aligned} \quad (11.18)$$

The implementation of this model in *Fusion* is straightforward:

```
Model::t M = new Model("Multi-processor scheduling"); auto _M = finally([&]() { M->
->dispose(); });

Variable::t x = M->variable("x", new_array_ptr<int, 1>({m, n}), Domain::binary());
Variable::t t = M->variable("t", 1, Domain::unbounded());

M->constraint( Expr::sum(x, 0), Domain::equalsTo(1.) );
M->constraint( Expr::sub( Var::repeat(t, m), Expr::mul(x, T) ), Domain::
->greaterThan(0.) );

M->objective( ObjectiveSense::Minimize, t );
```

Most of the code is self-explanatory. The only critical point is

```
M->constraint( Expr::sub( Var::repeat(t, m), Expr::mul(x, T) ), Domain::
->greaterThan(0.) );
```

that implements the set of constraints

$$t \geq \sum_{j \in J} T_j x_{ij}, \quad i \in M.$$

To fit in *Fusion* we restate the constraints as

$$t - \sum_{j \in J} T_j x_{ij} \geq 0, \quad i \in M,$$

which corresponds in matrix form to

$$t\mathbf{1} - xT \geq 0. \quad (11.19)$$

The function `Var.repeat` creates a vector of length m , as required for (11.19). The same result can be obtained via matrix multiplication, i.e. using `Expr.mul`, but in this particular case `Var.repeat` is faster as it only performs a logical operation.

Longest Processing Time first rule (LPT)

The multiprocessor scheduling is known to be an NP-complete problem (see [GJ79]). Nevertheless there are effective heuristics, with provable worst case bounds, that are able to provide a good integer solution quickly. In particular, we will use the so-called *Longest Processing Time first* rule (LPT, proposed in [Gra69]).

The informal algorithm sketch is the following:

- while M is not empty do
 - let k be the machine with the smallest load so far,
 - let i be the job in M with the longest completion time,
 - assign job i to machine k ,
 - update the load of machine k ,
 - remove i from M .

This simple algorithm is a $\frac{1}{3}(4 - \frac{1}{m})$ approximation. So for $m = 1$ we get the optimal solution (indeed there is no choice with a single machine); for $m \rightarrow \infty$ the approximation factor is no worse than $4/3$ (again see [Gra69]).

A simple implementation is given below.

```
//LPT heuristic
auto schedule = std::shared_ptr<ndarray<double, 1>>(new ndarray<double, 1>(m, 0.));
auto init = std::shared_ptr<ndarray<double, 1>>(new ndarray<double, 1>(n * m, 0.));

for (int i = 0; i < n; i++)
{
    auto pos = std::distance(schedule->begin(), std::min_element(schedule->begin(),
↪schedule->end()));
    (*schedule)[pos] += (*T)[i];
    (*init)[pos * n + i] = 1;
}
```

An efficient implementation of the LPT rule is beyond the scope of this section. The important part is that the scheduling produced by the LPT algorithm can be used as incumbent solution for the **MOSEK** mixed-integer linear programming solver. The availability of an integer feasible solution can significantly improve the performance of the solver.

To input the solution we only need to use the `Variable.setLevel` method, as shown below

```
x->setLevel(init);
```

We can test the program with and without providing the initial LPT solution. Our random datasets consists of a mix of tasks with long and short processing times and we accept a solution at relative optimality tolerance 0.01. Some results are shown in the table below.

Table 11.1: Sample test results for the makespan problem.

n	m	long tasks	short tasks	No LPT	With LPT
1000	8	20%	80%	13.36s	1.23s
1000	8	80%	20%	1.35s	1.24s
100	12	20%	80%	16.37s	0.11s
100	12	80%	20%	16.62s	10.01s
20	20	0%	100%	10.38s	21.88s

We can see that depending on the structure and parameters of the problem it may pay off to provide an initial LPT solution. Therefore it is always recommended to test the mixed-integer solver with different settings to find the most efficient setup for a given problem.

Listing 11.10: Complete code for the LPT scheduling example.

```
#include <iostream>
#include <random>
#include <sstream>

#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

int main(int arc, char** argv)
{
    double lb = 1.0;           //Bounds for the length of a short task
    double ub = 5.;

    int n = 30;                //Number of tasks
    int m = 6;                  //Number of processors

    double sh = 0.8;           //The proportion of short tasks
    int n_short = (int)(sh * n);
    int n_long = n - n_short;

    auto gen = std::bind(std::uniform_real_distribution<double>(lb, ub), std::mt19937(0));

    auto T = std::shared_ptr<ndarray<double, 1>>(new ndarray<double, 1>(n));
    for (int i = 0; i < n_short; i++) (*T)[i] = gen();
    for (int i = n_short; i < n; i++) (*T)[i] = 20 * gen();
    std::sort(T->begin(), T->end(), std::greater<double>());

    Model::t M = new Model("Multi-processor scheduling"); auto _M = finally([&]() { M->dispose(); });

    Variable::t x = M->variable("x", new_array_ptr<int, 1>({m, n}), Domain::binary());
    Variable::t t = M->variable("t", 1, Domain::unbounded());

    M->constraint( Expr::sum(x, 0), Domain::equalsTo(1.) );
    M->constraint( Expr::sub( Var::repeat(t, m), Expr::mul(x, T) ), Domain::greaterThan(0.) );

    M->objective( ObjectiveSense::Minimize, t );

    //LPT heuristic
    auto schedule = std::shared_ptr<ndarray<double, 1>>(new ndarray<double, 1>(m, 0.));
    auto init = std::shared_ptr<ndarray<double, 1>>(new ndarray<double, 1>(n * m, 0.));

    for (int i = 0; i < n; i++)
    {
        auto pos = std::distance(schedule->begin(), std::min_element(schedule->begin(),
        schedule->end()));
        (*schedule)[pos] += (*T)[i];
        (*init)[pos * n + i] = 1;
    }
}
```

(continues on next page)

```

}

//Comment this line to switch off feeding in the initial LPT solution
x->setLevel(init);

M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush; }␣
→);

M->setSolverParam("mioTolRelGap", .01);
M->solve();

std::cout << "initial solution: \n";
for (int i = 0; i < m; i++)
{
    std::cout << "M " << i << " [";
    for (int y = 0; y < n; y++)
        std::cout << int( (*init)[i * n + y] ) << ", ";
    std::cout << "]\n";
}

std::cout << "MOSEK solution:\n";
for (int i = 0; i < m; i++)
{
    std::cout << "M " << i << " [";
    for (int y = 0; y < n; y++)
        std::cout << int((*(x->index(i, y)->level()))[0]) << ", ";
    std::cout << "]\n";
}

return 0;
}

```

11.5 Logistic regression

Logistic regression is an example of a binary classifier, where the output takes one two values 0 or 1 for each data point. We call the two values *classes*.

Formulation as an optimization problem

Define the sigmoid function

$$S(x) = \frac{1}{1 + \exp(-x)}.$$

Next, given an observation $x \in \mathbb{R}^d$ and a weights $\theta \in \mathbb{R}^d$ we set

$$h_{\theta}(x) = S(\theta^T x) = \frac{1}{1 + \exp(-\theta^T x)}.$$

The weights vector θ is part of the setup of the classifier. The expression $h_{\theta}(x)$ is interpreted as the probability that x belongs to class 1. When asked to classify x the returned answer is

$$x \mapsto \begin{cases} 1 & h_{\theta}(x) \geq 1/2, \\ 0 & h_{\theta}(x) < 1/2. \end{cases}$$

When training a logistic regression algorithm we are given a sequence of training examples x_i , each labelled with its class $y_i \in \{0, 1\}$ and we seek to find the weights θ which maximize the likelihood

function

$$\prod_i h_\theta(x_i)^{y_i} (1 - h_\theta(x_i))^{1-y_i}.$$

Of course every single y_i equals 0 or 1, so just one factor appears in the product for each training data point. By taking logarithms we can define the logistic loss function:

$$J(\theta) = - \sum_{i:y_i=1} \log(h_\theta(x_i)) - \sum_{i:y_i=0} \log(1 - h_\theta(x_i)).$$

The training problem with regularization (a standard technique to prevent overfitting) is now equivalent to

$$\min_{\theta} J(\theta) + \lambda \|\theta\|_2.$$

This can equivalently be phrased as

$$\begin{aligned} & \text{minimize} && \sum_i t_i + \lambda r \\ & \text{subject to} && \begin{aligned} t_i &\geq -\log(h_\theta(x)) &= \log(1 + \exp(-\theta^T x_i)) & \text{if } y_i = 1, \\ t_i &\geq -\log(1 - h_\theta(x)) &= \log(1 + \exp(\theta^T x_i)) & \text{if } y_i = 0, \\ r &\geq \|\theta\|_2. \end{aligned} \end{aligned} \quad (11.20)$$

Implementation

As can be seen from (11.20) the key point is to implement the softplus bound $t \geq \log(1 + e^u)$, which is the simplest example of a log-sum-exp constraint for two terms. Here t is a scalar variable and u will be the affine expression of the form $\pm \theta^T x_i$. This is equivalent to

$$\exp(u - t) + \exp(-t) \leq 1$$

and further to

$$\begin{aligned} (z_1, 1, u - t) &\in K_{\text{exp}} & (z_1 \geq \exp(u - t)), \\ (z_2, 1, -t) &\in K_{\text{exp}} & (z_2 \geq \exp(-t)), \\ z_1 + z_2 &\leq 1. \end{aligned} \quad (11.21)$$

Listing 11.11: Implementation of $t \geq \log(1 + e^u)$ as in (11.21).

```
// t >= log( 1 + exp(u) ) coordinatewise
void softplus(Model::t M,
              Expression::t t,
              Expression::t u)
{
    int n = (*t->getShape())[0];
    auto z1 = M->variable(n);
    auto z2 = M->variable(n);
    M->constraint(Expr::add(z1, z2), Domain::equalsTo(1));
    M->constraint(Expr::hstack(z1, Expr::constTerm(n, 1.0), Expr::sub(u,t)), Domain::
    ↪ inPExpCone());
    M->constraint(Expr::hstack(z2, Expr::constTerm(n, 1.0), Expr::neg(t)), Domain::
    ↪ inPExpCone());
}
```

Once we have this subroutine, it is easy to implement a function that builds the regularized loss function model (11.20).

Listing 11.12: Implementation of (11.20).

```
// Model logistic regression (regularized with full 2-norm of theta)
// X - n x d matrix of data points
// y - length n vector classifying training points
// lamb - regularization parameter
std::pair<Model::t, Variable::t>
logisticRegression(std::vector<std::vector<double>> & X,
                  std::vector<bool> & y,
                  double lamb)
{
    int n = X.size();
    int d = X[0].size();    // num samples, dimension

    Model::t M = new Model();

    auto theta = M->variable(d);
    auto t      = M->variable(n);
    auto reg     = M->variable();

    M->objective(ObjectiveSense::Minimize, Expr::add(Expr::sum(t), Expr::mul(lamb,
    ↪ reg)));
    M->constraint(Var::vstack(reg, theta), Domain::inQCone());

    auto signs = std::make_shared<ndarray<double,1>>(shape(n), [y](ptrdiff_t i) {
    ↪ return y[i] ? -1 : 1; });

    softplus(M, t, Expr::mulElm(Expr::mul(new_array_ptr<double>(X), theta), signs));

    return std::make_pair(M, theta);
}
```

Example: 2D dataset fitting

In the next figure we apply logistic regression to the training set of 2D points taken from the example `ex2data2.txt`. The two-dimensional dataset was converted into a feature vector $x \in \mathbb{R}^{28}$ using monomial coordinates of degrees at most 6.

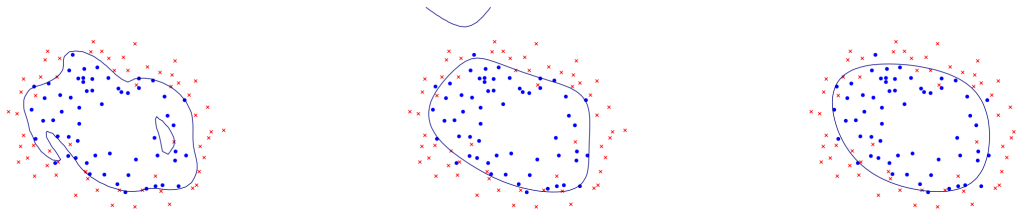


Fig. 11.6: Logistic regression example with none, medium and strong regularization (small, medium, large λ). Without regularization we get obvious overfitting.

11.6 Inner and outer Löwner-John Ellipsoids

In this section we show how to compute the Löwner-John *inner* and *outer* ellipsoidal approximations of a polytope. They are defined as, respectively, the largest volume ellipsoid contained inside the polytope and the smallest volume ellipsoid containing the polytope, as seen in Fig. 11.7.

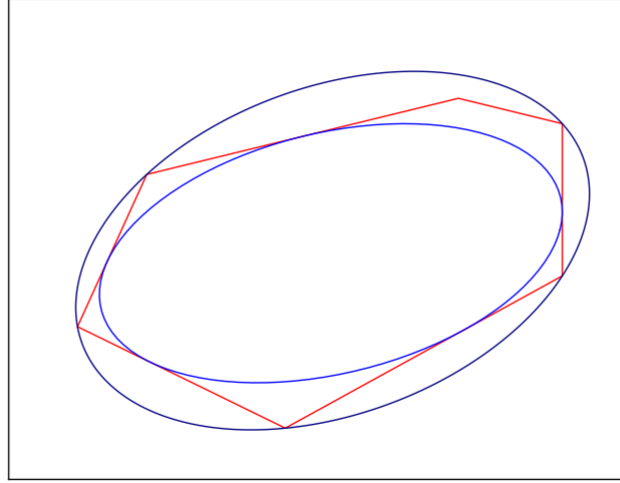


Fig. 11.7: The inner and outer Löwner-John ellipse of a polygon.

For further mathematical details, such as uniqueness of the two ellipsoids, consult [BenTalN01]. Our solution is a mix of conic and semidefinite programming. Among other things, in Sec. 11.6.3 we show how to implement bounds involving the determinant of a PSD matrix.

11.6.1 Inner Löwner-John Ellipsoids

Suppose we have a polytope given by an h-representation

$$\mathcal{P} = \{x \in \mathbb{R}^n \mid Ax \leq b\}$$

and we wish to find the inscribed ellipsoid with maximal volume. It will be convenient to parametrize the ellipsoid as an affine transformation of the standard disk:

$$\mathcal{E} = \{x \mid x = Cu + d, u \in \mathbb{R}^n, \|u\|_2 \leq 1\}.$$

Every non-degenerate ellipsoid has a parametrization such that C is a positive definite symmetric $n \times n$ matrix. Now the volume of $\mathcal{E}$ is proportional to $\det(C)^{1/n}$. The condition $\mathcal{E} \subseteq \mathcal{P}$ is equivalent to the inequality $A(Cu + d) \leq b$ for all u with $\|u\|_2 \leq 1$. After a short computation we obtain the formulation:

$$\begin{aligned} & \text{maximize} && t \\ & \text{subject to} && t \leq \det(C)^{1/n}, \\ & && (b - Ad)_i \geq \|(AC)_i\|_2, \quad i = 1, \dots, m, \\ & && C \succeq 0, \end{aligned} \tag{11.22}$$

where X_i denotes the i -th row of the matrix X . This can easily be implemented using *Fusion*, where the sequence of conic inequalities can be realized at once by feeding in the matrices $b - Ad$ and AC .

Listing 11.13: *Fusion* implementation of model (11.22).

```
std::pair<std::shared_ptr<ndarray<double, 1>>, std::shared_ptr<ndarray<double, 1>>>
    lownerjohn_inner
```

(continues on next page)

(continued from previous page)

```
( std::shared_ptr<ndarray<double, 2>> A,
  std::shared_ptr<ndarray<double, 1>> b)
{
  Model::t M = new Model("lowerjohn_inner"); auto _M = finally([&]() { M->dispose();
  ↪});
  int m = A->size(0);
  int n = A->size(1);

  // Setup variables
  Variable::t t = M->variable("t", 1, Domain::greaterThan(0.0));
  Variable::t C = det_rootn(M, t, n);
  Variable::t d = M->variable("d", n, Domain::unbounded());

  // quadratic cones
  M->constraint(Expr::hstack(Expr::sub(b, Expr::mul(A, d)), Expr::mul(A, C)),
               Domain::inQCone());

  // Objective: Maximize t
  M->objective(ObjectiveSense::Maximize, t);
  M->solve();

  return std::make_pair(C->level(), d->level());
}
```

The only black box is the method `det_rootn` which implements the constraint $t \leq \det(C)^{1/n}$. It will be described in [Sec. 11.6.3](#).

11.6.2 Outer Löwner-John Ellipsoids

To compute the outer ellipsoidal approximation to a polytope, let us now start with a v-representation

$$\mathcal{P} = \text{conv}\{x_1, x_2, \dots, x_m\} \subseteq \mathbb{R}^n,$$

of the polytope as a convex hull of a set of points. We are looking for an ellipsoid given by a quadratic inequality

$$\mathcal{E} = \{x \in \mathbb{R}^n \mid \|Px - c\|_2 \leq 1\},$$

whose volume is proportional to $\det(P)^{-1/n}$, so we are after maximizing $\det(P)^{1/n}$. Again, there is always such a representation with a symmetric, positive definite matrix P . The inclusion conditions $x_i \in \mathcal{E}$ translate into a straightforward problem formulation:

$$\begin{aligned} & \text{maximize} && t \\ & \text{subject to} && t \leq \det(P)^{1/n}, \\ & && \|Px_i - c\|_2 \leq 1, \quad i = 1, \dots, m, \\ & && P \succeq 0, \end{aligned} \tag{11.23}$$

and then directly into *Fusion* code:

Listing 11.14: *Fusion* implementation of model (11.23).

```
std::pair<std::shared_ptr<ndarray<double, 1>>, std::shared_ptr<ndarray<double, 1>>>
  lowerjohn_outer(std::shared_ptr<ndarray<double, 2>> x)
{
  Model::t M = new Model("lowerjohn_outer");
  int m = x->size(0);
  int n = x->size(1);
```

(continues on next page)

(continued from previous page)

```
// Setup variables
Variable::t t = M->variable("t", 1, Domain::greaterThan(0.0));
Variable::t P = det_rootn(M, t, n);
Variable::t c = M->variable("c", n, Domain::unbounded());

// (1, Px-c) \in Q
M->constraint(Expr::hstack(
    Expr::ones(m), Expr::sub(Expr::mul(x, P),
        Var::reshape(Var::repeat(c, m), new_array_ptr<int, 1>({m, n}))),
    ↪),
    Domain::inQCone());

// Objective: Maximize t
M->objective(ObjectiveSense::Maximize, t);
M->solve();

return std::make_pair(P->level(), c->level());
}
```

11.6.3 Bound on the Determinant Root

It remains to show how to express the bounds on $\det(X)^{1/n}$ for a symmetric positive definite $n \times n$ matrix X using PSD and conic quadratic variables. We want to model the set

$$C = \{(X, t) \in \mathcal{S}_+^n \times \mathbb{R} \mid t \leq \det(X)^{1/n}\}. \quad (11.24)$$

A standard approach when working with the determinant of a PSD matrix is to consider a semidefinite cone

$$\begin{pmatrix} X & Z \\ Z^T & \text{Diag}(Z) \end{pmatrix} \succeq 0 \quad (11.25)$$

where Z is a matrix of additional variables and where we intuitively identify $\text{Diag}(Z) = \{\lambda_1, \dots, \lambda_n\}$ with the eigenvalues of X . With this in mind, we are left with expressing the constraint

$$t \leq (\lambda_1 \cdots \lambda_n)^{1/n}. \quad (11.26)$$

but this is exactly the geometric mean cone [Domain.inPGeoMeanCone](#). We obtain the following model:

Listing 11.15: Bounding the n -th root of the determinant, see (11.25).

```
Variable::t det_rootn(Model::t M, Variable::t t, int n)
{
    // Setup variables
    Variable::t Y = M->variable(Domain::inPSDCone(2 * n));

    Variable::t X = Y->slice(new_array_ptr<int,1>({0, 0}), new_array_ptr<int,1>({n, n}
    ↪));
    Variable::t Z = Y->slice(new_array_ptr<int,1>({0, n}), new_array_ptr<int,1>({n, 2
    ↪ * n}));
    Variable::t DZ = Y->slice(new_array_ptr<int,1>({n, n}), new_array_ptr<int,1>({2 *
    ↪ n, 2 * n}));

    // Z is lower-triangular
    std::shared_ptr<ndarray<int,2>> low_tri( new ndarray<int,2>( shape_t<2>(n*(n-1)/2,
    ↪ 2) ));
    int k = 0;
```

(continues on next page)

```

for(int i = 0; i < n; i++)
    for(int j = i+1; j < n; j++)
        (*low_tri)(k,0) = i, (*low_tri)(k,1) = j, ++k;
M->constraint(Z->pick(low_tri), Domain::equalsTo(0.0));
// DZ = Diag(Z)
M->constraint(Expr::sub(DZ, Expr::mulElm(Z, Matrix::eye(n))), Domain::equalsTo(0.
↪0));

// (Z11*Z22*...*Znn) >= t^n
M->constraint(Expr::vstack(DZ->diag(), t), Domain::inPGeoMeanCone());

// Return an n x n PSD variable which satisfies t <= det(X)^(1/n)
return X;
}

```

11.7 SUDOKU

SUDOKU is a famous simple yet mind-blowing game. The objective is to fill a 9×9 grid with digits so that each column, each row, and each of the nine 3×3 sub-grids that compose the grid (also called *boxes*, *blocks*, *regions*, or *sub-squares*) contains all of the digits from 1 to 9. For more information see <http://en.wikipedia.org/wiki/Sudoku>. Here is a simple example:

				4				
	5	8			3			
	1		2	8		9		
	7	3	1			8	4	
	4	1			9	2	7	
		4		6	5		8	
			4			1	6	
				9				

A simple unsolved Sudoku

3	2	9	5	4	7	6	1	8
6	5	8	9	1	3	4	2	7
4	1	7	2	8	6	9	5	3
9	7	3	1	5	2	8	4	6
5	6	2	8	7	4	3	9	1
8	4	1	6	3	9	2	7	5
1	9	4	3	6	5	7	8	2
7	3	5	4	2	8	1	6	9
2	8	6	7	9	1	5	3	4

The solution

In a more general setting we are given a grid of dimension $n \times n$, with $n = m^2, m \in \mathbb{N}$. Each cell (i, j) must be filled with an integer $y_{ij} \in [1, n]$. Along each row and each column there must be no repetitions. No repetitions are allowed also in each sub-grid with corners $\{(mt, ml), (m(t+1)-1, m(l+1)-1)\}$, for $t, l = 0, \dots, m-1$ (we index cells from $(0, 0)$).

In general, each SUDOKU instance comes with a set F of predetermined values which:

- reduce the complexity of the game by removing symmetries and guiding the initial moves of the player;
- ensure that there will be a unique solution.

We represent the set F as list of triplets (i, j, v) , meaning that the cell (i, j) contains the value v .

Note that SUDOKU is a **feasibility** problem. A typical Integer Programming formulation is straightforward: let x_{ijk} be a binary variable that takes value 1 if k is written in cell (i, j) . Then we look for a feasible solution of a system of constraints given below.

SUDOKU is a typical assignment problem. Its constraints are commonly found in optimization problems concerning scheduling or resource allocation. SUDOKU has also been a nice problem to fiddle with for many researchers in the optimization community. Indeed, its simple structure and the easy way in which the results can be tested make it a perfect test problem.

We will approach SUDOKU as a standard integer linear program, and we will show how easily and elegantly it can be implemented in *Fusion*.

Mathematical Formulation

In this section we formulate SUDOKU as a mixed-integer linear optimization problem. Let's introduce a binary variable x_{ijk} that takes value 1 if k is written in the cell (i, j) , or 0 otherwise. We first ask that for each cell exactly one digit is selected:

$$\sum_{k=0}^{n-1} x_{ijk} = 1, \quad i, j = 0, \dots, n-1. \quad (11.27)$$

Similar constraints can be used to force each digit to appear only once in each row or column:

$$\begin{aligned} \sum_{i=0}^{n-1} x_{ijk} &= 1, & j, k &= 0, \dots, n-1, \\ \sum_{j=0}^{n-1} x_{ijk} &= 1, & i, k &= 0, \dots, n-1. \end{aligned} \quad (11.28)$$

To force a digit to appear only once in each sub-grid we can use the following

$$\sum_{i=0}^{m-1} \sum_{j=0}^{m-1} x_{(i+tm)(j+tl)k} = 1 \quad k = 0, \dots, n-1 \text{ and } t, l = 0, \dots, m-1 \quad (11.29)$$

If a cell (i, j) has a predetermined value, i.e. $(i, j, k) \in F$ then we set

$$x_{ijk} = 1.$$

Summarizing, and considering that there is no objective function to minimize, the optimization model for the SUDOKU problem takes the form

$$\begin{aligned} &\min 0 \\ &\text{s.t.} \\ &\sum_{i=0}^{n-1} x_{ijk} = 1, & j, k &= 0, \dots, n-1, \\ &\sum_{j=0}^{n-1} x_{ijk} = 1, & i, k &= 0, \dots, n-1, \\ &\sum_{k=0}^{n-1} x_{ijk} = 1, & i, j &= 0, \dots, n-1, \\ &\sum_{i=0}^{m-1} \sum_{j=0}^{m-1} x_{(i+tm)(j+tl)k} = 1, & k &= 0, \dots, n-1 \text{ and } \\ & & & t, l = 0, \dots, m-1, \\ &x_{ijk} = 1, & \forall (i, j, k) \in F. \end{aligned} \quad (11.30)$$

Implementation with *Fusion*

The implementation in *Fusion* is straightforward. First, we represent the variable x using a three dimensional *Fusion* variable:

```
Variable::t X = M->variable("X", new_array_ptr<int, 1>({n, n, n}), Domain::
  ↪binary());
```

Then we can define constraints (11.27) and (11.28) simply using the *Expr.sum* operator, that allows to sum the elements of an expression (in this case of the variable itself) along arbitrary dimensions. The code reads:

```
//each value only once per dimension
for (int d = 0; d < m; d++)
  M->constraint( Expr::sum(X, d), Domain::equalsTo(1.) );
```

The last set of constraints (11.29), i.e. the sum over block, needs a little more effort: we must loop over all blocks and select the proper slice:

```

//each number must appear only once in a block
for (int k = 0; k < n ; k++)
    for (int i = 0; i < m ; i++)
        for (int j = 0; j < m ; j++)
            M->constraint( Expr::sum( X->slice( new_array_ptr<int, 1>({i * m, j * m, k}),
                                                new_array_ptr<int, 1>({(i + 1)*m, (j +
↪1)*m, k + 1}) ) ),
                        Domain::equalsTo(1.) );

```

To set the triplets given in the set F we can use the `Variable.pick` method that returns a one dimensional view of an arbitrary set of elements of the variable.

```

auto fixed = std::shared_ptr< ndarray<int, 2> >( new ndarray<int, 2>( shape(nfixed,
↪3) ) );

for (int i = 0; i < nfixed; i++)
    for (int d = 0; d < m; d++)
        (*fixed)(i, d) = (*hr_fixed)(i, d) - 1;

M->constraint( X->pick( fixed ) , Domain::equalsTo(1.0) ) ;

```

SUDOKU: the complete example code.

The complete code for the SUDOKU problem is shown in [Listing 11.16](#).

Listing 11.16: *Fusion* implementation to solve SUDOKU.

```

#include <iostream>
#include <sstream>
#include <cmath>

#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;

void print_solution(int n, Variable::t X)
{
    using namespace std;

    cout << "\n";
    int m( std::sqrt(n) );
    for (int i = 0; i < n; i++)
    {
        stringstream ss;

        for (int j = 0; j < n; j++)
        {
            if (j % m == 0) ss << " |";

            for (int k = 0; k < n; k++)
            {
                auto x = X->index(new_array_ptr<int, 1>({i, j, k}))->level();
                if ( (*x)[0] > 0.5 )
                {
                    ss << " " << (k + 1);
                    break;
                }
            }
        }
    }
}

```

(continues on next page)

```

    }
}
cout << ss.str() << " |";

cout << "\n";
if ((i + 1) % m == 0)
    cout << "\n";
}
}

int main(int argc, char ** argv)
{

    int m = 3;
    int n = m * m;

    //fixed cells in human readable (i.e. 1-based) format
    auto hr_fixed = new_array_ptr<int, 2>(
    { {1, 5, 4},
      {2, 2, 5}, {2, 3, 8}, {2, 6, 3},
      {3, 2, 1}, {3, 4, 2}, {3, 5, 8}, {3, 7, 9},
      {4, 2, 7}, {4, 3, 3}, {4, 4, 1}, {4, 7, 8}, {4, 8, 4},
      {6, 2, 4}, {6, 3, 1}, {6, 6, 9}, {6, 7, 2}, {6, 8, 7},
      {7, 3, 4}, {7, 5, 6}, {7, 6, 5}, {7, 8, 8},
      {8, 4, 4}, {8, 7, 1}, {8, 8, 6},
      {9, 5, 9}
    }
    );

    int nfixed = hr_fixed->size() / m;

    Model::t M = new Model("SUDOKU"); auto _M = finally([&]() { M->dispose(); });

    M->setLogHandler([ = ](const std::string & msg) { std::cout << msg << std::flush; }
    ↪);

    Variable::t X = M->variable("X", new_array_ptr<int, 1>({n, n, n}), Domain::
    ↪binary());

    //each value only once per dimension
    for (int d = 0; d < m; d++)
        M->constraint( Expr::sum(X, d), Domain::equalsTo(1.) );

    //each number must appear only once in a block
    for (int k = 0; k < n ; k++)
        for (int i = 0; i < m ; i++)
            for (int j = 0; j < m ; j++)
                M->constraint( Expr::sum( X->slice( new_array_ptr<int, 1>({i * m, j * m, k}),
                ↪new_array_ptr<int, 1>({(i + 1)*m, (j +
                ↪1)*m, k + 1}) ) ),
                Domain::equalsTo(1.) );

    auto fixed = std::shared_ptr< ndarray<int, 2> >( new ndarray<int, 2>( shape(nfixed,
    ↪3) ) );

    for (int i = 0; i < nfixed; i++)

```



```

    for (int d = 0; d < m; d++)
        (*fixed)(i, d) = (*hr_fixed)(i, d) - 1;

M->constraint( X->pick( fixed ) , Domain::equalsTo(1.0) ) ;

M->solve();

//print the solution, if any...
if ( M->getPrimalSolutionStatus() == SolutionStatus::Optimal )
    print_solution(n, X);
else
    std::cout << "No solution found!\n";

return 0;
}

```

The problem instance corresponding to Fig. ?? is hard-coded for the sake of simplicity. It will produce the following output

```

Problem
  Name           : SUDOKU
  Objective sense : min
  Type           : LO (linear optimization problem)
  Constraints     : 350
  Cones          : 0
  Scalar variables : 1000
  Matrix variables : 0
  Integer variables : 729

Optimizer started.
Mixed integer optimizer started.
Threads used: 2
Presolve started.
Presolve terminated. Time = 0.00
Presolved problem: 0 variables, 0 constraints, 0 non-zeros
Presolved problem: 0 general integer, 0 binary, 0 continuous
ClIQUE table size: 0
BRANCHES RELAXS  ACT_NDS  DEPTH    BEST_INT_OBJ          BEST_RELAX_OBJ          REL_GAP(
->%)  TIME
0      1      0      0      0.0000000000e+00      0.0000000000e+00      0.
->00e+00  0.0
An optimal solution satisfying the relative gap tolerance of 1.00e-02(%) has been
->located.
The relative gap is 0.00e+00(%).
An optimal solution satisfying the absolute gap tolerance of 0.00e+00 has been
->located.
The absolute gap is 0.00e+00.

Objective of best integer solution : 0.000000000000e+00
Best objective bound                : -0.000000000000e+00
Construct solution objective        : Not employed
Construct solution # roundings     : 0
User objective cut value           : 0
Number of cuts generated           : 0
Number of branches                 : 0
Number of relaxations solved       : 1
Number of interior point iterations: 0

```

(continues on next page)

```

Number of simplex iterations      : 0
Time spend presolving the root    : 0.00
Time spend in the heuristic       : 0.00
Time spend in the sub optimizers  : 0.00
    Time spend optimizing the root : 0.00
Mixed integer optimizer terminated. Time: 0.02

```

```

Optimizer terminated. Time: 0.02

```

```

| 3 2 9 | 5 4 7 | 6 1 8 |
| 6 5 8 | 9 1 3 | 4 2 7 |
| 4 1 7 | 2 8 6 | 9 5 3 |

```

```

| 9 7 3 | 1 5 2 | 8 4 6 |
| 5 6 2 | 8 7 4 | 3 9 1 |
| 8 4 1 | 6 3 9 | 2 7 5 |

```

```

| 1 9 4 | 3 6 5 | 7 8 2 |
| 7 3 5 | 4 2 8 | 1 6 9 |
| 2 8 6 | 7 9 1 | 5 3 4 |

```

11.8 Travelling Salesman Problem (TSP)

The *Travelling Salesman Problem* is one of the most famous and studied problems in combinatorics and integer optimization. In this case study we shall:

- show how to compactly define a model with *Fusion*;
- implement an iterative algorithm that solves a sequence of optimization problems;
- modify an optimization problem by adding more constraints;
- show how to access the solution of an optimization problem.

The material presented in this section draws inspiration from [Pat03].

In a TSP instance we are given a directed graph $G = (N, A)$, where N is the set of nodes and A is the set of arcs. To each arc $(i, j) \in A$ corresponds a nonnegative cost c_{ij} . The goal is to find a minimum cost *Hamilton cycle* in G , that is a closed tour passing through each node exactly once. For example, consider the small directed graph in Fig. 11.8.

Its corresponding adjacency and cost matrices A and c are:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad c = \begin{bmatrix} 0 & 1 & 0.1 & 0.1 \\ 0.1 & 0 & 1 & 0 \\ 0 & 0.1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Typically, the problem is modeled introducing a set of binary variables x_{ij} such that

$$x_{ij} = \begin{cases} 0 & \text{if arc } (i, j) \text{ is in the tour,} \\ 1 & \text{otherwise.} \end{cases}$$

Now we can introduce the following simple model:

$$\begin{aligned} \min \quad & \sum_{i,j} c_{ij} x_{ij} \\ \text{subject to} \quad & \sum_i x_{ij} = 1 \quad \forall j = 1, \dots, n, \\ & \sum_j x_{ij} = 1 \quad \forall i = 1, \dots, n, \\ & x_{ij} \leq A_{ij} \quad \forall i, j, \\ & x_{ij} \in \{0, 1\} \quad \forall i, j. \end{aligned} \tag{11.31}$$

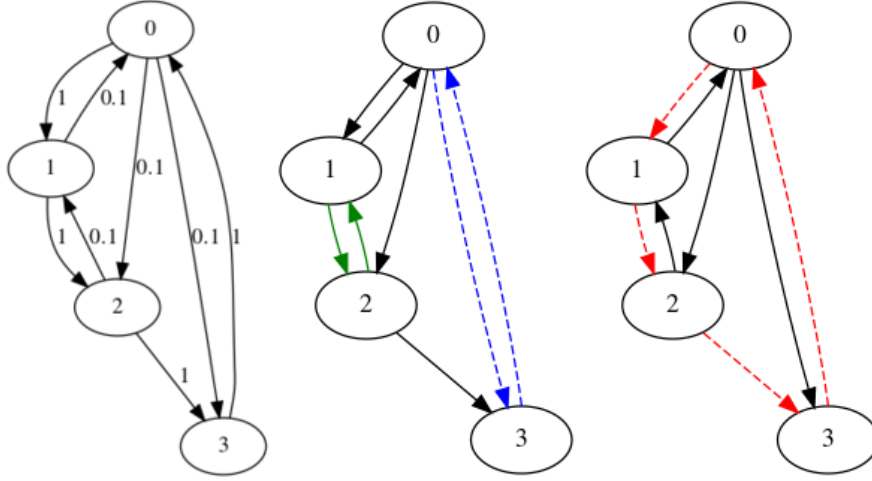


Fig. 11.8: (Left) a directed graph with costs. (Middle) The minimum cycle cover found in the first iteration. (Right) The minimum cost travelling salesman tour.

It describes the constraint that every vertex has exactly one incoming and one outgoing arc in the tour, and that only arcs present in the graph can be chosen. Problem (11.31) can be easily implemented in *Fusion*:

```
Model::t M = new Model();
auto M_ = finally([&]() { M->dispose(); });

auto x = M->variable("x", Set::make(n,n), Domain::binary());

M->constraint(Expr::sum(x, 0), Domain::equalsTo(1.0));
M->constraint(Expr::sum(x, 1), Domain::equalsTo(1.0));
M->constraint(x, Domain::lessThan( A ));

M->objective(ObjectiveSense::Minimize, Expr::dot(C, x));
```

Note in particular how:

- we can sum over rows and/or columns using the *Expr.sum* function;
- we use *Expr.dot* to compute the objective function.

The solution to problem (11.31) is not necessarily a closed tour. In fact (11.31) models another problem known as *minimum cost cycle cover*, whose solution may consist of more than one cycle. In our example we get the solution depicted in Fig. 11.8, i.e. there are two loops, namely $0 \rightarrow 3 \rightarrow 0$ and $1 \rightarrow 2 \rightarrow 1$.

A solution to (11.31) solves the TSP problem if and only if it consists of a single cycle. One classical approach ensuring this is the so-called *subtour elimination*: once we found a solution of (11.31) composed of at least two cycles, we add constraints that explicitly avoid that particular solution:

$$\sum_{(i,j) \in c} x_{ij} \leq |c| - 1 \quad \forall c \in C. \quad (11.32)$$

Thus the problem we want to solve at each step is

$$\begin{aligned} \min \quad & \sum_{i,j} c_{ij} x_{ij} \\ \text{subject to} \quad & \sum_i x_{ij} = 1 & \forall j = 1, \dots, n, \\ & \sum_j x_{ij} = 1 & \forall i = 1, \dots, n, \\ & x_{ij} \leq A_{ij} & \forall i, j, \\ & x_{ij} \in \{0, 1\} & \forall i, j, \\ & \sum_{(i,j) \in c} x_{ij} \leq |c| - 1 & \forall c \in C, \end{aligned} \quad (11.33)$$

where C is the set of cycles in all the cycle covers we have seen so far. The overall solution scheme is the following:

1. set C as the empty set,
2. solve problem (11.33),
3. **if** x has only one cycle **stop**,
4. **else** add the cycles of x to C and **goto** 2.

Cycle detection is a fairly easy task and we omit the procedure here for the sake of simplicity. Now we show how to add a constraint for each cycle. Since we have the list of arcs, and each one corresponds to a variable x_{ij} , we can use the function *Variable.pick* to compactly define constraints of the form (11.32):

```

for (auto c : cycles)
{
    int csize = c.size();

    auto tmp = std::shared_ptr<monty::ndarray<int, 2> >(new ndarray<int, 2>(
↳shape(csize, 2)) );
    for (auto i = 0; i < csize; ++i)
    {
        (*tmp)(i, 0) = std::get<0>(c[i]);
        (*tmp)(i, 1) = std::get<1>(c[i]);
    }

    M->constraint(Expr::sum(x->pick(tmp)), Domain::lessThan( 1.0 * csize - 1 ));
}

```

Executing our procedure will yield the following output:

```

it #1 - solution cost: 2.200000

cycles:
[0,3] - [3,0] -
[1,2] - [2,1] -

it #2 - solution cost: 4.000000

cycles:
[0,1] - [1,2] - [2,3] - [3,0] -

solution:
0 1 0 0
0 0 1 0
0 0 0 1
1 0 0 0

```

Thus we first discover the two-cycle solution; then the second iteration is forced not to include those cycles, and a new solution is located. This time it consists of one loop, and as expected the cost is higher. The solution is depicted in Fig. 11.8.

Formulation (11.33) can be improved in some cases by exploiting the graph structure. Some simple tricks follow.

Self-loops

Self-loops are never part of a TSP tour. Typically self-loops are removed by penalizing them with a huge cost c_{ii} . Although this works in practice, it is more advisable to just fix the corresponding variables to zero, i.e.

$$x_{ii} = 0 \quad \forall i = 1, \dots, n. \quad (11.34)$$

This removes redundant variables, and avoids unnecessarily large coefficients that can negatively affect the solver.

Constraints (11.34) are easily implemented as follows:

```
M->constraint(x->diag(), Domain::equalsTo(0.));
```

Two-arc loops removal

In networks with more than two nodes two-loop arcs can also be ignored. They are simple to detect and their number is of the same order as the size of the graph. The constraints we need to add are:

$$x_{ij} + x_{ji} \leq 1 \quad \forall i, j = 1, \dots, n. \quad (11.35)$$

Constraints (11.35) are easily implemented as follows:

```
M->constraint(Expr::add(x, x->transpose()), Domain::lessThan(1.0));
```

The complete working example

Listing 11.17: The complete code for the TSP examples.

```
void tsp(int n, Matrix::t A, Matrix::t C, bool remove_1_hop_loops, bool remove_2_hop_
→loops)
{
    Model::t M = new Model();
    auto M_ = finally([&]() { M->dispose(); });

    auto x = M->variable("x", Set::make(n,n), Domain::binary());

    M->constraint(Expr::sum(x, 0), Domain::equalsTo(1.0));
    M->constraint(Expr::sum(x, 1), Domain::equalsTo(1.0));
    M->constraint(x, Domain::lessThan( A ));

    M->objective(ObjectiveSense::Minimize, Expr::dot(C, x));

    if (remove_1_hop_loops)
        M->constraint(x->diag(), Domain::equalsTo(0.));

    if (remove_2_hop_loops)
        M->constraint(Expr::add(x, x->transpose()), Domain::lessThan(1.0));

    int it = 0;
    while (true)
    {
        M->solve();
        it++;

        typedef std::vector< std::tuple<int, int> > cycle_t;
        std::list< cycle_t > cycles;
```

(continues on next page)

```

auto xlevel = x->level();

for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    {
        if ( (*xlevel)[i * n + j] <= 0.5 )
            continue;

        bool found = false;
        for (auto && c : cycles)
        {
            for (auto && cc : c)
            {
                if ( i == std::get<0>(cc) || i == std::get<1>(cc) ||
                    j == std::get<0>(cc) || j == std::get<1>(cc) )
                {
                    c.push_back( std::make_tuple(i, j) );
                    found = true;
                    break;
                }
            }
            if (found) break;
        }

        if (!found)
            cycles.push_back(cycle_t(1, std::make_tuple(i, j)));
    }

std::cout << "Iteration " << it << "\n";
for (auto c : cycles) {
    for (auto cc : c)
        std::cout << "(" << std::get<0>(cc) << "," << std::get<1>(cc) << ") ";
    std::cout << "\n";
}

if (cycles.size() == 1) break;

for (auto c : cycles)
{
    int csize = c.size();

    auto tmp = std::shared_ptr<monty::ndarray<int, 2> >(new ndarray<int, 2>(
↳shape(csize, 2)) );
    for (auto i = 0; i < csize; ++i)
    {
        (*tmp)(i, 0) = std::get<0>(c[i]);
        (*tmp)(i, 1) = std::get<1>(c[i]);
    }

    M->constraint(Expr::sum(x->pick(tmp)), Domain::lessThan( 1.0 * csize - 1 ));
}
}
try {
    auto xlevel = x->level();
    std::cout << "Solution\n";

```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            std::cout << (int) (*xlevel)(i * n + j);
        std::cout << "\n";
    }
} catch (...) {}
}

int main()
{
    auto A_i = new_array_ptr<int, 1>({0, 1, 2, 3, 1, 0, 2, 0});
    auto A_j = new_array_ptr<int, 1>({1, 2, 3, 0, 0, 2, 1, 3});

    auto C_v = new_array_ptr<double, 1>({1., 1., 1., 1., 0.1, 0.1, 0.1, 0.1});

    int n = 4;
    tsp(n, Matrix::sparse(n, n, A_i, A_j, 1.), Matrix::sparse(n, n, A_i, A_j, C_v),
    →true, false);
    tsp(n, Matrix::sparse(n, n, A_i, A_j, 1.), Matrix::sparse(n, n, A_i, A_j, C_v),
    →true, true);

    return 0;
}

```

11.9 Nearest Correlation Matrix Problem

A *correlation matrix* is a symmetric positive definite matrix with unit diagonal. This term has origins in statistics, since the matrix whose entries are the correlation coefficients of a sequence of random variables has all these properties.

In this section we study variants of the problem of approximating a given symmetric matrix A with correlation matrices:

- find the correlation matrix X nearest to A in the *Frobenius norm*,
- find an approximation of the form $D + X$ where D is a diagonal matrix with positive diagonal and X is a positive semidefinite matrix of low rank, using the combination of Frobenius and *nuclear norm*.

Both problems are related to *portfolio optimization*, where one can often have a matrix A that only approximates the correlations of stocks. For subsequent optimizations one would like to approximate A with a correlation matrix or, in the factor model, with $D + VV^T$ with VV^T of small rank.

11.9.1 Nearest correlation with the Frobenius norm

The Frobenius norm of a real matrix M is defined as

$$\|M\|_F = \left(\sum_{i,j} M_{i,j}^2 \right)^{1/2}$$

and with respect to this norm our optimization problem can be expressed simply as:

$$\begin{aligned} & \text{minimize} && \|A - X\|_F \\ & \text{subject to} && \mathbf{diag}(X) = e, \\ & && X \succeq 0. \end{aligned} \tag{11.36}$$

We can exploit the symmetry of A and X to get a compact vector representation. To this end we make use of the following mapping from a symmetric matrix to a flattened vector containing the (scaled) lower

triangular part of the matrix:

$$\begin{aligned} \text{vec} : \quad & \mathbb{R}^{n \times n} \rightarrow \mathbb{R}^{n(n+1)/2} \\ \text{vec}(M) = & (\alpha_{11}M_{11}, \alpha_{21}M_{21}, \alpha_{22}M_{22}, \dots, \alpha_{n1}M_{n1}, \dots, \alpha_{nn}M_{nn}) \\ \alpha_{ij} = & \begin{cases} 1 & j = i \\ \sqrt{2} & j < i \end{cases} \end{aligned} \quad (11.37)$$

Note that $\|M\|_F = \|\text{vec}(M)\|_2$. The *Fusion* implementation of `vec` is as follows:

Listing 11.18: Implementation of function `vec` in (11.37).

```
Expression::t vec(Expression::t e)
{
    int N = (*e->getShape())[0];
    int dim = N * (N + 1) / 2;

    auto msubi = new_array_ptr<int, 1>(dim);
    auto msubj = new_array_ptr<int, 1>(dim);
    auto mcof = new_array_ptr<double, 1>(dim);

    for (int i = 0, k = 0; i < N; ++i)
        for (int j = 0; j < i + 1; ++j, ++k)
        {
            (*msubi)[k] = k;
            (*msubj)[k] = i * N + j;
            (*mcof)[k] = (i == j) ? 1.0 : std::sqrt(2.0);
        }

    Matrix::t S = Matrix::sparse(N * (N + 1) / 2, N * N, msubi, msubj, mcof);
    return Expr::mul(S, Expr::reshape(e, N * N));
}
```

That leads to an optimization problem with both conic quadratic and semidefinite constraints:

$$\begin{aligned} & \text{minimize} && t \\ & \text{subject to} && (t, \text{vec}(A - X)) \in \mathcal{Q}, \\ & && \text{diag}(X) = e, \\ & && X \succeq 0. \end{aligned} \quad (11.38)$$

Code example

Listing 11.19: Implementation of problem (11.38).

```
void nearestcorr( std::shared_ptr<ndarray<double, 2>> A)
{
    int N = A->size(0);

    // Create a model
    Model::t M = new Model("NearestCorrelation"); auto _M = finally([&]() { M->
    dispose(); });

    // Setting up the variables
    Variable::t X = M->variable("X", Domain::inPSDCone(N));
    Variable::t t = M->variable("t", 1, Domain::unbounded());

    // (t, vec(A-X)) \in Q
    M->constraint( Expr::vstack(t, vec(Expr::sub(A, X))), Domain::inQCone() );
}
```

(continues on next page)


```

// diag(X) = e
M->constraint(X->diag(), Domain::equalsTo(1.0));

// Objective: Minimize t
M->objective(ObjectiveSense::Minimize, t);

// Solve the problem
M->solve();

// Get the solution values
std::cout << "X = \n"; print_mat(std::cout, X->level());
std::cout << "t = " << *(t->level()->begin()) << std::endl;
}

```

We use the following input

Listing 11.20: Input for the nearest correlation problem.

```

int N = 5;
auto A = new_array_ptr<double, 2>(
{ { 0.0, 0.5, -0.1, -0.2, 0.5},
  { 0.5, 1.25, -0.05, -0.1, 0.25},
  { -0.1, -0.05, 0.51, 0.02, -0.05},
  { -0.2, -0.1, 0.02, 0.54, -0.1},
  { 0.5, 0.25, -0.05, -0.1, 1.25}
});

```

The expected output is the following (small differences may apply):

```

X =
[[ 1.          0.50001941 -0.09999994 -0.20000084  0.50001941]
 [ 0.50001941  1.          -0.04999551 -0.09999154  0.24999101]
 [-0.09999994 -0.04999551  1.          0.01999746 -0.04999551]
 [-0.20000084 -0.09999154  0.01999746  1.          -0.09999154]
 [ 0.50001941  0.24999101 -0.04999551 -0.09999154  1.          ]]

```

11.9.2 Nearest Correlation with Nuclear-norm Penalty

Next, we consider the approximation of A of the form $D + X$ where $D = \mathbf{diag}(w)$, $w \geq 0$ and $X \succeq 0$. We will also aim at minimizing the rank of X . This can be approximated by a relaxed linear objective penalizing the trace $\text{Tr}(X)$ (which in this case is the *nuclear norm* of X and happens to be the sum of its eigenvalues).

The combination of these constraints leads to a problem:

$$\begin{aligned} & \text{minimize} && \|X + \mathbf{diag}(w) - A\|_F + \gamma \text{Tr}(X), \\ & \text{subject to} && X \succeq 0, w \geq 0, \end{aligned}$$

where the parameter γ controls the tradeoff between the quality of approximation and the rank of X .

Exploit the mapping vec defined in (11.37) we can express this problem as:

$$\begin{aligned} & \text{minimize} && t + \gamma \text{Tr}(X) \\ & \text{subject to} && (t, \text{vec}(X + \mathbf{diag}(w) - A)) \in \mathcal{Q}, \\ & && X \succeq 0, w \geq 0. \end{aligned} \tag{11.39}$$

Code example

Listing 11.21: Implementation of problem (11.39).

```
void nearestcorr_nn(
    std::shared_ptr<ndarray<double, 2>> A,
    const std::vector<double>          & gammas,
    std::vector<double>                 & res,
    std::vector<double>                 & rank)
{
    int N = A->size(0);

    Model::t M = new Model("NucNorm"); auto M_ = monty::finally([&]() { M->dispose(); }
    ↪);

    // Setup variables
    Variable::t t = M->variable("t", 1, Domain::unbounded());
    Variable::t X = M->variable("X", Domain::inPSDCone(N));
    Variable::t w = M->variable("w", N, Domain::greaterThan(0.0));

    // (t, vec (X + diag(w) - A)) in Q
    Expression::t D = Expr::mulElm( Matrix::eye(N), Var::repeat(w, 1, N) );
    M->constraint( Expr::vstack( t, vec(Expr::sub(Expr::add(X, D), A)) ), Domain::
    ↪inQCone() );

    // Trace(X)
    auto TrX = Expr::sum(X->diag());

    for (int k = 0; k < gammas.size(); ++k)
    {
        // Objective: Minimize t + gamma*Tr(X)
        M->objective(ObjectiveSense::Minimize, Expr::add(t, Expr::mul(gammas[k], TrX)));
        M->solve();

        // Find the eigenvalues of X and approximate its rank
        auto d = new_array_ptr<double, 1>(N);
        mosek::LinAlg::sy eig(MSK_UPLO_LO, N, X->level(), d);
        int rnk = 0; for (int i = 0; i < N; ++i) if ((*d)[i] > 1e-6) ++rnk;

        res[k] = (*(t->level()))[0];
        rank[k] = rnk;
    }
}
```

We feed **MOSEK** with the same input as in Sec. 11.9.1. The problem is solved for a range of values γ values, to demonstrate how the penalty term helps achieve a low rank solution. To this extent we report both the rank of X and the residual norm $\|X + \mathbf{diag}(w) - A\|_F$.

```
--- Nearest Correlation with Nuclear Norm---
gamma=0.000000, res=3.076163e-01, rank=4
gamma=0.100000, res=4.251692e-01, rank=2
gamma=0.200000, res=5.112082e-01, rank=1
gamma=0.300000, res=5.298432e-01, rank=1
gamma=0.400000, res=5.592686e-01, rank=1
gamma=0.500000, res=6.045702e-01, rank=1
gamma=0.600000, res=6.764402e-01, rank=1
gamma=0.700000, res=8.009913e-01, rank=1
gamma=0.800000, res=1.062385e+00, rank=1
```

(continues on next page)

```
gamma=0.900000, res=1.129513e+00, rank=0
gamma=1.000000, res=1.129513e+00, rank=0
```

11.10 Semidefinite Relaxation of MIQCQO Problems

In this case study we will discuss a fairly common application for Semidefinite Optimization: to define a continuous semidefinite relaxation of a mixed-integer quadratic optimization problem. This section is based on the method by Park and Boyd [PB15].

We will focus on problems of the form:

$$\begin{aligned} & \text{minimize} && x^T P x + 2q^T x \\ & \text{subject to} && x \in \mathbb{Z}^n \end{aligned} \quad (11.40)$$

where $q \in \mathbb{R}^n$ and $P \in \mathcal{S}_+^{n \times n}$ is positive semidefinite. There are many important problems that can be reformulated as (11.40), for example:

- *integer least squares*: minimize $\|Ax - b\|_2^2$ subject to $x \in \mathbb{Z}^n$,
- *closest vector problem*: minimize $\|v - z\|_2$ subject to $z \in \{Bx \mid x \in \mathbb{Z}^n\}$.

Following [PB15], we can derive a relaxed continuous model. We first relax the integrality constraint

$$\begin{aligned} & \text{minimize} && x^T P x + 2q^T x \\ & \text{subject to} && x_i(x_i - 1) \geq 0 \quad i = 1, \dots, n. \end{aligned}$$

The last constraint is still non-convex. We introduce a new variable $X \in \mathbb{R}^{n \times n}$, such that $X = x \cdot x^T$. This allows us to write an equivalent formulation:

$$\begin{aligned} & \text{minimize} && \text{Tr}(PX) + 2q^T x \\ & \text{subject to} && \mathbf{diag}(X) \geq x, \\ & && X = x \cdot x^T. \end{aligned}$$

To get a conic problem we relax the last constraint and apply the Schur complement. The final relaxation follows:

$$\begin{aligned} & \text{minimize} && \text{Tr}(PX) + 2q^T x \\ & \text{subject to} && \mathbf{diag}(X) \geq x, \\ & && \begin{bmatrix} X & x \\ x^T & 1 \end{bmatrix} \in \mathcal{S}_+^{n+1}. \end{aligned} \quad (11.41)$$

Fusion Implementation

Implementing model (11.41) in *Fusion* is very simple. We assume the input n , P and q . Then we proceed creating the optimization model

```
Model::t M = new Model();
```

The important step is to define a single PSD variable

$$Z = \begin{bmatrix} X & x \\ x^T & 1 \end{bmatrix} \in \mathcal{S}_+^{n+1}.$$

Our code will create Z and two slices that correspond to X and x :

```
Variable::t Z = M->variable("Z", Domain::inPSDCone(n + 1));

Variable::t X = Z->slice(new_array_ptr<int, 1>({0, 0}), new_array_ptr<int, 1>({n, n}
↪));
Variable::t x = Z->slice(new_array_ptr<int, 1>({0, n}), new_array_ptr<int, 1>({n, n_
↪+ 1}));
```

Then we define the constraints:

```
M->constraint( Expr::sub(X->diag(), x), Domain::greaterThan(0.) );
M->constraint( Z->index(n, n), Domain::equalsTo(1.) );
```

The objective function uses several available linear expressions:

```
M->objective( ObjectiveSense::Minimize, Expr::add(
    Expr::sum( Expr::mulElm( P, X ) ),
    Expr::mul( 2.0, Expr::dot(x, q) )
) );
```

Note that the *trace* operator is not directly available in *Fusion*, but it can easily be defined from scratch.

Complete code

Listing 11.22: *Fusion* implementation of model (11.41).

```
Model::t miqcqp_sdo_relaxation(int n, Matrix::t P, const std::shared_ptr<ndarray
↪<double, 1>> & q) {
    Model::t M = new Model();

    Variable::t Z = M->variable("Z", Domain::inPSDCone(n + 1));

    Variable::t X = Z->slice(new_array_ptr<int, 1>({0, 0}), new_array_ptr<int, 1>({n, n}
↪));
    Variable::t x = Z->slice(new_array_ptr<int, 1>({0, n}), new_array_ptr<int, 1>({n, n}
↪+ 1));

    M->constraint( Expr::sub(X->diag(), x), Domain::greaterThan(0.) );
    M->constraint( Z->index(n, n), Domain::equalsTo(1.) );

    M->objective( ObjectiveSense::Minimize, Expr::add(
        Expr::sum( Expr::mulElm( P, X ) ),
        Expr::mul( 2.0, Expr::dot(x, q) )
    ) );

    return M;
}
```

Numerical Examples

We present now some simple numerical experiments for the integer least squares problem:

$$\begin{aligned} &\text{minimize} && \|Ax - b\|_2^2 \\ &\text{subject to} && x \in \mathbb{Z}^n. \end{aligned} \tag{11.42}$$

It corresponds to the problem (11.40) with $P = A^T A$ and $q = -A^T b$. Following [PB15] we will generate the input data by taking all entries of A from the normal distribution $\mathcal{N}(0, 1)$ and setting $b = Ac$ where c comes from the uniform distribution on $[0, 1]$.

We implement the linear algebra operations using the `LinAlg` module available in **MOSEK**.

An integer rounding `xRound` of the solution to (11.41) is a feasible integer solution to (11.42). We can compare it to the actual optimal integer solution `xOpt`, whenever the latter is available. Of course it is very simple to formulate the integer least squares problem in *Fusion*:

```
Model::t int_least_squares(int n, Matrix::t A, const std::shared_ptr<ndarray<double, 1>>
↪& b) {
    Model::t M = new Model();
```

(continues on next page)

(continued from previous page)

```
Variable::t x = M->variable("x", n, Domain::integral(Domain::unbounded()));
Variable::t t = M->variable("t", 1, Domain::unbounded());

M->constraint( Expr::vstack(t, Expr::sub(Expr::mul(A, x), b)), Domain::inQCone() );
M->objective( ObjectiveSense::Minimize, t );

return M;
}
```

All that remains is to compare the values of the objective function $\|Ax - b\|_2$ for the two solutions.

Listing 11.23: The comparison of two solutions.

```
// problem dimensions
int n = 20;
int m = 2 * n;

auto c = new_array_ptr<double, 1>(n);
auto A = new_array_ptr<double, 1>(n * m);
auto P = new_array_ptr<double, 1>(n * n);
auto b = new_array_ptr<double, 1>(m);
auto q = new_array_ptr<double, 1>(n);

std::generate(A->begin(), A->end(), std::bind(normal_distr, generator));
std::generate(c->begin(), c->end(), std::bind(unif_distr, generator));
std::fill(b->begin(), b->end(), 0.0);
std::fill(q->begin(), q->end(), 0.0);

// P = A^T A
syrk(MSK_UPLO_LO, MSK_TRANSPOSE_YES,
     n, m, 1.0, A, 0., P);
for (int j = 0; j < n; j++) for (int i = j + 1; i < n; i++) (*P)[i * n + j] =
→(*P)[j * n + i];

// q = -P c, b = A c
gemv(MSK_TRANSPOSE_NO, n, n, -1.0, P, c, 0., q);
gemv(MSK_TRANSPOSE_NO, m, n, 1.0, A, c, 0., b);

// Solve the problems
{
    Model::t M = miqcqp_sdo_relaxation(n, Matrix::dense(n, n, P), q);
    Model::t Mint = int_least_squares(n, Matrix::dense(n, m, A)->transpose(), b);
    M->solve();
    Mint->solve();

    auto xRound = M->getVariable("Z")->
        slice(new_array_ptr<int, 1>({0, n}), new_array_ptr<int, 1>({n, n +
→1}))->level();
    for (int i = 0; i < n; i++) (*xRound)[i] = round((*xRound)[i]);
    auto yRound = new_array_ptr<double, 1>(m);
    auto xOpt = Mint->getVariable("x")->level();
    auto yOpt = new_array_ptr<double, 1>(m);
    std::copy(b->begin(), b->end(), yRound->begin());
    std::copy(b->begin(), b->end(), yOpt->begin());
    gemv(MSK_TRANSPOSE_NO, m, n, 1.0, A, xRound, -1.0, yRound); // Ax_round-b
    gemv(MSK_TRANSPOSE_NO, m, n, 1.0, A, xOpt, -1.0, yOpt); // Ax_opt-b
}
```

(continues on next page)

```

    std::cout << M->getSolverDoubleInfo("optimizerTime") << " " << Mint->
↪getSolverDoubleInfo("optimizerTime") << "\n";
    double valRound, valOpt;
    dot(m, yRound, yRound, valRound); dot(m, yOpt, yOpt, valOpt);
    std::cout << sqrt(valRound) << " " << sqrt(valOpt) << "\n";
}

```

Experimentally the objective value for `xRound` approximates the optimal solution with a factor of 1.1-1.4. We refer to [PB15] for a more involved iterative rounding procedure, producing integer solutions of even better quality, and for a detailed discussion of test results.

Chapter 12

Problem Formulation and Solutions

In this chapter we will discuss the following topics:

- The formal, mathematical formulations of the problem types that **MOSEK** can solve and their duals.
- The solution information produced by **MOSEK**.
- The infeasibility certificate produced by **MOSEK** if the problem is infeasible.

For the underlying mathematical concepts, derivations and proofs see the [Modeling Cookbook](#) or any book on convex optimization. This chapter explains how the related data is organized specifically within the **MOSEK** API.

12.1 Linear Optimization

MOSEK accepts linear optimization problems of the form

$$\begin{array}{llllll} \text{minimize} & & c^T x + c^f & & & \\ \text{subject to} & l^c & \leq & Ax & \leq & u^c, \\ & l^x & \leq & x & \leq & u^x, \end{array} \tag{12.1}$$

where

- m is the number of constraints.
- n is the number of decision variables.
- $x \in \mathbb{R}^n$ is a vector of decision variables.
- $c \in \mathbb{R}^n$ is the linear part of the objective function.
- $c^f \in \mathbb{R}$ is a constant term in the objective
- $A \in \mathbb{R}^{m \times n}$ is the constraint matrix.
- $l^c \in \mathbb{R}^m$ is the lower limit on the activity for the constraints.
- $u^c \in \mathbb{R}^m$ is the upper limit on the activity for the constraints.
- $l^x \in \mathbb{R}^n$ is the lower limit on the activity for the variables.
- $u^x \in \mathbb{R}^n$ is the upper limit on the activity for the variables.

Lower and upper bounds can be infinite, or in other words the corresponding bound may be omitted.

A primal solution (x) is *(primal) feasible* if it satisfies all constraints in (12.1). If (12.1) has at least one primal feasible solution, then (12.1) is said to be (primal) feasible. In case (12.1) does not have a feasible solution, the problem is said to be *(primal) infeasible*.

12.1.1 Duality for Linear Optimization

Corresponding to the primal problem (12.1), there is a dual problem

$$\begin{aligned} & \text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x + c^f \\ & \text{subject to} && A^T y + s_l^x - s_u^x = c, \\ & && -y + s_l^c - s_u^c = 0, \\ & && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \end{aligned} \quad (12.2)$$

where

- s_l^c are the dual variables for lower bounds of constraints,
- s_u^c are the dual variables for upper bounds of constraints,
- s_l^x are the dual variables for lower bounds of variables,
- s_u^x are the dual variables for upper bounds of variables.

If a bound in the primal problem is plus or minus infinity, the corresponding dual variable is fixed at 0, and we use the convention that the product of the bound value and the corresponding dual variable is 0. This is equivalent to removing the corresponding dual variable from the dual problem. For example:

$$l_j^x = -\infty \quad \Rightarrow \quad (s_l^x)_j = 0 \text{ and } l_j^x \cdot (s_l^x)_j = 0.$$

A solution

$$(y, s_l^c, s_u^c, s_l^x, s_u^x)$$

to the dual problem is feasible if it satisfies all the constraints in (12.2). If (12.2) has at least one feasible solution, then (12.2) is *(dual) feasible*, otherwise the problem is *(dual) infeasible*.

A solution

$$(x^*, y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*)$$

is denoted a *primal-dual feasible solution*, if (x^*) is a solution to the primal problem (12.1) and $(y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*)$ is a solution to the corresponding dual problem (12.2). We also define an auxiliary vector

$$(x^c)^* := Ax^*$$

containing the activities of linear constraints.

For a primal-dual feasible solution we define the *duality gap* as the difference between the primal and the dual objective value,

$$\begin{aligned} & c^T x^* + c^f - \{ (l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* + c^f \} \\ & = \sum_{i=0}^{m-1} [(s_l^c)^*_i ((x_i^c)^* - l_i^c) + (s_u^c)^*_i (u_i^c - (x_i^c)^*)] \\ & + \sum_{j=0}^{n-1} [(s_l^x)^*_j (x_j^* - l_j^x) + (s_u^x)^*_j (u_j^x - x_j^*)] \geq 0 \end{aligned} \quad (12.3)$$

where the first relation can be obtained by transposing and multiplying the dual constraints (12.2) by x^* and $(x^c)^*$ respectively, and the second relation comes from the fact that each term in each sum is nonnegative. It follows that the primal objective will always be greater than or equal to the dual objective.

It is well-known that a linear optimization problem has an optimal solution if and only if there exist feasible primal-dual solution so that the duality gap is zero, or, equivalently, that the *complementarity conditions*

$$\begin{aligned} (s_l^c)^*_i ((x_i^c)^* - l_i^c) &= 0, & i = 0, \dots, m-1, \\ (s_u^c)^*_i (u_i^c - (x_i^c)^*) &= 0, & i = 0, \dots, m-1, \\ (s_l^x)^*_j (x_j^* - l_j^x) &= 0, & j = 0, \dots, n-1, \\ (s_u^x)^*_j (u_j^x - x_j^*) &= 0, & j = 0, \dots, n-1, \end{aligned}$$

are satisfied.

If (12.1) has an optimal solution and **MOSEK** solves the problem successfully, both the primal and dual solution are reported, including a status indicating the exact state of the solution.

12.1.2 Infeasibility for Linear Optimization

Primal Infeasible Problems

If the problem (12.1) is infeasible (has no feasible solution), **MOSEK** will report a certificate of primal infeasibility: The dual solution reported is the certificate of infeasibility, and the primal solution is undefined.

A certificate of primal infeasibility is a feasible solution to the modified dual problem

$$\begin{aligned} & \text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x \\ & \text{subject to} && A^T y + s_l^x - s_u^x = 0, \\ & && -y + s_l^c - s_u^c = 0, \\ & && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \end{aligned} \tag{12.4}$$

such that the objective value is strictly positive, i.e. a solution

$$(y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*)$$

to (12.4) so that

$$(l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* > 0.$$

Such a solution implies that (12.4) is unbounded, and that (12.1) is infeasible.

Dual Infeasible Problems

If the problem (12.2) is infeasible (has no feasible solution), **MOSEK** will report a certificate of dual infeasibility: The primal solution reported is the certificate of infeasibility, and the dual solution is undefined.

A certificate of dual infeasibility is a feasible solution to the modified primal problem

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && \hat{l}^c \leq Ax \leq \hat{u}^c, \\ & && \hat{l}^x \leq x \leq \hat{u}^x, \end{aligned} \tag{12.5}$$

where

$$\hat{l}_i^c = \begin{cases} 0 & \text{if } l_i^c > -\infty, \\ -\infty & \text{otherwise,} \end{cases} \quad \text{and} \quad \hat{u}_i^c := \begin{cases} 0 & \text{if } u_i^c < \infty, \\ \infty & \text{otherwise,} \end{cases}$$

and

$$\hat{l}_j^x = \begin{cases} 0 & \text{if } l_j^x > -\infty, \\ -\infty & \text{otherwise,} \end{cases} \quad \text{and} \quad \hat{u}_j^x := \begin{cases} 0 & \text{if } u_j^x < \infty, \\ \infty & \text{otherwise,} \end{cases}$$

such that

$$c^T x < 0.$$

Such a solution implies that (12.5) is unbounded, and that (12.2) is infeasible.

In case that both the primal problem (12.1) and the dual problem (12.2) are infeasible, **MOSEK** will report only one of the two possible certificates — which one is not defined (**MOSEK** returns the first certificate found).

12.1.3 Minimalization vs. Maximalization

When the objective sense of problem (12.1) is maximization, i.e.

$$\begin{aligned} & \text{maximize} && c^T x + c^f \\ & \text{subject to} && l^c \leq Ax \leq u^c, \\ & && l^x \leq x \leq u^x, \end{aligned}$$

the objective sense of the dual problem changes to minimization, and the domain of all dual variables changes sign in comparison to (12.2). The dual problem thus takes the form

$$\begin{aligned} & \text{minimize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x + c^f \\ & \text{subject to} && \\ & && A^T y + s_l^x - s_u^x = c, \\ & && -y + s_l^c - s_u^c = 0, \\ & && s_l^c, s_u^c, s_l^x, s_u^x \leq 0. \end{aligned}$$

This means that the duality gap, defined in (12.3) as the primal minus the dual objective value, becomes nonpositive. It follows that the dual objective will always be greater than or equal to the primal objective. The primal infeasibility certificate will be reported by **MOSEK** as a solution to the system

$$\begin{aligned} & A^T y + s_l^x - s_u^x = 0, \\ & -y + s_l^c - s_u^c = 0, \\ & s_l^c, s_u^c, s_l^x, s_u^x \leq 0, \end{aligned} \tag{12.6}$$

such that the objective value is strictly negative

$$(l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* < 0.$$

Similarly, the certificate of dual infeasibility is an x satisfying the requirements of (12.5) such that $c^T x > 0$.

12.2 Conic Optimization

Conic optimization is an extension of linear optimization (see Sec. 12.1) allowing conic domains to be specified for affine expressions. A conic optimization problem to be solved by **MOSEK** can be written as

$$\begin{aligned} & \text{minimize} && c^T x + c^f \\ & \text{subject to} && \begin{array}{lll} l^c & \leq & Ax & \leq & u^c, \\ l^x & \leq & x & \leq & u^x, \\ & & Fx + g & \in & \mathcal{D}, \end{array} \end{aligned} \tag{12.7}$$

where

- m is the number of constraints.
- n is the number of decision variables.
- $x \in \mathbb{R}^n$ is a vector of decision variables.
- $c \in \mathbb{R}^m$ is the linear part of the objective function.
- $c^f \in \mathbb{R}$ is a constant term in the objective
- $A \in \mathbb{R}^{m \times n}$ is the constraint matrix.
- $l^c \in \mathbb{R}^m$ is the lower limit on the activity for the constraints.
- $u^c \in \mathbb{R}^m$ is the upper limit on the activity for the constraints.
- $l^x \in \mathbb{R}^n$ is the lower limit on the activity for the variables.
- $u^x \in \mathbb{R}^n$ is the upper limit on the activity for the variables.

is the same as in Sec. 12.1 and moreover:

- $F \in \mathbb{R}^{k \times n}$ is the affine conic constraint matrix.,
- $g \in \mathbb{R}^k$ is the affine conic constraint constant term vector.,
- $\mathcal{D}$ is a Cartesian product of conic domains, namely $\mathcal{D} = \mathcal{D}_1 \times \cdots \times \mathcal{D}_p$, where p is the number of individual affine conic constraints (ACCs), and each domain is one from Sec. 14.8.

The total dimension of the domain $\mathcal{D}$ must be equal to k , the number of rows in F and g . Lower and upper bounds can be infinite, or in other words the corresponding bound may be omitted.

MOSEK supports also the cone of positive semidefinite matrices. In order not to obscure this section with additional notation, that extension is discussed in Sec. 12.3.

12.2.1 Duality for Conic Optimization

Corresponding to the primal problem (12.7), there is a dual problem

$$\begin{aligned}
& \text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x - g^T \dot{y} + c^f \\
& \text{subject to} && A^T y + s_l^x - s_u^x + F^T \dot{y} = c, \\
& && -y + s_l^c - s_u^c = 0, \\
& && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \\
& && \dot{y} \in \mathcal{D}^*,
\end{aligned} \tag{12.8}$$

where

- s_l^c are the dual variables for lower bounds of constraints,
- s_u^c are the dual variables for upper bounds of constraints,
- s_l^x are the dual variables for lower bounds of variables,
- s_u^x are the dual variables for upper bounds of variables,
- $\dot{y}$ are the dual variables for affine conic constraints,
- the dual domain $\mathcal{D}^* = \mathcal{D}_1^* \times \cdots \times \mathcal{D}_p^*$ is a Cartesian product of cones dual to $\mathcal{D}_i$.

One can check that the dual problem of the dual problem is identical to the original primal problem.

If a bound in the primal problem is plus or minus infinity, the corresponding dual variable is fixed at 0, and we use the convention that the product of the bound value and the corresponding dual variable is 0. This is equivalent to removing the corresponding dual variable $(s_l^x)_j$ from the dual problem. For example:

$$l_j^x = -\infty \quad \Rightarrow \quad (s_l^x)_j = 0 \text{ and } l_j^x \cdot (s_l^x)_j = 0.$$

A solution

$$(y, s_l^c, s_u^c, s_l^x, s_u^x, \dot{y})$$

to the dual problem is feasible if it satisfies all the constraints in (12.8). If (12.8) has at least one feasible solution, then (12.8) is *(dual) feasible*, otherwise the problem is *(dual) infeasible*.

A solution

$$(x^*, y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*, (\dot{y})^*)$$

is denoted a *primal-dual feasible solution*, if (x^*) is a solution to the primal problem (12.7) and $(y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*, (\dot{y})^*)$ is a solution to the corresponding dual problem (12.8). We also define an auxiliary vector

$$(x^c)^* := Ax^*$$

containing the activities of linear constraints.

For a primal-dual feasible solution we define the *duality gap* as the difference between the primal and the dual objective value,

$$\begin{aligned}
& c^T x^* + c^f - \{ (l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* - g^T (\dot{y})^* + c^f \} \\
& = \sum_{i=0}^{m-1} [(s_l^c)_i^* ((x_i^c)^* - l_i^c) + (s_u^c)_i^* (u_i^c - (x_i^c)^*)] \\
& + \sum_{j=0}^{n-1} [(s_l^x)_j^* (x_j - l_j^x) + (s_u^x)_j^* (u_j^x - x_j^*)] \\
& + ((\dot{y})^*)^T (Fx^* + g) \geq 0
\end{aligned} \tag{12.9}$$

where the first relation can be obtained by transposing and multiplying the dual constraints (12.2) by x^* and $(x^c)^*$ respectively, and the second relation comes from the fact that each term in each sum is nonnegative. It follows that the primal objective will always be greater than or equal to the dual objective.

It is well-known that, under some non-degeneracy assumptions that exclude ill-posed cases, a conic optimization problem has an optimal solution if and only if there exist feasible primal-dual solution so that the duality gap is zero, or, equivalently, that the *complementarity conditions*

$$\begin{aligned} (s_l^c)_i^* ((x_i^c)^* - l_i^c) &= 0, & i = 0, \dots, m-1, \\ (s_u^c)_i^* (u_i^c - (x_i^c)^*) &= 0, & i = 0, \dots, m-1, \\ (s_l^x)_j^* (x_j^* - l_j^x) &= 0, & j = 0, \dots, n-1, \\ (s_u^x)_j^* (u_j^x - x_j^*) &= 0, & j = 0, \dots, n-1, \\ ((y)^*)^T (Fx^* + g) &= 0, \end{aligned} \tag{12.10}$$

are satisfied.

If (12.7) has an optimal solution and **MOSEK** solves the problem successfully, both the primal and dual solution are reported, including a status indicating the exact state of the solution.

12.2.2 Infeasibility for Conic Optimization

Primal Infeasible Problems

If the problem (12.7) is infeasible (has no feasible solution), **MOSEK** will report a certificate of primal infeasibility: The dual solution reported is the certificate of infeasibility, and the primal solution is undefined.

A certificate of primal infeasibility is a feasible solution to the modified dual problem

$$\begin{aligned} &\text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x - g^T \dot{y} \\ &\text{subject to} && A^T y + s_l^x - s_u^x + F^T \dot{y} = 0, \\ & && -y + s_l^c - s_u^c = 0, \\ & && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \\ & && \dot{y} \in \mathcal{D}^*, \end{aligned} \tag{12.11}$$

such that the objective value is strictly positive, i.e. a solution

$$(y^*, (s_l^c)^*, (s_u^c)^*, (s_l^x)^*, (s_u^x)^*, (\dot{y})^*)$$

to (12.11) so that

$$(l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* - g^T \dot{y} > 0.$$

Such a solution implies that (12.11) is unbounded, and that (12.7) is infeasible.

Dual Infeasible Problems

If the problem (12.8) is infeasible (has no feasible solution), **MOSEK** will report a certificate of dual infeasibility: The primal solution reported is the certificate of infeasibility, and the dual solution is undefined.

A certificate of dual infeasibility is a feasible solution to the modified primal problem

$$\begin{aligned} &\text{minimize} && c^T x \\ &\text{subject to} && \hat{l}^c \leq Ax \leq \hat{u}^c, \\ & && \hat{l}^x \leq x \leq \hat{u}^x, \\ & && Fx \in \mathcal{D} \end{aligned} \tag{12.12}$$

where

$$\hat{l}_i^c = \begin{cases} 0 & \text{if } l_i^c > -\infty, \\ -\infty & \text{otherwise,} \end{cases} \quad \text{and} \quad \hat{u}_i^c := \begin{cases} 0 & \text{if } u_i^c < \infty, \\ \infty & \text{otherwise,} \end{cases} \tag{12.13}$$

and

$$\hat{l}_j^x = \begin{cases} 0 & \text{if } l_j^x > -\infty, \\ -\infty & \text{otherwise,} \end{cases} \quad \text{and} \quad \hat{u}_j^x := \begin{cases} 0 & \text{if } u_j^x < \infty, \\ \infty & \text{otherwise,} \end{cases} \tag{12.14}$$

such that

$$c^T x < 0.$$

Such a solution implies that (12.12) is unbounded, and that (12.8) is infeasible.

In case that both the primal problem (12.7) and the dual problem (12.8) are infeasible, **MOSEK** will report only one of the two possible certificates — which one is not defined (**MOSEK** returns the first certificate found).

12.2.3 Minimalization vs. Maximalization

When the objective sense of problem (12.7) is maximization, i.e.

$$\begin{array}{ll} \text{maximize} & c^T x + c^f \\ \text{subject to} & l^c \leq Ax \leq u^c, \\ & l^x \leq x \leq u^x, \\ & Fx + g \in \mathcal{D}, \end{array}$$

the objective sense of the dual problem changes to minimization, and the domain of all dual variables changes sign in comparison to (12.2). The dual problem thus takes the form

$$\begin{array}{ll} \text{minimize} & (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x - g^T \dot{y} + c^f \\ \text{subject to} & A^T y + s_l^x - s_u^x + F^T \dot{y} = c, \\ & -y + s_l^c - s_u^c = 0, \\ & s_l^c, s_u^c, s_l^x, s_u^x \leq 0, \\ & -\dot{y} \in \mathcal{D}^* \end{array}$$

This means that the duality gap, defined in (12.9) as the primal minus the dual objective value, becomes nonpositive. It follows that the dual objective will always be greater than or equal to the primal objective. The primal infeasibility certificate will be reported by **MOSEK** as a solution to the system

$$\begin{aligned} A^T y + s_l^x - s_u^x + F^T \dot{y} &= 0, \\ -y + s_l^c - s_u^c &= 0, \\ s_l^c, s_u^c, s_l^x, s_u^x &\leq 0, \\ -\dot{y} &\in \mathcal{D}^* \end{aligned} \tag{12.15}$$

such that the objective value is strictly negative

$$(l^c)^T (s_l^c)^* - (u^c)^T (s_u^c)^* + (l^x)^T (s_l^x)^* - (u^x)^T (s_u^x)^* - g^T \dot{y} < 0.$$

Similarly, the certificate of dual infeasibility is an x satisfying the requirements of (12.12) such that $c^T x > 0$.

12.3 Semidefinite Optimization

Semidefinite optimization is an extension of conic optimization (see Sec. 12.2) allowing positive semidefinite matrix variables to be used in addition to the usual scalar variables. All the other parts of the input are defined exactly as in Sec. 12.2, and the discussion from that section applies verbatim to all properties of problems with semidefinite variables. We only briefly indicate how the corresponding formulae should be modified with semidefinite terms.

A semidefinite optimization problem can be written as

$$\begin{array}{ll} \text{minimize} & c^T x + \langle \overline{C}, \overline{X} \rangle + c^f \\ \text{subject to} & l^c \leq Ax + \langle \overline{A}, \overline{X} \rangle \leq u^c, \\ & l^x \leq x \leq u^x, \\ & Fx + \langle \overline{F}, \overline{X} \rangle + g \in \mathcal{D}, \\ & \overline{X}_j \in \mathcal{S}_+^{r_j}, j = 1, \dots, s \end{array}$$

where

- m is the number of constraints.
- n is the number of decision variables.
- $x \in \mathbb{R}^n$ is a vector of decision variables.
- $c \in \mathbb{R}^n$ is the linear part of the objective function.
- $c^f \in \mathbb{R}$ is a constant term in the objective
- $A \in \mathbb{R}^{m \times n}$ is the constraint matrix.
- $l^c \in \mathbb{R}^m$ is the lower limit on the activity for the constraints.
- $u^c \in \mathbb{R}^m$ is the upper limit on the activity for the constraints.
- $l^x \in \mathbb{R}^n$ is the lower limit on the activity for the variables.
- $u^x \in \mathbb{R}^n$ is the upper limit on the activity for the variables.
- $F \in \mathbb{R}^{k \times n}$ is the affine conic constraint matrix.,
- $g \in \mathbb{R}^k$ is the affine conic constraint constant term vector.,
- $\mathcal{D}$ is a Cartesian product of conic domains, namely $\mathcal{D} = \mathcal{D}_1 \times \cdots \times \mathcal{D}_p$, where p is the number of individual affine conic constraints (ACCs), and each domain is one from [Sec. 14.8](#).

is the same as in [Sec. 12.2](#) and moreover:

- there are s symmetric positive semidefinite variables, the j -th of which is $\bar{X}_j \in \mathcal{S}_+^{r_j}$ of dimension r_j ,
- $\bar{C} = (\bar{C}_j)_{j=1,\dots,s}$ is a collection of symmetric coefficient matrices in the objective, with $\bar{C}_j \in \mathcal{S}^{r_j}$, and we interpret the notation $\langle \bar{C}, \bar{X} \rangle$ as a shorthand for

$$\langle \bar{C}, \bar{X} \rangle := \sum_{j=1}^s \langle \bar{C}_j, \bar{X}_j \rangle.$$

- $\bar{A} = (\bar{A}_{ij})_{i=1,\dots,m,j=1,\dots,s}$ is a collection of symmetric coefficient matrices in the constraints, with $\bar{A}_{ij} \in \mathcal{S}^{r_j}$, and we interpret the notation $\langle \bar{A}, \bar{X} \rangle$ as a shorthand for the vector

$$\langle \bar{A}, \bar{X} \rangle := \left(\sum_{j=1}^s \langle \bar{A}_{ij}, \bar{X}_j \rangle \right)_{i=1,\dots,m}.$$

- $\bar{F} = (\bar{F}_{ij})_{i=1,\dots,k,j=1,\dots,s}$ is a collection of symmetric coefficient matrices in the affine conic constraints, with $\bar{F}_{ij} \in \mathcal{S}^{r_j}$, and we interpret the notation $\langle \bar{F}, \bar{X} \rangle$ as a shorthand for the vector

$$\langle \bar{F}, \bar{X} \rangle := \left(\sum_{j=1}^s \langle \bar{F}_{ij}, \bar{X}_j \rangle \right)_{i=1,\dots,k}.$$

In each case the matrix inner product between symmetric matrices of the same dimension r is defined as

$$\langle U, V \rangle := \sum_{i=1}^r \sum_{j=1}^r U_{ij} V_{ij}.$$

To summarize, above the formulation extends that from [Sec. 12.2](#) by the possibility of including semidefinite terms in the objective, constraints and affine conic constraints.

Duality

The definition of the dual problem (12.8) becomes:

$$\begin{aligned}
& \text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x - g^T \dot{y} + c^f \\
& \text{subject to} && A^T y + s_l^x - s_u^x + F^T \dot{y} = c, \\
& && -y + s_l^c - s_u^c = 0, \\
& && \bar{C}_j - \sum_{i=1}^m y_i \bar{A}_{ij} - \sum_{i=1}^k \dot{y}_i \bar{F}_{ij} = S_j, \quad j = 1, \dots, s, \\
& && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \\
& && \dot{y} \in \mathcal{D}^*, \\
& && \bar{S}_j \in \mathcal{S}_+^{r_j}, \quad j = 1, \dots, s.
\end{aligned} \tag{12.16}$$

Complementarity conditions (12.10) include the additional relation:

$$\langle \bar{X}_j, \bar{S}_j \rangle = 0 \quad j = 1, \dots, s. \tag{12.17}$$

Infeasibility

A certificate of primal infeasibility (12.11) is now a feasible solution to:

$$\begin{aligned}
& \text{maximize} && (l^c)^T s_l^c - (u^c)^T s_u^c + (l^x)^T s_l^x - (u^x)^T s_u^x - g^T \dot{y} \\
& \text{subject to} && A^T y + s_l^x - s_u^x + F^T \dot{y} = 0, \\
& && -y + s_l^c - s_u^c = 0, \\
& && -\sum_{i=1}^m y_i \bar{A}_{ij} - \sum_{i=1}^k \dot{y}_i \bar{F}_{ij} = S_j, \quad j = 1, \dots, s, \\
& && s_l^c, s_u^c, s_l^x, s_u^x \geq 0, \\
& && \dot{y} \in \mathcal{D}^*, \\
& && \bar{S}_j \in \mathcal{S}_+^{r_j}, \quad j = 1, \dots, s.
\end{aligned} \tag{12.18}$$

such that the objective value is strictly positive.

Similarly, a dual infeasibility certificate (12.12) is a feasible solution to

$$\begin{aligned}
& \text{minimize} && c^T x + \langle \bar{C}, \bar{X} \rangle \\
& \text{subject to} && \hat{l}^c \leq Ax + \langle \bar{A}, \bar{X} \rangle \leq \hat{u}^c, \\
& && \hat{l}^x \leq x \leq \hat{u}^x, \\
& && Fx + \langle \bar{F}, \bar{X} \rangle \in \mathcal{D}, \\
& && \bar{X}_j \in \mathcal{S}_+^{r_j}, j = 1, \dots, s
\end{aligned} \tag{12.19}$$

where the modified bounds are as in (12.13) and (12.14) and the objective value is strictly negative.

Below is an outline of the different problem types for quick reference.

Continuous problem formulations

• Linear optimization (LO)

$$\begin{aligned}
& \text{minimize} && c^T x + c^f \\
& \text{subject to} && l^c \leq Ax \leq u^c, \\
& && l^x \leq x \leq u^x.
\end{aligned}$$

• Conic optimization (CO)

Conic optimization extends linear optimization with *affine conic constraints* (ACC):

$$\begin{aligned}
& \text{minimize} && c^T x + c^f \\
& \text{subject to} && l^c \leq Ax \leq u^c, \\
& && l^x \leq x \leq u^x, \\
& && Fx + g \in \mathcal{D},
\end{aligned}$$

where $\mathcal{D}$ is a product of domains from Sec. 14.8.

- **Semidefinite optimization (SDO)**

A conic optimization problem can be further extended with *semidefinite variables*:

$$\begin{array}{ll}
\text{minimize} & c^T x + \langle \overline{C}, \overline{X} \rangle + c^f \\
\text{subject to} & \begin{array}{ll} l^c & \leq Ax + \langle \overline{A}, \overline{X} \rangle & \leq u^c, \\ l^x & \leq x & \leq u^x, \\ & Fx + \langle \overline{F}, \overline{X} \rangle + g & \in \mathcal{D}, \\ & \overline{X} & \in \mathcal{S}_+, \end{array}
\end{array}$$

where $\mathcal{D}$ is a product of domains from Sec. 14.8 and $\mathcal{S}_+$ is a product of PSD cones meaning that $\overline{X}$ is a sequence of PSD matrix variables.

Mixed-integer extensions

Continuous problems can be extended with constraints requiring the mixed-integer optimizer. We outline them briefly here. The continuous part of a mixed-integer problem is formulated according to one of the continuous types above, however only the primal information and solution fields are relevant, there are no dual values and no infeasibility certificates.

- **Integer variables.** Specifies that a subset of variables take integer values, that is

$$x_I \in \mathbb{Z}$$

for some index set I . Available for problems of type LO, CO, QO and QCQO.

- **Disjunctive constraints.** Appends disjunctions of the form

$$\bigvee_{i=1}^t \bigwedge_{j=1}^{s_i} (D_{ij}x + d_{ij} \in \mathcal{D}_{ij})$$

ie. a disjunction of conjunctions of linear constraints, where each $D_{ij}x + d_{ij}$ is an affine expression of the optimization variables and each $\mathcal{D}_{ij}$ is an affine domain. Available for problems of type LO and CO.

Chapter 13

Optimizers

The most essential part of **MOSEK** are the optimizers:

- *primal simplex* (linear problems),
- *dual simplex* (linear problems),
- *interior-point* (linear, quadratic and conic problems),
- *mixed-integer* (problems with integer variables).

The structure of a successful optimization process is roughly:

- **Presolve**
 1. *Elimination*: Reduce the size of the problem.
 2. *Dualizer*: Choose whether to solve the primal or the dual form of the problem.
 3. *Scaling*: Scale the problem for better numerical stability.
- **Optimization**
 1. *Optimize*: Solve the problem using selected method.
 2. *Terminate*: Stop the optimization when specific termination criteria have been met.
 3. *Report*: Return the solution or an infeasibility certificate.

The preprocessing stage is transparent to the user, but useful to know about for tuning purposes. The purpose of the preprocessing steps is to make the actual optimization more efficient and robust. We discuss the details of the above steps in the following sections.

13.1 Presolve

Before an optimizer actually performs the optimization the problem is preprocessed using the so-called presolve. The purpose of the presolve is to

1. remove redundant constraints,
2. eliminate fixed variables,
3. remove linear dependencies,
4. substitute out (implied) free variables, and
5. reduce the size of the optimization problem in general.

After the presolved problem has been optimized the solution is automatically postsolved so that the returned solution is valid for the original problem. Hence, the presolve is completely transparent. For further details about the presolve phase, please see [AA95] and [AGMeszarosX96].

It is possible to fine-tune the behavior of the presolve or to turn it off entirely. If presolve consumes too much time or memory compared to the reduction in problem size gained it may be disabled. This is done by setting the parameter `presolveUse` to `"off"`.

In the following we describe in more detail the presolve applied to continuous, i.e., linear and conic optimization problems, see Sec. 13.2 and Sec. 13.3. The mixed-integer optimizer, Sec. 13.4, applies similar techniques. The two most time-consuming steps of the presolve for continuous optimization problems are

- the eliminator, and
- the linear dependency check.

Therefore, in some cases it is worthwhile to disable one or both of these.

Numerical issues in the presolve

During the presolve the problem is reformulated so that it hopefully solves faster. However, in rare cases the presolved problem may be harder to solve than the original problem. The presolve may also be infeasible although the original problem is not. If it is suspected that presolved problem is much harder to solve than the original, we suggest to first turn the eliminator off by setting the parameter `presolveEliminatorMaxNumTries` to 0. If that does not help, then trying to turn entire presolve off may help.

Since all computations are done in finite precision, the presolve employs some tolerances when concluding a variable is fixed or a constraint is redundant. If it happens that **MOSEK** incorrectly concludes a problem is primal or dual infeasible, then it is worthwhile to try to reduce the parameters `presolveTolX` and `presolveTolS`. However, if reducing the parameters actually helps then this should be taken as an indication that the problem is badly formulated.

Eliminator

The purpose of the eliminator is to eliminate free and implied free variables from the problem using substitution. For instance, given the constraints

$$\begin{aligned} y &= \sum_j x_j, \\ y, x &\geq 0, \end{aligned}$$

y is an implied free variable that can be substituted out of the problem, if deemed worthwhile. If the eliminator consumes too much time or memory compared to the reduction in problem size gained it may be disabled. This can be done by setting the parameter `presolveEliminatorMaxNumTries` to 0. In rare cases the eliminator may cause that the problem becomes much hard to solve.

Linear dependency checker

The purpose of the linear dependency check is to remove linear dependencies among the linear equalities. For instance, the three linear equalities

$$\begin{aligned} x_1 + x_2 + x_3 &= 1, \\ x_1 + 0.5x_2 &= 0.5, \\ 0.5x_2 + x_3 &= 0.5. \end{aligned}$$

contain exactly one linear dependency. This implies that one of the constraints can be dropped without changing the set of feasible solutions. Removing linear dependencies is in general a good idea since it reduces the size of the problem. Moreover, the linear dependencies are likely to introduce numerical problems in the optimization phase. It is best practice to build models without linear dependencies, but that is not always easy for the user to control. If the linear dependencies are removed at the modeling stage, the linear dependency check can safely be disabled by setting the parameter `presolveLindepUse` to `"off"`.

Dualizer

All linear, conic, and convex optimization problems have an equivalent dual problem associated with them. **MOSEK** has built-in heuristics to determine if it is more efficient to solve the primal or dual problem. The form (primal or dual) is displayed in the **MOSEK** log and available as an information item from the solver. Should the internal heuristics not choose the most efficient form of the problem it may be worthwhile to set the dualizer manually by setting the parameters:

- `intpntSolveForm`: In case of the interior-point optimizer.
- `simSolveForm`: In case of the simplex optimizer.

Note that currently only linear and conic (but not semidefinite) problems may be automatically dualized.

Scaling

Problems containing data with large and/or small coefficients, say $1.0e + 9$ or $1.0e - 7$, are often hard to solve. Significant digits may be truncated in calculations with finite precision, which can result in the optimizer relying on inaccurate data. Since computers work in finite precision, extreme coefficients should be avoided. In general, data around the same *order of magnitude* is preferred, and we will refer to a problem, satisfying this loose property, as being *well-scaled*. If the problem is not well scaled, **MOSEK** will try to scale (multiply) constraints and variables by suitable constants. **MOSEK** solves the scaled problem to improve the numerical properties.

The scaling process is transparent, i.e. the solution to the original problem is reported. It is important to be aware that the optimizer terminates when the termination criterion is met on the scaled problem, therefore significant primal or dual infeasibilities may occur after unscaling for badly scaled problems. The best solution of this issue is to reformulate the problem, making it better scaled.

By default **MOSEK** heuristically chooses a suitable scaling. The scaling for interior-point and simplex optimizers can be controlled with the parameters `intpntScaling` and `simScaling` respectively.

13.2 Linear Optimization

13.2.1 Optimizer Selection

Two different types of optimizers are available for linear problems: The default is an interior-point method, and the alternative is the simplex method (primal or dual). The optimizer can be selected using the parameter `optimizer`.

The Interior-point or the Simplex Optimizer?

Given a linear optimization problem, which optimizer is the best: the simplex or the interior-point optimizer? It is impossible to provide a general answer to this question. However, the interior-point optimizer behaves more predictably: it tends to use between 20 and 100 iterations, almost independently of problem size, but cannot perform warm-start. On the other hand the simplex method can take advantage of an initial solution, but is less predictable from cold-start. The interior-point optimizer is used by default.

The Primal or the Dual Simplex Variant?

MOSEK provides both a primal and a dual simplex optimizer. Predicting which simplex optimizer is faster is impossible, however, in recent years the dual optimizer has seen several algorithmic and computational improvements, which, in our experience, make it faster on average than the primal version. Still, it depends much on the problem structure and size. Setting the `optimizer` parameter to `"freeSimplex"` instructs **MOSEK** to choose one of the simplex variants automatically.

To summarize, if you want to know which optimizer is faster for a given problem type, it is best to try all the options.

13.2.2 The Interior-point Optimizer

The purpose of this section is to provide information about the algorithm employed in the **MOSEK** interior-point optimizer for linear problems and about its termination criteria.

The homogeneous primal-dual problem

In order to keep the discussion simple it is assumed that **MOSEK** solves linear optimization problems of standard form

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && Ax = b, \\ & && x \geq 0. \end{aligned} \tag{13.1}$$

This is in fact what happens inside **MOSEK**; for efficiency reasons **MOSEK** converts the problem to standard form before solving, then converts it back to the input form when reporting the solution.

Since it is not known beforehand whether problem (13.1) has an optimal solution, is primal infeasible or is dual infeasible, the optimization algorithm must deal with all three situations. This is the reason why **MOSEK** solves the so-called homogeneous model

$$\begin{aligned} Ax - b\tau &= 0, \\ A^T y + s - c\tau &= 0, \\ -c^T x + b^T y - \kappa &= 0, \\ x, s, \tau, \kappa &\geq 0, \end{aligned} \tag{13.2}$$

where y and s correspond to the dual variables in (13.1), and τ and κ are two additional scalar variables. Note that the homogeneous model (13.2) always has solution since

$$(x, y, s, \tau, \kappa) = (0, 0, 0, 0, 0)$$

is a solution, although not a very interesting one. Any solution

$$(x^*, y^*, s^*, \tau^*, \kappa^*)$$

to the homogeneous model (13.2) satisfies

$$x_j^* s_j^* = 0 \text{ and } \tau^* \kappa^* = 0.$$

Moreover, there is always a solution that has the property $\tau^* + \kappa^* > 0$.

First, assume that $\tau^* > 0$. It follows that

$$\begin{aligned} A \frac{x^*}{\tau^*} &= b, \\ A^T \frac{y^*}{\tau^*} + \frac{s^*}{\tau^*} &= c, \\ -c^T \frac{x^*}{\tau^*} + b^T \frac{y^*}{\tau^*} &= 0, \\ x^*, s^*, \tau^*, \kappa^* &\geq 0. \end{aligned}$$

This shows that $\frac{x^*}{\tau^*}$ is a primal optimal solution and $(\frac{y^*}{\tau^*}, \frac{s^*}{\tau^*})$ is a dual optimal solution; this is reported as the optimal interior-point solution since

$$(x, y, s) = \left\{ \frac{x^*}{\tau^*}, \frac{y^*}{\tau^*}, \frac{s^*}{\tau^*} \right\}$$

is a primal-dual optimal solution (see [Sec. 12.1](#) for the mathematical background on duality and optimality).

On other hand, if $\kappa^* > 0$ then

$$\begin{aligned} Ax^* &= 0, \\ A^T y^* + s^* &= 0, \\ -c^T x^* + b^T y^* &= \kappa^*, \\ x^*, s^*, \tau^*, \kappa^* &\geq 0. \end{aligned}$$

This implies that at least one of

$$c^T x^* < 0 \quad (13.3)$$

or

$$b^T y^* > 0 \quad (13.4)$$

is satisfied. If (13.3) is satisfied then x^* is a certificate of dual infeasibility, whereas if (13.4) is satisfied then y^* is a certificate of primal infeasibility.

In summary, by computing an appropriate solution to the homogeneous model, all information required for a solution to the original problem is obtained. A solution to the homogeneous model can be computed using a primal-dual interior-point algorithm [And09].

Interior-point Termination Criterion

For efficiency reasons it is not practical to solve the homogeneous model exactly. Hence, an exact optimal solution or an exact infeasibility certificate cannot be computed and a reasonable termination criterion has to be employed.

In the k -th iteration of the interior-point algorithm a trial solution

$$(x^k, y^k, s^k, \tau^k, \kappa^k)$$

to homogeneous model is generated, where

$$x^k, s^k, \tau^k, \kappa^k > 0.$$

Optimal case

Whenever the trial solution satisfies the criterion

$$\begin{aligned} \left\| A \frac{x^k}{\tau^k} - b \right\|_\infty &\leq \epsilon_p (1 + \|b\|_\infty), \\ \left\| A^T \frac{y^k}{\tau^k} + \frac{s^k}{\tau^k} - c \right\|_\infty &\leq \epsilon_d (1 + \|c\|_\infty), \text{ and} \\ \min \left(\frac{(x^k)^T s^k}{(\tau^k)^2}, \left| \frac{c^T x^k}{\tau^k} - \frac{b^T y^k}{\tau^k} \right| \right) &\leq \epsilon_g \max \left(1, \frac{\min(|c^T x^k|, |b^T y^k|)}{\tau^k} \right), \end{aligned} \quad (13.5)$$

the interior-point optimizer is terminated and

$$\frac{(x^k, y^k, s^k)}{\tau^k}$$

is reported as the primal-dual optimal solution. The interpretation of (13.5) is that the optimizer is terminated if

- $\frac{x^k}{\tau^k}$ is approximately primal feasible,
- $\left\{ \frac{y^k}{\tau^k}, \frac{s^k}{\tau^k} \right\}$ is approximately dual feasible, and
- the duality gap is almost zero.

Dual infeasibility certificate

On the other hand, if the trial solution satisfies

$$-\epsilon_i c^T x^k > \frac{\|c\|_\infty}{\max(1, \|b\|_\infty)} \|Ax^k\|_\infty$$

then the problem is declared dual infeasible and x^k is reported as a certificate of dual infeasibility. The motivation for this stopping criterion is as follows: First assume that $\|Ax^k\|_\infty = 0$; then x^k is an exact certificate of dual infeasibility. Next assume that this is not the case, i.e.

$$\|Ax^k\|_\infty > 0,$$

and define

$$\bar{x} := \epsilon_i \frac{\max(1, \|b\|_\infty)}{\|Ax^k\|_\infty \|c\|_\infty} x^k.$$

It is easy to verify that

$$\|A\bar{x}\|_\infty = \epsilon_i \frac{\max(1, \|b\|_\infty)}{\|c\|_\infty} \text{ and } -c^T \bar{x} > 1,$$

which shows $\bar{x}$ is an approximate certificate of dual infeasibility, where ϵ_i controls the quality of the approximation. A smaller value means a better approximation.

Primal infeasibility certificate

Finally, if

$$\epsilon_i b^T y^k > \frac{\|b\|_\infty}{\max(1, \|c\|_\infty)} \|A^T y^k + s^k\|_\infty$$

then y^k is reported as a certificate of primal infeasibility.

Adjusting optimality criteria

It is possible to adjust the tolerances ε_p , ε_d , ε_g and ε_i using parameters; see table for details.

Table 13.1: Parameters employed in termination criterion

ToleranceParameter	name
ε_p	<i>intpntTolPfeas</i>
ε_d	<i>intpntTolDfeas</i>
ε_g	<i>intpntTolRelGap</i>
ε_i	<i>intpntTolInfeas</i>

The default values of the termination tolerances are chosen such that for a majority of problems appearing in practice it is not possible to achieve much better accuracy. Therefore, tightening the tolerances usually is not worthwhile. However, an inspection of (13.5) reveals that the quality of the solution depends on $\|b\|_\infty$ and $\|c\|_\infty$; the smaller the norms are, the better the solution accuracy.

The interior-point method as implemented by **MOSEK** will converge toward optimality and primal and dual feasibility at the same rate [And09]. This means that if the optimizer is stopped prematurely then it is very unlikely that either the primal or dual solution is feasible. Another consequence is that in most cases all the tolerances, ε_p , ε_d , ε_g and ε_i , have to be relaxed together to achieve an effect.

The basis identification discussed in Sec. 13.2.2 requires an optimal solution to work well; hence basis identification should be turned off if the termination criterion is relaxed.

To conclude the discussion in this section, relaxing the termination criterion is usually not worthwhile.

Basis Identification

An interior-point optimizer does not return an optimal basic solution unless the problem has a unique primal and dual optimal solution. Therefore, the interior-point optimizer has an optional post-processing step that computes an optimal basic solution starting from the optimal interior-point solution. More information about the basis identification procedure may be found in [AY96]. In the following we provide an overall idea of the procedure.

There are some cases in which a basic solution could be more valuable:

- a basic solution is often more accurate than an interior-point solution,
- a basic solution can be used to warm-start the simplex algorithm in case of reoptimization,
- a basic solution is in general more sparse, i.e. more variables are fixed to zero. This is particularly appealing when solving continuous relaxations of mixed integer problems, as well as in all applications in which sparser solutions are preferred.

It is easy to see that all feasible solutions are also optimal. In particular, there are two basic solutions, namely

$$\begin{aligned}(x_1^*, y_1^*) &= (1, 0), \\ (x_2^*, y_2^*) &= (0, 1).\end{aligned}$$

In practice, when the algorithm gets close to the optimal solution, it is possible to construct in polynomial time an initial basis for the simplex algorithm from the current interior point solution. This basis is used to warm-start the simplex algorithm that will provide the optimal basic solution. In most cases the constructed basis is optimal, or very few iterations are required by the simplex algorithm to make it optimal and hence the final *clean-up* phase be short. However, for some cases of ill-conditioned problems the additional simplex clean up phase may take of lot a time.

- *intpntBasis*,
- *biIgnoreMaxIter*, and
- *biIgnoreNumError*

The type of simplex algorithm to be used (primal/dual) can be tuned with the parameter `biCleanOptimizer`, and the maximum number of iterations can be set with `biMaxIterations`.

The Interior-point Log

```

Optimizer - threads : 1
Optimizer - solved problem : the dual
Optimizer - Constraints : 2
Optimizer - Cones : 0
Optimizer - Scalar variables : 6 conic : 0
Optimizer - Semi-definite variables: 0 scalarized : 0
Factor - setup time : 0.00 dense det. time : 0.00
Factor - ML order time : 0.00 GP order time : 0.00
Factor - nonzeros before factor : 3 after factor : 3
Factor - dense dim. : 0 flops : 7.
↳00e+001
ITE PFEAS DFEAS GFEAS PRSTATUS POBJ DOBJ MU
↳ TIME
0 1.0e+000 8.6e+000 6.1e+000 1.00e+000 0.000000000e+000 -2.208000000e+003 1.
↳0e+000 0.00
1 1.1e+000 2.5e+000 1.6e-001 0.00e+000 -7.901380925e+003 -7.394611417e+003 2.
↳5e+000 0.00
2 1.4e-001 3.4e-001 2.1e-002 8.36e-001 -8.113031650e+003 -8.055866001e+003 3.3e-
↳001 0.00
3 2.4e-002 5.8e-002 3.6e-003 1.27e+000 -7.777530698e+003 -7.766471080e+003 5.7e-
↳002 0.01
4 1.3e-004 3.2e-004 2.0e-005 1.08e+000 -7.668323435e+003 -7.668207177e+003 3.2e-
↳004 0.01

```

163

(continued from previous page)

```
5  1.3e-008 3.2e-008 2.0e-009 1.00e+000 -7.668000027e+003 -7.668000015e+003 3.2e-
↪008 0.01
6  1.3e-012 3.2e-012 2.0e-013 1.00e+000 -7.667999994e+003 -7.667999994e+003 3.2e-
↪012 0.01
```

The first line displays the number of threads used by the optimizer and the second line indicates if the optimizer chose to solve the primal or dual problem (see [intpntSolveForm](#)). The next lines display the problem dimensions as seen by the optimizer, and the **Factor...** lines show various statistics. This is followed by the iteration log.

Using the same notation as in [Sec. 13.2.2](#) the columns of the iteration log have the following meaning:

- ITE: Iteration index k .
- PFEAS: $\|Ax^k - b\tau^k\|_\infty$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- DFEAS: $\|A^T y^k + s^k - c\tau^k\|_\infty$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- GFEAS: $|-c^T x^k + b^T y^k - \kappa^k|$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- PRSTATUS: This number converges to 1 if the problem has an optimal solution whereas it converges to -1 if that is not the case.
- POBJ: $c^T x^k / \tau^k$. An estimate for the primal objective value.
- DOBJ: $b^T y^k / \tau^k$. An estimate for the dual objective value.
- MU: $\frac{(x^k)^T s^k + \tau^k \kappa^k}{n+1}$. The numbers in this column should always converge to zero.
- TIME: Time spent since the optimization started.

13.2.3 The Simplex Optimizer

An alternative to the interior-point optimizer is the simplex optimizer. The simplex optimizer uses a different method that allows exploiting an initial guess for the optimal solution to reduce the solution time. Depending on the problem it may be faster or slower to use an initial guess; see [Sec. 13.2.1](#) for a discussion. **MOSEK** provides both a primal and a dual variant of the simplex optimizer.

Simplex Termination Criterion

The simplex optimizer terminates when it finds an optimal basic solution or an infeasibility certificate. A basic solution is optimal when it is primal and dual feasible; see [Sec. 12.1](#) for a definition of the primal and dual problem. Due to the fact that computations are performed in finite precision **MOSEK** allows violations of primal and dual feasibility within certain tolerances. The user can control the allowed primal and dual tolerances with the parameters [basisTolX](#) and [basisTolS](#).

Setting the parameter [optimizer](#) to *"freeSimplex"* instructs **MOSEK** to select automatically between the primal and the dual simplex optimizers. Hence, **MOSEK** tries to choose the best optimizer for the given problem and the available solution. The same parameter can also be used to force one of the variants.

Starting From an Existing Solution

When using the simplex optimizer it may be possible to reuse an existing solution and thereby reduce the solution time significantly. When a simplex optimizer starts from an existing solution it is said to perform a *warm-start*. If the user is solving a sequence of optimization problems by solving the problem, making modifications, and solving again, **MOSEK** will warm-start automatically.

By default **MOSEK** uses presolve when performing a warm-start. If the optimizer only needs very few iterations to find the optimal solution it may be better to turn off the presolve.

Numerical Difficulties in the Simplex Optimizers

Though **MOSEK** is designed to minimize numerical instability, completely avoiding it is impossible when working in finite precision. **MOSEK** treats a “numerically unexpected behavior” event inside the optimizer as a *set-back*. The user can define how many set-backs the optimizer accepts; if that number is exceeded, the optimization will be aborted. Set-backs are a way to escape long sequences where the optimizer tries to recover from an unstable situation.

Examples of set-backs are: repeated singularities when factorizing the basis matrix, repeated loss of feasibility, degeneracy problems (no progress in objective) and other events indicating numerical difficulties. If the simplex optimizer encounters a lot of set-backs the problem is usually badly scaled; in such a situation try to reformulate it into a better scaled problem. Then, if a lot of set-backs still occur, trying one or more of the following suggestions may be worthwhile:

- Raise tolerances for allowed primal or dual feasibility: increase the value of
 - *basisTolX*, and
 - *basisTolS*.
- Raise or lower pivot tolerance: Change the *simplexAbsTolPiv* parameter.
- Switch optimizer: Try another optimizer.
- Switch off crash: Set both *simPrimalCrash* and *simDualCrash* to 0.
- Experiment with other pricing strategies: Try different values for the parameters
 - *simPrimalSelection* and
 - *simDualSelection*.
- If you are using warm-starts, in rare cases switching off this feature may improve stability. This is controlled by the *simHotstart* parameter.
- Increase maximum number of set-backs allowed controlled by *simMaxNumSetbacks*.
- If the problem repeatedly becomes infeasible try switching off the special degeneracy handling. See the parameter *simDegen* for details.

The Simplex Log

Below is a typical log output from the simplex optimizer:

Optimizer	- solved problem	:	the primal			
Optimizer	- Constraints	:	667			
Optimizer	- Scalar variables	:	1424	conic	:	0
Optimizer	- hotstart	:	no			
ITER	DEGITER(%)	PFEAS	DFEAS	POBJ	DOBJ	
↪	TIME	TOTTIME				
0	0.00	1.43e+05	NA	6.5584140832e+03	NA	↪
↪	0.00	0.02				
1000	1.10	0.00e+00	NA	1.4588289726e+04	NA	↪
↪	0.13	0.14				
2000	0.75	0.00e+00	NA	7.3705564855e+03	NA	↪
↪	0.21	0.22				

(continues on next page)

(continued from previous page)

3000	0.67	0.00e+00	NA	6.0509727712e+03	NA	␣
↪	0.29	0.31				
4000	0.52	0.00e+00	NA	5.5771203906e+03	NA	␣
↪	0.38	0.39				
4533	0.49	0.00e+00	NA	5.5018458883e+03	NA	␣
↪	0.42	0.44				

The first lines summarize the problem the optimizer is solving. This is followed by the iteration log, with the following meaning:

- **ITER**: Number of iterations.
- **DEGITER(%)**: Ratio of degenerate iterations.
- **PFEAS**: Primal feasibility measure reported by the simplex optimizer. The numbers should be 0 if the problem is primal feasible (when the primal variant is used).
- **DFEAS**: Dual feasibility measure reported by the simplex optimizer. The number should be 0 if the problem is dual feasible (when the dual variant is used).
- **POBJ**: An estimate for the primal objective value (when the primal variant is used).
- **DOBJ**: An estimate for the dual objective value (when the dual variant is used).
- **TIME**: Time spent since this instance of the simplex optimizer was invoked (in seconds).
- **TOTTIME**: Time spent since optimization started (in seconds).

13.3 Conic Optimization - Interior-point optimizer

For conic optimization problems only an interior-point type optimizer is available.

13.3.1 The homogeneous primal-dual problem

The interior-point optimizer is an implementation of the so-called homogeneous and self-dual algorithm. For a detailed description of the algorithm, please see [ART03]. In order to keep our discussion simple we will assume that **MOSEK** solves a conic optimization problem of the form:

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && Ax = b, \\ & && x \in \mathcal{K} \end{aligned} \tag{13.6}$$

where $\mathcal{K}$ is a convex cone. The corresponding dual problem is

$$\begin{aligned} & \text{maximize} && b^T y \\ & \text{subject to} && A^T y + s = c, \\ & && s \in \mathcal{K}^* \end{aligned} \tag{13.7}$$

where $\mathcal{K}^*$ is the dual cone of $\mathcal{K}$. See Sec. 12.2 for definitions.

Since it is not known beforehand whether problem (13.6) has an optimal solution, is primal infeasible or is dual infeasible, the optimization algorithm must deal with all three situations. This is the reason that **MOSEK** solves the so-called homogeneous model

$$\begin{aligned} Ax - b\tau &= 0, \\ A^T y + s - c\tau &= 0, \\ -c^T x + b^T y - \kappa &= 0, \\ x &\in \mathcal{K}, \\ s &\in \mathcal{K}^*, \\ \tau, \kappa &\geq 0, \end{aligned} \tag{13.8}$$

where y and s correspond to the dual variables in (13.6), and τ and κ are two additional scalar variables. Note that the homogeneous model (13.8) always has a solution since

$$(x, y, s, \tau, \kappa) = (0, 0, 0, 0, 0)$$

is a solution, although not a very interesting one. Any solution

$$(x^*, y^*, s^*, \tau^*, \kappa^*)$$

to the homogeneous model (13.8) satisfies

$$(x^*)^T s^* + \tau^* \kappa^* = 0$$

i.e. complementarity. Observe that $x^* \in \mathcal{K}$ and $s^* \in \mathcal{K}^*$ implies

$$(x^*)^T s^* \geq 0$$

and therefore

$$\tau^* \kappa^* = 0.$$

since $\tau^*, \kappa^* \geq 0$. Hence, at least one of τ^* and κ^* is zero.

First, assume that $\tau^* > 0$ and hence $\kappa^* = 0$. It follows that

$$\begin{aligned} A \frac{x^*}{\tau^*} &= b, \\ A^T \frac{y^*}{\tau^*} + \frac{s^*}{\tau^*} &= c, \\ -c^T \frac{x^*}{\tau^*} + b^T \frac{y^*}{\tau^*} &= 0, \\ x^*/\tau^* &\in \mathcal{K}, \\ s^*/\tau^* &\in \mathcal{K}^*. \end{aligned}$$

This shows that $\frac{x^*}{\tau^*}$ is a primal optimal solution and $(\frac{y^*}{\tau^*}, \frac{s^*}{\tau^*})$ is a dual optimal solution; this is reported as the optimal interior-point solution since

$$(x, y, s) = \left(\frac{x^*}{\tau^*}, \frac{y^*}{\tau^*}, \frac{s^*}{\tau^*} \right)$$

is a primal-dual optimal solution.

On other hand, if $\kappa^* > 0$ then

$$\begin{aligned} Ax^* &= 0, \\ A^T y^* + s^* &= 0, \\ -c^T x^* + b^T y^* &= \kappa^*, \\ x^* &\in \mathcal{K}, \\ s^* &\in \mathcal{K}^*. \end{aligned}$$

This implies that at least one of

$$c^T x^* < 0 \tag{13.9}$$

or

$$b^T y^* > 0 \tag{13.10}$$

holds. If (13.9) is satisfied, then x^* is a certificate of dual infeasibility, whereas if (13.10) holds then y^* is a certificate of primal infeasibility.

In summary, by computing an appropriate solution to the homogeneous model, all information required for a solution to the original problem is obtained. A solution to the homogeneous model can be computed using a primal-dual interior-point algorithm [And09].

13.3.2 Interior-point Termination Criterion

Since computations are performed in finite precision, and for efficiency reasons, it is not possible to solve the homogeneous model exactly in general. Hence, an exact optimal solution or an exact infeasibility certificate cannot be computed and a reasonable termination criterion has to be employed.

In every iteration k of the interior-point algorithm a trial solution

$$(x^k, y^k, s^k, \tau^k, \kappa^k)$$

to the homogeneous model is generated, where

$$x^k \in \mathcal{K}, s^k \in \mathcal{K}^*, \tau^k, \kappa^k > 0.$$

Therefore, it is possible to compute the values:

$$\begin{aligned} \rho_p^k &= \arg \min_{\rho} \left\{ \rho \mid \left\| A \frac{x^k}{\tau^k} - b \right\|_{\infty} \leq \rho \varepsilon_p (1 + \|b\|_{\infty}) \right\}, \\ \rho_d^k &= \arg \min_{\rho} \left\{ \rho \mid \left\| A^T \frac{y^k}{\tau^k} + \frac{s^k}{\tau^k} - c \right\|_{\infty} \leq \rho \varepsilon_d (1 + \|c\|_{\infty}) \right\}, \\ \rho_g^k &= \arg \min_{\rho} \left\{ \rho \mid \left(\frac{(x^k)^T s^k}{(\tau^k)^2}, \left| \frac{c^T x^k}{\tau^k} - \frac{b^T y^k}{\tau^k} \right| \right) \leq \rho \varepsilon_g \max \left(1, \frac{\min(|c^T x^k|, |b^T y^k|)}{\tau^k} \right) \right\}, \\ \rho_{pi}^k &= \arg \min_{\rho} \left\{ \rho \mid \left\| A^T y^k + s^k \right\|_{\infty} \leq \rho \varepsilon_i b^T y^k, b^T y^k > 0 \right\} \text{ and} \\ \rho_{di}^k &= \arg \min_{\rho} \left\{ \rho \mid \left\| Ax^k \right\|_{\infty} \leq -\rho \varepsilon_i c^T x^k, c^T x^k < 0 \right\}. \end{aligned}$$

Note $\varepsilon_p, \varepsilon_d, \varepsilon_g$ and ε_i are nonnegative user specified tolerances.

Optimal Case

Observe ρ_p^k measures how far x^k/τ^k is from being a good approximate primal feasible solution. Indeed if $\rho_p^k \leq 1$, then

$$\left\| A \frac{x^k}{\tau^k} - b \right\|_{\infty} \leq \varepsilon_p (1 + \|b\|_{\infty}). \quad (13.11)$$

This shows the violations in the primal equality constraints for the solution x^k/τ^k is small compared to the size of b given ε_p is small.

Similarly, if $\rho_d^k \leq 1$, then $(y^k, s^k)/\tau^k$ is an approximate dual feasible solution. If in addition $\rho_g \leq 1$, then the solution $(x^k, y^k, s^k)/\tau^k$ is approximate optimal because the associated primal and dual objective values are almost identical.

In other words if $\max(\rho_p^k, \rho_d^k, \rho_g^k) \leq 1$, then

$$\frac{(x^k, y^k, s^k)}{\tau^k}$$

is an approximate optimal solution.

Dual Infeasibility Certificate

Next assume that $\rho_{di}^k \leq 1$ and hence

$$\|Ax^k\|_{\infty} \leq -\varepsilon_i c^T x^k \text{ and } -c^T x^k > 0$$

holds. Now in this case the problem is declared dual infeasible and x^k is reported as a certificate of dual infeasibility. The motivation for this stopping criterion is as follows. Let

$$\bar{x} := \frac{x^k}{-c^T x^k}$$

and it is easy to verify that

$$\|A\bar{x}\|_{\infty} \leq \varepsilon_i \text{ and } c^T \bar{x} = -1$$

which shows $\bar{x}$ is an approximate certificate of dual infeasibility, where ε_i controls the quality of the approximation.

Primal Infeasibility Certificate

Next assume that $\rho_{pi}^k \leq 1$ and hence

$$\|A^T y^k + s^k\|_\infty \leq \varepsilon_i b^T y^k \text{ and } b^T y^k > 0$$

holds. Now in this case the problem is declared primal infeasible and (y^k, s^k) is reported as a certificate of primal infeasibility. The motivation for this stopping criterion is as follows. Let

$$\bar{y} := \frac{y^k}{b^T y^k} \text{ and } \bar{s} := \frac{s^k}{b^T y^k}$$

and it is easy to verify that

$$\|A^T \bar{y} + \bar{s}\|_\infty \leq \varepsilon_i \text{ and } b^T \bar{y} = 1$$

which shows (y^k, s^k) is an approximate certificate of dual infeasibility, where ε_i controls the quality of the approximation.

13.3.3 Adjusting optimality criteria

It is possible to adjust the tolerances ε_p , ε_d , ε_g and ε_i using parameters; see the next table for details.

Table 13.2: Parameters employed in termination criterion

ToleranceParameter	Name (for conic problems)
ε_p	<i>intpntCoTolPfeas</i>
ε_d	<i>intpntCoTolDfeas</i>
ε_g	<i>intpntCoTolRelGap</i>
ε_i	<i>intpntCoTolInfeas</i>

The default values of the termination tolerances are chosen such that for a majority of problems appearing in practice it is not possible to achieve much better accuracy. Therefore, tightening the tolerances usually is not worthwhile. However, an inspection of (13.11) reveals that the quality of the solution depends on $\|b\|_\infty$ and $\|c\|_\infty$; the smaller the norms are, the better the solution accuracy.

The interior-point method as implemented by **MOSEK** will converge toward optimality and primal and dual feasibility at the same rate [And09]. This means that if the optimizer is stopped prematurely then it is very unlikely that either the primal or dual solution is feasible. Another consequence is that in most cases all the tolerances, ε_p , ε_d , ε_g and ε_i , have to be relaxed together to achieve an effect.

If the optimizer terminates without locating a solution that satisfies the termination criteria, for example because of a stall or other numerical issues, then it will check if the solution found up to that point satisfies the same criteria with all tolerances multiplied by the value of *intpntCoTolNearRel*. If this is the case, the solution is still declared as optimal.

To conclude the discussion in this section, relaxing the termination criterion is usually not worthwhile.

13.3.4 The Interior-point Log

Below is a typical log output from the interior-point optimizer:

Optimizer	- threads	:	20						
Optimizer	- solved problem	:	the primal						
Optimizer	- Constraints	:	1						
Optimizer	- Cones	:	2						
Optimizer	- Scalar variables	:	6	conic	:	6			
Optimizer	- Semi-definite variables:	0		scalarized	:	0			
Factor	- setup time	:	0.00	dense det. time	:	0.00			
Factor	- ML order time	:	0.00	GP order time	:	0.00			
Factor	- nonzeros before factor	:	1	after factor	:	1			
Factor	- dense dim.	:	0	flops	:	1.			
$\hookrightarrow 70e+01$									
ITE PFEAS	DFEAS	GFEAS	PRSTATUS	POBJ		DOBJ		MU	$\hookrightarrow$
$\hookrightarrow$ TIME									

(continues on next page)

(continued from previous page)

0	1.0e+00	2.9e-01	3.4e+00	0.00e+00	2.414213562e+00	0.000000000e+00	1.0e+00	┐
↪	0.01							
1	2.7e-01	7.9e-02	2.2e+00	8.83e-01	6.969257574e-01	-9.685901771e-03	2.7e-01	┐
↪	0.01							
2	6.5e-02	1.9e-02	1.2e+00	1.16e+00	7.606090061e-01	6.046141322e-01	6.5e-02	┐
↪	0.01							
3	1.7e-03	5.0e-04	2.2e-01	1.12e+00	7.084385672e-01	7.045122560e-01	1.7e-03	┐
↪	0.01							
4	1.4e-08	4.2e-09	4.9e-08	1.00e+00	7.071067941e-01	7.071067599e-01	1.4e-08	┐
↪	0.01							

The first line displays the number of threads used by the optimizer and the second line indicates if the optimizer chose to solve the primal or dual problem (see [intpntSolveForm](#)). The next lines display the problem dimensions as seen by the optimizer, and the **Factor...** lines show various statistics. This is followed by the iteration log.

Using the same notation as in [Sec. 13.3.1](#) the columns of the iteration log have the following meaning:

- **ITE**: Iteration index k .
- **PFEAS**: $\|Ax^k - b\tau^k\|_\infty$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- **DFEAS**: $\|A^T y^k + s^k - c\tau^k\|_\infty$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- **GFEAS**: $|-c^T x^k + b^T y^k - \kappa^k|$. The numbers in this column should converge monotonically towards zero but may stall at low level due to rounding errors.
- **PRSTATUS**: This number converges to 1 if the problem has an optimal solution whereas it converges to -1 if that is not the case.
- **POBJ**: $c^T x^k / \tau^k$. An estimate for the primal objective value.
- **DOBJ**: $b^T y^k / \tau^k$. An estimate for the dual objective value.
- **MU**: $\frac{(x^k)^T s^k + \tau^k \kappa^k}{n+1}$. The numbers in this column should always converge to zero.
- **TIME**: Time spent since the optimization started (in seconds).

13.4 The Optimizer for Mixed-Integer Problems

Solving optimization problems where one or more of the variables are constrained to be integer valued is called Mixed-Integer Optimization (MIO). For an introduction to model building with integer variables, the reader is recommended to consult the **MOSEK Modeling Cookbook**, and for further reading we highlight textbooks such as [\[Wol98\]](#) or [\[CCornuejolsZ14\]](#).

MOSEK can perform mixed-integer linear (MILO) and conic (MICO) problems, except for mixed-integer semidefinite problems.

By default the mixed-integer optimizer is run-to-run deterministic. This means that if a problem is solved twice on the same computer with identical parameter settings and no time limit, then the obtained solutions will be identical. The mixed-integer optimizer is parallelized, i.e., it can exploit multiple cores during the optimization.

In practice, a predominant special case of integer variables are binary variables, taking values in $\{0, 1\}$. Mixed- or pure binary problems are important subclasses of mixed-integer optimization where all integer variables are of this type. In the general setting however, an integer variable may have arbitrary lower and upper bounds.

13.4.1 Branch-and-Bound

In order to succeed in solving mixed-integer problems, it can be useful to have a basic understanding of the underlying solution algorithms. The most important concept in this regard is arguably the so-called Branch-and-Bound algorithm, employed also by **MOSEK**. In order to comprehend Branch-and-Bound, the concept of a *relaxation* is important.

Consider for example a mixed-integer linear optimization problem of minimization type

$$\begin{aligned} z^* = \quad & \text{minimize} && c^T x \\ & \text{subject to} && Ax = b \\ & && x \geq 0 \\ & && x_j \in \mathbb{Z}, \quad \forall j \in \mathcal{J}. \end{aligned} \quad (13.12)$$

It has the continuous relaxation

$$\begin{aligned} \underline{z} = \quad & \text{minimize} && c^T x \\ & \text{subject to} && Ax = b \\ & && x \geq 0, \end{aligned} \quad (13.13)$$

obtained simply by ignoring the integrality restrictions. The first step in Branch-and-Bound is to solve this so-called *root* relaxation, which is a continuous optimization problem. Since (13.13) is less constrained than (13.12), one certainly gets

$$\underline{z} \leq z^*,$$

and $\underline{z}$ is therefore called the *objective bound*: it bounds the optimal objective value from below.

After the solution of the root relaxation, in the most likely outcome there will be one or more integer constrained variables with fractional values, i.e., violating the integrality constraints. Branch-and-Bound now takes such a variable, $x_j = f_j \in \mathbb{R} \setminus \mathbb{Z}$ with $j \in \mathcal{J}$, say, and creates two branches leading to relaxations with the additional constraint $x_j \leq \lfloor f_j \rfloor$ or $x_j \geq \lceil f_j \rceil$, respectively. The intuitive idea here is to push the variable away from the fractional value, closer towards integrality. If the variable was binary, say, branching would lead to fixing its value to 0 in one branch, and to 1 in the other.

The Branch-and-Bound process continues in this way and successively solves relaxations and creates branches to refined relaxations. Whenever a relaxation solution $\hat{x}$ does not violate any integrality constraints, it is feasible to (13.12) and is called an *integer feasible solution*. Clearly, its solution value $\bar{z} := c^T \hat{x}$ is an upper bound on the optimal objective value,

$$z^* \leq \bar{z}.$$

Since refining a relaxation by adding constraints to it can only increase its solution value, the objective bound $\underline{z}$, now defined as the minimum over all solution values of so far solved relaxations, can only increase during the algorithm. If as upper bound $\bar{z}$ one records the solution value of the best integer feasible solution encountered so far, the so-called *incumbent solution*, $\bar{z}$ can only decrease during the algorithm. Since at any time we also have

$$\underline{z} \leq z^* \leq \bar{z},$$

objective bound and incumbent solution value are encapsulating the optimal objective value, eventually converging to it.

The Branch-and-Bound scheme can be depicted by means of a tree, where branches and relaxations correspond to edges and nodes. Figure Fig. 13.1 shows an example of such a tree. The strength of Branch-and-Bound is its ability to prune nodes in this tree, meaning that no new child nodes will be created. Pruning can occur in several cases:

- A relaxation leads to an integer feasible solution $\hat{x}$. In this case we may update the incumbent and its solution value $\bar{z}$, but no new branches need to be created.
- A relaxation is infeasible. The subtree rooted at this node cannot contain any feasible relaxation, so it can be discarded.
- A relaxation has a solution value that exceeds $\bar{z}$. The subtree rooted at this node cannot contain any integer feasible solution with a solution value better than the incumbent we already have, so it can be discarded.

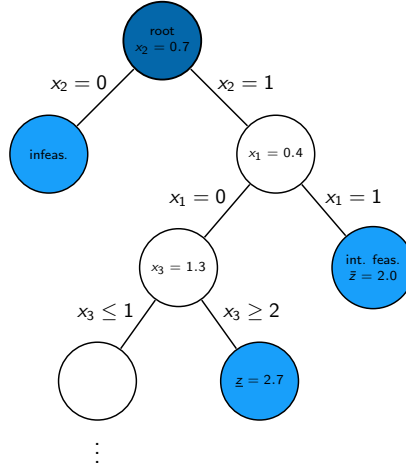


Fig. 13.1: An exemplary Branch-and-Bound tree. Pruned nodes are shown in light blue.

Having objective bound and incumbent solution value is a quite fundamental property of Branch-and-Bound, and helps to assess solution quality and control termination of the algorithm, as we detail in the next section. Note that the above explanation is coined for minimization problems, but the Branch-and-bound scheme has a straightforward extension to maximization problems.

13.4.2 Solution quality and termination criteria

The issue of terminating the mixed-integer optimizer is rather delicate. Recalling the Branch-and-Bound scheme from the previous section, one may see that mixed-integer optimization is generally much harder than continuous optimization; in fact, solving continuous sub-problems is just one component of a mixed-integer optimizer. Despite the ability to prune nodes in the tree, the computational effort required to solve mixed-integer problems grows exponentially with the size of the problem in a worst-case scenario (solving mixed-integer problems is NP-hard). For instance, a problem with n binary variables, may require the solution of 2^n relaxations. The value of 2^n is huge even for moderate values of n . In practice it is often advisable to accept near-optimal or approximate solutions in order to counteract this complexity burden. The user has numerous possibilities of influencing optimizer termination with various parameters, in particular related to solution quality, and the most important ones are highlighted here.

Solution quality in terms of optimality

In order to assess the quality of any incumbent solution in terms of its objective value, one may check the *optimality gap*, defined as

$$\epsilon = |(\text{incumbent solution value}) - (\text{objective bound})| = |\bar{z} - \underline{z}|.$$

It measures how much the objectives of the incumbent and the optimal solution can deviate in the worst case. Often it is more meaningful to look at the *relative optimality gap*

$$\epsilon_{\text{rel}} = \frac{|\bar{z} - \underline{z}|}{\max(\delta_1, |\bar{z}|)}.$$

This is essentially the above *absolute* optimality gap normalized against the magnitude of the incumbent solution value; the purpose of the (small) constant δ_1 is to avoid overweighing incumbent solution values that are very close to zero. The relative optimality gap can thus be interpreted as answering the question: “*Within what fraction of the optimal solution is the incumbent solution in the worst case?*”

Absolute and relative optimality gaps provide useful means to define termination criteria for the mixed-integer optimizer in **MOSEK**. The idea is to terminate the optimization process as soon as the quality of the incumbent solution, measured in absolute or relative gap, is good enough. In fact, whenever an incumbent solution is located, the criterion

$$\bar{z} - z \leq \max(\delta_2, \delta_3 \max(\delta_1, |\bar{z}|))$$

is checked. If satisfied, i.e., if either absolute or relative optimality gap are below the thresholds δ_2 or δ_3 , respectively, the optimizer terminates and reports the incumbent as an optimal solution. The optimality gaps can always be retrieved through the information items *"mioObjAbsGap"* and *"mioObjRelGap"*.

The tolerances discussed above can be adjusted using suitable parameters, see Table 13.3. By default, the optimality parameters δ_2 and δ_3 are quite small, i.e., restrictive. These default values for the absolute and relative gap amount to solving any instance to (almost) optimality: the incumbent is required to be within at most a tiny percentage of the optimal solution. As anticipated, this is not tractable in most practical situations, and one should resort to finding near-optimal solutions quickly rather than insisting on finding the optimal one. It may happen, for example, that an optimal or close-to-optimal solution is found very early by the optimizer, but it does not terminate because the objective bound $\underline{z}$ is of poor quality. Instead, the vast majority of computational time is spent on trying to improve $\underline{z}$: a typical situation that practioneers would want to avoid. The concept of optimality gaps is fundamental for controlling solution quality when resorting to near-optimal solutions.

MIO performance tweaks: termination criteria

One of the first things to do in order to cut down excessive solution time is to increase the relative gap tolerance *mioTolRelGap* to some non-default value, so as to not insist on finding optimal solutions. Typical values could be 0.01, 0.05 or 0.1, guaranteeing that the delivered solutions lie within 1%, 5% or 10% of the optimum. Increasing the tolerance will lead to less computational time spent by the optimizer.

Solution quality in terms of feasibility

For an optimizer relying on floating-point arithmetic like the mixed-integer optimizer in **MOSEK**, it may be hard to achieve exact integrality of the solution values of integer variables in most cases, and it makes sense to numerically relax this constraint. Any candidate solution $\hat{x}$ is accepted as integer feasible if the criterion

$$\min(\hat{x}_j - \lfloor \hat{x}_j \rfloor, \lceil \hat{x}_j \rceil - \hat{x}_j) \leq \delta_4 \quad \forall j \in \mathcal{J}$$

is satisfied, meaning that $\hat{x}_j$ is at most δ_4 away from the nearest integer. As above, δ_4 can be adjusted using a parameter, see Table 13.3, and impacts the quality of the acieved solution in terms of integer feasibility. By influencing what solution may be accepted as imcumbent, it can also have an impact on the termination of the optimizer.

MIO performance tweaks: feasibility criteria

Whether increasing the integer feasibility tolerance *mioTolAbsRelaxInt* leads to less solution time is highly problem dependent. Intuitively, the optimizer is more flexible in finding new incumbent soutions so as to improve $\bar{z}$. But this effect has do be examined with care on individuiual instances: it may worsen solution quality with no effect at all on the solution time. It may in some cases even lead to contrary effects on the solution time.

Table 13.3: Tolerances for the mixed-integer optimizer.

Tolerance	Parameter name	Default value
δ_1	<i>mioRelGapConst</i>	1.0e-10
δ_2	<i>mioTolAbsGap</i>	0.0
δ_3	<i>mioTolRelGap</i>	1.0e-4
δ_4	<i>mioTolAbsRelaxInt</i>	1.0e-5

Further controlling optimizer termination

There are more ways to limit the computational effort employed by the mixed-integer optimizer by simply limiting the number of explored branches, solved relaxations or updates of the incumbent solution. When any of the imposed limits is hit, the optimizer terminates and the incumbent solution may be retrieved. See Table 13.4 for a list of corresponding parameters. In contrast to the parameters discussed in Sec. 13.4.2, interfering with these does not maintain any guarantees in terms of solution quality.

Table 13.4: Other parameters affecting the integer optimizer termination criterion.

Parameter name	Explanation
<i>mioMaxNumBranches</i>	Maximum number of branches allowed.
<i>mioMaxNumRelaxs</i>	Maximum number of relaxations allowed.
<i>mioMaxNumSolutions</i>	Maximum number of feasible integer solutions allowed.

13.4.3 Additional components of the mixed-integer Optimizer

The Branch-and-Bound scheme from Sec. 13.4.1 is only the basic skeleton of the mixed-integer optimizer in **MOSEK**, and several components are built on top of that in order to enhance its functionality and increase its speed. A mixed-integer optimizer is sometimes referred to as a “*giant bag of tricks*”, and it would be impossible to describe all of these tricks here. Yet, some of the additional components are worth mentioning to the user. They can be influenced by various user parameters, and although the default values of these parameters are optimized to work well on average mixed-integer problems, it may pay off to adjust them for an individual problem, or a specific problem class.

Presolve

Similar to the case of continuous problems, see Sec. 13.1, the mixed-integer optimizer applies various presolve reductions before the actual solution process is initiated. Just as in the continuous case, the use of presolve can be controlled with the parameter *presolveUse*.

Primal Heuristics

Solving relaxations in the Branch-and-bound tree to an integer feasible solution $\hat{x}$ is not the only way to find new incumbent solutions. There is a variety of procedures that, given a mixed-integer problem in a generic form like (13.12), attempt to produce integer feasible solutions in an ad-hoc way. These procedures are called Primal Heuristics, and several of them are implemented in **MOSEK**. For example, whenever a relaxation leads to a fractional solution, one may round the solution values of the integer variables, in various ways, and hope that the outcome is still feasible to the remaining constraints. Primal heuristics are mostly employed while processing the root node, but play a role throughout the whole solution process. The goal of a primal heuristic is to improve the incumbent solution and thus the bound $\bar{z}$, and this can of course affect the quality of the solution that is returned after termination of the optimizer. The user parameters affecting primal heuristics are listed in Table 13.5.

MIO performance tweaks: primal heuristics

- If the mixed-integer optimizer struggles to improve the incumbent solution $\bar{z}$, see Sec. 13.4.4, it can be helpful to intensify the use of primal heuristics.
 - Set parameters related to primal heuristics to more aggressive values than the default ones, so that more effort is spent in this component. A List of the respective parameters can be found in Table 13.5. In particular, if the optimizer has difficulties finding any integer feasible solution at all, indicated by NA in the column BEST_INT_OBJ in the mixed-integer log, one may try to activate a construction heuristic like the Feasibility Pump with *mioFeaspumpLevel*.
 - Specify a good initial solution: In many cases a good feasible solution is either known or easily computed using problem-specific knowledge that the optimizer does not have. If so, it is usually worthwhile to use this as a starting point for the mixed-integer optimizer. See Sec. 7.7.2.

- For feasibility problems, i.e., problems having a constant objective, the goal is to find a single integer feasible solution, and this can be hard by itself on some instances. Try setting the objective to something meaningful anyway, even if the underlying application does not require this. After all, the feasible set is not changed, but the optimizer might benefit from being able to pursue a concrete goal.
- In rare cases it may also happen that the optimizer spends an excessive amount of time on primal heuristics without drawing any benefit from it, and one may try to limit their use with the respective parameters.

Table 13.5: Parameters affecting primal heuristics

Parameter name	Explanation
<i>mioHeuristicLevel</i>	Primal heuristics aggressivity level.
<i>mioRinsMaxNodes</i>	Maximum number of nodes allowed in the RINS heuristic.
<i>mioFeaspumpLevel</i>	Way of using the Feasibility Pump heuristic.

Cutting Planes

Cutting planes (cuts) are simply constraints that are valid for a mixed-integer problem, for example in the form (13.12), meaning they do not remove any integer feasible solutions from the feasible set. Therefore they are also called valid inequalities. They do not have to be valid for the relaxation (13.13) though, and of interest and potentially useful are those cuts that do remove solutions from the feasible set of the relaxation. The latter is a superset of the feasible region of the mixed-integer problem, and the rationale behind cuts is thus to bring the integer problem and its relaxation closer together in terms of their feasible sets.

As an example, take the constraints

$$2x_1 + 3x_2 + x_3 \leq 4, \quad x_1, x_2 \in \{0, 1\}, \quad x_3 \geq 0. \quad (13.14)$$

One may realize that there cannot be a feasible solution in which both binary variables take on a value of 1. So certainly

$$x_1 + x_2 \leq 1 \quad (13.15)$$

is a valid inequality. In fact, there is no integer solution satisfying (13.14), but violating (13.15). The latter does cut off a portion of the feasible region of the continuous relaxation of (13.14) though, obtained by replacing $x_1, x_2 \in \{0, 1\}$ with $x_1, x_2 \in [0, 1]$. For example, the fractional point $(x_1, x_2, x_3) = (0.5, 1, 0)$ is feasible to the relaxation, but violates the cut (13.15).

There are many classes of general-purpose cutting planes that may be generated for a mixed-integer problem in a generic form like (13.12), and **MOSEK**'s mixed-integer optimizer supports several of them. For instance, the above is an example of a so-called clique cut. The most effort on generating cutting planes is spent after the solution of the root relaxation, but cuts can also be generated later on in the Branch-and-Bound tree. Cuts aim at improving the objective bound $\underline{z}$ and can thus have significant impact on the solution time. The user parameters affecting cut generation can be seen in Table 13.6.

MIO performance tweaks: cutting planes

- If the mixed-integer optimizer struggles to improve the objective bound $\underline{z}$, see Sec. 13.4.4, it can be helpful to intensify the use of cutting planes.
 - Some types of cutting planes are not activated by default, but doing so may help to improve the objective bound.
 - The parameters *mioTolRelDualBoundImprovement* and *mioCutSelectionLevel* determine how aggressively cuts will be generated and selected.
 - If some valid inequalities can be deduced from problem-specific knowledge that the optimizer does not have, it may be helpful to add these to the problem formulation as constraints. This has to be done with care, since there is a tradeoff between the benefit obtained from an improved objective bound, and the amount of additional constraints that make the relaxations larger.

- In rare cases it may also be observed that the optimizer spends an excessive amount of time on cutting planes, see [Sec. 13.4.4](#), and one may limit their use with `mioMaxNumRootCutRounds`, or by disabling a certain type of cutting planes.

Table 13.6: Parameters affecting cutting planes

Parameter name	Explanation
<code>mioCutClique</code>	Should clique cuts be enabled?
<code>mioCutCmir</code>	Should mixed-integer rounding cuts be enabled?
<code>mioCutGmi</code>	Should GMI cuts be enabled?
<code>mioCutImpliedBound</code>	Should implied bound cuts be enabled?
<code>mioCutKnapsackCover</code>	Should knapsack cover cuts be enabled?
<code>mioCutLipro</code>	Should lift-and-project cuts be enabled?
<code>mioCutSelectionLevel</code>	Cut selection aggressivity level.
<code>mioMaxNumRootCutRounds</code>	Maximum number of root cut rounds.
<code>mioTolRelDualBoundImprovement</code>	Minimum required objective bound improvement during root cut generation.

13.4.4 The Mixed-Integer Log

Below is a typical log output from the mixed-integer optimizer:

Presolved problem: 1176 variables, 1344 constraints, 4968 non-zeros						
Presolved problem: 328 general integer, 392 binary, 456 continuous						
Clique table size: 55						
BRANCHES	RELAXS	ACT_NDS	DEPTH	BEST_INT_OBJ	BEST_RELAX_OBJ	REL_GAP(
→%)	TIME					
0	0	1	0	8.3888091139e+07	NA	NA
→	0.0					
0	1	1	0	8.3888091139e+07	2.5492512136e+07	69.61
→	0.1					
0	1	1	0	3.1273162420e+07	2.5492512136e+07	18.48
→	0.1					
0	1	1	0	2.6047699632e+07	2.5492512136e+07	2.13
→	0.2					
Cut generation started.						
0	1	1	0	2.6047699632e+07	2.5492512136e+07	2.13
→	0.2					
0	2	1	0	2.6047699632e+07	2.5589986247e+07	1.76
→	0.2					
Cut generation terminated. Time = 0.05						
0	4	1	0	2.5990071367e+07	2.5662741991e+07	1.26
→	0.3					
0	8	1	0	2.5971002767e+07	2.5662741991e+07	1.19
→	0.5					
0	11	1	0	2.5925040617e+07	2.5662741991e+07	1.01
→	0.5					
0	12	1	0	2.5915504014e+07	2.5662741991e+07	0.98
→	0.5					
2	23	1	0	2.5915504014e+07	2.5662741991e+07	0.98
→	0.6					
14	35	1	0	2.5915504014e+07	2.5662741991e+07	0.98
→	0.6					
[...]						

(continues on next page)

```

Objective of best integer solution : 2.578282162804e+07
Best objective bound               : 2.569877601306e+07
Construct solution objective      : Not employed
User objective cut value          : Not employed
Number of cuts generated          : 192
  Number of Gomory cuts           : 52
  Number of CMIR cuts             : 137
  Number of clique cuts           : 3
Number of branches                 : 29252
Number of relaxations solved       : 31280
Number of interior point iterations: 16
Number of simplex iterations       : 105440
Time spend presolving the root     : 0.03
Time spend optimizing the root     : 0.07
Mixed integer optimizer terminated. Time: 6.46

```

The first lines contain a summary of the problem after mixed-integer presolve has been applied. This is followed by the iteration log, reflecting the progress made during the Branch-and-bound process. The columns have the following meanings:

- **BRANCHES**: Number of branches / nodes generated.
- **RELAXS**: Number of relaxations solved.
- **ACT_NDS**: Number of active / non-processed nodes.
- **DEPTH**: Depth of the last solved node.
- **BEST_INT_OBJ**: The incumbent solution / best integer objective value, $\bar{z}$.
- **BEST_RELAX_OBJ**: The objective bound, $\underline{z}$.
- **REL_GAP(%)**: Relative optimality gap, $100\% \cdot \epsilon_{\text{rel}}$
- **TIME**: Time (in seconds) from the start of optimization.

The beginning and the end of the root cut generation is highlighted as well, and the number of log lines in between reflects to the computational effort spent here.

Finally there is a summary of the optimization process, containing also information on the type of generated cuts and the total number of iterations needed to solve all occurring continuous relaxations.

When the solution time for a mixed-integer problem has to be cut down, it can sometimes be useful to examine the log in order to understand where time is spent and what might be improved. In particular, it might happen that the values in either of the columns **BEST_INT_OBJ** or **BEST_RELAX_OBJ** stall over a long period of log lines, an indication that the optimizer has a hard time improving either the incumbent solution, i.e., $\bar{z}$, or the objective bound $\underline{z}$, see also [Sec. 13.4.3](#) and [Sec. 13.4.3](#).

13.4.5 Mixed-Integer Nonlinear Optimization

Due to the involved non-linearities, MI(QC)QO or MICO problems are on average harder than MILO problems of comparable size. Yet, the Branch-and-Bound scheme can be applied to these problem classes in a straightforward manner. The relaxations have to be solved as conic problems with the interior point algorithm in that case, see [Sec. 13.3](#), opposed to MILO where it is often beneficial to solve relaxations with the dual simplex method, see [Sec. 13.2.3](#). There is another solution approach for these types of problems implemented in **MOSEK**, namely the Outer-Approximation algorithm, making use of dynamically refined linear approximations of the non-linearities.

MICO performance tweaks: choice of algorithm

Whether conic Branch-and-Bound or Outer-Approximation is applied to a mixed-integer conic problem can be set with *mioConicOuterApproximation*. The best value for this option is highly problem dependent.

13.4.6 Disjunctive constraints

Problems with disjunctive constraints (DJC) see Sec. 7.8 are typically reformulated to mixed-integer problems, and even if this is not the case they are solved with an algorithm that is based on the mixed-integer optimizer. In **MOSEK**, these problems thus fall into the realm of MIO. In particular, **MOSEK** automatically attempts to replace any DJC by so called big-M constraints, potentially after transforming it to several, less complicated DJCs. As an example, take the DJC

$$[z = 0] \vee [z = 1, x_1 + x_2 \geq 1000],$$

where $z \in \{0, 1\}$ and $x_1, x_2 \in [0, 750]$. This is an example of a DJC formulation of a so-called indicator constraint. A big-M reformulation is given by

$$x_1 + x_2 \geq 1000 - M \cdot (1 - z),$$

where $M > 0$ is a large constant. The practical difficulty of these constructs is that M should always be sufficiently large, but ideally not larger. Too large values for M can be harmful for the mixed-integer optimizer. During presolve, and taking into account the bounds of the involved variables, **MOSEK** automatically reformulates DJCs to big-M constraints if the required M values do not exceed the parameter *mioDjcMaxBigm*. From a performance point-of-view, all DJCs would ideally be linearized to big-Ms after presolve without changing this parameter's default value of 1.0e6. Whether or not this is the case can be seen by retrieving the information item "*mioPresolvedNumdjc*", or by a line in the mixed-integer optimizer's log as in the example below. Both state the number of remaining disjunctions after presolve.

Presolved problem: 305 variables, 204 constraints, 708 non-zeros						
Presolved problem: 0 general integer, 100 binary, 205 continuous						
Presolved problem: 100 disjunctions						
Clique table size: 0						
BRANCHES	RELAXS	ACT_NDS	DEPTH	BEST_INT_OBJ	BEST_RELAX_OBJ	REL_GAP(
↪%)	TIME					
0	1	1	0	NA	0.0000000000e+00	NA
↪	0.0					
0	1	1	0	5.0574653969e+05	0.0000000000e+00	100.00
↪	0.0					
[...]						

DJC performance tweaks: managing variable bounds

- Always specify the tightest known bounds on the variables of any problem with DJCs, even if they seem trivial from the user-perspective. The mixed-integer optimizer can only benefit from these when reformulating DJCs and thus gain performance; even if bounds don't help with reformulations, it is very unlikely that they hurt the optimizer.
- Increasing *mioDjcMaxBigm* can lead to more DJC reformulations and thus increase optimizer speed, but it may in turn hurt numerical solution quality and has to be examined with care. The other way round, on numerically challenging instances with DJCs, decreasing *mioDjcMaxBigm* may lead to numerically more robust solutions.

13.4.7 Randomization

A mixed-integer optimizer is usually prone to performance variability, meaning that a small change in either

- problem data, or
- computer hardware, or
- algorithmic parameters

can lead to significant changes in solution time, due to different solution paths in the Branch-and-Bound tree. In extreme cases the exact same problem can vary from being solvable in less than a second to seemingly unsolvable in any reasonable amount of time on a different computer.

One practical implication of this is that one should ideally verify whether a seemingly beneficial set of parameters, established experimentally on a single problem, is still beneficial (on average) on a larger set of problems from the same problem class. This protects against making parameter changes that had positive effects only due to random effects on that single problem.

In the absence of a large set of test problems, one may also change the random seed of the optimizer to a series of different values in order to hedge against drawing such wrong conclusions regarding parameters. The random seed, accessible through *mioSeed*, impacts for example random tie-breaking in many of the mixed-integer optimizer's components. Changing the random seed can be combined with a permutation of the problem data to further incite randomness, accessible through the parameter *mioDataPermutationMethod*.

13.4.8 Further performance tweaks

In addition to what was mentioned previously, there may be other ways to speed up the solution of a given mixed-integer problem. For example, there are further user parameters affecting some algorithmic settings in the mixed-integer optimizer. As mentioned above, default parameter values are optimized to work well on average, but on individual problems they may be adjusted.

MIO performance tweaks: miscellaneous

- When relaxations in the the Branch-and-Bound tree are linear optimization problems (e.g., in MILO or when solving MICO problems with the Outer-Approximation method), it is usually best to employ the dual simplex method for their solution. In rare cases the primal simplex method may actually be the better choice, and this can be set with the parameter *mioNodeOptimizer*.
 - Some problems are numerically more challenging than others, for example if the ratio between the smallest and the largest involved coefficients is large, say $\geq 1e9$. An indication of numerical issues are, for example, large violations in the final solution, observable in the solution summary of the log output, see [Sec. 9.1.3](#). Similarly, a problem that is known to be feasible by the user may be declared infeasible by the optimizer. In such cases it is usually best to try to rescale the model. Otherwise, the mixed-integer optimizer can be instructed to be more cautious regarding numerics with the parameter *mioNumericalEmphasisLevel*. This may in turn be at the cost of solution speed though.
 - Improve the formulation: A MIO problem may be impossible to solve in one form and quite easy in another form. However, it is beyond the scope of this manual to discuss good formulations for mixed-integer problems. For discussions on this topic see for example [\[Wol98\]](#).
-

Chapter 14

Fusion API Reference

- *General API conventions.*
- **mosek::fusion** classes
 - Quick links: *Model*, *Expr*, *Variable*, *Parameter*, *Var*, *Domain*, *Matrix*, *Constraint*, *DJC*
 - *Full list*
- **Optimizer parameters:**
 - *Double*, *Integer*, *String*
 - *Full list*
 - *Browse by topic*
- **Optimizer information items:**
 - *Double*, *Integer*, *Long*
- *Enumerations*
- *Constants*
- *Exceptions*
- *List of supported domains*
- *Linear algebra utilities*

14.1 *Fusion* API conventions

14.1.1 General conventions

All the classes of the *Fusion* interface are contained in the namespace **mosek::fusion**. The user should not directly instantiate objects of any class other than creating a *Model*.

```
Model::t M = new Model();    auto _M = finally([&]() { M->dispose(); });
```

The model is the main access point to an optimization problem and its solution. All other objects should be created through the model (*Model.variable*, *Model.constraint*, etc.) or using static factory methods (*Matrix.sparse* etc.).

The C++ *Fusion* implements its own reference counting and array types to improve garbage collection. The user will require the type definitions from two namespaces:

```
#include "fusion.h"

using namespace mosek::fusion;
using namespace monty;
```

The following subsections contain the relevant API reference.

14.1.2 C++ Arrays and Pointers

Arrays

All arrays in *Fusion* are passed as objects of type `monty::ndarray` wrapped in a shared pointer:

```
std::shared_ptr<monty::ndarray<T,N>>
```

where `T` is the type of the elements and `N` is an integer denoting the number of dimensions. See `monty::ndarray` for ways to create, manipulate and iterate over arrays and `monty::new_array_ptr` for array factory methods.

Shapes and indexes

The shape of arrays is defined by a type `monty::shape_t`, which is also used to define an N -dimensional index.

Objects and reference counting

Objects of all classes defined in the `mosek::fusion` namespace are wrapped in a reference counting pointer of type `monty::rc_ptr`. Every *Fusion* class `C` contains a definition of a type

```
typedef monty::rc_ptr<C> t;
```

for example `Model::t`, `Variable::t` etc. These pointer types should be used instead of standard pointers to ensure proper garbage collection. It is also recommended to use a finalizer if possible, as below. For example

```
Model::t M = new Model("MyModel");
auto _M = finally([&]() { M->dispose(); });
Variable::t v = M->variable(5);
```

An object will be destroyed the moment nobody refers to it any more.

14.1.3 Using vectors

Standard C++ vectors can be converted into a *Fusion* array using `monty::new_array_ptr`.

```
std::vector<double> x(5);
// ....
auto a = monty::new_array_ptr<double>(x);
// Now we can use a in Fusion calls, eg:
auto product = Expr::dot(a, v);
```

Similar version of `monty::new_array_ptr` is available for 2D vectors:

```
std::vector<std::vector<double>> x({{1,1,1,1,1}, {1,2,3,4,5}});
// ....
auto product = Expr::mul(monty::new_array_ptr<double>(x), v);
```

For more details see `monty::new_array_ptr`, and `monty::new_vector_from_array_ptr`.

14.1.4 C++ extension reference

Multidimensional arrays

`monty::ndarray<T,N>`

A template class for all arrays in *Fusion*.

Template parameters

- `T` – The type of objects stored in the array.
- `N` – The number of dimensions.

`ndarray<T,N>.ndarray<T,N>`

```
ndarray(shape_t<N> shape)
ndarray(shape_t<N> shape, T value)
ndarray(const shape_t<N> & shape, const std::function<T(ptrdiff_t)> & fLin);
ndarray(const shape_t<N> & shape, const std::function<T(const shape_t<N> &)> & fInd)
ndarray(T* vals, shape_t<N> shape)
ndarray(init_t & init)
template<typename Iterator> ndarray(const shape_t<N> & shape, Iterator begin,
template<typename Iterable> ndarray(const shape_t<N> & shape, const typename
template<typename Iterable> ndarray(const shape_t<N> & shape, const const_
ndarray(const ndarray<T,N> & that)
ndarray(ndarray<T,N> && that)
```

Constructor of a multidimensional array object. The initializing data can have several forms:

- Default value, e.g. 0 for numerical types.
- Create a new array initialized with values generated by a generator function.

```
auto a1 = std::make_shared<ndarray<int,1>>(shape(5), [](ptrdiff_t i) {
auto a2 = std::make_shared<ndarray<int,2>>(shape(5,5), std::function
<int(const shape_t<2> &)>([](const shape_t<2> & p) { return p[0]+p[1]; }));
```

- Initialized with values from another array.
- Initialized with an initializer list.

Arrays can also be constructed with the factory method `monty::new_array_ptr`.

Parameters

- `shape` (*shape_t*) – The shape of the array.
- `value` (T) – The default value for all elements.
- `fLin` – An array-filling function taking the linear index as an argument.
- `fInd` – An array-filling function taking the multidimensional index as an argument.
- `vals` (T*) – An array with element values.
- `init` (*init_t*) – An initializer list. The type *init_t* is an *N*-fold nested initializer list of type `std::initializer_list`. For example, for 2-dimensional arrays this is an initializer list of initializer lists, and so forth.
- `begin`, `end` – The pointers to an iterator.
- `that` – Another iterable object (see `monty_base.h`) or an `ndarray` object used as initializer.

`ndarray<T,N>.operator()`

```
T& operator()(int i1, ..., int iN)
```

N-dimensional access operator. Returns the element at position $(i_1, \dots, i_N)$.

Example:

```
auto a = std::make_shared<ndarray<int,2>>(shape(5,5));
(*a)(3,4) = 15;
```

Parameters $i_1, \dots, i_N$ (int) – The multi-dimensional index of the element.

`ndarray<T,N>.operator[]`

```
T& operator[] (int i)
T& operator[] (const shape_t<N> & idx)
```

Access operator. Returns a specific element in the array.

Parameters

- i (int) – The linear index of the element, referring to its position in the one-dimensional row-wise flattening of the array.
- idx (*shape_t*) – The multi-dimensional index of the element.

`ndarray<T,N>.begin`

```
T* begin()
```

An iterator pointing to the beginning of the array. Note that the iterator sees the array as a one dimensional array obtained traversing the N -dimensional array row-wise.

Example:

```
auto a = new_array_ptr<double,2>({ { 1.1, 2.2 }, { 3.3, 4.4 } } );
for(auto x : *a) std::cout << x << " ";
```

`ndarray<T,N>.end`

```
T* end()
```

An iterator pointing to the end of the array.

`ndarray<T,N>.raw`

```
T* raw()
```

Returns a pointer to the raw data array.

`ndarray<T,N>.size`

```
size_t size()
```

Returns the number of elements in the array.

Shapes and indices

`monty::shape_t<N>`

Represents the shape of an N -dimensional array. It is in fact an N -tuple of integers. It is also used to specify an index in an N -dimensional array.

Template parameters N – The number of dimensions.

`shape_t<N>.shape_t<N>`

```
shape_t(int i1, ..., int iN)
shape_t(int i0, shape_t<N-1> is)
shape_t(const shape_t<N> & that)
```

Constructor of a new shape. Shapes can also be constructed with the factory method `monty::shape`.

Parameters

- `i1, ..., iN (int)` – The dimensions.
- `i0 (int)` – The first dimension.
- `is (shape_t)` – Shape of the remaining $N - 1$ dimensions.
- `that (shape_t)` – Another shape to be copied.

`shape_t<N>.operator[]`

```
int operator[] (int i)
```

Return the size in a specific dimension.

Parameters `i (int)` – The dimension.

`shape_t<N>.tolinear`

```
int tolinear(const shape_t<N> & point)
int tolinear(int i1, ..., int iN)
```

Compute the linear index of an element within this shape. The linear index is the position of the element in the one-dimensional row-wise flattening of the shape.

Example:

```
std::cout << shape(2,3).tolinear(1,1);
// 4
```

Parameters

- `point (shape_t)` – The coordinates of an index within the current shape.
- `i1, ..., iN (int)` – The coordinates of an index within the current shape.

`shape_t<N>.begin`

```
shape_iterator<N> begin()
```

An iterator to the shape. This iterator will traverse all indices belonging to the shape in increasing linear order, i.e. in the row-wise order in the multidimensional shape, and return a object of type *shape_t* for each index.

Example:

```
for(auto p : shape(2,3)) std::cout << p;
// (0,0) (0,1) (0,2) (1,0) (1,1) (1,2)
```

`shape_t<N>.end`

```
shape_iterator<N> end()
```

An iterator pointing to the end of shape.

Reference counting pointers

`monty::rc_ptr<T>`

A reference counting pointer wrapped around an object of type `T`. Implements standard pointer type operators such as `*` and `->` in the expected way. For *Fusion* classes, the reference counting pointer type `monty::rc_ptr<C>` is defined as `C::t`.

Exceptions

`monty::ArrayInitializerException`

An exception thrown when the array initializer is incorrect, for instance a 2-dimensional array is initialized with row lists of non-equal lengths.

Auxiliary functions in the `monty` namespace

`monty::new_array_ptr`

```
std::shared_ptr<ndarray<T,N>> new_array_ptr(init_t init)
std::shared_ptr<ndarray<T,1>> new_array_ptr(std::vector<T> & x)
std::shared_ptr<ndarray<T,2>> new_array_ptr(std::vector<std::vector<T>> & x2)
```

Create a new N -dimensional array of type `T` and wrap it in a shared pointer.

Examples:

```
auto a1 = new_array_ptr<double,1>({ 1.1, 2.2 , 3.3, 4.4 } );
auto a2 = new_array_ptr<double,2>({ { 1.1, 2.2 }, { 3.3, 4.4 } } );
```

```
std::vector<double> x(5);
// ....
auto a = monty::new_array_ptr<double>(x);
// Now we can use a in Fusion calls, eg:
auto product = Expr::dot(a, v);
```

```
std::vector<std::vector<double>> x({{1,1,1,1,1}, {1,2,3,4,5}});
// ....
auto product = Expr::mul(monty::new_array_ptr<double>(x), v);
```

Parameters

- `init (init_t)` – An initializer list. The type `init_t` is an N -fold nested initializer list of type `std::initializer_list`. For example, for 2-dimensional arrays this is an initializer list of initializer lists, and so forth.
- `x (std::vector<T>)` – A vector of type `T`. The new array will have length `x.size()`.
- `x2 (std::vector<std::vector<T>>)` – A vector of type `T`. All rows in `x2` must be vectors of the same length, otherwise an exception will be thrown.

`monty::new_vector_from_array_ptr`

```
std::vector<T> new_vector_from_array_ptr(std::shared_ptr<ndarray<T,1>> & a) {
```

Convert a *Fusion* array into a C++ vector.

Examples:

```
auto a = std::make_shared<ndarray<double,1>>(shape(5));
// ....
std::vector<double> v = monty::new_vector_from_array_ptr(a);
```

Parameters *a* (`std::shared_ptr<ndarray<T,1>>`) – A one-dimensional array.

`monty::shape`

```
shape_t<N> shape(int i1, ..., int iN)
```

Create a new shape with the given dimensions.

Example:

```
auto s = shape(2,3,4);
```

Parameters *i1, ..., iN* (`int`) – The dimensions.

14.2 Class list

Common

- *Constraint*: Abstract base class for Constraint objects.
- *DJC*: A class providing static methods to manipulate terms of disjunctive constraints (DJC).
- *Domain*: Base class for variable and constraint domains.
- *Expr*: Represents a linear expression and provides linear operators.
- *ExprScaleVecPSD*: Scale off-diagonal elements by $\sqrt{2}$ for use with SVECPD domain.
- *Expression*: Abstract base class for all objects which can be used as linear expressions.
- *Matrix*: Base class for all matrix objects.
- *Model*: The object containing all data related to a single optimization model.
- *Param*: Provides static methods for manipulating parameters
- *Parameter*: Abstract class representing model parameters whose values can be modified.
- *Set*: Handles shapes.
- *Var*: Provides basic operations on variable objects.
- *Variable*: Abstract base class for Variable objects.

Infrequent

- *BaseExpression*: Base class for most expressions
- *BaseVariable*: Abstract base class for Variable objects with default implementations.
- *BoundInterfaceConstraint*: Interface to either the upper bound or the lower bound of a ranged constraint.
- *BoundInterfaceVariable*: Interface to either the upper bound or the lower bound of a ranged variable.
- *ConeDomain*: Represent a domain defined by a conic constraints
- *ConicConstraint*: Represent a conic constraint.
- *ConicVariable*: Represent a conic variable.
- *Disjunction*: A class representing a disjunctive constraint.
- *LinearConstraint*: An object representing a block of linear constraints of the same type.

- *LinearDomain*: Represent a domain defined by linear constraints
- *LinearPSDConstraint*: Represents a semidefinite conic constraint.
- *LinearPSDVariable*: This class represents a positive semidefinite variable.
- *LinearVariable*: An object representing a block of linear variables of the same type.
- *ModelConstraint*: Represent a block of constraints.
- *ModelVariable*: Represent a block of variables.
- *NDSparseArray*: Representation of a sparse n-dimensional array.
- *PSDConstraint*: Represents a semidefinite conic constraint.
- *PSDDomain*: Represent the domain of PSD matrices.
- *PSDVariable*: This class represents a positive semidefinite variable.
- *RangeDomain*: The range domain represents a ranged subset of the euclidian space.
- *RangedConstraint*: Represents a ranged constraint.
- *RangedVariable*: Represents a ranged variable.
- *SimpleTerm*: A class representing simple term, a basic building block for disjunctive constraints.
- *SliceConstraint*: An alias for a subset of constraints from a single ModelConstraint.
- *SliceVariable*: An alias for a subset of variables from a single model variable.
- *SymmetricLinearDomain*: Represent a linear domain with symmetry.
- *SymmetricRangeDomain*: Represent a ranged domain with symmetry.
- *Term*: A class representing a term, which ultimately enters a disjunctive constraint.
- *WorkStack*: Stack object used to store expression evaluations. For internal use.

14.2.1 Class BaseExpression

`mosek::fusion::BaseExpression`
Base class for most expressions

Members *BaseExpression.eval* – Evaluate the expression and push the result onto the work stack.

BaseExpression.getDim – Return the d'th dimension in the expression.

BaseExpression.getND – Return the number of dimensions in the expression.

BaseExpression.getShape – Get the shape of the expression.

BaseExpression.getSize – Return the total number of elements in the expression (the product of the dimensions).

BaseExpression.index – Get a single element in the expression.

BaseExpression.pick – Pick a number of elements from the expression.

BaseExpression.slice – Get a slice of the expression.

BaseExpression.toString – Return a string representation of the expression object.

Implemented by *Expr*, *ExprScaleVecPSD*

`BaseExpression.eval`

```
void eval(WorkStack::t rs, WorkStack::t ws, WorkStack::t xs)
```

Evaluate the expression and push the result onto the `rs` work stack.

Parameters

- **rs** (*WorkStack*) – The stack where the result of the evaluation is stored.
- **ws** (*WorkStack*) – The stack used by evaluation to perform intermediate computations. It will be returned in the same state as when the function is called.
- **xs** (*WorkStack*) – An auxiliary stack.

BaseExpression.getDim

```
int getDim(int d)
```

Return the d'th dimension in the expression.

Parameters d (int)

Return (int)

BaseExpression.getND

```
int getND()
```

Return the number of dimensions in the expression.

Return (int)

BaseExpression.getShape

```
shared_ptr<ndarray<int,1>> getShape()
```

Get the shape of the expression.

Return (int[])

BaseExpression.getSize

```
long long getSize()
```

Return the total number of elements in the expression (the product of the dimensions).

Return (long long)

BaseExpression.index

```
Expression::t index(int i)
Expression::t index(shared_ptr<ndarray<int,1>> indexes)
```

Get a single element in the expression.

Parameters

- **i** (int) – Index of the element to pick.
- **indexes** (int[]) – Multi-dimensional index of the element to pick.

Return (*Expression*)

BaseExpression.pick

```
Expression::t pick(shared_ptr<ndarray<int,1>> indexes)
Expression::t pick(shared_ptr<ndarray<int,2>> indexrows)
```

Picks a number of elements from the expression and returns them as a one-dimensional expression.

Parameters

- `indexes (int[])` – Indexes of the elements to pick
- `indexrows (int[][])` – Indexes of the elements to pick. Each row defines a separate multi-dimensional index.

Return (*Expression*)

`BaseExpression.slice`

```
Expression::t slice(int first, int last)
Expression::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
→ lasta)
```

Get a slice of the expression.

Parameters

- `first (int)` – Index of the first element in the slice.
- `last (int)` – Index of the last element in the slice plus one.
- `firsta (int[])` – Multi-dimensional index of the first element in the slice.
- `lasta (int[])` – Multi-dimensional index of the element after the end of the slice.

Return (*Expression*)

`BaseExpression.toString`

```
string toString()
```

Return a string representation of the expression object.

Return (string)

14.2.2 Class BaseVariable

`mosek::fusion::BaseVariable`

An abstract variable object. This class provides various default implementations of methods in *Variable*.

Members *BaseVariable.antiDiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.remove – Remove the variable from the model.
BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.
BaseVariable.setLevel – Input solution values for this variable
BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.
BaseVariable.toString – Create a string representation of the variable.
BaseVariable.transpose – Transpose the variable.
BaseVariable.tril – Convert from a square variable to a trilinear representation.

Implemented by *ModelVariable*, *SliceVariable*

BaseVariable.antidiag

```
Variable::t antidiag()
Variable::t antidiag(int index)
```

Return the antidiagonal of a square variable matrix.

Parameters *index* (int) – Index of the anti-diagonal
Return (*Variable*)

BaseVariable.asExpr

```
Expression::t asExpr()
```

Create an *Expression* object corresponding to $I \cdot V$, where I is the identity matrix and V is this variable.

Return (*Expression*)

BaseVariable.diag

```
Variable::t diag()
Variable::t diag(int index)
```

Return the diagonal of a square variable matrix.

Parameters *index* (int) – Index of the anti-diagonal
Return (*Variable*)

BaseVariable.dual

```
shared_ptr<ndarray<double,1>> dual()
```

Get the dual solution value of the variable as an array. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (double[])

BaseVariable.eval

```
void eval(WorkStack::t rs, WorkStack::t ws, WorkStack::t xs)
```

Evaluate the expression and push the result onto the *rs* work stack.

Parameters
 • *rs* (*WorkStack*) – The stack where the result of the evaluation is stored.

- `ws` (*WorkStack*) – The stack used by evaluation to perform intermediate computations. It will be returned in the same state as when the function is called.
- `xs` (*WorkStack*) – An auxiliary stack.

`BaseVariable.fromTril`

```
Variable::t fromTril(int d)
Variable::t fromTril(int dim0, int d)
```

Convert from a trilinear representation into a square variable.

Parameters

- `d` (`int`) – Dimension of the square variable.
- `dim0` (`int`) – Index of the trilinear variable slices in a multi-dimensional representation.

Return (*Variable*)

`BaseVariable.getDim`

```
int getDim(int d)
```

Return the `d`'th dimension in the expression.

Parameters `d` (`int`)

Return (`int`)

`BaseVariable.getModel`

```
Model::t getModel()
```

Get the *Model* object that the variable belongs to.

Return (*Model*)

`BaseVariable.getND`

```
int getND()
```

Get the number of dimensions in the variable shape.

Return (`int`)

`BaseVariable.getShape`

```
shared_ptr<ndarray<int,1>> getShape()
```

Get the variable shape.

Return (`int[]`)

`BaseVariable.getSize`

```
long long getSize()
```

Get the total number of elements in the variable.

Return (`long long`)

BaseVariable.index

```
Variable::t index(int index)
Variable::t index(shared_ptr<ndarray<int,1>> index)
Variable::t index(int i0, int i1)
Variable::t index(int i0, int i1, int i2)
```

Return a variable slice of size 1 corresponding to a single element in the variable object..

Parameters

- `index (int)`
- `index (int[])`
- `i0 (int)` – Index in the first dimension of the element requested.
- `i1 (int)` – Index in the second dimension of the element requested.
- `i2 (int)` – Index in the second dimension of the element requested.

Return (*Variable*)

BaseVariable.level

```
shared_ptr<ndarray<double,1>> level()
```

Get the primal solution value of the variable as an array. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (`double[]`)

BaseVariable.makeContinuous

```
void makeContinuous()
```

Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger

```
void makeInteger()
```

Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick

```
Variable::t pick(shared_ptr<ndarray<int,1>> idxs)
Variable::t pick(shared_ptr<ndarray<int,2>> midxs)
Variable::t pick(shared_ptr<ndarray<int,1>> i0, shared_ptr<ndarray<int,1>> i1)
Variable::t pick(shared_ptr<ndarray<int,1>> i0, shared_ptr<ndarray<int,1>> i1, ↵
↳ shared_ptr<ndarray<int,1>> i2)
```

Create a one-dimensional variable by picking a list of indexes from this variable.

Parameters

- `idxs (int[])` – Indexes of the elements requested.
- `midxs (int[][])` – A sequence of multi-dimensional indexes of the elements requested.
- `i0 (int[])` – Index along the first dimension.
- `i1 (int[])` – Index along the second dimension.
- `i2 (int[])` – Index along the third dimension.

Return (*Variable*)

BaseVariable.remove

```
void remove()
```

Remove the variable from the model and remove it from any constraints where it appears. Using the variable object after this method has been called results in undefined behavior.

BaseVariable.reshape

```
Variable::t reshape(shared_ptr<ndarray<int,1>> shape)
Variable::t reshape(int dim0)
Variable::t reshape(int dim0, int dim1)
Variable::t reshape(int dim0, int dim1, int dim2)
```

Reshape the variable. The new shape must have the same total size as the current.

Parameters

- `shape (int[])` – The new shape.
- `dim0 (int)` – First dimension of new shape
- `dim1 (int)` – Second dimension of new shape
- `dim2 (int)` – Third dimension of new shape

Return (*Variable*)

BaseVariable.setLevel

```
void setLevel(shared_ptr<ndarray<double,1>> v)
```

Set values for an initial solution for this variable. Note that these values are buffered until the solver is called; they are not available through the `level()` methods.

Parameters `v (double[])` – An array of values to be assigned to the variable.

BaseVariable.slice

```
Variable::t slice(int first, int last)
Variable::t slice(shared_ptr<ndarray<int,1>> first, shared_ptr<ndarray<int,1>>
↳last)
```

Create a slice variable by picking a range of indexes for each variable dimension.

Parameters

- `first (int)` – The index from which the slice begins.
- `first (int[])` – The index from which the slice begins.
- `last (int)` – The index after the last element of the slice.
- `last (int[])` – The index after the last element of the slice.

Return (*Variable*)

BaseVariable.toString

```
string toString()
```

Create a string representation of the variable.

Return (`string`)

BaseVariable.transpose

```
Variable::t transpose()
```

Return the transpose of the current variable. The variable must have at most two dimensions.

Return (*Variable*)

BaseVariable.tril

```
Variable::t tril()  
Variable::t tril(int dim1, int dim2)
```

Convert from a square variable to a trilinear representation.

Parameters

- **dim1** (int) – First dimension in the current shape containing the square variables.
- **dim2** (int) – Second dimension in the current shape containing the square variables.

Return (*Variable*)

14.2.3 Class BoundInterfaceConstraint

mosek::fusion::BoundInterfaceConstraint

Interface to either the upper bound or the lower bound of a ranged constraint.

This class is never explicitly instantiated; it is created by a *RangedConstraint* to allow accessing a bound value and the dual variable value corresponding to the relevant bound as a separate object. The constraint

$$b_l \leq a^T x \leq b_u$$

has two bounds and two dual variables; these are not immediately available through the *RangedConstraint* object, but can be accessed through a *BoundInterfaceConstraint*.

Implements *SliceConstraint*

Members *BoundInterfaceConstraint.dual* – Get the dual solution values of the constraint.

BoundInterfaceConstraint.index – Get a single element from a one-dimensional constraint.

BoundInterfaceConstraint.slice – Create a slice constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.remove – Remove the constraint from the model.

Constraint.update – Update part of a constraint.

SliceConstraint.toString – Create a human readable string representation of the constraint.

BoundInterfaceConstraint.dual

```
shared_ptr<ndarray<double,1>> dual()
```

Get the dual solution value of the constraint as an array.

Return (double[])

BoundInterfaceConstraint.index

```
Constraint::t index(int idx)
Constraint::t index(shared_ptr<ndarray<int,1>> idxa)
```

Get a single element from a one-dimensional constraint.

Parameters

- `idx (int)` – The element index.
- `idxa (int[])` – Array of integer coordinates in each dimension.

Return (*Constraint*)

`BoundInterfaceConstraint.slice`

```
Constraint::t slice(int first, int last)
Constraint::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
→ lasta)
```

Create a slice constraint.

Parameters

- `first (int)` – Index of the first element in the slice.
- `last (int)` – Index of the first element after the end of the slice.
- `firsta (int[])` – The indexes of first elements in the slice along each dimension.
- `lasta (int[])` – The indexes of first elements after the end of the slice along each dimension.

Return (*Constraint*)

14.2.4 Class `BoundInterfaceVariable`

`mosek::fusion::BoundInterfaceVariable`

Interface to either the upper bound or the lower bound of a ranged variable.

This class is never explicitly instantiated; is created by a *RangedVariable* to allow accessing a bound value and the dual variable value corresponding to the relevant bound as a separate object. The variable

$$b_l \leq x \leq b_u$$

has two bounds and two dual variables; these are not immediately available through the *RangedVariable* object, but can be accessed through a *BoundInterfaceVariable*.

Implements *SliceVariable*

Members *BaseVariable.asExpr* – Create an expression corresponding to the variable object.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.remove – Remove the variable from the model.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.

BaseVariable.setLevel – Input solution values for this variable

BaseVariable.toString – Create a string representation of the variable.

BaseVariable.tril – Convert from a square variable to a trilinear representation.

BoundInterfaceVariable.antidiag – Return the antidiagonal of a square variable matrix.

BoundInterfaceVariable.diag – Return the diagonal of a square variable matrix.

BoundInterfaceVariable.dual – Get the dual solution value of the variable.

BoundInterfaceVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BoundInterfaceVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

BoundInterfaceVariable.transpose – Transpose the variable.

`BoundInterfaceVariable.antidiag`

```
Variable::t antidiag()
Variable::t antidiag(int index)
```

Return the antidiagonal of a square variable matrix.

Parameters `index (int)` – Index of the anti-diagonal

Return (*Variable*)

`BoundInterfaceVariable.diag`

```
Variable::t diag()
Variable::t diag(int index)
```

Return the diagonal of a square variable matrix.

Parameters `index (int)` – Index of the anti-diagonal

Return (*Variable*)

`BoundInterfaceVariable.dual`

```
shared_ptr<ndarray<double,1>> dual()
```

Get the dual solution value of the variable as an array. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (`double[]`)

`BoundInterfaceVariable.pick`

```
Variable::t pick(shared_ptr<ndarray<int,1>> idxs)
Variable::t pick(shared_ptr<ndarray<int,2>> midxs)
Variable::t pick(shared_ptr<ndarray<int,1>> i0, shared_ptr<ndarray<int,1>> i1)
Variable::t pick(shared_ptr<ndarray<int,1>> i0, shared_ptr<ndarray<int,1>> i1,
↳ shared_ptr<ndarray<int,1>> i2)
```

Create a one-dimensional variable by picking a list of indexes from this variable.

Parameters

- `idxs (int[])` – Indexes of the elements requested.
- `midxs (int[][])` – A sequence of multi-dimensional indexes of the elements requested.
- `i0 (int[])` – Index along the first dimension.
- `i1 (int[])` – Index along the second dimension.
- `i2 (int[])` – Index along the third dimension.

Return (*Variable*)

`BoundInterfaceVariable.slice`

```
Variable::t slice(int first, int last)
Variable::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
↳lasta)
```

Create a slice variable by picking a range of indexes for each variable dimension.

Parameters

- `first (int)` – The index from which the slice begins.
- `last (int)` – The index after the last element of the slice.
- `firsta (int[])`
- `lasta (int[])`

Return (*Variable*)

`BoundInterfaceVariable.transpose`

```
Variable::t transpose()
```

Return the transpose of the current variable. The variable must have at most two dimensions.

Return (*Variable*)

14.2.5 Class ConeDomain

`mosek::fusion::ConeDomain`

Represent a domain defined by a conic constraints

Members *ConeDomain.axis* – Set the dimension along which the cones are created.

ConeDomain.axisIsSet – Returns true if the cone axis was set

ConeDomain.GetAxis – Get the dimension along which the cones are created.

ConeDomain.integral – Creates a domain of integral variables.

ConeDomain.withNamesOnAxis – Set index names in a specific axis.

ConeDomain.withShape – Set the shape of the domain.

`ConeDomain.axis`

```
ConeDomain::t axis(int a)
```

Set the dimension along which the cones are created.

Parameters `a (int)`

Return (*ConeDomain*)

`ConeDomain.axisIsSet`

```
bool axisIsSet()
```

Returns true if the cone axis was set

Return (bool)

ConeDomain.GetAxis

```
int GetAxis()
```

Get the dimension along which the cones are created.

Return (int)

ConeDomain.integral

```
ConeDomain::t integral()
```

Modify a given domain restricting its elements to be integral.

Return (*ConeDomain*)

ConeDomain.withNamesOnAxis

```
ConeDomain::t withNamesOnAxis(shared_ptr<ndarray<string,1>> names, int axis)
```

Set index names in a specific axis.

Parameters

- **names** (string[]) – List of names, this must match the actual dimension on that axis.
- **axis** (int) – The axis to change names on.

Return (*ConeDomain*)

ConeDomain.withShape

```
ConeDomain::t withShape(shared_ptr<ndarray<int,1>> shp)
ConeDomain::t withShape(int dim0)
ConeDomain::t withShape(int dim0, int dim1)
ConeDomain::t withShape(int dim0, int dim1, int dim2)
```

Set the shape of the domain.

Parameters

- **shp** (int[]) – The shape of the domain
- **dim0** (int) – First dimension
- **dim1** (int) – Second dimension
- **dim2** (int) – Third dimension

Return (*ConeDomain*)

14.2.6 Class ConicConstraint

mosek::fusion::ConicConstraint

This class represents a conic constraint of the form

$$Ax - b \in \mathcal{K}$$

where $\mathcal{K}$ is a cone. Then class is never explicitly instantiated, but is created using *Model.constraint* by specifying a conic domain.

Note that a conic constraint in *Fusion* is always *dense* in the sense that all member constraints are created in the underlying optimization problem immediately.

Implements *ModelConstraint*

Members *ConicConstraint.toString* – Create a human readable string representation of the constraint.

Constraint.dual – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint’s shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

ModelConstraint.remove – Remove the constraint from the model.

ConicConstraint.toString

```
string toString()
```

Create a human readable string representation of the constraint.

Return (string)

14.2.7 Class ConicVariable

`mosek::fusion::ConicVariable`

This class represents a conic variable of the form

$$x \in \mathcal{K}$$

where $\mathcal{K}$ is a cone. Then class is never explicitly instantiated, but is created using *Model.variable* by specifying a conic domain.

Note that a conic variable in *Fusion* is always *dense* in the sense that all member variables are created in the underlying optimization problem immediately.

Implements *ModelVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d’tth dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.
BaseVariable.setLevel – Input solution values for this variable
BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.
BaseVariable.transpose – Transpose the variable.
BaseVariable.tril – Convert from a square variable to a trilinear representation.
ConicVariable.toString – Create a string representation of the variable.
ModelVariable.remove – Remove the variable from the model.

`ConicVariable.toString`

```
string toString()
```

Create a string representation of the variable.

Return (string)

14.2.8 Class Constraint

`mosek::fusion::Constraint`

An abstract constraint object. This is the base class for all constraint types in Fusion.

The *Constraint* object can be an interface to the normal model constraint, e.g. *LinearConstraint* and *ConicConstraint*, to slices of other constraints or to concatenations of other constraints.

Primal and dual solution values can be accessed through the *Constraint* object.

Members *Constraint.dual* – Get the dual solution values of the constraint.
Constraint.getModel – Return the model that the constraint belongs to.
Constraint.getND – Return the number of dimensions in the constraint shape.
Constraint.getShape – Return the constraint's shape.
Constraint.getSize – Return the total number of elements in the constraint.
Constraint.index – Get a single element from a constraint.
Constraint.level – Get the primal solution values of the constraint.
Constraint.remove – Remove the constraint from the model.
Constraint.slice – Create a slice constraint.
Constraint.toString – Create a human readable string representation of the constraint.
Constraint.update – Update part of a constraint.

Static members *Constraint.hstack* – Stack a number of constraints horizontally.

Constraint.stack – Stack a number of constraints.

Constraint.vstack – Stack a number of constraints vertically.

Implemented by *ModelConstraint*, *SliceConstraint*

`Constraint.dual`

```
shared_ptr<ndarray<double,1>> dual()
```

Get the dual solution values of the constraint or its slice. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (double[])

`Constraint.getModel`

```
Model::t getModel()
```

Return the model that the constraint belongs to.

Return (*Model*)

Constraint.getND

```
int getND()
```

Return the number of dimensions in the constraint shape.

Return (int)

Constraint.getShape

```
shared_ptr<ndarray<int,1>> getShape()
```

Return the constraint's shape.

Return (int[])

Constraint.getSize

```
int getSize()
```

Return the total number of elements in the constraint.

Return (int)

Constraint.hstack

```
Constraint::t Constraint::hstack(Constraint::t v1, Constraint::t v2)
Constraint::t Constraint::hstack(Constraint::t v1, Constraint::t v2, Constraint::
→t v3)
Constraint::t Constraint::hstack(shared_ptr<ndarray<Constraint::t,1>> clist)
```

Stack a number of constraints horizontally.

Parameters

- v1 (*Constraint*) – The first constraint in the stack.
- v2 (*Constraint*) – The second constraint in the stack.
- v3 (*Constraint*) – The third constraint in the stack.
- clist (*Constraint*[]) – The constraints in the stack.

Return (*Constraint*)

Constraint.index

```
Constraint::t index(int idx)
Constraint::t index(shared_ptr<ndarray<int,1>> idxa)
```

Get a single element from a one-dimensional constraint.

Parameters

- idx (int) – The index of the element.
- idxa (int[]) – A multi-dimensional index of the element.

Return (*Constraint*)

Constraint.level

```
shared_ptr<ndarray<double,1>> level()
```

Get the primal solution values of the constraint. This amounts to evaluating the Ax part of the constraint expression for the relevant solution value. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (double[])

Constraint.remove

```
void remove()
```

Remove the constraint from the model. Using the constraint object after this method has been called results in undefined behavior.

Constraint.slice

```
Constraint::t slice(int first, int last)
Constraint::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
→ lasta)
```

Create a slice constraint.

Parameters

- first (int) – Index of the first element in the slice.
- last (int) – Index of the first element after the end of the slice.
- firsta (int[]) – The indexes of first elements in the slice along each dimension.
- lasta (int[]) – The indexes of first elements after the end of the slice along each dimension.

Return (*Constraint*)

Constraint.stack

```
Constraint::t Constraint::stack(Constraint::t v1, Constraint::t v2, int dim)
Constraint::t Constraint::stack(Constraint::t v1, Constraint::t v2, Constraint::
→ t v3, int dim)
Constraint::t Constraint::stack(shared_ptr<ndarray<Constraint::t,1>> clist, int
→ dim)
```

Stack a number of constraints.

Parameters

- v1 (*Constraint*) – The first constraint in the stack.
- v2 (*Constraint*) – The second constraint in the stack.
- dim (int) – The dimension in which to stack, 0 means vertically.
- v3 (*Constraint*) – The third constraint in the stack.
- clist (*Constraint*[]) – The constraints in the stack.

Return (*Constraint*)

Constraint.toString

```
string toString()
```

Create a human readable string representation of the constraint.

Return (string)

Constraint.update

```

void update(Expression::t expr, Variable::t x)
void update(Expression::t expr, Variable::t x, bool bfixupdate)
void update(Expression::t expr)
void update(shared_ptr<ndarray<double,1>> bfix)

```

Update entire or part of the left-hand side of a constraint (the expression). See [Sec. 7.10](#) for a tutorial.

If only **expr** is given, replace the entire previous expression with **expr**. The shape of **expr** must match the shape of the constraint.

If only **bfix** is given, replace the constant term of the expression with **bfix**. The length of **bfix** must match the size of the expression.

If **x** is given, replace all columns in the constraint defined by **x** by terms defined by **expr**, possibly including constant terms. Currently it is only possible to update linear columns. Attempting to update columns of semidefinite variables will result in an error.

Parameters

- **expr** (*Expression*) – The expression to update with.
- **x** (*Variable*) – Variable object defining the columns to update.
- **bfixupdate** (bool) – Whether to include fixed terms as well.
- **bfix** (double[]) – The fixed term to update with.

Constraint.vstack

```

Constraint::t Constraint::vstack(Constraint::t v1, Constraint::t v2)
Constraint::t Constraint::vstack(Constraint::t v1, Constraint::t v2, Constraint::
→t v3)
Constraint::t Constraint::vstack(shared_ptr<ndarray<Constraint::t,1>> clist)

```

Stack a number of constraints vertically.

Parameters

- **v1** (*Constraint*) – The first constraint in the stack.
- **v2** (*Constraint*) – The second constraint in the stack.
- **v3** (*Constraint*) – The third constraint in the stack.
- **clist** (*Constraint*[]) – The constraints in the stack.

Return (*Constraint*)

14.2.9 Class DJC

mosek::fusion::DJC

A class providing static methods to manipulate terms of disjunctive constraints (DJC).

Static members *DJC.AND* – Create a conjunction of simple terms.

DJC.term – Create a simple term.

DJC.AND

```

Term::t DJC::AND(shared_ptr<ndarray<SimpleTerm::t,1>> slist)
Term::t DJC::AND(SimpleTerm::t s1)
Term::t DJC::AND(SimpleTerm::t s1, SimpleTerm::t s2)
Term::t DJC::AND(SimpleTerm::t s1, SimpleTerm::t s2, SimpleTerm::t s3)

```

Creates a conjunction of existing simple terms $S_1, \dots, S_n$, that is a term representing:

$$S_1 \text{ AND } \dots \text{ AND } S_n.$$

Parameters

- `slist` (*SimpleTerm*[]) – A list of simple terms in the conjunction.
- `s1` (*SimpleTerm*) – A simple term.
- `s2` (*SimpleTerm*) – A simple term.
- `s3` (*SimpleTerm*) – A simple term.

Return (*Term*)

DJC.term

```
SimpleTerm::t DJC::term(Variable::t x, LinearDomain::t dom)
SimpleTerm::t DJC::term(Expression::t expr, LinearDomain::t dom)
SimpleTerm::t DJC::term(Variable::t x, RangeDomain::t dom)
SimpleTerm::t DJC::term(Expression::t expr, RangeDomain::t dom)
```

Creates a simple term, that is a condition which can be used as a building block in disjunctive constraints. A simple term has the form *an expression belongs to a domain*. Only linear and ranged domains are allowed in a disjunctive constraint.

A simple term can be used to construct a conjunctive term with *DJC.AND* or it can enter directly as a term into a disjunctive constraint via *Model.disjunction*.

Parameters

- `x` (*Variable*) – A variable.
- `dom` (*LinearDomain*) – The domain of this simple term.
- `dom` (*RangeDomain*) – The domain of this simple term.
- `expr` (*Expression*) – An expression.

Return (*SimpleTerm*)

14.2.10 Class Disjunction

`mosek::fusion::Disjunction`

A class representing a disjunctive constraint.

14.2.11 Class Domain

`mosek::fusion::Domain`

The *Domain* class defines a set of static method for creating various variable and constraint domains. A *Domain* object specifies a subset of $\mathbb{R}^n$, which can be used to define the feasible domain of variables and expressions.

For further details on the use of these, see *Model.variable* and *Model.constraint*.

Static members *Domain.axis* – Set the dimension along which the cones are created.

Domain.binary – Creates a domain of binary variables.

Domain.equalsTo – Defines the domain consisting of a fixed point.

Domain.greaterThan – Defines the domain specified by a lower bound in each dimension.

Domain.inDExpCone – Defines the dual exponential cone.

Domain.inDGeoMeanCone – Defines the domain of dual geometric mean cones.

Domain.inDPowerCone – Defines the dual power cone.

Domain.inPExpCone – Defines the primal exponential cone.

Domain.inPGeoMeanCone – Defines the domain of primal geometric mean cones.

Domain.inPPowerCone – Defines the primal power cone.

Domain.inPSDCone – Creates a domain of Positive Semidefinite matrices.

Domain.inQCone – Defines the domain of quadratic cones.

Domain.inRange – Creates a domain specified by a range in each dimension.
Domain.inRotatedQCone – Defines the domain of rotated quadratic cones.
Domain.inSVecPSDCone – Creates a domain of vectorized Positive Semidefinite matrices.
Domain.integral – Creates a domain of integral variables.
Domain.isTrilPSD – Creates a domain of Positive Semidefinite matrices.
Domain.lessThan – Defines the domain specified by an upper bound in each dimension.
Domain.sparse – Use a sparse representation.
Domain.symmetric – Impose symmetry on a given linear domain.
Domain.unbounded – Creates a domain in which variables are unbounded.

Domain.axis

```
ConeDomain::t Domain::axis(ConeDomain::t c, int a)
```

Set the dimension along which the cones are created. If this conic domain is used for a variable or expression of dimension d , then the conic constraint will be applicable to all vectors obtained by fixing the coordinates other than a -th and moving along the a -th coordinate. If $d = 2$ this can be used to define the conditions “every row of the matrix is in a cone” and “every column of a matrix is in a cone”.

The default is the last dimension $a = d - 1$.

Parameters

- c (*ConeDomain*) – A conic domain.
- a (int) – The axis.

Return (*ConeDomain*)

Domain.binary

```
RangeDomain::t Domain::binary(int n)
RangeDomain::t Domain::binary(int m, int n)
RangeDomain::t Domain::binary(shared_ptr<ndarray<int,1>> dims)
RangeDomain::t Domain::binary()
```

Create a domain of binary variables. A binary domain can only be used when creating variables, but is not allowed in a constraint. Another way of restricting variables to be integers is the method *Variable.makeInteger*.

Parameters

- n (int) – Dimension size.
- m (int) – Dimension size.
- $dims$ (int[]) – A list of dimension sizes.

Return (*RangeDomain*)

Domain.equalsTo

```
LinearDomain::t Domain::equalsTo(double b)
LinearDomain::t Domain::equalsTo(double b, int n)
LinearDomain::t Domain::equalsTo(double b, int m, int n)
LinearDomain::t Domain::equalsTo(double b, shared_ptr<ndarray<int,1>> dims)
LinearDomain::t Domain::equalsTo(shared_ptr<ndarray<double,1>> a1)
LinearDomain::t Domain::equalsTo(shared_ptr<ndarray<double,2>> a2)
LinearDomain::t Domain::equalsTo(shared_ptr<ndarray<double,1>> a1, shared_ptr
  ↳<ndarray<int,1>> dims)
LinearDomain::t Domain::equalsTo(Matrix::t mx)
```

Defines the domain consisting of a fixed point.

Parameters

- **b** (`double`) – A single value. This is scalable: it means that each element in the variable or constraint is fixed to b .
- **n** (`int`) – Dimension size.
- **m** (`int`) – Dimension size.
- **dims** (`int[]`) – A list of dimension sizes.
- **a1** (`double[]`) – A one-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **a2** (`double[][]`) – A two-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **mx** (*Matrix*) – A matrix of bound values. The shape must match the variable or constraint with which it is used.

Return (*LinearDomain*)

`Domain.greaterThan`

```
LinearDomain::t Domain::greaterThan(double b)
LinearDomain::t Domain::greaterThan(double b, int n)
LinearDomain::t Domain::greaterThan(double b, int m, int n)
LinearDomain::t Domain::greaterThan(double b, shared_ptr<ndarray<int,1>> dims)
LinearDomain::t Domain::greaterThan(shared_ptr<ndarray<double,1>> a1)
LinearDomain::t Domain::greaterThan(shared_ptr<ndarray<double,2>> a2)
LinearDomain::t Domain::greaterThan(shared_ptr<ndarray<double,1>> a1, shared_ptr
↪<ndarray<int,1>> dims)
LinearDomain::t Domain::greaterThan(Matrix::t mx)
```

Defines the domain specified by a lower bound in each dimension.

Parameters

- **b** (`double`) – A single value. This is scalable: it means that each element in the variable or constraint is greater than or equal to b .
- **n** (`int`) – Dimension size.
- **m** (`int`) – Dimension size.
- **dims** (`int[]`) – A list of dimension sizes.
- **a1** (`double[]`) – A one-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **a2** (`double[][]`) – A two-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **mx** (*Matrix*) – A matrix of bound values. The shape must match the variable or constraint with which it is used.

Return (*LinearDomain*)

`Domain.inExpCone`

```
ConeDomain::t Domain::inExpCone()
ConeDomain::t Domain::inExpCone(int m)
ConeDomain::t Domain::inExpCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of dual exponential cones:

$$\left\{ x \in \mathbb{R}^3 : x_1 \geq -x_3 e^{-1} e^{x_2/x_3}, x_1 > 0, x_3 < 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to an exponential cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors

obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.

Return ([ConeDomain](#))

Domain.inDGeoMeanCone

```
ConeDomain::t Domain::inDGeoMeanCone()
ConeDomain::t Domain::inDGeoMeanCone(int n)
ConeDomain::t Domain::inDGeoMeanCone(int m, int n)
ConeDomain::t Domain::inDGeoMeanCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of dual geometric mean cones:

$$\left\{ x \in \mathbb{R}^n : (n-1) \left(\prod_{i=1}^{n-1} x_i \right)^{1/(n-1)} \geq |x_n|, x_1, \dots, x_{n-1} \geq 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to a cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- **n** (int) – The size of each cone; at least 2.
- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.

Return ([ConeDomain](#))

Domain.inDPowerCone

```
ConeDomain::t Domain::inDPowerCone(double alpha)
ConeDomain::t Domain::inDPowerCone(double alpha, int m)
ConeDomain::t Domain::inDPowerCone(double alpha, shared_ptr<ndarray<int,1>> dims)
ConeDomain::t Domain::inDPowerCone(shared_ptr<ndarray<double,1>> alphas)
ConeDomain::t Domain::inDPowerCone(shared_ptr<ndarray<double,1>> alphas, int m)
ConeDomain::t Domain::inDPowerCone(shared_ptr<ndarray<double,1>> alphas, shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of dual power cones. For a single double argument **alpha** it defines the set

$$\left\{ x \in \mathbb{R}^n : \left(\frac{x_1}{\alpha} \right)^\alpha \left(\frac{x_2}{1-\alpha} \right)^{1-\alpha} \geq \sqrt{\sum_{i=3}^n x_i^2}, x_1, x_2 \geq 0 \right\}.$$

For an array **alphas** of length n_l , consisting of weights for the cone, it defines the set

$$\left\{ x \in \mathbb{R}^n : \prod_{i=1}^{n_l} \left(\frac{x_i}{\beta_i} \right)^{\beta_i} \geq \sqrt{x_{n_l+1}^2 + \dots + x_n^2}, x_1, \dots, x_{n_l} \geq 0 \right\}.$$

where β_i are the weights normalized to add up to 1, ie. $\beta_i = \alpha_i / (\sum_j \alpha_j)$ for $i = 1, \dots, n_l$.

The conic domain scales as follows. If a variable or expression constrained to a power cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors

obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- **alpha** (double) – The exponent of the power cone. Must be between 0 and 1.
- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.
- **alphas** (double[]) – The weights of the power cone. Must be positive.

Return ([ConeDomain](#))

`Domain.inPExpCone`

```
ConeDomain::t Domain::inPExpCone()
ConeDomain::t Domain::inPExpCone(int m)
ConeDomain::t Domain::inPExpCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of primal exponential cones:

$$\left\{ x \in \mathbb{R}^3 : x_1 \geq x_2 e^{x_3/x_2}, x_1, x_2 > 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to an exponential cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.

Return ([ConeDomain](#))

`Domain.inPGeoMeanCone`

```
ConeDomain::t Domain::inPGeoMeanCone()
ConeDomain::t Domain::inPGeoMeanCone(int n)
ConeDomain::t Domain::inPGeoMeanCone(int m, int n)
ConeDomain::t Domain::inPGeoMeanCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of primal geometric mean cones:

$$\left\{ x \in \mathbb{R}^n : \left(\prod_{i=1}^{n-1} x_i \right)^{1/(n-1)} \geq |x_n|, x_1 \dots, x_{n-1} \geq 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to a cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- **n** (int) – The size of each cone; at least 2.
- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.

Return ([ConeDomain](#))

Domain.inPPowerCone

```

ConeDomain::t Domain::inPPowerCone(double alpha)
ConeDomain::t Domain::inPPowerCone(double alpha, int m)
ConeDomain::t Domain::inPPowerCone(double alpha, shared_ptr<ndarray<int,1>> dims)
ConeDomain::t Domain::inPPowerCone(shared_ptr<ndarray<double,1>> alphas)
ConeDomain::t Domain::inPPowerCone(shared_ptr<ndarray<double,1>> alphas, int m)
ConeDomain::t Domain::inPPowerCone(shared_ptr<ndarray<double,1>> alphas, shared_
→ptr<ndarray<int,1>> dims)

```

Defines the domain of primal power cones. For a single double argument `alpha` it defines the set

$$\left\{ x \in \mathbb{R}^n : x_1^\alpha x_2^{1-\alpha} \geq \sqrt{\sum_{i=3}^n x_i^2}, x_1, x_2 \geq 0 \right\}.$$

For an array `alphas` of length n_l , consisting of weights for the cone, it defines the set

$$\left\{ x \in \mathbb{R}^n : \prod_{i=1}^{n_l} x_i^{\beta_i} \geq \sqrt{x_{n_l+1}^2 + \dots + x_n^2}, x_1, \dots, x_{n_l} \geq 0 \right\}.$$

where β_i are the weights normalized to add up to 1, ie. $\beta_i = \alpha_i / (\sum_j \alpha_j)$ for $i = 1, \dots, n_l$.

The conic domain scales as follows. If a variable or expression constrained to a power cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also [Domain.axis](#).

If m was given the domain is a product of m such cones.

Parameters

- `alpha` (double) – The exponent of the power cone. Must be between 0 and 1.
- `m` (int) – The number of cones (default 1).
- `dims` (int[]) – Shape of the domain.
- `alphas` (double[]) – The weights of the power cone. Must be positive.

Return ([ConeDomain](#))

Domain.inPSDCone

```

PSDDomain::t Domain::inPSDCone()
PSDDomain::t Domain::inPSDCone(int n)
PSDDomain::t Domain::inPSDCone(int n, int m)

```

When used to create a new variable in [Model.variable](#) it defines a domain of symmetric positive semidefinite matrices, that is

$$\mathcal{S}_+^n = \{ X \in \mathbb{R}^{n \times n} : X = X^T, y^T X y \geq 0, \text{ for all } y \}.$$

The shape of the result is $n \times n$. If m was given the domain is a product of m such cones, that is of shape $m \times n \times n$.

When used to impose a constraint in [Model.constraint](#) it defines a domain

$$\left\{ X \in \mathbb{R}^{n \times n} : \frac{1}{2}(X + X^T) \in \mathcal{S}_+^n \right\}.$$

i.e. a positive semidefinite matrix without the symmetry assumption.

Parameters

- `n` (int) – Dimension of the PSD matrix.

- **m** (int) – Number of matrices (default 1).

Return (*PSDDomain*)

Domain.inQCone

```
ConeDomain::t Domain::inQCone()
ConeDomain::t Domain::inQCone(int n)
ConeDomain::t Domain::inQCone(int m, int n)
ConeDomain::t Domain::inQCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of quadratic cones:

$$\left\{ x \in \mathbb{R}^n : x_1^2 \geq \sum_{i=2}^n x_i^2, x_1 \geq 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to a quadratic cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also *Domain.axis*.

If m was given the domain is a product of m such cones.

Parameters

- **n** (int) – The size of each cone; at least 2.
- **m** (int) – The number of cones (default 1).
- **dims** (int[]) – Shape of the domain.

Return (*ConeDomain*)

Domain.inRange

```
RangeDomain::t Domain::inRange(double lb, double ub)
RangeDomain::t Domain::inRange(double lb, shared_ptr<ndarray<double,1>> uba)
RangeDomain::t Domain::inRange(shared_ptr<ndarray<double,1>> lba, double ub)
RangeDomain::t Domain::inRange(shared_ptr<ndarray<double,1>> lba, shared_ptr
↳<ndarray<double,1>> uba)
RangeDomain::t Domain::inRange(double lb, double ub, shared_ptr<ndarray<int,1>>
↳dims)
RangeDomain::t Domain::inRange(double lb, shared_ptr<ndarray<double,1>> uba,
↳shared_ptr<ndarray<int,1>> dims)
RangeDomain::t Domain::inRange(shared_ptr<ndarray<double,1>> lba, double ub,
↳shared_ptr<ndarray<int,1>> dims)
RangeDomain::t Domain::inRange(shared_ptr<ndarray<double,1>> lba, shared_ptr
↳<ndarray<double,1>> uba, shared_ptr<ndarray<int,1>> dims)
RangeDomain::t Domain::inRange(shared_ptr<ndarray<double,2>> lba, shared_ptr
↳<ndarray<double,2>> uba)
RangeDomain::t Domain::inRange(Matrix::t lbm, Matrix::t ubm)
```

Creates a domain specified by a range in each dimension.

Parameters

- **lb** (double) – The lower bound as a common scalar value.
- **ub** (double) – The upper bound as a common scalar value.
- **uba** (double[]) – The upper bounds as an array.
- **uba** (double[][]) – The upper bounds as an array.
- **lba** (double[]) – The lower bounds as an array.
- **lba** (double[][]) – The lower bounds as an array.
- **dims** (int[]) – A list of dimension sizes.

- `lbm (Matrix)` – The lower bounds as a *Matrix* object.
- `ubm (Matrix)` – The upper bounds as a *Matrix* object.

Return (*RangeDomain*)

`Domain.inRotatedQCone`

```
ConeDomain::t Domain::inRotatedQCone()
ConeDomain::t Domain::inRotatedQCone(int n)
ConeDomain::t Domain::inRotatedQCone(int m, int n)
ConeDomain::t Domain::inRotatedQCone(shared_ptr<ndarray<int,1>> dims)
```

Defines the domain of rotated quadratic cones:

$$\left\{ x \in \mathbb{R}^n : 2x_1x_2 \geq \sum_{i=3}^n x_i^2, x_1, x_2 \geq 0 \right\}$$

The conic domain scales as follows. If a variable or expression constrained to a quadratic cone is not a single vector but a d -dimensional variable then the conic domain is applicable to all vectors obtained by fixing the first $d - 1$ coordinates and moving along the last coordinate. If $d = 2$ it means that each row of a matrix must belong to a cone. See also *Domain.axis*.

If m was given the domain is a product of m such cones.

Parameters

- `n (int)` – The size of each cone; at least 3.
- `m (int)` – The number of cones (default 1).
- `dims (int[])` – Shape of the domain.

Return (*ConeDomain*)

`Domain.inSVecPSDCone`

```
ConeDomain::t Domain::inSVecPSDCone()
ConeDomain::t Domain::inSVecPSDCone(int n)
ConeDomain::t Domain::inSVecPSDCone(int d1, int d2)
ConeDomain::t Domain::inSVecPSDCone(shared_ptr<ndarray<int,1>> dims)
```

Creates a domain of vectorized Positive Semidefinite matrices:

$$\{(x_1, \dots, x_{d(d+1)/2}) \in \mathbb{R}^n : \text{sMat}(x) \in \mathcal{S}_+^d\} = \{\text{sVec}(X) : X \in \mathcal{S}_+^d\},$$

where

$$\text{sVec}(X) = (X_{11}, \sqrt{2}X_{21}, \dots, \sqrt{2}X_{d1}, X_{22}, \sqrt{2}X_{32}, \dots, X_{dd}),$$

and

$$\text{sMat}(x) = \begin{bmatrix} x_1 & x_2/\sqrt{2} & \cdots & x_d/\sqrt{2} \\ x_2/\sqrt{2} & x_{d+1} & \cdots & x_{2d-1}/\sqrt{2} \\ \cdots & \cdots & \cdots & \cdots \\ x_d/\sqrt{2} & x_{2d-1}/\sqrt{2} & \cdots & x_{d(d+1)/2} \end{bmatrix}.$$

In other words, the domain consists of vectorizations of the lower-triangular part of a positive semidefinite matrix, with the non-diagonal elements additionally rescaled.

Parameters

- `n (int)` – Length of the vectorization - this must be of the form $d * (d + 1) / 2$ for some positive integer d .
- `d1 (int)` – Size of first dimension of the domain.
- `d2 (int)` – Size of second dimension of the domain.

- `dims (int[])` – Shape of the domain.

Return (*ConeDomain*)

`Domain.integral`

```
ConeDomain::t Domain::integral(ConeDomain::t c)
LinearDomain::t Domain::integral(LinearDomain::t ld)
RangeDomain::t Domain::integral(RangeDomain::t rd)
```

Modify a given domain restricting its elements to be integral. An integral domain can only be used when creating variables, but is not allowed in a constraint. Another way of restricting variables to be integers is the method *Variable.makeInteger*.

Parameters

- `c (ConeDomain)` – A conic domain.
- `ld (LinearDomain)` – A linear domain.
- `rd (RangeDomain)` – A ranged domain.

Return

- (*ConeDomain*)
- (*LinearDomain*)
- (*RangeDomain*)

`Domain.isTrilPSD`

```
PSDDomain::t Domain::isTrilPSD()
PSDDomain::t Domain::isTrilPSD(int n)
PSDDomain::t Domain::isTrilPSD(int n, int m)
```

Creates an object representing a cone of the form

$$\{X \in \mathbb{R}^{n \times n} : \text{tril}(X) \in \mathcal{S}_+^n\}.$$

i.e. the lower triangular part of X defines the symmetric matrix that is positive semidefinite. The shape of the result is $n \times n$. If m was given the domain is a product of m such cones, that is of shape $m \times n \times n$.

Parameters

- `n (int)` – Dimension of the PSD matrix.
- `m (int)` – Number of matrices (default 1).

Return (*PSDDomain*)

`Domain.lessThan`

```
LinearDomain::t Domain::lessThan(double b)
LinearDomain::t Domain::lessThan(double b, int n)
LinearDomain::t Domain::lessThan(double b, int m, int n)
LinearDomain::t Domain::lessThan(double b, shared_ptr<ndarray<int,1>> dims)
LinearDomain::t Domain::lessThan(shared_ptr<ndarray<double,1>> a1)
LinearDomain::t Domain::lessThan(shared_ptr<ndarray<double,2>> a2)
LinearDomain::t Domain::lessThan(shared_ptr<ndarray<double,1>> a1, shared_ptr
-><ndarray<int,1>> dims)
LinearDomain::t Domain::lessThan(Matrix::t mx)
```

Defines the domain specified by an upper bound in each dimension.

Parameters

- **b** (`double`) – A single value. This is scalable: it means that each element in the variable or constraint is less than or equal to *b*.
- **n** (`int`) – Dimension size.
- **m** (`int`) – Dimension size.
- **dims** (`int[]`) – A list of dimension sizes.
- **a1** (`double[]`) – A one-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **a2** (`double[][]`) – A two-dimensional array of bounds. The shape must match the variable or constraint with which it is used.
- **mx** (*Matrix*) – A matrix of bound values. The shape must match the variable or constraint with which it is used.

Return (*LinearDomain*)

`Domain.sparse`

```
LinearDomain::t Domain::sparse(LinearDomain::t ld, shared_ptr<ndarray<int,1>>>
    ↪ sparsity)
LinearDomain::t Domain::sparse(LinearDomain::t ld, shared_ptr<ndarray<int,2>>>
    ↪ sparsity)
RangeDomain::t Domain::sparse(RangeDomain::t rd, shared_ptr<ndarray<int,1>>>
    ↪ sparsity)
RangeDomain::t Domain::sparse(RangeDomain::t rd, shared_ptr<ndarray<int,2>>>
    ↪ sparsity)
```

Given a linear domain, this method explicitly suggest to *Fusion* that a sparse representation is helpful.

Parameters

- **ld** (*LinearDomain*) – The linear sparse domain.
- **sparsity** (`int[]`) – Sparsity pattern.
- **sparsity** (`int[][]`) – Sparsity pattern.
- **rd** (*RangeDomain*) – The ranged sparse domain.

Return

- (*LinearDomain*)
- (*RangeDomain*)

`Domain.symmetric`

```
SymmetricLinearDomain::t Domain::symmetric(LinearDomain::t ld)
SymmetricRangeDomain::t Domain::symmetric(RangeDomain::t rd)
```

Given a linear domain *D* whose shape is that of square matrices, this method returns a domain consisting of symmetric matrices in *D*.

Parameters

- **ld** (*LinearDomain*) – The linear domain to be symmetrized.
- **rd** (*RangeDomain*) – The ranged domain to be symmetrized.

Return

- (*SymmetricLinearDomain*)
- (*SymmetricRangeDomain*)

`Domain.unbounded`

```

LinearDomain::t Domain::unbounded()
LinearDomain::t Domain::unbounded(int n)
LinearDomain::t Domain::unbounded(int m, int n)
LinearDomain::t Domain::unbounded(shared_ptr<ndarray<int,1>> dims)

```

Creates a domain in which variables are unbounded.

Parameters

- **n** (int) – Dimension size.
- **m** (int) – Dimension size.
- **dims** (int[]) – A list of dimension sizes.

Return (*LinearDomain*)

14.2.12 Class Expr

`mosek::fusion::Expr`

It represents an expression of the form $Ax + b$, where A is a matrix on sparse form, x is a variable vector and b is a vector of scalars.

Additionally, the class defines a set of static methods for constructing and manipulating various expressions.

Implements *BaseExpression*

Members *BaseExpression.getDim* – Return the d'th dimension in the expression.

BaseExpression.getND – Return the number of dimensions in the expression.

BaseExpression.getShape – Get the shape of the expression.

BaseExpression.getSize – Return the total number of elements in the expression (the product of the dimensions).

BaseExpression.index – Get a single element in the expression.

BaseExpression.pick – Pick a number of elements from the expression.

BaseExpression.slice – Get a slice of the expression.

BaseExpression.toString – Return a string representation of the expression object.

Expr.eval – Evaluate the expression and push the result onto the work stack.

Static members *Expr.add* – Compute the sum of expressions.

Expr.condense – Flatten expression and remove all structural zeros.

Expr.constTerm – Create an expression consisting of a constant vector of values.

Expr.dot – Return a scalar expression object representing the dot-product of two items.

Expr.flatten – Reshape the expression into a vector.

Expr.hstack – Stack a list of expressions horizontally (i.e. along the second dimension).

Expr.mul – Multiply two items.

Expr.mulDiag – Compute the diagonal of the product of two matrices.

Expr.mulElm – Element-wise product of two items.

Expr.neg – Change the sign of an expression

Expr.ones – Create an expression consisting of ones.

Expr.outer – Return the outer-product of two vectors.

Expr.repeat – Repeat an expression a number of times in the given dimension.

Expr.reshape – Reshape the expression into a different shape with the same number of elements.

Expr.stack – Stack a list of expressions in an arbitrary dimension.

Expr.sub – Compute the difference of two expressions.

Expr.sum – Sum the elements of an expression.

Expr.transpose – Transpose a two-dimensional expression.

Expr.vstack – Stack a list of expressions vertically (i.e. along the first dimension).

Expr.zeros – Create an expression consisting of zeros.

Expr.add

```

Expression::t Expr::add(Expression::t e1, Expression::t e2)
Expression::t Expr::add(Expression::t e1, shared_ptr<ndarray<double,1>> a1)
Expression::t Expr::add(Expression::t e1, shared_ptr<ndarray<double,2>> a2)
Expression::t Expr::add(shared_ptr<ndarray<double,1>> a1, Expression::t e2)
Expression::t Expr::add(shared_ptr<ndarray<double,2>> a2, Expression::t e2)
Expression::t Expr::add(Expression::t e1, double c)
Expression::t Expr::add(double c, Expression::t e2)
Expression::t Expr::add(Expression::t e1, Matrix::t m)
Expression::t Expr::add(Matrix::t m, Expression::t e2)
Expression::t Expr::add(Expression::t e1, NDSparseArray::t n)
Expression::t Expr::add(NDSparseArray::t n, Expression::t e2)
Expression::t Expr::add(shared_ptr<ndarray<Variable::t,1>> vs)
Expression::t Expr::add(shared_ptr<ndarray<Expression::t,1>> exps)

```

Computes the sum of two or more expressions or variables. The following types of arguments are allowed:

A	B
Variable	Variable
Expression	Expression
double	
double[]	
double[,]	
Matrix	
NDSparseArray	

By symmetry both `add(A,B)` and `add(B,A)` are available.

The arguments must have the same shapes and the returned expression also has that shape. If one of the arguments is a single scalar, it is promoted to the shape that matches the shape of the other argument, i.e. the scalar is added to all entries of the other argument.

Parameters

- `e1` (*Expression*) – An expression.
- `e2` (*Expression*) – An expression.
- `a1` (`double[]`) – A one-dimensional array of constants.
- `a2` (`double[][]`) – A two-dimensional array of constants.
- `c` (`double`) – A constant.
- `m` (*Matrix*) – A Matrix object.
- `n` (*NDSparseArray*) – An NDSparseArray object.
- `vs` (*Variable[]*) – A list of variables. All variables in the array must have the same shape. The list must contain at least one element.
- `exps` (*Expression[]*) – A list of expressions. All expressions in the array must have the same shape. The list must contain at least one element.

Return (*Expression*)

Expr.condense

```

Expression::t Expr::condense(Expression::t e)

```

Flatten expression and remove all structural zeros.

Parameters `e` (*Expression*) – Expression to be condensed.

Return (*Expression*)

Expr.constTerm

```
Expression::t Expr::constTerm(shared_ptr<ndarray<double,1>> vals1)
Expression::t Expr::constTerm(shared_ptr<ndarray<double,2>> vals2)
Expression::t Expr::constTerm(int size, double val)
Expression::t Expr::constTerm(shared_ptr<ndarray<int,1>> shp, double val)
Expression::t Expr::constTerm(shared_ptr<ndarray<int,1>> shp, shared_ptr<ndarray
↳<int,2>> sparsity, shared_ptr<ndarray<double,1>> vals1)
Expression::t Expr::constTerm(shared_ptr<ndarray<int,1>> shp, shared_ptr<ndarray
↳<int,2>> sparsity, double val)
Expression::t Expr::constTerm(double val)
Expression::t Expr::constTerm(Matrix::t m)
Expression::t Expr::constTerm(NDSparseArray::t nda)
```

Create an expression consisting of a constant vector of values.

Parameters

- vals1 (double[]) – A vector initializing the expression.
- vals2 (double[][]) – An array initializing the expression.
- size (int) – Length of the vector to be constructed.
- val (double) – A scalar value to be repeated in all entries of the expression.
- shp (int[]) – Defines the shape of the expression.
- sparsity (int[][]) – Sparsity pattern.
- m (*Matrix*) – A matrix of values initializing the expression.
- nda (*NDSparseArray*) – An multi-dimensional sparse array initializing the expression.

Return (*Expression*)

Expr.dot

```
Expression::t Expr::dot(Parameter::t p, Expression::t e)
Expression::t Expr::dot(Expression::t e, Parameter::t p)
Expression::t Expr::dot(shared_ptr<ndarray<double,1>> c1, Expression::t e)
Expression::t Expr::dot(shared_ptr<ndarray<double,2>> c2, Expression::t e)
Expression::t Expr::dot(NDSparseArray::t nda, Expression::t e)
Expression::t Expr::dot(Matrix::t m, Expression::t e)
Expression::t Expr::dot(Expression::t e, shared_ptr<ndarray<double,1>> c1)
Expression::t Expr::dot(Expression::t e, NDSparseArray::t nda)
Expression::t Expr::dot(Expression::t e, shared_ptr<ndarray<double,2>> c2)
Expression::t Expr::dot(Expression::t e, Matrix::t m)
```

Return an object representing the inner product (dot product) $x^T y = \sum_{i=1}^n x_i y_i$ of two objects x, y of size n .

Both arguments must have the same length when flattened. In particular, they can be two vectors of the same length or two matrices of the same shape.

Parameters

- p (*Parameter*) – A parameter.
- e (*Expression*) – An expression object.
- c1 (double[]) – A one-dimensional coefficient vector.
- c2 (double[][]) – A two-dimensional coefficient array.
- nda (*NDSparseArray*) – A multi-dimensional sparse array.
- m (*Matrix*) – A matrix object.

Return (*Expression*)

Expr.eval

```
void eval(WorkStack::t rs, WorkStack::t ws, WorkStack::t xs)
```

Evaluate the expression and push the result onto the `rs` work stack.

Parameters

- `rs` (*WorkStack*) – The stack where the result of the evaluation is stored.
- `ws` (*WorkStack*) – The stack used by evaluation to perform intermediate computations. It will be returned in the same state as when the function is called.
- `xs` (*WorkStack*) – An auxiliary stack.

Expr.flatten

```
Expression::t Expr::flatten(Expression::t e)
```

Reshape the expression into a vector.

Parameters `e` (*Expression*) – The expression to be flattened.

Return (*Expression*)

Expr.hstack

```
Expression::t Expr::hstack(shared_ptr<ndarray<Expression::t,1>> exprs)
Expression::t Expr::hstack(Expression::t e1, Expression::t e2)
Expression::t Expr::hstack(Expression::t e1, double a2)
Expression::t Expr::hstack(double a1, Expression::t e2)
Expression::t Expr::hstack(double a1, double a2, Expression::t e3)
Expression::t Expr::hstack(double a1, Expression::t e2, double a3)
Expression::t Expr::hstack(double a1, Expression::t e2, Expression::t e3)
Expression::t Expr::hstack(Expression::t e1, double a2, double a3)
Expression::t Expr::hstack(Expression::t e1, double a2, Expression::t e3)
Expression::t Expr::hstack(Expression::t e1, Expression::t e2, double a3)
Expression::t Expr::hstack(Expression::t e1, Expression::t e2, Expression::t e3)
```

Stack a list of expressions horizontally (i.e. along the second dimension). The expressions must have the same shape, except for the second dimension. The arguments may be any combination of expressions, scalar constants and variables.

For example, if x^1, x^2, x^3 are three vectors of length n then their horizontal stack is the matrix

$$\begin{bmatrix} | & | & | \\ x^1 & x^2 & x^3 \\ | & | & | \end{bmatrix}$$

of shape $(n,3)$.

Parameters

- `exprs` (*Expression*[]) – A list of expressions.
- `e1` (*Expression*) – An expression.
- `e2` (*Expression*) – An expression.
- `a2` (*double*) – A scalar constant.
- `a1` (*double*) – A scalar constant.
- `e3` (*Expression*) – An expression.
- `a3` (*double*) – A scalar constant.

Return (*Expression*)

Expr.mul

```

Expression::t Expr::mul(Parameter::t p, Expression::t expr)
Expression::t Expr::mul(Expression::t expr, Parameter::t p)
Expression::t Expr::mul(Matrix::t mx, Variable::t v)
Expression::t Expr::mul(Variable::t v, Matrix::t mx)
Expression::t Expr::mul(Matrix::t mx, Expression::t expr)
Expression::t Expr::mul(Expression::t expr, Matrix::t mx)
Expression::t Expr::mul(shared_ptr<ndarray<double,2>> a, Expression::t expr)
Expression::t Expr::mul(Expression::t expr, shared_ptr<ndarray<double,2>> a)
Expression::t Expr::mul(shared_ptr<ndarray<double,1>> a, Expression::t expr)
Expression::t Expr::mul(Expression::t expr, shared_ptr<ndarray<double,1>> a)
Expression::t Expr::mul(double c, Expression::t expr)
Expression::t Expr::mul(Expression::t expr, double c)

```

Compute the product (in the sense of matrix multiplication or scalar-by-matrix multiplication) of two arguments.

The operands must be at most two-dimensional. One of the arguments must be a constant, a vector of constants or a matrix of constants. The other argument can be a variable or expression. This allows to produce matrix expressions where the entries are linear combinations of variables.

The size and shape of the arguments must adhere to the rules of linear algebra.

Parameters

- **p** (*Parameter*) – A parameter object.
- **expr** (*Expression*) – An expression.
- **mx** (*Matrix*) – A matrix.
- **v** (*Variable*) – A variable.
- **a** (double[[[[]]]) – Scalar data.
- **a** (double[[[]]]) – Scalar data.
- **c** (double) – A scalar value.

Return (*Expression*)

Expr.mulDiag

```

Expression::t Expr::mulDiag(shared_ptr<ndarray<double,2>> a, Expression::t expr)
Expression::t Expr::mulDiag(shared_ptr<ndarray<double,2>> a, Variable::t v)
Expression::t Expr::mulDiag(Expression::t expr, shared_ptr<ndarray<double,2>> a)
Expression::t Expr::mulDiag(Variable::t v, shared_ptr<ndarray<double,2>> a)
Expression::t Expr::mulDiag(Matrix::t mx, Expression::t expr)
Expression::t Expr::mulDiag(Expression::t expr, Matrix::t mx)
Expression::t Expr::mulDiag(Matrix::t mx, Variable::t v)
Expression::t Expr::mulDiag(Variable::t v, Matrix::t mx)
Expression::t Expr::mulDiag(Parameter::t p, Expression::t expr)
Expression::t Expr::mulDiag(Expression::t expr, Parameter::t p)
Expression::t Expr::mulDiag(Parameter::t p, Variable::t v)
Expression::t Expr::mulDiag(Variable::t v, Parameter::t p)

```

Compute the diagonal of the product of two matrices. If $A \in \mathbb{M}(m, n)$ and $B \in \mathbb{M}(n, m)$, the result is a vector expression of length m equal to $\mathbf{diag}(AB)$.

Parameters

- **a** (double[[[[]]]) – A constant matrix.
- **expr** (*Expression*) – An expression object.
- **v** (*Variable*) – A variable object.
- **mx** (*Matrix*) – A matrix object.

- p (*Parameter*) – A parameter object.

Return (*Expression*)

Expr.mulElm

```
Expression::t Expr::mulElm(Expression::t expr, Parameter::t p)
Expression::t Expr::mulElm(Parameter::t p, Expression::t expr)
Expression::t Expr::mulElm(Expression::t expr, NDSparseArray::t spm)
Expression::t Expr::mulElm(Expression::t expr, shared_ptr<ndarray<double,1>> a1)
Expression::t Expr::mulElm(Expression::t expr, shared_ptr<ndarray<double,2>> a2)
Expression::t Expr::mulElm(Expression::t expr, Matrix::t m)
Expression::t Expr::mulElm(shared_ptr<ndarray<double,1>> a1, Expression::t expr)
Expression::t Expr::mulElm(shared_ptr<ndarray<double,2>> a2, Expression::t expr)
Expression::t Expr::mulElm(NDSparseArray::t spm, Expression::t expr)
Expression::t Expr::mulElm(Matrix::t m, Expression::t expr)
```

Returns the element-wise product of two items. The two operands must have the same shape and the returned expression also has this shape.

Parameters

- expr (*Expression*) – An expression object.
- p (*Parameter*) – A parameter object.
- spm (*NDSparseArray*) – A multidimensional sparse array object.
- a1 (double[]) – A one-dimensional coefficient array.
- a2 (double[][])) – A two-dimensional coefficient array.
- m (*Matrix*) – A matrix object.

Return (*Expression*)

Expr.neg

```
Expression::t Expr::neg(Expression::t e)
```

Return a new expression object representing the given one with opposite sign.

Parameters e (*Expression*) – An expression object.

Return (*Expression*)

Expr.ones

```
Expression::t Expr::ones(int size)
Expression::t Expr::ones(shared_ptr<ndarray<int,1>> shp)
Expression::t Expr::ones(shared_ptr<ndarray<int,1>> shp, shared_ptr<ndarray<int,
→2>> sparsity)
Expression::t Expr::ones()
```

Create an expression consisting of ones.

Parameters

- size (int) – Length of the vector to be constructed.
- shp (int[]) – Defines the shape of the expression.
- sparsity (int[][])) – Defines the sparsity pattern of the expression - everything outside the sparsity pattern will be zero.

Return (*Expression*)

Expr.outer

```

Expression::t Expr::outer(Expression::t e, shared_ptr<ndarray<double,1>> a)
Expression::t Expr::outer(shared_ptr<ndarray<double,1>> a, Expression::t e)
Expression::t Expr::outer(Expression::t e, Matrix::t m)
Expression::t Expr::outer(Matrix::t m, Expression::t e)
Expression::t Expr::outer(Expression::t e, Parameter::t p)
Expression::t Expr::outer(Parameter::t p, Expression::t e)

```

Return an expression representing the outer product xy^T of two vectors x, y . If x has length k and y has length n then the result is of shape (k,n) .

Parameters

- e (*Expression*) – A vector expression.
- a (`double[]`) – A vector of constants.
- m (*Matrix*) – A one-dimensional matrix.
- p (*Parameter*) – A vector parameter.

Return (*Expression*)

`Expr.repeat`

```

Expression::t Expr::repeat(Expression::t e, int n, int d)
Expression::t Expr::repeat(Variable::t x, int n, int d)

```

Repeat an expression a number of times in the given dimension. This is equivalent to stacking n copies of the expression in dimension d ; see *Expr.stack*.

Parameters

- e (*Expression*) – The expression to repeat.
- n (`int`) – Number of times to repeat. Must be strictly positive.
- d (`int`) – The dimension in which to repeat. Must define a valid dimension index.
- x (*Variable*) – The variable to repeat.

Return (*Expression*)

`Expr.reshape`

```

Expression::t Expr::reshape(Expression::t e, shared_ptr<ndarray<int,1>> newshape)
Expression::t Expr::reshape(Expression::t e, int size)
Expression::t Expr::reshape(Expression::t e, int dimi, int dimj)

```

Reshape the expression into a different shape with the same number of elements.

Parameters

- e (*Expression*) – The expression to reshape.
- `newshape` (`int[]`) – Reshape into an expression of this shape.
- `size` (`int`) – Reshape into a one-dimensional expression of this size.
- `dimi` (`int`) – The first dimension size.
- `dimj` (`int`) – The second dimension size.

Return (*Expression*)

`Expr.stack`

```

Expression::t Expr::stack(int dim, shared_ptr<ndarray<Expression::t,1>> exprs)
Expression::t Expr::stack(int dim, Expression::t e1, Expression::t e2)
Expression::t Expr::stack(int dim, Expression::t e1, double a2)

```

(continues on next page)

(continued from previous page)

```
Expression::t Expr::stack(int dim, double a1, Expression::t e2)
Expression::t Expr::stack(int dim, double a1, double a2, Expression::t e1)
Expression::t Expr::stack(int dim, double a1, Expression::t e2, double a3)
Expression::t Expr::stack(int dim, double a1, Expression::t e2, Expression::t e3)
Expression::t Expr::stack(int dim, Expression::t e1, double a2, double a3)
Expression::t Expr::stack(int dim, Expression::t e1, double a2, Expression::t e3)
Expression::t Expr::stack(int dim, Expression::t e1, Expression::t e2, double a3)
Expression::t Expr::stack(int dim, Expression::t e1, Expression::t e2, ␣
→ Expression::t e3)
Expression::t Expr::stack(shared_ptr<ndarray<Expression::t,2>> exprs)
```

Stack a list of expressions along an arbitrary dimension. All expressions must have the same shape, except for dimension `dim`. The arguments may be any combination of expressions, scalar constants and variables.

For example, suppose A, B are two $n \times m$ matrices. Then stacking them in the first dimension produces a matrix of shape $(2n, m)$:

$$\begin{bmatrix} A \\ B \end{bmatrix},$$

stacking them in the second dimension produces a matrix of shape $(n, 2m)$:

$$\begin{bmatrix} A & B \end{bmatrix},$$

and stacking them in the third dimension produces a three-dimensional array of shape $(n, m, 2)$.

The version which takes a two-dimensional array of expressions constructs a block matrix with the given expressions as blocks. The dimensions of the blocks must be suitably compatible.

Parameters

- `dim` (`int`) – The dimension in which to stack.
- `exprs` (`Expression[]`) – A list of expressions.
- `exprs` (`Expression[][]`) – A list of expressions.
- `e1` (`Expression`) – An expression.
- `e2` (`Expression`) – An expression.
- `a2` (`double`) – A scalar constant.
- `a1` (`double`) – A scalar constant.
- `a3` (`double`) – A scalar constant.
- `e3` (`Expression`) – An expression.

Return (`Expression`)

`Expr.sub`

```
Expression::t Expr::sub(Expression::t e1, Expression::t e2)
Expression::t Expr::sub(Expression::t e1, shared_ptr<ndarray<double,1>> a1)
Expression::t Expr::sub(Expression::t e1, shared_ptr<ndarray<double,2>> a2)
Expression::t Expr::sub(shared_ptr<ndarray<double,1>> a1, Expression::t e2)
Expression::t Expr::sub(shared_ptr<ndarray<double,2>> a2, Expression::t e2)
Expression::t Expr::sub(Expression::t e1, double c)
Expression::t Expr::sub(double c, Expression::t e2)
Expression::t Expr::sub(Expression::t e1, Matrix::t m)
Expression::t Expr::sub(Matrix::t m, Expression::t e2)
Expression::t Expr::sub(Expression::t e1, NDSparseArray::t n)
Expression::t Expr::sub(NDSparseArray::t n, Expression::t e2)
```

Computes the difference of two expressions. The expressions must have the same shape and the result will be also an expression of that shape. The allowed combinations of arguments are the same as for `Expr.add`.

Parameters

- `e1` (*Expression*) – An expression.
- `e2` (*Expression*) – An expression.
- `a1` (`double[]`) – An array of constants.
- `a2` (`double[][]`) – An array of constants.
- `c` (`double`) – A constant.
- `m` (*Matrix*) – A Matrix object.
- `n` (*NDSparseArray*) – An NDSparseArray object.

Return (*Expression*)

`Expr.sum`

```
Expression::t Expr::sum(Expression::t expr)
Expression::t Expr::sum(Expression::t expr, int dim)
```

Sum the elements of an expression. Without extra arguments, all elements are summed into a scalar expression of size 1.

With argument `dim`, elements are summed along a specific dimension, resulting in an expression of reduced dimension. Note that the result of summing over a dimension of size 0 is 0.0. This means that for an expression of shape $(2,0,2)$, summing over the second dimension yields an expression of shape $(2,2)$ of zeros.

For example, if the argument is an $n \times m$ matrix then summing along the 0-th dimension computes the $1 \times m$ vector of column sums, and summing along the 1-st dimension computes the $n \times 1$ vector of row sums.

Parameters

- `expr` (*Expression*) – An expression object.
- `dim` (`int`) – The dimension along which to sum.

Return (*Expression*)

`Expr.transpose`

```
Expression::t Expr::transpose(Expression::t e)
```

Transpose a two-dimensional expression.

Parameters `e` (*Expression*) – Expression to transpose.

Return (*Expression*)

`Expr.vstack`

```
Expression::t Expr::vstack(shared_ptr<ndarray<Expression::t,1>> exprs)
Expression::t Expr::vstack(Expression::t e1, Expression::t e2)
Expression::t Expr::vstack(Expression::t e1, double a2)
Expression::t Expr::vstack(double a1, Expression::t e2)
Expression::t Expr::vstack(Expression::t e1, Expression::t e2, Expression::t e3)
Expression::t Expr::vstack(Expression::t e1, Expression::t e2, double a3)
Expression::t Expr::vstack(Expression::t e1, double a2, Expression::t e3)
Expression::t Expr::vstack(Expression::t e1, double a2, double a3)
Expression::t Expr::vstack(double a1, Expression::t e2, Expression::t e3)
Expression::t Expr::vstack(double a1, Expression::t e2, double a3)
Expression::t Expr::vstack(double a1, double a2, Expression::t e3)
Expression::t Expr::vstack(double a1, double a2, double a3)
```

Stack a list of expressions vertically (i.e. along the first dimension). The expressions must have the same shape, except for the first dimension. The arguments may be any combination of expressions, scalar constants and variables.

For example, if y^1, y^2, y^3 are three horizontal vectors of length n (and shape $(1, n)$) then their vertical stack is the matrix

$$\begin{bmatrix} -y^1 - \\ -y^2 - \\ -y^3 - \end{bmatrix}$$

of shape $(3, n)$.

Parameters

- **exprs** (*Expression*[]) – A list of expressions.
- **e1** (*Expression*) – An expression.
- **e2** (*Expression*) – An expression.
- **a2** (double) – A scalar constant.
- **a1** (double) – A scalar constant.
- **e3** (*Expression*) – An expression.
- **a3** (double) – A scalar constant.

Return (*Expression*)

`Expr.zeros`

```
Expression::t Expr::zeros(int size)
Expression::t Expr::zeros(shared_ptr<ndarray<int,1>> shp)
```

Create an expression consisting of zeros.

Parameters

- **size** (int) – Length of the vector to be constructed.
- **shp** (int[]) – Defines the shape of the expression.

Return (*Expression*)

14.2.13 Class ExprScaleVecPSD

`mosek::fusion::ExprScaleVecPSD`

Scale off-diagonal elements by $\sqrt{2}$ for use with SVECPD domain.

Implements *BaseExpression*

Members *BaseExpression.getDim* – Return the d'th dimension in the expression.

BaseExpression.getND – Return the number of dimensions in the expression.

BaseExpression.getShape – Get the shape of the expression.

BaseExpression.getSize – Return the total number of elements in the expression (the product of the dimensions).

BaseExpression.index – Get a single element in the expression.

BaseExpression.pick – Pick a number of elements from the expression.

BaseExpression.slice – Get a slice of the expression.

BaseExpression.toString – Return a string representation of the expression object.

ExprScaleVecPSD.eval – Evaluate the expression and push the result onto the work stack.

`ExprScaleVecPSD.eval`

```
void eval(WorkStack::t rs, WorkStack::t ws, WorkStack::t xs)
```

Evaluate the expression and push the result onto the `rs` work stack.

Parameters

- `rs` (*WorkStack*) – The stack where the result of the evaluation is stored.
- `ws` (*WorkStack*) – The stack used by evaluation to perform intermediate computations. It will be returned in the same state as when the function is called.
- `xs` (*WorkStack*) – An auxiliary stack.

14.2.14 Class Expression

`mosek::fusion::Expression`

Abstract base class for all objects which can be used as linear expressions of the form $Ax + b$.

The main use of this class is to store the result of expressions created by the static methods provided by *Expr*.

Members *Expression.eval* – Evaluate the expression and push the result onto the work stack.

Expression.getDim – Return the d'th dimension in the expression.

Expression.getND – Return the number of dimensions in the expression.

Expression.getShape – Get the shape of the expression.

Expression.getSize – Return the total number of elements in the expression (the product of the dimensions).

Expression.index – Get a single element in the expression.

Expression.pick – Pick a number of elements from the expression.

Expression.slice – Get a slice of the expression.

Expression.toString – Return a string representation of the expression object.

Implemented by *BaseExpression*, *Parameter*, *Variable*

`Expression.eval`

```
void eval(WorkStack::t rs, WorkStack::t ws, WorkStack::t xs)
```

Evaluate the expression and push the result onto the `rs` work stack.

Parameters

- `rs` (*WorkStack*) – The stack where the result of the evaluation is stored.
- `ws` (*WorkStack*) – The stack used by evaluation to perform intermediate computations. It will be returned in the same state as when the function is called.
- `xs` (*WorkStack*) – An auxiliary stack.

`Expression.getDim`

```
int getDim(int d)
```

Return the d'th dimension in the expression.

Parameters `d` (int)

Return (int)

`Expression.getND`

```
int getND()
```

Return the number of dimensions in the expression.

Return (int)

Expression.getShape

```
shared_ptr<ndarray<int,1>> getShape()
```

Get the shape of the expression.

Return (int[])

Expression.getSize

```
long long getSize()
```

Return the total number of elements in the expression (the product of the dimensions).

Return (long long)

Expression.index

```
Expression::t index(int i)
Expression::t index(shared_ptr<ndarray<int,1>> indexes)
```

Get a single element in the expression.

Parameters

- **i** (int) – Index of the element to pick.
- **indexes** (int[]) – Multi-dimensional index of the element to pick.

Return (*Expression*)

Expression.pick

```
Expression::t pick(shared_ptr<ndarray<int,1>> indexes)
Expression::t pick(shared_ptr<ndarray<int,2>> indexrows)
```

Picks a number of elements from the expression and returns them as a one-dimensional expression.

Parameters

- **indexes** (int[]) – Indexes of the elements to pick
- **indexrows** (int[][][]) – Indexes of the elements to pick. Each row defines a separate multi-dimensional index.

Return (*Expression*)

Expression.slice

```
Expression::t slice(int first, int last)
Expression::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
→ lasta)
```

Get a slice of the expression.

Parameters

- **first** (int) – Index of the first element in the slice.
- **last** (int) – Index of the last element in the slice plus one.
- **firsta** (int[]) – Multi-dimensional index of the first element in the slice.
- **lasta** (int[]) – Multi-dimensional index of the element after the end of the slice.

Return (*Expression*)

Expression.toString

```
string toString()
```

Return a string representation of the expression object.

Return (string)

14.2.15 Class LinearConstraint

mosek::fusion::LinearConstraint

A linear constraint defines a block of constraints with the same linear domain. The domain is either a product of product of one-dimensional half-spaces (linear inequalities), a fixed value vector (equalities) or the whole space (free constraints).

The *type* of a linear variable is immutable; it is either free, an inequality or an equality, but the linear expression and the right-hand side can be modified.

The class is not meant to be instantiated directly, but must be created by calling the *Model.variable* method.

Implements *ModelConstraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

LinearConstraint.toString – Create a human readable string representation of the constraint.

ModelConstraint.remove – Remove the constraint from the model.

LinearConstraint.toString

```
string toString()
```

Create a human readable string representation of the constraint.

Return (string)

14.2.16 Class LinearDomain

mosek::fusion::LinearDomain

Represent a domain defined by linear constraints

Members *LinearDomain.integral* – Creates a domain of integral variables.

LinearDomain.sparse – Creates a domain exploiting sparsity.

LinearDomain.symmetric – Creates a symmetric domain

LinearDomain.withNamesOnAxis – Set index names in a specific axis.

LinearDomain.withShape – Set the shape of the domain.

LinearDomain.integral

```
LinearDomain::t integral()
```

Modify a given domain restricting its elements to be integral.

Return (*LinearDomain*)

LinearDomain.sparse

```
LinearDomain::t sparse()
LinearDomain::t sparse(shared_ptr<ndarray<int,1>> sparsity)
LinearDomain::t sparse(shared_ptr<ndarray<int,2>> sparsity)
```

Creates a domain exploiting sparsity.

Parameters

- sparsity (int[])
- sparsity (int[][])

Return (*LinearDomain*)

LinearDomain.symmetric

```
SymmetricLinearDomain::t symmetric()
```

Creates a symmetric domain

Return (*SymmetricLinearDomain*)

LinearDomain.withNamesOnAxis

```
LinearDomain::t withNamesOnAxis(shared_ptr<ndarray<string,1>> names, int axis)
```

Set index names in a specific axis.

Parameters

- names (string[]) – List of names, this must match the actual dimension on that axis.
- axis (int) – The axis to change names on.

Return (*LinearDomain*)

LinearDomain.withShape

```
LinearDomain::t withShape(shared_ptr<ndarray<int,1>> shp)
LinearDomain::t withShape(int dim0)
LinearDomain::t withShape(int dim0, int dim1)
LinearDomain::t withShape(int dim0, int dim1, int dim2)
```

Set the shape of the domain.

Parameters

- shp (int[]) – The shape of the domain
- dim0 (int) – First dimension
- dim1 (int) – Second dimension
- dim2 (int) – Third dimension

Return (*LinearDomain*)

14.2.17 Class LinearPSDConstraint

`mosek::fusion::LinearPSDConstraint`

This class represents a semidefinite conic constraint of the form

$$Ax - b \succeq 0$$

i.e. $Ax - b$ must be positive semidefinite

Implements *ModelConstraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

ModelConstraint.remove – Remove the constraint from the model.

ModelConstraint.toString – Create a human readable string representation of the constraint.

14.2.18 Class LinearPSDVariable

`mosek::fusion::LinearPSDVariable`

This class represents a positive semidefinite variable.

Implements *ModelVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.

BaseVariable.setLevel – Input solution values for this variable

BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

BaseVariable.transpose – Transpose the variable.

BaseVariable.tril – Convert from a square variable to a trilinear representation.

LinearPSDVariable.toString – Create a string representation of the variable.
ModelVariable.remove – Remove the variable from the model.

`LinearPSDVariable.toString`

```
string toString()
```

Create a string representation of the variable.

Return (string)

14.2.19 Class LinearVariable

`mosek::fusion::LinearVariable`

A linear variable defines a block of variables with the same linear domain. The domain is either a product of product of one-dimensional half-spaces (linear inequalities), a fixed value vector (equalities) or the whole space (free variables).

The *type* of a linear variable is immutable; it is either free, an inequality or an equality.

The class is not meant to be instantiated directly, but must be created by calling the *Model.variable* method.

Implements *ModelVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.

BaseVariable.setLevel – Input solution values for this variable

BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

BaseVariable.transpose – Transpose the variable.

BaseVariable.tril – Convert from a square variable to a trilinear representation.

LinearVariable.toString – Create a string representation of the variable.

ModelVariable.remove – Remove the variable from the model.

`LinearVariable.toString`

```
string toString()
```

Create a string representation of the variable.

Return (string)

14.2.20 Class Matrix

mosek::fusion::Matrix

Base class for all matrix objects. It can be used to create and manipulate matrices of constant coefficients both in dense and sparse format. To operate with matrices containing variables and linear expressions use the classes *Expr* and *Variable*.

Members *Matrix.get* – Get a single entry.

Matrix.getDataAsArray – Return a dense array of values.

Matrix.getDataAsTriplets – Return the matrix data in sparse triplet format.

Matrix.isSparse – Returns true if the matrix is sparse.

Matrix.numColumns – Returns the number of columns in the matrix.

Matrix.numNonzeros – Returns the number of non-zeros in the matrix.

Matrix.numRows – Returns the number of rows in the matrix.

Matrix.toString – Get a string representation of the matrix.

Matrix.transpose – Transpose the matrix.

Static members *Matrix.antiDiag* – Create a sparse square matrix with a given vector as anti-diagonal.

Matrix.dense – Create a dense matrix from the given data.

Matrix.diag – Create a sparse square matrix with a given vector as diagonal.

Matrix.eye – Create the identity matrix.

Matrix.ones – Create a matrix filled with all ones.

Matrix.sparse – Create a sparse matrix from the given data.

Matrix.antiDiag

```
Matrix::t Matrix::antiDiag(shared_ptr<ndarray<double,1>> d)
Matrix::t Matrix::antiDiag(shared_ptr<ndarray<double,1>> d, int k)
Matrix::t Matrix::antiDiag(int n, double val)
Matrix::t Matrix::antiDiag(int n, double val, int k)
```

Create a sparse square matrix with a given vector as anti-diagonal.

Parameters

- **d** (double[]) – The anti-diagonal vector.
- **k** (int) – The anti-diagonal index. $k = 0$ is the default and means the main anti-diagonal. $k > 0$ means above, and $k < 0$ means below the main anti-diagonal.
- **n** (int) – The dimension of the matrix.
- **val** (double) – Use this value for all anti-diagonal elements.

Return (*Matrix*)

Matrix.dense

```
Matrix::t Matrix::dense(shared_ptr<ndarray<double,2>> data)
Matrix::t Matrix::dense(int dimi, int dimj, shared_ptr<ndarray<double,1>> data)
Matrix::t Matrix::dense(int dimi, int dimj, double value)
Matrix::t Matrix::dense(Matrix::t other)
```

Create a dense matrix from the given data.

Parameters

- **data** (double[[[[]]]) – A one- or two-dimensional array of matrix coefficients.

- `data (double[])` – A one- or two-dimensional array of matrix coefficients.
- `dimi (int)` – Number of rows.
- `dimj (int)` – Number of columns.
- `value (double)` – Use this value for all elements.
- `other (Matrix)` – Create a dense matrix from another matrix.

Return (*Matrix*)

`Matrix.diag`

```
Matrix::t Matrix::diag(shared_ptr<ndarray<double,1>> d)
Matrix::t Matrix::diag(shared_ptr<ndarray<double,1>> d, int k)
Matrix::t Matrix::diag(int n, double val)
Matrix::t Matrix::diag(int n, double val, int k)
Matrix::t Matrix::diag(shared_ptr<ndarray<Matrix::t,1>> md)
Matrix::t Matrix::diag(int num, Matrix::t mv)
```

Create a sparse square matrix with a given vector as diagonal.

Parameters

- `d (double[])` – The diagonal vector.
- `k (int)` – The diagonal index. $k = 0$ is the default and means the main diagonal. $k > 0$ means above, and $k < 0$ means below the main diagonal.
- `n (int)` – The dimension of the matrix.
- `val (double)` – Use this value for all diagonal elements.
- `md (Matrix[])` – A list of square matrices that are used to create a block-diagonal square matrix.
- `num (int)` – Number of times to repeat the `mv` matrix.
- `mv (Matrix)` – A matrix to be repeated in all blocks of a block-diagonal square matrix.

Return (*Matrix*)

`Matrix.eye`

```
Matrix::t Matrix::eye(int n)
```

Construct the identity matrix of size n .

Parameters `n (int)` – The dimension of the matrix.

Return (*Matrix*)

`Matrix.get`

```
double get(int i, int j)
```

Get a single entry.

Parameters

- `i (int)` – Row index.
- `j (int)` – Column index.

Return (`double`)

`Matrix.getDataAsArray`

```
shared_ptr<ndarray<double,1>> getDataAsArray()
```

Return the matrix elements as a dense array in row-major format.

Return (double[])

`Matrix.getDataAsTriplets`

```
void getDataAsTriplets(shared_ptr<ndarray<int,1>> subi, shared_ptr<ndarray<int,1>
↳> subj, shared_ptr<ndarray<double,1>> val)
```

Return the matrix data in sparse triplet format. Data is copied to the arrays `subi`, `subj` and `val` which must be pre-allocated to hold at least the number of non-zeros in the matrix.

The data returned must be ordered with `subi` as primary key and `subj` as secondary key.

Parameters

- `subi` (int[]) – Row subscripts are returned in this array.
- `subj` (int[]) – Column subscripts are returned in this array.
- `val` (double[]) – Coefficient values are returned in this array.

`Matrix.isSparse`

```
bool isSparse()
```

Returns true if the matrix is sparse.

Return (bool)

`Matrix.numColumns`

```
int numColumns()
```

Returns the number of columns in the matrix.

Return (int)

`Matrix.numNonzeros`

```
long long numNonzeros()
```

Returns the number of non-zeros in the matrix.

Return (long long)

`Matrix.numRows`

```
int numRows()
```

Returns the number of rows in the matrix.

Return (int)

`Matrix.ones`

```
Matrix::t Matrix::ones(int n, int m)
```

Construct a matrix filled with ones.

Parameters

- `n` (int) – Number of rows.
- `m` (int) – Number of columns.

Return (*Matrix*)

`Matrix.sparse`

```
Matrix::t Matrix::sparse(int nrow, int ncol, shared_ptr<ndarray<int,1>> subi,
↳shared_ptr<ndarray<int,1>> subj, shared_ptr<ndarray<double,1>> val)
Matrix::t Matrix::sparse(shared_ptr<ndarray<int,1>> subi, shared_ptr<ndarray<int,
↳1>> subj, shared_ptr<ndarray<double,1>> val)
Matrix::t Matrix::sparse(shared_ptr<ndarray<int,1>> subi, shared_ptr<ndarray<int,
↳1>> subj, double val)
Matrix::t Matrix::sparse(int nrow, int ncol, shared_ptr<ndarray<int,1>> subi,
↳shared_ptr<ndarray<int,1>> subj, double val)
Matrix::t Matrix::sparse(int nrow, int ncol)
Matrix::t Matrix::sparse(shared_ptr<ndarray<double,2>> data)
Matrix::t Matrix::sparse(shared_ptr<ndarray<Matrix::t,2>> blocks)
Matrix::t Matrix::sparse(Matrix::t mx)
```

Create a sparse matrix from the given data.

Parameters

- `nrow` (`int`) – Number of rows.
- `ncol` (`int`) – Number of columns.
- `subi` (`int[]`) – Row subscripts of non-zero elements.
- `subj` (`int[]`) – Column subscripts of non-zero elements.
- `val` (`double[]`) – Coefficients of non-zero elements.
- `val` (`double`) – Coefficients of non-zero elements.
- `data` (`double[][]`) – Dense data array.
- `blocks` (*Matrix* `[][]`) – The matrix data in block format. All elements in a row must have the same height, and all elements in a column must have the same width. Entries that are NULL will be interpreted as a block of zeros whose height and width are deduced from the other elements in the same row and column. Any row that contains only NULL entries will have height 0, and any column that contains only NULL entries will have width 0.
- `mx` (*Matrix*) – A *Matrix* object.

Return (*Matrix*)

`Matrix.toString`

```
string toString()
```

Get a string representation of the matrix.

Return (`string`)

`Matrix.transpose`

```
Matrix::t transpose()
```

Transpose the matrix.

Return (*Matrix*)

14.2.21 Class Model

`mosek::fusion::Model`

The object containing all data related to a single optimization model.

Implements `BaseModel`

Members *`Model.acceptedSolutionStatus`* – Set the accepted solution status.
`Model.breakSolver` – Request that the solver terminates as soon as possible.
`Model.clone` – Return a copy of the model.
`Model.constraint` – Create a new constraint in the model.
`Model.disjunction` – Create a new disjunctive constraint in the model.
`Model.dispose` – Destroy the Model object
`Model.dualObjValue` – Get the dual objective value in the current solution.
`Model.flushParameters` – Flush all parameters to the underlying task.
`Model.flushSolutions` – If any solution values have been provided, flush those values to the underlying task.
`Model.getAcceptedSolutionStatus` – Get the accepted solution status.
`Model.getConstraint` – Get the constraint matching the given name or linear index.
`Model.getDualSolutionStatus` – Return the status of the dual solution.
`Model.getName` – Return the model name, or an empty string if it has not been set.
`Model.getParameter` – Get the parameter matching the given name.
`Model.getPrimalSolutionStatus` – Return the status of the primal solution.
`Model.getProblemStatus` – Return the status of the problem.
`Model.getSolverDoubleInfo` – Fetch a solution information item from the solver
`Model.getSolverIntInfo` – Fetch a solution information item from the solver
`Model.getSolverLIntInfo` – Fetch a solution information item from the solver
`Model.getTask` – Return the underlying MOSEK task object.
`Model.getVariable` – Get the variable matching the given name or linear index.
`Model.hasConstraint` – Check whether the model contains a constraint with a given name.
`Model.hasParameter` – Check whether the model contains a parameter with a given name.
`Model.hasVariable` – Check whether the model contains a variable with a given name.
`Model.objective` – Replace the objective expression.
`Model.optserverHost` – Specify an OptServer for remote calls.
`Model.parameter` – Create a new parameter in the model.
`Model.primalObjValue` – Get the primal objective value in the current solution.
`Model.selectedSolution` – Chooses a solution.
`Model.setCallbackHandler` – Attach a progress callback handler.
`Model.setDataCallbackHandler` – Attach a data callback handler.
`Model.setLogHandler` – Attach a log handler.
`Model.setSolverParam` – Set a solver parameter
`Model.solve` – Attempt to optimize the model.
`Model.updateObjective` – Update part of the objective.
`Model.variable` – Create a new variable in the model.
`Model.writeTask` – Dump the current solver task to a file.
`Model.writeTaskStream` – Write the current solver task to a stream.
Static members *`Model.getVersion`* – Return MOSEK version.
`Model.putlicensecode` – Set the license code in the global environment.
`Model.putlicensepath` – Set the license path in the global environment.
`Model.putlicensewait` – Set the license wait flag in the global environment.
`Model.solveBatch` – Attempt to optimize a collection of models in parallel.

`Model.acceptedSolutionStatus`

<code>void acceptedSolutionStatus(AccSolutionStatus what)</code>
--

Set the accepted solution status. This defines which solution status values are considered as *acceptable* when fetching a solution. Requesting a solution value for a variable or constraint when the status does not match at least the accepted value will cause an error.

By default the accepted solution status is `AccSolutionStatus.Optimal`. It is necessary to change the accepted status to access sub-optimal solutions and infeasibility certificates.

The methods `Model.getPrimalSolutionStatus` and `Model.getDualSolutionStatus` can be used to get the *actual* status of the solutions.

Parameters `what` (`AccSolutionStatus`) – The new accepted solution status.

`Model.breakSolver`

```
void breakSolver()
```

Request that the solver terminates as soon as possible. This must be called from another thread than the one in which `solve()` was called, or from a callback function.

The method does not stop the solver directly, rather it sets a flag that the solver checks occasionally, indicating it should terminate.

`Model.clone`

```
Model::t clone()
```

Return a copy of the model.

Return (`Model`)

`Model.constraint`

```
Constraint::t constraint(string name, Expression::t expr, PSDDomain::t psddom)
Constraint::t constraint(Expression::t expr, PSDDomain::t psddom)
Constraint::t constraint(string name, Expression::t expr, LinearDomain::t ldom)
Constraint::t constraint(Expression::t expr, LinearDomain::t ldom)
RangedConstraint::t constraint(string name, Expression::t expr, RangeDomain::t rdom)
RangedConstraint::t constraint(Expression::t expr, RangeDomain::t rdom)
Constraint::t constraint(string name, Expression::t expr, ConeDomain::t qdom)
Constraint::t constraint(Expression::t expr, ConeDomain::t qdom)
```

Adds a new constraint to the model. A constraint is always a statement that *an expression or variable* belongs to *a domain*. Constraints can have optional names.

Typical domains used for defining constraints include:

- `Domain.lessThan`, `Domain.greaterThan`, `Domain.inRange`, `Domain.equalsTo` — puts linear bounds $E \leq u$, $l \leq E$, $l \leq E \leq u$ or $E = c$ on an expression E .
- `Domain.inQCone`, `Domain.inRotatedQCone` — constrains a vector or matrix expression E to a second-order cone.
- `Domain.inPExpCone`, `Domain.inPPowerCone` — constrains a vector or matrix expression E to an exponential or power cone.
- `Domain.inPSDCone` — constrains a square matrix expression E to be positive semidefinite.

See `Domain` for a full list of domains.

Parameters

- `name` (`string`) – Name of the constraint. This must be unique among all constraints in the model. The value `NULL` is allowed instead of a unique name.
- `expr` (`Expression`) – An expression.

- `psddom` (*PSDDomain*) – A positive semidefinite domain.
- `ldom` (*LinearDomain*) – A linear domain.
- `rdom` (*RangeDomain*) – A ranged domain.
- `qdom` (*ConeDomain*) – A domain in a cone.

Return

- (*Constraint*)
- (*RangedConstraint*)

`Model.disjunction`

```
Disjunction::t disjunction(string name, Term::t t1)
Disjunction::t disjunction(string name, Term::t t1, Term::t t2)
Disjunction::t disjunction(string name, Term::t t1, Term::t t2, Term::t t3)
Disjunction::t disjunction(Term::t t1)
Disjunction::t disjunction(Term::t t1, Term::t t2)
Disjunction::t disjunction(Term::t t1, Term::t t2, Term::t t3)
Disjunction::t disjunction(shared_ptr<ndarray<Term::t,1>> terms)
Disjunction::t disjunction(string name, shared_ptr<ndarray<Term::t,1>> terms)
```

Adds a new disjunctive constraint to the model. A disjunctive constraint with terms $T_1, \dots, T_n$ is:

$$T_1 \text{ OR } \dots \text{ OR } T_n.$$

Each term T_i of a disjunctive constraint is a *Term*, which can be created with *DJC.term* or with *DJC.AND*.

Parameters

- `name` (*string*) – The name of this disjunctive constraint.
- `t1` (*Term*) – A term in the disjunction.
- `t2` (*Term*) – A term in the disjunction.
- `t3` (*Term*) – A term in the disjunction.
- `terms` (*Term*[]) – A list of terms forming the disjunctive constraint.

Return (*Disjunction*)

`Model.dispose`

```
void dispose()
```

Destroy the Model object. This removes all references to other objects from the Model.

This helps garbage collection by removing cyclic references, and in some cases it is necessary to ensure that the garbage collector can collect the Model object and associated objects.

`Model.dualObjValue`

```
double dualObjValue()
```

Get the dual objective value in the current solution.

Return (*double*)

`Model.flushParameters`

```
void flushParameters()
```

Flush all parameters to the underlying task.

`Model.flushSolutions`


```
void flushSolutions()
```

If any solution values have been provided, flush those values to the underlying task.
`Model.getAcceptedSolutionStatus`

```
AccSolutionStatus getAcceptedSolutionStatus()
```

Get the accepted solution status.

Return (*AccSolutionStatus*)

`Model.getConstraint`

```
Constraint::t getConstraint(string name)  
Constraint::t getConstraint(int index)
```

Get the constraint matching the given name or linear index. Constraints are assigned indices in the order they are added to the model.

Parameters

- **name** (*string*) – The constraint's name.
- **index** (*int*) – The constraint's linear index.

Return (*Constraint*)

`Model.getDualSolutionStatus`

```
SolutionStatus getDualSolutionStatus(SolutionType which)  
SolutionStatus getDualSolutionStatus()
```

Return the status of the dual solution. If no solution type is given the solution set with *Model.selectedSolution* is checked. It is recommended to check the problem and solution status before accessing the solution values.

Parameters **which** (*SolutionType*) – The type of the solution for which status is requested.

Return (*SolutionStatus*)

`Model.getName`

```
string getName()
```

Return the model name, or an empty string if it has not been set.

Return (*string*)

`Model.getParameter`

```
Parameter::t getParameter(string name)
```

Get the parameter matching the given name.

Parameters **name** (*string*) – The parameter's name.

Return (*Parameter*)

`Model.getPrimalSolutionStatus`

```
SolutionStatus getPrimalSolutionStatus(SolutionType which)
SolutionStatus getPrimalSolutionStatus()
```

Return the status of the primal solution. If no solution type is given the solution set with *Model.selectedSolution* is checked. It is recommended to check the problem and solution status before accessing the solution values.

Parameters which (*SolutionType*) – The type of the solution for which status is requested.

Return (*SolutionStatus*)

`Model.getProblemStatus`

```
ProblemStatus getProblemStatus(SolutionType which)
ProblemStatus getProblemStatus()
```

Return the status of the problem. If no solution type is given the solution set with *Model.selectedSolution* is checked. It is recommended to check the problem and solution status before accessing the solution values.

Parameters which (*SolutionType*) – The type of the solution.

Return (*ProblemStatus*)

`Model.getSolverDoubleInfo`

```
double getSolverDoubleInfo(string name)
```

This method returns the value for the specified double solver information item. The information items become available during and after the solver execution. A runtime exception is thrown if a non-existing information item is requested. The double information items are listed in Section *Double information items*.

Parameters name (*string*) – A string name of the information item.

Return (*double*)

`Model.getSolverIntInfo`

```
int getSolverIntInfo(string name)
```

This method returns the value for the specified integer solver information item. The information items become available during and after the solver execution. A runtime exception is thrown if a non-existing information item is requested. The integer information items are listed in Section *Integer information items*.

Parameters name (*string*) – A string name of the information item.

Return (*int*)

`Model.getSolverLIntInfo`

```
long long getSolverLIntInfo(string name)
```

This method returns the value for the specified long solver information item. The information items become available during and after the solver execution. A runtime exception is thrown if a non-existing information item is requested. The long integer information items are listed in Section *Long integer information items*.

Parameters name (*string*) – A string name of the information item.

Return (*long long*)

Model.getTask

```
mosek.Task getTask()
```

Returns the underlying **MOSEK** Task object. Note that the returned object is the actual underlying object, not a copy. This means if the returned object is modified by the user, the *Model* object may become invalid. Accessing the task object should never be necessary, except maybe for advanced debugging. For details on the Task object see the Optimizer API documentation.

Return (Task)

Model.getVariable

```
Variable::t getVariable(string name)  
Variable::t getVariable(int index)
```

Get the variable matching the given name or linear index. Variables are assigned indices in the order they are added to the model.

Parameters

- **name** (string) – The variable’s name.
- **index** (int) – The variable’s linear index.

Return (*Variable*)

Model.getVersion

```
string Model::getVersion()
```

Returns the **MOSEK** version as a string, for example “10.0.44”.

Return (string)

Model.hasConstraint

```
bool hasConstraint(string name)
```

Check whether the model contains a constraint with a given name.

Parameters **name** (string) – The constraint name.

Return (bool)

Model.hasParameter

```
bool hasParameter(string name)
```

Check whether the model contains a parameter with a given name.

Parameters **name** (string) – The parameter’s name.

Return (bool)

Model.hasVariable

```
bool hasVariable(string name)
```

Check whether the model contains a variable with a given name.

Parameters **name** (string) – The variable name.

Return (bool)

Model.objective

```
void objective(string name, ObjectiveSense sense, Expression::t expr)
void objective(string name, ObjectiveSense sense, double c)
void objective(string name, double c)
void objective(ObjectiveSense sense, Expression::t expr)
void objective(ObjectiveSense sense, double c)
void objective(double c)
```

Replace the objective expression. This method must be called at least once before the first *Model.solve*.

Parameters

- **name** (string) – Name of the objective. This may be any string, and it has no function except when writing the problem to an external file format.
- **sense** (*ObjectiveSense*) – The objective sense. Defines whether the objective must be minimized or maximized.
- **expr** (*Expression*) – The objective expression. This must be an affine expression that evaluates to a scalar.
- **c** (double) – A constant scalar.

Model.optserverHost

```
void optserverHost(string addr)
```

Specify an OptServer URL for remote calls. The URL should contain protocol, host and port in the form `http://server:port` or `https://server:port`. If the URL is set using this function, all subsequent calls to *Model.solve* will be sent to the specified OptServer instead of being executed locally. Passing NULL deactivates this redirection.

Parameters **addr** (string) – Address of the OptServer. It should be a valid URL, for example `http://server:port` or `https://server:port`.

Model.parameter

```
Parameter::t parameter(shared_ptr<ndarray<int,1>> shape, shared_ptr<ndarray<int,
↪2>> sparsity)
Parameter::t parameter(shared_ptr<ndarray<int,1>> shape, shared_ptr<ndarray<long
↪long,1>> sp)
Parameter::t parameter(shared_ptr<ndarray<int,1>> shape)
Parameter::t parameter(int d1)
Parameter::t parameter(int d1, int d2)
Parameter::t parameter(int d1, int d2, int d3)
Parameter::t parameter()
Parameter::t parameter(string name, shared_ptr<ndarray<int,1>> shape, shared_ptr
↪<ndarray<int,2>> sparsity)
Parameter::t parameter(string name, shared_ptr<ndarray<int,1>> shape, shared_ptr
↪<ndarray<long long,1>> sp)
Parameter::t parameter(string name, shared_ptr<ndarray<int,1>> shape)
Parameter::t parameter(string name, int d1)
Parameter::t parameter(string name, int d1, int d2)
Parameter::t parameter(string name, int d1, int d2, int d3)
Parameter::t parameter(string name)
```

Create a new parameter in the model. A parameter is a placeholder for a constant (scalar or array) value that can be assigned and reset after the model is built and between optimizations.

If the shape is not provided, the parameter is a scalar parameter. The default value of a newly created parameter is 0. To set the value use *Parameter.setValue*.

Parameters

- **shape** (`int[]`) – Shape of the parameter.
- **sparsity** (`int[][]`) – Non-zero sparsity pattern, if the parameter is sparse.
- **sp** (`long long[]`) – Non-zero sparsity pattern as a list of linear indexes, if the parameter is sparse.
- **d1** (`int`) – First dimension of a parameter.
- **d2** (`int`) – Second dimension of a parameter.
- **d3** (`int`) – Third dimension of a parameter.
- **name** (`string`) – Name of the parameter.

Return (*Parameter*)

`Model.primalObjValue`

```
double primalObjValue()
```

Get the primal objective value in the current solution.

Return (`double`)

`Model.putlicensecode`

```
void Model::putlicensecode(shared_ptr<ndarray<int,1>> code)
```

Set the license code in the global environment.

Parameters `code` (`int[]`)

`Model.putlicensepath`

```
void Model::putlicensepath(string licfile)
```

Set the license path in the global environment.

Parameters `licfile` (`string`)

`Model.putlicensewait`

```
void Model::putlicensewait(bool wait)
```

Set the license wait flag in the global environment. If set, **MOSEK** will wait until a license becomes available.

Parameters `wait` (`bool`)

`Model.selectedSolution`

```
void selectedSolution(SolutionType soltype)
```

Chooses a solution. The values of variables and constraints will be read from the chosen solution. The default is to consider all solution types in the order of *SolutionType.Default*.

Parameters `soltype` (*SolutionType*) – The solution type to select as default.

Model.setCallbackHandler

```
void setCallbackHandler(System.CallbackHandler h)
```

Attach a progress callback handler. During optimization this handler will be called, providing a code with the current state of the solver. Passing NULL detaches the current handler. See Section [Progress and data callback](#) for details and examples and the Optimizer API for information about callback codes.

The progress callback handler is a function of type `std::function<int(MSKcallbackcodee)>`.

Parameters `h` (`CallbackHandler`) – The callback handler or NULL.

Model.setDataCallbackHandler

```
void setDataCallbackHandler(System.DataCallbackHandler h)
```

Attach a data callback handler. During optimization this handler will be called, providing various information about the current state of the solution and solver. Passing NULL detaches the current handler. See Section [Progress and data callback](#) for details and examples and the Optimizer API for information about callback codes.

The data callback handler is a function of type `std::function<bool(MSKcallbackcodee, const double *, const int32_t *, const int64_t *)>`.

Parameters `h` (`DataCallbackHandler`) – The callback handler or NULL.

Model.setLogHandler

```
void setLogHandler(System.StreamWriter h)
```

Attach a log handler. The solver log information will be sent to the stream handler. Passing NULL detaches the current handler.

The log handler is a function of type `std::function<void(const std::string &)>`, for example

```
M->setLogHandler([](const std::string & msg) { std::cout << msg << std::flush; }  
→);
```

Parameters `h` (`StreamWriter`) – The log handler object or NULL.

Model.setSolverParam

```
void setSolverParam(string name, string strval)  
void setSolverParam(string name, int intval)  
void setSolverParam(string name, double floatval)
```

Set a solver parameter. Solver parameter values can be either symbolic values, integers or doubles, depending on the parameter. The value is automatically converted to a suitable type whenever possible. If this fails, an exception will be thrown. For example, if the parameter accepts a double value and is given a string, the string will be parsed to produce a double.

See Section [Parameters \(alphabetical list sorted by type\)](#) for a listing of all parameter settings.

Parameters

- `name` (`string`) – Name of the parameter to set
- `strval` (`string`) – A string value to assign to the parameter.
- `intval` (`int`) – An integer value to assign to the parameter.
- `floatval` (`double`) – A floating point value to assign to the parameter.

`Model.solve`

```
void solve()
void solve(string addr, string accesstoken)
```

This calls the **MOSEK** solver to solve the problem defined in the model.

If no error occurs, on exit a solution status will be defined for the primal and the dual solutions. These can be obtained with `Model.getPrimalSolutionStatus` and `Model.getDualSolutionStatus`. Depending on the solution status, various values may be defined:

- If the model is primal-dual feasible, or nearly so, and the solver found a solution, the solution values can be accessed through the `Variable` and `Constraint` objects in the model. For integer problems only the primal solution is defined, while for continuous problems both primal and dual solutions are available.
- If the model is primal or dual infeasible, *only* the primal *or* the dual solution is defined, depending on the solution status. The available solution contains a certificate of infeasibility.
- If the status is unknown the solver ran into problems and did not find anything useful. In this case the solution values may be garbage.

The solution can be obtained with `Model.primalObjValue` and `Variable.level` and their dual analogues.

By default, trying to fetch a non-optimal solution using `Variable.level` or `Variable.dual` will cause an exception. To fetch infeasibility certificates or other less optimal solutions it is necessary to change the accepted solution flag with `Model.acceptedSolutionStatus`.

Parameters

- `addr` (string) – Address of the OptServer if optimizing remotely. It should be a valid URL, for example `http://server:port` or `https://server:port`.
- `accesstoken` (string) – Access token if optimizing remotely with authentication.

`Model.solveBatch`

```
shared_ptr<ndarray<SolverStatus,1>> Model::solveBatch(bool israce, double ↵
↵ maxtime, int numthreads, shared_ptr<ndarray<Model::t,1>> models)
```

Calls the **MOSEK** solver to solve all the provided models in parallel. All callbacks and log output streams are disabled.

Assuming that each model takes about same time and there many more models than threads then a linear speedup can be achieved, also known as strong scaling. A typical application of this method is to solve many small models of similar type; in this case it is recommended that each of them is allocated a single thread by setting `numThreads` to 1.

If the parameters `israce` or `maxtime` are used, then the result may not be deterministic, in the sense that the models which complete first may vary between runs.

The remaining behavior is the same as if each model was solved separately with `Model.solve`. The return array describes if each model solved, failed or wasn't attempted because another one finished first (if `israce` is used). Debugging a failed model can be done by solving it individually.

This method parallelizes the call to the numerical solver; additional operations such as postprocessing the optimizer solution into the solution of the *Fusion* model are still performed sequentially.

Parameters

- `israce` (bool) – If true, then the function is terminated after the first model completed.
- `maxtime` (double) – Time limit: if nonnegative, then the function is terminated after this time (seconds).

- `numthreads` (`int`) – The number of threads for the whole pool available to all models. If set to 0 the number of threads used will be equal to the number of cores detected on the machine.
- `models` (`Model[]`) – An array of models to be solved.

Return (`SolverStatus[]`)

`Model.updateObjective`

```
void updateObjective(Expression::t expr, Variable::t x)
```

Update the columns in the objective expression defined by `x`.

Parameters

- `expr` (`Expression`) – The expression to update with. This must have size 1.
- `x` (`Variable`) – The columns to replace.

`Model.variable`

```
Variable::t variable(string name)
Variable::t variable(string name, int size)
Variable::t variable(string name, int size, LinearDomain::t ldom)
RangedVariable::t variable(string name, int size, RangeDomain::t rdom)
Variable::t variable(string name, int size, ConeDomain::t qdom)
Variable::t variable(string name, shared_ptr<ndarray<int,1>> shp)
Variable::t variable(string name, shared_ptr<ndarray<int,1>> shp, LinearDomain::
    ↳t ldom)
RangedVariable::t variable(string name, shared_ptr<ndarray<int,1>> shp,
    ↳RangeDomain::t rdom)
Variable::t variable(string name, shared_ptr<ndarray<int,1>> shp, ConeDomain::t
    ↳qdom)
Variable::t variable(string name, LinearDomain::t ldom)
RangedVariable::t variable(string name, RangeDomain::t rdom)
Variable::t variable(string name, ConeDomain::t qdom)
Variable::t variable()
Variable::t variable(int size)
Variable::t variable(int size, LinearDomain::t ldom)
RangedVariable::t variable(int size, RangeDomain::t rdom)
Variable::t variable(int size, ConeDomain::t qdom)
Variable::t variable(shared_ptr<ndarray<int,1>> shp)
Variable::t variable(shared_ptr<ndarray<int,1>> shp, LinearDomain::t ldom)
RangedVariable::t variable(shared_ptr<ndarray<int,1>> shp, RangeDomain::t rdom)
Variable::t variable(shared_ptr<ndarray<int,1>> shp, ConeDomain::t qdom)
Variable::t variable(LinearDomain::t ldom)
RangedVariable::t variable(RangeDomain::t rdom)
Variable::t variable(ConeDomain::t qdom)
Variable::t variable(string name, shared_ptr<ndarray<int,1>> shp, PSDDomain::t
    ↳psddom)
Variable::t variable(string name, int n, PSDDomain::t psddom)
Variable::t variable(string name, int n, int m, PSDDomain::t psddom)
Variable::t variable(string name, PSDDomain::t psddom)
Variable::t variable(int n, PSDDomain::t psddom)
Variable::t variable(int n, int m, PSDDomain::t psddom)
Variable::t variable(PSDDomain::t psddom)
```

Create a new variable in the model. All variables must be created using this method. The many versions of the method accept a name (optional), the shape of the variable and its domain. The domain must be suitable for the given variable shape. If the domain is not provided, it is assumed

that the variable is unbounded. If the dimension is not provided the variable is a single scalar variable.

Typical domains used for creating variables include:

- *Domain.lessThan*, *Domain.greaterThan*, *Domain.inRange* — creates a variable x with bounds $x \leq u$, $l \leq x$ or $l \leq x \leq u$.
- *Domain.inPSDCone* — creates a symmetric positive definite variable of dimension n .

Parameters

- **name** (string) – Name of the variable. This must be unique among all variables in the model. The value NULL is allowed instead of a unique name.
- **size** (int) – Size of the variable. The variable becomes a one-dimensional vector of the given size.
- **ldom** (*LinearDomain*) – A linear domain for the variable.
- **rdom** (*RangeDomain*) – A ranged domain for the variable.
- **qdom** (*ConeDomain*) – A conic domain for the variable.
- **shp** (int[]) – Defines the shape of the variable.
- **psddom** (*PSDDomain*) – A semidefinite domain for the variable.
- **n** (int) – Dimension of the semidefinite variable.
- **m** (int) – Number of semidefinite variables.

Return

- (*Variable*)
- (*RangedVariable*)

`Model.writeTask`

```
void writeTask(string filename)
```

Dump the current solver task to a file. The file extension determines the file format, see Section *Supported File Formats* for details. The file can be read with the command line **MOSEK** or with the Optimizer API for debugging purposes.

Parameters `filename` (string) – Name of the output file.

`Model.writeTaskStream`

```
void writeTaskStream(string ext, System.DataStream stream)
```

Write the current solver task to a stream object. The extension determines the file format, see Section *Supported File Formats*, and it should be a string such as "ptf", "task.gz", etc. that would be used for file extension in *Model.writeTask*.

`stream` is an object of type `std::ostream`, for example

```
M->writeTaskStream("jtask", std::cout);

std::ofstream outfile("dump.ptf", std::ofstream::binary);
M->writeTaskStream("ptf", outfile);
outfile.close();
```

Parameters

- **ext** (string) – Extension, which determines the file format.
- **stream** (DataStream) – The output stream.

14.2.22 Class ModelConstraint

`mosek::fusion::ModelConstraint`

Base class for all constraints that directly corresponds to a block of constraints in the underlying task, i.e. all objects created from *Model.constraint*.

Implements *Constraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

ModelConstraint.remove – Remove the constraint from the model.

ModelConstraint.toString – Create a human readable string representation of the constraint.

Implemented by *LinearConstraint*, *ConicConstraint*, *RangedConstraint*, *PSDConstraint*, *LinearPSDConstraint*

`ModelConstraint.remove`

```
void remove()
```

Remove the constraint from the model. Using the constraint object after this method has been called results in undefined behavior.

`ModelConstraint.toString`

```
string toString()
```

Create a human readable string representation of the constraint.

Return (string)

14.2.23 Class ModelVariable

`mosek::fusion::ModelVariable`

Base class for all variables that directly corresponds to a block of variables in the underlying task, i.e. all objects created from *Model.variable*.

Implements *BaseVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.
BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.
BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.
BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.
BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.
BaseVariable.setLevel – Input solution values for this variable
BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.
BaseVariable.toString – Create a string representation of the variable.
BaseVariable.transpose – Transpose the variable.
BaseVariable.tril – Convert from a square variable to a trilinear representation.
ModelVariable.remove – Remove the variable from the model.

Implemented by *ConicVariable*, *LinearVariable*, *PSDVariable*, *LinearPSDVariable*, *RangedVariable*

`ModelVariable.remove`

```
void remove()
```

Remove the variable from the model and remove it from any constraints where it appears. Using the variable object after this method has been called results in undefined behavior.

14.2.24 Class NDSparseArray

`mosek::fusion::NDSparseArray`

Representation of a sparse n-dimensional array.

Static members *NDSparseArray.make* – Create a sparse n-dimensional matrix (tensor).

`NDSparseArray.make`

```

NDSparseArray::t NDSparseArray::make(shared_ptr<ndarray<int,1>> dims, shared_ptr
→<ndarray<int,2>> sub, shared_ptr<ndarray<double,1>> cof)
NDSparseArray::t NDSparseArray::make(shared_ptr<ndarray<int,1>> dims, shared_ptr
→<ndarray<long long,1>> inst, shared_ptr<ndarray<double,1>> cof)
NDSparseArray::t NDSparseArray::make(Matrix::t m)

```

Create a sparse n-dimensional matrix (tensor).

Parameters

- `dims (int[])` – Dimensions.
- `sub (int[][])` – Positions of nonzeros. Array where each row is an n -dimensional index.
- `cof (double[])` – Values of nonzero elements. Array of coefficients corresponding to subscripts.
- `inst (long long[])` – Positions of nonzeros using linear indexes into the array.
- `m (Matrix)` – An initializing matrix.

Return (*NDSparseArray*)

14.2.25 Class PSDConstraint

mosek::fusion::PSDConstraint

This class represents a semidefinite conic constraint of the form

$$Ax - b \succeq 0$$

i.e. $Ax - b$ must be positive semidefinite

Implements *ModelConstraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

ModelConstraint.remove – Remove the constraint from the model.

PSDConstraint.toString – Create a human readable string representation of the constraint.

PSDConstraint.toString

`string toString()`

Create a human readable string representation of the constraint.

Return (string)

14.2.26 Class PSDDomain

mosek::fusion::PSDDomain

Represent the domain of PSD matrices.

Members *PSDDomain.axis* – Set the dimension along which the cones are created in a multi-dimensional domain.

PSDDomain.withNamesOnAxis – Set index names in a specific axis.

PSDDomain.axis

`PSDDomain::t axis(int conedim1, int conedim2)`

Set the dimension along which the cones are created in a multi-dimensional domain.

Parameters

- `conedim1 (int)` – First dimension.
- `conedim2 (int)` – Second dimension.

Return (*PSDDomain*)

PSDDomain.withNamesOnAxis

`PSDDomain::t withNamesOnAxis(shared_ptr<ndarray<string,1>> names, int axis)`

Set index names in a specific axis.

Parameters

- `names (string[])` – List of names, this must match the actual dimension on that axis.
- `axis (int)` – The axis to change names on.

Return (*PSDDomain*)

14.2.27 Class PSDVariable

`mosek::fusion::PSDVariable`

This class represents a positive semidefinite variable.

Implements *ModelVariable*

Members *BaseVariable.antiDiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.

BaseVariable.setLevel – Input solution values for this variable

BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

BaseVariable.transpose – Transpose the variable.

BaseVariable.tril – Convert from a square variable to a trilinear representation.

ModelVariable.remove – Remove the variable from the model.

PSDVariable.toString – Create a string representation of the variable.

`PSDVariable.toString`

```
string toString()
```

Create a string representation of the variable.

Return (string)

14.2.28 Class Param

mosek::fusion::Param

The class defines a set of static methods for manipulating parameters.

Static members *Param.hstack* – Stack a list of parameters horizontally (i.e. along the second dimension).

Param.repeat – Repeat a parameter a number of times in the given dimension.

Param.stack – Stack a list of parameters in an arbitrary dimension.

Param.vstack – Stack a list of parameters vertically (i.e. along the first dimension).

Param.hstack

```
Parameter::t Param::hstack(shared_ptr<ndarray<Parameter::t,1>> p)
Parameter::t Param::hstack(Parameter::t p1, Parameter::t p2)
Parameter::t Param::hstack(Parameter::t p1, Parameter::t p2, Parameter::t p3)
```

Stack a list of parameters horizontally (i.e. along the second dimension). The parameters must have the same shape, except for the second dimension.

For example, if p^1, p^2, p^3 are three parameters of shape $(n,1)$ then their horizontal stack is the two-dimensional parameter

$$\begin{bmatrix} | & | & | \\ p^1 & p^2 & p^3 \\ | & | & | \end{bmatrix}$$

of shape $(n,3)$.

Parameters

- p (*Parameter*) – Parameters to stack.
- $p1$ (*Parameter*) – First parameter to stack.
- $p2$ (*Parameter*) – Second parameter to stack.
- $p3$ (*Parameter*) – Third parameter to stack.

Return (*Parameter*)

Param.repeat

```
Parameter::t Param::repeat(Parameter::t p, int n, int dim)
```

Repeat a parameter a number of times in the given dimension. This is equivalent to stacking n copies of the parameter in dimension dim ; see *Param.stack*.

Parameters

- p (*Parameter*) – The parameter to repeat.
- n (*int*) – Number of times to repeat. Must be strictly positive.
- dim (*int*) – The dimension in which to repeat. Must define a valid dimension index.

Return (*Parameter*)

Param.stack

```
Parameter::t Param::stack(shared_ptr<ndarray<Parameter::t,2>> p)
Parameter::t Param::stack(int dim, shared_ptr<ndarray<Parameter::t,1>> p)
Parameter::t Param::stack(int dim, Parameter::t p1, Parameter::t p2)
Parameter::t Param::stack(int dim, Parameter::t p1, Parameter::t p2, Parameter::t p3)
```

Stack a list of parameters along an arbitrary dimension. All parameters must have the same shape, except for dimension `dim`.

For example, suppose P, Q are two $n \times m$ parameters. Then stacking them in the first dimension produces a parameter of shape $(2n, m)$:

$$\begin{bmatrix} P \\ Q \end{bmatrix},$$

stacking them in the second dimension produces a parameter of shape $(n, 2m)$:

$$\begin{bmatrix} P & Q \end{bmatrix},$$

and stacking them in the third dimension produces a three-dimensional parameter of shape $(n, m, 2)$.

The version which takes a two-dimensional array of parameters constructs a block matrix parameter with the given parameters as blocks. The dimensions of the blocks must be suitably compatible.

Parameters

- `p` (*Parameter*[]) – Parameters to stack.
- `p` (*Parameter*[]) – Parameters to stack.
- `dim` (`int`) – The dimension in which to stack.
- `p1` (*Parameter*) – First parameter to stack.
- `p2` (*Parameter*) – Second parameter to stack.
- `p3` (*Parameter*) – Third parameter to stack.

Return (*Parameter*)

`Param.vstack`

```
Parameter::t Param::vstack(shared_ptr<ndarray<Parameter::t,1>> p)
Parameter::t Param::vstack(Parameter::t p1, Parameter::t p2)
Parameter::t Param::vstack(Parameter::t p1, Parameter::t p2, Parameter::t p3)
```

Stack a list of parameters vertically (i.e. along the first dimension). The parameters must have the same shape, except for the first dimension.

For example, if p^1, p^2, p^3 are three parameters of shape $(1, n)$ then their vertical stack is the two-dimensional parameter

$$\begin{bmatrix} -p^1- \\ -p^2- \\ -p^3- \end{bmatrix}$$

of shape $(3, n)$.

Parameters

- `p` (*Parameter*[]) – Parameters to stack.
- `p1` (*Parameter*) – First parameter to stack.
- `p2` (*Parameter*) – Second parameter to stack.
- `p3` (*Parameter*) – Third parameter to stack.

Return (*Parameter*)

14.2.29 Class Parameter

`mosek::fusion::Parameter`

The parameter class defines a parameter belonging to a specific model. A parameter is a placeholder for a constant (of any shape) whose value can be changed at any point in the life of the model. The Parameter object is not created directly, but through the factory methods *Model.parameter*.

Implements *Expression*

Members *Expression.eval* – Evaluate the expression and push the result onto the work stack.

Expression.index – Get a single element in the expression.

Expression.pick – Pick a number of elements from the expression.

Expression.toString – Return a string representation of the expression object.

Parameter.asExpr – Convert parameter to an expression

Parameter.getNumNonzero – Get number of non-zero elements in the parameter. This means the number of elements in the sparsity pattern - *not* the number of numeric non-zeros.

Parameter.getSize – Get the total number of elements in the parameter

Parameter.getValue – Get the current parameter values.

Parameter.isSparse – Return whether the parameter has a sparsity pattern.

Parameter.reshape – Reshape the parameter. The new shape must have the same size as the old.

Parameter.setValue – Set the parameter values.

Parameter.slice – Take a slice of the parameter.

`Parameter.asExpr`

`Expression::t asExpr()`

Convert parameter to an expression

Return (*Expression*)

`Parameter.getNumNonzero`

`int getNumNonzero()`

Get number of non-zero elements in the parameter. This means the number of elements in the sparsity pattern - *not* the number of numeric non-zeros.

Return (int)

`Parameter.getSize`

`long long getSize()`

Get the total number of elements in the parameter

Return (long long)

`Parameter.getValue`

`shared_ptr<ndarray<double,1>> getValue()`

Get the current parameter values.

Return (double[])

`Parameter.isSparse`


```
bool isSparse()
```

Return whether the parameter has a sparsity pattern.

Return (bool)

`Parameter.reshape`

```
Parameter::t reshape(shared_ptr<ndarray<int,1>> dims)
```

Reshape the parameter. The new shape must have the same size as the old.

Parameters `dims (int[])` – The new shape

Return (*Parameter*)

`Parameter.setValue`

```
void setValue(double value)
void setValue(shared_ptr<ndarray<double,1>> values)
void setValue(shared_ptr<ndarray<double,2>> values2)
```

Sets the value of this parameter.

The number of input values must match the size of the shape of the parameter. If the parameter is sparse, values on zero-positions are simply ignored.

Parameters

- `value (double)` – Set all parameter elements to this value.
- `values (double[])` – Set parameter elements to these values. The length of the array must match the size of the parameter. This form is valid no matter the shape of the parameter.
- `values2 (double[][])` – Set parameter elements to these values. The shape must exactly match the shape of the two-dimensional parameter.

`Parameter.slice`

```
Parameter::t slice(int start, int stop)
Parameter::t slice(shared_ptr<ndarray<int,1>> astart, shared_ptr<ndarray<int,1>>
→astop)
```

Take a slice of the parameter.

Parameters

- `start (int)` – The first index in a one-dimensional parameter.
- `stop (int)` – The last-plus-one index in a one-dimensional parameter.
- `astart (int[])` – The first index in the parameter.
- `astop (int[])` – The last-plus-one index in the parameter.

Return (*Parameter*)

14.2.30 Class RangeDomain

mosek::fusion::RangeDomain

The *RangeDomain* object is never instantiated directly: Instead use the relevant methods in *Domain*.

Members *RangeDomain.integral* – Creates a domain of integral variables.

RangeDomain.sparse – Creates a domain exploiting sparsity.

RangeDomain.symmetric – Creates a symmetric domain.

RangeDomain.withNamesOnAxis – Set index names in a specific axis.

RangeDomain.withShape – Set the shape of the domain.

Implemented by *SymmetricRangeDomain*

RangeDomain.integral

```
RangeDomain::t integral()
```

Modify a given domain restricting its elements to be integral.

Return (*RangeDomain*)

RangeDomain.sparse

```
RangeDomain::t sparse()
RangeDomain::t sparse(shared_ptr<ndarray<int,1>> sparsity)
RangeDomain::t sparse(shared_ptr<ndarray<int,2>> sparsity)
```

Creates a domain exploiting sparsity.

Parameters

- sparsity (int[])
- sparsity (int[][])

Return (*RangeDomain*)

RangeDomain.symmetric

```
SymmetricRangeDomain::t symmetric()
```

Creates a symmetric domain.

Return (*SymmetricRangeDomain*)

RangeDomain.withNamesOnAxis

```
RangeDomain::t withNamesOnAxis(shared_ptr<ndarray<string,1>> names, int axis)
```

Set index names in a specific axis.

Parameters

- names (string[]) – List of names, this must match the actual dimension on that axis.
- axis (int) – The axis to change names on.

Return (*RangeDomain*)

RangeDomain.withShape

```
RangeDomain::t withShape(shared_ptr<ndarray<int,1>> shp)
RangeDomain::t withShape(int dim0)
RangeDomain::t withShape(int dim0, int dim1)
RangeDomain::t withShape(int dim0, int dim1, int dim2)
```

Set the shape of the domain.

Parameters

- `shp (int[])` – The shape of the domain
- `dim0 (int)` – First dimension
- `dim1 (int)` – Second dimension
- `dim2 (int)` – Third dimension

Return (*RangeDomain*)

14.2.31 Class RangedConstraint

`mosek::fusion::RangedConstraint`

Represents a ranged constraint. The dual of a ranged constraint is the difference between upper and lower dual. To get them separately use the intermediate objects created with *RangedConstraint.lowerBoundCon* and *RangedConstraint.upperBoundCon*.

Implements *ModelConstraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

ModelConstraint.remove – Remove the constraint from the model.

ModelConstraint.toString – Create a human readable string representation of the constraint.

RangedConstraint.lowerBoundCon – Obtain the interface to the variable's lower bound.

RangedConstraint.upperBoundCon – Obtain the interface to the variable's upper bound.

`RangedConstraint.lowerBoundCon`

`BoundInterfaceConstraint::t lowerBoundCon()`

Obtain the interface to the variable's lower bound.

Return (*BoundInterfaceConstraint*)

`RangedConstraint.upperBoundCon`

`BoundInterfaceConstraint::t upperBoundCon()`

Obtain the interface to the variable's upper bound.

Return (*BoundInterfaceConstraint*)

14.2.32 Class RangedVariable

`mosek::fusion::RangedVariable`

Represents a ranged variable.

The dual of a ranged variable is the difference between upper and lower dual. To get them separately use the intermediate objects created with *RangedVariable.lowerBoundVar* and *RangedVariable.upperBoundVar*.

Implements *ModelVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.

BaseVariable.getND – Get the number of dimensions in the variable shape.

BaseVariable.getShape – Get the variable shape.

BaseVariable.getSize – Get the total number of elements in the variable.

BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..

BaseVariable.level – Get the primal solution value of the variable.

BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.

BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.

BaseVariable.setLevel – Input solution values for this variable

BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

BaseVariable.toString – Create a string representation of the variable.

BaseVariable.transpose – Transpose the variable.

BaseVariable.tril – Convert from a square variable to a trilinear representation.

ModelVariable.remove – Remove the variable from the model.

RangedVariable.lowerBoundVar – Obtain the interface to the variable's lower bound.

RangedVariable.upperBoundVar – Obtain the interface to the variable's upper bound.

`RangedVariable.lowerBoundVar`

`BoundInterfaceVariable::t lowerBoundVar()`

Obtain the interface to the variable's lower bound.

Return (*BoundInterfaceVariable*)

`RangedVariable.upperBoundVar`

`BoundInterfaceVariable::t upperBoundVar()`

Obtain the interface to the variable's upper bound.

Return (*BoundInterfaceVariable*)

14.2.33 Class Set

`mosek::fusion::Set`

This class contains static methods for creating and manipulating shape specifications.

Static members *Set.make* – Creates a shape.

Set.scalar – Create a shape of size 1.

Set.strides – Compute the strides from a shape.

`Set.make`

```
shared_ptr<ndarray<int,1>> Set::make(shared_ptr<ndarray<string,1>> names)
shared_ptr<ndarray<int,1>> Set::make(int sz)
shared_ptr<ndarray<int,1>> Set::make(int s1, int s2)
shared_ptr<ndarray<int,1>> Set::make(int s1, int s2, int s3)
shared_ptr<ndarray<int,1>> Set::make(shared_ptr<ndarray<int,1>> sizes)
shared_ptr<ndarray<int,1>> Set::make(shared_ptr<ndarray<int,1>> set1, shared_ptr
→<ndarray<int,1>> set2)
```

This static method is a factory for different kind of set objects:

- A (multi-dimensional) set of integers (shape).
- A set whose elements are strings.
- A set obtained as Cartesian product of sets given in a list.

Parameters

- `names (string[])` – A list of strings for a set of strings.
- `sz (int)` – The size of a one-dimensional set of integers.
- `s1 (int)` – Size of the first dimension.
- `s2 (int)` – Size of the second dimension.
- `s3 (int)` – Size of the third dimension.
- `sizes (int[])` – The sizes of dimensions for a multi-dimensional integer set.
- `set1 (int[])` – First factor in a Cartesian product.
- `set2 (int[])` – Second factor in a Cartesian product.

Return (`int[]`)

`Set.scalar`

```
shared_ptr<ndarray<int,1>> Set::scalar()
```

Create a shape of size 1.

Return (`int[]`)

`Set.strides`

```
shared_ptr<ndarray<long long,1>> Set::strides(shared_ptr<ndarray<int,1>> shape)
```

Compute the strides from a shape.

Parameters `shape (int[])`

Return (`long long[]`)

14.2.34 Class SimpleTerm

`mosek::fusion::SimpleTerm`

A class representing simple term, a basic building block for disjunctive constraints.

Implements *Term*

Members *Term.size* – Total size of the term.

14.2.35 Class SliceConstraint

`mosek::fusion::SliceConstraint`

An alias for a subset of constraints from a single *ModelConstraint*.

This class acts as a proxy for accessing a portion of a *ModelConstraint*. It is possible to access and modify the properties of the original variable using this alias. It does not access the *Model* directly, only through the original variable.

Implements *Constraint*

Members *Constraint.dual* – Get the dual solution values of the constraint.

Constraint.getModel – Return the model that the constraint belongs to.

Constraint.getND – Return the number of dimensions in the constraint shape.

Constraint.getShape – Return the constraint's shape.

Constraint.getSize – Return the total number of elements in the constraint.

Constraint.index – Get a single element from a constraint.

Constraint.level – Get the primal solution values of the constraint.

Constraint.remove – Remove the constraint from the model.

Constraint.slice – Create a slice constraint.

Constraint.update – Update part of a constraint.

SliceConstraint.toString – Create a human readable string representation of the constraint.

Implemented by *BoundInterfaceConstraint*

`SliceConstraint.toString`

`string toString()`

Create a human readable string representation of the constraint.

Return (string)

14.2.36 Class SliceVariable

`mosek::fusion::SliceVariable`

An alias for a subset of variables from a single *ModelVariable*.

This class acts as a proxy for accessing a portion of a *ModelVariable*. It is possible to access and modify the properties of the original variable using this alias, and the object can be used in expressions as any other *Variable* object.

Implements *BaseVariable*

Members *BaseVariable.antidiag* – Return the antidiagonal of a square variable matrix.

BaseVariable.asExpr – Create an expression corresponding to the variable object.

BaseVariable.diag – Return the diagonal of a square variable matrix.

BaseVariable.dual – Get the dual solution value of the variable.

BaseVariable.eval – Evaluate the expression and push the result onto the work stack.

BaseVariable.fromTril – Convert from a trilinear representation into a square variable.

BaseVariable.getDim – Return the d'th dimension in the expression.

BaseVariable.getModel – Get the *Model* object that the variable belongs to.
BaseVariable.getND – Get the number of dimensions in the variable shape.
BaseVariable.getShape – Get the variable shape.
BaseVariable.getSize – Get the total number of elements in the variable.
BaseVariable.index – Return a variable slice of size 1 corresponding to a single element in the variable object..
BaseVariable.level – Get the primal solution value of the variable.
BaseVariable.makeContinuous – Drop integrality constraints on the variable, if any.
BaseVariable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.
BaseVariable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.
BaseVariable.remove – Remove the variable from the model.
BaseVariable.reshape – Reshape the variable. The new shape must have the same total size as the current.
BaseVariable.setLevel – Input solution values for this variable
BaseVariable.slice – Create a slice variable by picking a range of indexes for each variable dimension.
BaseVariable.toString – Create a string representation of the variable.
BaseVariable.transpose – Transpose the variable.
BaseVariable.tril – Convert from a square variable to a trilinear representation.

Implemented by *BoundInterfaceVariable*

14.2.37 Class SymmetricLinearDomain

mosek::fusion::SymmetricLinearDomain

Represent a linear domain with symmetry.

Members *SymmetricLinearDomain.integral* – Creates a domain of integral variables.
SymmetricLinearDomain.sparse – Creates a domain exploiting sparsity.

SymmetricLinearDomain.integral

```
SymmetricLinearDomain::t integral()
```

Modify a given domain restricting its elements to be integral.

Return (*SymmetricLinearDomain*)

SymmetricLinearDomain.sparse

```
SymmetricLinearDomain::t sparse(shared_ptr<ndarray<int,1>> sparsity)
SymmetricLinearDomain::t sparse(shared_ptr<ndarray<int,2>> sparsity)
```

Modify a given domain exploiting sparsity, i.e only instantiating the variables that are actually used in the model.

Parameters

- sparsity (int[])
- sparsity (int[][])

Return (*SymmetricLinearDomain*)

14.2.38 Class SymmetricRangeDomain

mosek::fusion::SymmetricRangeDomain

Represent a ranged domain with symmetry.

Implements *RangeDomain*

Members *RangeDomain.integral* – Creates a domain of integral variables.

RangeDomain.sparse – Creates a domain exploiting sparsity.

RangeDomain.symmetric – Creates a symmetric domain.

RangeDomain.withNamesOnAxis – Set index names in a specific axis.

RangeDomain.withShape – Set the shape of the domain.

14.2.39 Class Term

mosek::fusion::Term

A class representing a term, which ultimately enters a disjunctive constraint. A term can be a simple term, such as a (possibly multidimensional) inequality or equality constructed with *DJC.term* or a conjunction of simple terms constructed with *DJC.AND*.

Members *Term.size* – Total size of the term.

Implemented by *SimpleTerm*

Term.size

```
int size()
```

Total size of the term.

Return (int)

14.2.40 Class Var

mosek::fusion::Var

Contains several static methods for manipulating variable objects and creating new variables from old ones.

Primal and dual solution values and additional operations on variables are available from the *Variable* class.

Static members *Var.compress* – Reshape a variable object by removing all dimensions of size 1.

Var.empty – Produce a new empty variable of the given shape.

Var.flatten – Create a one-dimensional logical view of a variable object.

Var.hrepeat – Repeat a variable a number of times in the second dimension.

Var.hstack – Stack a list of variables horizontally (i.e. along the second dimension).

Var.promote – Pad variable shape.

Var.repeat – Repeat a variable a number of times in the given dimension.

Var.reshape – Create a reshaped view of the given variable.

Var.stack – Stack a list of variables in an arbitrary dimension.

Var.vrepeat – Repeat a variable a number of times in the first dimension.

Var.vstack – Stack a list of variables vertically (i.e. along the first dimension).

Var.compress

```
Variable::t Var::compress(Variable::t v)
```

Reshape a variable object by removing all dimensions of size 1. The result contains the same number of elements, but all dimensions are larger than 1 (except if the original variable contains exactly one element).

Parameters v (*Variable*) – The variable object to compress.

Return (*Variable*)

`Var.empty`

```
Variable::t Var::empty(shared_ptr<ndarray<int,1>> shape)
```

Produce a new empty variable of the given shape.

Parameters `shape` (`int[]`) – The shape of a variable.

Return (*Variable*)

`Var.flatten`

```
Variable::t Var::flatten(Variable::t v)
```

Create a one-dimensional *Variable* object that represents a *logical* view of the k -dimensional variable V . The V matrix is traversed starting from the innermost dimension. For a two-dimensional matrix this means it is traversed row after row.

The returned view is a one-dimensional array of size equal to the product of dimensions of V .

Parameters v (*Variable*) – The variable to be flattened.

Return (*Variable*)

`Var.hrepeat`

```
Variable::t Var::hrepeat(Variable::t v, int n)
```

Repeat a variable a number of times in the second dimension. This is equivalent to horizontal stacking of n copies of the variable; see *Var.hstack*.

Parameters

- v (*Variable*) – A variable object.
- n (`int`) – Number of times to repeat v .

Return (*Variable*)

`Var.hstack`

```
Variable::t Var::hstack(shared_ptr<ndarray<Variable::t,1>> v)
Variable::t Var::hstack(Variable::t v1, Variable::t v2)
Variable::t Var::hstack(Variable::t v1, Variable::t v2, Variable::t v3)
```

Stack a list of variables horizontally (i.e. along the second dimension). The variables must have the same shape, except for the second dimension.

For example, if x^1, x^2, x^3 are three one-dimensional variables of length n then their horizontal stack is the matrix variable

$$\begin{bmatrix} | & | & | \\ x^1 & x^2 & x^3 \\ | & | & | \end{bmatrix}$$

of shape $(n, 3)$.

Parameters

- v (*Variable*[]) – List of variables to stack.
- $v1$ (*Variable*) – First variable in the stack.
- $v2$ (*Variable*) – Second variable in the stack.

- `v3` (*Variable*) – Third variable in the stack.
- Return** (*Variable*)

`Var.promote`

```
Variable::t Var::promote(Variable::t v, int nd)
```

Pad the variable shape up to `nd` dimensions.

Parameters

- `v` (*Variable*) – Variable to pad.
- `nd` (`int`) – Final number of dimensions.

Return (*Variable*)

`Var.repeat`

```
Variable::t Var::repeat(Variable::t v, int dim, int n)
Variable::t Var::repeat(Variable::t v, int n)
```

Repeat a variable a number of times in the given dimension. If `dim` is non-negative this is equivalent to stacking `n` copies of the variable in dimension `dim`; see *Var.stack*.

If `dim` is negative then a new dimension is added in front, so the new variable has shape `(n, v.shape())`.

The default is repeating in the first dimension as in *Var.vrepeat*.

Parameters

- `v` (*Variable*) – A variable object.
- `dim` (`int`) – Dimension to repeat in.
- `n` (`int`) – Number of times to repeat `v`.

Return (*Variable*)

`Var.reshape`

```
Variable::t Var::reshape(Variable::t v, shared_ptr<ndarray<int,1>> shape)
Variable::t Var::reshape(Variable::t v, int d1, int d2)
Variable::t Var::reshape(Variable::t v, int d1)
```

Create a reshaped view of the given variable.

Parameters

- `v` (*Variable*) – A variable object.
- `shape` (`int[]`) – An array containing the shape of the new variable.
- `d1` (`int`) – Size of first dimension in the result.
- `d2` (`int`) – Size of second dimension in the result.

Return (*Variable*)

`Var.stack`

```
Variable::t Var::stack(int dim, Variable::t v1, Variable::t v2)
Variable::t Var::stack(int dim, Variable::t v1, Variable::t v2, Variable::t v3)
Variable::t Var::stack(int dim, shared_ptr<ndarray<Variable::t,1>> v)
Variable::t Var::stack(Variable::t v1, Variable::t v2, int dim)
Variable::t Var::stack(Variable::t v1, Variable::t v2, Variable::t v3, int dim)
Variable::t Var::stack(shared_ptr<ndarray<Variable::t,1>> v, int dim)
Variable::t Var::stack(shared_ptr<ndarray<Variable::t,2>> vlist)
```

Stack a list of variables along an arbitrary dimension. All variables must have the same shape, except for dimension `dim`.

For example, suppose x, y are two matrix variables of shape $n \times m$. Then stacking them in the first dimension produces a matrix variable of shape $(2n, m)$:

$$\begin{bmatrix} x \\ y \end{bmatrix},$$

stacking them in the second dimension produces a matrix variable of shape $(n, 2m)$:

$$\begin{bmatrix} x & y \end{bmatrix},$$

and stacking them in the third dimension produces a three-dimensional variable of shape $(n, m, 2)$.

The version which takes a two-dimensional array of variables constructs a block matrix variable with the given variables as blocks. The dimensions of the blocks must be suitably compatible. The variables may be more than two-dimensional, if they have the same size in the remaining dimensions; the block stacking still takes place in the first and second dimension.

Parameters

- `dim (int)` – Dimension in which to stack.
- `v1 (Variable)` – First variable in the stack.
- `v2 (Variable)` – Second variable in the stack.
- `v3 (Variable)` – Third variable in the stack.
- `v (Variable[])` – List of variables to stack.
- `vlist (Variable[][])` – List of variables to stack.

Return `(Variable)`

`Var.vrepeat`

```
Variable::t Var::vrepeat(Variable::t v, int n)
```

Repeat a variable a number of times in the first dimension. This is equivalent to vertically stacking of n copies of the variable; see `Var.vstack`.

Parameters

- `v (Variable)` – A variable object.
- `n (int)` – Number of times to repeat `v`.

Return `(Variable)`

`Var.vstack`

```
Variable::t Var::vstack(shared_ptr<ndarray<Variable::t,1>> v)
Variable::t Var::vstack(Variable::t v1, Variable::t v2)
Variable::t Var::vstack(Variable::t v1, Variable::t v2, Variable::t v3)
```

Stack a list of variables vertically (i.e. along the first dimension). The variables must have the same shape, except for the first dimension.

For example, if y^1, y^2, y^3 are three horizontal vector variables of length n (and shape $(1, n)$) then their vertical stack is the matrix variable

$$\begin{bmatrix} -y^1- \\ -y^2- \\ -y^3- \end{bmatrix}$$

of shape $(3, n)$.

Parameters

- `v (Variable[])` – List of variables to stack.

- `v1` (*Variable*) – First variable in the stack.
- `v2` (*Variable*) – Second variable in the stack.
- `v3` (*Variable*) – Third variable in the stack.

Return (*Variable*)

14.2.41 Class Variable

`mosek::fusion::Variable`

An abstract variable object. This is the base class for all variable types in *Fusion*, and it contains several methods for manipulating variable objects.

The *Variable* object can be an interface to the normal model variables, e.g. *LinearVariable* and *ConicVariable*, to slices of other variables or to concatenations of other variables.

Primal and dual solution values can be accessed through the *Variable* object.

More static methods to manipulate variables are available in the *Var* class.

Implements *Expression*

Members *Expression.eval* – Evaluate the expression and push the result onto the work stack.

Expression.getDim – Return the d'th dimension in the expression.

Variable.antidiag – Return the anti-diagonal of a variable matrix.

Variable.asExpr – Create an expression corresponding to the variable object.

Variable.diag – Return the diagonal of a variable matrix.

Variable.dual – Get the dual solution value of the variable.

Variable.fromTril – Convert from a linear representation of the lower triangular part of a square variable into a square variable.

Variable.getModel – Return the model to which the variable belongs.

Variable.getND – Get the number of dimensions in the variable shape.

Variable.getShape – Return the shape of the variable.

Variable.getSize – Get the total number of elements in the variable.

Variable.index – Return a single entry in the variable.

Variable.level – Return the primal value of the variable as an array.

Variable.makeContinuous – Drop integrality constraints on the variable, if any.

Variable.makeInteger – Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

Variable.pick – Create a one-dimensional variable by picking a list of indexes from this variable.

Variable.remove – Remove the variable from the model.

Variable.reshape – Reshape the variable. The new shape must have the same total size as the current.

Variable.setLevel – Input solution values for this variable

Variable.slice – Create a slice variable by picking a range of indexes for each variable dimension.

Variable.toString – Create a string representation of the variable.

Variable.transpose – Transpose the variable.

Variable.tril – Convert a square variable into a linear representation of the lower triangular part of the variable.

Implemented by *BaseVariable*

`Variable.antidiag`

```
Variable::t antidiag(int index)
```

```
Variable::t antidiag()
```

Return the anti-diagonal of a variable matrix in a one-dimensional variable object. The main anti-diagonal is defined as starting with the element in the first row and last column.

Parameters `index` (int) – Index of the anti-diagonal. Index 0 means the main anti-diagonal, negative indices are below it and positive indices are above it.

Return (*Variable*)

`Variable.asExpr`

```
Expression::t asExpr()
```

Create an *Expression* object corresponding to $I \cdot V$, where I is the identity matrix and V is this variable.

Return (*Expression*)

`Variable.diag`

```
Variable::t diag(int index)
Variable::t diag()
```

Return the diagonal of a square variable matrix in a one-dimensional variable object. The main diagonal is defined as starting with the element in the first row and first column.

Parameters `index (int)` – Index of the diagonal. Index 0 means the main diagonal, negative indices are below it and positive indices are above it.

Return (*Variable*)

`Variable.dual`

```
shared_ptr<ndarray<double,1>> dual()
```

Get the dual solution value of the variable as an array. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (`double[]`)

`Variable.fromTril`

```
Variable::t fromTril(int dim)
```

Convert from a linear representation of the lower triangular part of a square variable into a square variable.

Parameters `dim (int)` – Dimension of the square variable.

Return (*Variable*)

`Variable.getModel`

```
Model::t getModel()
```

Get the *Model* object that the variable belongs to.

Return (*Model*)

`Variable.getND`

```
int getND()
```

Get the number of dimensions in the variable shape.

Return (`int`)

Variable.getShape

```
shared_ptr<ndarray<int,1>> getShape()
```

Get the variable shape.

Return (int[])

Variable.getSize

```
long long getSize()
```

Get the total number of elements in the variable.

Return (long long)

Variable.index

```
Variable::t index(int i1)
Variable::t index(int i1, int i2)
Variable::t index(int i1, int i2, int i3)
Variable::t index(shared_ptr<ndarray<int,1>> idx)
```

Return a variable of size one corresponding to a single element in the variable object.

Parameters

- *i1* (int) – Index in the first dimension of the element requested.
- *i2* (int) – Index in the second dimension of the element requested.
- *i3* (int) – Index in the third dimension of the element requested.
- *idx* (int[]) – List of indexes of the elements requested.

Return (*Variable*)

Variable.level

```
shared_ptr<ndarray<double,1>> level()
```

Get the primal solution value of the variable as an array. When the selected slice is multi-dimensional, this corresponds to the flattened slice of solution values.

Return (double[])

Variable.makeContinuous

```
void makeContinuous()
```

Drop integrality constraints on the variable, if any.

Variable.makeInteger

```
void makeInteger()
```

Apply integrality constraints on the variable. Has no effect on elements of semidefinite matrix variables.

Variable.pick

```
Variable::t pick(shared_ptr<ndarray<int,1>> idxs)
Variable::t pick(shared_ptr<ndarray<int,2>> midxs)
Variable::t pick(shared_ptr<ndarray<int,1>> i1, shared_ptr<ndarray<int,1>> i2)
Variable::t pick(shared_ptr<ndarray<int,1>> i1, shared_ptr<ndarray<int,1>> i2,
↳ shared_ptr<ndarray<int,1>> i3)
```

Create a one-dimensional variable by picking a list of indexes from this variable.

Parameters

- `idxs (int[])` – Indexes of the elements requested.
- `midxs (int[][])` – A sequence of multi-dimensional indexes of the elements requested.
- `i1 (int[])` – Index along the first dimension.
- `i2 (int[])` – Index along the second dimension.
- `i3 (int[])` – Index along the third dimension.

Return (*Variable*)

`Variable.remove`

```
void remove()
```

Remove the variable from the model and remove it from any constraints where it appears. Using the variable object after this method has been called results in undefined behavior.

`Variable.reshape`

```
Variable::t reshape(shared_ptr<ndarray<int,1>> shape)
Variable::t reshape(int dim0)
Variable::t reshape(int dim0, int dim1)
Variable::t reshape(int dim0, int dim1, int dim2)
```

Reshape the variable. The new shape must have the same total size as the current.

Parameters

- `shape (int[])` – The new shape.
- `dim0 (int)` – First dimension of new shape
- `dim1 (int)` – Second dimension of new shape
- `dim2 (int)` – Third dimension of new shape

Return (*Variable*)

`Variable.setLevel`

```
void setLevel(shared_ptr<ndarray<double,1>> v)
```

Set values for an initial solution for this variable. Note that these values are buffered until the solver is called; they are not available through the `level()` methods.

Parameters `v (double[])` – An array of values to be assigned to the variable.

`Variable.slice`

```
Variable::t slice(int first, int last)
Variable::t slice(shared_ptr<ndarray<int,1>> firsta, shared_ptr<ndarray<int,1>>
↳ lasta)
```

Create a slice variable by picking a range of indexes for each variable dimension.

Parameters

- **first** (*int*) – The index from which the slice begins.
- **last** (*int*) – The index after the last element of the slice.
- **firsta** (*int*[]) – The indices from which the slice of a multidimensional variable begins.
- **lasta** (*int*[]) – The indices after the last element of slice of a multidimensional variable.

Return (*Variable*)

`Variable.toString`

`string toString()`

Create a string representation of the variable.

Return (*string*)

`Variable.transpose`

`Variable::t transpose()`

Return the transpose of the current variable. The variable must have at most two dimensions.

Return (*Variable*)

`Variable.tril`

`Variable::t tril()`

Convert a square variable into a linear representation of the lower triangular part of the variable.

Return (*Variable*)

14.2.42 Class WorkStack

`mosek::fusion::WorkStack`

Stack object used to store expression evaluations. For internal use.

Members *WorkStack.allocf64* – Allocate n doubles and return the stack index of the first

WorkStack.alloci32 – Allocate n int32s and return the stack index of the first

WorkStack.alloci64 – Allocate n int64s and return the stack index of the first

WorkStack.clear – Clear all stacks

WorkStack.ensuref64 – Make sure that the double stack has capacity for n new items

WorkStack.ensurei32 – Make sure that the int32 stack has capacity for n new items

WorkStack.ensurei64 – Make sure that the int64 stack has capacity for n new items

WorkStack.peekf64 – Peek at one double item from stack

WorkStack.peeki32 – Peek at one int32 item from stack

WorkStack.peeki64 – Peek at one int64 item from stack

WorkStack.popf64 – Pop one or more double items from stack

WorkStack.popi32 – Pop one or more int32 items from stack

WorkStack.popi64 – Pop one or more int64 items from stack

WorkStack.pushf64 – Push an double value

WorkStack.pushi32 – Push an int32 value

WorkStack.pushi64 – Push an int64 value

`WorkStack.allocf64`


```
int allocf64(int n)
```

Allocate n doubles and return the stack index of the first

Parameters n (int)

Return (int)

WorkStack.alloci32

```
int alloci32(int n)
```

Allocate n int32s and return the stack index of the first

Parameters n (int)

Return (int)

WorkStack.alloci64

```
int alloci64(int n)
```

Allocate n int64s and return the stack index of the first

Parameters n (int)

Return (int)

WorkStack.clear

```
void clear()
```

Clear all stacks

WorkStack.ensuref64

```
void ensuref64(int n)
```

Make sure that the double stack has capacity for n new items

Parameters n (int) – Number of items to make space for

WorkStack.ensurei32

```
void ensurei32(int n)
```

Make sure that the int32 stack has capacity for n new items

Parameters n (int) – Number of items to make space for

WorkStack.ensurei64

```
void ensurei64(int n)
```

Make sure that the int64 stack has capacity for n new items

Parameters n (int) – Number of items to make space for

WorkStack.peekf64

```
double peekf64(int i)
double peekf64()
```

Peek at one double item from stack

Parameters *i* (int) – Peek at this index on the stack (default is 0)

Return (double)

WorkStack.peeki32

```
int peeki32(int i)
int peeki32()
```

Peek at one int32 item from stack

Parameters *i* (int) – Peek at this index on the stack (default is 0)

Return (int)

WorkStack.peeki64

```
long long peeki64(int i)
long long peeki64()
```

Peek at one int64 item from stack

Parameters *i* (int) – Peek at this index on the stack (default is 0)

Return (long long)

WorkStack.popf64

```
double popf64()
void popf64(int n, shared_ptr<ndarray<double,1>> r, int ofs)
int popf64(int n)
```

Pop one or more double items from stack

Parameters

- *n* (int) – Number of items to pop (default is 1)
- *r* (double[]) – Copy popped item to this array
- *ofs* (int) – Copy popped items to this offset in *r*

Return

- (double)
- (int)

WorkStack.popi32

```
int popi32()
void popi32(int n, shared_ptr<ndarray<int,1>> r, int ofs)
int popi32(int n)
```

Pop one or more int32 items from stack

Parameters

- *n* (int) – Number of items to pop (default is 1)
- *r* (int[]) – Copy popped item to this array
- *ofs* (int) – Copy popped items to this offset in *r*

Return (int)

WorkStack.popi64

```
long long popi64()
void popi64(int n, shared_ptr<ndarray<long long,1>> r, int ofs)
int popi64(int n)
```

Pop one or more int64 items from stack

Parameters

- **n** (int) – Number of items to pop (default is 1)
- **r** (long long[]) – Copy popped item to this array
- **ofs** (int) – Copy popped items to this offset in *r*

Return

- (long long)
- (int)

WorkStack.pushf64

```
void pushf64(double v)
```

Push an double value

Parameters **v** (double) – The value to store

WorkStack.pushi32

```
void pushi32(int v)
```

Push an int32 value

Parameters **v** (int) – The value to store

WorkStack.pushi64

```
void pushi64(long long v)
```

Push an int64 value

Parameters **v** (long long) – The value to store

14.3 Parameters grouped by topic

Basis identification

- *simLuToLRelPiv*
- *biCleanOptimizer*
- *biIgnoreMaxIter*
- *biIgnoreNumError*
- *biMaxIterations*
- *intpntBasis*
- *logBi*
- *logBiFreq*

Conic interior-point method

- *intpntCoTolDfeas*
- *intpntCoTolInfeas*
- *intpntCoTolMuRed*
- *intpntCoTolNearRel*
- *intpntCoTolPfeas*
- *intpntCoTolRelGap*

Data input/output

- *infeasReportAuto*
- *logFile*
- *writeJsonIndentation*
- *writeLpFullObj*
- *writeLpLineWidth*
- *basSolFileName*
- *dataFileName*
- *intSolFileName*
- *itrSolFileName*
- *writeLpGenVarName*

Dual simplex

- *simDualCrash*
- *simDualRestrictSelection*
- *simDualSelection*

Infeasibility report

- *logInfeasAna*

Interior-point method

- *intpntCoTolDfeas*
- *intpntCoTolInfeas*
- *intpntCoTolMuRed*
- *intpntCoTolNearRel*
- *intpntCoTolPfeas*
- *intpntCoTolRelGap*
- *intpntTolDfeas*
- *intpntTolDsafe*

- *intpntTolInfeas*
- *intpntTolMuRed*
- *intpntTolPath*
- *intpntTolPfeas*
- *intpntTolPsafe*
- *intpntTolRelGap*
- *intpntTolRelStep*
- *intpntTolStepSize*
- *biIgnoreMaxIter*
- *biIgnoreNumError*
- *intpntBasis*
- *intpntDiffStep*
- *intpntMaxIterations*
- *intpntMaxNumCor*
- *intpntOffColTrh*
- *intpntOrderGpNumSeeds*
- *intpntOrderMethod*
- *intpntRegularizationUse*
- *intpntScaling*
- *intpntSolveForm*
- *intpntStartingPoint*
- *logIntpnt*

License manager

- *cacheLicense*
- *licenseDebug*
- *licensePauseTime*
- *licenseSuppressExpireWrns*
- *licenseTrhExpiryWrn*
- *licenseWait*

Logging

- *log*
- *logBi*
- *logBiFreq*
- *logCutSecondOpt*
- *logExpand*
- *logFile*
- *logInfeasAna*
- *logIntpnt*
- *logLocalInfo*
- *logMio*
- *logMioFreq*
- *logOrder*
- *logPresolve*
- *logResponse*
- *logSim*
- *logSimFreq*

Mixed-integer optimization

- *mioDjcMaxBgm*
- *mioMaxTime*
- *mioRelGapConst*
- *mioTolAbsGap*
- *mioTolAbsRelaxInt*
- *mioTolFeas*
- *mioTolRelDualBoundImprovement*
- *mioTolRelGap*
- *logMio*
- *logMioFreq*
- *mioBranchDir*
- *mioConicOuterApproximation*
- *mioConstructSol*
- *mioCutClique*
- *mioCutCmir*
- *mioCutGmi*
- *mioCutImpliedBound*

- *mioCutKnapsackCover*
- *mioCutLipro*
- *mioCutSelectionLevel*
- *mioDataPermutationMethod*
- *mioFeaspumpLevel*
- *mioHeuristicLevel*
- *mioMaxNumBranches*
- *mioMaxNumRelaxs*
- *mioMaxNumRootCutRounds*
- *mioMaxNumSolutions*
- *mioMemoryEmphasisLevel*
- *mioNodeOptimizer*
- *mioNodeSelection*
- *mioNumericalEmphasisLevel*
- *mioPerspectiveReformulate*
- *mioProbingLevel*
- *mioPropagateObjectiveConstraint*
- *mioQcqpReformulationMethod*
- *mioRinsMaxNodes*
- *mioRootOptimizer*
- *mioRootRepeatPresolveLevel*
- *mioSeed*
- *mioSymmetryLevel*
- *mioVbDetectionLevel*

Output information

- *licenseSuppressExpireWrns*
- *licenseTrhExpiryWrn*
- *log*
- *logBi*
- *logBiFreq*
- *logCutSecondOpt*
- *logExpand*
- *logFile*
- *logInfeasAna*
- *logIntpnt*

- *logLocalInfo*
- *logMio*
- *logMioFreq*
- *logOrder*
- *logResponse*
- *logSim*
- *logSimFreq*
- *logSimMinor*

Overall solver

- *biCleanOptimizer*
- *infeasPreferPrimal*
- *licenseWait*
- *mioMode*
- *optimizer*
- *presolveLevel*
- *presolveUse*

Overall system

- *autoUpdateSolInfo*
- *licenseWait*
- *mtSpincount*
- *numThreads*
- *removeUnusedSolutions*
- *remoteOptserverHost*
- *remoteTlsCert*
- *remoteTlsCertPath*

Presolve

- *presolveTolAbsLindep*
- *presolveTolAij*
- *presolveTolPrimalInfeasPerturbation*
- *presolveTolRelLindep*
- *presolveTolS*
- *presolveTolX*
- *mioPresolveAggregatorUse*

- *presolveEliminatorMaxFill*
- *presolveEliminatorMaxNumTries*
- *presolveLevel*
- *presolveLindepAbsWorkTrh*
- *presolveLindepRelWorkTrh*
- *presolveLindepUse*
- *presolveMaxNumPass*
- *presolveUse*

Primal simplex

- *simPrimalCrash*
- *simPrimalRestrictSelection*
- *simPrimalSelection*

Simplex optimizer

- *basisRelTolS*
- *basisTolS*
- *basisTolX*
- *simLuTolRelPiv*
- *simplexAbsTolPiv*
- *logSim*
- *logSimFreq*
- *logSimMinor*
- *simBasisFactorUse*
- *simDegen*
- *simDualPhaseoneMethod*
- *simExploitDupvec*
- *simHotstart*
- *simHotstartLu*
- *simMaxIterations*
- *simMaxNumSetbacks*
- *simNonSingular*
- *simPrimalPhaseoneMethod*
- *simRefactorFreq*
- *simReformulation*
- *simSaveLu*

- *simScaling*
- *simScalingMethod*
- *simSeed*
- *simSolveForm*
- *simSwitchOptimizer*

Solution input/output

- *infeasReportAuto*
- *basSolFileName*
- *intSolFileName*
- *itrSolFileName*

Termination criteria

- *basisRelTolS*
- *basisTolS*
- *basisTolX*
- *intpntCoTolDfeas*
- *intpntCoTolInfeas*
- *intpntCoTolMuRed*
- *intpntCoTolNearRel*
- *intpntCoTolPfeas*
- *intpntCoTolRelGap*
- *intpntTolDfeas*
- *intpntTolInfeas*
- *intpntTolMuRed*
- *intpntTolPfeas*
- *intpntTolRelGap*
- *lowerObjCut*
- *lowerObjCutFiniteTrh*
- *mioMaxTime*
- *mioRelGapConst*
- *mioTolRelGap*
- *optimizerMaxTime*
- *upperObjCut*
- *upperObjCutFiniteTrh*
- *biMaxIterations*

- *intpntMaxIterations*
- *mioMaxNumBranches*
- *mioMaxNumRootCutRounds*
- *mioMaxNumSolutions*
- *simMaxIterations*

Other

- *remoteUseCompression*

14.4 Parameters (alphabetical list sorted by type)

- *Double parameters*
- *Integer parameters*
- *String parameters*

14.4.1 Double parameters

"basisRelTolS"

Maximum relative dual bound violation allowed in an optimal basic solution.

Default 1.0e-12

Accepted [0.0; +inf]

Example M->setSolverParam("basisRelTolS", 1.0e-12)

Generic name MSK_DPAR_BASIS_REL_TOL_S

Groups *Simplex optimizer, Termination criteria*

"basisTolS"

Maximum absolute dual bound violation in an optimal basic solution.

Default 1.0e-6

Accepted [1.0e-9; +inf]

Example M->setSolverParam("basisTolS", 1.0e-6)

Generic name MSK_DPAR_BASIS_TOL_S

Groups *Simplex optimizer, Termination criteria*

"basisTolX"

Maximum absolute primal bound violation allowed in an optimal basic solution.

Default 1.0e-6

Accepted [1.0e-9; +inf]

Example M->setSolverParam("basisTolX", 1.0e-6)

Generic name MSK_DPAR_BASIS_TOL_X

Groups *Simplex optimizer, Termination criteria*

"intpntCoTolDfeas"

Dual feasibility tolerance used by the interior-point optimizer for conic problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntCoTolDfeas", 1.0e-8)

See also *intpntCoTolNearRel*

Generic name MSK_DPAR_INTPNT_CO_TOL_DFEAS

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntCoTolInfeas"

Infeasibility tolerance used by the interior-point optimizer for conic problems. Controls when the interior-point optimizer declares the model primal or dual infeasible. A small number means the optimizer gets more conservative about declaring the model infeasible.

Default 1.0e-12

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntCoTolInfeas", 1.0e-12)

Generic name MSK_DPAR_INTPNT_CO_TOL_INFEAS

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntCoTolMuRed"

Relative complementarity gap tolerance used by the interior-point optimizer for conic problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntCoTolMuRed", 1.0e-8)

Generic name MSK_DPAR_INTPNT_CO_TOL_MU_RED

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntCoTolNearRel"

Optimality tolerance used by the interior-point optimizer for conic problems. If **MOSEK** cannot compute a solution that has the prescribed accuracy then it will check if the solution found satisfies the termination criteria with all tolerances multiplied by the value of this parameter. If yes, then the solution is also declared optimal.

Default 1000

Accepted [1.0; +inf]

Example M->setSolverParam("intpntCoTolNearRel", 1000)

Generic name MSK_DPAR_INTPNT_CO_TOL_NEAR_REL

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntCoTolPfeas"

Primal feasibility tolerance used by the interior-point optimizer for conic problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntCoTolPfeas", 1.0e-8)

See also [*intpntCoTolNearRel*](#)

Generic name MSK_DPAR_INTPNT_CO_TOL_PFEAS

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntCoTolRelGap"

Relative gap termination tolerance used by the interior-point optimizer for conic problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntCoTolRelGap", 1.0e-8)

See also [*intpntCoTolNearRel*](#)

Generic name MSK_DPAR_INTPNT_CO_TOL_REL_GAP

Groups *Interior-point method, Termination criteria, Conic interior-point method*

"intpntTolDfeas"

Dual feasibility tolerance used by the interior-point optimizer for linear problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntTolDfeas", 1.0e-8)

Generic name MSK_DPAR_INTPNT_TOL_DFEAS

Groups *Interior-point method, Termination criteria*

"intpntTolDsafe"

Controls the initial dual starting point used by the interior-point optimizer. If the interior-point optimizer converges slowly and/or the constraint or variable bounds are very large, then it might be worthwhile to increase this value.

Default 1.0

Accepted [1.0e-4; +inf]

Example M->setSolverParam("intpntTolDsafe", 1.0)

Generic name MSK_DPAR_INTPNT_TOL_DSAFE

Groups *Interior-point method*

"intpntTolInfeas"

Infeasibility tolerance used by the interior-point optimizer for linear problems. Controls when the interior-point optimizer declares the model primal or dual infeasible. A small number means the optimizer gets more conservative about declaring the model infeasible.

Default 1.0e-10

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntTolInfeas", 1.0e-10)

Generic name MSK_DPAR_INTPNT_TOL_INFEAS

Groups *Interior-point method, Termination criteria*

"intpntTolMuRed"

Relative complementarity gap tolerance used by the interior-point optimizer for linear problems.

Default 1.0e-16

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntTolMuRed", 1.0e-16)

Generic name MSK_DPAR_INTPNT_TOL_MU_RED

Groups *Interior-point method, Termination criteria*

"intpntTolPath"

Controls how close the interior-point optimizer follows the central path. A large value of this parameter means the central path is followed very closely. On numerically unstable problems it may be worthwhile to increase this parameter.

Default 1.0e-8

Accepted [0.0; 0.9999]

Example M->setSolverParam("intpntTolPath", 1.0e-8)

Generic name MSK_DPAR_INTPNT_TOL_PATH

Groups *Interior-point method*

"intpntTolPfeas"

Primal feasibility tolerance used by the interior-point optimizer for linear problems.

Default 1.0e-8

Accepted [0.0; 1.0]

Example M->setSolverParam("intpntTolPfeas", 1.0e-8)

Generic name MSK_DPAR_INTPNT_TOL_PFEAS

Groups *Interior-point method, Termination criteria*

"intpntTolPsafe"

Controls the initial primal starting point used by the interior-point optimizer. If the interior-point optimizer converges slowly and/or the constraint or variable bounds are very large, then it may be worthwhile to increase this value.

Default 1.0
Accepted [1.0e-4; +inf]
Example M->setSolverParam("intpntTolPsafe", 1.0)
Generic name MSK_DPAR_INTPNT_TOL_PSAFE
Groups *Interior-point method*

"intpntTolRelGap"

Relative gap termination tolerance used by the interior-point optimizer for linear problems.

Default 1.0e-8
Accepted [1.0e-14; +inf]
Example M->setSolverParam("intpntTolRelGap", 1.0e-8)
Generic name MSK_DPAR_INTPNT_TOL_REL_GAP
Groups *Termination criteria, Interior-point method*

"intpntTolRelStep"

Relative step size to the boundary for linear and quadratic optimization problems.

Default 0.9999
Accepted [1.0e-4; 0.999999]
Example M->setSolverParam("intpntTolRelStep", 0.9999)
Generic name MSK_DPAR_INTPNT_TOL_REL_STEP
Groups *Interior-point method*

"intpntTolStepSize"

Minimal step size tolerance. If the step size falls below the value of this parameter, then the interior-point optimizer assumes that it is stalled. In other words the interior-point optimizer does not make any progress and therefore it is better to stop.

Default 1.0e-6
Accepted [0.0; 1.0]
Example M->setSolverParam("intpntTolStepSize", 1.0e-6)
Generic name MSK_DPAR_INTPNT_TOL_STEP_SIZE
Groups *Interior-point method*

"lowerObjCut"

If either a primal or dual feasible solution is found proving that the optimal objective value is outside the interval [*lowerObjCut*, *upperObjCut*], then **MOSEK** is terminated.

Default -1.0e30
Accepted [-inf; +inf]
Example M->setSolverParam("lowerObjCut", -1.0e30)
See also *lowerObjCutFiniteTrh*
Generic name MSK_DPAR_LOWER_OBJ_CUT
Groups *Termination criteria*

"lowerObjCutFiniteTrh"

If the lower objective cut is less than the value of this parameter value, then the lower objective cut i.e. *lowerObjCut* is treated as $-\infty$.

Default -0.5e30
Accepted [-inf; +inf]
Example M->setSolverParam("lowerObjCutFiniteTrh", -0.5e30)
Generic name MSK_DPAR_LOWER_OBJ_CUT_FINITE_TRH
Groups *Termination criteria*

"mioDjcMaxBigm"

Maximum allowed big-M value when reformulating disjunctive constraints to linear constraints. Higher values make it more likely that a disjunction is reformulated to linear constraints, but also increase the risk of numerical problems.

Default 1.0e6

Accepted [0; +inf]

Example M->setSolverParam("mioDjcMaxBigm", 1.0e6)

Generic name MSK_DPAR_MIO_DJC_MAX_BIGM

Groups *Mixed-integer optimization*

"mioMaxTime"

This parameter limits the maximum time spent by the mixed-integer optimizer. A negative number means infinity.

Default -1.0

Accepted [-inf; +inf]

Example M->setSolverParam("mioMaxTime", -1.0)

Generic name MSK_DPAR_MIO_MAX_TIME

Groups *Mixed-integer optimization, Termination criteria*

"mioRelGapConst"

This value is used to compute the relative gap for the solution to an integer optimization problem.

Default 1.0e-10

Accepted [1.0e-15; +inf]

Example M->setSolverParam("mioRelGapConst", 1.0e-10)

Generic name MSK_DPAR_MIO_REL_GAP_CONST

Groups *Mixed-integer optimization, Termination criteria*

"mioTolAbsGap"

Absolute optimality tolerance employed by the mixed-integer optimizer.

Default 0.0

Accepted [0.0; +inf]

Example M->setSolverParam("mioTolAbsGap", 0.0)

Generic name MSK_DPAR_MIO_TOL_ABS_GAP

Groups *Mixed-integer optimization*

"mioTolAbsRelaxInt"

Absolute integer feasibility tolerance. If the distance to the nearest integer is less than this tolerance then an integer constraint is assumed to be satisfied.

Default 1.0e-5

Accepted [1e-9; +inf]

Example M->setSolverParam("mioTolAbsRelaxInt", 1.0e-5)

Generic name MSK_DPAR_MIO_TOL_ABS_RELAX_INT

Groups *Mixed-integer optimization*

"mioTolFeas"

Feasibility tolerance for mixed integer solver.

Default 1.0e-6

Accepted [1e-9; 1e-3]

Example M->setSolverParam("mioTolFeas", 1.0e-6)

Generic name MSK_DPAR_MIO_TOL_FEAS

Groups *Mixed-integer optimization*

"mioTolRelDualBoundImprovement"

If the relative improvement of the dual bound is smaller than this value, the solver will terminate the root cut generation. A value of 0.0 means that the value is selected automatically.

Default 0.0

Accepted [0.0; 1.0]

Example M->setSolverParam("mioTolRelDualBoundImprovement", 0.0)

Generic name MSK_DPAR_MIO_TOL_REL_DUAL_BOUND_IMPROVEMENT

Groups *Mixed-integer optimization*

"mioTolRelGap"

Relative optimality tolerance employed by the mixed-integer optimizer.

Default 1.0e-4

Accepted [0.0; +inf]

Example M->setSolverParam("mioTolRelGap", 1.0e-4)

Generic name MSK_DPAR_MIO_TOL_REL_GAP

Groups *Mixed-integer optimization, Termination criteria*

"optimizerMaxTime"

Maximum amount of time the optimizer is allowed to spent on the optimization. A negative number means infinity.

Default -1.0

Accepted [-inf; +inf]

Example M->setSolverParam("optimizerMaxTime", -1.0)

Generic name MSK_DPAR_OPTIMIZER_MAX_TIME

Groups *Termination criteria*

"presolveTolAbsLindep"

Absolute tolerance employed by the linear dependency checker.

Default 1.0e-6

Accepted [0.0; +inf]

Example M->setSolverParam("presolveTolAbsLindep", 1.0e-6)

Generic name MSK_DPAR_PREOLVE_TOL_ABS_LINDEP

Groups *Presolve*

"presolveTolAij"

Absolute zero tolerance employed for a_{ij} in the presolve.

Default 1.0e-12

Accepted [1.0e-15; +inf]

Example M->setSolverParam("presolveTolAij", 1.0e-12)

Generic name MSK_DPAR_PREOLVE_TOL_AIJ

Groups *Presolve*

"presolveTolPrimalInfeasPerturbation"

The presolve is allowed to perturb a bound on a constraint or variable by this amount if it removes an infeasibility.

Default 1.0e-6

Accepted [0.0; +inf]

Example M->setSolverParam("presolveTolPrimalInfeasPerturbation", 1.0e-6)

Generic name MSK_DPAR_PREOLVE_TOL_PRIMAL_INFEAS_PERTURBATION

Groups *Presolve*

"presolveTolRelLindep"

Relative tolerance employed by the linear dependency checker.

Default 1.0e-10

Accepted [0.0; +inf]

Example M->setSolverParam("presolveTolRelLindep", 1.0e-10)

Generic name MSK_DPAR_PREOLVE_TOL_REL_LINDEP

Groups *Presolve*

"presolveTolS"

Absolute zero tolerance employed for s_i in the presolve.

Default 1.0e-8

Accepted [0.0; +inf]

Example M->setSolverParam("presolveTolS", 1.0e-8)

Generic name MSK_DPAR_PREOLVE_TOL_S

Groups *Presolve*

"presolveTolX"

Absolute zero tolerance employed for x_j in the presolve.

Default 1.0e-8

Accepted [0.0; +inf]

Example M->setSolverParam("presolveTolX", 1.0e-8)

Generic name MSK_DPAR_PREOLVE_TOL_X

Groups *Presolve*

"simLuTolRelPiv"

Relative pivot tolerance employed when computing the LU factorization of the basis in the simplex optimizers and in the basis identification procedure. A value closer to 1.0 generally improves numerical stability but typically also implies an increase in the computational work.

Default 0.01

Accepted [1.0e-6; 0.999999]

Example M->setSolverParam("simLuTolRelPiv", 0.01)

Generic name MSK_DPAR_SIM_LU_TOL_REL_PIV

Groups *Basis identification, Simplex optimizer*

"simplexAbsTolPiv"

Absolute pivot tolerance employed by the simplex optimizers.

Default 1.0e-7

Accepted [1.0e-12; +inf]

Example M->setSolverParam("simplexAbsTolPiv", 1.0e-7)

Generic name MSK_DPAR_SIMPLEX_ABS_TOL_PIV

Groups *Simplex optimizer*

"upperObjCut"

If either a primal or dual feasible solution is found proving that the optimal objective value is outside the interval [*lowerObjCut*, *upperObjCut*], then **MOSEK** is terminated.

Default 1.0e30

Accepted [-inf; +inf]

Example M->setSolverParam("upperObjCut", 1.0e30)

See also *upperObjCutFiniteTrh*

Generic name MSK_DPAR_UPPER_OBJ_CUT

Groups *Termination criteria*

"upperObjCutFiniteTrh"

If the upper objective cut is greater than the value of this parameter, then the upper objective cut *upperObjCut* is treated as ∞ .

Default 0.5e30

Accepted [-inf; +inf]

Example M->setSolverParam("upperObjCutFiniteTrh", 0.5e30)

Generic name MSK_DPAR_UPPER_OBJ_CUT_FINITE_TRH

Groups *Termination criteria*

14.4.2 Integer parameters

"autoUpdateSolInfo"

Controls whether the solution information items are automatically updated after an optimization is performed.

Default "off"

Accepted "on", "off"

Example M->setSolverParam("autoUpdateSolInfo", "off")

Generic name MSK_IPAR_AUTO_UPDATE_SOL_INFO

Groups *Overall system*

"biCleanOptimizer"

Controls which simplex optimizer is used in the clean-up phase. Anything else than "primalSimplex" or "dualSimplex" is equivalent to "freeSimplex".

Default "free"

Accepted "free", "intpnt", "conic", "primalSimplex", "dualSimplex", "freeSimplex", "mixedInt"

Example M->setSolverParam("biCleanOptimizer", "free")

Generic name MSK_IPAR_BI_CLEAN_OPTIMIZER

Groups *Basis identification, Overall solver*

"biIgnoreMaxIter"

If the parameter *intpntBasis* has the value "noError" and the interior-point optimizer has terminated due to maximum number of iterations, then basis identification is performed if this parameter has the value "on".

Default "off"

Accepted "on", "off"

Example M->setSolverParam("biIgnoreMaxIter", "off")

Generic name MSK_IPAR_BI_IGNORE_MAX_ITER

Groups *Interior-point method, Basis identification*

"biIgnoreNumError"

If the parameter *intpntBasis* has the value "noError" and the interior-point optimizer has terminated due to a numerical problem, then basis identification is performed if this parameter has the value "on".

Default "off"

Accepted "on", "off"

Example M->setSolverParam("biIgnoreNumError", "off")

Generic name MSK_IPAR_BI_IGNORE_NUM_ERROR

Groups *Interior-point method, Basis identification*

"biMaxIterations"

Controls the maximum number of simplex iterations allowed to optimize a basis after the basis identification.

Default 1000000
Accepted [0; +inf]
Example M->setSolverParam("biMaxIterations", 1000000)
Generic name MSK_IPAR_BI_MAX_ITERATIONS
Groups *Basis identification, Termination criteria*

"cacheLicense"

Specifies if the license is kept checked out for the lifetime of the **MOSEK** environment/model/process ("**on**") or returned to the server immediately after the optimization ("**off**"). By default the license is checked out for the lifetime of the process by the first call to *Model.solve*. Check-in and check-out of licenses have an overhead. Frequent communication with the license server should be avoided.

Default "**on**"
Accepted "**on**", "**off**"
Example M->setSolverParam("cacheLicense", "on")
Generic name MSK_IPAR_CACHE_LICENSE
Groups *License manager*

"infeasPreferPrimal"

If both certificates of primal and dual infeasibility are supplied then only the primal is used when this option is turned on.

Default "**on**"
Accepted "**on**", "**off**"
Example M->setSolverParam("infeasPreferPrimal", "on")
Generic name MSK_IPAR_INFEAS_PREFER_PRIMAL
Groups *Overall solver*

"infeasReportAuto"

Controls whether an infeasibility report is automatically produced after the optimization if the problem is primal or dual infeasible.

Default "**off**"
Accepted "**on**", "**off**"
Example M->setSolverParam("infeasReportAuto", "off")
Generic name MSK_IPAR_INFEAS_REPORT_AUTO
Groups *Data input/output, Solution input/output*

"intpntBasis"

Controls whether the interior-point optimizer also computes an optimal basis.

Default "**always**"
Accepted "**never**", "**always**", "**noError**", "**ifFeasible**", "**reserved**"
Example M->setSolverParam("intpntBasis", "always")
See also *biIgnoreMaxIter*, *biIgnoreNumError*, *biMaxIterations*, *biCleanOptimizer*
Generic name MSK_IPAR_INTPNT_BASIS
Groups *Interior-point method, Basis identification*

"intpntDiffStep"

Controls whether different step sizes are allowed in the primal and dual space.

Default "**on**"
Accepted

- "**on**": Different step sizes are allowed.
- "**off**": Different step sizes are not allowed.

Example `M->setSolverParam("intpntDiffStep", "on")`

Generic name `MSK_IPAR_INTPNT_DIFF_STEP`

Groups *Interior-point method*

"intpntMaxIterations"

Controls the maximum number of iterations allowed in the interior-point optimizer.

Default 400

Accepted `[0; +inf]`

Example `M->setSolverParam("intpntMaxIterations", 400)`

Generic name `MSK_IPAR_INTPNT_MAX_ITERATIONS`

Groups *Interior-point method, Termination criteria*

"intpntMaxNumCor"

Controls the maximum number of correctors allowed by the multiple corrector procedure. A negative value means that **MOSEK** is making the choice.

Default -1

Accepted `[-1; +inf]`

Example `M->setSolverParam("intpntMaxNumCor", -1)`

Generic name `MSK_IPAR_INTPNT_MAX_NUM_COR`

Groups *Interior-point method*

"intpntOffColTrh"

Controls how many offending columns are detected in the Jacobian of the constraint matrix.

0	no detection
1	aggressive detection
> 1	higher values mean less aggressive detection

Default 40

Accepted `[0; +inf]`

Example `M->setSolverParam("intpntOffColTrh", 40)`

Generic name `MSK_IPAR_INTPNT_OFF_COL_TRH`

Groups *Interior-point method*

"intpntOrderGpNumSeeds"

The GP ordering is dependent on a random seed. Therefore, trying several random seeds may lead to a better ordering. This parameter controls the number of random seeds tried.

A value of 0 means that MOSEK makes the choice.

Default 0

Accepted `[0; +inf]`

Example `M->setSolverParam("intpntOrderGpNumSeeds", 0)`

Generic name `MSK_IPAR_INTPNT_ORDER_GP_NUM_SEEDS`

Groups *Interior-point method*

"intpntOrderMethod"

Controls the ordering strategy used by the interior-point optimizer when factorizing the Newton equation system.

Default *"free"*

Accepted *"free", "appminloc", "experimental", "tryGraphpar", "forceGraphpar", "none"*

Example `M->setSolverParam("intpntOrderMethod", "free")`

Generic name `MSK_IPAR_INTPNT_ORDER_METHOD`

Groups *Interior-point method*

"intpntRegularizationUse"

Controls whether regularization is allowed.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("intpntRegularizationUse", "on")`

Generic name `MSK_IPAR_INTPNT_REGULARIZATION_USE`

Groups *Interior-point method*

"intpntScaling"

Controls how the problem is scaled before the interior-point optimizer is used.

Default *"free"*

Accepted *"free", "none"*

Example `M->setSolverParam("intpntScaling", "free")`

Generic name `MSK_IPAR_INTPNT_SCALING`

Groups *Interior-point method*

"intpntSolveForm"

Controls whether the primal or the dual problem is solved.

Default *"free"*

Accepted *"free", "primal", "dual"*

Example `M->setSolverParam("intpntSolveForm", "free")`

Generic name `MSK_IPAR_INTPNT_SOLVE_FORM`

Groups *Interior-point method*

"intpntStartingPoint"

Starting point used by the interior-point optimizer.

Default *"free"*

Accepted *"free", "guess", "constant", "satisfyBounds"*

Example `M->setSolverParam("intpntStartingPoint", "free")`

Generic name `MSK_IPAR_INTPNT_STARTING_POINT`

Groups *Interior-point method*

"licenseDebug"

This option is used to turn on debugging of the license manager.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("licenseDebug", "off")`

Generic name `MSK_IPAR_LICENSE_DEBUG`

Groups *License manager*

"licensePauseTime"

If *licenseWait* is *"on"* and no license is available, then **MOSEK** sleeps a number of milliseconds between each check of whether a license has become free.

Default 100

Accepted `[0; 1000000]`

Example `M->setSolverParam("licensePauseTime", 100)`

Generic name `MSK_IPAR_LICENSE_PAUSE_TIME`

Groups *License manager*

"licenseSuppressExpireWrns"

Controls whether license features expire warnings are suppressed.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("licenseSuppressExpireWrns", "off")`

Generic name `MSK_IPAR_LICENSE_SUPPRESS_EXPIRE_WRNS`

Groups *License manager, Output information*

"licenseTrhExpiryWrn"

If a license feature expires in a numbers of days less than the value of this parameter then a warning will be issued.

Default `7`

Accepted `[0; +inf]`

Example `M->setSolverParam("licenseTrhExpiryWrn", 7)`

Generic name `MSK_IPAR_LICENSE_TRH_EXPIRY_WRN`

Groups *License manager, Output information*

"licenseWait"

If all licenses are in use **MOSEK** returns with an error code. However, by turning on this parameter **MOSEK** will wait for an available license.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("licenseWait", "off")`

Generic name `MSK_IPAR_LICENSE_WAIT`

Groups *Overall solver, Overall system, License manager*

"log"

Controls the amount of log information. The value 0 implies that all log information is suppressed. A higher level implies that more information is logged.

Please note that if a task is employed to solve a sequence of optimization problems the value of this parameter is reduced by the value of *logCutSecondOpt* for the second and any subsequent optimizations.

Default `10`

Accepted `[0; +inf]`

Example `M->setSolverParam("log", 10)`

See also *logCutSecondOpt*

Generic name `MSK_IPAR_LOG`

Groups *Output information, Logging*

"logBi"

Controls the amount of output printed by the basis identification procedure. A higher level implies that more information is logged.

Default `1`

Accepted `[0; +inf]`

Example `M->setSolverParam("logBi", 1)`

Generic name `MSK_IPAR_LOG_BI`

Groups *Basis identification, Output information, Logging*

"logBiFreq"

Controls how frequently the optimizer outputs information about the basis identification and how frequent the user-defined callback function is called.

Default `2500`

Accepted `[0; +inf]`

Example `M->setSolverParam("logBiFreq", 2500)`

Generic name `MSK_IPAR_LOG_BI_FREQ`

Groups *Basis identification, Output information, Logging*

"logCutSecondOpt"

If a task is employed to solve a sequence of optimization problems, then the value of the log levels is reduced by the value of this parameter. E.g *log* and *logSim* are reduced by the value of this parameter for the second and any subsequent optimizations.

Default 1

Accepted [0; +inf]

Example M->setSolverParam("logCutSecondOpt", 1)

See also *log*, *logIntpnt*, *logMio*, *logSim*

Generic name MSK_IPAR_LOG_CUT_SECOND_OPT

Groups *Output information*, *Logging*

"logExpand"

Controls the amount of logging when a data item such as the maximum number constraints is expanded.

Default 0

Accepted [0; +inf]

Example M->setSolverParam("logExpand", 0)

Generic name MSK_IPAR_LOG_EXPAND

Groups *Output information*, *Logging*

"logFile"

If turned on, then some log info is printed when a file is written or read.

Default 1

Accepted [0; +inf]

Example M->setSolverParam("logFile", 1)

Generic name MSK_IPAR_LOG_FILE

Groups *Data input/output*, *Output information*, *Logging*

"logInfeasAna"

Controls amount of output printed by the infeasibility analyzer procedures. A higher level implies that more information is logged.

Default 1

Accepted [0; +inf]

Example M->setSolverParam("logInfeasAna", 1)

Generic name MSK_IPAR_LOG_INFEAS_ANA

Groups *Infeasibility report*, *Output information*, *Logging*

"logIntpnt"

Controls amount of output printed by the interior-point optimizer. A higher level implies that more information is logged.

Default 1

Accepted [0; +inf]

Example M->setSolverParam("logIntpnt", 1)

Generic name MSK_IPAR_LOG_INTPNT

Groups *Interior-point method*, *Output information*, *Logging*

"logLocalInfo"

Controls whether local identifying information like environment variables, filenames, IP addresses etc. are printed to the log.

Note that this will only affect some functions. Some functions that specifically emit system information will not be affected.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("logLocalInfo", "on")`

Generic name MSK_IPAR_LOG_LOCAL_INFO

Groups *Output information, Logging*

"logMio"

Controls the log level for the mixed-integer optimizer. A higher level implies that more information is logged.

Default 4

Accepted `[0; +inf]`

Example `M->setSolverParam("logMio", 4)`

Generic name MSK_IPAR_LOG_MIO

Groups *Mixed-integer optimization, Output information, Logging*

"logMioFreq"

Controls how frequent the mixed-integer optimizer prints the log line. It will print line every time *logMioFreq* relaxations have been solved.

Default 10

Accepted `[-inf; +inf]`

Example `M->setSolverParam("logMioFreq", 10)`

Generic name MSK_IPAR_LOG_MIO_FREQ

Groups *Mixed-integer optimization, Output information, Logging*

"logOrder"

If turned on, then factor lines are added to the log.

Default 1

Accepted `[0; +inf]`

Example `M->setSolverParam("logOrder", 1)`

Generic name MSK_IPAR_LOG_ORDER

Groups *Output information, Logging*

"logPresolve"

Controls amount of output printed by the presolve procedure. A higher level implies that more information is logged.

Default 1

Accepted `[0; +inf]`

Example `M->setSolverParam("logPresolve", 1)`

Generic name MSK_IPAR_LOG_PREOLVE

Groups *Logging*

"logResponse"

Controls amount of output printed when response codes are reported. A higher level implies that more information is logged.

Default 0

Accepted `[0; +inf]`

Example `M->setSolverParam("logResponse", 0)`

Generic name MSK_IPAR_LOG_RESPONSE

Groups *Output information, Logging*

"logSim"

Controls amount of output printed by the simplex optimizer. A higher level implies that more information is logged.

Default 4

Accepted [0; +inf]
Example M->setSolverParam("logSim", 4)
Generic name MSK_IPAR_LOG_SIM
Groups *Simplex optimizer, Output information, Logging*

"logSimFreq"

Controls how frequent the simplex optimizer outputs information about the optimization and how frequent the user-defined callback function is called.

Default 1000
Accepted [0; +inf]
Example M->setSolverParam("logSimFreq", 1000)
Generic name MSK_IPAR_LOG_SIM_FREQ
Groups *Simplex optimizer, Output information, Logging*

"logSimMinor"

Currently not in use.

Default 1
Accepted [0; +inf]
Example M->setSolverParam("logSimMinor", 1)
Generic name MSK_IPAR_LOG_SIM_MINOR
Groups *Simplex optimizer, Output information*

"mioBranchDir"

Controls whether the mixed-integer optimizer is branching up or down by default.

Default *"free"*
Accepted *"free", "up", "down", "near", "far", "rootLp", "guided", "pseudocost"*
Example M->setSolverParam("mioBranchDir", "free")
Generic name MSK_IPAR_MIO_BRANCH_DIR
Groups *Mixed-integer optimization*

"mioConicOuterApproximation"

If this option is turned on outer approximation is used when solving relaxations of conic problems; otherwise interior point is used.

Default *"off"*
Accepted *"on", "off"*
Example M->setSolverParam("mioConicOuterApproximation", "off")
Generic name MSK_IPAR_MIO_CONIC_OUTER_APPROXIMATION
Groups *Mixed-integer optimization*

"mioConstructSol"

If set to *"on"* and all integer variables have been given a value for which a feasible mixed integer solution exists, then **MOSEK** generates an initial solution to the mixed integer problem by fixing all integer values and solving the remaining problem.

Default *"off"*
Accepted *"on", "off"*
Example M->setSolverParam("mioConstructSol", "off")
Generic name MSK_IPAR_MIO_CONSTRUCT_SOL
Groups *Mixed-integer optimization*

"mioCutClique"

Controls whether clique cuts should be generated.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutClique", "on")`

Generic name `MSK_IPAR_MIO_CUT_CLIQUE`

Groups *Mixed-integer optimization*

"mioCutCmir"

Controls whether mixed integer rounding cuts should be generated.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutCmir", "on")`

Generic name `MSK_IPAR_MIO_CUT_CMIR`

Groups *Mixed-integer optimization*

"mioCutGmi"

Controls whether GMI cuts should be generated.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutGmi", "on")`

Generic name `MSK_IPAR_MIO_CUT_GMI`

Groups *Mixed-integer optimization*

"mioCutImpliedBound"

Controls whether implied bound cuts should be generated.

Default *"on"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutImpliedBound", "on")`

Generic name `MSK_IPAR_MIO_CUT_IMPLIED_BOUND`

Groups *Mixed-integer optimization*

"mioCutKnapsackCover"

Controls whether knapsack cover cuts should be generated.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutKnapsackCover", "off")`

Generic name `MSK_IPAR_MIO_CUT_KNAPSACK_COVER`

Groups *Mixed-integer optimization*

"mioCutLipro"

Controls whether lift-and-project cuts should be generated.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioCutLipro", "off")`

Generic name `MSK_IPAR_MIO_CUT_LIPRO`

Groups *Mixed-integer optimization*

"mioCutSelectionLevel"

Controls how aggressively generated cuts are selected to be included in the relaxation.

- -1. The optimizer chooses the level of cut selection
- 0. Generated cuts less likely to be added to the relaxation
- 1. Cuts are more aggressively selected to be included in the relaxation

Default *-1*

Accepted [-1; +1]

Example M->setSolverParam("mioCutSelectionLevel", -1)

Generic name MSK_IPAR_MIO_CUT_SELECTION_LEVEL

Groups *Mixed-integer optimization*

"mioDataPermutationMethod"

Controls what problem data permutation method is applied to mixed-integer problems.

Default *"none"*

Accepted *"none", "cyclicShift", "random"*

Example M->setSolverParam("mioDataPermutationMethod", "none")

Generic name MSK_IPAR_MIO_DATA_PERMUTATION_METHOD

Groups *Mixed-integer optimization*

"mioFeaspumpLevel"

Controls the way the Feasibility Pump heuristic is employed by the mixed-integer optimizer.

- -1. The optimizer chooses how the Feasibility Pump is used
- 0. The Feasibility Pump is disabled
- 1. The Feasibility Pump is enabled with an effort to improve solution quality
- 2. The Feasibility Pump is enabled with an effort to reach feasibility early

Default -1

Accepted [-1; 2]

Example M->setSolverParam("mioFeaspumpLevel", -1)

Generic name MSK_IPAR_MIO_FEASPUMP_LEVEL

Groups *Mixed-integer optimization*

"mioHeuristicLevel"

Controls the heuristic employed by the mixed-integer optimizer to locate an initial good integer feasible solution. A value of zero means the heuristic is not used at all. A larger value than 0 means that a gradually more sophisticated heuristic is used which is computationally more expensive. A negative value implies that the optimizer chooses the heuristic. Normally a value around 3 to 5 should be optimal.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("mioHeuristicLevel", -1)

Generic name MSK_IPAR_MIO_HEURISTIC_LEVEL

Groups *Mixed-integer optimization*

"mioMaxNumBranches"

Maximum number of branches allowed during the branch and bound search. A negative value means infinite.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("mioMaxNumBranches", -1)

Generic name MSK_IPAR_MIO_MAX_NUM_BRANCHES

Groups *Mixed-integer optimization, Termination criteria*

"mioMaxNumRelaxs"

Maximum number of relaxations allowed during the branch and bound search. A negative value means infinite.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("mioMaxNumRelaxs", -1)

Generic name MSK_IPAR_MIO_MAX_NUM_RELAXS

Groups *Mixed-integer optimization*

"mioMaxNumRootCutRounds"

Maximum number of cut separation rounds at the root node.

Default 100

Accepted [0; +inf]

Example M->setSolverParam("mioMaxNumRootCutRounds", 100)

Generic name MSK_IPAR_MIO_MAX_NUM_ROOT_CUT_ROUNDS

Groups *Mixed-integer optimization, Termination criteria*

"mioMaxNumSolutions"

The mixed-integer optimizer can be terminated after a certain number of different feasible solutions has been located. If this parameter has the value $n > 0$, then the mixed-integer optimizer will be terminated when n feasible solutions have been located.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("mioMaxNumSolutions", -1)

Generic name MSK_IPAR_MIO_MAX_NUM_SOLUTIONS

Groups *Mixed-integer optimization, Termination criteria*

"mioMemoryEmphasisLevel"

Controls how much emphasis is put on reducing memory usage. Being more conservative about memory usage may come at the cost of decreased solution speed.

- 0. The optimizer chooses
- 1. More emphasis is put on reducing memory usage and less on speed

Default 0

Accepted [0; +1]

Example M->setSolverParam("mioMemoryEmphasisLevel", 0)

Generic name MSK_IPAR_MIO_MEMORY_EMPHASIS_LEVEL

Groups *Mixed-integer optimization*

"mioMode"

Controls whether the optimizer includes the integer restrictions and disjunctive constraints when solving a (mixed) integer optimization problem.

Default *"satisfied"*

Accepted *"ignored", "satisfied"*

Example M->setSolverParam("mioMode", "satisfied")

Generic name MSK_IPAR_MIO_MODE

Groups *Overall solver*

"mioNodeOptimizer"

Controls which optimizer is employed at the non-root nodes in the mixed-integer optimizer.

Default *"free"*

Accepted *"free", "intpnt", "conic", "primalSimplex", "dualSimplex", "freeSimplex", "mixedInt"*

Example M->setSolverParam("mioNodeOptimizer", "free")

Generic name MSK_IPAR_MIO_NODE_OPTIMIZER

Groups *Mixed-integer optimization*

"mioNodeSelection"

Controls the node selection strategy employed by the mixed-integer optimizer.

Default *"free"*
Accepted *"free", "first", "best", "pseudo"*
Example `M->setSolverParam("mioNodeSelection", "free")`
Generic name `MSK_IPAR_MIO_NODE_SELECTION`
Groups *Mixed-integer optimization*

"mioNumericalEmphasisLevel"

Controls how much emphasis is put on reducing numerical problems possibly at the expense of solution speed.

- 0. The optimizer chooses
- 1. More emphasis is put on reducing numerical problems
- 2. Even more emphasis

Default `0`
Accepted `[0; +2]`
Example `M->setSolverParam("mioNumericalEmphasisLevel", 0)`
Generic name `MSK_IPAR_MIO_NUMERICAL_EMPHASIS_LEVEL`
Groups *Mixed-integer optimization*

"mioPerspectiveReformulate"

Enables or disables perspective reformulation in presolve.

Default *"on"*
Accepted *"on", "off"*
Example `M->setSolverParam("mioPerspectiveReformulate", "on")`
Generic name `MSK_IPAR_MIO_PERSPECTIVE_REFORMULATE`
Groups *Mixed-integer optimization*

"mioPresolveAggregatorUse"

Controls if the aggregator should be used.

Default *"on"*
Accepted *"on", "off"*
Example `M->setSolverParam("mioPresolveAggregatorUse", "on")`
Generic name `MSK_IPAR_MIO_PRESOLVE_AGGREGATOR_USE`
Groups *Presolve*

"mioProbingLevel"

Controls the amount of probing employed by the mixed-integer optimizer in presolve.

- -1. The optimizer chooses the level of probing employed
- 0. Probing is disabled
- 1. A low amount of probing is employed
- 2. A medium amount of probing is employed
- 3. A high amount of probing is employed

Default `-1`
Accepted `[-1; 3]`
Example `M->setSolverParam("mioProbingLevel", -1)`
Generic name `MSK_IPAR_MIO_PROBING_LEVEL`
Groups *Mixed-integer optimization*

"mioPropagateObjectiveConstraint"

Use objective domain propagation.

Default *"off"*

Accepted *"on", "off"*

Example `M->setSolverParam("mioPropagateObjectiveConstraint", "off")`

Generic name `MSK_IPAR_MIO_PROPAGATE_OBJECTIVE_CONSTRAINT`

Groups *Mixed-integer optimization*

"mioQcqpReformulationMethod"

Controls what reformulation method is applied to mixed-integer quadratic problems.

Default *"free"*

Accepted *"free", "none", "linearization", "eigenValMethod", "diagSdp", "relaxSdp"*

Example `M->setSolverParam("mioQcqpReformulationMethod", "free")`

Generic name `MSK_IPAR_MIO_QCQP_REFORMULATION_METHOD`

Groups *Mixed-integer optimization*

"mioRinsMaxNodes"

Controls the maximum number of nodes allowed in each call to the RINS heuristic. The default value of -1 means that the value is determined automatically. A value of zero turns off the heuristic.

Default *-1*

Accepted *[-1; +inf]*

Example `M->setSolverParam("mioRinsMaxNodes", -1)`

Generic name `MSK_IPAR_MIO_RINS_MAX_NODES`

Groups *Mixed-integer optimization*

"mioRootOptimizer"

Controls which optimizer is employed at the root node in the mixed-integer optimizer.

Default *"free"*

Accepted *"free", "intpnt", "conic", "primalSimplex", "dualSimplex", "freeSimplex", "mixedInt"*

Example `M->setSolverParam("mioRootOptimizer", "free")`

Generic name `MSK_IPAR_MIO_ROOT_OPTIMIZER`

Groups *Mixed-integer optimization*

"mioRootRepeatPresolveLevel"

Controls whether presolve can be repeated at root node.

- -1. The optimizer chooses whether presolve is repeated
- 0. Never repeat presolve
- 1. Always repeat presolve

Default *-1*

Accepted *[-1; 1]*

Example `M->setSolverParam("mioRootRepeatPresolveLevel", -1)`

Generic name `MSK_IPAR_MIO_ROOT_REPEAT_PRESOLVE_LEVEL`

Groups *Mixed-integer optimization*

"mioSeed"

Sets the random seed used for randomization in the mixed integer optimizer. Selecting a different seed can change the path the optimizer takes to the optimal solution.

Default *42*

Accepted *[0; +inf]*

Example `M->setSolverParam("mioSeed", 42)`

Generic name `MSK_IPAR_MIO_SEED`

Groups *Mixed-integer optimization*

"mioSymmetryLevel"

Controls the amount of symmetry detection and handling employed by the mixed-integer optimizer in presolve.

- -1. The optimizer chooses the level of symmetry detection and handling employed
- 0. Symmetry detection and handling is disabled
- 1. A low amount of symmetry detection and handling is employed
- 2. A medium amount of symmetry detection and handling is employed
- 3. A high amount of symmetry detection and handling is employed
- 4. An extremely high amount of symmetry detection and handling is employed

Default -1

Accepted [-1; 4]

Example M->setSolverParam("mioSymmetryLevel", -1)

Generic name MSK_IPAR_MIO_SYMMETRY_LEVEL

Groups *Mixed-integer optimization*

"mioVbDetectionLevel"

Controls how much effort is put into detecting variable bounds.

- -1. The optimizer chooses
- 0. No variable bounds are detected
- 1. Only detect variable bounds that are directly represented in the problem
- 2. Detect variable bounds in probing

Default -1

Accepted [-1; +2]

Example M->setSolverParam("mioVbDetectionLevel", -1)

Generic name MSK_IPAR_MIO_VB_DETECTION_LEVEL

Groups *Mixed-integer optimization*

"mtSpincount"

Set the number of iterations to spin before sleeping.

Default 0

Accepted [0; 1000000000]

Example M->setSolverParam("mtSpincount", 0)

Generic name MSK_IPAR_MT_SPINCOUNT

Groups *Overall system*

"numThreads"

Controls the number of threads employed by the optimizer. If set to 0 the number of threads used will be equal to the number of cores detected on the machine.

Default 0

Accepted [0; +inf]

Example M->setSolverParam("numThreads", 0)

Generic name MSK_IPAR_NUM_THREADS

Groups *Overall system*

"optimizer"

The parameter controls which optimizer is used to optimize the task.

Default *"free"*

Accepted *"free", "intpnt", "conic", "primalSimplex", "dualSimplex", "freeSimplex", "mixedInt"*

Example M->setSolverParam("optimizer", "free")

Generic name MSK_IPAR_OPTIMIZER

Groups *Overall solver*

"presolveEliminatorMaxFill"

Controls the maximum amount of fill-in that can be created by one pivot in the elimination phase of the presolve. A negative value means the parameter value is selected automatically.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("presolveEliminatorMaxFill", -1)

Generic name MSK_IPAR_PRESOLVE_ELIMINATOR_MAX_FILL

Groups *Presolve*

"presolveEliminatorMaxNumTries"

Control the maximum number of times the eliminator is tried. A negative value implies **MOSEK** decides.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("presolveEliminatorMaxNumTries", -1)

Generic name MSK_IPAR_PRESOLVE_ELIMINATOR_MAX_NUM_TRIES

Groups *Presolve*

"presolveLevel"

Currently not used.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("presolveLevel", -1)

Generic name MSK_IPAR_PRESOLVE_LEVEL

Groups *Overall solver, Presolve*

"presolveLindepAbsWorkTrh"

Controls linear dependency check in presolve. The linear dependency check is potentially computationally expensive.

Default 100

Accepted [-inf; +inf]

Example M->setSolverParam("presolveLindepAbsWorkTrh", 100)

Generic name MSK_IPAR_PRESOLVE_LINDEP_ABS_WORK_TRH

Groups *Presolve*

"presolveLindepRelWorkTrh"

Controls linear dependency check in presolve. The linear dependency check is potentially computationally expensive.

Default 100

Accepted [-inf; +inf]

Example M->setSolverParam("presolveLindepRelWorkTrh", 100)

Generic name MSK_IPAR_PRESOLVE_LINDEP_REL_WORK_TRH

Groups *Presolve*

"presolveLindepUse"

Controls whether the linear constraints are checked for linear dependencies.

Default *"on"*

Accepted *"on", "off"*

Example M->setSolverParam("presolveLindepUse", "on")

Generic name MSK_IPAR_PRESOLVE_LINDEP_USE

Groups *Presolve*

"presolveMaxNumPass"

Control the maximum number of times presolve passes over the problem. A negative value implies MOSEK decides.

Default -1

Accepted [-inf; +inf]

Example M->setSolverParam("presolveMaxNumPass", -1)

Generic name MSK_IPAR_PREOLVE_MAX_NUM_PASS

Groups *Presolve*

"presolveUse"

Controls whether the presolve is applied to a problem before it is optimized.

Default "free"

Accepted "off", "on", "free"

Example M->setSolverParam("presolveUse", "free")

Generic name MSK_IPAR_PREOLVE_USE

Groups *Overall solver, Presolve*

"remoteUseCompression"

Use compression when sending data to an optimization server.

Default "zstd"

Accepted "none", "free", "gzip", "zstd"

Example M->setSolverParam("remoteUseCompression", "zstd")

Generic name MSK_IPAR_REMOTE_USE_COMPRESSION

"removeUnusedSolutions"

Removes unused solutions before the optimization is performed.

Default "off"

Accepted "on", "off"

Example M->setSolverParam("removeUnusedSolutions", "off")

Generic name MSK_IPAR_REMOVE_UNUSED_SOLUTIONS

Groups *Overall system*

"simBasisFactorUse"

Controls whether an LU factorization of the basis is used in a hot-start. Forcing a refactorization sometimes improves the stability of the simplex optimizers, but in most cases there is a performance penalty.

Default "on"

Accepted "on", "off"

Example M->setSolverParam("simBasisFactorUse", "on")

Generic name MSK_IPAR_SIM_BASIS_FACTOR_USE

Groups *Simplex optimizer*

"simDegen"

Controls how aggressively degeneration is handled.

Default "free"

Accepted "none", "free", "aggressive", "moderate", "minimum"

Example M->setSolverParam("simDegen", "free")

Generic name MSK_IPAR_SIM_DEGEN

Groups *Simplex optimizer*

"simDualCrash"

Controls whether crashing is performed in the dual simplex optimizer. If this parameter is set to x , then a crash will be performed if a basis consists of more than $(100 - x) \bmod f_v$ entries, where f_v is the number of fixed variables.

Default 90

Accepted [0; +inf]

Example M->setSolverParam("simDualCrash", 90)

Generic name MSK_IPAR_SIM_DUAL_CRASH

Groups *Dual simplex*

"simDualPhaseoneMethod"

An experimental feature.

Default 0

Accepted [0; 10]

Example M->setSolverParam("simDualPhaseoneMethod", 0)

Generic name MSK_IPAR_SIM_DUAL_PHASEONE_METHOD

Groups *Simplex optimizer*

"simDualRestrictSelection"

The dual simplex optimizer can use a so-called restricted selection/pricing strategy to choose the outgoing variable. Hence, if restricted selection is applied, then the dual simplex optimizer first choose a subset of all the potential outgoing variables. Next, for some time it will choose the outgoing variable only among the subset. From time to time the subset is redefined. A larger value of this parameter implies that the optimizer will be more aggressive in its restriction strategy, i.e. a value of 0 implies that the restriction strategy is not applied at all.

Default 50

Accepted [0; 100]

Example M->setSolverParam("simDualRestrictSelection", 50)

Generic name MSK_IPAR_SIM_DUAL_RESTRICT_SELECTION

Groups *Dual simplex*

"simDualSelection"

Controls the choice of the incoming variable, known as the selection strategy, in the dual simplex optimizer.

Default *"free"*

Accepted *"free", "full", "ase", "deve", "se", "partial"*

Example M->setSolverParam("simDualSelection", "free")

Generic name MSK_IPAR_SIM_DUAL_SELECTION

Groups *Dual simplex*

"simExploitDupvec"

Controls if the simplex optimizers are allowed to exploit duplicated columns.

Default *"off"*

Accepted *"on", "off", "free"*

Example M->setSolverParam("simExploitDupvec", "off")

Generic name MSK_IPAR_SIM_EXPLOIT_DUPVEC

Groups *Simplex optimizer*

"simHotstart"

Controls the type of hot-start that the simplex optimizer perform.

Default *"free"*

Accepted *"none", "free", "statusKeys"*

Example M->setSolverParam("simHotstart", "free")

Generic name MSK_IPAR_SIM_HOTSTART

Groups *Simplex optimizer*

"simHotstartLu"

Determines if the simplex optimizer should exploit the initial factorization.

Default *"on"*

Accepted

- *"on"*: Factorization is reused if possible.
- *"off"*: Factorization is recomputed.

Example M->setSolverParam("simHotstartLu", "on")

Generic name MSK_IPAR_SIM_HOTSTART_LU

Groups *Simplex optimizer*

"simMaxIterations"

Maximum number of iterations that can be used by a simplex optimizer.

Default 10000000

Accepted [0; +inf]

Example M->setSolverParam("simMaxIterations", 10000000)

Generic name MSK_IPAR_SIM_MAX_ITERATIONS

Groups *Simplex optimizer, Termination criteria*

"simMaxNumSetbacks"

Controls how many set-backs are allowed within a simplex optimizer. A set-back is an event where the optimizer moves in the wrong direction. This is impossible in theory but may happen due to numerical problems.

Default 250

Accepted [0; +inf]

Example M->setSolverParam("simMaxNumSetbacks", 250)

Generic name MSK_IPAR_SIM_MAX_NUM_SETBACKS

Groups *Simplex optimizer*

"simNonSingular"

Controls if the simplex optimizer ensures a non-singular basis, if possible.

Default *"on"*

Accepted *"on", "off"*

Example M->setSolverParam("simNonSingular", "on")

Generic name MSK_IPAR_SIM_NON_SINGULAR

Groups *Simplex optimizer*

"simPrimalCrash"

Controls whether crashing is performed in the primal simplex optimizer. In general, if a basis consists of more than (100-this parameter value)% fixed variables, then a crash will be performed.

Default 90

Accepted [0; +inf]

Example M->setSolverParam("simPrimalCrash", 90)

Generic name MSK_IPAR_SIM_PRIMAL_CRASH

Groups *Primal simplex*

"simPrimalPhaseoneMethod"

An experimental feature.

Default 0

Accepted [0; 10]

Example M->setSolverParam("simPrimalPhaseoneMethod", 0)

Generic name MSK_IPAR_SIM_PRIMAL_PHASEONE_METHOD

Groups *Simplex optimizer*

"simPrimalRestrictSelection"

The primal simplex optimizer can use a so-called restricted selection/pricing strategy to choose the outgoing variable. Hence, if restricted selection is applied, then the primal simplex optimizer first choose a subset of all the potential incoming variables. Next, for some time it will choose the incoming variable only among the subset. From time to time the subset is redefined. A larger value of this parameter implies that the optimizer will be more aggressive in its restriction strategy, i.e. a value of 0 implies that the restriction strategy is not applied at all.

Default 50

Accepted [0; 100]

Example M->setSolverParam("simPrimalRestrictSelection", 50)

Generic name MSK_IPAR_SIM_PRIMAL_RESTRICT_SELECTION

Groups *Primal simplex*

"simPrimalSelection"

Controls the choice of the incoming variable, known as the selection strategy, in the primal simplex optimizer.

Default *"free"*

Accepted *"free", "full", "ase", "devex", "se", "partial"*

Example M->setSolverParam("simPrimalSelection", "free")

Generic name MSK_IPAR_SIM_PRIMAL_SELECTION

Groups *Primal simplex*

"simRefactorFreq"

Controls how frequent the basis is refactorized. The value 0 means that the optimizer determines the best point of refactorization. It is strongly recommended NOT to change this parameter.

Default 0

Accepted [0; +inf]

Example M->setSolverParam("simRefactorFreq", 0)

Generic name MSK_IPAR_SIM_REFACTOR_FREQ

Groups *Simplex optimizer*

"simReformulation"

Controls if the simplex optimizers are allowed to reformulate the problem.

Default *"off"*

Accepted *"on", "off", "free", "aggressive"*

Example M->setSolverParam("simReformulation", "off")

Generic name MSK_IPAR_SIM_REFORMULATION

Groups *Simplex optimizer*

"simSaveLu"

Controls if the LU factorization stored should be replaced with the LU factorization corresponding to the initial basis.

Default *"off"*

Accepted *"on", "off"*

Example M->setSolverParam("simSaveLu", "off")

Generic name MSK_IPAR_SIM_SAVE_LU

Groups *Simplex optimizer*

"simScaling"

Controls how much effort is used in scaling the problem before a simplex optimizer is used.

Default *"free"*
Accepted *"free", "none"*
Example M->setSolverParam("simScaling", "free")
Generic name MSK_IPAR_SIM_SCALING
Groups *Simplex optimizer*

"simScalingMethod"

Controls how the problem is scaled before a simplex optimizer is used.

Default *"pow2"*
Accepted *"pow2", "free"*
Example M->setSolverParam("simScalingMethod", "pow2")
Generic name MSK_IPAR_SIM_SCALING_METHOD
Groups *Simplex optimizer*

"simSeed"

Sets the random seed used for randomization in the simplex optimizers.

Default 23456
Accepted [0; 32749]
Example M->setSolverParam("simSeed", 23456)
Generic name MSK_IPAR_SIM_SEED
Groups *Simplex optimizer*

"simSolveForm"

Controls whether the primal or the dual problem is solved by the primal-/dual-simplex optimizer.

Default *"free"*
Accepted *"free", "primal", "dual"*
Example M->setSolverParam("simSolveForm", "free")
Generic name MSK_IPAR_SIM_SOLVE_FORM
Groups *Simplex optimizer*

"simSwitchOptimizer"

The simplex optimizer sometimes chooses to solve the dual problem instead of the primal problem. This implies that if you have chosen to use the dual simplex optimizer and the problem is dualized, then it actually makes sense to use the primal simplex optimizer instead. If this parameter is on and the problem is dualized and furthermore the simplex optimizer is chosen to be the primal (dual) one, then it is switched to the dual (primal).

Default *"off"*
Accepted *"on", "off"*
Example M->setSolverParam("simSwitchOptimizer", "off")
Generic name MSK_IPAR_SIM_SWITCH_OPTIMIZER
Groups *Simplex optimizer*

"writeJsonIndentation"

When set, the JSON task and solution files are written with indentation for better readability.

Default *"off"*
Accepted *"on", "off"*
Example M->setSolverParam("writeJsonIndentation", "off")
Generic name MSK_IPAR_WRITE_JSON_INDENTATION
Groups *Data input/output*

"writeLpFullObj"

Write all variables, including the ones with 0-coefficients, in the objective.

Default *"on"*

Accepted *"on", "off"*

Example M->setSolverParam("writeLpFullObj", "on")

Generic name MSK_IPAR_WRITE_LP_FULL_OBJ

Groups *Data input/output*

"writeLpLineWidth"

Maximum width of line in an LP file written by **MOSEK**.

Default 80

Accepted [40; +inf]

Example M->setSolverParam("writeLpLineWidth", 80)

Generic name MSK_IPAR_WRITE_LP_LINE_WIDTH

Groups *Data input/output*

14.4.3 String parameters

"basSolFileName"

Name of the bas solution file.

Accepted Any valid file name.

Example M->setSolverParam("basSolFileName", "somevalue")

Generic name MSK_SPAR_BAS_SOL_FILE_NAME

Groups *Data input/output, Solution input/output*

"dataFileName"

Data are read and written to this file.

Accepted Any valid file name.

Example M->setSolverParam("dataFileName", "somevalue")

Generic name MSK_SPAR_DATA_FILE_NAME

Groups *Data input/output*

"intSolFileName"

Name of the int solution file.

Accepted Any valid file name.

Example M->setSolverParam("intSolFileName", "somevalue")

Generic name MSK_SPAR_INT_SOL_FILE_NAME

Groups *Data input/output, Solution input/output*

"itrSolFileName"

Name of the itr solution file.

Accepted Any valid file name.

Example M->setSolverParam("itrSolFileName", "somevalue")

Generic name MSK_SPAR_ITR_SOL_FILE_NAME

Groups *Data input/output, Solution input/output*

"remoteOptserverHost"

URL of the remote optimization server in the format (http|https)://server:port. If set, all subsequent calls to any **MOSEK** function that involves synchronous optimization will be sent to the specified OptServer instead of being executed locally. Passing empty string deactivates this redirection.

Accepted Any valid URL.

Example M->setSolverParam("remoteOptserverHost", "somevalue")

Generic name MSK_SPAR_REMOTE_OPTSERVER_HOST

Groups *Overall system*

"remoteTlsCert"

List of known server certificates in PEM format.

Accepted PEM files separated by new-lines.

Example M->setSolverParam("remoteTlsCert", "somevalue")

Generic name MSK_SPAR_REMOTE_TLS_CERT

Groups *Overall system*

"remoteTlsCertPath"

Path to known server certificates in PEM format.

Accepted Any valid path.

Example M->setSolverParam("remoteTlsCertPath", "somevalue")

Generic name MSK_SPAR_REMOTE_TLS_CERT_PATH

Groups *Overall system*

"writeLpGenVarName"

Sometimes when an LP file is written additional variables must be inserted. They will have the prefix denoted by this parameter.

Default xmskgen

Accepted Any valid string.

Example M->setSolverParam("writeLpGenVarName", "xmskgen")

Generic name MSK_SPAR_WRITE_LP_GEN_VAR_NAME

Groups *Data input/output*

14.5 Enumerations

AccSolutionStatus

Constants used for defining which solutions statuses are acceptable.

Anything

Accept all solution status except *SolutionStatus.Undefined*.

Optimal

Accept only optimal solution status.

Feasible

Accept any feasible solution, even if not optimal.

Certificate

Accept only a certificate.

ObjectiveSense

Used in *Model.objective* to define the objective sense of the *Model*.

Undefined

The sense is not defined; trying to optimize a *Model* whose objective sense is undefined is an error.

Minimize

Minimize the objective.

Maximize

Maximize the objective.

ProblemStatus

Constants defining the problem status.

Unknown

Unknown problem status.

PrimalAndDualFeasible

The problem is primal and dual feasible.

PrimalFeasible

The problem is at least primal feasible.

DualFeasible
The problem is at least least dual feasible.

PrimalInfeasible
The problem is primal infeasible.

DualInfeasible
The problem is dual infeasible.

PrimalAndDualInfeasible
The problem is primal and dual infeasible.

IllPosed
The problem is illposed.

PrimalInfeasibleOrUnbounded
The problem is primal infeasible or unbounded.

SolutionStatus
Defines properties of either a primal or a dual solution. A model may contain multiple solutions which may have different status. Specifically, there will be individual solutions, and thus solution statuses, for the interior-point, simplex and integer solvers.

Undefined
Undefined solution. This means that no values exist for the relevant solution.

Unknown
The solution status is unknown; this will happen if the user inputs values or a solution is read from a file or the solver stalled.

Optimal
The solution values are feasible and optimal.

Feasible
The solution is feasible.

Certificate
The solution is a certificate of infeasibility.

IllposedCert
The solution is a certificate of illposedness.

SolutionType
Used when requesting a specific solution from a *Model*.

Default
Auto-select the default solution; usually this will be the integer solution, if available, otherwise the basic solution, if available, otherwise the interior-point solution.

Basic
Select the basic solution.

Interior
Select the interior-point solution.

Integer
Select the integer solution.

SolverStatus
Constants used for reporting solver status from *Model.solveBatch*.

OK
No error.

Error
An error occurred.

LostRace
The model was not solved because it lost the race.

14.6 Constants

14.6.1 Basis identification

"never"
Never do basis identification.

"always"
Basis identification is always performed even if the interior-point optimizer terminates abnormally.

"noError"
Basis identification is performed if the interior-point optimizer terminates without an error.

"ifFeasible"
Basis identification is not performed if the interior-point optimizer terminates with a problem status saying that the problem is primal or dual infeasible.

"reserved"
Not currently in use.

14.6.2 Bound keys

"lo"
The constraint or variable has a finite lower bound and an infinite upper bound.

"up"
The constraint or variable has an infinite lower bound and a finite upper bound.

"fx"
The constraint or variable is fixed.

"fr"
The constraint or variable is free.

"ra"
The constraint or variable is ranged.

14.6.3 Mark

"lo"
The lower bound is selected for sensitivity analysis.

"up"
The upper bound is selected for sensitivity analysis.

14.6.4 Degeneracy strategies

"none"
The simplex optimizer should use no degeneracy strategy.

"free"
The simplex optimizer chooses the degeneracy strategy.

"aggressive"
The simplex optimizer should use an aggressive degeneracy strategy.

"moderate"
The simplex optimizer should use a moderate degeneracy strategy.

"minimum"
The simplex optimizer should use a minimum degeneracy strategy.

14.6.5 Transposed matrix.

"no"
No transpose is applied.
"yes"
A transpose is applied.

14.6.6 Triangular part of a symmetric matrix.

"lo"
Lower part.
"up"
Upper part.

14.6.7 Problem reformulation.

"on"
Allow the simplex optimizer to reformulate the problem.
"off"
Disallow the simplex optimizer to reformulate the problem.
"free"
The simplex optimizer can choose freely.
"aggressive"
The simplex optimizer should use an aggressive reformulation strategy.

14.6.8 Exploit duplicate columns.

"on"
Allow the simplex optimizer to exploit duplicated columns.
"off"
Disallow the simplex optimizer to exploit duplicated columns.
"free"
The simplex optimizer can choose freely.

14.6.9 Hot-start type employed by the simplex optimizer

"none"
The simplex optimizer performs a coldstart.
"free"
The simplex optimizer chooses the hot-start type.
"statusKeys"
Only the status keys of the constraints and variables are used to choose the type of hot-start.

14.6.10 Hot-start type employed by the interior-point optimizers.

"none"
The interior-point optimizer performs a coldstart.
"primal"
The interior-point optimizer exploits the primal solution only.
"dual"
The interior-point optimizer exploits the dual solution only.
"primalDual"
The interior-point optimizer exploits both the primal and dual solution.

14.6.11 Solution purification employed optimizer.

"none"
The optimizer performs no solution purification.

"primal"
The optimizer purifies the primal solution.

"dual"
The optimizer purifies the dual solution.

"primalDual"
The optimizer purifies both the primal and dual solution.

"auto"
TBD

14.6.12 Progress callback codes

"beginBi"
The basis identification procedure has been started.

"beginConic"
The callback function is called when the conic optimizer is started.

"beginDualBi"
The callback function is called from within the basis identification procedure when the dual phase is started.

"beginDualSensitivity"
Dual sensitivity analysis is started.

"beginDualSetupBi"
The callback function is called when the dual BI phase is started.

"beginDualSimplex"
The callback function is called when the dual simplex optimizer started.

"beginDualSimplexBi"
The callback function is called from within the basis identification procedure when the dual simplex clean-up phase is started.

"beginInfeasAna"
The callback function is called when the infeasibility analyzer is started.

"beginIntpnt"
The callback function is called when the interior-point optimizer is started.

"beginLicenseWait"
Begin waiting for license.

"beginMio"
The callback function is called when the mixed-integer optimizer is started.

"beginOptimizer"
The callback function is called when the optimizer is started.

"beginPresolve"
The callback function is called when the presolve is started.

"beginPrimalBi"
The callback function is called from within the basis identification procedure when the primal phase is started.

"beginPrimalRepair"
Begin primal feasibility repair.

"beginPrimalSensitivity"
Primal sensitivity analysis is started.

"beginPrimalSetupBi"
The callback function is called when the primal BI setup is started.

"beginPrimalSimplex"
The callback function is called when the primal simplex optimizer is started.

"beginPrimalSimplexBi"
The callback function is called from within the basis identification procedure when the primal simplex clean-up phase is started.

"beginQcqpReformulate"
 Begin QCQP reformulation.

"beginRead"
MOSEK has started reading a problem file.

"beginRootCutgen"
 The callback function is called when root cut generation is started.

"beginSimplex"
 The callback function is called when the simplex optimizer is started.

"beginSimplexBi"
 The callback function is called from within the basis identification procedure when the simplex clean-up phase is started.

"beginSolveRootRelax"
 The callback function is called when solution of root relaxation is started.

"beginToConic"
 Begin conic reformulation.

"beginWrite"
MOSEK has started writing a problem file.

"conic"
 The callback function is called from within the conic optimizer after the information database has been updated.

"dualSimplex"
 The callback function is called from within the dual simplex optimizer.

"endBi"
 The callback function is called when the basis identification procedure is terminated.

"endConic"
 The callback function is called when the conic optimizer is terminated.

"endDualBi"
 The callback function is called from within the basis identification procedure when the dual phase is terminated.

"endDualSensitivity"
 Dual sensitivity analysis is terminated.

"endDualSetupBi"
 The callback function is called when the dual BI phase is terminated.

"endDualSimplex"
 The callback function is called when the dual simplex optimizer is terminated.

"endDualSimplexBi"
 The callback function is called from within the basis identification procedure when the dual clean-up phase is terminated.

"endInfeasAna"
 The callback function is called when the infeasibility analyzer is terminated.

"endIntpnt"
 The callback function is called when the interior-point optimizer is terminated.

"endLicenseWait"
 End waiting for license.

"endMio"
 The callback function is called when the mixed-integer optimizer is terminated.

"endOptimizer"
 The callback function is called when the optimizer is terminated.

"endPresolve"
 The callback function is called when the presolve is completed.

"endPrimalBi"
 The callback function is called from within the basis identification procedure when the primal phase is terminated.

"endPrimalRepair"
 End primal feasibility repair.

"endPrimalSensitivity"
 Primal sensitivity analysis is terminated.

"endPrimalSetupBi"
The callback function is called when the primal BI setup is terminated.

"endPrimalSimplex"
The callback function is called when the primal simplex optimizer is terminated.

"endPrimalSimplexBi"
The callback function is called from within the basis identification procedure when the primal clean-up phase is terminated.

"endQcqrReformulate"
End QCQO reformulation.

"endRead"
MOSEK has finished reading a problem file.

"endRootCutgen"
The callback function is called when root cut generation is terminated.

"endSimplex"
The callback function is called when the simplex optimizer is terminated.

"endSimplexBi"
The callback function is called from within the basis identification procedure when the simplex clean-up phase is terminated.

"endSolveRootRelax"
The callback function is called when solution of root relaxation is terminated.

"endToConic"
End conic reformulation.

"endWrite"
MOSEK has finished writing a problem file.

"imBi"
The callback function is called from within the basis identification procedure at an intermediate point.

"imConic"
The callback function is called at an intermediate stage within the conic optimizer where the information database has not been updated.

"imDualBi"
The callback function is called from within the basis identification procedure at an intermediate point in the dual phase.

"imDualSensitivity"
The callback function is called at an intermediate stage of the dual sensitivity analysis.

"imDualSimplex"
The callback function is called at an intermediate point in the dual simplex optimizer.

"imIntpnt"
The callback function is called at an intermediate stage within the interior-point optimizer where the information database has not been updated.

"imLicenseWait"
MOSEK is waiting for a license.

"imLu"
The callback function is called from within the LU factorization procedure at an intermediate point.

"imMio"
The callback function is called at an intermediate point in the mixed-integer optimizer.

"imMioDualSimplex"
The callback function is called at an intermediate point in the mixed-integer optimizer while running the dual simplex optimizer.

"imMioIntpnt"
The callback function is called at an intermediate point in the mixed-integer optimizer while running the interior-point optimizer.

"imMioPrimalSimplex"
The callback function is called at an intermediate point in the mixed-integer optimizer while running the primal simplex optimizer.

"imOrder"
The callback function is called from within the matrix ordering procedure at an intermediate point.

"imPresolve"
The callback function is called from within the presolve procedure at an intermediate stage.

"imPrimalBi"
The callback function is called from within the basis identification procedure at an intermediate point in the primal phase.

"imPrimalSensitivity"
The callback function is called at an intermediate stage of the primal sensitivity analysis.

"imPrimalSimplex"
The callback function is called at an intermediate point in the primal simplex optimizer.

"imQoReformulate"
The callback function is called at an intermediate stage of the conic quadratic reformulation.

"imRead"
Intermediate stage in reading.

"imRootCutgen"
The callback is called from within root cut generation at an intermediate stage.

"imSimplex"
The callback function is called from within the simplex optimizer at an intermediate point.

"imSimplexBi"
The callback function is called from within the basis identification procedure at an intermediate point in the simplex clean-up phase. The frequency of the callbacks is controlled by the *logSimFreq* parameter.

"intpnt"
The callback function is called from within the interior-point optimizer after the information database has been updated.

"newIntMio"
The callback function is called after a new integer solution has been located by the mixed-integer optimizer.

"primalSimplex"
The callback function is called from within the primal simplex optimizer.

"readOpf"
The callback function is called from the OPF reader.

"readOpfSection"
A chunk of Q non-zeros has been read from a problem file.

"solvingRemote"
The callback function is called while the task is being solved on a remote server.

"updateDualBi"
The callback function is called from within the basis identification procedure at an intermediate point in the dual phase.

"updateDualSimplex"
The callback function is called in the dual simplex optimizer.

"updateDualSimplexBi"
The callback function is called from within the basis identification procedure at an intermediate point in the dual simplex clean-up phase. The frequency of the callbacks is controlled by the *logSimFreq* parameter.

"updatePresolve"
The callback function is called from within the presolve procedure.

"updatePrimalBi"
The callback function is called from within the basis identification procedure at an intermediate point in the primal phase.

"updatePrimalSimplex"
The callback function is called in the primal simplex optimizer.

"updatePrimalSimplexBi"
The callback function is called from within the basis identification procedure at an intermediate point in the primal simplex clean-up phase. The frequency of the callbacks is controlled by the *logSimFreq* parameter.

"updateSimplex"
The callback function is called from simplex optimizer.

"writeOpf"

The callback function is called from the OPF writer.

14.6.13 Types of convexity checks.

"none"

No convexity check.

"simple"

Perform simple and fast convexity check.

"full"

Perform a full convexity check.

14.6.14 Compression types

"none"

No compression is used.

"free"

The type of compression used is chosen automatically.

"gzip"

The type of compression used is gzip compatible.

"zstd"

The type of compression used is zstd compatible.

14.6.15 Cone types

"quad"

The cone is a quadratic cone.

"rquad"

The cone is a rotated quadratic cone.

"pexp"

A primal exponential cone.

"dexp"

A dual exponential cone.

"ppow"

A primal power cone.

"dpow"

A dual power cone.

"zero"

The zero cone.

14.6.16 Cone types

"r"

R.

"rzero"

The zero vector.

"rplus"

The positive orthant.

"rminus"

The negative orthant.

"quadraticCone"

The quadratic cone.

"rquadraticCone"

The rotated quadratic cone.

"primalExpCone"

The primal exponential cone.

"dualExpCone"
 The dual exponential cone.

"primalPowerCone"
 The primal power cone.

"dualPowerCone"
 The dual power cone.

"primalGeoMeanCone"
 The primal geometric mean cone.

"dualGeoMeanCone"
 The dual geometric mean cone.

"svecPsdCone"
 The vectorized positive semidefinite cone.

14.6.17 Name types

"gen"
 General names. However, no duplicate and blank names are allowed.

"mps"
 MPS type names.

"lp"
 LP type names.

14.6.18 Cone types

"sparse"
 Sparse symmetric matrix.

14.6.19 Data format types

"extension"
 The file extension is used to determine the data file format.

"mps"
 The data file is MPS formatted.

"lp"
 The data file is LP formatted.

"op"
 The data file is an optimization problem formatted file.

"freeMps"
 The data a free MPS formatted file.

"task"
 Generic task dump file.

"ptf"
 (P)retty (T)ext (F)format.

"cb"
 Conic benchmark format,

"jsonTask"
 JSON based task format.

14.6.20 Data format types

"extension"

The file extension is used to determine the data file format.

"b"

Simple binary format

"task"

Tar based format.

"jsonTask"

JSON based format.

14.6.21 Double information items

"anaProScalarizedConstraintMatrixDensity"

Density percentage of the scalarized constraint matrix.

"biCleanDualTime"

Time spent within the dual clean-up optimizer of the basis identification procedure since its invocation.

"biCleanPrimalTime"

Time spent within the primal clean-up optimizer of the basis identification procedure since its invocation.

"biCleanTime"

Time spent within the clean-up phase of the basis identification procedure since its invocation.

"biDualTime"

Time spent within the dual phase basis identification procedure since its invocation.

"biPrimalTime"

Time spent within the primal phase of the basis identification procedure since its invocation.

"biTime"

Time spent within the basis identification procedure since its invocation.

"intpntDualFeas"

Dual feasibility measure reported by the interior-point optimizer. (For the interior-point optimizer this measure is not directly related to the original problem because a homogeneous model is employed.)

"intpntDualObj"

Dual objective value reported by the interior-point optimizer.

"intpntFactorNumFlops"

An estimate of the number of flops used in the factorization.

"intpntOptStatus"

A measure of optimality of the solution. It should converge to +1 if the problem has a primal-dual optimal solution, and converge to -1 if the problem is (strictly) primal or dual infeasible. If the measure converges to another constant, or fails to settle, the problem is usually ill-posed.

"intpntOrderTime"

Order time (in seconds).

"intpntPrimalFeas"

Primal feasibility measure reported by the interior-point optimizer. (For the interior-point optimizer this measure is not directly related to the original problem because a homogeneous model is employed).

"intpntPrimalObj"

Primal objective value reported by the interior-point optimizer.

"intpntTime"

Time spent within the interior-point optimizer since its invocation.

"mioCliqueSeparationTime"

Separation time for clique cuts.

"mioCmirSeparationTime"

Separation time for CMIR cuts.

"mioConstructSolutionObj"

If **MOSEK** has successfully constructed an integer feasible solution, then this item contains the optimal objective value corresponding to the feasible solution.

"mioDualBoundAfterPresolve"
 Value of the dual bound after presolve but before cut generation.

"mioGmiSeparationTime"
 Separation time for GMI cuts.

"mioImpliedBoundTime"
 Separation time for implied bound cuts.

"mioInitialFeasibleSolutionObj"
 If the user provided solution was found to be feasible this information item contains it's objective value.

"mioKnapsackCoverSeparationTime"
 Separation time for knapsack cover.

"mioLiproSeparationTime"
 Separation time for lift-and-project cuts.

"mioObjAbsGap"
 Given the mixed-integer optimizer has computed a feasible solution and a bound on the optimal objective value, then this item contains the absolute gap defined by

$$|(\text{objective value of feasible solution}) - (\text{objective bound})|.$$

Otherwise it has the value -1.0.

"mioObjBound"
 The best known bound on the objective function. This value is undefined until at least one relaxation has been solved: To see if this is the case check that *"mioNumRelax"* is strictly positive.

"mioObjInt"
 The primal objective value corresponding to the best integer feasible solution. Please note that at least one integer feasible solution must have been located i.e. check *"mioNumIntSolutions"*.

"mioObjRelGap"
 Given that the mixed-integer optimizer has computed a feasible solution and a bound on the optimal objective value, then this item contains the relative gap defined by

$$\frac{|(\text{objective value of feasible solution}) - (\text{objective bound})|}{\max(\delta, |(\text{objective value of feasible solution})|)}.$$

where δ is given by the parameter *mioRelGapConst*. Otherwise it has the value -1.0 .

"mioProbingTime"
 Total time for probing.

"mioRootCutgenTime"
 Total time for cut generation.

"mioRootOptimizerTime"
 Time spent in the continuous optimizer while processing the root node relaxation.

"mioRootPresolveTime"
 Time spent presolving the problem at the root node.

"mioRootTime"
 Time spent processing the root node.

"mioTime"
 Time spent in the mixed-integer optimizer.

"mioUserObjCut"
 If the objective cut is used, then this information item has the value of the cut.

"optimizerTime"
 Total time spent in the optimizer since it was invoked.

"presolveEliTime"
 Total time spent in the eliminator since the presolve was invoked.

"presolveLindepTime"
 Total time spent in the linear dependency checker since the presolve was invoked.

"presolveTime"
 Total time (in seconds) spent in the presolve since it was invoked.

"presolveTotalPrimalPerturbation"
 Total perturbation of the bounds of the primal problem.

"primalRepairPenaltyObj"
The optimal objective value of the penalty function.

"qcqoReformulateMaxPerturbation"
Maximum absolute diagonal perturbation occurring during the QCQO reformulation.

"qcqoReformulateTime"
Time spent with conic quadratic reformulation.

"qcqoReformulateWorstCholeskyColumnScaling"
Worst Cholesky column scaling.

"qcqoReformulateWorstCholeskyDiagScaling"
Worst Cholesky diagonal scaling.

"readDataTime"
Time spent reading the data file.

"remoteTime"
The total real time in seconds spent when optimizing on a server by the process performing the optimization on the server

"simDualTime"
Time spent in the dual simplex optimizer since invoking it.

"simFeas"
Feasibility measure reported by the simplex optimizer.

"simObj"
Objective value reported by the simplex optimizer.

"simPrimalTime"
Time spent in the primal simplex optimizer since invoking it.

"simTime"
Time spent in the simplex optimizer since invoking it.

"solBasDualObj"
Dual objective value of the basic solution. Updated if *autoUpdateSolInfo* is set .

"solBasDviolcon"
Maximal dual bound violation for x^c in the basic solution. Updated if *autoUpdateSolInfo* is set .

"solBasDviolvar"
Maximal dual bound violation for x^x in the basic solution. Updated if *autoUpdateSolInfo* is set .

"solBasNrmBarx"
Infinity norm of $\bar{X}$ in the basic solution.

"solBasNrmSlc"
Infinity norm of s_l^c in the basic solution.

"solBasNrmSlx"
Infinity norm of s_l^x in the basic solution.

"solBasNrmSuc"
Infinity norm of s_u^c in the basic solution.

"solBasNrmSux"
Infinity norm of s_u^X in the basic solution.

"solBasNrmXc"
Infinity norm of x^c in the basic solution.

"solBasNrmXx"
Infinity norm of x^x in the basic solution.

"solBasNrmY"
Infinity norm of y in the basic solution.

"solBasPrimalObj"
Primal objective value of the basic solution. Updated if *autoUpdateSolInfo* is set .

"solBasPviolcon"
Maximal primal bound violation for x^c in the basic solution. Updated if *autoUpdateSolInfo* is set .

"solBasPviolvar"
Maximal primal bound violation for x^x in the basic solution. Updated if *autoUpdateSolInfo* is set .

"solItgNrmBarx"
Infinity norm of $\bar{X}$ in the integer solution.

"solItgNrmXc"
Infinity norm of x^c in the integer solution.

"solItgNrmXx"
Infinity norm of x^x in the integer solution.

"solItgPrimalObj"
Primal objective value of the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolacc"
Maximal primal violation for affine conic constraints in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolbarvar"
Maximal primal bound violation for $\overline{X}$ in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolcon"
Maximal primal bound violation for x^c in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolcones"
Maximal primal violation for primal conic constraints in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolajc"
Maximal primal violation for disjunctive constraints in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolitg"
Maximal violation for the integer constraints in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItgPviolvar"
Maximal primal bound violation for x^x in the integer solution. Updated if *autoUpdateSolInfo* is set .

"solItrDualObj"
Dual objective value of the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrDviolacc"
Maximal dual violation for the affine conic constraints in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrDviolbarvar"
Maximal dual bound violation for $\overline{X}$ in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrDviolcon"
Maximal dual bound violation for x^c in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrDviolcones"
Maximal dual violation for conic constraints in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrDviolvar"
Maximal dual bound violation for x^x in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrNrmBars"
Infinity norm of $\overline{S}$ in the interior-point solution.

"solItrNrmBarx"
Infinity norm of $\overline{X}$ in the interior-point solution.

"solItrNrmSlc"
Infinity norm of s_l^c in the interior-point solution.

"solItrNrmSlx"
Infinity norm of s_l^x in the interior-point solution.

"solItrNrmSnx"
Infinity norm of s_n^x in the interior-point solution.

"solItrNrmSuc"
Infinity norm of s_u^c in the interior-point solution.

"solItrNrmSux"
Infinity norm of s_u^X in the interior-point solution.

"solItrNrmXc"
Infinity norm of x^c in the interior-point solution.

"solItrNrmXx"
Infinity norm of x^x in the interior-point solution.

"solItrNrmY"
Infinity norm of y in the interior-point solution.

"solItrPrimalObj"
Primal objective value of the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrPviolacc"
Maximal primal violation for affine conic constraints in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrPviolbarvar"
Maximal primal bound violation for $\bar{X}$ in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrPviolcon"
Maximal primal bound violation for x^c in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrPviolcones"
Maximal primal violation for conic constraints in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"solItrPviolvar"
Maximal primal bound violation for x^x in the interior-point solution. Updated if *autoUpdateSolInfo* is set .

"toConicTime"
Time spent in the last to conic reformulation.

"writeDataTime"
Time spent writing the data file.

14.6.22 License feature

"pts"
Base system.

"pton"
Conic extension.

14.6.23 Long integer information items.

"anaProScalarizedConstraintMatrixNumColumns"
Number of columns in the scalarized constraint matrix.

"anaProScalarizedConstraintMatrixNumNz"
Number of non-zero entries in the scalarized constraint matrix.

"anaProScalarizedConstraintMatrixNumRows"
Number of rows in the scalarized constraint matrix.

"biCleanDualDegIter"
Number of dual degenerate clean iterations performed in the basis identification.

"biCleanDualIter"
Number of dual clean iterations performed in the basis identification.

"biCleanPrimalDegIter"
Number of primal degenerate clean iterations performed in the basis identification.

"biCleanPrimalIter"
Number of primal clean iterations performed in the basis identification.

"biDualIter"
Number of dual pivots performed in the basis identification.

"biPrimalIter"
Number of primal pivots performed in the basis identification.

"intpntFactorNumNz"
 Number of non-zeros in factorization.

"mioAnz"
 Number of non-zero entries in the constraint matrix of the problem to be solved by the mixed-integer optimizer.

"mioIntpntIter"
 Number of interior-point iterations performed by the mixed-integer optimizer.

"mioNumDualIllposedCer"
 Number of dual illposed certificates encountered by the mixed-integer optimizer.

"mioNumPrimIllposedCer"
 Number of primal illposed certificates encountered by the mixed-integer optimizer.

"mioPresolvedAnz"
 Number of non-zero entries in the constraint matrix of the problem after the mixed-integer optimizer's presolve.

"mioSimplexIter"
 Number of simplex iterations performed by the mixed-integer optimizer.

"rdNumacc"
 Number of affine conic constraints.

"rdNumanz"
 Number of non-zeros in A that is read.

"rdNumdjC"
 Number of disjunctive constraints.

"rdNumqnz"
 Number of Q non-zeros.

"simplexIter"
 Number of iterations performed by the simplex optimizer.

14.6.24 Integer information items.

"anaProNumCon"
 Number of constraints in the problem.

"anaProNumConEq"
 Number of equality constraints.

"anaProNumConFr"
 Number of unbounded constraints.

"anaProNumConLo"
 Number of constraints with a lower bound and an infinite upper bound.

"anaProNumConRa"
 Number of constraints with finite lower and upper bounds.

"anaProNumConUp"
 Number of constraints with an upper bound and an infinite lower bound.

"anaProNumVar"
 Number of variables in the problem.

"anaProNumVarBin"
 Number of binary (0-1) variables.

"anaProNumVarCont"
 Number of continuous variables.

"anaProNumVarEq"
 Number of fixed variables.

"anaProNumVarFr"
 Number of free variables.

"anaProNumVarInt"
 Number of general integer variables.

"anaProNumVarLo"
 Number of variables with a lower bound and an infinite upper bound.

"anaProNumVarRa"
 Number of variables with finite lower and upper bounds.

"anaProNumVarUp"
 Number of variables with an upper bound and an infinite lower bound.

"intpntFactorDimDense"
 Dimension of the dense sub system in factorization.

"intpntIter"
 Number of interior-point iterations since invoking the interior-point optimizer.

"intpntNumThreads"
 Number of threads that the interior-point optimizer is using.

"intpntSolveDual"
 Non-zero if the interior-point optimizer is solving the dual problem.

"mioAbsgapSatisfied"
 Non-zero if absolute gap is within tolerances.

"mioCliqueTableSize"
 Size of the clique table.

"mioConstructSolution"
 This item informs if **MOSEK** constructed an initial integer feasible solution.

- -1: tried, but failed,
- 0: no partial solution supplied by the user,
- 1: constructed feasible solution.

"mioInitialFeasibleSolution"
 This item informs if **MOSEK** found the solution provided by the user to be feasible

- 0: solution provided by the user was not found to be feasible for the current problem,
- 1: user provided solution was found to be feasible.

"mioNodeDepth"
 Depth of the last node solved.

"mioNumActiveNodes"
 Number of active branch and bound nodes.

"mioNumBranch"
 Number of branches performed during the optimization.

"mioNumCliqueCuts"
 Number of clique cuts.

"mioNumCmirCuts"
 Number of Complemented Mixed Integer Rounding (CMIR) cuts.

"mioNumGomoryCuts"
 Number of Gomory cuts.

"mioNumImpliedBoundCuts"
 Number of implied bound cuts.

"mioNumIntSolutions"
 Number of integer feasible solutions that have been found.

"mioNumKnapsackCoverCuts"
 Number of clique cuts.

"mioNumLiproCuts"
 Number of lift-and-project cuts.

"mioNumRelax"
 Number of relaxations solved during the optimization.

"mioNumRepeatedPresolve"
 Number of times presolve was repeated at root.

"mioNumbin"
 Number of binary variables in the problem to be solved by the mixed-integer optimizer.

"mioNumbinconevar"
 Number of binary cone variables in the problem to be solved by the mixed-integer optimizer.

"mioNumcon"
 Number of constraints in the problem to be solved by the mixed-integer optimizer.

"mioNumcone"
 Number of cones in the problem to be solved by the mixed-integer optimizer.

"mioNumconevar"
 Number of cone variables in the problem to be solved by the mixed-integer optimizer.

"mioNumcont"
 Number of continuous variables in the problem to be solved by the mixed-integer optimizer.

"mioNumcontconevar"
 Number of continuous cone variables in the problem to be solved by the mixed-integer optimizer.

"mioNumdexpcones"
 Number of dual exponential cones in the problem to be solved by the mixed-integer optimizer.

"mioNumdjc"
 Number of disjunctive constraints in the problem to be solved by the mixed-integer optimizer.

"mioNumdpowcones"
 Number of dual power cones in the problem to be solved by the mixed-integer optimizer.

"mioNumint"
 Number of integer variables in the problem to be solved by the mixed-integer optimizer.

"mioNumintconevar"
 Number of integer cone variables in the problem to be solved by the mixed-integer optimizer.

"mioNumpexpcones"
 Number of primal exponential cones in the problem to be solved by the mixed-integer optimizer.

"mioNumpowcones"
 Number of primal power cones in the problem to be solved by the mixed-integer optimizer.

"mioNumqcones"
 Number of quadratic cones in the problem to be solved by the mixed-integer optimizer.

"mioNumrqcones"
 Number of rotated quadratic cones in the problem to be solved by the mixed-integer optimizer.

"mioNumvar"
 Number of variables in the problem to be solved by the mixed-integer optimizer.

"mioObjBoundDefined"
 Non-zero if a valid objective bound has been found, otherwise zero.

"mioPresolvedNumbin"
 Number of binary variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumbinconevar"
 Number of binary cone variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumcon"
 Number of constraints in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumcone"
 Number of cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumconevar"
 Number of cone variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumcont"
 Number of continuous variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumcontconevar"
 Number of continuous cone variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumdexpcones"
 Number of dual exponential cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumdjc"
 Number of disjunctive constraints in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumdpowcones"
 Number of dual power cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumint"
 Number of integer variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumintconevar"
 Number of integer cone variables in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumpexpcones"
 Number of primal exponential cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumpowcones"
 Number of primal power cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumqcones"
 Number of quadratic cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumrqcones"
 Number of rotated quadratic cones in the problem after the mixed-integer optimizer's presolve.

"mioPresolvedNumvar"
 Number of variables in the problem after the mixed-integer optimizer's presolve.

"mioRelgapSatisfied"
 Non-zero if relative gap is within tolerances.

"mioTotalNumCuts"
 Total number of cuts generated by the mixed-integer optimizer.

"mioUserObjCut"
 If it is non-zero, then the objective cut is used.

"optNumcon"
 Number of constraints in the problem solved when the optimizer is called.

"optNumvar"
 Number of variables in the problem solved when the optimizer is called

"optimizeResponse"
 The response code returned by optimize.

"presolveNumPrimalPerturbations"
 Number perturbations to thhe bounds of the primal problem.

"purifyDualSuccess"
 Is nonzero if the dual solution is purified.

"purifyPrimalSuccess"
 Is nonzero if the primal solution is purified.

"rdNumbarvar"
 Number of symmetric variables read.

"rdNumcon"
 Number of constraints read.

"rdNumcone"
 Number of conic constraints read.

"rdNumintvar"
 Number of integer-constrained variables read.

"rdNumq"
 Number of nonempty Q matrices read.

"rdNumvar"
 Number of variables read.

"rdPrototype"
 Problem type.

"simDualDegIter"
 The number of dual degenerate iterations.

"simDualHotstart"
 If 1 then the dual simplex algorithm is solving from an advanced basis.

"simDualHotstartLu"
 If 1 then a valid basis factorization of full rank was located and used by the dual simplex algorithm.

"simDualInfIter"
 The number of iterations taken with dual infeasibility.

"simDualIter"
 Number of dual simplex iterations during the last optimization.

"simNumcon"
 Number of constraints in the problem solved by the simplex optimizer.

"simNumvar"
 Number of variables in the problem solved by the simplex optimizer.

"simPrimalDegIter"
 The number of primal degenerate iterations.

"simPrimalHotstart"
 If 1 then the primal simplex algorithm is solving from an advanced basis.

"simPrimalHotstartLu"
 If 1 then a valid basis factorization of full rank was located and used by the primal simplex algorithm.

"simPrimalInfIter"
The number of iterations taken with primal infeasibility.

"simPrimalIter"
Number of primal simplex iterations during the last optimization.

"simSolveDual"
Is non-zero if dual problem is solved.

"solBasProsta"
Problem status of the basic solution. Updated after each optimization.

"solBasSolsta"
Solution status of the basic solution. Updated after each optimization.

"solItgProsta"
Problem status of the integer solution. Updated after each optimization.

"solItgSolsta"
Solution status of the integer solution. Updated after each optimization.

"solItrProsta"
Problem status of the interior-point solution. Updated after each optimization.

"solItrSolsta"
Solution status of the interior-point solution. Updated after each optimization.

"stoNumARealloc"
Number of times the storage for storing A has been changed. A large value may indicate that memory fragmentation may occur.

14.6.25 Information item types

"douType"
Is a double information type.

"intType"
Is an integer.

"lintType"
Is a long integer.

14.6.26 Input/output modes

"read"
The file is read-only.

"write"
The file is write-only. If the file exists then it is truncated when it is opened. Otherwise it is created when it is opened.

"readwrite"
The file is to read and write.

14.6.27 Specifies the branching direction.

"free"
The mixed-integer optimizer decides which branch to choose.

"up"
The mixed-integer optimizer always chooses the up branch first.

"down"
The mixed-integer optimizer always chooses the down branch first.

"near"
Branch in direction nearest to selected fractional variable.

"far"
Branch in direction farthest from selected fractional variable.

"rootLp"
Chose direction based on root lp value of selected variable.

"guided"
Branch in direction of current incumbent.

"pseudocost"

Branch based on the pseudocost of the variable.

14.6.28 Specifies the reformulation method for mixed-integer quadratic problems.

"free"

The mixed-integer optimizer decides which reformulation method to apply.

"none"

No reformulation method is applied.

"linearization"

A reformulation via linearization is applied.

"eigenValMethod"

The eigenvalue method is applied.

"diagSdp"

A perturbation of matrix diagonals via the solution of SDPs is applied.

"relaxSdp"

A Reformulation based on the solution of an SDP-relaxation of the problem is applied.

14.6.29 Specifies the problem data permutation method for mixed-integer problems.

"none"

No problem data permutation is applied.

"cyclicShift"

A random cyclic shift is applied to permute the problem data.

"random"

A random permutation is applied to the problem data.

14.6.30 Continuous mixed-integer solution type

"none"

No interior-point or basic solution are reported when the mixed-integer optimizer is used.

"root"

The reported interior-point and basic solutions are a solution to the root node problem when mixed-integer optimizer is used.

"itg"

The reported interior-point and basic solutions are a solution to the problem with all integer variables fixed at the value they have in the integer solution. A solution is only reported in case the problem has a primal feasible solution.

"itgRel"

In case the problem is primal feasible then the reported interior-point and basic solutions are a solution to the problem with all integer variables fixed at the value they have in the integer solution. If the problem is primal infeasible, then the solution to the root node problem is reported.

14.6.31 Integer restrictions

"ignored"

The integer constraints are ignored and the problem is solved as a continuous problem.

"satisfied"

Integer restrictions should be satisfied.

14.6.32 Mixed-integer node selection types

"free"

The optimizer decides the node selection strategy.

"first"

The optimizer employs a depth first node selection strategy.

"best"

The optimizer employs a best bound node selection strategy.

"pseudo"

The optimizer employs selects the node based on a pseudo cost estimate.

14.6.33 MPS file format type

"strict"

It is assumed that the input file satisfies the MPS format strictly.

"relaxed"

It is assumed that the input file satisfies a slightly relaxed version of the MPS format.

"free"

It is assumed that the input file satisfies the free MPS format. This implies that spaces are not allowed in names. Otherwise the format is free.

"cplex"

The CPLEX compatible version of the MPS format is employed.

14.6.34 Objective sense types

"minimize"

The problem should be minimized.

"maximize"

The problem should be maximized.

14.6.35 On/off

"on"

Switch the option on.

"off"

Switch the option off.

14.6.36 Optimizer types

"conic"

The optimizer for problems having conic constraints.

"dualSimplex"

The dual simplex optimizer is used.

"free"

The optimizer is chosen automatically.

"freeSimplex"

One of the simplex optimizers is used.

"intpnt"

The interior-point optimizer is used.

"mixedInt"

The mixed-integer optimizer.

"primalSimplex"

The primal simplex optimizer is used.

14.6.37 Ordering strategies

"free"

The ordering method is chosen automatically.

"appminloc"

Approximate minimum local fill-in ordering is employed.

"experimental"

This option should not be used.

"tryGraphpar"

Always try the graph partitioning based ordering.

"forceGraphpar"

Always use the graph partitioning based ordering even if it is worse than the approximate minimum local fill ordering.

"none"

No ordering is used.

14.6.38 Presolve method.

"off"

The problem is not presolved before it is optimized.

"on"

The problem is presolved before it is optimized.

"free"

It is decided automatically whether to presolve before the problem is optimized.

14.6.39 Parameter type

"invalidType"

Not a valid parameter.

"douType"

Is a double parameter.

"intType"

Is an integer parameter.

"strType"

Is a string parameter.

14.6.40 Problem data items

"var"

Item is a variable.

"con"

Item is a constraint.

"cone"

Item is a cone.

14.6.41 Problem types

"lo"

The problem is a linear optimization problem.

"qo"

The problem is a quadratic optimization problem.

"qcqo"

The problem is a quadratically constrained optimization problem.

"conic"

A conic optimization.

"mixed"

General nonlinear constraints and conic constraints. This combination can not be solved by **MOSEK**.

14.6.42 Problem status keys

"unknown"	Unknown problem status.
"primAndDualFeas"	The problem is primal and dual feasible.
"primFeas"	The problem is primal feasible.
"dualFeas"	The problem is dual feasible.
"primInfeas"	The problem is primal infeasible.
"dualInfeas"	The problem is dual infeasible.
"primAndDualInfeas"	The problem is primal and dual infeasible.
"illPosed"	The problem is ill-posed. For example, it may be primal and dual feasible but have a positive duality gap.
"primInfeasOrUnbounded"	The problem is either primal infeasible or unbounded. This may occur for mixed-integer problems.

14.6.43 XML writer output mode

"row"	Write in row order.
"col"	Write in column order.

14.6.44 Response code type

"ok"	The response code is OK.
"wrn"	The response code is a warning.
"trm"	The response code is an optimizer termination status.
"err"	The response code is an error.
"unk"	The response code does not belong to any class.

14.6.45 Scaling type

"free"	The optimizer chooses the scaling heuristic.
"none"	No scaling is performed.

14.6.46 Scaling method

"pow2"

Scales only with power of 2 leaving the mantissa untouched.

"free"

The optimizer chooses the scaling heuristic.

14.6.47 Sensitivity types

"basis"

Basis sensitivity analysis is performed.

14.6.48 Simplex selection strategy

"free"

The optimizer chooses the pricing strategy.

"full"

The optimizer uses full pricing.

"ase"

The optimizer uses approximate steepest-edge pricing.

"devex"

The optimizer uses devex steepest-edge pricing (or if it is not available an approximate steep-edge selection).

"se"

The optimizer uses steepest-edge selection (or if it is not available an approximate steep-edge selection).

"partial"

The optimizer uses a partial selection approach. The approach is usually beneficial if the number of variables is much larger than the number of constraints.

14.6.49 Solution items

"xc"

Solution for the constraints.

"xx"

Variable solution.

"y"

Lagrange multipliers for equations.

"slc"

Lagrange multipliers for lower bounds on the constraints.

"suc"

Lagrange multipliers for upper bounds on the constraints.

"slx"

Lagrange multipliers for lower bounds on the variables.

"sux"

Lagrange multipliers for upper bounds on the variables.

"snx"

Lagrange multipliers corresponding to the conic constraints on the variables.

14.6.50 Solution status keys

"unknown"
Status of the solution is unknown.

"optimal"
The solution is optimal.

"primFeas"
The solution is primal feasible.

"dualFeas"
The solution is dual feasible.

"primAndDualFeas"
The solution is both primal and dual feasible.

"primInfeasCer"
The solution is a certificate of primal infeasibility.

"dualInfeasCer"
The solution is a certificate of dual infeasibility.

"primIllposedCer"
The solution is a certificate that the primal problem is illposed.

"dualIllposedCer"
The solution is a certificate that the dual problem is illposed.

"integerOptimal"
The primal solution is integer optimal.

14.6.51 Solution types

"bas"
The basic solution.

"itr"
The interior solution.

"itg"
The integer solution.

14.6.52 Solve primal or dual form

"free"
The optimizer is free to solve either the primal or the dual problem.

"primal"
The optimizer should solve the primal problem.

"dual"
The optimizer should solve the dual problem.

14.6.53 Status keys

"unk"
The status for the constraint or variable is unknown.

"bas"
The constraint or variable is in the basis.

"supbas"
The constraint or variable is super basic.

"low"
The constraint or variable is at its lower bound.

"upr"
The constraint or variable is at its upper bound.

"fix"
The constraint or variable is fixed.

"inf"
The constraint or variable is infeasible in the bounds.

14.6.54 Starting point types

"free"

The starting point is chosen automatically.

"guess"

The optimizer guesses a starting point.

"constant"

The optimizer constructs a starting point by assigning a constant value to all primal and dual variables. This starting point is normally robust.

"satisfyBounds"

The starting point is chosen to satisfy all the simple bounds on nonlinear variables. If this starting point is employed, then more care than usual should be employed when choosing the bounds on the nonlinear variables. In particular very tight bounds should be avoided.

14.6.55 Stream types

"log"

Log stream. Contains the aggregated contents of all other streams. This means that a message written to any other stream will also be written to this stream.

"msg"

Message stream. Log information relating to performance and progress of the optimization is written to this stream.

"err"

Error stream. Error messages are written to this stream.

"wrn"

Warning stream. Warning messages are written to this stream.

14.6.56 Integer values

"maxStrLen"

Maximum string length allowed in **MOSEK**.

"licenseBufferLength"

The length of a license key buffer.

14.6.57 Variable types

"typeCont"

Is a continuous variable.

"typeInt"

Is an integer variable.

14.7 Exceptions

- *DeletionError*: This item cannot be removed.
- *DimensionError*: Thrown when a given object has the wrong number of dimensions, or they have not the right size.
- *DomainError*: Invalid domain.
- *ExpressionError*: Tried to construct an expression from invalid.
- *FatalError*: A fatal error has happened.
- *FusionException*: Base class for all normal exceptions in fusion.
- *FusionRuntimeException*: Base class for all run-time exceptions in fusion.
- *IOError*: Error when reading or writing a stream, or opening a file.
- *IndexError*: Index out of bound, or a multi-dimensional index had wrong number of dimensions.

- *LengthError*: An array did not have the required length, or two arrays were expected to have same length.
- *MatrixError*: Thrown if data used in construction of a matrix contained inconsistencies or errors.
- *ModelError*: Thrown when objects from different models were mixed.
- *NameError*: Name clash; tries to add a variable or constraint with a name that already exists.
- *OptimizeError*: An error occurred during optimization.
- *ParameterError*: Tried to use an invalid parameter for a value that was invalid for a specific parameter.
- *RangeError*: Invalid range specified
- *SetDefinitionError*: Invalid data for constructing set.
- *SliceError*: Invalid slice definition, negative slice or slice index out of bounds.
- *SolutionError*: Requested a solution that was undefined or whose status was not acceptable.
- *SparseFormatError*: The given sparsity patterns was invalid or specified an index that was out of bounds.
- *UnexpectedError*: An unexpected error has happened. No specific exception could have been risen.
- *UnimplementedError*: Called a stub. Functionality has not yet been implemented.
- *UpdateError*: Invalid slice definition, negative slice or slice index out of bounds.
- *ValueConversionError*: Error casting or converting a value.

14.7.1 Exception DeletionError

`mosek::fusion::DeletionError`

This item cannot be removed.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.2 Exception DimensionError

`mosek::fusion::DimensionError`

Thrown when a given object has the wrong number of dimensions, or they have not the right size.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.3 Exception DomainError

`mosek::fusion::DomainError`

Invalid domain.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.4 Exception ExpressionError

mosek::fusion::ExpressionError

Tried to construct an expression from invalid.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.5 Exception FatalError

mosek::fusion::FatalError

A fatal error has happened.

Implements *RuntimeException*

Members *RuntimeException.toString* – Return the exception message.

14.7.6 Exception FusionException

mosek::fusion::FusionException

Base class for all normal exceptions in fusion.

Implements *Exception*

Members *FusionException.toString* – Return the exception message.

Implemented by *SolutionError*

FusionException.toString

```
string toString()
```

Return the exception message.

Return (string)

14.7.7 Exception FusionRuntimeException

mosek::fusion::FusionRuntimeException

Base class for all run-time exceptions in fusion.

Implements *RuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

Implemented by *IOError*, *ExpressionError*, *ParameterError*,
ValueConversionError, *DomainError*, *IndexError*, *RangeError*,
LengthError, *DimensionError*, *MatrixError*, *ModelError*,
DeletionError, *NameError*, *OptimizeError*, *SetDefinitionError*,
UpdateError, *SliceError*, *SparseFormatError*

FusionRuntimeException.toString

```
string toString()
```

Return the exception message.

Return (string)

14.7.8 Exception IOError

`mosek::fusion::IOError`

Error when reading or writing a stream, or opening a file.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.9 Exception IndexError

`mosek::fusion::IndexError`

Index out of bound, or a multi-dimensional index had wrong number of dimensions.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.10 Exception LengthError

`mosek::fusion::LengthError`

An array did not have the required length, or two arrays were expected to have same length.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.11 Exception MatrixError

`mosek::fusion::MatrixError`

Thrown if data used in construction of a matrix contained inconsistencies or errors.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.12 Exception ModelError

`mosek::fusion::ModelError`

Thrown when objects from different models were mixed.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.13 Exception NameError

`mosek::fusion::NameError`

Name clash; tries to add a variable or constraint with a name that already exists.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.14 Exception OptimizeError

`mosek::fusion::OptimizeError`

An error occurred during optimization.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.15 Exception ParameterError

`mosek::fusion::ParameterError`

Tried to use an invalid parameter for a value that was invalid for a specific parameter.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.16 Exception RangeError

`mosek::fusion::RangeError`

Invalid range specified

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.17 Exception SetDefinitionError

`mosek::fusion::SetDefinitionError`

Invalid data for constructing set.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.18 Exception SliceError

`mosek::fusion::SliceError`

Invalid slice definition, negative slice or slice index out of bounds.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.19 Exception SolutionError

`mosek::fusion::SolutionError`

Requested a solution that was undefined or whose status was not acceptable.

Implements *FusionException*

Members *FusionException.toString* – Return the exception message.

14.7.20 Exception SparseFormatError

`mosek::fusion::SparseFormatError`

The given sparsity patterns was invalid or specified an index that was out of bounds.

Implements *FusionRuntimeException*

Members *FusionRuntimeException.toString* – Return the exception message.

14.7.21 Exception UnexpectedError

`mosek::fusion::UnexpectedError`

An unexpected error has happened. No specific exception could have been risen.

Implements *RuntimeException*

Members *RuntimeException.toString* – Return the exception message.

14.7.22 Exception UnimplementedError

`mosek::fusion::UnimplementedError`

Called a stub. Functionality has not yet been implemented.

Implements `RuntimeException`

Members `RuntimeException.toString` – Return the exception message.

14.7.23 Exception UpdateError

`mosek::fusion::UpdateError`

Invalid slice definition, negative slice or slice index out of bounds.

Implements `FusionRuntimeException`

Members `FusionRuntimeException.toString` – Return the exception message.

14.7.24 Exception ValueConversionError

`mosek::fusion::ValueConversionError`

Error casting or converting a value.

Implements `FusionRuntimeException`

Members `FusionRuntimeException.toString` – Return the exception message.

14.8 Supported domains

This section lists the domains supported by **MOSEK**. See [Sec. 7](#) for how to apply domains to specify conic constraints and disjunctive constraints (DJs).

14.8.1 Affine domains

- `Domain.equalsTo`: the **fixed domain** consisting of a single point,
- `Domain.lessThan`: the **upper-bounded domain** specified by an upper bound in each dimension,
- `Domain.greaterThan`: the **lower-bounded domain** specified by a lower bound in each dimension,
- `Domain.inRange`: the **ranged domain** specified by an interval in each dimension,
- `Domain.unbounded`: the **unbounded domain** $\mathbb{R}$.

Membership in an affine domain imposes linear constraints in the model. The unbounded domain imposes no restriction.

14.8.2 Quadratic cone domains

The quadratic domains are determined by the dimension n .

- `Domain.inQCone`: the **quadratic cone domain** is the subset of $\mathbb{R}^n$ defined as

$$\mathcal{Q}^n = \left\{ x \in \mathbb{R}^n : x_1 \geq \sqrt{x_2^2 + \cdots + x_n^2} \right\}.$$

- `Domain.inRotatedQCone`: the **rotated quadratic cone domain** is the subset of $\mathbb{R}^n$ defined as

$$\mathcal{Q}_r^n = \left\{ x \in \mathbb{R}^n : 2x_1x_2 \geq x_3^2 + \cdots + x_n^2, x_1, x_2 \geq 0 \right\}.$$

14.8.3 Exponential cone domains

- *Domain.inPExpCone*: the **primal exponential cone domain** is the subset of $\mathbb{R}^3$ defined as

$$K_{\text{exp}} = \{(x_1, x_2, x_3) \in \mathbb{R}^3 : x_1 \geq x_2 \exp(x_3/x_2), x_1, x_2 \geq 0\}.$$

- *Domain.inDExpCone*: the **dual exponential cone domain** is the subset of $\mathbb{R}^3$ defined as

$$K_{\text{exp}}^* = \{(x_1, x_2, x_3) \in \mathbb{R}^3 : x_1 \leq -x_3 \exp(x_2/x_3 - 1), x_1 \geq 0, x_3 \leq 0\}.$$

14.8.4 Power cone domains

A power cone domain is determined by the dimension n and a sequence of $1 \leq n_l < n$ positive real numbers (weights) $\alpha_1, \dots, \alpha_{n_l}$.

- *Domain.inPPowerCone*: the **primal power cone domain** is the subset of $\mathbb{R}^n$ defined as

$$\mathcal{P}_n^{(\alpha_1, \dots, \alpha_{n_l})} = \left\{ x \in \mathbb{R}^n : \prod_{i=1}^{n_l} x_i^{\beta_i} \geq \sqrt{x_{n_l+1}^2 + \dots + x_n^2}, x_1, \dots, x_{n_l} \geq 0 \right\}.$$

where β_i are the weights normalized to add up to 1, ie. $\beta_i = \alpha_i / (\sum_j \alpha_j)$ for $i = 1, \dots, n_l$. The name n_l reads as “n left”, the length of the product on the left-hand side of the definition.

- *Domain.inDPowerCone*: the **dual power cone domain** is the subset of $\mathbb{R}^n$ defined as

$$\left(\mathcal{P}_n^{(\alpha_1, \dots, \alpha_{n_l})} \right)^* = \left\{ x \in \mathbb{R}^n : \prod_{i=1}^{n_l} \left(\frac{x_i}{\beta_i} \right)^{\beta_i} \geq \sqrt{x_{n_l+1}^2 + \dots + x_n^2}, x_1, \dots, x_{n_l} \geq 0 \right\}.$$

where β_i are the weights normalized to add up to 1, ie. $\beta_i = \alpha_i / (\sum_j \alpha_j)$ for $i = 1, \dots, n_l$. The name n_l reads as “n left”, the length of the product on the left-hand side of the definition.

- **Remark:** in MOSEK 9 power cones were available only in the special case with $n_l = 2$ and weights $(\alpha, 1 - \alpha)$ for some $0 < \alpha < 1$ specified as cone parameter.

14.8.5 Geometric mean cone domains

A geometric mean cone domain is determined by the dimension n .

- *Domain.inPGeoMeanCone*: the **primal geometric mean cone domain** is the subset of $\mathbb{R}^n$ defined as

$$\mathcal{GM}^n = \left\{ x \in \mathbb{R}^n : \left(\prod_{i=1}^{n-1} x_i \right)^{1/(n-1)} \geq |x_n|, x_1, \dots, x_{n-1} \geq 0 \right\}.$$

It is a special case of the primal power cone domain with $n_l = n - 1$ and weights $\alpha = (1, \dots, 1)$.

- *Domain.inDGeoMeanCone*: the **dual geometric mean cone domain** is the subset of $\mathbb{R}^n$ defined as

$$(\mathcal{GM}^n)^* = \left\{ x \in \mathbb{R}^n : (n-1) \left(\prod_{i=1}^{n-1} x_i \right)^{1/(n-1)} \geq |x_n|, x_1, \dots, x_{n-1} \geq 0 \right\}.$$

It is a special case of the dual power cone domain with $n_l = n - 1$ and weights $\alpha = (1, \dots, 1)$.

14.8.6 Positive semidefinite cone domain

- *Domain.inPSDCone* is the domain $\mathcal{S}_+^d$ of symmetric positive-semidefinite variables of a given dimension d . It can only be applied to objects of shape (d, d) .

14.9 Class LinAlg

mosek::LinAlg

BLAS/LAPACK linear algebra routines.

Static members *LinAlg.axy* – Computes vector addition and multiplication by a scalar.

LinAlg.dot – Computes the inner product of two vectors.

LinAlg.gemm – Performs a dense matrix multiplication.

LinAlg.gemv – Computes dense matrix times a dense vector product.

LinAlg.potrf – Computes a Cholesky factorization of a dense matrix.

LinAlg.syeig – Computes all eigenvalues of a symmetric dense matrix.

LinAlg.syevd – Computes all the eigenvalues and eigenvectors of a symmetric dense matrix, and thus its eigenvalue decomposition.

LinAlg.syrk – Performs a rank-k update of a symmetric matrix.

LinAlg.axy

```
void LinAlg::axy(int n, double alpha, shared_ptr<ndarray<double,1>> x, shared_ptr<ndarray<double,1>> y)
```

Adds αx to y , i.e. performs the update

$$y := \alpha x + y.$$

Note that the result is stored overwriting y . It must not overlap with the other input arrays.

Parameters

- **n** (int) – Length of the vectors.
- **alpha** (double) – The scalar that multiplies x .
- **x** (double[]) – The x vector.
- **y** (double[]) – The y vector.

LinAlg.dot

```
double LinAlg::dot(int n, shared_ptr<ndarray<double,1>> x, shared_ptr<ndarray<double,1>> y)
```

Computes the inner product of two vectors x, y of length $n \geq 0$, i.e

$$x \cdot y = \sum_{i=1}^n x_i y_i.$$

Note that if $n = 0$, then the result of the operation is 0.

Parameters

- **n** (int) – Length of the vectors.
- **x** (double[]) – The x vector.
- **y** (double[]) – The y vector.

Return (double)

LinAlg.gemm


```
void LinAlg::gemm(mosek.transpose transa, mosek.transpose transb, int m, int n,
→int k, double alpha, shared_ptr<ndarray<double,1>> a, shared_ptr<ndarray
→<double,1>> b, double beta, shared_ptr<ndarray<double,1>> c)
```

Performs a matrix multiplication plus addition of dense matrices. Given A , B and C of compatible dimensions, this function computes

$$C := \alpha op(A)op(B) + \beta C$$

where α, β are two scalar values. The function $op(X)$ denotes X if `transX` is `NO`, or X^T if set to `YES`. The matrix C has m rows and n columns, and the other matrices must have compatible dimensions.

The result of this operation is stored in C . It must not overlap with the other input arrays.

Parameters

- **transa (transpose)** – Indicates if A should be transposed. See the Optimizer API documentation for the definition of these constants.
- **transb (transpose)** – Indicates if B should be transposed. See the Optimizer API documentation for the definition of these constants.
- **m (int)** – Indicates the number of rows of matrix C .
- **n (int)** – Indicates the number of columns of matrix C .
- **k (int)** – Specifies the common dimension along which $op(A)$ and $op(B)$ are multiplied. For example, if neither A nor B are transposed, then this is the number of columns in A and also the number of rows in B .
- **alpha (double)** – A scalar value multiplying the result of the matrix multiplication.
- **a (double[])** – The pointer to the array storing matrix A in a column-major format.
- **b (double[])** – The pointer to the array storing matrix B in a column-major format.
- **beta (double)** – A scalar value that multiplies C .
- **c (double[])** – The pointer to the array storing matrix C in a column-major format.

`LinAlg.gemv`

```
void LinAlg::gemv(mosek.transpose trans, int m, int n, double alpha, shared_ptr
→<ndarray<double,1>> a, shared_ptr<ndarray<double,1>> x, double beta, shared_ptr
→<ndarray<double,1>> y)
```

Computes the multiplication of a scaled dense matrix times a dense vector, plus a scaled dense vector. Precisely, if `trans` is `NO` then the update is

$$y := \alpha Ax + \beta y,$$

and if `trans` is `YES` then

$$y := \alpha A^T x + \beta y,$$

where α, β are scalar values and A is a matrix with m rows and n columns.

Note that the result is stored overwriting y . It must not overlap with the other input arrays.

Parameters

- **trans (transpose)** – Indicates if A should be transposed. See the Optimizer API documentation for the definition of these constants.
- **m (int)** – Specifies the number of rows of the matrix A .

- **n** (int) – Specifies the number of columns of the matrix A .
- **alpha** (double) – A scalar value multiplying the matrix A .
- **a** (double[]) – A pointer to the array storing matrix A in a column-major format.
- **x** (double[]) – A pointer to the array storing the vector x .
- **beta** (double) – A scalar value multiplying the vector y .
- **y** (double[]) – A pointer to the array storing the vector y .

LinAlg.potrf

```
void LinAlg::potrf(mosek.uplo uplo, int n, shared_ptr<ndarray<double,1>> a)
```

Computes a Cholesky factorization of a real symmetric positive definite dense matrix.

Parameters

- **uplo** (uplo) – Indicates whether the upper or lower triangular part of the matrix is used. See the Optimizer API documentation for the definition of these constants.
- **n** (int) – Specifies the dimension of the symmetric matrix.
- **a** (double[]) – A symmetric matrix stored in column-major order. Only the lower or the upper triangular part is used, accordingly with the **uplo** argument. It will contain the result on exit.

LinAlg.syeig

```
void LinAlg::syeig(mosek.uplo uplo, int n, shared_ptr<ndarray<double,1>> a,
↳ shared_ptr<ndarray<double,1>> w)
```

Computes all eigenvalues of a real symmetric matrix A . Given a matrix $A \in \mathbb{R}^{n \times n}$ it returns a vector $w \in \mathbb{R}^n$ containing the eigenvalues of A .

Parameters

- **uplo** (uplo) – Indicates whether the upper or lower triangular part of the matrix is used. See the Optimizer API documentation for the definition of these constants.
- **n** (int) – Specifies the dimension of the symmetric matrix.
- **a** (double[]) – A symmetric matrix stored in column-major order. Only the lower or the upper triangular part is used, accordingly with the **uplo** argument. It will contain the result on exit.
- **w** (double[]) – Array of length at least **n** containing the eigenvalues of A .

LinAlg.syevd

```
void LinAlg::syevd(mosek.uplo uplo, int n, shared_ptr<ndarray<double,1>> a,
↳ shared_ptr<ndarray<double,1>> w)
```

Computes all the eigenvalues and eigenvectors a real symmetric matrix. Given the input matrix $A \in \mathbb{R}^{n \times n}$, this function returns a vector $w \in \mathbb{R}^n$ containing the eigenvalues of A and it also computes the eigenvectors of A . Therefore, this function computes the eigenvalue decomposition of A as

$$A = UVU^T,$$

where $V = \text{diag}(w)$ and U contains the eigenvectors of A .

Note that the matrix U overwrites the input data A .

Parameters

- **uplo** (**uplo**) – Indicates whether the upper or lower triangular part of the matrix is used. See the Optimizer API documentation for the definition of these constants.
- **n** (**int**) – Specifies the dimension of the symmetric matrix.
- **a** (**double**[]) – A symmetric matrix stored in column-major order. Only the lower or the upper triangular part is used, accordingly with the **uplo** argument. It will contain the result on exit.
- **w** (**double**[]) – Array of length at least **n** containing the eigenvalues of A .

LinAlg.syrk

```
void LinAlg::syrk(mosek.uplo uplo, mosek.transpose trans, int n, int k, double
↪alpha, shared_ptr<ndarray<double,1>> a, double beta, shared_ptr<ndarray<double,
↪1>> c)
```

Performs a symmetric rank- k update for a symmetric matrix.

Given a symmetric matrix $C \in \mathbb{R}^{n \times n}$, two scalars α, β and a matrix A of rank $k \leq n$, it computes either

$$C := \alpha AA^T + \beta C,$$

when **trans** is set to **NO** and $A \in \mathbb{R}^{n \times k}$, or

$$C := \alpha A^T A + \beta C,$$

when **trans** is set to **YES** and $A \in \mathbb{R}^{k \times n}$.

Only the part of C indicated by **uplo** is used and only that part is updated with the result. It must not overlap with the other input arrays.

Parameters

- **uplo** (**uplo**) – Indicates whether the upper or lower triangular part of C is used. See the Optimizer API documentation for the definition of these constants.
- **trans** (**transpose**) – Indicates if A should be transposed. See the Optimizer API documentation for the definition of these constants.
- **n** (**int**) – Specifies the order of C .
- **k** (**int**) – Indicates the number of rows or columns of A , depending on whether or not it is transposed, and its rank.
- **alpha** (**double**) – A scalar value multiplying the result of the matrix multiplication.
- **a** (**double**[]) – The pointer to the array storing matrix A in a column-major format.
- **beta** (**double**) – A scalar value that multiplies C .
- **c** (**double**[]) – The pointer to the array storing matrix C in a column-major format.

Chapter 15

Supported File Formats

MOSEK supports a range of problem and solution formats listed in [Table 15.1](#) and [Table 15.2](#).

The most important are:

- the **Task format**, **MOSEK**'s native binary format which supports all features that **MOSEK** supports. It is the closest possible representation of the internal data in a task and it is ideal for submitting problem data support questions.
- the **PTF format**, **MOSEK**'s human-readable format that supports all linear, conic and mixed-integer features. It is ideal for debugging. It is not an exact copy of all the data in the task, but it contains all information required to reconstruct it, presented in a readable fashion.
- **MPS**, **LP**, **CBF** formats are industry standards, each supporting some limited set of features, and potentially requiring some degree of reformulation during read/write.

Problem formats

Table 15.1: List of supported file formats for optimization problems.

Format Type	Ext.	Binary/Text	LP	QCQO	ACC	SDP	DJC	Sol	Param
<i>LP</i>	lp	plain text	X	X					
<i>MPS</i>	mps	plain text	X	X					
<i>PTF</i>	ptf	plain text	X		X	X	X	X	
<i>CBF</i>	cbf	plain text	X		X	X			
<i>Task format</i>	task	binary	X	X	X	X	X	X	X
<i>Jtask format</i>	jtask	text/JSON	X	X	X	X	X	X	X
<i>OPF</i> (deprecated for conic problems)	opf	plain text	X	X				X	X

The columns of the table indicate if the specified file format supports:

- LP - linear problems,
- QCQO - quadratic objective or constraints,
- ACC - affine conic constraints,
- SDP - semidefinite cone/variables,
- DJC - disjunctive constraints,
- Sol - solutions,
- Param - optimizer parameters.

Solution formats

Table 15.2: List of supported solution formats.

Format Type	Ext.	Binary/Text	Description
<i>SOL</i>	sol	plain text	Interior Solution
	bas	plain text	Basic Solution
	int	plain text	Integer
<i>Jsol format</i>	jsol	text/JSON	All solutions

Compression

MOSEK supports GZIP and Zstandard compression. Problem files with extension `.gz` (for GZIP) and `.zst` (for Zstandard) are assumed to be compressed when read, and are automatically compressed when written. For example, a file called

`problem.mps.zst`

will be considered as a Zstandard compressed MPS file.

15.1 The LP File Format

MOSEK supports the LP file format with some extensions. The LP format is not a completely well-defined standard and hence different optimization packages may interpret the same LP file in slightly different ways. **MOSEK** tries to emulate as closely as possible CPLEX's behavior, but tries to stay backward compatible.

The LP file format can specify problems of the form

$$\begin{aligned}
 & \text{minimize/maximize} && c^T x + \frac{1}{2} q^o(x) \\
 & \text{subject to} && l^c \leq Ax + \frac{1}{2} q(x) \leq u^c, \\
 & && l^x \leq x \leq u^x, \\
 & && x_{\mathcal{J}} \text{ integer},
 \end{aligned}$$

where

- $x \in \mathbb{R}^n$ is the vector of decision variables.
- $c \in \mathbb{R}^n$ is the linear term in the objective.
- $q^o : \mathbb{R}^n \rightarrow \mathbb{R}$ is the quadratic term in the objective where

$$q^o(x) = x^T Q^o x$$

and it is assumed that

$$Q^o = (Q^o)^T.$$

- $A \in \mathbb{R}^{m \times n}$ is the constraint matrix.
- $l^c \in \mathbb{R}^m$ is the lower limit on the activity for the constraints.
- $u^c \in \mathbb{R}^m$ is the upper limit on the activity for the constraints.
- $l^x \in \mathbb{R}^n$ is the lower limit on the activity for the variables.
- $u^x \in \mathbb{R}^n$ is the upper limit on the activity for the variables.
- $q : \mathbb{R}^n \rightarrow \mathbb{R}$ is a vector of quadratic functions. Hence,

$$q_i(x) = x^T Q^i x$$

where it is assumed that

$$Q^i = (Q^i)^T.$$

- $\mathcal{J} \subseteq \{1, 2, \dots, n\}$ is an index set of the integer constrained variables.

15.1.1 File Sections

An LP formatted file contains a number of sections specifying the objective, constraints, variable bounds, and variable types. The section keywords may be any mix of upper and lower case letters.

Objective Function

The first section beginning with one of the keywords

```
max
maximum
maximize
min
minimum
minimize
```

defines the objective sense and the objective function, i.e.

$$c^T x + \frac{1}{2} x^T Q x.$$

The objective may be given a name by writing

```
myname:
```

before the expressions. If no name is given, then the objective is named **obj**.

The objective function contains linear and quadratic terms. The linear terms are written as

```
4 x1 + x2 - 0.1 x3
```

and so forth. The quadratic terms are written in square brackets (`[ ]/2`) and are either squared or multiplied as in the examples

```
x1^2
```

and

```
x1 * x2
```

There may be zero or more pairs of brackets containing quadratic expressions.

An example of an objective section is

```
minimize
myobj: 4 x1 + x2 - 0.1 x3 + [ x1^2 + 2.1 x1 * x2 ]/2
```

Please note that the quadratic expressions are multiplied with $\frac{1}{2}$, so that the above expression means

$$\text{minimize } 4x_1 + x_2 - 0.1 \cdot x_3 + \frac{1}{2}(x_1^2 + 2.1 \cdot x_1 \cdot x_2)$$

If the same variable occurs more than once in the linear part, the coefficients are added, so that `4 x1 + 2 x1` is equivalent to `6 x1`. In the quadratic expressions `x1 * x2` is equivalent to `x2 * x1` and, as in the linear part, if the same variables multiplied or squared occur several times their coefficients are added.

Constraints

The second section beginning with one of the keywords

```
subj to
subject to
s.t.
st
```

defines the linear constraint matrix A and the quadratic matrices Q^i .

A constraint contains a name (optional), expressions adhering to the same rules as in the objective and a bound:

```
subject to
con1: x1 + x2 + [ x3^2 ]/2 <= 5.1
```

The bound type (here $\leq$) may be any of $<$, $\leq$, $=$, $>$, $\geq$ ($<$ and $\leq$ mean the same), and the bound may be any number.

In the standard LP format it is not possible to define more than one bound per line, but **MOSEK** supports defining ranged constraints by using double-colon ($::$) instead of a single-colon ($:$) after the constraint name, i.e.

$$-5 \leq x_1 + x_2 \leq 5 \quad (15.1)$$

may be written as

```
con:: -5 < x_1 + x_2 < 5
```

By default **MOSEK** writes ranged constraints this way.

If the files must adhere to the LP standard, ranged constraints must either be split into upper bounded and lower bounded constraints or be written as an equality with a slack variable. For example the expression (15.1) may be written as

$$x_1 + x_2 - sl_1 = 0, \quad -5 \leq sl_1 \leq 5.$$

Bounds

Bounds on the variables can be specified in the bound section beginning with one of the keywords

```
bound
bounds
```

The bounds section is optional but should, if present, follow the **subject to** section. All variables listed in the bounds section must occur in either the objective or a constraint.

The default lower and upper bounds are 0 and $+\infty$. A variable may be declared free with the keyword **free**, which means that the lower bound is $-\infty$ and the upper bound is $+\infty$. Furthermore it may be assigned a finite lower and upper bound. The bound definitions for a given variable may be written in one or two lines, and bounds can be any number or $\pm\infty$ (written as **+inf/-inf/+infinity/-infinity**) as in the example

```
bounds
x1 free
x2 <= 5
0.1 <= x2
x3 = 42
2 <= x4 < +inf
```

Variable Types

The final two sections are optional and must begin with one of the keywords

```
bin
binaries
binary
```

and

```
gen
general
```

Under **general** all integer variables are listed, and under **binary** all binary (integer variables with bounds 0 and 1) are listed:

```
general
x1 x2
binary
x3 x4
```

Again, all variables listed in the binary or general sections must occur in either the objective or a constraint.

Terminating Section

Finally, an LP formatted file must be terminated with the keyword

```
end
```

15.1.2 LP File Examples

Linear example lo1.lp

```
\ File: lo1.lp
maximize
obj: 3 x1 + x2 + 5 x3 + x4
subject to
c1: 3 x1 + x2 + 2 x3 = 30
c2: 2 x1 + x2 + 3 x3 + x4 >= 15
c3: 2 x2 + 3 x4 <= 25
bounds
0 <= x1 <= +infinity
0 <= x2 <= 10
0 <= x3 <= +infinity
0 <= x4 <= +infinity
end
```

Mixed integer example milo1.lp

```
maximize
obj: x1 + 6.4e-01 x2
subject to
c1: 5e+01 x1 + 3.1e+01 x2 <= 2.5e+02
c2: 3e+00 x1 - 2e+00 x2 >= -4e+00
bounds
0 <= x1 <= +infinity
0 <= x2 <= +infinity
general
x1 x2
end
```


15.1.3 LP Format peculiarities

Comments

Anything on a line after a \ is ignored and is treated as a comment.

Names

A name for an objective, a constraint or a variable may contain the letters a-z, A-Z, the digits 0-9 and the characters

!"#\$%&()/,.;?@_'\`|~

The first character in a name must not be a number, a period or the letter e or E. Keywords must not be used as names.

MOSEK accepts any character as valid for names, except \0. A name that is not allowed in LP file will be changed and a warning will be issued.

The algorithm for making names LP valid works as follows: The name is interpreted as an **utf-8** string. For a Unicode character *c*:

- If *c*==_ (underscore), the output is __ (two underscores).
- If *c* is a valid LP name character, the output is just *c*.
- If *c* is another character in the ASCII range, the output is _XX, where XX is the hexadecimal code for the character.
- If *c* is a character in the range 127-65535, the output is _uXXXX, where XXXX is the hexadecimal code for the character.
- If *c* is a character above 65535, the output is _UXXXXXXXX, where XXXXXXXX is the hexadecimal code for the character.

Invalid **utf-8** substrings are escaped as _XX', and if a name starts with a period, e or E, that character is escaped as _XX.

Variable Bounds

Specifying several upper or lower bounds on one variable is possible but **MOSEK** uses only the tightest bounds. If a variable is fixed (with =), then it is considered the tightest bound.

15.2 The MPS File Format

MOSEK supports the standard MPS format with some extensions. For a detailed description of the MPS format see the book by Nazareth [Naz87].

15.2.1 MPS File Structure

The version of the MPS format supported by **MOSEK** allows specification of an optimization problem of the form

$$\begin{aligned}
 &\text{maximize/minimize} && c^T x + q_0(x) \\
 &l^c \leq && Ax + q(x) \leq u^c, \\
 &l^x \leq && x \leq u^x, \\
 &&& x \in \mathcal{K}, \\
 &&& x_{\mathcal{J}} \text{ integer},
 \end{aligned} \tag{15.2}$$

where

- $x \in \mathbb{R}^n$ is the vector of decision variables.
- $A \in \mathbb{R}^{m \times n}$ is the constraint matrix.
- $l^c \in \mathbb{R}^m$ is the lower limit on the activity for the constraints.

- $u^c \in \mathbb{R}^m$ is the upper limit on the activity for the constraints.
- $l^x \in \mathbb{R}^n$ is the lower limit on the activity for the variables.
- $u^x \in \mathbb{R}^n$ is the upper limit on the activity for the variables.
- $q : \mathbb{R}^n \rightarrow \mathbb{R}$ is a vector of quadratic functions. Hence,

$$q_i(x) = \frac{1}{2}x^T Q^i x$$

where it is assumed that $Q^i = (Q^i)^T$. Please note the explicit $\frac{1}{2}$ in the quadratic term and that Q^i is required to be symmetric. The same applies to q_0 .

- $\mathcal{K}$ is a convex cone.
- $\mathcal{J} \subseteq \{1, 2, \dots, n\}$ is an index set of the integer-constrained variables.
- c is the vector of objective coefficients.

An MPS file with one row and one column can be illustrated like this:

```
*          1          2          3          4          5          6
*23456789012345678901234567890123456789012345678901234567890
NAME          [name]
OBJSENSE
    [objsense]
OBJNAME          [objname]
ROWS
?  [cname1]
COLUMNS
    [vname1]  [cname1]  [value1]          [cname2]  [value2]
RHS
    [name]    [cname1]  [value1]          [cname2]  [value2]
RANGES
    [name]    [cname1]  [value1]          [cname2]  [value2]
QSECTION          [cname1]
    [vname1]  [vname2]  [value1]          [vname3]  [value2]
QMATRIX
    [vname1]  [vname2]  [value1]
QUADOBJ
    [vname1]  [vname2]  [value1]
QCMATRIX          [cname1]
    [vname1]  [vname2]  [value1]
BOUNDS
?? [name]      [vname1]  [value1]
CSECTION          [kname1]  [value1]          [ktype]
    [vname1]
ENDATA
```

Here the names in capitals are keywords of the MPS format and names in brackets are custom defined names or values. A couple of notes on the structure:

- Fields: All items surrounded by brackets appear in *fields*. The fields named “**valueN**” are numerical values. Hence, they must have the format

$[+|-]XXXXXXXX.XXXXXX[e|E][+|-]XXX$

where

$X = [0|1|2|3|4|5|6|7|8|9].$

- Sections: The MPS file consists of several sections where the names in capitals indicate the beginning of a new section. For example, COLUMNS denotes the beginning of the columns section.

- Comments: Lines starting with an ***** are comment lines and are ignored by **MOSEK**.
- Keys: The question marks represent keys to be specified later.
- Extensions: The sections **QSECTION** and **CSECTION** are specific **MOSEK** extensions of the MPS format. The sections **QMATRIX**, **QUADOBJ** and **QCMATRIX** are included for sake of compatibility with other vendors extensions to the MPS format.
- The standard MPS format is a fixed format, i.e. everything in the MPS file must be within certain fixed positions. **MOSEK** also supports a *free format*. See [Sec. 15.2.5](#) for details.

Linear example lo1.mps

A concrete example of a MPS file is presented below:

```
* File: lo1.mps
NAME          lo1
OBJSENSE
    MAX
ROWS
N  obj
E  c1
G  c2
L  c3
COLUMNS
    x1      obj      3
    x1      c1       3
    x1      c2       2
    x2      obj      1
    x2      c1       1
    x2      c2       1
    x2      c3       2
    x3      obj      5
    x3      c1       2
    x3      c2       3
    x4      obj      1
    x4      c2       1
    x4      c3       3
RHS
    rhs     c1      30
    rhs     c2      15
    rhs     c3      25
RANGES
BOUNDS
UP bound    x2      10
ENDATA
```

Subsequently each individual section in the MPS format is discussed.

NAME (optional)

In this section a name (**[name]**) is assigned to the problem.

OBJSENSE (optional)

This is an optional section that can be used to specify the sense of the objective function. The **OBJSENSE** section contains one line at most which can be one of the following:

```
MIN
MINIMIZE
MAX
MAXIMIZE
```

It should be obvious what the implication is of each of these four lines.

OBJNAME (optional)

This is an optional section that can be used to specify the name of the row that is used as objective function. **objname** should be a valid row name.

ROWS

A record in the **ROWS** section has the form

```
? [cname1]
```

where the requirements for the fields are as follows:

Field	Starting Position	Max Width	required	Description
?	2	1	Yes	Constraint key
[cname1]	5	8	Yes	Constraint name

Hence, in this section each constraint is assigned a unique name denoted by [cname1]. Please note that [cname1] starts in position 5 and the field can be at most 8 characters wide. An initial key ? must be present to specify the type of the constraint. The key can have values E, G, L, or N with the following interpretation:

Constraint type	l_i^c	u_i^c
E (equal)	finite	$= l_i^c$
G (greater)	finite	∞
L (lower)	$-\infty$	finite
N (none)	$-\infty$	∞

In the MPS format the objective vector is not specified explicitly, but one of the constraints having the key N will be used as the objective vector c . In general, if multiple N type constraints are specified, then the first will be used as the objective vector c , unless something else was specified in the section **OBJNAME**.

COLUMNS

In this section the elements of A are specified using one or more records having the form:

```
[vname1] [cname1] [value1] [cname2] [value2]
```

where the requirements for each field are as follows:

Field	Starting Position	Max Width	required	Description
[vname1]	5	8	Yes	Variable name
[cname1]	15	8	Yes	Constraint name
[value1]	25	12	Yes	Numerical value
[cname2]	40	8	No	Constraint name
[value2]	50	12	No	Numerical value

Hence, a record specifies one or two elements a_{ij} of A using the principle that [vname1] and [cname1] determines j and i respectively. Please note that [cname1] must be a constraint name specified in the

ROWS section. Finally, [value1] denotes the numerical value of a_{ij} . Another optional element is specified by [cname2], and [value2] for the variable specified by [vname1]. Some important comments are:

- All elements belonging to one variable must be grouped together.
- Zero elements of A should not be specified.
- At least one element for each variable should be specified.

RHS (optional)

A record in this section has the format

[name]	[cname1]	[value1]	[cname2]	[value2]
--------	----------	----------	----------	----------

where the requirements for each field are as follows:

Field	Starting Position	Max Width	required	Description
[name]	5	8	Yes	Name of the RHS vector
[cname1]	15	8	Yes	Constraint name
[value1]	25	12	Yes	Numerical value
[cname2]	40	8	No	Constraint name
[value2]	50	12	No	Numerical value

The interpretation of a record is that [name] is the name of the RHS vector to be specified. In general, several vectors can be specified. [cname1] denotes a constraint name previously specified in the ROWS section. Now, assume that this name has been assigned to the i -th constraint and v_1 denotes the value specified by [value1], then the interpretation of v_1 is:

Constraint	l_i^c	u_i^c
E	v_1	v_1
G	v_1	
L		v_1
N		

An optional second element is specified by [cname2] and [value2] and is interpreted in the same way. Please note that it is not necessary to specify zero elements, because elements are assumed to be zero.

RANGES (optional)

A record in this section has the form

[name]	[cname1]	[value1]	[cname2]	[value2]
--------	----------	----------	----------	----------

where the requirements for each fields are as follows:

Field	Starting Position	Max Width	required	Description
[name]	5	8	Yes	Name of the RANGE vector
[cname1]	15	8	Yes	Constraint name
[value1]	25	12	Yes	Numerical value
[cname2]	40	8	No	Constraint name
[value2]	50	12	No	Numerical value

The records in this section are used to modify the bound vectors for the constraints, i.e. the values in l^c and u^c . A record has the following interpretation: [name] is the name of the RANGE vector and [cname1] is a valid constraint name. Assume that [cname1] is assigned to the i -th constraint and let v_1 be the value specified by [value1], then a record has the interpretation:

Constraint type	Sign of v_1	l_i^c	u_i^c
E	—	$u_i^c + v_1$	
E	+		$l_i^c + v_1$
G	— or +		$l_i^c + v_1 $
L	— or +	$u_i^c - v_1 $	
N			

Another constraint bound can optionally be modified using [cname2] and [value2] the same way.

QSECTION (optional)

Within the QSECTION the label [cname1] must be a constraint name previously specified in the ROWS section. The label [cname1] denotes the constraint to which the quadratic terms belong. A record in the QSECTION has the form

[vname1]	[vname2]	[value1]	[vname3]	[value2]
----------	----------	----------	----------	----------

where the requirements for each field are:

Field	Starting Position	Max Width	required	Description
[vname1]	5	8	Yes	Variable name
[vname2]	15	8	Yes	Variable name
[value1]	25	12	Yes	Numerical value
[vname3]	40	8	No	Variable name
[value2]	50	12	No	Numerical value

A record specifies one or two elements in the lower triangular part of the Q^i matrix where [cname1] specifies the i . Hence, if the names [vname1] and [vname2] have been assigned to the k -th and j -th variable, then Q_{kj}^i is assigned the value given by [value1]. An optional second element is specified in the same way by the fields [vname1], [vname3], and [value2].

The example

$$\begin{array}{ll}
\text{minimize} & -x_2 + \frac{1}{2}(2x_1^2 - 2x_1x_3 + 0.2x_2^2 + 2x_3^2) \\
\text{subject to} & x_1 + x_2 + x_3 \geq 1, \\
& x \geq 0
\end{array}$$

has the following MPS file representation

```

* File: qo1.mps
NAME          qo1
ROWS
  N  obj
  G  c1
COLUMNS
  x1      c1      1.0
  x2      obj     -1.0
  x2      c1      1.0
  x3      c1      1.0
RHS
  rhs     c1      1.0
QSECTION  obj
  x1      x1      2.0
  x1      x3     -1.0
  x2      x2      0.2
  x3      x3      2.0
ENDATA

```

Regarding the QSECTIONS please note that:

- Only one QSECTION is allowed for each constraint.

- The QSECTIONs can appear in an arbitrary order after the COLUMNS section.
- All variable names occurring in the QSECTION must already be specified in the COLUMNS section.
- All entries specified in a QSECTION are assumed to belong to the lower triangular part of the quadratic term of Q .

QMATRIX/QUADOBJ (optional)

The QMATRIX and QUADOBJ sections allow to define the quadratic term of the objective function. They differ in how the quadratic term of the objective function is stored:

- QMATRIX stores all the nonzeros coefficients, without taking advantage of the symmetry of the Q matrix.
- QUADOBJ stores the upper diagonal nonzero elements of the Q matrix.

A record in both sections has the form:

[vname1]	[vname2]	[value1]
----------	----------	----------

where the requirements for each field are:

Field	Starting Position	Max Width	required	Description
[vname1]	5	8	Yes	Variable name
[vname2]	15	8	Yes	Variable name
[value1]	25	12	Yes	Numerical value

A record specifies one elements of the Q matrix in the objective function. Hence, if the names [vname1] and [vname2] have been assigned to the k -th and j -th variable, then Q_{kj} is assigned the value given by [value1]. Note that a line must appear for each off-diagonal coefficient if using a QMATRIX section, while only one entry is required in a QUADOBJ section. The quadratic part of the objective function will be evaluated as $1/2x^T Qx$.

The example

$$\begin{aligned}
 &\text{minimize} && -x_2 + \frac{1}{2}(2x_1^2 - 2x_1x_3 + 0.2x_2^2 + 2x_3^2) \\
 &\text{subject to} && x_1 + x_2 + x_3 \geq 1, \\
 &&& x \geq 0
 \end{aligned}$$

has the following MPS file representation using QMATRIX

```

* File: qo1_matrix.mps
NAME          qo1_qmatrix
ROWS
  N  obj
  G  c1
COLUMNS
  x1      c1      1.0
  x2      obj     -1.0
  x2      c1      1.0
  x3      c1      1.0
RHS
  rhs     c1      1.0
QMATRIX
  x1      x1      2.0
  x1      x3     -1.0
  x3      x1     -1.0
  x2      x2      0.2
  x3      x3      2.0
ENDATA

```

or the following using QUADOBJ

```

* File: qo1_quadobj.mps
NAME          qo1_quadobj
ROWS
  N  obj
  G  c1
COLUMNS
  x1      c1      1.0
  x2      obj     -1.0
  x2      c1      1.0
  x3      c1      1.0
RHS
  rhs     c1      1.0
QUADOBJ
  x1      x1      2.0
  x1      x3     -1.0
  x2      x2      0.2
  x3      x3      2.0
ENDATA

```

Please also note that:

- A QMATRIX/QUADOBJ section can appear in an arbitrary order after the COLUMNS section.
- All variable names occurring in the QMATRIX/QUADOBJ section must already be specified in the COLUMNS section.

QMATRIX (optional)

A QMATRIX section allows to specify the quadratic part of a given constraint. Within the QMATRIX the label [cname1] must be a constraint name previously specified in the ROWS section. The label [cname1] denotes the constraint to which the quadratic term belongs. A record in the QSECTION has the form

[vname1]	[vname2]	[value1]
----------	----------	----------

where the requirements for each field are:

Field	Starting Position	Max Width	required	Description
[vname1]	5	8	Yes	Variable name
[vname2]	15	8	Yes	Variable name
[value1]	25	12	Yes	Numerical value

A record specifies an entry of the Q^i matrix where [cname1] specifies the i . Hence, if the names [vname1] and [vname2] have been assigned to the k -th and j -th variable, then Q_{kj}^i is assigned the value given by [value1]. Moreover, the quadratic term is represented as $1/2x^T Qx$.

The example

$$\begin{aligned}
& \text{minimize} && x_2 \\
& \text{subject to} && x_1 + x_2 + x_3 \geq 1, \\
& && \frac{1}{2}(-2x_1x_3 + 0.2x_2^2 + 2x_3^2) \leq 10, \\
& && x \geq 0
\end{aligned}$$

has the following MPS file representation

```

* File: qo1.mps
NAME          qo1
ROWS
  N  obj
  G  c1
  L  q1
COLUMNS

```

(continues on next page)

(continued from previous page)

x1	c1	1.0
x2	obj	-1.0
x2	c1	1.0
x3	c1	1.0
RHS		
rhs	c1	1.0
rhs	q1	10.0
QCMATRIX		
q1	q1	
x1	x1	2.0
x1	x3	-1.0
x3	x1	-1.0
x2	x2	0.2
x3	x3	2.0
ENDATA		

Regarding the QCMATRIXs please note that:

- Only one QCMATRIX is allowed for each constraint.
- The QCMATRIXs can appear in an arbitrary order after the COLUMNS section.
- All variable names occurring in the QSECTION must already be specified in the COLUMNS section.
- QCMATRIX does not exploit the symmetry of Q : an off-diagonal entry (i, j) should appear twice.

BOUNDS (optional)

In the BOUNDS section changes to the default bounds vectors l^x and u^x are specified. The default bounds vectors are $l^x = 0$ and $u^x = \infty$. Moreover, it is possible to specify several sets of bound vectors. A record in this section has the form

??	[name]	[vname1]	[value1]
----	--------	----------	----------

where the requirements for each field are:

Field	Starting Position	Max Width	Required	Description
??	2	2	Yes	Bound key
[name]	5	8	Yes	Name of the BOUNDS vector
[vname1]	15	8	Yes	Variable name
[value1]	25	12	No	Numerical value

Hence, a record in the BOUNDS section has the following interpretation: [name] is the name of the bound vector and [vname1] is the name of the variable for which the bounds are modified by the record. ?? and [value1] are used to modify the bound vectors according to the following table:

??	l_j^x	u_j^x	Made integer (added to $\mathcal{J}$)
FR	$-\infty$	∞	No
FX	v_1	v_1	No
LO	v_1	unchanged	No
MI	$-\infty$	unchanged	No
PL	unchanged	∞	No
UP	unchanged	v_1	No
BV	0	1	Yes
LI	$\lceil v_1 \rceil$	unchanged	Yes
UI	unchanged	$\lfloor v_1 \rfloor$	Yes

Here v_1 is the value specified by [value1].

CSECTION (optional)

The purpose of the CSECTION is to specify the conic constraint

$$x \in \mathcal{K}$$

in (15.2). It is assumed that $\mathcal{K}$ satisfies the following requirements. Let

$$x^t \in \mathbb{R}^{n^t}, \quad t = 1, \dots, k$$

be vectors comprised of parts of the decision variables x so that each decision variable is a member of exactly **one** vector x^t , for example

$$x^1 = \begin{bmatrix} x_1 \\ x_4 \\ x_7 \end{bmatrix} \quad \text{and} \quad x^2 = \begin{bmatrix} x_6 \\ x_5 \\ x_3 \\ x_2 \end{bmatrix}.$$

Next define

$$\mathcal{K} := \{x \in \mathbb{R}^n : x^t \in \mathcal{K}_t, \quad t = 1, \dots, k\}$$

where $\mathcal{K}_t$ must have one of the following forms:

- $\mathbb{R}$ set:

$$\mathcal{K}_t = \mathbb{R}^{n^t}.$$

- Zero cone:

$$\mathcal{K}_t = \{0\} \subseteq \mathbb{R}^{n^t}. \quad (15.3)$$

- Quadratic cone:

$$\mathcal{K}_t = \left\{ x \in \mathbb{R}^{n^t} : x_1 \geq \sqrt{\sum_{j=2}^{n^t} x_j^2} \right\}. \quad (15.4)$$

- Rotated quadratic cone:

$$\mathcal{K}_t = \left\{ x \in \mathbb{R}^{n^t} : 2x_1x_2 \geq \sum_{j=3}^{n^t} x_j^2, \quad x_1, x_2 \geq 0 \right\}. \quad (15.5)$$

- Primal exponential cone:

$$\mathcal{K}_t = \{x \in \mathbb{R}^3 : x_1 \geq x_2 \exp(x_3/x_2), \quad x_1, x_2 \geq 0\}. \quad (15.6)$$

- Primal power cone (with parameter $0 < \alpha < 1$):

$$\mathcal{K}_t = \left\{ x \in \mathbb{R}^{n^t} : x_1^\alpha x_2^{1-\alpha} \geq \sqrt{\sum_{j=3}^{n^t} x_j^2}, \quad x_1, x_2 \geq 0 \right\}. \quad (15.7)$$

- Dual exponential cone:

$$\mathcal{K}_t = \{x \in \mathbb{R}^3 : x_1 \geq -x_3 e^{-1} \exp(x_2/x_3), \quad x_3 \leq 0, x_1 \geq 0\}. \quad (15.8)$$

- Dual power cone (with parameter $0 < \alpha < 1$):

$$\mathcal{K}_t = \left\{ x \in \mathbb{R}^{n^t} : \left(\frac{x_1}{\alpha}\right)^\alpha \left(\frac{x_2}{1-\alpha}\right)^{1-\alpha} \geq \sqrt{\sum_{j=3}^{n^t} x_j^2}, \quad x_1, x_2 \geq 0 \right\}. \quad (15.9)$$

In general, membership in the $\mathbb{R}$ set is not specified. If a variable is not a member of any other cone then it is assumed to be a member of the $\mathbb{R}$ cone.

Next, let us study an example. Assume that the power cone

$$x_4^{1/3} x_5^{2/3} \geq |x_8|$$

and the rotated quadratic cone

$$2x_3x_7 \geq x_1^2 + x_0^2, \quad x_3, x_7 \geq 0,$$

should be specified in the MPS file. One CSECTION is required for each cone and they are specified as follows:

*	1	2	3	4	5	6
*234567890123456789012345678901234567890123456789012345678901234567890						
CSECTION	konea	3e-1		PPOW		
x4						
x5						
x8						
CSECTION	koneb	0.0		RQUAD		
x7						
x3						
x1						
x0						

In general, a CSECTION header has the format

CSECTION	[kname1]	[value1]	[ktype]
----------	----------	----------	---------

where the requirements for each field are as follows:

Field	Starting Position	Max Width	Required	Description
[kname1]	15	8	Yes	Name of the cone
[value1]	25	12	No	Cone parameter
[ktype]	40		Yes	Type of the cone.

The possible cone type keys are:

[ktype]	Members	[value1]	Interpretation.
ZERO	≥ 0	unused	Zero cone (15.3).
QUAD	≥ 1	unused	Quadratic cone (15.4).
RQUAD	≥ 2	unused	Rotated quadratic cone (15.5).
PEXP	3	unused	Primal exponential cone (15.6).
PPOW	≥ 2	α	Primal power cone (15.7).
DEXP	3	unused	Dual exponential cone (15.8).
DPOW	≥ 2	α	Dual power cone (15.9).

A record in the CSECTION has the format

[vname1]

where the requirements for each field are

Field	Starting Position	Max Width	required	Description
[vname1]	5	8	Yes	A valid variable name

A variable must occur in at most one CSECTION.

ENDATA

This keyword denotes the end of the MPS file.

15.2.2 Integer Variables

Using special bound keys in the `BOUNDS` section it is possible to specify that some or all of the variables should be integer-constrained i.e. be members of $\mathcal{J}$. However, an alternative method is available. This method is available only for backward compatibility and we recommend that it is not used. This method requires that markers are placed in the `COLUMNS` section as in the example:

```
COLUMNS
x1      obj      -10.0      c1      0.7
x1      c2        0.5      c3      1.0
x1      c4        0.1
* Start of integer-constrained variables.
MARK000 'MARKER'          'INTORG'
x2      obj      -9.0      c1      1.0
x2      c2        0.833333333 c3      0.66666667
x2      c4        0.25
x3      obj      1.0      c6      2.0
MARK001 'MARKER'          'INTEND'
* End of integer-constrained variables.
```

Please note that special marker lines are used to indicate the start and the end of the integer variables. Furthermore be aware of the following

- All variables between the markers are assigned a default lower bound of 0 and a default upper bound of 1. **This may not be what is intended.** If it is not intended, the correct bounds should be defined in the `BOUNDS` section of the MPS formatted file.
- **MOSEK** ignores field 1, i.e. `MARK0001` and `MARK001`, however, other optimization systems require them.
- Field 2, i.e. `MARKER`, must be specified including the single quotes. This implies that no row can be assigned the name `MARKER`.
- Field 3 is ignored and should be left blank.
- Field 4, i.e. `INTORG` and `INTEND`, must be specified.
- It is possible to specify several such integer marker sections within the `COLUMNS` section.

15.2.3 General Limitations

- An MPS file should be an ASCII file.

15.2.4 Interpretation of the MPS Format

Several issues related to the MPS format are not well-defined by the industry standard. However, **MOSEK** uses the following interpretation:

- If a matrix element in the `COLUMNS` section is specified multiple times, then the multiple entries are added together.
- If a matrix element in a `QSECTION` section is specified multiple times, then the multiple entries are added together.

15.2.5 The Free MPS Format

MOSEK supports a free format variation of the MPS format. The free format is similar to the MPS file format but less restrictive, e.g. it allows longer names. However, a name must not contain any blanks.

Warning: This file format is to a large extent deprecated. While it can still be used for linear and quadratic problems, for conic problems the [Sec. 15.5](#) is recommended.

15.3 The OPF Format

The *Optimization Problem Format (OPF)* is an alternative to LP and MPS files for specifying optimization problems. It is row-oriented, inspired by the CPLEX LP format.

Apart from containing objective, constraints, bounds etc. it may contain complete or partial solutions, comments and extra information relevant for solving the problem. It is designed to be easily read and modified by hand and to be forward compatible with possible future extensions.

Intended use

The OPF file format is meant to replace several other files:

- The LP file format: Any problem that can be written as an LP file can be written as an OPF file too; furthermore it naturally accommodates ranged constraints and variables as well as arbitrary characters in names, fixed expressions in the objective, empty constraints, and conic constraints.
- Parameter files: It is possible to specify integer, double and string parameters along with the problem (or in a separate OPF file).
- Solution files: It is possible to store a full or a partial solution in an OPF file and later reload it.

15.3.1 The File Format

The format uses tags to structure data. A simple example with the basic sections may look like this:

```
[comment]
This is a comment. You may write almost anything here...
[/comment]

# This is a single-line comment.

[objective min 'myobj']
x + 3 y + x^2 + 3 y^2 + z + 1
[/objective]

[constraints]
[con 'con01'] 4 <= x + y  [/con]
[/constraints]

[bounds]
[b] -10 <= x,y <= 10  [/b]

[cone quad] x,y,z [/cone]
[/bounds]
```

A scope is opened by a tag of the form `[tag]` and closed by a tag of the form `[/tag]`. An opening tag may accept a list of unnamed and named arguments, for examples:

```
[tag value] tag with one unnamed argument [/tag]
[tag arg=value] tag with one named argument [/tag]
```

Unnamed arguments are identified by their order, while named arguments may appear in any order, but never before an unnamed argument. The **value** can be a quoted, single-quoted or double-quoted text string, i.e.

```
[tag 'value']      single-quoted value [/tag]
[tag arg='value']  single-quoted value [/tag]
[tag "value"]      double-quoted value [/tag]
[tag arg="value"]  double-quoted value [/tag]
```

15.3.2 Sections

The recognized tags are

`[comment]`

A comment section. This can contain *almost* any text: Between single quotes (') or double quotes (") any text may appear. Outside quotes the markup characters ([and]) must be prefixed by backslashes. Both single and double quotes may appear alone or inside a pair of quotes if it is prefixed by a backslash.

`[objective]`

The objective function: This accepts one or two parameters, where the first one (in the above example `min`) is either `min` or `max` (regardless of case) and defines the objective sense, and the second one (above `myobj`), if present, is the objective name. The section may contain linear and quadratic expressions.

If several objectives are specified, all but the last are ignored.

`[constraints]`

This does not directly contain any data, but may contain subsections `con` defining a linear constraint.

`[con]`

Defines a single constraint; if an argument is present (`[con NAME]`) this is used as the name of the constraint, otherwise it is given a null-name. The section contains a constraint definition written as linear and quadratic expressions with a lower bound, an upper bound, with both or with an equality. Examples:

```
[constraints]
[con 'con1'] 0 <= x + y      [/con]
[con 'con2'] 0 >= x + y      [/con]
[con 'con3'] 0 <= x + y <= 10 [/con]
[con 'con4']      x + y = 10 [/con]
[/constraints]
```

Constraint names are unique. If a constraint is specified which has the same name as a previously defined constraint, the new constraint replaces the existing one.

`[bounds]`

This does not directly contain any data, but may contain subsections `b` (linear bounds on variables) and `cone` (cones).

[b]

Bound definition on one or several variables separated by comma (,). An upper or lower bound on a variable replaces any earlier defined bound on that variable. If only one bound (upper or lower) is given only this bound is replaced. This means that upper and lower bounds can be specified separately. So the OPF bound definition:

```
[b] x,y >= -10 [/b]
[b] x,y <= 10  [/b]
```

results in the bound $-10 \leq x, y \leq 10$.

[cone]

Specifies a cone. A cone is defined as a sequence of variables which belong to a single unique cone. The supported cone types are:

- **quad**: a quadratic cone of n variables $x_1, \dots, x_n$ defines a constraint of the form

$$x_1^2 \geq \sum_{i=2}^n x_i^2, \quad x_1 \geq 0.$$

- **rquad**: a rotated quadratic cone of n variables $x_1, \dots, x_n$ defines a constraint of the form

$$2x_1x_2 \geq \sum_{i=3}^n x_i^2, \quad x_1, x_2 \geq 0.$$

- **pexp**: primal exponential cone of 3 variables x_1, x_2, x_3 defines a constraint of the form

$$x_1 \geq x_2 \exp(x_3/x_2), \quad x_1, x_2 \geq 0.$$

- **ppow** with parameter $0 < \alpha < 1$: primal power cone of n variables $x_1, \dots, x_n$ defines a constraint of the form

$$x_1^\alpha x_2^{1-\alpha} \geq \sqrt{\sum_{j=3}^n x_j^2}, \quad x_1, x_2 \geq 0.$$

- **dexp**: dual exponential cone of 3 variables x_1, x_2, x_3 defines a constraint of the form

$$x_1 \geq -x_3 e^{-1} \exp(x_2/x_3), \quad x_3 \leq 0, x_1 \geq 0.$$

- **dpow** with parameter $0 < \alpha < 1$: dual power cone of n variables $x_1, \dots, x_n$ defines a constraint of the form

$$\left(\frac{x_1}{\alpha}\right)^\alpha \left(\frac{x_2}{1-\alpha}\right)^{1-\alpha} \geq \sqrt{\sum_{j=3}^n x_j^2}, \quad x_1, x_2 \geq 0.$$

- **zero**: zero cone of n variables $x_1, \dots, x_n$ defines a constraint of the form

$$x_1 = \dots = x_n = 0$$

A [bounds]-section example:

```
[bounds]
[b] 0 <= x,y <= 10 [/b] # ranged bound
[b] 10 >= x,y >= 0 [/b] # ranged bound
[b] 0 <= x,y <= inf [/b] # using inf
[b] x,y free [/b] # free variables
# Let (x,y,z,w) belong to the cone K
[cone rquad] x,y,z,w [/cone] # rotated quadratic cone
[cone ppow '3e-01' 'a'] x1, x2, x3 [/cone] # power cone with alpha=1/3 and name 'a'
[/bounds]
```

By default all variables are free.

[variables]

This defines an ordering of variables as they should appear in the problem. This is simply a space-separated list of variable names.

[integer]

This contains a space-separated list of variables and defines the constraint that the listed variables must be integer-valued.

[hints]

This may contain only non-essential data; for example estimates of the number of variables, constraints and non-zeros. Placed before all other sections containing data this may reduce the time spent reading the file.

In the `hints` section, any subsection which is not recognized by **MOSEK** is simply ignored. In this section a hint is defined as follows:

```
[hint ITEM] value [/hint]
```

The hints recognized by **MOSEK** are:

- `numvar` (number of variables),
- `numcon` (number of linear/quadratic constraints),
- `numanz` (number of linear non-zeros in constraints),
- `numqnz` (number of quadratic non-zeros in constraints).

[solutions]

This section can contain a set of full or partial solutions to a problem. Each solution must be specified using a `[solution]`-section, i.e.

```
[solutions]
[solution]...[/solution] #solution 1
[solution]...[/solution] #solution 2
#other solutions....
[solution]...[/solution] #solution n
[/solutions]
```

The syntax of a `[solution]`-section is the following:

```
[solution SOLTYPE status=STATUS]...[/solution]
```

where `SOLTYPE` is one of the strings

- `interior`, a non-basic solution,
- `basic`, a basic solution,
- `integer`, an integer solution,

and `STATUS` is one of the strings

- `UNKNOWN`,
- `OPTIMAL`,
- `INTEGER_OPTIMAL`,
- `PRIM_FEAS`,
- `DUAL_FEAS`,
- `PRIM_AND_DUAL_FEAS`,

- NEAR_OPTIMAL,
- NEAR_PRIM_FEAS,
- NEAR_DUAL_FEAS,
- NEAR_PRIM_AND_DUAL_FEAS,
- PRIM_INFEAS_CER,
- DUAL_INFEAS_CER,
- NEAR_PRIM_INFEAS_CER,
- NEAR_DUAL_INFEAS_CER,
- NEAR_INTEGER_OPTIMAL.

Most of these values are irrelevant for input solutions; when constructing a solution for simplex hot-start or an initial solution for a mixed integer problem the safe setting is UNKNOWN.

A [solution]-section contains [con] and [var] sections. Each [con] and [var] section defines solution information for a single variable or constraint, specified as list of KEYWORD/value pairs, in any order, written as

```
KEYWORD=value
```

Allowed keywords are as follows:

- **sk**. The status of the item, where the **value** is one of the following strings:
 - LOW, the item is on its lower bound.
 - UPR, the item is on its upper bound.
 - FIX, it is a fixed item.
 - BAS, the item is in the basis.
 - SUPBAS, the item is super basic.
 - UNK, the status is unknown.
 - INF, the item is outside its bounds (infeasible).
- **lv1** Defines the level of the item.
- **s1** Defines the level of the dual variable associated with its lower bound.
- **su** Defines the level of the dual variable associated with its upper bound.
- **sn** Defines the level of the variable associated with its cone.
- **y** Defines the level of the corresponding dual variable (for constraints only).

A [var] section should always contain the items **sk**, **lv1**, **s1** and **su**. Items **s1** and **su** are not required for integer solutions.

A [con] section should always contain **sk**, **lv1**, **s1**, **su** and **y**.

An example of a solution section

```
[solution basic status=UNKNOWN]
[var x0] sk=LOW    lv1=5.0    [/var]
[var x1] sk=UPR    lv1=10.0   [/var]
[var x2] sk=SUPBAS lv1=2.0    s1=1.5 su=0.0 [/var]

[con c0] sk=LOW    lv1=3.0 y=0.0 [/con]
[con c0] sk=UPR    lv1=0.0 y=5.0 [/con]
[/solution]
```

- **[vendor]** This contains solver/vendor specific data. It accepts one argument, which is a vendor ID – for **MOSEK** the ID is simply **mosek** – and the section contains the subsection **parameters** defining solver parameters. When reading a vendor section, any unknown vendor can be safely ignored. This is described later.

Comments using the **#** may appear anywhere in the file. Between the **#** and the following line-break any text may be written, including markup characters.

15.3.3 Numbers

Numbers, when used for parameter values or coefficients, are written in the usual way by the **printf** function. That is, they may be prefixed by a sign (+ or -) and may contain an integer part, decimal part and an exponent. The decimal point is always **.** (a dot). Some examples are

```
1
1.0
.0
1.
1e10
1e+10
1e-10
```

Some *invalid* examples are

```
e10    # invalid, must contain either integer or decimal part
.       # invalid
.e10   # invalid
```

More formally, the following standard regular expression describes numbers as used:

```
[+|-]?([0-9]+[.][0-9]*|.[0-9]+)([eE][+|-]?[0-9]+)?
```

15.3.4 Names

Variable names, constraint names and objective name may contain arbitrary characters, which in some cases must be enclosed by quotes (single or double) that in turn must be preceded by a backslash. Unquoted names must begin with a letter (**a-z** or **A-Z**) and contain only the following characters: the letters **a-z** and **A-Z**, the digits **0-9**, braces (**{** and **}**) and underscore (**_**).

Some examples of legal names:

```
an_unquoted_name
another_name{123}
'single quoted name'
"double quoted name"
"name with \"quote\" in it"
"name with []s in it"
```

15.3.5 Parameters Section

In the **vendor** section solver parameters are defined inside the **parameters** subsection. Each parameter is written as

```
[p PARAMETER_NAME] value [/p]
```

where **PARAMETER_NAME** is replaced by a **MOSEK** parameter name, usually of the form **MSK_IPAR_...**, **MSK_DPAR_...** or **MSK_SPAR_...**, and the **value** is replaced by the value of that parameter; both integer values and named values may be used. Some simple examples are

```
[vendor mosek]
[parameters]
[p MSK_IPAR_OPF_MAX_TERMS_PER_LINE] 10      [/p]
[p MSK_IPAR_OPF_WRITE_PARAMETERS]    MSK_ON [/p]
[p MSK_DPAR_DATA_TOL_BOUND_INF]      1.0e18 [/p]
[/parameters]
[/vendor]
```

15.3.6 Writing OPF Files from MOSEK

To write an OPF file add the `.opf` extension to the file name.

15.3.7 Examples

This section contains a set of small examples written in OPF and describing how to formulate linear, quadratic and conic problems.

Linear Example `lo1.opf`

Consider the example:

$$\begin{array}{llllll} \text{maximize} & 3x_0 & + & 1x_1 & + & 5x_2 & + & 1x_3 \\ \text{subject to} & 3x_0 & + & 1x_1 & + & 2x_2 & & = & 30, \\ & 2x_0 & + & 1x_1 & + & 3x_2 & + & 1x_3 & \geq & 15, \\ & & & 2x_1 & & & + & 3x_3 & \leq & 25, \end{array}$$

having the bounds

$$\begin{array}{llll} 0 & \leq & x_0 & \leq & \infty, \\ 0 & \leq & x_1 & \leq & 10, \\ 0 & \leq & x_2 & \leq & \infty, \\ 0 & \leq & x_3 & \leq & \infty. \end{array}$$

In the OPF format the example is displayed as shown in [Listing 15.1](#).

Listing 15.1: Example of an OPF file for a linear problem.

```
[comment]
  The lo1 example in OPF format
[/comment]

[hints]
  [hint NUMVAR] 4 [/hint]
  [hint NUMCON] 3 [/hint]
  [hint NUMANZ] 9 [/hint]
[/hints]

[variables disallow_new_variables]
  x1 x2 x3 x4
[/variables]

[objective maximize 'obj']
  3 x1 + x2 + 5 x3 + x4
[/objective]

[constraints]
  [con 'c1'] 3 x1 +   x2 + 2 x3           = 30 [/con]
  [con 'c2'] 2 x1 +   x2 + 3 x3 +   x4 >= 15 [/con]
```

(continues on next page)

```

[con 'c3']          2 x2          + 3 x4 <= 25 [/con]
[/constraints]

[bounds]
[b] 0 <= * [/b]
[b] 0 <= x2 <= 10 [/b]
[/bounds]

```

Quadratic Example qo1.opf

An example of a quadratic optimization problem is

$$\begin{aligned}
 &\text{minimize} && x_1^2 + 0.1x_2^2 + x_3^2 - x_1x_3 - x_2 \\
 &\text{subject to} && 1 \leq x_1 + x_2 + x_3, \\
 &&& x \geq 0.
 \end{aligned}$$

This can be formulated in `opf` as shown below.

Listing 15.2: Example of an OPF file for a quadratic problem.

```

[comment]
  The qo1 example in OPF format
[/comment]

[hints]
[hint NUMVAR] 3 [/hint]
[hint NUMCON] 1 [/hint]
[hint NUMANZ] 3 [/hint]
[hint NUMQNZ] 4 [/hint]
[/hints]

[variables disallow_new_variables]
  x1 x2 x3
[/variables]

[objective minimize 'obj']
  # The quadratic terms are often written with a factor of 1/2 as here,
  # but this is not required.

  - x2 + 0.5 ( 2.0 x1 ^ 2 - 2.0 x3 * x1 + 0.2 x2 ^ 2 + 2.0 x3 ^ 2 )
[/objective]

[constraints]
[con 'c1'] 1.0 <= x1 + x2 + x3 [/con]
[/constraints]

[bounds]
[b] 0 <= * [/b]
[/bounds]

```

Conic Quadratic Example `cqo1.opf`

Consider the example:

$$\begin{array}{llll} \text{minimize} & x_3 + x_4 + x_5 & & \\ \text{subject to} & x_0 + x_1 + 2x_2 & = & 1, \\ & x_0, x_1, x_2 & \geq & 0, \\ & x_3 \geq \sqrt{x_0^2 + x_1^2}, & & \\ & 2x_4x_5 \geq x_2^2. & & \end{array}$$

Please note that the type of the cones is defined by the parameter to `[cone ...]`; the content of the `cone`-section is the names of variables that belong to the cone. The resulting OPF file is in [Listing 15.3](#).

Listing 15.3: Example of an OPF file for a conic quadratic problem.

```
[comment]
  The cqo1 example in OPF format.
[/comment]

[hints]
  [hint NUMVAR] 6 [/hint]
  [hint NUMCON] 1 [/hint]
  [hint NUMANZ] 3 [/hint]
[/hints]

[variables disallow_new_variables]
  x1 x2 x3 x4 x5 x6
[/variables]

[objective minimize 'obj']
  x4 + x5 + x6
[/objective]

[constraints]
  [con 'c1'] x1 + x2 + 2e+00 x3 = 1e+00 [/con]
[/constraints]

[bounds]
  # We let all variables default to the positive orthant
  [b] 0 <= * [/b]

  # ...and change those that differ from the default
  [b] x4,x5,x6 free [/b]

  # Define quadratic cone: x4 >= sqrt( x1^2 + x2^2 )
  [cone quad 'k1'] x4, x1, x2 [/cone]

  # Define rotated quadratic cone: 2 x5 x6 >= x3^2
  [cone rquad 'k2'] x5, x6, x3 [/cone]
[/bounds]
```

Mixed Integer Example milo1.opf

Consider the mixed integer problem:

$$\begin{array}{llll} \text{maximize} & x_0 + 0.64x_1 & & \\ \text{subject to} & 50x_0 + 31x_1 & \leq & 250, \\ & 3x_0 - 2x_1 & \geq & -4, \\ & x_0, x_1 \geq 0 & & \text{and integer} \end{array}$$

This can be implemented in OPF with the file in [Listing 15.4](#).

Listing 15.4: Example of an OPF file for a mixed-integer linear problem.

```
[comment]
  The milo1 example in OPF format
[/comment]

[hints]
  [hint NUMVAR] 2 [/hint]
  [hint NUMCON] 2 [/hint]
  [hint NUMANZ] 4 [/hint]
[/hints]

[variables disallow_new_variables]
  x1 x2
[/variables]

[objective maximize 'obj']
  x1 + 6.4e-1 x2
[/objective]

[constraints]
  [con 'c1'] 5e+1 x1 + 3.1e+1 x2 <= 2.5e+2 [/con]
  [con 'c2'] -4 <= 3 x1 - 2 x2 [/con]
[/constraints]

[bounds]
  [b] 0 <= * [/b]
[/bounds]

[integer]
  x1 x2
[/integer]
```

15.4 The CBF Format

This document constitutes the technical reference manual of the *Conic Benchmark Format* with file extension: `.cbf` or `.CBF`. It unifies linear, second-order cone (also known as conic quadratic), exponential cone, power cone and semidefinite optimization with mixed-integer variables. The format has been designed with benchmark libraries in mind, and therefore focuses on compact and easily parsable representations. The CBF format separates problem structure from the problem data.

15.4.1 How Instances Are Specified

This section defines the spectrum of conic optimization problems that can be formulated in terms of the keywords of the CBF format.

In the CBF format, conic optimization problems are considered in the following form:

$$\begin{aligned} & \min / \max && g^{obj} \\ & \text{s.t.} && \begin{aligned} & g_i \in \mathcal{K}_i, & i \in \mathcal{I}, \\ & G_i \in \mathcal{K}_i, & i \in \mathcal{I}^{PSD}, \\ & x_j \in \mathcal{K}_j, & j \in \mathcal{J}, \\ & \overline{X}_j \in \mathcal{K}_j, & j \in \mathcal{J}^{PSD}. \end{aligned} \end{aligned} \tag{15.10}$$

- **Variables** are either scalar variables, x_j for $j \in \mathcal{J}$, or matrix variables, $\overline{X}_j$ for $j \in \mathcal{J}^{PSD}$. Scalar variables can also be declared as integer.
- **Constraints** are affine expressions of the variables, either scalar-valued g_i for $i \in \mathcal{I}$, or matrix-valued G_i for $i \in \mathcal{I}^{PSD}$

$$\begin{aligned} g_i &= \sum_{j \in \mathcal{J}^{PSD}} \langle F_{ij}, X_j \rangle + \sum_{j \in \mathcal{J}} a_{ij} x_j + b_i, \\ G_i &= \sum_{j \in \mathcal{J}} x_j H_{ij} + D_i. \end{aligned}$$

- The **objective function** is a scalar-valued affine expression of the variables, either to be minimized or maximized. We refer to this expression as g^{obj}

$$g^{obj} = \sum_{j \in \mathcal{J}^{PSD}} \langle F_j^{obj}, X_j \rangle + \sum_{j \in \mathcal{J}} a_j^{obj} x_j + b^{obj}.$$

As of version 4 of the format, CBF files can represent the following non-parametric cones $\mathcal{K}$:

- **Free domain** - A cone in the linear family defined by

$$\{x \in \mathbb{R}^n\}, \text{ for } n \geq 1.$$

- **Positive orthant** - A cone in the linear family defined by

$$\{x \in \mathbb{R}^n \mid x_j \geq 0 \text{ for } j = 1, \dots, n\}, \text{ for } n \geq 1.$$

- **Negative orthant** - A cone in the linear family defined by

$$\{x \in \mathbb{R}^n \mid x_j \leq 0 \text{ for } j = 1, \dots, n\}, \text{ for } n \geq 1.$$

- **Fixpoint zero** - A cone in the linear family defined by

$$\{x \in \mathbb{R}^n \mid x_j = 0 \text{ for } j = 1, \dots, n\}, \text{ for } n \geq 1.$$

- **Quadratic cone** - A cone in the second-order cone family defined by

$$\left\{ \begin{pmatrix} p \\ x \end{pmatrix} \in \mathbb{R} \times \mathbb{R}^{n-1}, p^2 \geq x^T x, p \geq 0 \right\}, \text{ for } n \geq 2.$$

- **Rotated quadratic cone** - A cone in the second-order cone family defined by

$$\left\{ \begin{pmatrix} p \\ q \\ x \end{pmatrix} \in \mathbb{R} \times \mathbb{R} \times \mathbb{R}^{n-2}, 2pq \geq x^T x, p \geq 0, q \geq 0 \right\}, \text{ for } n \geq 3.$$

- **Exponential cone** - A cone in the exponential cone family defined by

$$\text{cl}(S_1) = S_1 \cup S_2$$

where,

$$S_1 = \left\{ \begin{pmatrix} t \\ s \\ r \end{pmatrix} \in \mathbb{R}^3, t \geq s e^{\frac{r}{s}}, s \geq 0 \right\}.$$

and,

$$S_2 = \left\{ \begin{pmatrix} t \\ s \\ r \end{pmatrix} \in \mathbb{R}^3, t \geq 0, r \leq 0, s = 0 \right\}.$$

- **Dual Exponential cone** - A cone in the exponential cone family defined by

$$\text{cl}(S_1) = S_1 \cup S_2$$

where,

$$S_1 = \left\{ \begin{pmatrix} t \\ s \\ r \end{pmatrix} \in \mathbb{R}^3, et \geq (-r)e^{\frac{s}{r}}, -r \geq 0 \right\}.$$

and,

$$S_2 = \left\{ \begin{pmatrix} t \\ s \\ r \end{pmatrix} \in \mathbb{R}^3, et \geq 0, s \geq 0, r = 0 \right\}.$$

- **Radial geometric mean cone** - A cone in the power cone family defined by

$$\left\{ \begin{pmatrix} p \\ x \end{pmatrix} \in \mathbb{R}_+^k \times \mathbb{R}^1, \left(\prod_{j=1}^k p_j \right)^{\frac{1}{k}} \geq |x| \right\}, \text{ for } n = k + 1 \geq 2.$$

- **Dual radial geometric mean cone** - A cone in the power cone family defined by

$$\left\{ \begin{pmatrix} p \\ x \end{pmatrix} \in \mathbb{R}_+^k \times \mathbb{R}^1, \left(\prod_{j=1}^k k p_j \right)^{\frac{1}{k}} \geq |x| \right\}, \text{ for } n = k + 1 \geq 2.$$

and, the following parametric cones:

- **Radial power cone** - A cone in the power cone family defined by

$$\left\{ \begin{pmatrix} p \\ x \end{pmatrix} \in \mathbb{R}_+^k \times \mathbb{R}^{n-k}, \left(\prod_{j=1}^k p_j^{\alpha_j} \right)^{\frac{1}{\sigma}} \geq \|x\|_2 \right\}, \text{ for } n \geq k \geq 1.$$

where, $\sigma = \sum_{j=1}^k \alpha_j$ and $\alpha = \mathbb{R}_{++}^k$.

- **Dual radial power cone** - A cone in the power cone family defined by

$$\left\{ \begin{pmatrix} p \\ x \end{pmatrix} \in \mathbb{R}_+^k \times \mathbb{R}^{n-k}, \left(\prod_{j=1}^k \left(\frac{\sigma p_j}{\alpha_j} \right)^{\alpha_j} \right)^{\frac{1}{\sigma}} \geq \|x\|_2 \right\}, \text{ for } n \geq k \geq 1.$$

where, $\sigma = \sum_{j=1}^k \alpha_j$ and $\alpha = \mathbb{R}_{++}^k$.

15.4.2 The Structure of CBF Files

This section defines how information is written in the CBF format, without being specific about the type of information being communicated.

All information items belong to exactly one of the three groups of information. These information groups, and the order they must appear in, are:

1. File format.
2. Problem structure.
3. Problem data.

The first group, file format, provides information on how to interpret the file. The second group, problem structure, provides the information needed to deduce the type and size of the problem instance. Finally, the third group, problem data, specifies the coefficients and constants of the problem instance.

Information items

The format is composed as a list of information items. The first line of an information item is the **KEYWORD**, revealing the type of information provided. The second line - of some keywords only - is the **HEADER**, typically revealing the size of information that follows. The remaining lines are the **BODY** holding the actual information to be specified.

```
KEYWORD
BODY
```

```
KEYWORD
HEADER
BODY
```

The **KEYWORD** determines how each line in the **HEADER** and **BODY** is structured. Moreover, the number of lines in the **BODY** follows either from the **KEYWORD**, the **HEADER**, or from another information item required to precede it.

File encoding and line width restrictions

The format is based on the US-ASCII printable character set with two extensions as listed below. Note, by definition, that none of these extensions can be misinterpreted as printable US-ASCII characters:

- A line feed marks the end of a line, carriage returns are ignored.
- Comment-lines may contain unicode characters in UTF-8 encoding.

The line width is restricted to 512 bytes, with 3 bytes reserved for the potential carriage return, line feed and null-terminator.

Integers and floating point numbers must follow the ISO C decimal string representation in the standard C locale. The format does not impose restrictions on the magnitude of, or number of significant digits in numeric data, but the use of 64-bit integers and 64-bit IEEE 754 floating point numbers should be sufficient to avoid loss of precision.

Comment-line and whitespace rules

The format allows single-line comments respecting the following rule:

- Lines having first byte equal to '#' (US-ASCII 35) are comments, and should be ignored. Comments are only allowed between information items.

Given that a line is not a comment-line, whitespace characters should be handled according to the following rules:

- Leading and trailing whitespace characters should be ignored.
 - The separator between multiple pieces of information on one line, is either one or more whitespace characters.
- Lines containing only whitespace characters are empty, and should be ignored. Empty lines are only allowed between information items.

15.4.3 Problem Specification

The problem structure

The problem structure defines the objective sense, whether it is minimization and maximization. It also defines the index sets, $\mathcal{J}$, $\mathcal{J}^{PSD}$, $\mathcal{I}$ and $\mathcal{I}^{PSD}$, which are all numbered from zero, $\{0, 1, \dots\}$, and empty until explicitly constructed.

- **Scalar variables** are constructed in vectors restricted to a conic domain, such as $(x_0, x_1) \in \mathbb{R}_+^2$, $(x_2, x_3, x_4) \in \mathcal{Q}^3$, etc. In terms of the Cartesian product, this generalizes to

$$x \in \mathcal{K}_1^{n_1} \times \mathcal{K}_2^{n_2} \times \dots \times \mathcal{K}_k^{n_k}$$

which in the CBF format becomes:

```
VAR
n k
K1 n1
K2 n2
...
Kk nk
```

where $\sum_i n_i = n$ is the total number of scalar variables. The list of supported cones is found in Table 15.3. Integrality of scalar variables can be specified afterwards.

- **PSD variables** are constructed one-by-one. That is, $X_j \succeq \mathbf{0}^{n_j \times n_j}$ for $j \in \mathcal{J}^{PSD}$, constructs a matrix-valued variable of size $n_j \times n_j$ restricted to be symmetric positive semidefinite. In the CBF format, this list of constructions becomes:

```

PSDVAR
N
n1
n2
...
nN

```

where N is the total number of PSD variables.

- **Scalar constraints** are constructed in vectors restricted to a conic domain, such as $(g_0, g_1) \in \mathbb{R}_+^2$, $(g_2, g_3, g_4) \in \mathcal{Q}^3$, etc. In terms of the Cartesian product, this generalizes to

$$g \in \mathcal{K}_1^{m_1} \times \mathcal{K}_2^{m_2} \times \dots \times \mathcal{K}_k^{m_k}$$

which in the CBF format becomes:

```

CON
m k
K1 m1
K2 m2
..
Kk mk

```

where $\sum_i m_i = m$ is the total number of scalar constraints. The list of supported cones is found in [Table 15.3](#).

- **PSD constraints** are constructed one-by-one. That is, $G_i \succeq \mathbf{0}^{m_i \times m_i}$ for $i \in \mathcal{I}^{PSD}$, constructs a matrix-valued affine expressions of size $m_i \times m_i$ restricted to be symmetric positive semidefinite. In the CBF format, this list of constructions becomes

```

PSDCON
M
m1
m2
..
mM

```

where M is the total number of PSD constraints.

With the objective sense, variables (with integer indications) and constraints, the definitions of the many affine expressions follow in problem data.

Problem data

The problem data defines the coefficients and constants of the affine expressions of the problem instance. These are considered zero until explicitly defined, implying that instances with no keywords from this information group are, in fact, valid. Duplicating or conflicting information is a failure to comply with the standard. Consequently, two coefficients written to the same position in a matrix (or to transposed positions in a symmetric matrix) is an error.

The affine expressions of the objective, g^{obj} , of the scalar constraints, g_i , and of the PSD constraints, G_i , are defined separately. The following notation uses the standard trace inner product for matrices, $\langle X, Y \rangle = \sum_{i,j} X_{ij} Y_{ij}$.

- The affine expression of the objective is defined as

$$g^{obj} = \sum_{j \in \mathcal{J}^{PSD}} \langle F_j^{obj}, X_j \rangle + \sum_{j \in \mathcal{J}} a_j^{obj} x_j + b^{obj},$$

in terms of the symmetric matrices, F_j^{obj} , and scalars, a_j^{obj} and b^{obj} .

- The affine expressions of the scalar constraints are defined, for $i \in \mathcal{I}$, as

$$g_i = \sum_{j \in \mathcal{J}^{PSD}} \langle F_{ij}, X_j \rangle + \sum_{j \in \mathcal{J}} a_{ij} x_j + b_i,$$

in terms of the symmetric matrices, F_{ij} , and scalars, a_{ij} and b_i .

- The affine expressions of the PSD constraints are defined, for $i \in \mathcal{I}^{PSD}$, as

$$G_i = \sum_{j \in \mathcal{J}} x_j H_{ij} + D_i,$$

in terms of the symmetric matrices, H_{ij} and D_i .

List of cones

The format uses an explicit syntax for symmetric positive semidefinite cones as shown above. For scalar variables and constraints, constructed in vectors, the supported conic domains and their sizes are given as follows.

Table 15.3: Cones available in the CBF format

Name	CBF keyword	Cone family	Cone size
Free domain	F	linear	$n \geq 1$
Positive orthant	L+	linear	$n \geq 1$
Negative orthant	L-	linear	$n \geq 1$
Fixpoint zero	L=	linear	$n \geq 1$
Quadratic cone	Q	second-order	$n \geq 1$
Rotated quadratic cone	QR	second-order	$n \geq 2$
Exponential cone	EXP	exponential	$n = 3$
Dual exponential cone	EXP*	exponential	$n = 3$
Radial geometric mean cone	GMEANABS	power	$n = k + 1 \geq 2$
Dual radial geometric mean cone	GMEANABS*	power	$n = k + 1 \geq 2$
Radial power cone (parametric)	POW	power	$n \geq k \geq 1$
Dual radial power cone (parametric)	POW*	power	$n \geq k \geq 1$

15.4.4 File Format Keywords

VER

Description: The version of the Conic Benchmark Format used to write the file.

HEADER: None

BODY: One line formatted as:

INT

This is the version number.

Must appear exactly once in a file, as the first keyword.

POWCONES

Description: Define a lookup table for power cone domains.

HEADER: One line formatted as:

INT INT

This is the number of cones to be specified and the combined length of their dense parameter vectors.

BODY: A list of chunks each specifying the dense parameter vector of a power cone.

CHUNKHEADER: One line formatted as:

INT

This is the parameter vector length.

CHUNKBODY: A list of lines formatted as:

REAL

This is the parameter vector values. The number of lines should match the number stated in the chunk header.

The cone specified at index k (with 0-based indexing) is registered under the CBF name @ k :POW.

POW*CONES

Description: Define a lookup table for dual power cone domains.

HEADER: One line formatted as:

INT INT

This is the number of cones to be specified and the combined length of their dense parameter vectors.

BODY: A list of chunks each specifying the dense parameter vector of a dual power cone.

CHUNKHEADER: One line formatted as:

INT

This is the parameter vector length.

CHUNKBODY: A list of lines formatted as:

REAL

This is the parameter vector values. The number of lines should match the number stated in the chunk header.

The cone specified at index k (with 0-based indexing) is registered under the CBF name @ k :POW*.

OBJSENSE

Description: Define the objective sense.

HEADER: None

BODY: One line formatted as:

STR

having MIN indicates minimize, and MAX indicates maximize. Upper-case letters are required.

Must appear exactly once in a file.

PSDVAR

Description: Construct the PSD variables.

HEADER: One line formatted as:

INT

This is the number of PSD variables in the problem.

BODY: A list of lines formatted as:

INT

This indicates the number of rows (equal to the number of columns) in the matrix-valued PSD variable. The number of lines should match the number stated in the header.

VAR

Description: Construct the scalar variables.

HEADER: One line formatted as:

INT INT

This is the number of scalar variables, followed by the number of conic domains they are restricted to.

BODY: A list of lines formatted as:

STR INT

This indicates the cone name (see [Table 15.3](#)), and the number of scalar variables restricted to this cone. These numbers should add up to the number of scalar variables stated first in the header. The number of lines should match the second number stated in the header.

INT

Description: Declare integer requirements on a selected subset of scalar variables.

HEADER: one line formatted as:

INT

This is the number of integer scalar variables in the problem.

BODY: a list of lines formatted as:

INT

This indicates the scalar variable index $j \in \mathcal{J}$. The number of lines should match the number stated in the header.

Can only be used after the keyword **VAR**.

PSDCON

Description: Construct the PSD constraints.

HEADER: One line formatted as:

INT

This is the number of PSD constraints in the problem.

BODY: A list of lines formatted as:

INT

This indicates the number of rows (equal to the number of columns) in the matrix-valued affine expression of the PSD constraint. The number of lines should match the number stated in the header.

Can only be used after these keywords: **PSDVAR**, **VAR**.

CON

Description: Construct the scalar constraints.

HEADER: One line formatted as:

INT INT

This is the number of scalar constraints, followed by the number of conic domains they restrict to.

BODY: A list of lines formatted as:

STR INT

This indicates the cone name (see [Table 15.3](#)), and the number of affine expressions restricted to this cone. These numbers should add up to the number of scalar constraints stated first in the header. The number of lines should match the second number stated in the header.

Can only be used after these keywords: **PSDVAR**, **VAR**

OBJFCOORD

Description: Input sparse coordinates (quadruplets) to define the symmetric matrices F_j^{obj} , as used in the objective.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT INT INT REAL

This indicates the PSD variable index $j \in \mathcal{J}^{PSD}$, the row index, the column index and the coefficient value. The number of lines should match the number stated in the header.

OBJACOORD

Description: Input sparse coordinates (pairs) to define the scalars, a_j^{obj} , as used in the objective.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT REAL

This indicates the scalar variable index $j \in \mathcal{J}$ and the coefficient value. The number of lines should match the number stated in the header.

OBJBCOORD

Description: Input the scalar, b^{obj} , as used in the objective.

HEADER: None.

BODY: One line formatted as:

REAL

This indicates the coefficient value.

FCOORD

Description: Input sparse coordinates (quintuplets) to define the symmetric matrices, F_{ij} , as used in the scalar constraints.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT INT INT INT REAL

This indicates the scalar constraint index $i \in \mathcal{I}$, the PSD variable index $j \in \mathcal{J}^{PSD}$, the row index, the column index and the coefficient value. The number of lines should match the number stated in the header.

ACCOORD

Description: Input sparse coordinates (triplets) to define the scalars, a_{ij} , as used in the scalar constraints.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT INT REAL

This indicates the scalar constraint index $i \in \mathcal{I}$, the scalar variable index $j \in \mathcal{J}$ and the coefficient value. The number of lines should match the number stated in the header.

BCOORD

Description: Input sparse coordinates (pairs) to define the scalars, b_i , as used in the scalar constraints.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT REAL

This indicates the scalar constraint index $i \in \mathcal{I}$ and the coefficient value. The number of lines should match the number stated in the header.

HCOORD

Description: Input sparse coordinates (quintuplets) to define the symmetric matrices, H_{ij} , as used in the PSD constraints.

HEADER: One line formatted as:

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as

INT INT INT INT REAL

This indicates the PSD constraint index $i \in \mathcal{I}^{PSD}$, the scalar variable index $j \in \mathcal{J}$, the row index, the column index and the coefficient value. The number of lines should match the number stated in the header.

DCOORD

Description: Input sparse coordinates (quadruplets) to define the symmetric matrices, D_i , as used in the PSD constraints.

HEADER: One line formatted as

INT

This is the number of coordinates to be specified.

BODY: A list of lines formatted as:

INT INT INT REAL

This indicates the PSD constraint index $i \in \mathcal{I}^{PSD}$, the row index, the column index and the coefficient value. The number of lines should match the number stated in the header.

15.4.5 CBF Format Examples

Minimal Working Example

The conic optimization problem (15.11) , has three variables in a quadratic cone - first one is integer - and an affine expression in domain 0 (equality constraint).

$$\begin{aligned} & \text{minimize} && 5.1 x_0 \\ & \text{subject to} && 6.2 x_1 + 7.3 x_2 - 8.4 \in \{0\} \\ & && x \in \mathcal{Q}^3, x_0 \in \mathbb{Z}. \end{aligned} \tag{15.11}$$

Its formulation in the Conic Benchmark Format begins with the version of the CBF format used, to safeguard against later revisions.

```
VER
4
```

Next follows the problem structure, consisting of the objective sense, the number and domain of variables, the indices of integer variables, and the number and domain of scalar-valued affine expressions (i.e., the equality constraint).

```
OBJSENSE
MIN

VAR
3 1
Q 3

INT
1
0

CON
1 1
L= 1
```

Finally follows the problem data, consisting of the coefficients of the objective, the coefficients of the constraints, and the constant terms of the constraints. All data is specified on a sparse coordinate form.

```
OBJCOORD
1
0 5.1

ACCOORD
2
0 1 6.2
0 2 7.3

BCCOORD
1
0 -8.4
```

This concludes the example.

Mixing Linear, Second-order and Semidefinite Cones

The conic optimization problem (15.12), has a semidefinite cone, a quadratic cone over unordered subindices, and two equality constraints.

$$\begin{aligned}
 & \text{minimize} && \left\langle \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}, X_1 \right\rangle + x_1 \\
 & \text{subject to} && \left\langle \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, X_1 \right\rangle + x_1 &= 1.0, \\
 & && \left\langle \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, X_1 \right\rangle + x_0 + x_2 &= 0.5, \\
 & && x_1 \geq \sqrt{x_0^2 + x_2^2}, \\
 & && X_1 \succeq \mathbf{0}.
 \end{aligned} \tag{15.12}$$

The equality constraints are easily rewritten to the conic form, $(g_0, g_1) \in \{0\}^2$, by moving constants such that the right-hand-side becomes zero. The quadratic cone does not fit under the **VAR** keyword in this variable permutation. Instead, it takes a scalar constraint $(g_2, g_3, g_4) = (x_1, x_0, x_2) \in \mathcal{Q}^3$, with scalar variables constructed as $(x_0, x_1, x_2) \in \mathbb{R}^3$. Its formulation in the CBF format is reported in the following list

```

# File written using this version of the Conic Benchmark Format:
#       | Version 4.
VER
4

# The sense of the objective is:
#       | Minimize.
OBJSENSE
MIN

# One PSD variable of this size:
#       | Three times three.
PSDVAR
1
3

# Three scalar variables in this one conic domain:
#       | Three are free.
VAR
3 1
F 3

# Five scalar constraints with affine expressions in two conic domains:
#       | Two are fixed to zero.
#       | Three are in conic quadratic domain.
CON
5 2
L= 2
Q 3

# Five coordinates in F~{obj}_j coefficients:
#       | F~{obj}[0][0,0] = 2.0
#       | F~{obj}[0][1,0] = 1.0
#       | and more...
OBJFCOORD
5

```

(continues on next page)

```

0 0 0 2.0
0 1 0 1.0
0 1 1 2.0
0 2 1 1.0
0 2 2 2.0

# One coordinate in a{obj}j coefficients:
#       | a{obj}[1] = 1.0
OBJCOORD
1
1 1.0

# Nine coordinates in Fij coefficients:
#       | F[0,0][0,0] = 1.0
#       | F[0,0][1,1] = 1.0
#       | and more...
FCOORD
9
0 0 0 0 1.0
0 0 1 1 1.0
0 0 2 2 1.0
1 0 0 0 1.0
1 0 1 0 1.0
1 0 2 0 1.0
1 0 1 1 1.0
1 0 2 1 1.0
1 0 2 2 1.0

# Six coordinates in aij coefficients:
#       | a[0,1] = 1.0
#       | a[1,0] = 1.0
#       | and more...
ACCOORD
6
0 1 1.0
1 0 1.0
1 2 1.0
2 1 1.0
3 0 1.0
4 2 1.0

# Two coordinates in bi coefficients:
#       | b[0] = -1.0
#       | b[1] = -0.5
BCOORD
2
0 -1.0
1 -0.5

```

Mixing Semidefinite Variables and Linear Matrix Inequalities

The standard forms in semidefinite optimization are usually based either on semidefinite variables or linear matrix inequalities. In the CBF format, both forms are supported and can even be mixed as shown.

$$\begin{aligned}
 & \text{minimize} && \left\langle \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, X_1 \right\rangle + x_1 + x_2 + 1 \\
 & \text{subject to} && \left\langle \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, X_1 \right\rangle - x_1 - x_2 && \geq 0.0, \\
 & && x_1 \begin{bmatrix} 0 & 1 \\ 1 & 3 \end{bmatrix} + x_2 \begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} && \succeq \mathbf{0}, \\
 & && X_1 \succeq \mathbf{0}.
 \end{aligned} \tag{15.13}$$

Its formulation in the CBF format is written in what follows

```

# File written using this version of the Conic Benchmark Format:
#   | Version 4.
VER
4

# The sense of the objective is:
#   | Minimize.
OBJSENSE
MIN

# One PSD variable of this size:
#   | Two times two.
PSDVAR
1
2

# Two scalar variables in this one conic domain:
#   | Two are free.
VAR
2 1
F 2

# One PSD constraint of this size:
#   | Two times two.
PSDCON
1
2

# One scalar constraint with an affine expression in this one conic domain:
#   | One is greater than or equal to zero.
CON
1 1
L+ 1

# Two coordinates in F^{obj}_j coefficients:
#   | F^{obj}[0][0,0] = 1.0
#   | F^{obj}[0][1,1] = 1.0
OBJFCOORD
2
0 0 0 1.0
0 1 1 1.0

# Two coordinates in a^{obj}_j coefficients:

```

(continues on next page)

```

#      | a^{obj}[0] = 1.0
#      | a^{obj}[1] = 1.0
OBJCOORD
2
0 1.0
1 1.0

# One coordinate in b^{obj} coefficient:
#      | b^{obj} = 1.0
OBJBCOORD
1.0

# One coordinate in F_{ij} coefficients:
#      | F[0,0][1,0] = 1.0
FCOORD
1
0 0 1 0 1.0

# Two coordinates in a_{ij} coefficients:
#      | a[0,0] = -1.0
#      | a[0,1] = -1.0
ACCOORD
2
0 0 -1.0
0 1 -1.0

# Four coordinates in H_{ij} coefficients:
#      | H[0,0][1,0] = 1.0
#      | H[0,0][1,1] = 3.0
#      | and more...
HCOORD
4
0 0 1 0 1.0
0 0 1 1 3.0
0 1 0 0 3.0
0 1 1 0 1.0

# Two coordinates in D_i coefficients:
#      | D[0][0,0] = -1.0
#      | D[0][1,1] = -1.0
DCCOORD
2
0 0 0 -1.0
0 1 1 -1.0

```

The exponential cone

The conic optimization problem (15.14), has one equality constraint, one quadratic cone constraint and an exponential cone constraint.

$$\begin{aligned} & \text{minimize} && x_0 - x_3 \\ & \text{subject to} && x_0 + 2x_1 - x_2 \in \{0\} \\ & && (5.0, x_0, x_1) \in \mathcal{Q}^3 \\ & && (x_2, 1.0, x_3) \in EXP. \end{aligned} \tag{15.14}$$

The nonlinear conic constraints enforce $\sqrt{x_0^2 + x_1^2} \leq 0.5$ and $x_3 \leq \log(x_2)$.

```
# File written using this version of the Conic Benchmark Format:
#       | Version 3.
VER
3

# The sense of the objective is:
#       | Minimize.
OBJSENSE
MIN

# Four scalar variables in this one conic domain:
#       | Four are free.
VAR
4 1
F 4

# Seven scalar constraints with affine expressions in three conic domains:
#       | One is fixed to zero.
#       | Three are in conic quadratic domain.
#       | Three are in exponential cone domain.
CON
7 3
L= 1
Q 3
EXP 3

# Two coordinates in a^{obj}_j coefficients:
#       | a^{obj}[0] = 1.0
#       | a^{obj}[3] = -1.0
OBJCOORD
2
0 1.0
3 -1.0

# Seven coordinates in a_ij coefficients:
#       | a[0,0] = 1.0
#       | a[0,1] = 2.0
#       | and more...
ACCOORD
7
0 0 1.0
0 1 2.0
0 2 -1.0
2 0 1.0
3 1 1.0
4 2 1.0
6 3 1.0
```

(continues on next page)

(continued from previous page)

```
# Two coordinates in b_i coefficients:
#       | b[1] = 5.0
#       | b[5] = 1.0
BCCOORD
2
1 5.0
5 1.0
```

Parametric cones

The problem (15.15), has three variables in a power cone with parameter $\alpha_1 = (1, 1)$ and two power cone constraints each with parameter $\alpha_0 = (8, 1)$.

$$\begin{aligned} & \text{minimize} && x_3 \\ & \text{subject to} && (1.0, x_1, x_1 + x_2) \in POW_{\alpha_0} \\ & && (1.0, x_2, x_1 + x_2) \in POW_{\alpha_0} \\ & && x \in POW_{\alpha_1}. \end{aligned} \tag{15.15}$$

The nonlinear conic constraints enforce $x_3 \leq x_1 x_2$ and $x_1 + x_2 \leq \min(x_1^{\frac{1}{9}}, x_2^{\frac{1}{9}})$.

```
# File written using this version of the Conic Benchmark Format:
#       | Version 3.
VER
3

# Two power cone domains defined in a total of four parameters:
#       | @0:POW (specification 0) has two parameters:
#       | alpha[0] = 8.0.
#       | alpha[1] = 1.0.
#       | @1:POW (specification 1) has two parameters:
#       | alpha[0] = 1.0.
#       | alpha[1] = 1.0.
POWCONES
2 4
2
8.0
1.0
2
1.0
1.0

# The sense of the objective is:
#       | Maximize.
OBJSENSE
MAX

# Three scalar variable in this one conic domain:
#       | Three are in power cone domain (specification 1).
VAR
3 1
@1:POW 3

# Six scalar constraints with affine expressions in two conic domains:
#       | Three are in power cone domain (specification 0).
#       | Three are in power cone domain (specification 0).
```

(continues on next page)

```

CON
6 2
@0:POW 3
@0:POW 3

# One coordinate in a^{obj}_j coefficients:
#       | a^{obj}[2] = 1.0
OBJCOORD
1
2 1.0

# Six coordinates in a_ij coefficients:
#       | a[1,0] = 1.0
#       | a[2,0] = 1.0
#       | and more...
ACCOORD
6
1 0 1.0
2 0 1.0
2 1 1.0
4 1 1.0
5 0 1.0
5 1 1.0

# Two coordinates in b_i coefficients:
#       | b[0] = 1.0
#       | b[3] = 1.0
BCCOORD
2
0 1.0
3 1.0

```

15.5 The PTF Format

The PTF format is a human-readable, natural text format supporting all of **MOSEK** optimization problems in conic form, possibly with integer variables and disjunctive constraints.

15.5.1 The overall format

The format is indentation based, where each section is started by a head line and followed by a section body with deeper indentation than the head line. For example:

```

Header line
  Body line 1
  Body line 1
  Body line 1

```

Section can also be nested:

```

Header line A
  Body line in A
  Header line A.1
    Body line in A.1
    Body line in A.1
  Body line in A

```


The indentation of blank lines is ignored, so a subsection can contain a blank line with no indentation. The character # defines a line comment and anything between the # character and the end of the line is ignored.

In a PTF file, the first section must be a **Task** section. The order of the remaining section is arbitrary, and sections may occur multiple times or not at all.

MOSEK will ignore any top-level section it does not recognize.

Names

In the description of the format we use following definitions for name strings:

```
NAME: PLAIN_NAME | QUOTED_NAME
PLAIN_NAME: [a-zA-Z_] [a-zA-Z0-9_-.!|]
QUOTED_NAME: '"' ( [^'\\r\n] | "\\" ( [\\rn] | "x" [0-9a-fA-F] [0-9a-fA-F] ) ) * "'
```

Expressions

An expression is a sum of terms. A term is either a linear term (a coefficient and a variable name, where the coefficient can be left out if it is 1.0), or a matrix inner product.

An expression:

```
EXPR: EMPTY | [+]? TERM ( [+]? TERM ) *
TERM: LINEAR_TERM | MATRIX_TERM
```

A linear term

```
LINEAR_TERM: FLOAT? NAME
```

A matrix term

```
MATRIX_TERM: "<" FLOAT? NAME ( [+]? FLOAT? NAME ) * ";" NAME ">"
```

Here the right-hand name is the name of a (semidefinite) matrix variable, and the left-hand side is a sum of symmetric matrixes. The actual matrixes are defined in a separate section.

Expressions can span multiple lines by giving subsequent lines a deeper indentation.

For example following two section are equivalent:

```
# Everything on one line:
x1 + x2 + x3 + x4

# Split into multiple lines:
x1
  + x2
  + x3
  + x4
```

15.5.2 Task section

The first section of the file must be a **Task**. The text in this section is not used and may contain comments, or meta-information from the writer or about the content.

Format:

```
Task NAME
  Anything goes here...
```

NAME is a the task name.

15.5.3 Objective section

The **Objective** section defines the objective name, sense and function. The format:

```
"Objective" NAME?  
  ( "Minimize" | "Maximize" ) EXPR
```

For example:

```
Objective 'obj'  
  Minimize x1 + 0.2 x2 + < M1 ; X1 >
```

15.5.4 Constraints section

The constraints section defines a series of constraints. A constraint defines a term $A \cdot x + b \in K$. For linear constraints A is just one row, while for conic constraints it can be multiple rows. If a constraint spans multiple rows these can either be written inline separated by semi-colons, or each expression in a separate sub-section.

Simple linear constraints:

```
"Constraints"  
  NAME? "[" [-+] (FLOAT | "Inf") (";" [-+] (FLOAT | "Inf"))? "]" EXPR
```

If the brackets contain two values, they are used as upper and lower bounds. If they contain one value the constraint is an equality.

For example:

```
Constraints  
'c1' [0;10] x1 + x2 + x3  
[0] x1 + x2 + x3
```

Constraint blocks put the expression either in a subsection or inline. The cone type (domain) is written in the brackets, and **MOSEK** currently supports following types:

- SOC(N) Second order cone of dimension N
- RSOC(N) Rotated second order cone of dimension N
- PSD(N) Symmetric positive semidefinite cone of dimension N. This contains $N \cdot (N+1) / 2$ elements.
- PEXP Primal exponential cone of dimension 3
- DEXP Dual exponential cone of dimension 3
- PPOW(N,P) Primal power cone of dimension N with parameter P
- DPOW(N,P) Dual power cone of dimension N with parameter P
- ZERO(N) The zero-cone of dimension N.

```
"Constraints"  
  NAME? "[" DOMAIN "]" EXPR_LIST
```

For example:

```
Constraints  
'K1' [SOC(3)] x1 + x2 ; x2 + x3 ; x3 + x1  
'K2' [RSOC(3)]  
  x1 + x2  
  x2 + x3  
  x3 + x1
```

15.5.5 Variables section

Any variable used in an expression must be defined in a variable section. The variable section defines each variable domain.

```
"Variables"
  NAME "[" [-+] (FLOAT | "Inf") (";" [-+] (FLOAT | "Inf"))? "]"
  NAME "[" DOMAIN "]" NAMES
```

For example, a linear variable

```
Variables
  x1 [0;Inf]
```

As with constraints, members of a conic domain can be listed either inline or in a subsection:

```
Variables
  k1 [SOC(3)] x1 ; x2 ; x3
  k2 [RSOC(3)]
    x1
    x2
    x3
```

15.5.6 Integer section

This section contains a list of variables that are integral. For example:

```
Integer
  x1 x2 x3
```

15.5.7 SymmetricMatrixes section

This section defines the symmetric matrixes used for matrix coefficients in matrix inner product terms. The section lists named matrixes, each with a size and a number of non-zeros. Only non-zeros in the lower triangular part should be defined.

```
"SymmetricMatrixes"
  NAME "SYMMAT" "(" INT ")" ( "(" INT "," INT "," FLOAT ")" ) *
  ...
```

For example:

```
SymmetricMatrixes
  M1 SYMMAT(3) (0,0,1.0) (1,1,2.0) (2,1,0.5)
  M2 SYMMAT(3)
    (0,0,1.0)
    (1,1,2.0)
    (2,1,0.5)
```

15.5.8 Solutions section

Each subsection defines a solution. A solution defines for each constraint and for each variable exactly one primal value and either one (for conic domains) or two (for linear domains) dual values. The values follow the same logic as in the **MOSEK C API**. A primal and a dual solution status defines the meaning of the values primal and dual (solution, certificate, unknown, etc.)

The format is this:

```
"Solutions"
  "Solution" WHICHSOL
    "ProblemStatus" PROSTA PROSTA?
  "SolutionStatus" SOLSTA SOLSTA?
  "Objective" FLOAT FLOAT
  "Variables"
    # Linear variable status: level, slx, sux
    NAME "[" STATUS "]" FLOAT (FLOAT FLOAT)?
    # Conic variable status: level, snx
    NAME
      "[" STATUS "]" FLOAT FLOAT?
    ...
  "Constraints"
    # Linear variable status: level, slx, sux
    NAME "[" STATUS "]" FLOAT (FLOAT FLOAT)?
    # Conic variable status: level, snx
    NAME
      "[" STATUS "]" FLOAT FLOAT?
    ...
```

Following values for WHICHSOL are supported:

- **interior** Interior solution, the result of an interior-point solver.
- **basic** Basic solution, as produced by a simplex solver.
- **integer** Integer solution, the solution to a mixed-integer problem. This does not define a dual solution.

Following values for PROSTA are supported:

- **unknown** The problem status is unknown
- **feasible** The problem has been proven feasible
- **infeasible** The problem has been proven infeasible
- **illposed** The problem has been proved to be ill posed
- **infeasible_or_unbounded** The problem is infeasible or unbounded

Following values for SOLSTA are supported:

- **unknown** The solution status is unknown
- **feasible** The solution is feasible
- **optimal** The solution is optimal
- **infeas_cert** The solution is a certificate of infeasibility
- **illposed_cert** The solution is a certificate of illposedness

Following values for STATUS are supported:

- **unknown** The value is unknown
- **super_basic** The value is super basic

- `at_lower` The value is basic and at its lower bound
- `at_upper` The value is basic and at its upper bound
- `fixed` The value is basic fixed
- `infinite` The value is at infinity

15.5.9 Examples

Linear example `lo1.ptf`

```
Task ''
# Written by MOSEK v10.0.13
# problemtype: Linear Problem
# number of linear variables: 4
# number of linear constraints: 3
# number of old-style A nonzeros: 9
Objective obj
  Maximize + 3 x1 + x2 + 5 x3 + x4
Constraints
  c1 [3e+1] + 3 x1 + x2 + 2 x3
  c2 [1.5e+1;+inf] + 2 x1 + x2 + 3 x3 + x4
  c3 [-inf;2.5e+1] + 2 x2 + 3 x4
Variables
  x1 [0;+inf]
  x2 [0;1e+1]
  x3 [0;+inf]
  x4 [0;+inf]
```

Conic example `cq01.ptf`

```
Task ''
# Written by MOSEK v10.0.17
# problemtype: Conic Problem
# number of linear variables: 6
# number of linear constraints: 1
# number of old-style cones: 0
# number of positive semidefinite variables: 0
# number of positive semidefinite matrixes: 0
# number of affine conic constraints: 2
# number of disjunctive constraints: 0
# number scalar affine expressions/nonzeros : 6/6
# number of old-style A nonzeros: 3
Objective obj
  Minimize + x4 + x5 + x6
Constraints
  c1 [1] + x1 + x2 + 2 x3
  k1 [QUAD(3)]
    @ac1: + x4
    @ac2: + x1
    @ac3: + x2
  k2 [RQUAD(3)]
    @ac4: + x5
    @ac5: + x6
    @ac6: + x3
Variables
```

(continues on next page)

```

x4
x1 [0;+inf]
x2 [0;+inf]
x5
x6
x3 [0;+inf]

```

Disjunctive example djc1.ptf

```

Task djc1
Objective ''
  Minimize + 2 'x[0]' + 'x[1]' + 3 'x[2]' + 'x[3]'
Constraints
  @c0 [-10;+inf] + 'x[0]' + 'x[1]' + 'x[2]' + 'x[3]'
  @D0 [OR]
    [AND]
      [NEGATIVE(1)]
        + 'x[0]' - 2 'x[1]' + 1
      [ZERO(2)]
        + 'x[2]'
        + 'x[3]'
    [AND]
      [NEGATIVE(1)]
        + 'x[2]' - 3 'x[3]' + 2
      [ZERO(2)]
        + 'x[0]'
        + 'x[1]'
  @D1 [OR]
    [ZERO(1)]
      + 'x[0]' - 2.5
    [ZERO(1)]
      + 'x[1]' - 2.5
    [ZERO(1)]
      + 'x[2]' - 2.5
    [ZERO(1)]
      + 'x[3]' - 2.5
Variables
  'x[0]'
  'x[1]'
  'x[2]'
  'x[3]'

```

15.6 The Task Format

The Task format is **MOSEK**'s native binary format. It contains a complete image of a **MOSEK** task, i.e.

- Problem data: Linear, conic, semidefinite and quadratic data
- Problem item names: Variable names, constraints names, cone names etc.
- Parameter settings
- Solutions

There are a few things to be aware of:

- Status of a solution read from a file will *always* be unknown.
- Parameter settings in a task file *always override* any parameters set on the command line or in a parameter file.

The format is based on the *TAR* (USTar) file format. This means that the individual pieces of data in a `.task` file can be examined by unpacking it as a *TAR* file. Please note that the inverse may not work: Creating a file using *TAR* will most probably not create a valid **MOSEK** Task file since the order of the entries is important.

15.7 The JSON Format

MOSEK provides the possibility to read/write problems and solutions in JSON format. The official JSON website <http://www.json.org> provides plenty of information along with the format definition. JSON is an industry standard for data exchange and JSON files can be easily written and read in most programming languages using dedicated libraries.

MOSEK uses two JSON-based formats:

- **JTASK**, for storing problem instances together with solutions and parameters. The JTASK format contains the same information as a native **MOSEK** task *task format*, that is a very close representation of the internal data storage in the task object.

You can write a JTASK file specifying the extension `.jtask`. When the parameter `writeJsonIndentation` is set the JTASK file will be indented to slightly improve readability.

- **JSOL**, for storing solutions and information items.

It is not directly accessible via Fusion API for C++ but only from the lower-level Optimizer API and command line tools.

15.7.1 JTASK Specification

The JTASK is a dictionary containing the following sections. All sections are optional and can be omitted if irrelevant for the problem.

- **\$schema**: JSON schema.
- **Task/name**: The name of the task (string).
- **Task/INFO**: Information about problem data dimensions and similar. These are treated as hints when reading the file.
 - **numvar**: number of variables (int32).
 - **numcon**: number of constraints (int32).
 - **numcone**: number of cones (int32, deprecated).
 - **numbarvar**: number of symmetric matrix variables (int32).
 - **numanz**: number of nonzeros in A (int64).
 - **numsymmat**: number of matrices in the symmetric matrix storage E (int64).
 - **numafe**: number of affine expressions in AFE storage (int64).
 - **numfnz**: number of nonzeros in F (int64).
 - **numacc**: number of affine conic constraints (ACCs) (int64).
 - **numdjcc**: number of disjunctive constraints (DJCs) (int64).
 - **numdom**: number of domains (int64).
 - **mosekver**: MOSEK version (list(int32)).
- **Task/data**: Numerical and structural data of the problem.
 - **var**: Information about variables. All fields present must have the same length as **bk**. All or none of **bk**, **bl**, and **bu** must appear.

- * **name**: Variable names (list(string)).
- * **bk**: Bound keys (list(string)).
- * **bl**: Lower bounds (list(double)).
- * **bu**: Upper bounds (list(double)).
- * **type**: Variable types (list(string)).
- **con**: Information about linear constraints. All fields present must have the same length as **bk**. All or none of **bk**, **bl**, and **bu** must appear.
 - * **name**: Constraint names (list(string)).
 - * **bk**: Bound keys (list(string)).
 - * **bl**: Lower bounds (list(double)).
 - * **bu**: Upper bounds (list(double)).
- **barvar**: Information about symmetric matrix variables. All fields present must have the same length as **dim**.
 - * **name**: Barvar names (list(string)).
 - * **dim**: Dimensions (list(int32)).
- **objective**: Information about the objective.
 - * **name**: Objective name (string).
 - * **sense**: Objective sense (string).
 - * **c**: The linear part c of the objective as a sparse vector. Both arrays must have the same length.
 - **subj**: indices of nonzeros (list(int32)).
 - **val**: values of nonzeros (list(double)).
 - * **cfix**: Constant term in the objective (double).
 - * **Q**: The quadratic part Q^o of the objective as a sparse matrix, only lower-triangular part included. All arrays must have the same length.
 - **subi**: row indices of nonzeros (list(int32)).
 - **subj**: column indices of nonzeros (list(int32)).
 - **val**: values of nonzeros (list(double)).
 - * **barc**: The semidefinite part $\overline{C}$ of the objective (list). Each element of the list is a list describing one entry $\overline{C}_j$ using three fields:
 - index j (int32).
 - weights of the matrices from the storage E forming $\overline{C}_j$ (list(double)).
 - indices of the matrices from the storage E forming $\overline{C}_j$ (list(int64)).
- **A**: The linear constraint matrix A as a sparse matrix. All arrays must have the same length.
 - * **subi**: row indices of nonzeros (list(int32)).
 - * **subj**: column indices of nonzeros (list(int32)).
 - * **val**: values of nonzeros (list(double)).
- **bara**: The semidefinite part $\overline{A}$ of the constraints (list). Each element of the list is a list describing one entry $\overline{A}_{ij}$ using four fields:
 - * index i (int32).
 - * index j (int32).
 - * weights of the matrices from the storage E forming $\overline{A}_{ij}$ (list(double)).
 - * indices of the matrices from the storage E forming $\overline{A}_{ij}$ (list(int64)).
- **AfE**: The affine expression storage.
 - * **numafe**: number of rows in the storage (int64).
 - * **F**: The matrix F as a sparse matrix. All arrays must have the same length.
 - **subi**: row indices of nonzeros (list(int64)).
 - **subj**: column indices of nonzeros (list(int32)).
 - **val**: values of nonzeros (list(double)).

- * **g**: The vector g of constant terms as a sparse vector. Both arrays must have the same length.
 - **subi**: indices of nonzeros (list(int64)).
 - **val**: values of nonzeros (list(double)).
- * **barf**: The semidefinite part $\bar{F}$ of the expressions in AFE storage (list). Each element of the list is a list describing one entry $\bar{F}_{ij}$ using four fields:
 - index i (int64).
 - index j (int32).
 - weights of the matrices from the storage E forming $\bar{F}_{ij}$ (list(double)).
 - indices of the matrices from the storage E forming $\bar{F}_{ij}$ (list(int64)).
- **domains**: Information about domains. All fields present must have the same length as **type**.
 - * **name**: Domain names (list(string)).
 - * **type**: Description of the type of each domain (list). Each element of the list is a list describing one domain using at least one field:
 - domain type (string).
 - (except **pexp**, **dexp**) dimension (int64).
 - (only **ppow**, **dpow**) weights (list(double)).
- **ACC**: Information about affine conic constraints (ACC). All fields present must have the same length as **domain**.
 - * **name**: ACC names (list(string)).
 - * **domain**: Domains (list(int64)).
 - * **afeidx**: AFE indices, grouped by ACC (list(list(int64))).
 - * **b**: constant vectors b , grouped by ACC (list(list(double))).
- **DJC**: Information about disjunctive constraints (DJC). All fields present must have the same length as **termsize**.
 - * **name**: DJC names (list(string)).
 - * **termsize**: Term sizes, grouped by DJC (list(list(int64))).
 - * **domain**: Domains, grouped by DJC (list(list(int64))).
 - * **afeidx**: AFE indices, grouped by DJC (list(list(int64))).
 - * **b**: constant vectors b , grouped by DJC (list(list(double))).
- **MatrixStore**: The symmetric matrix storage E (list). Each element of the list is a list describing one entry E using four fields in sparse matrix format, lower-triangular part only:
 - * dimension (int32).
 - * row indices of nonzeros (list(int32)).
 - * column indices of nonzeros (list(int32)).
 - * values of nonzeros (list(double)).
- **Q**: The quadratic part Q^c of the constraints (list). Each element of the list is a list describing one entry Q_i^c using four fields in sparse matrix format, lower-triangular part only:
 - * the row index i (int32).
 - * row indices of nonzeros (list(int32)).
 - * column indices of nonzeros (list(int32)).
 - * values of nonzeros (list(double)).
- **qcone** (deprecated). The description of cones. All fields present must have the same length as **type**.
 - * **name**: Cone names (list(string)).
 - * **type**: Cone types (list(string)).
 - * **par**: Additional cone parameters (list(double)).
 - * **members**: Members, grouped by cone (list(list(int32))).

- **Task/solutions**: Solutions. This section can contain up to three subsections called:

- interior
- basic
- integer

corresponding to the three solution types in MOSEK. Each of these sections has the same structure:

- **prosta**: problem status (string).
- **solsta**: solution status (string).
- **xx**, **xc**, **y**, **slc**, **suc**, **slx**, **sux**, **snx**: one for each component of the solution of the same name (list(double)).
- **skx**, **skc**, **skn**: status keys (list(string)).
- **doty**: the dual y solution, grouped by ACC (list(list(double))).
- **barx**, **bars**: the primal/dual semidefinite solution, grouped by matrix variable (list(list(double))).

- **Task/parameters**: Parameters.

- **iparam**: Integer parameters (dictionary). A dictionary with entries of the form **name:value**, where **name** is a shortened parameter name (without leading **MSK_IPAR_**) and **value** is either an integer or string if the parameter takes values from an enum.
- **dparam**: Double parameters (dictionary). A dictionary with entries of the form **name:value**, where **name** is a shortened parameter name (without leading **MSK_DPAR_**) and **value** is a double.
- **sparam**: String parameters (dictionary). A dictionary with entries of the form **name:value**, where **name** is a shortened parameter name (without leading **MSK_SPAR_**) and **value** is a string. Note that this section is allowed but MOSEK ignores it both when writing and reading JTASK files.

15.7.2 JSOL Specification

The JSOL is a dictionary containing the following sections. All sections are optional and can be omitted if irrelevant for the problem.

- **\$schema**: JSON schema.
- **Task/name**: The name of the task (string).
- **Task/solutions**: Solutions. This section can contain up to three subsections called:
 - interior
 - basic
 - integer

corresponding to the three solution types in MOSEK. Each of these section has the same structure:

- **prosta**: problem status (string).
- **solsta**: solution status (string).
- **xx**, **xc**, **y**, **slc**, **suc**, **slx**, **sux**, **snx**: one for each component of the solution of the same name (list(double)).
- **skx**, **skc**, **skn**: status keys (list(string)).
- **doty**: the dual y solution, grouped by ACC (list(list(double))).
- **barx**, **bars**: the primal/dual semidefinite solution, grouped by matrix variable (list(list(double))).

- **Task/information**: Information items from the optimizer.

- **int32**: int32 information items (dictionary). A dictionary with entries of the form **name:value**.

- int64: int64 information items (dictionary). A dictionary with entries of the form **name**: **value**.
- double: double information items (dictionary). A dictionary with entries of the form **name**: **value**.

15.7.3 A jtask example

Listing 15.5: A formatted jtask file for a simple portfolio optimization problem.

```
{
  "$schema": "http://mosek.com/json/schema#",
  "Task/name": "Markowitz portfolio with market impact",
  "Task/INFO": { "numvar": 7, "numcon": 1, "numcone": 0, "numbarvar": 0, "numanz": 6, "numsymmat"
  ↪ ": 0, "numafe": 13, "numfnz": 12, "numacc": 4, "numdjc": 0, "numdom": 3, "mosekver": [10, 0, 0, 3] },
  "Task/data": {
    "var": {
      "name": ["1.0", "x[0]", "x[1]", "x[2]", "t[0]", "t[1]", "t[2]"],
      "bk": ["fx", "lo", "lo", "lo", "fr", "fr", "fr"],
      "bl": [1, 0.0, 0.0, 0.0, -1e+30, -1e+30, -1e+30],
      "bu": [1, 1e+30, 1e+30, 1e+30, 1e+30, 1e+30, 1e+30],
      "type": ["cont", "cont", "cont", "cont", "cont", "cont", "cont"]
    },
    "con": {
      "name": ["budget[]"],
      "bk": ["fx"],
      "bl": [1],
      "bu": [1]
    },
    "objective": {
      "sense": "max",
      "name": "obj",
      "c": {
        "subj": [1, 2, 3],
        "val": [0.1073, 0.0737, 0.0627]
      },
      "cfix": 0.0
    },
    "A": {
      "subi": [0, 0, 0, 0, 0, 0],
      "subj": [1, 2, 3, 4, 5, 6],
      "val": [1, 1, 1, 0.01, 0.01, 0.01]
    },
    "AFE": {
      "numafe": 13,
      "F": {
        "subi": [1, 1, 1, 2, 2, 3, 4, 6, 7, 9, 10, 12],
        "subj": [1, 2, 3, 2, 3, 3, 4, 1, 5, 2, 6, 3],
        "val": [0.166673333200005, 0.0232190712557243, 0.0012599496030238, 0.
        ↪ 102863378954911, -0.00222873156550421, 0.0338148677744977, 1, 1, 1, 1, 1, 1]
      },
      "g": {
        "subi": [0, 5, 8, 11],
        "val": [0.035, 1, 1, 1]
      }
    },
    "domains": {
```

(continues on next page)

```

        "type": [{"r",0},
                  ["quad",4],
                  ["ppow",3,[0.6666666666666666,0.3333333333333337]]]
    },
    "ACC":{
        "name":["risk[]","tz[0]","tz[1]","tz[2]"],
        "domain":[1,2,2,2],
        "afeidx":[[0,1,2,3],
                   [4,5,6],
                   [7,8,9],
                   [10,11,12]]
    }
},
"Task/solutions":{
    "interior":{
        "prosta":"unknown",
        "solsta":"unknown",
        "skx":["fix","supbas","supbas","supbas","supbas","supbas","supbas"],
        "skc":["fix"],
        "xx":[1,0.10331580274282556,0.11673185566457132,0.7724326587076371,0.
↪033208600335718846,0.03988270849469869,0.6788769587942524],
        "xc":[1],
        "slx":[0.0,-5.585840467641202e-10,-8.945844685006369e-10,-7.815248786428623e-
↪11,0.0,0.0,0.0],
        "sux":[0.0,0.0,0.0,0.0,0.0,0.0,0.0],
        "snx":[0.0,0.0,0.0,0.0,0.0,0.0,0.0],
        "slc":[0.0],
        "suc":[-0.046725814048521205],
        "y":[0.046725814048521205],
        "doty":[[-0.6062603164682975,0.3620818321879349,0.17817754087278295,0.
↪4524390346223723],
                [-4.6725842015519993e-4,-7.708781121860897e-6,2.24800624747081e-4],
                [-4.6725842015519993e-4,-9.268264309496919e-6,2.390390600079771e-4],
                [-4.6725842015519993e-4,-1.5854982159992136e-4,6.159249331148646e-4]]
    }
},
"Task/parameters":{
    "iparam":{
        "LICENSE_DEBUG":"ON",
        "MIO_SEED":422
    },
    "dparam":{
        "MIO_MAX_TIME":100
    },
    "sparam":{
    }
}
}

```

15.8 The Solution File Format

MOSEK can output solutions to a text file:

- *basis solution file* (extension `.bas`) if the problem is optimized using the simplex optimizer or basis identification is performed,
- *interior solution file* (extension `.sol`) if a problem is optimized using the interior-point optimizer and no basis identification is required,
- *integer solution file* (extension `.int`) if the problem is solved with the mixed-integer optimizer.

All solution files have the format:

NAME	:	<problem name>					
PROBLEM STATUS	:	<status of the problem>					
SOLUTION STATUS	:	<status of the solution>					
OBJECTIVE NAME	:	<name of the objective function>					
PRIMAL OBJECTIVE	:	<primal objective value corresponding to the solution>					
DUAL OBJECTIVE	:	<dual objective value corresponding to the solution>					
CONSTRAINTS							
INDEX	NAME	AT	ACTIVITY	LOWER LIMIT	UPPER LIMIT	DUAL LOWER	DUAL UPPER
?	<name>	??	<a value>	<a value>	<a value>	<a value>	<a value>
AFFINE CONIC CONSTRAINTS							
INDEX	NAME	I	ACTIVITY	DUAL			
?	<name>	<a value>	<a value>	<a value>			
VARIABLES							
INDEX	NAME	AT	ACTIVITY	LOWER LIMIT	UPPER LIMIT	DUAL LOWER	DUAL UPPER
↪[CONIC DUAL]							
?	<name>	??	<a value>	<a value>	<a value>	<a value>	<a value>
↪[<a value>]							
SYMMETRIC MATRIX VARIABLES							
INDEX	NAME	I	J	PRIMAL	DUAL		
?	<name>	<a value>	<a value>	<a value>	<a value>		

The fields `?`, `??` and `<>` will be filled with problem and solution specific information as described below. The solution contains sections corresponding to parts of the input. Empty sections may be omitted and fields in `[]` are optional, depending on what type of problem is solved. The notation below follows the **MOSEK** naming convention for parts of the solution as defined in the problem specifications in [Sec. 12](#).

- **HEADER** In this section, first the name of the problem is listed and afterwards the problem and solution status are shown. Next the primal and dual objective values are displayed.
- **CONSTRAINTS**
 - **INDEX**: A sequential index assigned to the constraint by **MOSEK**
 - **NAME**: The name of the constraint assigned by the user or autogenerated.
 - **AT**: The status key `bkc` of the constraint as in [Table 15.4](#).
 - **ACTIVITY**: the activity `xc` of the constraint expression.
 - **LOWER LIMIT**: the lower bound `blc` of the constraint.
 - **UPPER LIMIT**: the upper bound `buc` of the constraint.
 - **DUAL LOWER**: the dual multiplier `slc` corresponding to the lower limit on the constraint.
 - **DUAL UPPER**: the dual multiplier `suc` corresponding to the upper limit on the constraint.
- **AFFINE CONIC CONSTRAINTS**

- INDEX: A sequential index assigned to the affine expressions by **MOSEK**
- NAME: The name of the affine conic constraint assigned by the user or autogenerated.
- I: The sequential index of the affine expression in the affine conic constraint.
- ACTIVITY: the activity of the I-th affine expression in the affine conic constraint.
- DUAL: the dual multiplier `doty` for the I-th entry in the affine conic constraint.

- VARIABLES

- INDEX: A sequential index assigned to the variable by **MOSEK**
- NAME: The name of the variable assigned by the user or autogenerated.
- AT: The status key `bkx` of the variable as in Table 15.4.
- ACTIVITY: the value `xx` of the variable.
- LOWER LIMIT: the lower bound `blx` of the variable.
- UPPER LIMIT: the upper bound `bux` of the variable.
- DUAL LOWER: the dual multiplier `slx` corresponding to the lower limit on the variable.
- DUAL UPPER: the dual multiplier `sux` corresponding to the upper limit on the variable.
- CONIC DUAL: the dual multiplier `skx` corresponding to a conic variable (deprecated).

- SYMMETRIC MATRIX VARIABLES

- INDEX: A sequential index assigned to each symmetric matrix entry by **MOSEK**
- NAME: The name of the symmetric matrix variable assigned by the user or autogenerated.
- I: The row index in the symmetric matrix variable.
- J: The column index in the symmetric matrix variable.
- PRIMAL: the value of `barx` for the (I, J)-th entry in the symmetric matrix variable.
- DUAL: the dual multiplier `bars` for the (I, J)-th entry in the symmetric matrix variable.

Table 15.4: Status keys.

Status key	Interpretation
UN	Unknown status
BS	Is basic
SB	Is superbasic
LL	Is at the lower limit (bound)
UL	Is at the upper limit (bound)
EQ	Lower limit is identical to upper limit
**	Is infeasible i.e. the lower limit is greater than the upper limit.

Example.

Below is an example of a solution file.

Listing 15.6: An example of `.sol` file.

```

NAME          :
PROBLEM STATUS : PRIMAL_AND_DUAL_FEASIBLE
SOLUTION STATUS : OPTIMAL
OBJECTIVE NAME  : OBJ
PRIMAL OBJECTIVE : 0.70571049347734
DUAL OBJECTIVE  : 0.70571048919757

CONSTRAINTS
INDEX      NAME      AT ACTIVITY      LOWER LIMIT      UPPER LIMIT
↪          DUAL LOWER      DUAL UPPER

```

(continues on next page)

(continued from previous page)

AFFINE CONIC CONSTRAINTS

INDEX	NAME	I	ACTIVITY	DUAL
0	A1	0	1.0000000009656	0.54475821296644
1	A1	1	0.50000000152223	0.32190455246225
2	A2	0	0.25439922724695	0.4552417870329
3	A2	1	0.17988741850378	-0.32190455246178
4	A2	2	0.17988741850378	-0.32190455246178

VARIABLES

INDEX	NAME	AT	ACTIVITY	LOWER LIMIT	UPPER LIMIT
↪	DUAL LOWER		DUAL UPPER		
0	X1	SB	0.25439922724695	NONE	NONE
↪	0		0		
1	X2	SB	0.17988741850378	NONE	NONE
↪	0		0		
2	X3	SB	0.17988741850378	NONE	NONE
↪	0		0		

SYMMETRIC MATRIX VARIABLES

INDEX	NAME	I	J	PRIMAL	DUAL
0	BARX1	0	0	0.21725733689874	1.1333372337141
1	BARX1	1	0	-0.25997257078534	0.
↪	67809544651396				
2	BARX1	2	0	0.21725733648507	-0.
↪	3219045527104				
3	BARX1	1	1	0.31108610088839	1.1333372332693
4	BARX1	2	1	-0.25997257078534	0.
↪	67809544651435				
5	BARX1	2	2	0.21725733689874	1.1333372337145
6	BARX2	0	0	4.8362272828127e-10	0.
↪	54475821339698				
7	BARX2	1	0	0	0
8	BARX2	1	1	4.8362272828127e-10	0.
↪	54475821339698				

Chapter 16

List of examples

List of examples shipped in the distribution of Fusion API for C++:

Table 16.1: List of distributed examples

File	Description
TrafficNetworkModel.cc	Demonstrates a traffic network problem as a conic quadratic problem (CQO)
alan.cc	A portfolio choice model <code>alan.gms</code> from the GAMS model library
baker.cc	A small bakery revenue maximization linear problem
breaksolver.cc	Shows how to break a long-running task
callback.cc	An example of data/progress callback
ceo1.cc	A simple conic exponential problem
cqo1.cc	A simple conic quadratic problem
diet.cc	Solving Stigler's Nutrition model <code>diet</code> from the GAMS model library
djc1.cc	A simple problem with disjunctive constraints (DJC)
duality.cc	Shows how to access the dual solution
elastic.cc	Linear regression with elastic net. Demonstrates model parametrization.
facility_location.cc	Demonstrates a small one-facility location problem (CQO)
gp1.cc	A simple geometric program (GP) in conic form
lo1.cc	A simple linear problem
logistic.cc	Implements logistic regression and simple log-sum-exp (CEO)
lownerjohn_ellipsoids.cc	Computes the Lowner-John inner and outer ellipsoidal approximations of a polytope (SDO, Power Cone)
lpt.cc	Demonstrates how to solve the multi-processor scheduling problem and input an integer feasible point (MIP)
mico1.cc	A simple mixed-integer conic problem
milo1.cc	A simple mixed-integer linear problem
miointsol.cc	A simple mixed-integer linear problem with an initial guess
nearestcorr.cc	Solves the nearest correlation matrix problem (SDO, CQO)
opt_server_sync.cc	Uses MOSEK OptServer to solve an optimization problem synchronously
parallel.cc	Demonstrates parallel optimization using a batch method in MOSEK
parameters.cc	Shows how to set optimizer parameters and read information items
pinfeas.cc	Shows how to obtain and analyze a primal infeasibility certificate
portfolio_1_basic.cc	Portfolio optimization - basic Markowitz model
portfolio_2_frontier.cc	Portfolio optimization - efficient frontier
portfolio_3_impact.cc	Portfolio optimization - market impact costs

continues on next page

Table 16.1 – continued from previous page

File	Description
<code>portfolio_4_transaction.cc</code>	Portfolio optimization - transaction costs
<code>portfolio_5_cardinality.cc</code>	Portfolio optimization - cardinality constraints
<code>portfolio_6_factor.cc</code>	Portfolio optimization - factor model
<code>pow1.cc</code>	A simple power cone problem
<code>primal_svm.cc</code>	Implements a simple soft-margin Support Vector Machine (CQO)
<code>qcqp_sdp_relaxation.cc</code>	Demonstrate how to use SDP to solve convex relaxation of a mixed-integer QCQP problem
<code>reoptimization.cc</code>	Demonstrate how to modify and re-optimize a linear problem
<code>response.cc</code>	Demonstrates proper response handling
<code>sdo1.cc</code>	A simple semidefinite problem with one matrix variable and a quadratic cone
<code>sdo2.cc</code>	A simple semidefinite problem with two matrix variables
<code>sdo3.cc</code>	A simple semidefinite problem with many matrix variables of the same dimension
<code>sospoly.cc</code>	Models the cone of nonnegative polynomials and nonnegative trigonometric polynomials using Nesterov's framework
<code>sudoku.cc</code>	A SUDOKU solver (MIP)
<code>total_variation.cc</code>	Demonstrates how to solve a total variation problem (CQO)
<code>tsp.cc</code>	Solves a simple Travelling Salesman Problem and shows how to add constraints to a model and re-optimize (MIP)

Additional examples can be found on the **MOSEK** website and in other **MOSEK** publications.

Chapter 17

Interface changes

The section shows interface-specific changes to the **MOSEK** Fusion API for C++ in version 10.0 compared to version 9. See the [release notes](#) for general changes and new features of the **MOSEK** Optimization Suite.

17.1 Important changes compared to version 9

- **Parameters.** Users who set parameters to tune the performance and numerical properties of the solver (termination criteria, tolerances, solving primal or dual, presolve etc.) are recommended to reevaluate such tuning. It may be that other, or default, parameter settings will be more beneficial in the current version. The hints in [Sec. 9](#) may be useful for some cases.
- **Multithreading.** In the interior-point optimizer it is possible to set the number of threads with `numThreads` before each optimization, and not just once per process. The parameter `MSK_IPAR_INTPNT_MULTI_THREAD` is no longer relevant and was removed.
- **OptServer.** The arguments used in remote calls from the **MOSEK** API change from `(server, port)` to `(addr, accesstoken)`, where `addr` is the full URL such as `http://server:port` or `https://server:port`. See the documentation of the relevant functions.
- **MIO initial solution.** In order for the mixed-integer solver to utilize a partial integer solution the parameter `mioConstructSol` must be set. See [Sec. 7.7.2](#) for details. In version 9 this action happened by default.
- **Text file formats.** Due to new internal representation of the model, *conic problems can no longer be written in OPF format*. Instead write them in PTF format (extension `*.ptf`, see [Sec. 15.5](#)).

17.2 Changes compared to version 9

17.3 Parameters compared to version 9

Added

- `mioDjcMaxBigm`
- `presolveTolPrimalInfeasPerturbation`
- `mioConstructSol`
- `mioCutLipro`
- `mioDataPermutationMethod`
- `mioMemoryEmphasisLevel`
- `mioNumericalEmphasisLevel`

- *mioPresolveAggregatorUse*
- *mioQcgoReformulationMethod*
- *mioSymmetryLevel*
- *remoteUseCompression*
- *writeJsonIndentation*
- *remoteOptserverHost*
- *remoteTlsCert*
- *remoteTlsCertPath*

Removed

- `intpntMultiThread`
- `readLpDropNewVarsInBou`
- `readLpQuotedNames`
- `writeLpQuotedNames`
- `writeLpStrictFormat`
- `writeLpTermsPerLine`
- `writePrecision`
- `remoteAccessToken`
- `statFileName`

17.4 Constants compared to version 9

Added

Removed

- `beginFullConvexityCheck`
- `endFullConvexityCheck`
- `imFullConvexityCheck`
- `rdTime`
- `aggressive`
- `moderate`

Bibliography

- [AA95] E. D. Andersen and K. D. Andersen. Presolving in linear programming. *Math. Programming*, 71(2):221–245, 1995.
- [AGMeszarosX96] E. D. Andersen, J. Gondzio, Cs. Mészáros, and X. Xu. Implementation of interior point methods for large scale linear programming. In T. Terlaky, editor, *Interior-point methods of mathematical programming*, pages 189–252. Kluwer Academic Publishers, 1996.
- [ART03] E. D. Andersen, C. Roos, and T. Terlaky. On implementing a primal-dual interior-point method for conic quadratic optimization. *Math. Programming*, February 2003.
- [AY96] E. D. Andersen and Y. Ye. Combining interior-point and pivoting algorithms. *Management Sci.*, 42(12):1719–1731, December 1996.
- [And09] Erling D. Andersen. The homogeneous and self-dual model and algorithm for linear optimization. Technical Report TR-1-2009, MOSEK ApS, 2009. URL: <http://docs.mosek.com/whitepapers/homolo.pdf>.
- [And13] Erling D. Andersen. On formulating quadratic functions in optimization models. Technical Report TR-1-2013, MOSEK ApS, 2013. Last revised 23-feb-2016. URL: <http://docs.mosek.com/whitepapers/qmodel.pdf>.
- [BKVH07] S. Boyd, S.J. Kim, L. Vandenberghe, and A. Hassibi. A Tutorial on Geometric Programming. *Optimization and Engineering*, 8(1):67–127, 2007. Available at http://www.stanford.edu/boyd/gp_tutorial.html.
- [Chvatal83] V. Chvátal. *Linear programming*. W.H. Freeman and Company, 1983.
- [CCornuejolsZ14] M. Conforti, G. Cornu'ejols, and G. Zambelli. *Integer programming*. Springer, 2014.
- [GJ79] Michael R Gary and David S Johnson. Computers and intractability: a guide to the theory of np-completeness. 1979.
- [GY05] Donald Goldfarb and Wotao Yin. Second-order cone programming methods for total variation-based image restoration. *SIAM Journal on Scientific Computing*, 27(2):622–645, 2005.
- [Gra69] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM journal on Applied Mathematics*, 17(2):416–429, 1969.
- [GK00] Richard C. Grinold and Ronald N. Kahn. *Active portfolio management*. McGraw-Hill, New York, 2 edition, 2000.
- [Naz87] J. L. Nazareth. *Computer Solution of Linear Programs*. Oxford University Press, New York, 1987.
- [PB15] Jaehyun Park and Stephen Boyd. A semidefinite programming method for integer convex quadratic minimization. *arXiv preprint arXiv:1504.07672*, 2015.
- [Pat03] Gábor Pataki. Teaching integer programming formulations using the traveling salesman problem. *SIAM review*, 45(1):116–123, 2003.
- [Wol98] L. A. Wolsey. *Integer programming*. John Wiley and Sons, 1998.
- [BenTalN01] A. Ben-Tal and A. Nemirovski. *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. MPS/SIAM Series on Optimization. SIAM, 2001.

Symbol Index

Classes

`ndarray<T,N>`, 181
`rc_ptr<T>`, 185
`shape_t<N>`, 183
`BaseExpression`, 187
`BaseVariable`, 189
`BoundInterfaceConstraint`, 194
`BoundInterfaceVariable`, 195
`ConeDomain`, 197
`ConicConstraint`, 198
`ConicVariable`, 199
`Constraint`, 200
`Disjunction`, 204
`DJC`, 203
`Domain`, 204
`Expr`, 214
`Expression`, 224
`ExprScaleVecPSD`, 223
`LinearConstraint`, 226
`LinearDomain`, 226
`LinearPSDConstraint`, 228
`LinearPSDVariable`, 228
`LinearVariable`, 229
`Matrix`, 230
`Model`, 234
`ModelConstraint`, 246
`ModelVariable`, 246
`NDSparseArray`, 247
`Param`, 250
`Parameter`, 252
`PSDConstraint`, 248
`PSDDomain`, 248
`PSDVariable`, 249
`RangedConstraint`, 255
`RangeDomain`, 254
`RangedVariable`, 256
`Set`, 257
`SimpleTerm`, 258
`SliceConstraint`, 258
`SliceVariable`, 258
`SymmetricLinearDomain`, 259
`SymmetricRangeDomain`, 260
`Term`, 260
`Var`, 260
`Variable`, 264
`WorkStack`, 268
`LinAlg`, 340

Enumerations

`AccSolutionStatus`, 307
`AccSolutionStatus.Optimal`, 307
`AccSolutionStatus.Feasible`, 307
`AccSolutionStatus.Certificate`, 307
`AccSolutionStatus.Anything`, 307
`ObjectiveSense`, 307
`ObjectiveSense.Undefined`, 307
`ObjectiveSense.Minimize`, 307
`ObjectiveSense.Maximize`, 307
`ProblemStatus`, 307
`ProblemStatus.Unknown`, 307
`ProblemStatus.PrimalInfeasibleOrUnbounded`, 308
`ProblemStatus.PrimalInfeasible`, 308
`ProblemStatus.PrimalFeasible`, 307
`ProblemStatus.PrimalAndDualInfeasible`, 308
`ProblemStatus.PrimalAndDualFeasible`, 307
`ProblemStatus.IllPosed`, 308
`ProblemStatus.DualInfeasible`, 308
`ProblemStatus.DualFeasible`, 308
`SolutionStatus`, 308
`SolutionStatus.Unknown`, 308
`SolutionStatus.Undefined`, 308
`SolutionStatus.Optimal`, 308
`SolutionStatus.IllposedCert`, 308
`SolutionStatus.Feasible`, 308
`SolutionStatus.Certificate`, 308
`SolutionType`, 308
`SolutionType.Interior`, 308
`SolutionType.Integer`, 308
`SolutionType.Default`, 308
`SolutionType.Basic`, 308
`SolverStatus`, 308
`SolverStatus.OK`, 308
`SolverStatus.LostRace`, 308
`SolverStatus.Error`, 308

Exceptions

`monty::ArrayInitializerException`, 185
`DeletionError`, 334
`DimensionError`, 334
`DomainError`, 334
`ExpressionError`, 335
`FatalError`, 335
`FusionException`, 335
`FusionRuntimeException`, 335
`IndexError`, 336
`IOError`, 336

- LengthError, 336
- MatrixError, 336
- ModelError, 336
- NameError, 336
- OptimizeError, 336
- ParameterError, 337
- RangeError, 337
- SetDefinitionError, 337
- SliceError, 337
- SolutionError, 337
- SparseFormatError, 337
- UnexpectedError, 337
- UnimplementedError, 338
- UpdateError, 338
- ValueConversionError, 338

Parameters

- Double parameters, 279
- basisRelTolS, 279
- basisTolS, 279
- basisTolX, 279
- intpntCoTolDfeas, 279
- intpntCoTolInfeas, 280
- intpntCoTolMuRed, 280
- intpntCoTolNearRel, 280
- intpntCoTolPfeas, 280
- intpntCoTolRelGap, 280
- intpntTolDfeas, 280
- intpntTolDsafe, 281
- intpntTolInfeas, 281
- intpntTolMuRed, 281
- intpntTolPath, 281
- intpntTolPfeas, 281
- intpntTolPsafe, 281
- intpntTolRelGap, 282
- intpntTolRelStep, 282
- intpntTolStepSize, 282
- lowerObjCut, 282
- lowerObjCutFiniteTrh, 282
- mioDjcMaxBigm, 282
- mioMaxTime, 283
- mioRelGapConst, 283
- mioTolAbsGap, 283
- mioTolAbsRelaxInt, 283
- mioTolFeas, 283
- mioTolRelDualBoundImprovement, 283
- mioTolRelGap, 284
- optimizerMaxTime, 284
- presolveTolAbsLindep, 284
- presolveTolAij, 284
- presolveTolPrimalInfeasPerturbation, 284
- presolveTolRelLindep, 284
- presolveTolS, 285
- presolveTolX, 285
- simLuTolRelPiv, 285
- simplexAbsTolPiv, 285
- upperObjCut, 285
- upperObjCutFiniteTrh, 285

- Integer parameters, 286
- autoUpdateSolInfo, 286
- biCleanOptimizer, 286
- biIgnoreMaxIter, 286
- biIgnoreNumError, 286
- biMaxIterations, 286
- cacheLicense, 287
- infeasPreferPrimal, 287
- infeasReportAuto, 287
- intpntBasis, 287
- intpntDiffStep, 287
- intpntMaxIterations, 288
- intpntMaxNumCor, 288
- intpntOffColTrh, 288
- intpntOrderGpNumSeeds, 288
- intpntOrderMethod, 288
- intpntRegularizationUse, 288
- intpntScaling, 289
- intpntSolveForm, 289
- intpntStartingPoint, 289
- licenseDebug, 289
- licensePauseTime, 289
- licenseSuppressExpireWrns, 289
- licenseTrhExpiryWrn, 290
- licenseWait, 290
- log, 290
- logBi, 290
- logBiFreq, 290
- logCutSecondOpt, 291
- logExpand, 291
- logFile, 291
- logInfeasAna, 291
- logIntpnt, 291
- logLocalInfo, 291
- logMio, 292
- logMioFreq, 292
- logOrder, 292
- logPresolve, 292
- logResponse, 292
- logSim, 292
- logSimFreq, 293
- logSimMinor, 293
- mioBranchDir, 293
- mioConicOuterApproximation, 293
- mioConstructSol, 293
- mioCutClique, 293
- mioCutCmir, 294
- mioCutGmi, 294
- mioCutImpliedBound, 294
- mioCutKnapsackCover, 294
- mioCutLipro, 294
- mioCutSelectionLevel, 294
- mioDataPermutationMethod, 295
- mioFeaspumpLevel, 295
- mioHeuristicLevel, 295
- mioMaxNumBranches, 295
- mioMaxNumRelaxs, 295
- mioMaxNumRootCutRounds, 296

- mioMaxNumSolutions, 296
- mioMemoryEmphasisLevel, 296
- mioMode, 296
- mioNodeOptimizer, 296
- mioNodeSelection, 296
- mioNumericalEmphasisLevel, 297
- mioPerspectiveReformulate, 297
- mioPresolveAggregatorUse, 297
- mioProbingLevel, 297
- mioPropagateObjectiveConstraint, 297
- mioQcqrReformulationMethod, 298
- mioRinsMaxNodes, 298
- mioRootOptimizer, 298
- mioRootRepeatPresolveLevel, 298
- mioSeed, 298
- mioSymmetryLevel, 298
- mioVbDetectionLevel, 299
- mtSpincount, 299
- numThreads, 299
- optimizer, 299
- presolveEliminatorMaxFill, 300
- presolveEliminatorMaxNumTries, 300
- presolveLevel, 300
- presolveLindepAbsWorkTrh, 300
- presolveLindepRelWorkTrh, 300
- presolveLindepUse, 300
- presolveMaxNumPass, 301
- presolveUse, 301
- remoteUseCompression, 301
- removeUnusedSolutions, 301
- simBasisFactorUse, 301
- simDegen, 301
- simDualCrash, 301
- simDualPhaseoneMethod, 302
- simDualRestrictSelection, 302
- simDualSelection, 302
- simExploitDupvec, 302
- simHotstart, 302
- simHotstartLu, 303
- simMaxIterations, 303
- simMaxNumSetbacks, 303
- simNonSingular, 303
- simPrimalCrash, 303
- simPrimalPhaseoneMethod, 303
- simPrimalRestrictSelection, 304
- simPrimalSelection, 304
- simRefactorFreq, 304
- simReformulation, 304
- simSaveLu, 304
- simScaling, 304
- simScalingMethod, 305
- simSeed, 305
- simSolveForm, 305
- simSwitchOptimizer, 305
- writeJsonIndentation, 305
- writeLpFullObj, 305
- writeLpLineWidth, 306
- String parameters, 306

- basSolFileName, 306
- dataFileName, 306
- intSolFileName, 306
- itrSolFileName, 306
- remoteOptserverHost, 306
- remoteTlsCert, 306
- remoteTlsCertPath, 307
- writeLpGenVarName, 307

Response codes

Index

A

- algorithm
 - approximation, 121, 144
- approximation
 - algorithm, 121, 144
 - correlation matrix, 139
- asset, *see* portfolio optimization
- assignment problem, 129

B

- basic
 - solution, 60
- basis identification, 162
- bound
 - constraint, 24, 147, 150, 154
 - linear optimization, 24
 - variable, 24, 147, 150, 154
- Branch-and-Bound, 170

C

- callback, 70
- cardinality constraints, 108
- CBF format, 370
- certificate, 61
 - dual, 149, 152
 - infeasibility, 56
 - infeasible, 56
 - primal, 149, 152
- Cholesky factorization, 101
- compile
 - Linux, examples, 12
- complementarity, 148, 152
- cone, 15
 - dual, 151
 - dual exponential, 31
 - exponential, 31
 - power, 29, 128
 - quadratic, 26
 - rotated quadratic, 26
 - semidefinite, 36
- conic exponential optimization, 31
- conic optimization, 26, 29, 31, 150
 - interior-point, 166
 - mixed-integer, 177
 - modeling, 15
 - termination criteria, 168
- conic quadratic optimization, 26
- constraint
 - bound, 24, 147, 150, 154

- linear optimization, 24
- matrix, 24, 147, 150, 154
- modeling, 17
- constraint programming, 45
- correlation matrix, 95, 139
 - approximation, 139
- covariance matrix, *see* correlation matrix
- cuts, 175
- cutting planes, 175

D

- denoising, 116
- dense
 - matrix, 91
- determinant, 128
- determinism, 91
- disjunction, 45
- disjunctive constraints, 45
- DJC, 45
- domain, 338
- dual
 - certificate, 149, 152
 - cone, 151
 - feasible, 148
 - infeasible, 148, 149, 152
 - problem, 148, 151, 155
 - solution, 27, 30, 32, 62
 - variable, 148, 151
- duality
 - conic, 151
 - linear, 148
 - semidefinite, 155
- dualizer, 158

E

- efficient frontier, 99
- elastic net, 48
- eliminator, 158
- ellipsoid, 126
- error
 - optimization, 60
- errors, 64
- examples
 - compile Linux, 12
- exceptions, 64
- exponential cone, 31
- expression
 - modeling, 16

F

- factor model, 101, 139
- feasibility
 - integer feasibility, 173
- feasibility problem, 129
- feasible
 - dual, 148
 - primal, 147, 160, 167
 - problem, 147
- format, 67
 - CBF, 370
 - json, 395
 - LP, 345
 - MPS, 349
 - OPF, 361
 - PTF, 388
 - sol, 401
 - task, 394
- Frobenius norm, 139
- Fusion
 - reformulation, 14

G

- geometric mean, 128
- geometric programming, 33
- GP, 33

H

- heuristic, 174
- hot-start, 164

I

- I/O, 67
- infeasibility, 61, 149, 152
 - linear optimization, 149
 - semidefinite, 155
- infeasibility certificate, 56
- infeasible
 - dual, 148, 149, 152
 - primal, 147, 149, 152, 160, 167
 - problem, 147, 149, 155
- information item, 68, 70
- installation, 9
 - makefile, 12
 - requirements, 9
 - troubleshooting, 9
 - Visual Studio, 12
- integer
 - solution, 60
 - variable, 42
- integer feasibility, 173
 - feasibility, 173
- integer optimization, 42, 45
 - initial solution, 43, 121
 - parameter, 42
- interior-point
 - conic optimization, 166
 - linear optimization, 160

- logging, 163, 169
- optimizer, 160, 166
- solution, 60
- termination criteria, 161, 168

J

- json format, 395

L

- Löwner-John ellipsoid, 126
- lasso, 48
- least squares
 - integer, 143
- license, 93
 - checkout, 93
 - parameter, 93
 - path, 93
- limitations, 89
- linear constraint matrix, 24
- linear dependency, 158
- linear optimization, 24, 147
 - bound, 24
 - constraint, 24
 - infeasibility, 149
 - interior-point, 160
 - objective, 24
 - simplex, 164
 - termination criteria, 161, 164
 - variable, 24

Linux

- examples compile, 12
- log-sum-exp, 124
- logging, 66
 - interior-point, 163, 169
 - mixed-integer optimizer, 176
 - optimizer, 163, 165, 169
 - simplex, 165
- logistic regression, 123
- LP format, 345

M

- machine learning
 - large margin classification, 110
 - logistic regression, 123
 - separating hyperplane, 110
 - Support-Vector Machine, 110
- makespan, 120
- market impact cost, 105
- Markowitz model, 95
- matrix
 - constraint, 24, 147, 150, 154
 - dense, 91
 - low rank, 141
 - modeling, 18
 - semidefinite, 36
 - sparse, 91
 - symmetric, 36
- memory management, 89

- MI(QC)QO, 177
- MICO, 177
- MIP, *see* integer optimization
- mixed-integer, *see* integer
 - conic optimization, 177
 - optimizer, 170
 - presolve, 174
 - quadratic, 177
- mixed-integer optimization, *see* integer optimization, 170
- mixed-integer optimizer
 - logging, 176
- modeling
 - conic optimization, 15
 - constraint, 17
 - design, 12
 - expression, 16
 - matrix, 18
 - objective, 17
 - variable, 15
- MPS format, 349
 - free, 360

N

- norm
 - Frobenius, 139
 - nuclear, 141
- nuclear norm, 141
- numerical issues
 - presolve, 158
 - scaling, 159
 - simplex, 165

O

- objective, 147, 150, 154
 - linear optimization, 24
 - modeling, 17
- OPF format, 361
- optimal
 - solution, 61
- optimality gap, 172
- optimization
 - conic, 150
 - conic quadratic, 150
 - error, 60
 - integer, 45
 - linear, 24, 147
 - semidefinite, 153
- optimizer
 - determinism, 91
 - interior-point, 160, 166
 - interrupt, 69, 70
 - logging, 163, 165, 169
 - mixed-integer, 45, 170
 - parallel, 55
 - selection, 158, 159
 - simplex, 164
 - termination, 172
 - time limit, 69
- Optimizer API, 73
 - reformulation, 14

P

- parallel optimization, 55
- parallelization, 91
- parameter, 67
 - integer optimization, 42
 - license, 93
 - simplex, 165
- parameters, 48
- parametrization, 92
- Pareto optimality, 95
- path
 - license, 93
- penalty, 111
- portfolio optimization, 95
 - cardinality constraints, 108
 - correlation matrix, 139
 - efficient frontier, 99
 - factor model, 101, 139
 - market impact cost, 105
 - Markowitz model, 95
 - Pareto optimality, 95
 - slippage cost, 104
 - transaction cost, 107
- power cone, 29, 128
- power cone optimization, 29
- presolve, 157
 - eliminator, 158
 - linear dependency check, 158
 - mixed-integer, 174
 - numerical issues, 158
- primal
 - certificate, 149, 152
 - feasible, 147, 160, 167
 - infeasible, 147, 149, 152, 160, 167
 - problem, 148, 151, 155
 - solution, 27, 30, 32, 62, 147
- primal heuristics, 174
- primal-dual
 - problem, 160, 166
 - solution, 148
- problem
 - dual, 148, 151, 155
 - feasible, 147
 - infeasible, 147, 149, 155
 - load, 67
 - primal, 148, 151, 155
 - primal-dual, 160, 166
 - save, 67
 - status, 60
 - unbounded, 149, 153
- PTF format, 388

Q

- quadratic

- mixed-integer, 177
- quadratic cone, 26

R

- regression, 48
 - logistic, 123
- regularization, 48
- relaxation, 143, 170
- reoptimization, 21, 48, 135
- response code, 64
- ridge, 48
- rotated quadratic cone, 26

S

- scaling, 159
- scheduling, 120
- Schur complement, 143
- semidefinite
 - cone, 36
 - infeasibility, 155
 - matrix, 36
 - variable, 36
- semidefinite optimization, 36, 153
- separating hyperplane, 110
- simplex
 - linear optimization, 164
 - logging, 165
 - numerical issues, 165
 - optimizer, 164
 - parameter, 165
 - termination criteria, 164
- slice
 - variable, 19, 92, 116
- slippage cost, 104
- sol format, 401
- solution
 - basic, 60
 - dual, 27, 30, 32, 62
 - file format, 401
 - integer, 60
 - interior-point, 60
 - optimal, 61
 - primal, 27, 30, 32, 62, 147
 - primal-dual, 148
 - retrieve, 60
 - status, 61
- sparse
 - matrix, 91
- stacking, 19
- status
 - problem, 60
 - solution, 61
- symmetric
 - matrix, 36

T

- task format, 394
- termination, 60

- optimizer, 172
- termination criteria, 70, 172
 - conic optimization, 168
 - interior-point, 161, 168
 - linear optimization, 161, 164
 - simplex, 164
 - tolerance, 162, 169
- thread, 91
- time limit, 69, 70
- tolerance
 - termination criteria, 162, 169
- transaction cost, 107
- travelling salesman problem, 134
- troubleshooting
 - installation, 9

U

- unbounded
 - problem, 149, 153
- user callback, *see* callback

V

- valid inequalities, 175
- variable, 147, 150, 154
 - bound, 24, 147, 150, 154
 - dual, 148, 151
 - integer, 42
 - limitations, 89
 - linear optimization, 24
 - modeling, 15
 - semidefinite, 36
 - slice, 19, 92, 116
- vectorization, 20, 92
- Visual Studio
 - installation, 12